Fundamentals *of*
# Multiagent Systems
with NetLogo Examples

*José M Vidal*

March 1, 2010

*Fundamentos de Sistemas*
*Multiagentes com Exemplos*
*NetLogo Jos´e M Vidal*

March 1, 2010

**Fundamentals of Multiagent Systems**
by José M. Vidal
Copyright © 2007 José M. Vidal. All rights reserved.

Fundamentos de Sistemas Multiagentes por Jos´e M. Vidal Copyright c◯ 2007Jos´e M. Vidal. Todos os direitos reservados.

## Contents


**Preface** **7**
- 0.1 Usage . . . 7
- 0.2 Acknowledgments . . . 8

**1 Multiagent Problem Formulation** **9**
- 1.1 Utility . . . 9
  - 1.1.1 Utility is Not Money . . . 10
  - 1.1.2 Expected Utility . . . 11
- 1.2 Markov Decision Processes . . . 12
  - 1.2.1 Multiagent Markov Decision Processes . . . 14
  - 1.2.2 Partially Observable MDPs . . . 15
- 1.3 Planning . . . 15
  - 1.3.1 Hierarchical Planning . . . 16
- 1.4 Summary . . . 17
- Exercises . . . 17

**2 Distributed Constraints** **19**
- 2.1 Distributed Constraint Satisfaction . . . 19
  - 2.1.1 Filtering Algorithm . . . 21
  - 2.1.2 Hyper-Resolution Based Consistency Algorithm . . . 22
  - 2.1.3 Asynchronous Backtracking . . . 24
  - 2.1.4 Asynchronous Weak-Commitment Search . . . 27
  - 2.1.5 Distributed Breakout . . . 29
- 2.2 Distributed Constraint Optimization . . . 31
  - 2.2.1 Adopt . . . 32
  - 2.2.2 OptAPO . . . 37
- Exercises . . . 38

**3 Standard and Extended Form Games** **39**
- 3.1 Games in Normal Form . . . 39
  - 3.1.1 Solution Concepts . . . 40
  - 3.1.2 Famous Games . . . 42
  - 3.1.3 Repeated Games . . . 44
- 3.2 Games in Extended Form . . . 45
  - 3.2.1 Solution Concepts . . . 46
- 3.3 Finding a Solution . . . 47
- Exercises . . . 47

**4 Characteristic Form Games and Coalition Formation** **49**
- 4.1 Characteristic Form Games . . . 49
  - 4.1.1 Solution Concepts . . . 50
  - 4.1.2 Finding the Optimal Coalition Structure . . . 55
- 4.2 Coalition Formation . . . 57
- Exercises . . . 57




## Conteúdo


**Prefácio** **7**
0.1 Uso.7 0,2 Agradecimentos.8

**1 Formulação de Problema Multiagente 9**
1.1 Utilidade.9 1.1.1 Utilidade não é dinheiro.10 1.1.2 Utilidade Esperada.11 1.2 Processos de Decisão de Markov.12 1.2.1 Processos de Decisão Markov Multiagentes.14 1.2.2 MDPs Parcialmente Observáveis.15 1.3 Planejamento.15 1.3.1 Planejamento Hierárquico.16 1.4 Resumo.17 Exercícios.17

**2 Restrições Distribuídas 19**
2.1 Satisfação de Restrições Distribuídas.19 2.1.1 Algoritmo de Filtragem.21 2.1.2 Algoritmo de consistência baseado em hiperresolução . . . . . 22 2.1.3 Retrocesso Assíncrono.24 2.1.4 Pesquisa assíncrona de comprometimento fraco.27 2.1.5 Breakout Distribuído.29 2.2 Otimização de Restrições Distribuídas.31 2.2.1 Adotar.32 2.2.2 OptAPO.37 Exercícios.38

**3 Jogos de Forma Padrão e Estendida 39**
3.1 Jogos na Forma Normal.39 3.1.1 Conceitos de Solução.40 3.1.2 Jogos Famosos.42 3.1.3 Jogos Repetidos.44 3.2 Jogos em Formato Estendido.45 3.2.1 Conceitos de Solução.46 3.3 Encontrando uma solução.47 Exercícios.47

**4 Jogos de Forma Característica e Formação de Coalizão 49**
4.1 Forma Característica dos Jogos.49 4.1.1 Conceitos de Solução.50 4.1.2 Encontrando a Estrutura de Coalizão Ideal.55 4.2 Formação de Coalizão.57 Exercícios.57

**5 Learning in Multiagent Systems** **59**
5.1 The Machine Learning Problem . . . 59
5.2 Cooperative Learning . . . 61
5.3 Repeated Games . . . 61
5.3.1 Fictitious Play . . . 61
5.3.2 Replicator Dynamics . . . 63
5.3.3 The AWESOME Algorithm . . . 66
5.4 Stochastic Games . . . 67
5.4.1 Reinforcement Learning . . . 68
5.5 General Theories for Learning Agents . . . 71
5.5.1 CLRI Model . . . 71
5.5.2 N-Level Agents . . . 73
5.6 Collective Intelligence . . . 74
5.7 Summary . . . 76
5.8 Recent Advances . . . 76
Exercises . . . 76

**6 Negotiation** **77**
6.1 The Bargaining Problem . . . 77
6.1.1 Axiomatic Solution Concepts . . . 78
6.1.2 Strategic Solution Concepts . . . 81
6.2 Monotonic Concession Protocol . . . 83
6.2.1 The Zeuthen Strategy . . . 84
6.2.2 One-Step Protocol . . . 86
6.3 Negotiation as Distributed Search . . . 86
6.4 Ad-hoc Negotiation Strategies . . . 87
6.5 The Task Allocation Problem . . . 88
6.5.1 Payments . . . 89
6.5.2 Lying About Tasks . . . 91
6.5.3 Contracts . . . 92
6.6 Complex Deals . . . 94
6.6.1 Annealing Over Complex Deals . . . 95
6.7 Argumentation-Based Negotiation . . . 96
6.8 Negotiation Networks . . . 98
6.8.1 Network Exchange Theory . . . 99
Exercises . . . 101

**7 Auctions** **103**
7.1 Valuations . . . 103
7.2 Simple Auctions . . . 104
7.2.1 Analysis . . . 105
7.2.2 Auction Design . . . 107
7.3 Combinatorial Auctions . . . 107
7.3.1 Centralized Winner Determination . . . 108
7.3.2 Distributed Winner Determination . . . 113
7.3.3 Bidding Languages . . . 115
7.3.4 Preference Elicitation . . . 116
7.3.5 VCG Payments . . . 119
Exercises . . . 119

**8 Voting and Mechanism Design** **121**
8.1 The Voting Problem . . . 121
8.1.1 Possible Solutions . . . 122
8.1.2 Voting Summary . . . 124
8.2 Mechanism Design . . . 124
8.2.1 Problem Description . . . 124
8.2.2 Distributed Mechanism Design . . . 131



**5 Aprendizagem em Sistemas Multiagentes 59**
5.1 O problema do aprendizado de máquina.59 5.2 Aprendizagem Cooperativa.61 5.3 Jogos Repetidos.61 5.3.1 Jogo Fictício.61 5.3.2 Dinâmica do Replicador.63 5.3.3 O Algoritmo INCRÍVEL.66 5.4 Jogos Estocásticos.67 5.4.1 Aprendizado por Reforço.68 5.5 Teorias Gerais para Agentes de Aprendizagem.71 5.5.1 Modelo CLRI.71 5.5.2 Agentes de nível N.73 5.6 Inteligência Coletiva.74 5.7 Resumo.76 5.8 Avanços Recentes.76 Exercícios.76

**6 Negociação 77**
6.1 O problema da negociação.77 6.1.1 Conceitos de Solução Axiomática.78 6.1.2 Conceitos de Soluções Estratégicas.81 6.2 Protocolo de Concessão Monotônica.83 6.2.1 A Estratégia de Zeuthen.84 6.2.2 Protocolo de uma etapa.86 6.3 Negociação como Pesquisa Distribuída.86 6.4 Estratégias de negociação ad hoc.87 6.5 O problema de alocação de tarefas.88 6.5.1 Pagamentos.89 6.5.2 Mentir sobre tarefas.91 6.5.3 Contratos.92 6.6 Transações Complexas.94 6.6.1 Recozimento em Transações Complexas.95 6.7 Negociação Baseada em Argumentação.96 6.8 Redes de Negociação.98 6.8.1 Teoria da Troca de Redes.99 Exercícios.101

**7 Leilões 103**
7.1 Avaliações.103 7.2 Leilões Simples.104 7.2.1 Análise.105 7.2.2 Design de Leilão.107 7.3 Leilões Combinatórios.107 7.3.1 Determinação Centralizada do Vencedor.108 7.3.2 Determinação Distribuída do Vencedor.113 7.3.3 Idiomas de Licitação.115 7.3.4 Elicitação de Preferências.116 7.3.5 Pagamentos VCG.119 Exercícios.119

**8 Votação e Mecanismo Design 121**
8.1 O problema da votação.121 8.1.1 Possíveis soluções.122 8.1.2 Resumo da votação.124

8.2 Projeto do Mecanismo.124
8.2.1 Descrição do Problema.124 8.2.2 Projeto de Mecanismo Distribuído.131

8.2.3 Mechanism Design Summary . . . . . 134

**9 Coordination Using Goal and Plan Hierarchies 137**
9.1 TÆMS . . . . . 137
9.2 GPGP . . . . . 139
9.2.1 Agent Architecture . . . . . 139
9.2.2 Coordination . . . . . 140
9.2.3 Design-to-Criteria Scheduler . . . . . 141
9.2.4 GPGP/TÆMS Summary . . . . . 141

**10 Nature-Inspired Approaches 143**
10.1 Ants and Termites . . . . . 143
10.2 Immune System . . . . . 143
10.3 Physics . . . . . 143

**Bibliography 145**

**Index 153**



8.2.3 Resumo do Projeto do Mecanismo.134

**9 CoordenaçãoUsando Hierarquias de Metas e Planos 137**
9.1 TÆMS .137 9.2 GPGP.139 9.2.1 Arquitetura do Agente.139 9.2.2 Coordenação.140 9.2.3 Agendador de projeto de acordo com critérios.141 9.2.4 GPGP/TÆMS Resumo.141

**10Abordagens inspiradas na natureza 143**
10.1 Formigas e Cupins.143 10.2 Sistema Imunológico.143 10.3 Física.143

**Bibliografia 145**

**Índice 153**

# Preface

The goal of this book is to cover all the material that a competent multiagent practitioner or researcher should be familiar with. Of course, since this is a relatively new field the list of required material is still growing and there is some uncertainty as to what are the most important ideas. I have chosen to concentrate on the theoretical aspects of multiagent systems since these ideas have been around for a long time and are important for a wide variety of applications. I have stayed away from technological issues because these are evolving very fast. Also, it would require another textbook to describe all the distributed programming tools available. A reader interested in the latest multiagent technologies should visit the website `www.multiagent.com`.

The book is linked to a large number of sample NetLogo programs (Wilensky, 1999). These programs are meant to be used by the reader as an aid in understanding emergent decentralized behaviors. It is hard for people, especially those new to distributed systems, to fully grasp the order that can arise from seemingly chaotic simple interactions. As Resnick notices:

> *"But even as the influence of decentralized ideas grows, there is a deep-seated resistance to such ideas. At some deep level, people seem to have strong attachments to centralized ways of thinking. When people see patterns in the world (like a flock of birds), they often assume that there is some type of centralized control (a leader of the flock). According to this way of thinking, a pattern can exist only if someone (or something) creates and orchestrates the pattern. Everything must have a single cause, and ultimate controlling factor. The continuing resistance to evolutionary theories is an example: many people still insist that someone or something must have explicitly designed the complex, orderly structures that we call Life."* (Resnick, 1994)

Resnick points to examples such as surveys of 8–15 year old kids, half of which believe that the government sets all prices and salaries.

Resnick created StarLogo in order to teach the *decentralized* mindset. Wilensky, one of his students, later extended StarLogo and created NetLogo.

The reader is assumed to be familiar with basic Artificial Intelligence techniques (Russell and Norvig, 2003). The reader should also be comfortable with mathematical notation and basic computer science algorithms. The book is written for a graduate or advanced undergraduate audience. I also recommend (Mas-Colell et al., 1995; Osborne and Rubinstein, 1999) as reference books.

## 0.1 Usage

If you are using the PDF version of this document you can click on any of the citations and your PDF reader will take you to the appropriate place in the bibliography. From there you can click on the title of the paper and your web browser will take you to a page with a full description of the paper. You can also get the full paper if you have the user-name and password I provide in class. The password is only available to my students due to licensing restrictions on some of the papers.

Whenever you see an icon such as the one on this margin it means that we have NetLogo implementation of a relevant problem. If you are using the PDF version of this document you can just click on the name and your browser will take you to the appropriate applet. Otherwise, the URL is formed by pre-pending `http://jmvidal.cse.sc.edu/netlogomas/` to the name and appending `.html` at the end. So the URL for this icon is `http://jmvidal.cse.sc.edu/netlogomas/ABTgc.html`.



ABTgc



# Prefácio

O objetivo deste livro é cobrir todo o material com o qual um profissional ou pesquisador multiagente competente deve estar familiarizado. É claro que, como este é um campo relativamente novo, a lista de materiais necessários ainda está a crescer e há alguma incerteza quanto a quais são as ideias mais importantes. Optei por me concentrar nos aspectos teóricos dos sistemas multiagentes, uma vez que essas ideias já existem há muito tempo e são importantes para uma ampla variedade de aplicações. Tenho me afastado das questões tecnológicas porque elas estão evoluindo muito rápido. Além disso, seria necessário outro livro para descrever todas as ferramentas de programação distribuída disponíveis. Um leitor interessado nas mais recentes tecnologias multiagentes deve visitar o website www.multiagent.com.

O livro está vinculado a um grande número de exemplos de programas NetLogo (Wilensky, 1999). Esses programas destinam-se a ser usados pelo leitor como uma ajuda na compreensão dos comportamentos descentralizados emergentes. É difícil para as pessoas, especialmente aquelas que são novas em sistemas distribuídos, compreender totalmente a ordem que pode surgir de i nterações simples aparentemente caóticas. Como Resnick observa:

> *"Mas mesmo à medida que a influência das ideias descentralizadas cresce, há uma resistência profunda a tais ideias. Em algum nível profundo, as pessoas parecem ter fortes apegos a formas centralizadas de pensar. Quando as pessoas vêem padrões no mundo (como um bando de pássaros), muitas vezes assumem que existe algum tipo de controle centralizado (um líder do bando). De acordo com esta forma de pensar, um padrão só pode existir se alguém (ou algo) criar e orquestrar o padrão. Tudo deve ter uma causa única e final. O factor controlador. A resistência contínua às teorias evolucionistas é um exemplo: muitas pessoas ainda insistem que alguém ou alguma coisa deve ter concebido explicitamente as estruturas complexas e ordenadas a que chamamos Vida." (Resnick, 1994)*

Resnick aponta exemplos co mo inquéritos a crianças entre os 8 e os 15 anos, metade das quais acredita que o governo define todos os preços e salários.

Resnick criou o StarLogo para ensinar a *decentralized* mentalidade. Wilensky, um de seus alunos, mais tarde ampliou o StarLogo e criou o NetLogo.

Presume-se que o leitor esteja familiarizado com as técnicas básicas de Inteligência Artificial (Russell e Norvig, 2003). O leitor também deve estar confortável com notação matemática e algoritmos básicos de ciência da computação. O livro foi escrito para um público de pós-graduação ou graduação avançada. Também recomendo (Mas-Colell et al., 1995; Osborne e Rubinstein, 1999) como livros de referência.

## 0.1 Uso

Se você estiver usando a versão PDF deste documento, poderá clicar em qualquer uma das citações e seu PDF leitor o levará ao local apropriado na bibliografia. A partir daí você pode clicar no título do artigo e seu navegador o levará a uma página com uma descrição completa do artigo. Você também pode obter o artigo completo se tiver o nome de usuário e a senha que forneci em aula. A senha está disponível apenas para meus alunos devido a restrições de licenciamento em alguns artigos.

Sempre que você vir um ícone como este nesta margem significa que temos Implementação NetLogo de um problema relevante. Se você estiver usando a versão PDF deste documento, basta clicar no nome e seu navegador o levará ao miniaplicativo apropriado. Caso contrário, o URL é formado pré-pendente http://jmvidal.cse.sc.edu/netlogomas/ ao nome e acrescentando .html no final. Portanto, o URL para este ícone é http://jmvidal.cse.sc.edu/netlogomas/ABTgc.html.



ABTgc

## 0.2 Acknowledgments

I would like to thank the students at the University of South Carolina who have provided much needed feedback on all revisions of this book. Specifically, I thank Jimmy Cleveland, Jiangbo Dang, Huang Jingshan, and Alicia Ruvinsky. I am also especially grateful to faculty members from other Universities who have used this book in their classes and provided me with invaluable feedback. Specifically I thank Ramón F. Brena, Muaz Niazi, and Iyad Rahwan.



## 0.2 Agradecimentos

Gostaria de agradecer aos estudantes da Universidade da Carolina do Sul que forneceram o tão necessário feedback sobre todas as revisões deste livro. Especificamente, agradeço a Jimmy Cleveland, Jiangbo Dang, Huang Jingshan e Alicia Ruvinsky. Também sou especialmente grato aos professores de outras universidades que usaram este livro em suas aulas e me forneceram comentários valiosos. Agradeço especificamente a Ram´a F. Brena, Muaz Niazi e Iyad Rahwan.

## Chapter 1

# Multiagent Problem Formulation

The goal of multiagent systems' research is to find methods that allow us to build complex systems composed of autonomous agents who, while operating on local knowledge and possessing only limited abilities, are nonetheless capable of enacting the desired global behaviors. We want to know how to take a description of what a *system of agents* should do and break it down into individual agent behaviors. At its most ambitious, multiagent systems aims at reverse-engineering emergent phenomena as typified by ant colonies, the economy, and the immune system. Multiagent systems approaches the problem using the well proven tools from game theory, Economics, and Biology. It supplements these with ideas and algorithms from artificial intelligence research, namely planning, reasoning methods, search methods, and machine learning.

A more relaxed introduction to the topics in this chapter is given in (Russell and Norvig, 2003, Chapters 16–17).

These disparate influences have lead to the development of many different approaches, some of which end up being incompatible with each other. That is, it is sometimes not clear if two researchers are studying variations of the same problem or completely different problems. Still, the model that has thus far gained most attention, probably due to its flexibility as well as its well established roots in game theory and artificial intelligence, is that of modeling agents as utility maximizers who inhabit some kind of Markov decision process. This is the model we present in this chapter and the one we will use most often throughout the book. Note, however, that while this is a very general model it is not always the best way to describe the problem. We will also examine the traditional artificial intelligence planning model and, in later chapters, models inspired by Biology. Another popular model is that of agents as logical inference machines. This model is favored by those working on semantic or logical applications. We will sometimes make reference to **deductive** agents which can deduce facts based on the rules of logic, and **inductive** agents (see Chapter 5) which use machine learning techniques to extrapolate conclusions from the given evidence.

DEDUCTIVE
INDUCTIVE

## 1.1 Utility

A common simplifying assumption is that an agent's preferences are captured by a **utility function**. This function provides a map from the states of the world or outcome of game to a real number. The bigger the number the more the agent likes that particular state. Specifically, given that $S$ is the set of states in the world the agent can perceive then agent $i$'s utility function is of the form

UTILITY FUNCTION

In game theory it is known as the von Neumann-Morgenstern utility function.

$$u_i : S \rightarrow \Re. \tag{1.1}$$

Notice also that the states are defined as those states of the world that the agent can perceive. For example, if a robot has only one sensor that feeds him a binary input, say 1 if it is bright and 0 if its dark, then that robot has a utility function defined over only two states regardless of how complicated the real world might be, such as $u_i(0) = 5$, $u_i(1) = 10$. In practice, agents have sophisticated inputs and it is impractical to define a different output for each input. Thus, most agents also end up mapping their raw inputs to a smaller set of world states. Creating



## Capítulo 1

# Formulação de problema multiagente

O objectivo da investigação em sistemas multiagentes é encontrar métodos que nos permitam construir sistemas complexos compostos por agentes autónomos que, embora operem com base no conhecimento local e possuam apenas capacidades limitadas, sejam, no entanto, capazes de implementar os comportamentos globais desejados. Queremos saber como obter uma descrição do que um sistema de agentes deve fazer e decompô-la em comportamentos individuais dos agentes. Os seus sistemas multiagentes mais ambiciosos visam a engenharia reversa de fenómenos emergentes, tipificados pelas colónias de f ormigas, pela economia e pelo sistema imunitário. Os sistemas multiagentes abordam o problema usando ferramentas comprovadas da teoria dos jogos, economia e biologia. Suplementa-os com ideias e algoritmos de investigação em inteligência artificial, nomeadamente planeamento, métodos de raciocínio, métodos de pesquisa e aprendizagem automática.

Uma introdução mais descontraída aos tópicos deste capítulo é apresentada em (Russell e Norvig, 2003, Capítulos 16–17).

Estas influências díspares levaram ao desenvolvimento de muitas abordagens diferentes, algumas das quais acabam por ser incompatíveis entre si. Ou seja, às vezes não fica claro se dois pesquisadores estão estudando variações do mesmo problema ou problemas completamente diferentes. Ainda assim, o modelo que até agora tem recebido mais atenção, provavelmente devido à sua flexibilidade, bem como às suas raízes bem estabelecidas na teoria dos jogos e na inteligência artificial, é o da modelação de agentes como maximizadores de utilidade que habitam algum tipo de processo de decisão de Markov. Este é o modelo que apresentamos neste capítulo e aquele que usaremos com mais frequência ao longo do livro. Observe, entretanto, que embora este seja um modelo muito geral, nem sempre é a melhor maneira de descrever o problema. Examinaremos também o modelo tradicional de planejamento de inteligência artificial e, em capítulos posteriores, modelos inspirados na Biologia. Outro modelo popular é o dos agentes como máquinas de inferência lógica. Este modelo é preferido por quem trabalha com aplicações semânticas ou lógicas. Às vezes faremos referência a dedutivos

agentes que podem deduzir fatos com base nas regras da lógica e agentes indutivos (ver Capítulo 5) que utilizam técnicas de aprendizado de máquina para extrapolar conclusões a partir das evidências fornecidas.

DEDUCTIVE
INDUCTIVE

## 1.1 Utilidade

Uma suposição simplificadora comum é que as preferências de um agente são capturadas por uma função de utilidade. Esta função fornece um mapa dos estados do mundo ou

resultado do jogo para um número real. Quanto maior o número mais o agente gosta aquele estado específico. Especificamente, dado que $S$ é o conjunto de estados no mundo que o agente pode perceber, então a função de utilidade do agente $i$ é da forma

UTILITY FUNCTION

Na teoria dos jogos, é conhecida como função de utilidade de von Neumann-Morgenstern.

$$u_i : S \rightarrow \Re. \tag{1.1}$$

Observe também que os estados são definidos como aqueles estados do mundo que o agente pode perceber. Por exemplo, se um robô tem apenas um sensor que o alimenta com uma entrada binária, digamos 1 se for claro e 0 se for escuro, então esse robô tem uma f unção de utilidade definida em apenas dois estados, independentemente de quão co mplicado o mundo real possa ser, como $u_i(0) = 5$, $u_i(1) = 10$. Na prática, os agentes possuem entradas sofisticadas e é impraticável definir uma saída diferente para cada entrada. Assim, a maioria dos agentes acaba também por mapear os seus insumos brutos para um conjunto menor de estados mundiais. Criando

this mapping function can be challenging as it requires a deep understanding of the problem setting.

PREFERENCE ORDERING

Given an agent's utility function we can define a **preference ordering** for the agent over the states of the world. By comparing the utility values of two states we can determine which one is preferred by the agent. This ordering has the following properties.

- *reflexive*: $u_i(s) \geq u_i(s)$
- *transitive*: If $u_i(a) \geq u_i(b)$ and $u_i(b) \geq u_i(c)$ then $u_i(a) \geq u_i(c)$.
- *comparable*: $\forall_{a,b}$ either $u_i(a) \geq u_i(b)$ or $u_i(b) \geq u_i(a)$.

We can use utility functions to describe the behavior of almost every agent. Utility functions are also useful for capturing the various tradeoffs that an agent must make, along with the value or expected value of its actions. For example, we can say that a robot receives a certain payment for delivering a package but also incurs a cost in terms of the electricity used as well as the opportunity cost—he could have been delivering other packages. If we translate all these payments and costs into utility numbers then we can easily study the tradeoffs among them.

Once we have defined a utility function for all the agents then all they have to do is take actions which maximize their utility. As in game theory, we use the word **selfish** to refer to a rational agent that wants to maximize its utility. Notice that this use is slightly different from the everyday usage of the word which often implies a desire to harm others, a true selfish agent cares exclusively about its utility. The use of selfish agents does not preclude the implementation of cooperative multiagent systems. We can view a cooperative multiagent system as one where the agents' utility functions have been defined in such a way so that the agents seem to cooperate. For example, if an agent receives a higher utility for helping other agents then the resulting behavior will seem cooperative to an observer even though the agent is acting selfishly.

SELFISH

**selfish**, adj. Devoid of consideration for the selfishness of others. —*The Devil's Dictionary*

We use utility functions as a succinct way of representing an agent's behavior. In the actual implementations these functions will sometimes be probabilistic, sometimes they will be learned as the agent acts in the world, sometimes they will be incomplete, sometimes they will be the result of some inferencing or induction. Still, by assuming that the function exists we can more easily design and study a multiagent system.

### 1.1.1 Utility is Not Money

Note that while utility represents an agent's preferences it is not necessarily equated with money. In fact, the utility of money has been found to be roughly logarithmic. For example, say Bill has $100 million while Tim has $0 in the bank. They both contemplate the possibility of winning one million dollars. Clearly, that extra million will make a significant difference in Tim's lifestyle while Bill's lifestyle will remain largely unchanged, thus Tim's utility for the same million dollars is much larger than Bill's. There is experimental evidence that shows most people have these type of conditional preferences. Very roughly, people's utility for smaller amounts of money is linear but for larger amounts it becomes logarithmic. Notice that we are considering the **marginal utility** of money, that is, it is the utility for the next million dollars. We assume that both Bill and Tim have the same utility for their first million dollars.

MARGINAL UTILITY

You can determine your own curve by repeatedly asking yourself for increasing values of $x$: which one would you rather have, $x$ dollars, or a 50/50 chance at winning $2 \cdot x$ dollars? When you start preferring the first choice you are dropping below the $y = x$ line.

Recent results in behavioral economics have shown that the true utility function is more complicated than what can be captured by a mathematical function (Camerer et al., 2003). One experiment shows that people are less likely to take a gamble when the wording of the question is such that the person must give back some money, even if the gamble is mathematically identical to another one that uses different wording. The experiment in question is the following: which option would you prefer (a) I give you $10,000 and a 50/50 chance at winning another $10,000,



esta função de mapeamento pode ser desafiadora, pois requer uma compreensão profunda da configuração do problema.

PREFERENCE ORDERING

Dada a função de utilidade de um agente, podemos definir uma ordem de preferência para o agente sobre os estados do mundo. Comparando os valores de utilidade de dois estados podemos determinar qual deles é o preferido pelo agente. Essa ordenação possui as seguintes propriedades.

- *reflexivo:* $u_i(s) \geq u_i(s)$
- transitivo: se $u_i(a) \geq u_i(b)$ e $u_i(b) \geq u_i(c)$ então $u_i(a) \geq u_i(c)$.
- comparável: $\forall_{a,b}$ ou $u_i(a) \geq u_i(b)$ ou $u_i(b) \geq u_i(a)$.

Podemos usar funções utilitárias para descrever o comportamento de quase todos os agentes. As funções de utilidade também são úteis para capturar as diversas compensações que um agente deve fazer, juntamente com o valor ou valor esperado de suas ações. Por exemplo, podemos dizer que um robô recebe um determinado pagamento pela entrega de um pacote, mas também incorre num custo em termos da eletricidade utilizada, bem como do custo de oportunidade – ele poderia estar entregando outros pacotes. Se traduzirmos t odos esses pagamentos e custos em números de serviços públicos, poderemos facilmente estudar as compensações entre eles.

Uma vez definida uma função de utilidade para todos os agentes, tudo o que eles precisam f azer é realizar ações que maximizem sua utilidade. Tal como na teoria dos jogos, usamos a palavra egoísta para nos referirmos a um agente racional que pretende maximizar a sua utilidade.

SELFISH

egoísta, adj. Desprovido de consideração pelo egoísmo dos outros. —*The Devil's Dictionary*

embora este uso seja ligeiramente diferente do uso diário da palavra, que muitas vezes implica um desejo de prejudicar os outros, um verdadeiro agente egoísta preocupa-se exclusivamente com a sua utilidade. A utilização de agentes egoístas não impede a implementação de sistemas multiagentes cooperativos. Podemos ver um sistema multiagente cooperativo como aquele em que o

as funções de utilidade dos agentes foram definidas de tal forma que os agentes parecem cooperar. Por exemplo, se um agente recebe uma utilidade maior por ajudar outros agentes, então o comportamento resultante parecerá cooperativo para um observador, mesmo que o agente esteja agindo de forma egoísta.

Usamos funções de utilidade como uma forma sucinta de representar o comportamento de um agente. Nas implementações reais estas funções serão por vezes probabilísticas, por vezes serão aprendidas à medida que o agente actua no mundo, por vezes serão incompletas, por vezes serão o resultado de alguma inferência ou indução. Ainda assim, assumindo que a função existe, podemos projetar e estudar mais facilmente um sistema multiagente.

### 1.1.1 Utilidade não é dinheiro

Note-se que embora a utilidade represente as preferências de um agente, não é necessariamente equiparada a dinheiro. Na verdade, descobriu-se que a utilidade do dinheiro é aproximadamente logarítmica. Por exemplo, digamos que Bill tenha US$ 100 milhões enquanto Tim tenha US$ 0 no banco. Ambos contemplam a possibilidade de ganhar um milhão de dólares. Claramente, esse milhão extra fará uma diferença significativa no estilo de vida de Tim, enquanto o estilo de vida de Bill permanecerá praticamente inalterado, portanto, a utilidade de Tim para o mesmo milhão de dólares é muito maior do que a de Bill. Existem evidências experimentais que mostram que a maioria das pessoas tem esse tipo de preferência condicional. Muito grosso modo, a utilidade das pessoas para quantias menores de dinheiro é linear, mas para quantias maiores torna-se logarítmica. Observe que estamos considerando a utilidade marginal do dinheiro, ou seja, é a utilidade para o próximo milhões de dólares. Assumimos que Bill e Tim têm a mesma utilidade para o seu primeiro milhão de dólares.

MARGINAL UTILITY

Você pode determinar sua própria curva perguntando-se repetidamente sobre valores crescentes de$x$: qual você preferiria ter, $x$ dólares ou uma $50/50$chance de ganhar $2 \cdot x$ dólares? Quando você começa a preferir a primeira opção, você fica abaixo da linha $y = x$ .

Resultados recentes em economia comportamental mostraram que a verdadeira função de utilidade é mais complicada do que aquilo que pode ser capturado por uma função matemática (Camerer et al., 2003). Uma experiência mostra que as pessoas são menos propensas a apostar quando a formulação da pergunta é tal que a pessoa deve devolver algum dinheiro, mesmo que a aposta seja matematicamente idêntica a outra que utilize uma formulação diferente. O experimento em questão é o seguinte: qual opção você prefere (a) eu lhe dou US$ 10.000 e uma chance de 50/50 de ganhar outros US$ 10.000,

or (b) I give you $20,000 and then flip a coin and if it comes out heads you have to give me $10,000. Both of these are equivalent but a large majority of people prefer option (a).

The fact that people are not entirely rational when making choices about money becomes important to us when we start to think about building agents that will buy, sell, or negotiate for humans. It is reasonable to assume that in these systems people will demand agents that behave like people. Otherwise, the agent would be making decisions the user finds disagreeable. People's apparent irrationality also opens up the possibility that we could build agents that are better negotiators than us.

### 1.1.2 Expected Utility

Once we have utility functions we must then determine how the agents will use them. We assume each agent has some sensors which it can use to determine the state of the world and some effectors which it can use to take action. These actions can lead to new states of the world. For example, an agent senses its location and decides to move forward one foot. Of course, these sensors and effectors might not operate perfectly: the agent might not move exactly one foot or its sensors might be noisy. Let's assume, however, that the agent does know the probability of reaching state $s'$ given that it is in state $s$ and takes action $a$. This probability is given by $T(s, a, s')$ which we call the **transition function**. Since the transition function returns a probability then it must be true that the sum of $T(s, a, s')$ over all possible $a$ and $s'$ is equal to one.

TRANSITION FUNCTION

Using this transition function the agent can then calculate its **expected utility** for taking action $a$ in state $s$ as

EXPECTED UTILITY

$$E[u_i, s, a] = \sum_{s' \in S} T(s, a, s')u_i(s'), \tag{1.2}$$

where $S$ is the set of all possible states.

An agent can use the expected utility function to determine the value of a piece of added information it might acquire such as an extra sensor reading or a message from another agent. For example, consider a piece of new information that leads the agent to determine that it is not really in state $s$ but it is instead in state $t$. Perhaps a second agent tells him that he is near a precipice and not in the middle of a plateau as the agent originally thought. In this case the agent can compare the expected utility it would have received under the old state against the new one it will now receive. That is, the **value of information** for that piece of information is given by

VALUE OF INFORMATION

$$E[u_i, t, \pi_i(t)] - E[u_i, t, \pi_i(s)]. \tag{1.3}$$

The value of the piece of information is the expected utility the agent receives now that he knows he is in $t$ and takes action accordingly minus the utility it would have received if he had instead mistakenly assumed it was in $s$ and taken action accordingly. This equation provides a simple and robust way for agents to make meta-level decisions about what knowledge to seek: what messages to send, which sensors to turn on, how much deliberation to perform, etc.

Specifically, an agent can use it to play *"what if"* games to determine its best action in the same way a person might. For example, say a doctor has made a preliminary decision about which drug to prescribe for a patient but is uncertain about the patient's specific disease. She can calculate the value of information acquired from performing various tests which could identify the specific disease. If a test, regardless of its outcome, would still lead the doctor to prescribe the same drug then the value of the information from that test's result is zero: it does not lead to a change in action so (1.3) is zero. On the other hand, a test that is likely to change the doctor's action will likely have a high value of information.



ou (b) eu lhe dou $ 20.000 e depois jogue uma moeda e, se der cara, você terá que me dar $ 10.000. Ambas são equivalentes, mas a grande maioria das pessoas prefere a opção (a).

O facto de as pessoas não serem inteiramente racionais quando fazem escolhas sobre dinheiro torna-se importante para nós quando começamos a pensar na construção de agentes que comprem, vendam ou negociem em nome dos humanos. É razoável supor que nestes sistemas as pessoas exigirão agentes que se comportem co mo pessoas. Caso contrário, o agente estaria tomando decisões que o usuário considera desagradáveis. A aparente irracionalidade das pessoas também abre a possibilidade de podermos construir agentes que sejam melhores negociadores do que nós.

### 1.1.2 Utilidade Esperada

Uma vez que tenhamos funções de utilidade, devemos determinar como os agentes as utilizarão. Assumimos que cada agente possui alguns sensores que pode usar para determinar o estado do mundo e alguns efetores que pode usar para agir. Essas ações podem levar a novos estados do mundo. Por exemplo, um agente sente sua localização e decide avançar um pé. É claro que esses sensores e efetores podem não funcionar perfeitamente: o agente pode não mover exatamente um pé ou seus sensores podem ser barulhentos. Suponhamos, entretanto, que o agente conheça a probabilidade de atingir o estado $s'$, visto que está no estado $s$ e executa a ação $a$. Esta probabilidade é dada por $T(s, a, s')$ que chamamos de função de transição. Como a função de transição

TRANSITION FUNCTION

retorna uma probabilidade, então deve ser verdade que a soma de $T(s, a, s')$ sobre todos os possíveis$a$ e $s'$ é igual a um.

Usando esta função de transição, o agente pode então calcular sua utilidade esperada por agir $a$ no estado $s$ como

EXPECTED UTILITY

$$E[u_i, s, a] = \sum_{s' \in S} T(s, a, s')u_i(s'), \tag{1.2}$$

onde $S$ é o conjunto de todos os estados possíveis.

Um agente pode usar a função de utilidade esperada para determinar o valor de uma informação adicional que ele pode adquirir, como uma leitura extra de um sensor ou uma mensagem de outro agente. Por exemplo, considere uma nova informação que leva o agente a determinar que não está realmente no estado $s$, mas sim no estado $t$. Talvez um segundo agente lhe diga que ele está perto de um precipício e não no meio de um platô como o agente pensava originalmente. Neste caso, o agente pode co mparar a utilidade esperada que teria recebido no estado antigo com a nova que receberá agora. Ou seja, o valor da informação para aquela informação

VALUE OF INFORMATION

é dado por

$$E[u_i, t, \pi_i(t)] - E[u_i, t, \pi_i(s)]. \tag{1.3}$$

O valor da informação é a utilidade esperada que o agente recebe agora que sabe que está em $t$ e age de acordo, menos a utilidade que teria recebido se, em vez disso, tivesse assumido erroneamente que estava em $s$ e agido de acordo. Esta equação fornece uma maneira simples e robusta para os agentes tomarem decisões de meta-nível sobre qual conhecimento buscar: quais mensagens enviar, quais sensores ativar, quanta deliberação realizar, etc.

Especificamente, um agente pode usá-lo para jogar jogos "e se" para determinar s ua melhor ação, da mesma forma que uma pessoa faria. Por exemplo, digamos que um médico tomou uma decisão preliminar sobre qual medicamento prescrever para um paciente, mas não tem certeza sobre a doença específica do paciente. Ela pode calcular o valor das informações adquiridas na realização de diversos testes que podem identificar a doença específica. Se um teste, independentemente do seu resultado, ainda levar o médico a prescrever o mesmo medicamento, então o valor da informação do resultado desse teste é zero: não leva a uma mudança na ação, pelo que (1.3) é zero. Por outro lado, um exame que provavelmente mudará a ação do médico provavelmente terá um alto valor de informação.

| $s$ | $r(s)$ |
|---|---|
| $s_1$ | 0 |
| $s_2$ | 0 |
| $s_3$ | 1 |
| $s_4$ | 0 |

| $s_i$ | $a$ | $s_j$ | $T(s_i, a, s_j)$ |
|---|---|---|---|
| $s_1$ | $a_1$ | $s_1$ | 0.2 |
| $s_1$ | $a_1$ | $s_2$ | 0.8 |
| $s_1$ | $a_2$ | $s_1$ | 0.2 |
| $s_1$ | $a_2$ | $s_4$ | 0.8 |
| $s_2$ | $a_2$ | $s_2$ | 0.2 |
| $s_2$ | $a_2$ | $s_3$ | 0.8 |
| $s_2$ | $a_3$ | $s_2$ | 0.2 |
| $s_2$ | $a_3$ | $s_1$ | 0.8 |
| $s_3$ | $a_4$ | $s_2$ | 1 |
| $s_3$ | $a_3$ | $s_4$ | 1 |
| $s_4$ | $a_1$ | $s_4$ | 0.1 |
| $s_4$ | $a_1$ | $s_3$ | 0.9 |
| $s_4$ | $a_4$ | $s_4$ | 0.2 |
| $s_4$ | $a_4$ | $s_1$ | 0.8 |

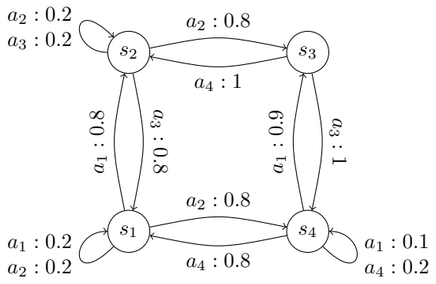



**Figure 1.1:** Graphical representation of a sample Markov decision process along with values for the transition and reward functions. We let the start state be $s_1$.



Andrey Markov. 1856–1922.

MARKOV DECISION PROCESS DETERMINISTIC

NON-DETERMINISTIC

POLICY

OPTIMAL POLICY

MAXIMUM EXPECTED UTILITY

## 1.2 Markov Decision Processes

So far we have assumed that only the agent can change the state of the world. But, the reality in most cases is that agents inhabit an environment whose state changes either because of the agent's action or due to some external event. We can think of the agent sensing the state of the world then taking an action which leads it to a new state. We also make the further simplifying assumption that the choice of the new state therefore depends only on the agent's current state and the agent's action. This idea is formally captured by a **Markov decision process** or MDP.

**Definition 1.1** (Markov Decision Processess)**.** *An* MDP *consists of an initial state* $s_1$ *taken from a set of states* $S$*, a transition function* $T(s, a, s')$ *and a reward function* $r : S \to \Re$*.*

The transition function $T(s, a, s')$ gives us the probability that an agent in state $s$ who takes action $a$ will end up in state $s'$. For a purely **deterministic** world we have that this function will be zero for all $s'$ of each given $s, a$ pair except one, for which it will be one. That is, in a deterministic world an agent's action has a completely predictable effect. This is not the case for a **non-deterministic** environment where an agent's action could lead to a number of different states, depending on the value of this fixed probability function.

Figure 1.1 shows a typical visualization of an MDP, this one with four states. Notice that in this example the agent's actions are limited based on the state, for example, in state $s_1$ the agent can only take actions $a_1$ and $a_2$. Also note that when the agent is in state $s_1$ and it takes action $a_1$ there is a 0.2 probability that it will stay in the same state. In this example the agent receives a reward of 1 only when it reaches $s_3$, at which point its only option is to leave $s_3$.

The agent's behavior is captured by a **policy** which is a mapping from states to actions. We will use $\pi$ to denote the agent's policy. The agent's goal then becomes that of finding its **optimal policy** which is the one that maximizes its expected utility, as we might expect a selfish agent to do. This strategy is known as the principle of **maximum expected utility**. Agent $i$'s optimal policy can thus be defined as

$$\pi_i^*(s) = \arg\max_{a \in A} E[u_i, s, a]. \tag{1.4}$$

But, in order to expand the expected value within that equation we must first determine how to handle future rewards. That is, is it better to take 100 actions with no reward just to get to a state which gives the agent a reward of 100, or is



| $s$ | $r(s)$ |
|---|---|
| $s_1$ | 0 |
| $s_2$ | 0 |
| $s_3$ | 1 |
| $s_4$ | 0 |

| $s_i$ | $a$ | $s_j$ | $T(s_i, a, s_j)$ |
|---|---|---|---|
| $s_1$ | $a_1$ | $s_1$ | 0.2 |
| $s_1$ | $a_1$ | $s_2$ | 0.8 |
| $s_1$ | $a_2$ | $s_1$ | 0.2 |
| $s_1$ | $a_2$ | $s_4$ | 0.8 |
| $s_2$ | $a_2$ | $s_2$ | 0.2 |
| $s_2$ | $a_2$ | $s_3$ | 0.8 |
| $s_2$ | $a_3$ | $s_2$ | 0.2 |
| $s_2$ | $a_3$ | $s_1$ | 0.8 |
| $s_3$ | $a_4$ | $s_2$ | 1 |
| $s_3$ | $a_3$ | $s_4$ | 1 |
| $s_4$ | $a_1$ | $s_4$ | 0.1 |
| $s_4$ | $a_1$ | $s_3$ | 0.9 |
| $s_4$ | $a_4$ | $s_4$ | 0.2 |
| $s_4$ | $a_4$ | $s_1$ | 0.8 |

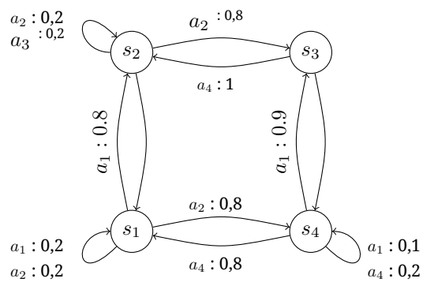



Figura 1.1: Representação gráfica de uma amostra de processo de decisão de Markov juntamente com valores para as funções de transição e recompensa. Deixamos o estado inicial ser $s_1$.



Andrey Markov. 1856–1922.

MARKOV DECISION PROCESS DETERMINISTIC

NON-DETERMINISTIC

POLICY

OPTIMAL POLICY

MAXIMUM EXPECTED UTILITY

### 1.2 Processos de Decisão Markovianos

Até agora assumimos que apenas o agente pode mudar o estado do mundo. Mas, na maioria dos casos, a realidade é que os agentes habitam um ambiente cujo estado muda devido à ação do agente ou devido a algum evento externo. Podemos pensar no agente que sente o estado do mundo e depois toma uma acção que o leva a um novo estado. Também fazemos a suposição ainda mais simplificada de que a escolha do novo estado depende, portanto, apenas do estado atual do agente e da ação do agente. Esta ideia é formalmente capturada por um processo de decisão de Markov ou MDP.

*Definição 1.1 (Processo de Decisão de Markov). Um* MDP *consiste em um estado inicial* $s_1$ *retirado de um conjunto de estados* $S$*, uma função de transição* $T(s, a, s')$ *e uma função de recompensa* $r : S \to \Re$*.*

A função de transição $T(s, a, s')$ nos dá a probabilidade de que um agente no estado $s$ que executa uma ação $a$ acabe no estado $s'$. Por um mundo puramente determinista temos que esta função será zero para todos $s'$ de cada par $s, a$ dado, exceto um, para o qual será um. Isto é, num mundo determinista a acção de um agente tem um efeito completamente previsível. Este não é o caso de uma abordagem não determinística ambiente onde a ação de um agente pode levar a vários estados diferentes, dependendo do valor desta função de probabilidade fixa.

A Figura 1.1 mostra uma visualização típica de um MDP, este com quatro estados. Observe que neste exemplo as ações do agente são limitadas com base no estado, por exemplo, no estado $s_1$ o agente só pode realizar ações $a_1$ e $a_2$. Observe também que quando o agente está no estado $s_1$ e executa uma ação $a_1$ , há uma probabilidade de 0,2 de que ele permaneça no mesmo estado. Neste exemplo o agente recebe uma recompensa de 1 somente quando chega a $s_3$, momento em que sua única opção é sair de $s_3$.

O comportamento do agente é capturado por uma política que é um mapeamento de estados para ações. Usaremos $\pi$ para indicar a política do agente. O objetivo do agente passa então a ser encontrar a sua política ótima, que é aquela que maximiza o seu valor esperado. utilidade, como poderíamos esperar que um agente egoísta fizesse. Esta estratégia é conhecida como princípio da utilidade máxima esperada. A política ótima do agente $i$ pode, portanto, ser definido como

$$\pi_i^*(s) = \arg\max_{a \in A} E[u_i, s, a]. \tag{1.4}$$

Mas, para expandir o valor esperado dentro dessa equação, devemos primeiro determinar como lidar com as recompensas futuras. Ou seja, é melhor realizar 100 ações sem recompensa apenas para chegar a um estado que dê ao agente uma recompensa de 100, ou é

it better to take 100 actions each of which gives the agent a reward of 1? Since no agent is likely to live forever we will not want to wait forever for that big reward. On the other hand, it seems reasonable to give up a small reward in the next couple of steps if we know there is a very big reward afterwards. As such, we generally prefer to use **discounted rewards** which allow us to smoothly reduce the impact of rewards that are farther off in the future. We do this by multiplying the agent's future rewards by an ever-decreasing **discount factor** represented by $\gamma$, which is a number between zero and one.

DISCOUNTED REWARDS

DISCOUNT FACTOR

For example, if an agent using policy $\pi$, starts out in state $s_1$ and visits states $s_1, s_2, s_3, \ldots$ then we say that its discounted reward is given by

$$\gamma^0 r(s_1) + \gamma^1 r(s_2) + \gamma^2 r(s_3) + \cdots \tag{1.5}$$

Note, however, that we only know $s_1$. The rest of the states, $s_2, s_3, \ldots$ depend on the transition function $T$. That is, from state $s_1$ we know that the probability of arriving at any other state $s'$ given that we took action $a$ is $T(s, a, s')$ but we do not know which specific state the agent will reach. If we assume that the agent is an utility maximizing agent then we know that it is going to take the action which maximizes its expected utility. So, when in state $s$ the agent will take action given by (1.4), which when expanded using (1.2) gives us

$$\pi^*(s) = \arg\max_a \sum_{s'} T(s, a, s') u(s'), \tag{1.6}$$

where $u(s')$ is the utility the agent can expect from reaching $s'$ and then continuing on to get more rewards for successive states while using $\pi^*$. We can now work backwards and determine what is the value of $u(s)$. We know that when the agent arrives in $s$ it receives a reward of $r(s)$, but we now also know that because it is in $s$ it can take its action based on $\pi^*(s)$ and will get a new reward at the next time. Of course, that reward will be discounted by $\gamma$. As such, we can define the real utility the agent receives for being in state $s$ as

$$u(s) = r(s) + \gamma \max_a \sum_{s'} T(s, a, s') u(s'). \tag{1.7}$$



Richard Bellman. 1920–1984. Norbert Wiener prize, IEEE Medal of Honor.

BELLMAN EQUATION

This is known as the **Bellman equation** and it captures the fact that an agent's utility depends not only on its immediate rewards but also on its future discounted rewards. Notice that, once we have this function defined for all $s$ then we also have the agent's true optimal policy $\pi^*(s)$, namely, the agent should take the action which maximizes its expected utility, as given in (1.6).

The problem we now face is how to calculate $u(s)$ given the MDP definition. One approach is to solve the set of equations formed. Given $n$ states we have $n$ Bellman equations each one with a different variable. We thus have a system of $n$ equations with $n$ variables so, theoretically, we can find values for all these variables. In practice, however, solving this set of equations is not easy because of the max operator in the Bellman equation which makes the equations non-linear.

Another approach for solving the problem is to use **value iteration**. In this method we start by setting the values of $u(s)$ to some arbitrary numbers and then iteratively improve these numbers using the **Bellman update** equation:

VALUE ITERATION

BELLMAN UPDATE

$$u^{t+1}(s) \leftarrow r(s) + \gamma \max_a \sum_{s'} T(s, a, s') u^t(s'). \tag{1.8}$$

The reader will notice that this equation is nearly the same as (1.7), except that we are now updating the $u$ values over time. It has been shown that this process will eventually, and often rapidly, converge to the real values of $u(s)$. We also know that if the maximum change in utility for a particular time step is less than $\epsilon(1-\gamma)/\gamma$ then the error is less than $\epsilon$. This fact can be used as a stopping condition.



`valueiter`

The VALUE-ITERATION algorithm is shown in figure 1.2. This algorithm is instance of **dynamic programming** in which the optimal solution to a problem is

DYNAMIC PROGRAMMING



é melhor realizar 100 ações, cada uma das quais dá ao agente uma recompensa de 1? Como nenhum agente provavelmente viverá para sempre, não desejaremos esperar para sempre por essa grande recompensa. Por outro lado, parece razoável abrir mão de uma pequena recompensa nas próximas etapas se soubermos que haverá uma recompensa muito grande posteriormente. Como tal, geralmente preferimos usar recompensas com desconto que nos permitam reduzir suave mente o impacto

de recompensas que estão mais distantes no futuro. Fazemos isso multiplicando as recompensas futuras do agente por um fator de desconto cada vez menor representado por $\gamma$, que é um

DISCOUNTED REWARDS

DISCOUNT FACTOR

número entre zero e um.

Por exemplo, se um agente que usa a política $\pi$ começa no estado $s_1$ e visita os estados$s_1, s_2, s_3, \ldots$ então dizemos que sua recompensa com desconto é dada por

$$\gamma^0 r(s_1) + \gamma^1 r(s_2) + \gamma^2 r(s_3) + \cdots \tag{1.5}$$

Observe, entretanto, que conhecemos apenas $s_1$. O resto dos estados, $s_2, s_3, \ldots$ dependem da função de transição $T$. Ou seja, a partir do estado $s_1$ sabemos que a probabilidade de chegar a qualquer outro estado $s'$ dado que tomamos medidas $a$ é $T(s, a, s')$ mas não sabemos qual estado específico o agente alcançará. Se assumirmos que o agente é um agente maximizador de utilidade, então sabemos que ele executará a ação que maximiza sua utilidade esperada. Então, quando estiver no estado $s$ , o agente executará a ação dada por (1.4), que quando expandida usando (1.2) nos dá

$$\pi^*(s) = \arg\max_a \sum_{s'} T(s, a, s') u(s'), \tag{1.6}$$

onde $u(s')$ é a utilidade que o agente pode esperar alcançar $s'$ e então continuar para obter mais recompensas para estados sucessivos ao usar $\pi^*$. Agora podemos trabalhar de trás para frente e determinar qual é o valor de $u(s)$. Sabemos que quando o agente chega em $s$ ele recebe uma recompensa de $r(s)$, mas agora também sabemos que por estar em $s$ele pode realizar sua ação com base em $\pi^*(s)$ e receberá uma nova recompensa na próxima vez. Claro, essa recompensa terá um desconto de $\gamma$. Assim, podemos definir a utilidade real que o agente recebe por estar no estado $s$ como

$$u(s) = r(s) + \gamma \max_a \sum_{s'} T(s, a, s') u(s'). \tag{1.7}$$



Richard Bellman. 1920–1984. Norbert Wienerprize, Medalha de Honra do IEEE.

BELLMAN EQUATION

Isso é conhecido como equação de Bellman e captura o fato de que o a utilidade depende não apenas de suas recompensas imediatas, mas também de suas f uturas recompensas com desconto. Observe que, uma vez definida esta função para todos $s$ , então também temos a verdadeira política ótima do agente $\pi^*(s)$, ou seja, o agente deve realizar a ação que maximiza sua utilidade esperada, conforme dado em (1.6).

O problema que enfrentamos agora é como calcular $u(s)$ dada a definição MDP. Uma abordagem é resolver o conjunto de equações formadas. Dados $n$ estados, temos $n$equações de Bellman, cada uma com uma variável diferente. Temos, portanto, um sistema de $n$ equações com $n$ variáveis, então, teoricamente, podemos encontrar valores para todas essas variáveis. Na prática, porém, resolver este conjunto de equações não é fácil por causa do operador max na equação de Bellman, que torna as equações não lineares.

VALUE ITERATION

BELLMAN UPDATE

Outra abordagem para resolver o problema é usar iteração de valor. Nesta método, começamos definindo os valores de $u(s)$ para alguns números arbitrários e, em seguida, melhoramos iterativamente esses números usando a equação de atualização de Bellman:

$$u^{t+1}(s) \leftarrow r(s) + \gamma \max_a \sum_{s'} T(s, a, s') u^t(s'). \tag{1.8}$$

O leitor notará que esta equação é quase igual a (1.7), exceto que agora estamos atualizando os valores $u$ ao longo do tempo. Foi demonstrado que este processo irá eventualmente, e muitas vezes rapidamente, convergir para os valores reais de $u(s)$. Também sabemos que se a mudança máxima na utilidade para um determinado intervalo de tempo for menor que $\epsilon($ $1-\gamma)/\gamma$então o erro será menor que $\epsilon$. Este fato pode ser usado como condição de parada.



avaliador

O algoritmo VALUE-ITERATIONé mostrado na figura 1.2. Este algoritmo é um exemplo de programação dinâmica em que a solução ótima para um problema é

DYNAMIC PROGRAMMING

VALUE-ITERATION$(T, r, \gamma, \epsilon)$
1 **do**
2 $u \leftarrow u'$
3 $\delta \leftarrow 0$
4 **for** $s \in S$
5 **do** $u'(s) \leftarrow r(s) + \gamma \max_a \sum_{s'} T(s, a, s')u(s')$
6 **if** $|u'(s) - u(s)| > \delta$
7 **then** $\delta \leftarrow |u'(s) - u(s)|$
8 **until** $\delta < \epsilon(1 - \gamma)/\gamma$
9 **return** $u$

**Figure 1.2:** The VALUE-ITERATION algorithm. It takes as input an MDP given by $T(\cdot)$, a discount value $\gamma$, and an error $\epsilon$. It returns a $u(s)$ for that MDP that is guaranteed to have an error less than $\epsilon$.

| | Time ($t$) | | | | |
|---|---|---|---|---|---|
| | 0 | 1 | 2 | 3 | 4 |
| $u(s_1)$ | 0 | 0 | 0 | $.5(.8).45 = .18$ | $.5(.036 + .376) = .206$ |
| $u(s_2)$ | 0 | 0 | $.5(.8)1 = .4$ | $.5(.88)1 = .44$ | $.5(.088 + .96) = .52$ |
| $u(s_3)$ | 0 | 1 | 1 | $1 + .5(1).45 = 1.2$ | $1 + .5(.47) = 1.2$ |
| $u(s_4)$ | 0 | 0 | $.5(.9)1 = .45$ | $.5(.9 + .045) = .47$ | $.5(1.1 + .047) = .57$ |

| $s$ | $\pi^*(s)$ |
|---|---|
| $s_1$ | $a_2$ |
| $s_2$ | $a_2$ |
| $s_3$ | $a_3$ |
| $s_4$ | $a_1$ |

**Figure 1.3:** Example application of the VALUE-ITERATION algorithm to the MDP from figure 1.1, assuming $\gamma = .5$ and $\epsilon = .15$. The algorithm stops after $t = 4$. The bottom table shows the optimal policy given the utility values found by the algorithm.

found by first finding the optimal solutions to sub-problems. In our case, the sub-problems are the variables themselves. Finding a value for one variable helps us find values for other variables.

For example, figure 1.3 shows how the utility values change as the VALUE-ITERATION algorithm is applied to the example from figure 1.1. The utilities start at 0 but $s_3$ is quickly set to 1 because it receives a reward of 1. This reward then propagates back to its immediate neighbors $s_2$ and $s_4$ at time 2 and then at time 3 it reaches $s_1$. At time 4 the algorithm stops because the biggest change in utility from time 3 to time 4 was 0.13, for $s_4$, which is less than or equal to $\epsilon(1 - \gamma)/\gamma = .15$.

### 1.2.1 Multiagent Markov Decision Processes

The MDP model represents the problems of only one agent, not of a multiagent system. There are several ways of transforming an MDP into a multiagent MDP. The easiest way is to simply place all the other agents' effects into the transition function. That is, assume the other agents don't really exist as entities and are merely part of the environment. This technique can work for simple cases where the agents are not changing their behavior since the transition function in an MDP must be fixed. Unfortunately, agents that change their policies over time, either because of their own learning or because of input from the users, are very common.

A better method is to extend the definition of MDP to include multiple agents all of which can take an action at each time step. As such, instead of having a transition function $T(s, a, s')$ we have a transition function $T(s, \vec{a}, s')$, where $\vec{a}$ is a vector of size equal to the number of agents where each element is an agent's action, or a symbol representing non-action by that agent. We also need to determine how the reward $r(s)$ is to be doled out amongst the agents. One possibility is to divide it evenly among the agents. Unfortunately, such a simplistic division can mislead agents by rewarding them for bad actions. A better method is to give each agent a reward proportional to his contribution to the system's reward. We will see how this can be done in chapter 5.6.



VALUE-ITERATION$(T, r, \gamma, \epsilon)$
1 **do**
2 $u \leftarrow u'$
3 $\delta \leftarrow 0$
4 **for** $s \in S$
5 **do** $u'(s) \leftarrow r(s) + \gamma \max_a \sum_{s'} T(s, a, s')u(s')$
6 **if** $|u'(s) - u(s)| > \delta$
7 **then** $\delta \leftarrow |u'(s) - u(s)|$
8 **until** $\delta < \epsilon(1 - \gamma)/\gamma$
9 **return** $u$

Figura 1.2: O algoritmo VALUE-ITERATION. Toma como entrada um MDP dado por $T(\cdot)$, um valor de desconto $\gamma$ e um erro $\epsilon$. Ele retorna um $u(s)$ para aquele MDP que tem garantia de erro menor que $\epsilon$.

| | Tempo ($t$) | | | | |
|---|---|---|---|---|---|
| | 0 | 1 | 2 | 3 | 4 |
| $u(s_1)$ | 0 | 0 | 0 | $0{,}5(0{,}8).45 = .18$ | $.5(.036 + .376) = .206$ |
| $u(s_2)$ | 0 | 0 | $0{,}5(0{,}8)1 = .4$ | $0{,}5(0{,}88)1 = .44$ | $.5(.088 + .96) = .52$ |
| $u(s_3)$ | 0 | 1 | 1 | $1 + .5(1).45 = 1.2$ | $1 + .5(.47) = 1.2$ |
| $u(s_4)$ | 0 | 0 | $0{,}5(0{,}9)1 = .45$ | $.5(.9 + .045) = .47$ | $.5(1.1 + .047) = .57$ |

| $s$ | $\pi^*(s)$ |
|---|---|
| $s_1$ | $a_2$ |
| $s_2$ | $a_2$ |
| $s_3$ | $a_3$ |
| $s_4$ | $a_1$ |

Figura 1.3: Exemplo de aplicação do algoritmo VALUE-ITERATION ao MDP da figura 1.1, assumindo $\gamma = .5$ e $\epsilon = .15$. O algoritmo para após $t = 4$. A tabela inferior mostra a política ótima dados os valores de utilidade encontrados pelo algoritmo.

encontrado encontrando primeiro as soluções ótimas para os subproblemas. No nosso caso, os subproblemas são as próprias variáveis. Encontrar um valor para uma variável nos ajuda a encontrar valores para outras variáveis.

Por exemplo, a figura 1.3 mostra como os valores de utilidade mudam à medida que o algoritmo VALUE-ITERATION é aplicado ao exemplo da figura 1.1. Os utilitários começam em 0, mas $s_3$ é rapidamente definido como 1 porque recebe uma recompensa de 1. Essa recompensa então se propaga de volta para seus vizinhos imediatos $s_2$ e $s_4$ no tempo 2 e então no tempo 3 atinge $s_1$. No tempo 4, o algoritmo para porque a maior mudança na utilidade do tempo 3 para o tempo 4 foi 0,13, para $s_4$, que é menor ou igual a $\epsilon(1 - \gamma)/\gamma = .15$.

### 1.2.1 Processos de Decisão Markov Multiagente

O modelo MDP representa os problemas de apenas um agente, não de um sistema multiagente. Existem diversas maneiras de transformar um MDP em multiagente MDP. A maneira mais fácil é simplesmente colocar todos os efeitos dos outros agentes na função de transição. Isto é, suponha que os outros agentes não existam realmente como entidades e sejam apenas parte do ambiente. Esta técnica pode funcionar para casos simples onde os agentes não estão mudando seu comportamento já que a função de transição em um MDP deve ser corrigida. Infelizmente, são muito comuns os agentes que alteram as suas políticas ao longo do tempo, quer devido à sua própria aprendizagem, quer devido à contribuição dos utilizadores.

Um método melhor é estender a definição de MDP para incluir vários agentes, todos os quais podem realizar uma ação em cada intervalo de tempo. Como tal, em vez de ter uma função de transição $T(s, a, s')$ temos uma função de transição $T(s, \vec{a}, s')$, onde $\vec{a}$ é um vetor de tamanho igual ao número de agentes onde cada elemento é uma ação de um agente, ou um símbolo que representa a não ação desse agente. Também precisamos determinar como a recompensa $r(s)$ será distribuída entre os agentes. U ma possibilidade é dividi-lo igualmente entre os agentes. Infelizmente, uma divisão tão simplista pode enganar os agentes, recompensando-os por más ações. Um método melhor é dar a cada agente uma recompensa proporcional à sua contribuição para a recompensa do sistema. Veremos como isso pode ser feito no capítulo 5.6.

### 1.2.2 Partially Observable MDPs

In many situations it is not possible for the agent to sense the full state of the world. Also, an agent's observations are often subject to noise. For example, a robot has only limited sensors and might not be able to see behind walls or hear soft sounds and its microphone might sometimes fail to pick up sounds. For these scenarios we would like to be able to describe the fact that the agent does not know in which state it is in but, instead, believes that it can be in any number of states with certain probability. For example, a robot might believe that it is in any one of a number of rooms, each with equal probability, but that it is definitely not outdoors. We can capture this problem by modeling the agent's **belief state** $\vec{b}$ instead of the world state. This belief state is merely a probability distribution over the set of possible states and it indicates the agent's belief that it is in that state. For the case with four states, the vector $\vec{b} = \langle \frac{1}{2}, \frac{1}{2}, 0, 0 \rangle$ indicates that the agent believes it is either in $s_1$ or $s_2$, with equal probability, and that it is definitely not in $s_3$ or $s_4$.

BELIEF STATE

We also need an **observation model** $O(s, o)$ which tells the agent the probability that it will perceive observation $o$ when in state $s$. The agent can then use the observations it receives to update its current belief $\vec{b}$. Specifically, if the agent's current belief is $\vec{b}$ and it takes action $a$ then its new belief vector $\vec{b}'$ can be determined using

OBSERVATION MODEL

$$\forall_{s'} \; \vec{b}'(s') = \alpha O(s', o) \sum_s T(s, a, s') \, \vec{b}(s), \tag{1.9}$$

where $\vec{b}(s)$ is the value of $\vec{b}$ for $s$ and $\alpha$ is a normalizing constant that makes the belief state sum to 1. When we put all these requirements together we have a **partially observable Markov decision process** or POMDP which are a very natural way of describing the problems faced by an agent with limited sensors. Of course, since the agent does not know the state then it cannot use value iteration to solve this problem.

PARTIALLY OBSERVABLE MARKOV DECISION PROCESS

Luckily, it turns out that we can use (1.9) to define a new transition function

$$\tau(\vec{b}, a, \vec{b}') = \begin{cases} \sum_{s'} O(s', o) \sum_s T(s, a, s') \, \vec{b}(s) & \text{if (1.9) is true for } \vec{b}, a, \vec{b}' \\ 0 & \text{otherwise,} \end{cases} \tag{1.10}$$

and a new reward function

$$\rho(\vec{b}) = \sum_s \vec{b}(s) r(s). \tag{1.11}$$

Solving a POMDP on a physical state amounts to solving this MDP on the belief state. Unfortunately, this MDP can have an infinite number of states since the beliefs are continuous values. Luckily, there do exist algorithms for solving these type of MDPs. These algorithms work by grouping together beliefs into regions and associating actions with each region. In general, however, when faced with a POMDP problem it is usually easier to use a dynamic decision networks to represent and solve the problem.

See (Russell and Norvig, 2003, Chapter 17.5) for introduction to dynamic decision networks.

## 1.3 Planning

In the **artificial intelligence planning** problem an agent is also given a set of possible states and is asked for a sequence of actions that will lead it to a desirable world state. However, instead of a transition function the agent is given a set of operators. Each operator has pre-requisites that specify when it can be used—in which states—and effects which specify the changes the operator will cause in the state. The planning problem is to find a sequence of operators that take the agent from the start state to the goal state.

ARTIFICIAL INTELLIGENCE PLANNING

It should be clear that this problem is a special case of an MDP, one where only one state provides a reward and all the transitions have probability of 1 or 0. The transitions are generated by applying the operators to the current state. By



### 1.2.2 MDPs Parcialmente Observáveis

Em muitas situações não é possível ao agente sentir o estado completo do mundo. Além disso, as observações de um agente estão frequentemente sujeitas a ruído. Por exemplo, um robô t em apenas sensores limitados e pode não ser capaz de ver atrás das paredes ou ouvir sons suaves e o seu microfone pode, por vezes, não conseguir captar sons. Para estes cenários gostaríamos de poder descrever o fato de que o agente não sabe em que estado se encontra, mas, em vez disso, acredita que pode estar em qualquer número de estados com certa probabilidade. Por exemplo, um robô pode acreditar que está em qualquer uma de várias salas, cada uma com igual probabilidade, mas que definitivamente não está ao ar livre. Podemos capturar esse problema modelando o estado de crença do agente $b$ em vez do mundo

BELIEF STATE

estado. Este estado de crença é apenas uma distribuição de probabilidade sobre o conjunto de estados possíveis e indica a crença do agente de que está nesse estado. Para o caso com quatro estados, o vetor $\vec{b} = \langle \frac{1}{2}, \frac{1}{2}, 0, 0 \rangle$ indica que o agente acredita que está em $s_1$ ou em $s_2$, com igual probabilidade, e que definitivamente não está em $s_3$ ou em $s_4$.

Também precisamos de um modelo de observação $O(s, o)$ que informe ao agente a probabilidade

OBSERVATION MODEL

probabilidade de perceber a observação $o$ quando estiver no estado $s$. O agente pode então usar as observações que recebe para atualizar sua crença atual $b$. Especificamente, se a crença atual do agente for $\vec{b}$ e ele agir $a$ então seu novo vetor de crenças $\vec{b}'$ pode ser determinado usando

$$\forall_{s'} \; \vec{b}'(s') = \alpha O(s', o) \sum_s T(s, a, s') \, \vec{b}(s), \tag{1.9}$$

onde $\vec{b}(s)$ é o valor de $b$ para $s$ e $\alpha$ é uma constante de normalização que faz com que a soma do estado de crença seja 1. Quando colocamos todos esses requisitos juntos, temos um processo de decisão de Markov parcialmente observável ou POMDP que são uma forma muito natural

PARTIALLY OBSERVABLE MARKOV DECISION PROCESS

de descrever os problemas enfrentados por um agente com sensores limitados. É claro que, como o agente não conhece o estado, ele não pode usar a iteração de valores para resolver esse problema.

Felizmente, podemos usar (1.9) para definir uma nova função de transição

$$\tau(\vec{b}, a, \vec{b}') = \begin{cases} \sum_{s'} O(s', o) \sum_s T(s, a, s') \, \vec{b}(s) & \text{if (1.9) is true for } \vec{b}, a, \vec{b}' \\ 0 & \text{otherwise,} \end{cases} \tag{1.10}$$

e uma nova função de recompensa

$$\rho(\vec{b}) = \sum_s \vec{b}(s) r(s). \tag{1.11}$$

Resolver um POMDP num estado físico equivale a resolver este MDP no estado de crença. Infelizmente, este MDP pode ter um número infinito de estados, uma vez que as crenças são valores contínuos. Felizmente, existem algoritmos para resolver esses tipos de MDPs. Esses algoritmos funcionam agrupando crenças em regiões e associando ações a cada região. Em geral, porém, quando confrontado com um POMDP

Veja (Russell e Norvig, 2003, Capítulo 17.5) para introdução às redes de decisão dinâmicas.

problema, geralmente é mais fácil usar redes de decisão dinâmicas para representar e resolver o problema.

## 1.3 Planejamento

No problema de planejamento de inteligência artificial, um agente também recebe um conjunto de estados possíveis e é solicitada uma sequência de ações que o levará a um estado mundial desejável. No entanto, em vez de uma função de transição, o agente recebe u m conjunto de operadores. Cada operador possui pré-requisitos que especificam quando ele pode ser usado – em quais estados – e efeitos que especificam as mudanças que o operador causará no estado. O problema de planejamento é encontrar uma sequência de operadores que leve o agente do estado inicial ao estado objetivo.

ARTIFICIAL INTELLIGENCE PLANNING

Deve ficar claro que este problema é um caso especial de MDP, onde apenas um estado fornece uma recompensa e todas as transições têm probabilidade de 1 ou 0. As transições são geradas aplicando os operadores ao estado atual. Por

describing the problem using operators we achieve a much more succinct definition of the problem than by enumerating all states and transition probabilities as done in an MDP. Still, our goal is solving the problem, not defining it with as few bits as possible. The more detailed description of the states does, however, provide an advantage over the simple state listing used in MDP, namely, it provides added information about the composition of the state. For example, if we want to reach a state that has properties of *is-blue?* and *is-wet?* then we can use this knowledge to set about getting the state of the world blue and, once that is done, to get it wet without changing its color. This kind of information is opaque in a MDP description which simply describes a wet blue state as $s_{11}$ and a dry blue state as $s_{23}$. Operators and their corresponding state descriptions thus provide the agent with more knowledge about the problem domain.

There exists many planning algorithms which solve the basic planning problem and variations of it, including cases where there is a possibility that the operators don't have the desired effect (making it even more like an MDP), cases where the agent must start taking actions before the plan can be finished, and cases where there is uncertainty as to what state the agent is in (like POMDP). Most modern planning algorithms use a graphical representation of the problem. That is, they end up turning the problem into an MDP-like problem and solving that. These algorithms are sophisticated and are available as libraries to use with your programs. If the need arises to solve a planning problem or an MDP it is advisable to use one of the many available libraries or, at worst, implement one of the published algorithms rather than trying to implement your own.

See (Russell and Norvig, 2003, Chapters 11–12) for pointers to these programs.

### 1.3.1 Hierarchical Planning

A very successful technique for handling complexity is to recursively divide a problem into smaller ones until we find problems that are small enough that they can be solved in a reasonable amount of time. This general idea is known as the **divide and conquer** approach in AI. Within planning, the idea can be applied by developing plans whose primitive operators are other plans. For example, in order to achieve your goal of a holiday at the beach you must first arrange transportation, arrange lodging, and arrange meals. Each one of these can be further decomposed into smaller actions, and there might be different ways of performing this decomposition. Within the planning model this amounts to building virtual operators which are planning problems themselves, instead of atomic actions. The problem then becomes one of deciding which virtual operators to build. In fact, the problem is very similar to that of deciding which functions or classes to write when implementing a large software project.

DIVIDE AND CONQUER

Within the MDP model we can imagine building a hierarchy of policies, each one taking of from states that fit a particular description (say, dry states) to other types of states (say, red and wet). We can then define new MDP problems which use sets of states as atomic states and the new policies as transition functions, similar to how we built POMDPs. These techniques are studied in **hierarchical learning**.

HIERARCHICAL LEARNING

Hierarchical planning and learning are usually studied as ways of making the planning and learning problems easier, that is, as ways to develop algorithms that will find solutions faster. However, once developed these hierarchies provide added benefits in a multiagent system. Specifically, they can be used to enable coordination among agents (Durfee, 1999). By exchanging top-level plan names (or policies) the agents have a general idea of what each other is doing without the need for exchanging detailed instructions of what they are doing at each step. For example, with two robots moving boxes in a room one might tell the other that its moving a box from the South corner to the East corner, the other agent then knows that if it stays in the Northwest part of the room it does not need to worry about the exact location of its partner—the robots stay out of each other's way without detailed coordination. This technique of using goal/plan/policy hierarchies for multiagent coordination has been very successful. We will examine it in more detail in



descrevendo o problema usando operadores, conseguimos uma definição muito mais sucinta do problema do que enumerando todos os estados e probabilidades de tr ansição como feito em um MDP. Ainda assim, nosso objetivo é resolver o problema, e não defini-lo com o mínimo de bits possível. A descrição mais detalhada dos estados, no entanto, oferece uma vantagem sobre a simples listagem de estados usada em MDP, ou seja, fornece informações adicionais sobre a composição do estado. Por exemplo, se quisermos atingir um estado que tenha propriedades de is-blue? e está molhado? então poderemos usar esse conhecimento para começar a tornar o estado do mundo azul e, uma vez feito isso, molhá-lo sem alterar sua cor. Este tipo de informação é opaca em uma descrição MDP que simplesmente descreve um estado azul úmido como $s_{11}$ e um estado azul seco como $s_{23}$. Os operadores e suas correspondentes descrições de estado fornecem ao agente mais conhecimento sobre o domínio do problema.

Existem muitos algoritmos de planejamento que resolvem o problema básico de planejamento e suas variações, incluindo casos em que existe a possibilidade de os operadores não terem o efeito desejado (tornando-o ainda mais parecido com um MDP), casos em que o agente deve começar a tomar ações antes que o plano possa ser concluído e casos em que há incerteza quanto ao estado em que o agente se encontra (como POMDP). A maioria dos algoritmos de planejamento modernos usa uma representação gráfica do problema. Ou seja, eles acabam transformando o problema em um problema do tipo MDP e resolvendo isso. Esses algoritmos são sofisticados e estão disponíveis como bibliotecas para uso com seus programas. Se o

Veja (Russell e Norvig, 2003, Capítulos 11–12) para obter dicas sobre esses programas.

surge a necessidade de resolver um problema de planejamento ou MDP é aconselhável usar uma das muitas bibliotecas disponíveis ou, na pior das hipóteses, implementar u m dos algoritmos publicados em vez de tentar implementar o seu próprio.

#### 1.3.1 Planejamento Hierárquico

Uma técnica muito bem-sucedida para lidar com a complexidade é dividir recursivamente um problema em problemas menores até encontrarmos problemas que sejam pequenos o suficiente para que possam s er resolvidos em um período de tempo razoável. Essa ideia geral é conhecida como abordagem de dividir para conquistar em AI. Dentro do planejamento, a ideia pode ser aplicada pelo desenvolvimento

DIVIDE AND CONQUER

planos de operação cujos operadores primitivos são outros planos. Por exemplo, para atingir o seu objetivo de férias na praia, você deve primeiro providenciar transporte, hospedagem e refeições. Cada uma delas pode ser decomposta em ações menores, e pode haver diferentes maneiras de realizar essa decomposição. Dentro do modelo de planejamento, isso equivale à construção de operadores virtuais que planejam eles próprios problemas, em vez de ações atômicas. O problema então passa a ser decidir quais operadores virtuais construir. Na verdade, o problema é muito semelhante ao de decidir quais funções ou classes escrever ao implementar um grande projeto de software.

Dentro do modelo MDP , podemos imaginar a construção de uma hierarquia de políticas, cada uma partindo de estados que se enquadram em uma descrição específica (digamos, estados secos) para outros tipos de estados (digamos, vermelhos e úmidos). Podemos então definir novos problemas MDP que usam conjuntos de estados como estados atômicos e as novas políticas como funções de transição, semelhante a como construímos POMDPs. Essas técnicas são estudadas na aprendizagem hierárquica.

HIERARCHICAL LEARNING

O planejamento e o aprendizado hierárquicos costumam ser estudados como formas de facilitar o planejamento e o aprendizado dos problemas, ou seja, como formas de desenvolver algoritmos que encontrarão soluções mais rapidamente. No entanto, uma vez desenvolvidas, estas hierarquias proporcionam benefícios adicionais num sistema multiagente. Especificamente, podem ser usados para permitir a coordenação entre agentes (Durfee, 1999). Ao trocar nomes de planos (ou políticas) de nível superior, os agentes têm uma ideia geral do que cada um está fazendo, sem a necessidade de trocar instruções detalhadas sobre o que estão fazendo em cada etapa. Por exemplo, com dois robôs movendo caixas em uma sala, um pode dizer ao outro que está movendo uma caixa do canto Sul para o canto Leste, o outro agente então sabe que se permanecer na parte Noroeste da sala, não precisa se preocupar com a localização exata de seu parceiro – os robôs ficam fora do caminho um do outro sem coordenação detalhada. Esta técnica de utilização de hierarquias de objectivos/planos/políticas para a coordenação de vários agentes tem sido muito bem sucedida. Iremos examiná-lo com mais detalhes em

Chapter 9.

## 1.4 Summary

The view of an autonomous agent as an utility-maximizing agent that inhabits an MDP, or variation thereof, is most popular because of its flexibility, applicability to disparate domains, and amenability to formal analysis with mathematical or computational tools. As such, this book will largely adopt this model as the basis for formulating the various multiagent problems. Note, however, that the MDP is a tool for describing the problem faced by an agent. In practice, it is rare that a practical solution to a real world problem is also implemented as an algorithm to solve the raw MDP. More commonly we find algorithms which use a much more succinct, and therefore practical, method for representing the problem. Unfortunately, these representations tend to be very domain specific; they cannot be used in different domains.

## Exercises

1.1 Marvin is a robot that inhabits a 3 by 3 grid and can only move North, South, East, and West. Marvin's wheels sometimes spin so that on each action there is a .2 probability that Marvin remains stationary. Marvin receives a reward of 1 whenever it visits the center square.

1. Draw an MDP which describes Marvin's problem.
2. What is the optimal policy?

1.2 The dynamic programming approach can be used to solve a wide variety of problems. For example, the Google search engine ranks results using the PageRank algorithm in which the pagerank of a webpage is proportional to the number of other pages that link to it weighted by their own pagerank. Thus, the pagerank can be calculated using a simple variation of the value iteration algorithm. However, one problem is that we do not know how long it will take for the algorithm to converge.

Implement the VALUE-ITERATION algorithm from figure 1.2 and run experiments to determine how much time it takes to converge as you increase the number of states given a fixed edge to node ratio. Try different ratios.

1.3 You have a table with three blocks: A, B, C. Assume the state of the world is completely defined by the position of the blocks relative to each other, where a block can only be on the table on on top of another block. At most one block can be on top of another block block. You are further given the following operators:

- MOVE-TO-TABLE($block$): Requires that $block$ has no other block on top of it. Results in the block now being on the table.
- MOVE-TO-BLOCK($b1, b2$): Requires that block $b2$ not have any other block on top of it. Results in block $b1$ being on top of block $b2$.

Draw an MDP for this domain assuming operators always have their intended results.

1.4 Implement a NetLogo program where the patches are randomly set to one of three colors: black, red, and green.There is a turtle in this world who can only move North, South, East, and West by one patch at a time. Each time it lands in a patch it receives a reward determined by the color of the patch: black is 0, red is -1, and green in 1.

The turtle's movement is noisy, so when it decides to move North it ends up at the desired North tile with probability .5, and the tile NorthEast of its



Capítulo 9.

## 1.4 Resumo

A visão de um agente autônomo como um agente maximizador de utilidade que habita um MDP, ou uma variação dele, é mais popular devido à sua flexibilidade, aplicabilidade a domínios díspares e facilidade para análise formal com ferramentas matemáticas ou computacionais. Como tal, este livro adotará amplamente esse modelo como base para a formulação de vários problemas multiagentes. Observe, entretanto, que MDPé uma ferramenta para descrever o problema enfrentado por um agente. Na prática, é raro que uma solução prática para um problema do mundo real também seja i mplementada como um algoritmo para resolver o MDP bruto. Mais comumente encontramos algoritmos que usam um método muito mais sucinto e, portanto, prático, para representar o problema. Infelizmente, essas representações tendem a ser muito específicas de domínio; eles não podem ser usados em domínios diferentes.

## Exercícios

1.1 Marvin é um robô que habita uma grade 3 por 3 e só pode se mover para Norte, Sul, Leste e Oeste. As rodas de Marvin às vezes giram de modo que em cada ação há uma probabilidade de 0,2 de que Marvin permaneça parado. Marvin recebe uma recompensa de 1 sempre que visita a praça central.

1. Desenhe um MDP que descreva o problema de Marvin. 2. Qual é a política ideal?

1.2 A abordagem de programação dinâmica pode ser usada para resolver uma ampla variedade de problemas. Por exemplo, o mecanismo de pesquisa Google classifica os resultados usando o algoritmo PageRank, no qual o pagerank de uma página da web é proporcional ao número de outras páginas vinculadas a ela, ponderado por seu próprio pagerank. Assim, o pagerank pode ser calculado usando uma variação simples do algoritmo de iteração de valor. Entretanto, um problema é que não sabemos quanto tempo levará para o algoritmo convergir.

Implemente o VALUE-ITERATION algoritmo da figura 1.2 e execute experimentos para determinar quanto tempo leva para convergir à medida que você aumenta o número de estados dada uma proporção fixa entre arestas e nós. Experimente proporções diferentes.

1.3 Você tem uma mesa com três blocos: A, B, C. Suponha que o estado do mundo seja completamente definido pela posição dos blocos em relação uns aos outros, onde um bloco só pode estar na mesa em cima de outro bloco. No máximo um bloco pode estar em cima de outro bloco. Você ainda recebe os seguintes operadores:

- MOVE-TO-TABLE(block ): Requer que o bloco não tenha outro bloco em cima dele. O resultado é que o bloco agora está sobre a mesa.
- MOVE-TO-BLOCK(b1, b2): Requer que o bloco b2 não tenha nenhum outro bloco em cima dele. Resulta no bloco b1 no topo do bloco b2.

Desenhe um MDP para este domínio, assumindo que os operadores sempre têm os resultados pretendidos.

1.4 Implemente um programa NetLogo onde os patches são definidos aleatoriamente em uma das três cores: preto, vermelho e verde. Há uma tartaruga neste mundo que só pode se mover para Norte, Sul, Leste e Oeste, um patch de cada vez. Cada vez que pousa em uma área, ele recebe uma recompensa determinada pela cor da área: preto é 0, vermelho é -1 e verde é 1.

O movimento da tartaruga é barulhento, então quando ela decide se mover para o Norte ela termina na peça Norte desejada com probabilidade 0,5, e a peça Nordeste de sua

current location (a diagonally adjacent tile) with probability .25, and at the tile NorthWest of its current location with a probability of .25.

Implement the VALUE-ITER algorithm for this domain and find the optimal policy.



localização atual (um bloco diagonalmente adjacente) com probabilidade de 0,25, e no bloco Noroeste de sua localização atual com uma probabilidade de 0,25.

Implemente o algoritmo VALUE-ITER para este domínio e encontre a política ideal.

# Chapter 2

# Distributed Constraints

Most multiagent systems are characterized by a set of autonomous agents each with local information and ability to perform an action when the set of actions of all must be coordinated so as to achieve a desired global behavior. In all these cases you own all the agents in question and can program them to do whatever you want. Thus, there is no need to properly incentivise them as there would be in an open system, which we study in later Chapters. However, there is still the problem of coordination. Since each agent only has local information it might be hard for it to decide what to do.

In this chapter we look at some algorithms for performing distributed search in cooperative multiagent systems where each agent has some local information and where the goal is to get all the agents to set themselves to a state such that the set of states in the system is optimal. For example, imagine a group of small sensors in a field. Each sensor can communicate only with those that are near him and has to decide which of its modalities to use (sense temperature, point radar North, point radar South, etc.) so that the group of sensors gets a complete view of the important events in the field. Or, imagine a group of kids who have been told to stand in a circle, each one must move find a spot to stand on but the position of that spot depends on everyone else's location. In these examples the agents have local information, can take any one of their available actions at any time, but the utility of their actions depends on the actions of others. We find that many multiagent problems can be reduced to a distributed constraints problem. Thus, the algorithms we present in this chapter have many different applications.

## 2.1 Distributed Constraint Satisfaction

We start by formally describing the problem. In a **constraint satisfaction problem** (CSP) we are given a set of variables, each with its own domain along with a set of constraints. The goal is to set every variable to a value from its domain such that no constraints are violated. Formally,

CONSTRAINT SATISFACTION PROBLEM

**Definition 2.1** (Constraint Satisfaction Problem)**.** *Given a set of variables* $x_1$, $x_2$,...,$x_n$ *with domains* $D_1, D_2, \ldots, D_n$ *and a set of boolean constraints* $P$ *of the form* $p_k(x_{k1}, x_{k2}, \ldots, x_{kj}) \rightarrow \{0, 1\}$*, find assignments for all the variables such that no constraints are violated.*

The most widely studied instance of a constraint satisfaction problem is the graph coloring problem, shown in figure 2.1. In this problem we are given a graph and a set of colors. The problem is to find out if there is a way to color each node with one of the given colors such that no two nodes that are connected by an edge have the same color. We can easily map the graph coloring problem to the formal definition of a CSP by considering each node to be a variable, the domains to be the set of colors, and each edge becomes a constraint between its two nodes that is true only if their values are different. Notice that, in graph coloring constraints are only over two variables instead of over any set of variables as we find in the general constraint satisfaction problem.

The constraint satisfaction problem is NP-complete. As such, me use search algorithms to find a solution and hope that the actual running time will be less than



**Capítulo 2**

# Restrições Distribuídas

A maioria dos sistemas multiagentes são caracterizados por um conjunto de agentes autônomos, cada um com informações locais e capacidade de executar uma ação quando o conjunto de ações de todos deve ser coordenado de modo a alcançar um comportamento global desejado. Em todos esses casos você possui todos os agentes em questão e pode programá-los para fazer o que quiser. Assim, não há necessidade de incentivá-los adequadamente, como aconteceria num sistema aberto, que estudaremos em capítulos posteriores. No entanto, ainda existe o problema de coordenação. Como cada agente possui apenas informações locais, pode ser difícil decidir o que fazer.

Neste capítulo, examinamos alguns algoritmos para realizar buscas distribuídas e m sistemas multiagentes cooperativos onde cada agente possui alguma informação l ocal e onde o objetivo é fazer com que todos os agentes se ajustem a um estado tal que o conjunto de estados no sistema seja ótimo. Por exemplo, imagine um grupo de pequenos sensores num campo. Cada sensor pode se comunicar apenas com aqueles que estão próximos a ele e deve decidir qual de suas modalidades utilizar (sensor de temperatura, radar pontual Norte, radar pontual Sul, etc.) para que o grupo de sensores obtenha uma visão completa dos eventos importantes no campo. Ou imagine um grupo de crianças que foram instruídas a ficar em círculo, cada uma deve se mover e encontrar um lugar para ficar de pé, mas a posição desse ponto depende da localização de todos os outros. Nestes exemplos os agentes possuem informações locais, podem realizar qualquer uma das ações disponíveis a qualquer momento, mas a utilidade de suas ações depende das ações de outros. Descobrimos que muitos problemas multiagentes podem ser reduzidos a um problema de restrições distribuídas. Assim, os algoritmos que apresentamos neste capítulo têm muitas aplicações diferentes.

## 2.1 Satisfação de Restrições Distribuídas

Começamos descrevendo formalmente o problema. Em um problema de satisfação de restrições (CSP), recebemos um conjunto de variáveis, cada uma com seu próprio domínio junto com um conjunto de restrições. O objetivo é definir cada variável com um valor de seu do mínio, de modo que nenhuma restrição seja violada. Formalmente,

CONSTRAINT SATISFACTION PROBLEM

*Definição 2.1 (Problema de Satisfação de Restrições). Dado um conjunto de variáveis* $x_1, x_2, \ldots, x_n$ *com domínios* $D_1, D_2, \ldots, D_n$ *e um conjunto de restrições booleanas* $P$ *da forma* $p_k(x_{k1}, x_{k2}, \ldots, x_{kj}) \rightarrow \{0, 1\}$*, encontre atribuições para todas as variáveis de forma que nenhuma restrição seja violada.*

O exemplo mais amplamente estudado de um problema de satisfação de restrições é o problema de coloração de grafos, mostrado na figura 2.1. Neste problema, recebemos um gráfico e um conjunto de cores. O problema é descobrir se existe uma maneira de colorir cada nó com uma das cores fornecidas, de modo que não haja dois nós conectados por uma aresta com a mesma cor. Podemos facilmente mapear o problema de coloração de grafos para a definição formal de um CSP considerando cada nó como uma variável, os domínios como o conjunto de cores, e cada aresta se torna uma restrição entre seus dois nós que é verdadeira apenas se seus valores forem diferentes. Observe que, na coloração de grafos, as restrições são apenas sobre duas variáveis, em vez de sobre qualquer conjunto de variáveis, como encontramos no problema geral de satisfação de restrições.

O problema de satisfação de restrições é NP-completo. Como tal, uso algoritmos de busca para encontrar uma solução e espero que o tempo real de execução seja menor que

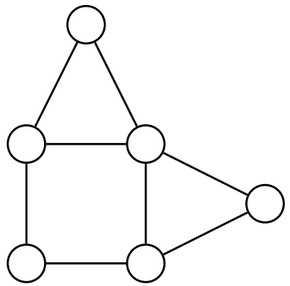


**Figure 2.1:** Sample graph coloring problem. You must color each node either black, white, or gray so that no two nodes connected by an edge have the same color. Can you find a coloring?

DEPTH-FIRST-SEARCH-CSP$(i, g)$

```
1  if i > n
2      then return g
3  for v ∈ D_i
4      do if setting x_i ← v does not violate any constraint in P given g
5          then g' ← DEPTH-FIRST-SEARCH-CSP(i + 1, g + {x_i ← v})
6               if g' ≠ ∅
7                  then return g'
8
9  return ∅
```

**Figure 2.2:** A centralized depth first search algorithm for the CSP. Variable $g$ holds the partial assignment of variable values. The algorithm is called with DEPTH-FIRST-SEARCH-CSP$(1, \emptyset)$.

N-QUEENS PROBLEM

the worst-case exponential time. In practice we find that there are many special cases where we can solve the problem really fast. For example, in the **n-queens problem** we are given an $8 \times 8$ chess board with 8 queens for which we must find places on the board such that no queen can take another queen. This problem is a special case of the CSP but it is one for which there are algorithms that can find a solution very quickly for large numbers of queens as the problem is under constrained.

DEPTH FIRST SEARCH

A common modification is to improve the variables' ordering.

A simple but effective method for solving the centralized constraint satisfaction problem is with the use of a **depth first search** algorithm, as shown in figure 2.2. The algorithm simply chooses an arbitrary ordering for the variables. On each call it tries to set the current variable $x_i$ to all its possible values to see which ones do not violate any of the constraints in $P$ given the partial variable assignment $g$. Whenever a value $v$ works, the algorithm adds that assignment to $g$ and recursively tries to assign the next variable in the list. This algorithm performs a complete search and is thus guaranteed to find a solution if one exists. Depth first search is also practical because it only requires an amount of memory that is linear on the number of variables, that is, it has $O(n)$ memory usage. As we shall see later in this chapter, some of the distributed algorithms try to parallelize this basic algorithm.

DISTRIBUTED CONSTRAINT SATISFACTION

In the **distributed constraint satisfaction** (DCSP) problem we give each agent one variable. The agent is responsible for setting the value of its own variable. The agents do not know the values of any other variable but can communicate with other agents. Formally, we define:

**Definition 2.2** (Distributed Constraint Satisfaction Problem)**.** *Give each agent one of the variables in a* CSP*. Agents are responsible for finding a value for their variable and can find out the values of the other agents' variables via communication*

Usually the agents are limited so that they can only communicate with their neighbors—the agents with whom they share a constraint. Some algorithms, however, allow agents to communicate with any other agent. Also, most algorithms assume that the agents know about all the constraints that involve their variable.

The DCSP is pervasive in multiagent research and many real-world problems can be reduced to a DCSP, or a variation thereof. As such, many DCSP algorithms have



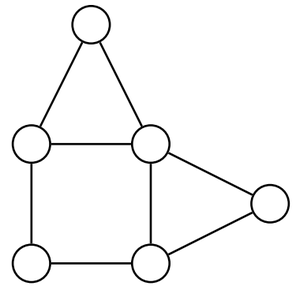


Figura 2.1: Exemplo de problema de coloração de gráfico. Você deve colorir cada nó com preto, branco ou cinza para que dois nós conectados por uma aresta não tenham a mesma cor. Você consegue encontrar uma coloração?

DEPTH-FIRST-SEARCH-CSP$(i, g)$

1 se $i > n$2 então retorne $g$3 para $v \in D_i$4 faça se a configuração $x_i \leftarrow v$ não viola nenhuma restrição em $P$ dado $g$5 então $g' \leftarrow$ DEPTH-FIRST-SEARCH-CSP$($ $i + 1, g + \{x_i \leftarrow v\}$) 6 se $g' \neq \emptyset$7 então retorne $g'$8 9 retorne $\emptyset$

Figura 2.2: Um algoritmo de busca em profundidade centralizado para CSP. A variável $g$ contém a atribuição parcial dos valores das variáveis. O algorit mo é chamado com DEPTH-FIRST-SEARCH-CSP$(1, \emptyset)$.

N-QUEENS PROBLEM

o tempo exponencial do pior caso. Na prática, descobrimos que existem muitos casos especiais em que podemos resolver o problema muito rapidamente. Por exemplo, no problema das n-rainhas recebemos um $8 \times 8$ tabuleiro de xadrez com 8 rainhas para as quais devemos encontrar lugares no tabuleiro de modo que nenhuma rainha possa pegar outra rainha. Este problema é um caso especial de CSP, mas é aquele para o qual existem algoritmos que podem encontrar uma solução muito rapidamente para um grande número de rainhas, pois o problema está sob restrições.

DEPTH FIRST SEARCH

Uma modificação comum é melhorar a ordenação das variáveis.

Um método simples, mas eficaz, para resolver o problema centralizado de satisfação de restrições é com o uso de um algoritmo de busca em profundidade, conforme mostrado na figura 2.2.

O algoritmo simplesmente escolhe uma ordem arbitrária para as variáveis. Em cada chamada ele tenta definir a variável atual $x_i$ com todos os seus valores possíveis para ver quais deles não violam nenhuma das restrições em $P$ dada a atribuição parcial da variável $g$. Sempre que um valor $v$ funciona, o algoritmo adiciona essa atribuição a $g$ e tenta recursivamente atribuir a próxima variável na lista. Este algoritmo realiza uma busca completa e, portanto, garante encontrar uma solução, se existir. A primeira pesquisa em profundidade também é prática porque requer apenas uma quantidade de memória que é linear no número de variáveis, ou seja, tem uso de memória $O(n)$. Como veremos mais adiante neste capítulo, alguns dos algoritmos distribuídos tentam paralelizar esse algoritmo básico.

DISTRIBUTED CONSTRAINT SATISFACTION

**No problema de satisfação de restrição distribuída (DCSP), damos a cada agente** uma variável. O agente é responsável por definir o valor de sua própria variável. Os agentes não conhecem os valores de nenhuma outra variável, mas podem co municar-se com outros agentes. Formalmente, definimos:

*Definição 2.2 (Problema de Satisfação de Restrições Distribuídas). Dê a cada agente uma das variáveis em um* CSP*. Os agentes são responsáveis por encontrar um valor para sua variável e podem descobrir os valores das variáveis dos outros agentes via comunicação*

Geralmente os agentes são limitados de modo que só podem se comunicar com seus vizinhos – os agentes com quem compartilham uma restrição. Alguns algoritmos, entretanto, permitem que os agentes se comuniquem com qualquer outro agente. Além disso, a maioria dos algoritmos assume que os agentes conhecem todas as restrições que envolvem sua variável.

O DCSP é difundido na pesquisa multiagente e muitos problemas do mundo real podem ser reduzidos a um DCSP ou a uma variação dele. Como tal, muitos DCSP algoritmos têm

```
FILTERING()
1  for j ∈ {neighbors of i}          ▷ i is this agent.
2      do REVISE(x_i, x_j)

HANDLE-NEW-DOMAIN(j, D')
1  D_j ← D'
2  REVISE(x_i, x_j)

REVISE(x_i, x_j)
1  old-domain ← D_i
2  for v_i ∈ D_i
3      do if there is no v_j ∈ D_j consistent with v_i
4             then D_i ← D_i − v_i
5  if old-domain ≠ D_i
6     then ∀_{k∈{neighbors of i}} k.HANDLE-NEW-DOMAIN(i, D_i)
```

**Figure 2.3:** The filtering algorithm. Each agent $i$ executes FILTERING().

been developed, each one with its own set of strengths and weaknesses. In the next sections we examine some of the most popular algorithms.

### 2.1.1 Filtering Algorithm

Picture yourself as one of the agents. You know your constraints, the domain of your variable, and the domain of the variables of your neighbors. You can use this information to determine if any of the values from your variable's domain is impossible. That is, if there is a value which violates some constraint no matter what value one of your neighbors uses then that value will clearly not be part of the solution so you can get rid of it. Similarly, all other agents might do the same.



This idea is implemented by the **filtering algorithm** (Waltz, 1975) shown in figure 2.3. Every agent $i$ executes the FILTERING procedure which leads it to do a REVISE with every one of its neighbors' variables $x_j$. If the agent does succeed in eliminating one or more values from its domain it then tells its neighbors its new domain. When an agent receives new domains from its neighbors it again executes REVISE for those neighbors. This process repeats itself until all agents finish their computations and no more messages are sent.

The filtering algorithm is guaranteed to stop since there are only a finite number of values in all the domains and the algorithm only sends a message when a value is eliminated from a domain. Once it stops, the filtering algorithm might have reduced the size of some domains. Unfortunately, it might stop at a point where one or more of the agents still has many values in its domain. Thus, it can stop before a solution is found.

Figure 2.4 shows an example of the filtering algorithm as applied to the graph coloring problem. In this problem the agents have three possible colors but start out with already limited domains as indicated by the black crosses. For example, agent $x_1$ can only be either black or gray. At the first step $x_1$ executes REVISE$(x_1, x_3)$ and realizes that since $x_3$ must be gray that it cannot be gray. As such, it eliminates gray from its domain and tells all the other agents about this. The other agents incorporate this knowledge. Then, agent $x_2$ does a REVISE$(x_2, x_3)$ and realizes that it also can't be gray and thus eliminates gray from its domain. At this point $x_2$ knows the final configuration. When it tells the other agents about its new domain then all the agents will also know the final solution.

In this example the filtering algorithm found a specific solution. Unfortunately, this is a rare occurrence. More often we find that the filtering algorithm only slightly reduces the size of the domains. For example, if we run the example from figure 2.4 but start with all possible colors in all domains then the filtering algorithm will not be able to do anything. That is, none of the agents are able to eliminate any color



```
FILTERING()
1 para  j ∈ {vizinhos de  i}          ▷ i é este agente. 2
faça  REVISE(x_i, x_j)

HANDLE-NEW-DOMAIN(j, D')
1   D_j ← D'
2   REVISE(x_i, x_j)

REVISE(x_i, x_j)
1 domínio antigo  ← D_i2 para  v_i ∈ D_i3 faça se não houver
 v_j ∈ D_j consistente com  v_i4 então  D_i ← D_i − v_i5 se domínio
antigo  ≠ D_i6 então ∀_{k∈{neighbors de  i}} k.HANDLE-NEW-DOMAIN(i, D_i)
```

Figura 2.3: O algoritmo de filtragem. Cada agente $i$ executa FILTERING().

foram desenvolvidos, cada um com seu próprio conjunto de pontos fortes e fracos. Nas próximas seções examinaremos alguns dos algoritmos mais populares.

#### 2.1.1 Algoritmo de Filtragem

Imagine-se como um dos agentes. Você conhece suas restrições, o domínio da sua variável e o domínio das variáveis dos seus vizinhos. Você pode usar essas informações para determinar se algum dos valores do domínio da sua variável é impossível. Isto é, se houver um valor que viole alguma restrição, não importa qual valor um de seus vizinhos use, então esse valor claramente não fará parte da solução, então você pode se livrar dele. Da mesma forma, todos os outros agentes podem fazer o mesmo.



Esta ideia é implementada pelo algoritmo de filtragem (Waltz, 1975) mostrado em figura 2.3. Cada agente  $i$ executa o procedimento  FILTERING que o leva a fazer umREVISE com cada uma das variáveis de seus vizinhos  $x_j$. Se o agente conseguir eliminar um ou mais valores de seu domínio, ele informará aos vizinhos seu novo domínio. Quando u m agente recebe novos domínios de seus vizinhos, ele executa novamenteREVISE para esses vizinhos. Este processo se repete até que todos os agentes terminem seus cálculos e nenhuma mensagem seja enviada.

O algoritmo de filtragem tem garantia de parada, pois há apenas um número finito de valores em todos os domínios e o algoritmo só envia uma mensagem quando um valor é eliminado de um domínio. Assim que parar, o algoritmo de filtragem pode ter reduzido o tamanho de alguns domínios. Infelizmente, ele pode parar num ponto em que um ou mais agentes ainda tenham muitos valores em seu domínio. Assim, pode parar antes que uma solução seja encontrada.

A Figura 2.4 mostra um exemplo de algoritmo de filtragem aplicado ao problema de coloração de grafos. Neste problema os agentes têm três cores possíveis, mas começam com domínios já limitados, conforme indicado pelas cruzes pretas. Por exemplo, agente$x_1$ só pode ser preto ou cinza. Na primeira etapa  $x_1$ executa  REVISE$(x_1, x_3)$ e percebe que como  $x_3$ deve ser cinza, ele não pode ser cinza. Como tal, elimina o cinza de seu domínio e informa todos os outros agentes sobre isso. Os demais agentes incorporam esse conhecimento. Então, o agente  $x_2$ faz um  REVISE$(x_2, x_3)$ e percebe que também não pode ser cinza e assim elimina o cinza de seu domínio. Neste ponto  $x_2$ conhecemos a configuração final. Quando informar aos outros agentes sobre seu novo domínio, todos os agentes também saberão a solução final.

Neste exemplo, o algoritmo de filtragem encontrou uma solução específica. Infeliz mente, esta é uma ocorrência rara. Mais frequentemente descobrimos que o algoritmo de filtragem reduz apenas ligeiramente o tamanho dos domínios. Por exemplo, se executarmos o exemplo da figura 2.4, mas começarmos com todas as cores possíveis em todos os do mínios, o algoritmo de filtragem não será capaz de fazer nada. Ou seja, nenhum dos agentes é capaz de eliminar qualquer cor

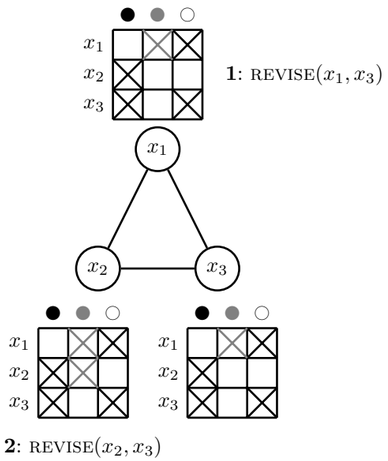



**Figure 2.4:** Filtering example. The agents start out with some prohibited colors as indicated by the black crosses. On the first step $x_1$ does his REVISE and eliminates the color gray from consideration. It then tells everyone else about this. Then $x_2$ does its revise and eliminates the color gray from its domain.

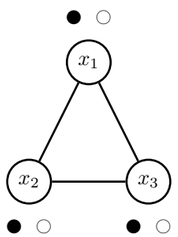



**Figure 2.5:** Example of a problem that does not have a solution and the filtering algorithm cannot that fact.

from their domain because many colorings are possible.

The filtering algorithm might also fail to eliminate values from the domains even when no coloring exists. That is, the filtering algorithm cannot reliably detect problems that do not have a solution. This problem can be illustrated with the example in figure 2.5. Here the nodes have to choose between either white or black. We know that there is no possible way to color these nodes without violating a constraint. However, the filtering algorithm will get stuck at the first step in this example and will not be able to reduce the size of the domains. Namely, each agent considers that if he is black then the other two could be white and if he is white then the other two could be black. As such, no agent eliminates colors from its domain.

### 2.1.2 Hyper-Resolution Based Consistency Algorithm

The reason the filtering algorithm fails for this example is because the REVISE function considers only the agent's own value and one other agent's value at a time. More generally, we need to consider the agent's value against all possible combinations of the values of two of its neighbors, of three of its neighbors, and so on all the way to considering the values of all of its neighbors at the same time.

More formally, we say that if an agent can find a suitable value regardless of the values of $k$ of its neighbors then it can be consistent with anything they might do. If we can say this for all agents then we have achieved **$k$-consistency**.

$k$-CONSISTENCY

**Definition 2.3** ($k$-consistency)**.** *Given any instantiation of any $k-1$ variables that satisfy all constraints it is possible to find an instantiation of any $k^{th}$ variable such that all $k$ variable values satisfy all constraints.*

If we can prove that problem is $j$-consistent for $j = 2, 3, \ldots$ up to some number $k$ then we could say that the problem is strongly $k$-consistent.



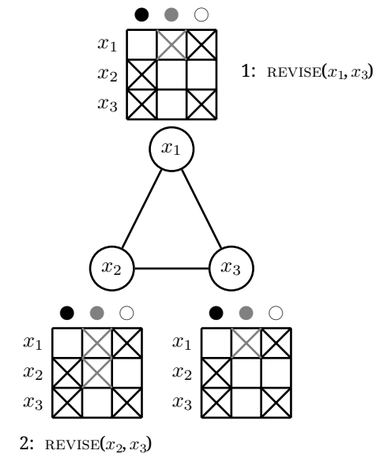



Figura 2.4: Exemplo de filtragem. Os agentes começam com algumas cores proibidas, conforme indicado pelas cruzes pretas. Na primeira etapa $x_1$ faz o seuREVISEe elimina a cor cinza de consideração. Em seguida, ele conta a todos sobre isso. Então $x_2$ faz sua revisão e elimina a cor cinza de seu domínio.

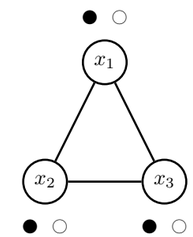



Figura 2.5: Exemplo de problema que não tem solução e o algoritmo de filtragem não pode esse fato.

do seu domínio porque muitas colorações são possíveis.

O algoritmo de filtragem também pode falhar na eliminação de valores dos domínios mesmo quando não existe coloração. Ou seja, o algoritmo de filtragem não consegue detectar de forma confiável problemas que não possuem solução. Este problema pode ser ilustrado com o exemplo da figura 2.5. Aqui os nós devem escolher entre branco ou preto. Sabemos que não há maneira possível de colorir esses nós sem violar uma restrição. No entanto, o algoritmo de filtragem ficará preso na primeira etapa deste exemplo e não será capaz de reduzir o tamanho dos domínios. Ou seja, cada agente considera que se for preto então os outros dois poderiam ser brancos e se ele for branco então os outros dois poderiam ser pretos. Como tal, nenhum agente elimina cores do seu domínio.

### 2.1.2 Algoritmo de consistência baseado em hiper-resolução

A razão pela qual o algoritmo de filtragem falha neste exemplo é porque a função REVISE considera apenas o valor do próprio agente e o valor de outro agente por vez. De for ma mais geral, precisamos considerar o valor do agente em relação a todas as combinações possíveis dos valores de dois dos seus vizinhos, de três dos seus vizinhos, e assim por diante, até considerar os valores de todos os seus vizinhos ao mesmo tempo.

Mais formalmente, dizemos que se um agente puder encontrar um valor adequado independentemente dos valores de $k$ de seus vizinhos, então ele poderá ser consistente com qualquer coisa que eles possam fazer. Se pudermos dizer isso para todos os agentes, então alcançamos consistência $k$.

$k$-CONSISTENCY

*Definição 2.3 ($k$-consistência). Dada qualquer instanciação de qualquer variável $k-1$ que satisfaça todas as restrições, é possível encontrar uma instanciação de qualquer variável $k^{th}$ tal que todos os valores da variável $k$ satisfaçam todas as restrições.*

Se pudermos provar que o problema é consistente com $j$ para $j = 2, 3, \ldots$ até algum número$k$ então poderíamos dizer que o problema é fortemente consistente com $k$.

| Time | $\mathbf{x_1}$ | $\mathbf{x_2}$ | $\mathbf{x_3}$ |
|---|---|---|---|
| | $x_1 = ○ \vee x_1 = ●$ | $x_2 = ○ \vee x_2 = ●$ | $x_3 = ○ \vee x_3 = ●$ |
| | $\neg(x_1 = ○ \wedge x_2 = ○)$ | $\neg(x_1 = ○ \wedge x_2 = ○)$ | $\neg(x_1 = ○ \wedge x_2 = ○)$ |
| | $\neg(x_1 = ● \wedge x_2 = ●)$ | $\neg(x_1 = ● \wedge x_2 = ●)$ | $\neg(x_1 = ● \wedge x_2 = ●)$ |
| 1 | $\neg(x_2 = ○ \wedge x_3 = ●)$ | $\neg(x_2 = ○ \wedge x_3 = ●)$ | $\neg(x_2 = ○ \wedge x_3 = ●)$ |
| 2 | $\neg(x_2 = ● \wedge x_3 = ○)$ | $\neg(x_3 = ●)$ | $\neg(x_2 = ● \wedge x_3 = ○)$ |
| 3 | | $\neg(x_2 = ● \wedge x_3 = ○)$ | $\neg(x_3 = ○)$ |
| 4 | | $\neg(x_3 = ○)$ | $\neg(x_3 = ●)$ |

**Table 2.1:** Example databases for a sample run of the hyper-resolution based consistency algorithm as applied to the graph of figure 2.5. Only a few of the nogoods produced by the graph's constraints are shown due to space constraints. You can infer the rest. New statements are added starting at time 1.

**Definition 2.4** (Strongly $k$-consistent)**.** *A problem is* strongly $k$-consistent *if it is $j$-consistent for all $j \leq k$.*

We now face the problem of proving that a particular DCSP is strongly $k$-consistent. Notice that the filtering algorithm guarantees 2-consistency since it ensures that all agents can find a value regardless of the value any one of their neighbors might set. We need to extend this procedure to more than one neighbor. This can be done using the **hyper-resolution rule**.

HYPER-RESOLUTION RULE

**Definition 2.5** (Hyper-Resolution Rule)**.**

$$\frac{\begin{array}{c} A_1 \vee A_2 \vee \cdots \vee A_m \\ \neg(A_1 \wedge A_{11} \wedge \cdots) \\ \neg(A_2 \wedge A_{21} \wedge \cdots) \\ \vdots \\ \neg(A_m \wedge A_{m1} \wedge \cdots) \end{array}}{\neg(A_{11} \wedge \cdots \wedge A_{21} \wedge \cdots \wedge A_{m1} \wedge \cdots)}.$$

The hyper-resolution rule is a simple deduction rule from formal logic whose form just so happens to correspond exactly to what we need. The idea is that the agents' domains can be represented as an OR statement like the first one in the rule. For example, if agent $x_1$ can be either gray, black, or white then we write $x_1 = \text{gray} \vee x_1 = \text{black} \vee x_1 = \text{white}$. The constraints are represented as negations like the one in the second line of the hyperesolution rule. For example, if there is a constraint that says that $x_1$ and $x_2$ cannot both be white we would write that as $\neg(x_1 = \text{white} \wedge x_2 = \text{white}$. Once we have represented both the domain and the constraints in this logical form then we can use the hyper-resolution rule to generate new constraints which are added to the agent's knowledge and communicated to the other agents. Each one of these new constraints is called a **nogood**.

NOGOOD

We can use the hyper-resolution rule to implement a distributed constraint satisfaction algorithm that, unlike the filtering algorithm, will detect all cases where there is no solution and will find a solution when there is one. The algorithm is basically the same as the filtering algorithm but we modify the REVISE function to instead use the hyper-resolution to generate new nogoods. These nogoods are then sent to the other agents. When an agent receives nogoods from other agents it adds them to its set of nogoods. The process continues until no new nogoods can be derived by any agent or until a contradiction is reached. When a contradiction is found it means that no solution exists to this problem.

We now show how the hyper-resolution algorithm would solve the problem from figure 2.5. Table 2.1 shows a few of the initial statements in the agents' databases. Note that many of the nogoods are missing due to space limitations but you can easily infer them as they correspond to the constraints in the problem. One possible run of the algorithm is as follows:

Let $x_1$ start out by doing



| Time | $\mathbf{x_1}$ | $\mathbf{x_2}$ | $\mathbf{x_3}$ |
|---|---|---|---|
| | $x_1 = ○ \vee x_1 = ●$ | $x_2 = ○ \vee x_2 = ●$ | $x_3 = ○ \vee x_3 = ●$ |
| | $\neg(x_1 = ○ \wedge x_2 = ○)$ | $\neg(x_1 = ○ \wedge x_2 = ○)$ | $\neg(x_1 = ○ \wedge x_2 = ○)$ |
| | $\neg(x_1 = ● \wedge x_2 = ●)$ | $\neg(x_1 = ● \wedge x_2 = ●)$ | $\neg(x_1 = ● \wedge x_2 = ●)$ |
| 1 | $\neg(x_2 = ○ \wedge x_3 = ●)$ | $\neg(x_2 = ○ \wedge x_3 = ●)$ | $\neg(x_2 = ○ \wedge x_3 = ●)$ |
| 2 | $\neg(x_2 = ● \wedge x_3 = ○)$ | $\neg(x_3 = ●)$ | $\neg(x_2 = ● \wedge x_3 = ○)$ |
| 3 | | $\neg(x_2 = ● \wedge x_3 = ○)$ | $\neg(x_3 = ○)$ |
| 4 | | $\neg(x_3 = ○)$ | $\neg(x_3 = ●)$ |

Tabela 2.1: Exemplos de bancos de dados para uma amostra do algoritmo de consistência baseado em hiperresolução aplicado ao gráfico da figura 2.5. Apenas alguns dos não-bens produzidos pelas restrições do gráfico são mostrados devido a restrições de espaço. Você pode inferir o resto. Novas declarações são adicionadas a partir de time1.

Definição 2.4 (fortemente $k$consistente). Um problema é fortemente consistente com $k$ se for consistente com $j$ para todos $j \leq k$.

Enfrentamos agora o problema de provar que um determinado DCSP é fortemente $k$-consistente. Observe que o algoritmo de filtragem garante consistência 2, pois garante que todos os agentes possam encontrar um valor, independentemente do valor que qualquer um de seus vizinhos possa definir. Precisamos estender este procedimento a mais de um vizinho. Isso pode ser feito usando a regra de hiperresolução.

HYPER-RESOLUTION RULE

Definição 2.5 (Regra de Hiper-Resolução).

$$\frac{\begin{array}{c} A_1 \vee A_2 \vee \cdots \vee A_m \\ \neg(A_1 \wedge A_{11} \wedge \cdots) \\ \neg(A_2 \wedge A_{21} \wedge \cdots) \\ \vdots \\ \neg(A_m \wedge A_{m1} \wedge \cdots) \end{array}}{\neg(A_{11} \wedge \cdots \wedge A_{21} \wedge \cdots \wedge A_{m1} \wedge \cdots)}.$$

A regra de hiper-resolução é uma regra de dedução simples da lógica formal cuja forma corresponde exatamente ao que precisamos. A ideia é que os domínios dos agentes possam ser representados como uma instrução OR como a primeira da regra. Por exemplo, se o agente $x_1$ pode ser cinza, preto ou branco, então escrevemos$x_1 =$ cinza $\vee x_1 =$ preto $\vee x_1 =$ branco. As restrições são representadas como negações como a da segunda linha da regra de hiperresolução. Por exemplo, se houver uma restrição que diz que $x_1$ e $x_2$ não podem ser ambos brancos, escreveríamos isso como $\neg(x_1 =$ branco $\wedge x_2 =$ branco. Depois de representarmos o domínio e as restrições nesta forma lógica, podemos usar a regra de hiper-resolução para gerar novas restrições que são adicionadas ao conhecimento do agente e comunicadas aos outros agentes. Cada uma dessas novas restrições é chamada de nogood.

NOGOOD

Podemos usar a regra de hiper-resolução para implementar um algoritmo distribuído de satisfação de restrições que, diferentemente do algoritmo de filtragem, detectará todos os casos onde não há solução e encontrará uma solução quando houver. O algoritmo é basicamente o mesmo que o algoritmo de filtragem, mas modificamos a função REVISE para usar a hiper-resolução para gerar novos bens inúteis. Esses bens inúteis são então enviados para os outros agentes. Quando um agente não recebe bens de outros agentes, ele os adiciona ao seu conjunto de não bens. O processo continua até que nenhum novo bem possa ser derivado por qualquer agente ou até que uma contradição seja alcançada. Quando uma contradição é encontrada significa que não existe solução para este problema.

Mostramos agora como o algoritmo de hiper-resolução resolveria o problema da figura 2.5. A Tabela 2.1 mostra algumas das declarações iniciais nas bases de dados dos agentes. Observe que muitos dos bens não estão faltando devido a limitações de espaço, mas você pode inferi-los facilmente, pois eles correspondem às restrições do problema. Uma execução possível do algoritmo é a seguinte:

Vamos $x_1$ começar fazendo

$$\frac{\begin{array}{c}x_1 = \bigcirc \vee x_1 = \bullet \\ \neg(x_1 = \bigcirc \wedge x_2 = \bigcirc) \\ \neg(x_1 = \bullet \wedge x_3 = \bullet)\end{array}}{\neg(x_2 = \bigcirc \wedge x_3 = \bullet)}$$

at time 1. It then sends $\neg(x_2 = \bigcirc \wedge x_3 = \bullet)$ to $x_2$ and $x_3$. $x_2$ then can do

$$\frac{\begin{array}{c}x_2 = \bigcirc \vee x_2 = \bullet \\ \neg(x_2 = \bigcirc \wedge x_3 = \bullet) \\ \neg(x_2 = \bullet \wedge x_3 = \bullet)\end{array}}{\neg(x_3 = \bullet)}$$

Meanwhile, $x_1$ can do

$$\frac{\begin{array}{c}x_1 = \bigcirc \vee x_1 = \bullet \\ \neg(x_1 = \bullet \wedge x_2 = \bullet) \\ \neg(x_1 = \bigcirc \wedge x_3 = \bigcirc)\end{array}}{\neg(x_2 = \bullet \wedge x_3 = \bigcirc)}$$

$x_1$ then sends the nogood $\neg(x_2 = \bullet \wedge x_3 = \bigcirc)$ to $x_2$ and $x_3$. Using this message, $x_2$ can now do

$$\frac{\begin{array}{c}x_2 = \bigcirc \vee x_2 = \bullet \\ \neg(x_2 = \bigcirc \wedge x_3 = \bigcirc) \\ \neg(x_2 = \bullet \wedge x_3 = \bigcirc)\end{array}}{\neg(x_3 = \bigcirc)}$$

$x_2$ then sends $\neg(x_3 = \bigcirc)$ and $\neg(x_3 = \bullet)$ to $x_3$. Using this message, $x_3$ can then do

$$\frac{\begin{array}{c}x_3 = \bigcirc \vee x_3 = \bullet \\ \neg(x_3 = \bigcirc) \\ \neg(x_3 = \bullet)\end{array}}{\text{Contradiction}}$$

This last step derives a contradiction which means that the algorithm has been able to prove that no solution exists.

In practice, the hyper-resolution algorithm is very slow because there is often little guidance as to which nogoods should be derived first. That is, at any one time each agent can derive a large number of different nogoods. Most of these are useless to the other agents and there is no easy way for the agent to determine which nogoods might be most useful since that would require that it also know about the other agents' databases. As such, we find that in practice the hyper-resolution algorithm spends a lot of time deriving useless nogoods. This is especially true as the size of the nogoods—their number of terms—increases. Also, if the problem does have a solution then the algorithm only stops when all the agents prove to themselves that they cannot generate any new nogood, which can take a very long time. In summary, hyper-resolution in its raw form is impractical for most cases with the possible exception of highly over-constrained problems.

### 2.1.3 Asynchronous Backtracking

Earlier, in figure 2.2, we saw a depth first search algorithm for solving a constraint satisfaction problem. The algorithm sets a variable to a specific value and, recursively, tries to set a new variable to some value which would not violate any of the constraints with existing values. If no such value exists the algorithm backs up and re-sets one of the variables it has already set. This type of depth first search is also known as a **backtracking** algorithm because we set the variables to some values and then if there is a conflict we backtrack, unset variables, and try again with different values.





$$\frac{\begin{array}{c}x_1 = \bigcirc \vee x_1 = \bullet \\ \neg(x_1 = \bigcirc \wedge x_2 = \bigcirc) \\ \neg(x_1 = \bullet \wedge x_3 = \bullet)\end{array}}{\neg(x_2 = \bigcirc \wedge x_3 = \bullet)}$$

no tempo 1. Em seguida, envia $\neg(x_2 = \bigcirc \wedge x_3 = \bullet)$ para $x_2$ e $x_3$. $x_2$ então posso fazer

$$\frac{\begin{array}{c}x_2 = \bigcirc \vee x_2 = \bullet \\ \neg(x_2 = \bigcirc \wedge x_3 = \bullet) \\ \neg(x_2 = \bullet \wedge x_3 = \bullet)\end{array}}{\neg(x_3 = \bullet)}$$

Enquanto isso, $x_1$ podemos fazer

$$\frac{\begin{array}{c}x_1 = \bigcirc \vee x_1 = \bullet \\ \neg(x_1 = \bullet \wedge x_2 = \bullet) \\ \neg(x_1 = \bigcirc \wedge x_3 = \bigcirc)\end{array}}{\neg(x_2 = \bullet \wedge x_3 = \bigcirc)}$$

$x_1$ em seguida, envia o nogood $\neg(x_2 = \bullet \wedge x_3 = \bigcirc)$ para $x_2$ e $x_3$. Usando esta mensagem,$x_2$ agora podemos fazer

$$\frac{\begin{array}{c}x_2 = \bigcirc \vee x_2 = \bullet \\ \neg(x_2 = \bigcirc \wedge x_3 = \bigcirc) \\ \neg(x_2 = \bullet \wedge x_3 = \bigcirc)\end{array}}{\neg(x_3 = \bigcirc)}$$

$x_2$ em seguida, envia $\neg(x_3 = \bigcirc)$ e $\neg(x_3 = \bullet)$ para $x_3$. Usando esta mensagem, $x_3$ podemos então fazer

$$\frac{\begin{array}{c}x_3 = \bigcirc \vee x_3 = \bullet \\ \neg(x_3 = \bigcirc) \\ \neg(x_3 = \bullet)\end{array}}{\text{Contradiction}}$$

Este último passo deriva uma contradição que significa que o algoritmo foi capaz de provar que não existe solução.

Na prática, o algoritmo de hiper-resolução é muito lento porque muitas vezes há pouca orientação sobre quais bens não devem ser derivados primeiro. Ou seja, a qualquer momento, cada agente pode derivar um grande número de bens diferentes. A maioria deles é inútil para os outros agentes e não há uma maneira fácil para o agente determinar quais bens inúteis podem ser mais úteis, uma vez que isso exigiria que ele também conhecesse os bancos de dados dos outros agentes. Como tal, descobrimos que, na prática, o algoritmo de hiper-resolução gasta muito tempo derivando bens inúteis. Isto é especialmente verdadeiro à medida que o tamanho dos bens inúteis – o seu número de termos – aumenta. Além disso, se o problema tiver solução, o algoritmo só para quando todos os agentes provam a si mesmos que não podem gerar nenhum novo mal, o que pode levar muito tempo. Em resumo, a hiper-resolução na sua forma bruta é impraticável para a maioria dos casos, com a possível excepção de problemas altamente limitados.

### 2.1.3 Retrocesso Assíncrono

Anteriormente, na figura 2.2, vimos um algoritmo de busca em profundidade para resolver um problema de satisfação de restrições. O algoritmo define uma variável para um valor específico e, recursivamente, tenta definir uma nova variável para algum valor que não viole nenhuma das restrições com valores existentes. Se não existir tal valor, o algoritmo faz backup e redefine uma das variáveis que já definiu. Este tipo de pesquisa em profundidade também é conhecido como algoritmo de retrocesso porque definimos as variáveis com alguns valores



e então, se houver um conflito, recuamos, desativamos as variáveis e tentamos novamente com valores diferentes.

ASYNCHRONOUS BACKTRACKING

The **asynchronous backtracking** algorithm (ABT), is a distributed asynchronous version of the centralized depth first search algorithm for constraint satisfaction (Yokoo and Hirayama, 2000). ABT performs the same kind of search but it does so via the use of messages between agents and under the understanding that only the owner of the variable can change its value.

The ABT algorithm also implements some performance improvements on the basic depth first search. As we mentioned earlier, there is often much to be gained by properly ordering the variables. In general, we want to place variables that are involved in more constraints towards the top of the tree and search them first so as to avoid having to do some backtracking later on. This heuristic has been shown to work well and is employed by ABT.

Each agent $i$ in ABT is responsible for variable $x_i$. All agents are given a fixed *priority* ordering, from 1 to $n$. Each agent $i$ can change the value of its variable $x_i$. The agents keep track of the others' variables in *local-view*. The list of *neighbors* is initially set to be just the other agents with whom the agent shares a constraint but, as we will see, this list can grow. The agents find out other agents' variable values by sending messages. There are only three types of messages that ABT sends. Thus, only three remote procedures need to be implemented.

1. HANDLE-OK?$(j, x_j)$ where $j$ is the name of another agent and $x_j$ is its current variable value. This message asks the receiver if that assignment does not violate any of his constraints.

2. HANDLE-NOGOOD$(j, nogood)$ which means that $j$ is reporting that it can't find a value for his variable because of *nogood*.

3. HANDLE-ADD-NEIGHBOR$(j)$ which requests the agent to add some other agent $j$ as its neighbor. This message is handled by simply adding that agent to *neighbors*.

The full algorithm can be seen in figure 2.6. All agents start out by setting their $x_i$ variable to some value and then asking all their neighbors to HANDLE-OK? After that they become event-driven, handling messages as they arrive. ABT assumes that messages are never lost and arrive and in the same order they were sent.

The procedure CHECK-LOCAL-VIEW checks that the agent's current value $x_i$ is consistent with the values it thinks its neighbors have. If not then it checks to see if it can find some other value that will work. If it can't find one then this means it must BACKTRACK and find a new value. If it can then it sets $x_i$ to that value and informs its neighbors.

Nogoods are handled by HANDLE-NOGOOD. The first thing this procedure does is to see if any of the agents in the nogood is not in *neighbors* so it can add it. This step is needed to ensure the algorithm will arrive at a correct answer. The nogood is added to the set of constraints and CHECK-LOCAL-VIEW is called to ensure there are no conflicts.

Finally, the BACKTRACK procedure is responsible for generating and sending new nogoods. As we saw, it is only called when there is no way for the agent to set is value so that no constraints are violated. This means that someone else must change their value. That someone else is the agent with the lowest priority in the nogood. BACKTRACK first uses the hyper-resolution rule (or similar) to generate one or more nogoods. Each one of these nogoods is then sent to the agent in that nogood that has the lowest priority. Also, if the hyper-resolution generated a contradiction, represented as an empty set then this means that no solution exists and thus the algorithm is terminated.

Notice that, since the nogoods are always sent to the lowest priority agent and the agents have linear priorities, the nogoods always flow from the high priority agents to the low priority agents. Namely, the highest priority agent will never receive a nogood, the second highest priority agent will only receive nogoods from the highest priority agent, and so on. This suggests to us that the lower priority agents will end up doing more work.

Some versions of ABT use one nogood in line 1 of BACKTRACK, others use more.



ABTgc



ASYNCHRONOUS BACKTRACKING

O algoritmo de retrocesso assíncrono (ABT), é um algoritmo assíncrono distribuído versão nous do algoritmo de busca de profundidade centralizada para satisfação de restrições (Yokoo e Hirayama, 2000). ABT realiza o mesmo tipo de pesquisa, mas através do uso de mensagens entre agentes e sob o entendimento de que somente o proprietário da variável pode alterar seu valor.

O algoritmo ABT também implementa algumas melhorias de desempenho na primeira pesquisa básica em profundidade. Como mencionamos anteriormente, muitas vezes há muito a ganhar ordenando adequadamente as variáveis. Em geral, queremos colocar variáveis que estão envolvidas em mais restrições no topo da árvore e pesquisá-las primeiro para evitar ter que fazer algum retrocesso mais tarde. Esta heurística demonstrou funcionar bem e é empregada por ABT.

Cada agente $i$ em ABT é responsável pela variável $x_i$. Todos os agentes recebem uma ordem de prioridade fixa, de 1 a $n$. Cada agente $i$ pode alterar o valor de sua variável $x_i$. Os agentes acompanham as variáveis dos outros em local-view . A lista de vizinhos é inicialmente definida como sendo apenas os outros agentes com os quais o agente compartilha uma restrição, mas, como veremos, esta lista pode crescer. Os agentes descobrem os valores das variáveis de outros agentes enviando mensagens. Existem apenas três tipos de mensagens que ABT envia. Assim, apenas três procedimentos remotos precisa m ser implementados.

1. HANDLE-OK?$(j, x_j)$ onde $j$ é o nome de outro agente e $x_j$ é o valor da variável atual. Esta mensagem pergunta ao receptor se aquela atribuição não viola nenhu ma de suas restrições. 2. HANDLE-NOGOOD$(j$, nogood ) o que significa que $j$ está relatando que não consegue encontrar um valor para sua variável por causa de *nogood*. 3. HANDLE-ADD-NEIGHBOR$(j)$ que solicita ao agente para adicionar outro agente$j$ como seu vizinho. Esta mensagem é tratada simplesmente adicionando esse agente aos vizinhos.

O algoritmo completo pode ser visto na figura 2.6. Todos os agentes começam definindo sua$x_i$ variável com algum valor e depois pedem a todos os seus vizinhos para HANDLE-OK? Depois disso, eles se tornam orientados a eventos, manipulando as mensagens à medida que chegam. ABT assume que as mensagens nunca são perdidas e chegam na mesma ordem em que foram enviadas.

O procedimento CHECK-LOCAL-VIEW verifica se o valor atual do agente $x_i$ é consistente com os valores que ele acha que seus vizinhos possuem. Caso contrário, ele verifica se consegue encontrar algum outro valor que funcione. Se não conseguir encontrar um, isso significa que ele deve BACKTRACK e encontrar um novo valor. Se puder, ele define $x_i$ com esse valor e informa seus vizinhos.

Nenhum bem é tratado por HANDLE-NOGOOD. A primeira coisa que este procedimento faz é verificar se algum dos agentes no nogood não está em vizinhos para poder adicioná-lo. Esta etapa é necessária para garantir que o algoritmo chegará a u ma resposta correta. O nogood é adicionado ao conjunto de restrições e CHECK-LOCAL-VIEW é chamado para garantir que não haja conflitos.

Por fim, o procedimento BACKTRACK é responsável por gerar e enviar novos nogoods. Como vimos, ele só é chamado quando não há como o agente definir seu valor para que nenhuma restrição seja violada. Isso significa que outra pessoa deve alterar seu valor. Essa outra pessoa é o agente com a prioridade mais baixa no que não é bom. BACKTRACKprimeiro usa a regra de hiper-resolução (ou similar) para gerar um ou mais bens inúteis. Cada um desses itens inúteis é então enviado ao agente naquela forma inútil.

que tem a prioridade mais baixa. Além disso, se a hiper-resolução gerou uma contradição, representada como um conjunto vazio, isso significa que não existe solução e, portanto, o algoritmo é encerrado.

Observe que, como os bens inúteis são sempre enviados para o agente de prioridade mais baixa e os agentes têm prioridades lineares, os bens inúteis sempre fluem dos agentes de alta prioridade para os agentes de baixa prioridade. Ou seja, o agente de maior prioridade nunca receberá nada de bom, o segundo agente de maior prioridade só receberá nada de bom do agente de maior prioridade e assim por diante. Isto sugere-nos que os agentes de menor prioridade acabarão por trabalhar mais.

Algumas versões de ABT usa m um nogood na linha 1 de BACKTRACK, outras usam mais.



ABTgc

HANDLE-OK?$(j, x_j)$
1 *local-view* $\leftarrow$ *local-view* $+(j, x_j)$
2 CHECK-LOCAL-VIEW()

CHECK-LOCAL-VIEW()
1 **if** *local-view* and $x_i$ are not consistent
2 **then if** no value in $D_i$ is consistent with *local-view*
3 **then** BACKTRACK()
4 **else** select $d \in D_i$ consistent with *local-view*
5 $x_i \leftarrow d$
6 $\forall_{k \in neighbors}$ $k$.HANDLE-OK?$(i, x_i)$

HANDLE-ADD-NEIGHBOR$(j)$
1 *neighbors* $\leftarrow$ *neighbors* $+j$

HANDLE-NOGOOD$(j,$ *nogood*$)$
1 record *nogood* as a new constraint
2 **for** $(k, x_k) \in$ *nogood* where $k \notin$ *neighbors*
3 **do** $k$.HANDLE-ADD-NEIGHBOR$(i)$
4 *neighbors* $\leftarrow$ *neighbors* $+k$
5 *local-view* $\leftarrow$ *local-view* $+(k, x_k)$
6 *old-value* $\leftarrow x_i$
7 CHECK-LOCAL-VIEW()
8 **if** *old-value* $\neq x_i$
9 **then** $j$.HANDLE-OK?$(i, x_i)$

BACKTRACK()
1 *nogoods* $\leftarrow \{V \mid V =$ inconsistent subset of *local-view* using hyper-resolution rule$\}$
2 **if** an empty set is an element of *nogoods*
3 **then** broadcast that there is no solution
4 terminate this algorithm
5 **for** $V \in$ *nogoods*
6 **do** select $(j, x_j)$ where $j$ has lowest priority in $V$
7 $j$.HANDLE-NOGOOD$(i, V)$
8 *local-view* $\leftarrow$ *local-view* $-(j, x_j)$
9 CHECK-LOCAL-VIEW()

**Figure 2.6:** ABT algorithm. All agents start by setting themselves to some color and invoking a HANDLE-OK? on all their neighbors. After this they become event driven.

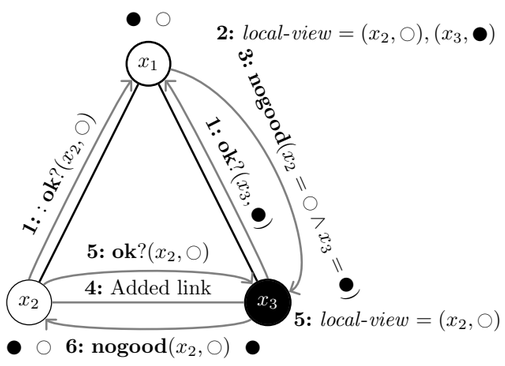



**Figure 2.7:** ABT example. The numbers preceding each message indicate the time at which the particular action took place. Only the first six steps are shown.



HANDLE-OK?*(j, $x_j$) 1 visualização l*
*ocal* $\leftarrow$*visualização local* $+(j, x_j)$
2 CHECK-LOCAL-VIEW*()*

CHECK-LOCAL-VIEW()
1 se a visualização local e $x_i$ não forem consistentes 2 então se nenhum valor em $D_i$ for consistente com a visualização local 3 então BACKTRACK() 4 caso contrário selecione $d \in D_i$ consistente com a visualização local5 $x_i \leftarrow d$
6 $\forall_{k \in neighbors}$ $k$.HANDLE-OK?*(i, $x_i$)*

HANDLE-ADD-NEIGHBOR*(j)*
*1 vizinhos* $\leftarrow$ *vizinhos* $+j$

HANDLE-NOGOOD*(j, nada bom)*

*1 registro* *nogood* *como uma nova restrição 2 para (k, $x_k$)* $\in$ *não é bom onde* $k \notin$ *vizinhos 3 fazem* $k$.HANDLE-ADD-NEIGHBOR*(i) 4 vizinhos* $\leftarrow$ *vizinhos* $+k$*5 visualização local* $\leftarrow$*local -view* $+$ *(k, $x_k$) 6 valor antigo* $\leftarrow x_i$7 CHECK-LOCAL-VIEW *() 8 se valor antigo* $\neq x_i$*9 então* $j$.HANDLE-OK?*(i, $x_i$)*

BACKTRACK()
1 *nogoods* $\leftarrow \{V \mid V =$ subconjunto inconsistente de visualização local usando regra de hiper-resolução$\}$2 se um conjunto vazio for um elemento de *nogoods*3 então transmita que não há solução 4 encerre este algoritmo 5 para $V \in$ *nogoods*6 selecione $(j, x_j)$ onde $j$ tem prioridade mais baixa em $V$7 $j$.HANDLE-NOGOOD$(i, V)$ 8 visualização local $\leftarrow$local -view $-(j, x_j)$9 CHECK-LOCAL-VIEW()

Figura 2.6: ABT algoritmo. Todos os agentes começam configurando alguma cor para si mesmos e invocando a HANDLE-OK? em todos os seus vizinhos. Depois disso, eles se tornam orientados a eventos.

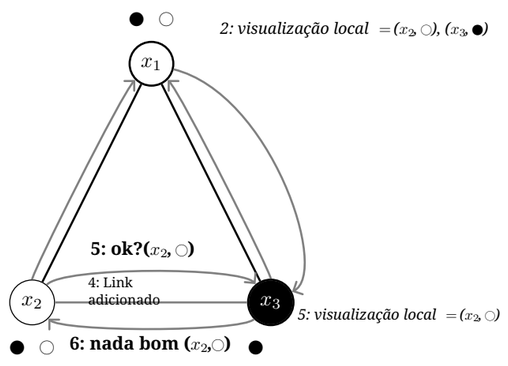



Figura 2.7: ABTexemplo. Os números que precedem cada mensagem indicam a hora em que a acção específica ocorreu. Apenas os primeiros seis passos são mostrados.

An example of ABT at work can be seen in figure 2.7. Here agents $x_2$ and $x_3$ start by setting their colors to be gray and black, respectively, and asking $x_1$ to HANDLE-OK?. Agent $x_1$ then generates a new nogood and sends a *nogood* to $x_3$. $x_3$ notices that $x_2$ is in the nogood but not in his *neighbors* and so adds it as his neighbor and asks $x_2$ to add him as a neighbor. In response to this $x_2$ asks $x_3$ to HANDLE-OK? with his new color. Agent $x_3$ uses $x_2$'s new color to generate a new nogood that it sends to $x_2$. The figure ends at this point but the algorithm would continue with $x_2$ realizing that it must set itself to black and sending the appropriate HANDLE-OK? messages, and so on.

The ABT algorithm has been shown to be complete.

**Theorem 2.1** (ABT is Complete)**.** *The* ABT *algorithm always finds a solution if one exists and terminates with the appropriate message if there is no solution (Yokoo et al., 1998).*

*Proof.* The proof works by induction. In order to prove it we must first show that the agent with the highest priority never enters an infinite loop. Then show that given that all the agents with lower priority that $k$ never fall into an infinite loop then $k$ will not fall into an infinite loop. □

Unfortunately, after running a few examples of this algorithm one quickly notices some problems. The most important is the uneven division of labor. In ABT the lowest priority agent ends up doing many times more work than the highest priority agents, which is reflected by the number of messages each agent receives. The difference only gets worse as the number of agents increases. Another drawback is the fact that in practice ABT can take very long as it also suffers from the same problem as regular backtracking. That is, if the high priority agents make bad decisions (choose colors that will not work) then it takes a long time to fix these as all the other agents have to prove that those colors will not work by trying all possible combinations of their colors. In the graph coloring problem we notice that the time to solve a problem increases exponentially as the ratio of edges to nodes increases. This means that as the problem becomes more constrained the time to find a solution explodes.

Another possible improvement to ABT can be found in line 1 of BACKTRACKING which tells us to find new nogoods. As you can see, the line does not specify which subset of nogoods to generate. One way to improve ABT is by managing which nogoods get generated such that only those that are likely to lead to a solution get generated (Jiang and Vidal, 2005; Jiang and Vidal, 2006). This strategy has the effect of greatly reducing the number of cycles needed to find a solution.

We can also extend ABT by allowing agents to solve the problem without the need to add new links, thus limiting the agent to agent communication. This enhancement has been implemented by the kernel ABT algorithm (Bessière et al., 2005). This algorithm could be used in applications where it is impossible to change the original communication network.

### 2.1.4 Asynchronous Weak-Commitment Search

As we mentioned, ABT suffers from the fact that the agents have a fixed priority so that if the highest priority agent makes a bad decision then all the other agents must perform an exhaustive search to prove him wrong. The **Asynchronous Weak-Commitment** (AWC) algorithm tries to avoid this problem by giving the agents dynamic priorities. The agents all start out with the same priority but whenever an agent is forced to backtrack and resolve a new nogood then it raises its priority to be higher than anyone else's. The agents inform each other about their priorities by including them in the HANDLE-OK? and HANDLE-NOGOOD remote procedure calls.

AWC also uses the min-conflict heuristic (Minton et al., 1992) in order to reduce the risk of making a bad decision. The min-conflict heuristic tells the agents to choose the value for their variable which minimizes the number of constraint violations with other agents given the current values.

ABTmm

ABTkgc

ASYNCHRONOUS WEAK-COMMITMENT

AWCgc



Um exemplo de ABT no trabalho pode ser visto na figura 2.7. Aqui, os agentes $x_2$ e $x_3$ começam definindo suas cores como cinza e preto, respectivamente, e pedindo $x_1$ para HANDLE-OK?. O agente $x_1$ então gera um novo nogood e envia um nogood para $x_3$. $x_3$ percebe que $x_2$ está no nogood mas não em seus vizinhos e então o adiciona como seu vizinho e pede $x_2$ para adicioná-lo como seu vizinho. Em resposta a isso, $x_2$ pede $x_3$ para HANDLE-OK?com sua nova cor. O agente $x_3$ usa a nova cor de $x_2$ para gerar um novo nogood que envia para $x_2$. A figura termina neste ponto, mas o algoritmo continuaria $x_2$percebendo que deve se definir como preto e enviando as mensagens HANDLE-OK?apropriadas, e assim por diante.

O algoritmo ABT mostrou-se completo.

*Teorema 2.1 (*ABT*está completo). O algoritmo* ABT *sempre encontra uma solução, se existir, e termina com a mensagem apropriada se não houver solução (Yokoo et al., 1998).*

Prova. A prova funciona por indução. Para provar isso devemos primeiro mostrar que o agente com maior prioridade nunca entra em um loop infinito. Em seguida, mostre que, dado que todos os agentes com prioridade mais baixa que $k$ nunca caem em um loop infinito, então $k$ não cairão em um loop infinito. □

Infelizmente, depois de executar alguns exemplos deste algoritmo percebe-se rapidamente alguns problemas. O mais importante é a divisão desigual do trabalho. No ABTo agente de prioridade mais baixa acaba realizando muitas vezes mais trabalho do que os agentes de prioridade mais alta, o que se reflete no número de mensagens que cada agente recebe. A diferença só piora à medida que aumenta o número de agentes. Outra desvantagem é o fato de que, na prática, ABT pode demorar muito, pois também sofre do mesmo problema do backtracking regular. Isto é, se os agentes de alta prioridade tomarem decisões erradas (escolher cores que não funcionarão), então levará muito tempo para corrigi-las, pois todos os outros agentes terão que provar que essas cores não funcionarão, tentando todas as combinações possíveis de suas cores. No problema de coloração de grafos notamos que o tempo para resolver um problema aumenta exponencialmente à medida que a proporção entre arestas e nós aumenta. Ist o significa que à medida que o problema se torna mais limitado, o tempo para encontrar uma solução explode.

Outra possível melhoria para ABT pode ser encontrada na linha 1 de BACKTRACKINGque nos diz para encontrar novos bens inúteis. Como você pode ver, a linha não especifica qual subconjunto de bens não-bens deve ser gerado. Uma maneira de melhorar ABT é gerenciar quais bens não são gerados, de modo que apenas aqueles que provavelmente levarão a uma solução sejam gerados (Jiang e Vidal, 2005; Jiang e Vidal, 2006). Esta estratégia tem o efeito de reduzir significativamente o número de ciclos necessários para encontrar uma solução.

Também podemos estender ABT permitindo que os agentes resolvam o problema sem a necessidade de adicionar novos links, limitando assim a comunicação entre o agente e o agente. Esse aprimoramento foi implementado pelo algoritmo ABT do kernel (Bessi`ere et al., 2005). Este algoritmo pode ser utilizado em aplicações onde é i mpossível alterar a rede de comunicação original.

### 2.1.4 Pesquisa assíncrona de compromisso fraco

Como mencionamos, ABT sofre com o fato de os agentes terem uma prioridade fixa, de modo que se o agente de maior prioridade tomar uma decisão errada, todos os outros agentes deverão realizar uma busca exaustiva para provar que ele está errado. O algoritmo Asynchronous Weak-Commitment (AWC) tenta evitar esse problema dando aos agentes prioridades dinâmicas. Todos os agentes começam com a mesma prioridade, mas sempre que um agente é forçado a voltar atrás e resolver um novo problema, ele aumenta sua prioridade para ser maior do que a de qualquer outro. Os agentes informam uns aos outros sobre suas prioridades, incluindo-as nas chamadas de procedimento remoto HANDLE-OK? e HANDLE-NOGOOD .

AWC também utiliza a heurística do mínimo conflito (Minton et al., 1992) para reduzir o risco de tomar uma decisão errada. A heurística de conflito mínimo diz aos agentes para escolherem o valor para sua variável, o que minimiza o número de violações de restrições com outros agentes, dados os valores atuais.

ABTmm

ABTkgc

ASYNCHRONOUS WEAK-COMMITMENT

AWCgc

CHECK-LOCAL-VIEW

1 **if** $x_i$ is consistent with *local-view*
2 **then return**
3 **if** no value in $D_i$ is consistent with *local-view*
4 **then** BACKTRACK()
5 **else** select $d \in D_i$ consistent with *local-view* and which minimizes constraint violations with lower priority agents.
6 $x_i \leftarrow d$
7 $\forall_{k \in neighbors}$ $k$.HANDLE-OK?$(i, x_i, priority)$

BACKTRACK

1 generate a nogood $V$
2 **if** $V$ is empty nogood
3 **then** broadcast that there is no solution
4 terminate this algorithm
5 **if** $V$ is a new nogood
6 **then** $\forall_{(k,x_k) \in V}$ $k$.HANDLE-NOGOOD$(i, j, priority)$
7 $priority \leftarrow 1 + \max\{neighbors$'priorities$\}$
8 select $d \in D_i$ consistent with *local-view* and which minimizes constraint violations with lower priority agents.
9 $x_i \leftarrow d$
10 $\forall_{k \in neighbors}$ $k$.HANDLE-OK?$(i, x_i, priority)$

**Figure 2.8:** Asynchronous weak-commitment search algorithm. The rest of the procedures are the same as in ABT, from figure 2.6 except that they now also record the calling agent's priority.

The AWC algorithm is the same as ABT except that it re-defines both CHECK-LOCAL-VIEW and BACKTRACK. The re-defined functions from the AWC algorithm can be seen in figure 2.8. As you can see, it amounts to only a small modification to ABT. The agents now must keep track of all their neighbors' priorities. Whenever an agent detects a nogood it sets its priority to be higher than all its neighbors. This technique results in having the most constrained agent pick its value first.

Note that the algorithm, as presented in figure 2.8, glosses over some important implementation details. Line 5 of BACKTRACK asks us to check if $V$ is a new nogood. This means that the nogoods need to be stored in a data structure with a fast search function, perhaps a hash table or a database. Line 5 in CHECK-LOCAL-VIEW, as well as line 8 in BACKTRACK, ask us to solve a minimization problem which could also take a long time for agents with high priority. Finally, line 1 of BACKTRACK does not specify which nogood we should generate and, as we mentioned in the previous section, is thus amenable to improvements.

In practice, a problem that plagues both AWC and ABT is the fact that the implementation of a good hyper-resolution function is not easy. Even if implemented correctly, finding a new nogood could take a very long time. In fact, one expects that the time required to generate new nogoods would increase linearly, perhaps exponentially, as the number of known nogoods increases.

Just like its predecessor, AWC has been shown to be complete. The only tricky part to proving its completeness lies in those changing priorities. Luckily, we note that these priorities cannot be forever changing as there are only a finite number of impossible value assignments. Thus, at some point we will have tried all possible value combinations and the priorities will stop increasing.

**Theorem 2.2** (AWC is complete)**.** *The* AWC *algorithm always finds a solution if one exists and terminates with the appropriate message if there is no solution (Yokoo and Hirayama, 2000).*

*Proof.* The priority values are changed if and only if a new nogood is found. Since the number of possible nogoods is finite the priority values cannot be changed indefinitely. When the priority values are stable AWC becomes ABT, which is complete. □

It has been shown that AWC outperforms ABT in that it finds the solution much



CHECK-LOCAL-VIEW

1 se $x_i$ for consistente com a visualização local 2 então retorne 3 se nenhum valor em $D_i$ for consistente com a visualização local 4 então BACKTRACK() 5 caso contrário selecione $d \in D_i$ consistente com a visualização local e que minimiza violações de restrições com agentes de prioridade mais baixa.6 $x_i \leftarrow d$7 $\forall_{k \in neighbors}$ $k$.HANDLE-OK?(i, $x_i$, prioridade)

BACKTRACK

1 gera um nogood $V$2 se $V$ está vazio nogood 3 então transmite que não há solução 4 termina este algoritmo 5 se $V$ é um novo nogood 6 então $\forall_{(k,x_k) \in V}$ $k$.HANDLE-NOGOOD(i, j, prioridade) 7 prioridade $\leftarrow 1 +$max{prioridades dos vizinhos}8 selecione $d \in D_i$ consistente com a visualização local e que minimiza violações de restrições com agentes de prioridade mais baixa.9 $x_i \leftarrow d$10 $\forall_{k \in neighbors}$ $k$.HANDLE-OK?(i, $x_i$, prioridade)

Figura 2.8: Algoritmo de busca assíncrona de comprometimento fraco. O resto dos procedimentos são os mesmos que em ABT, da figura 2.6, exceto que agora também registram a prioridade do agente chamador.

O algoritmo AWC é o mesmo que ABT , exceto que redefine CHECK-LOCAL-VIEW e BACKTRACK. As funções redefinidas do algoritmo AWC podem ser vistas na figura 2.8. Co mo você pode ver, isso representa apenas uma pequena modificação em ABT. Os agentes devem agora acompanhar todas as prioridades dos seus vizinhos. Sempre que um agente detecta algo ruim, ele define sua prioridade como maior que a de todos os seus vizinhos. Essa técnica faz com que o agente mais restrito escolha seu valor primeiro.

Observe que o algoritmo, conforme apresentado na figura 2.8, encobre alguns detalhes importantes de implementação. A linha 5 de BACKTRACK nos pede para verificar se $V$ novamente não é bom. Isso significa que os bens inúteis precisam ser armazenados em uma estrutura de dados com uma função de pesquisa rápida, talvez uma tabela hash ou um banco de dados. A linha 5 em CHECK-LOCAL-VIEW, assim como a linha 8 em BACKTRACK, nos pedem para resolver um problema de minimização que também pode levar muito tempo para agentes com alta prioridade. Por fim, a linha 1 de BACKTRACKnão especifica qual não-bom devemos gerar e, como mencionamos na seção anterior, é, portanto, passível de melhorias.

Na prática, um problema que assola tanto AWC quanto ABT é o fato de que a implementação de uma boa função de hiper-resolução não é fácil. Mesmo se implementado corretamente, encontrar um novo mal pode levar muito tempo. Na verdade, espera-se que o tempo necessário para gerar novos bens inúteis aumente linearmente, talvez exponencialmente, à medida que aumenta o número de bens inúteis conhecidos.

Assim como seu antecessor, AWC demonstrou ser completo. A única parte complicada para provar a sua integralidade reside na mudança de prioridades. Felizmente, notamos que estas prioridades não podem mudar para sempre, pois existe apenas um número finito de atribuições de valores impossíveis. Assim, em algum momento teremos tentado todas as combinações de valores possíveis e as prioridades deixarão de aumentar.

*Teorema 2.2 (*AWC*está completo). O algoritmo* AWC *sempre encontra uma solução, se existir, e termina com a mensagem apropriada se não houver solução (Yokoo e Hirayama, 2000).*

Prova. Os valores de prioridade são alterados se e somente se um novo erro for encontrado. Dado que o número de bens possíveis é finito, os valores de prioridade não podem ser alterados indefinidamente. Quando os valores de prioridade são estáveis, AWC torna-se ABT, que está completo. □

Foi demonstrado que AWC supera ABT na medida em que encontra a solução muito

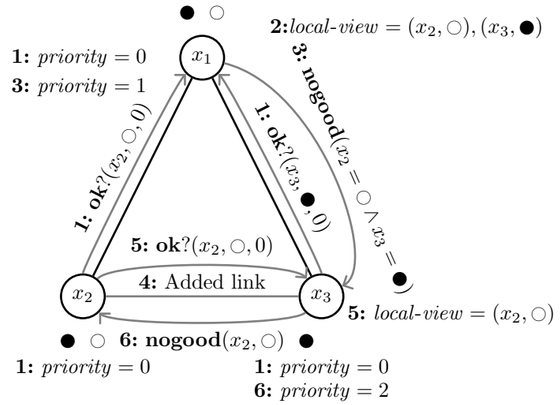



**Figure 2.9:** AWC example. The numbers indicate the time at which the particular action took place. Only the first six steps are shown. Note how the priorities of the agents change.

faster in randomly generated graphs (Yokoo and Hirayama, 2000). Neither ABT nor AWC provide us with guidance as to which nogood should be generated. That is, at any step there are many different nogoods that can be generated and these nogoods get bigger as the number of neighbors increases.

### 2.1.5 Distributed Breakout

Another way to approach the problem of distributed constraint satisfaction, different from the backtracking idea, is to use a hill-climbing method. In hill-climbing we start by assigning all variables randomly chosen values, then we change the values so as to minimize the constraint violations. We repeat the process until we cannot improve on the current solution. In a distributed version of hill-climbing each agent is a variable and all agents change their color in parallel, sending messages to their neighbors which indicate their new color.

All hill-climbing algorithms suffer from the problem of local minima. That is, it is possible for the system to reach a state were no one agent can find a better value for its variable and yet that is not the best possible value assignment for the system. That is, a local minimum can be recognized by the fact that no one agent can reduce the total number of constraint violations by changing its color but that number could be reduced by having two or more agents change their color simultaneously. As such, identifying the fact that the system is in a local minimum requires all agents to broadcast the fact that they are in a local minimum and these broadcasts must be further annotated to ensure that the agents' local views are coherent with each other—they were all in the same global state when they sent out their broadcasts. Implementing all these techniques would lead to a complex algorithm with large amounts of communications and would result in long run times.

A local minimum is akin to a Nash equilibrium while the optimal solution is akin to the utilitarian solution in game theory.

The **distributed breakout** algorithm tries to bypass this problem by recognizing a **quasi-local minimum** instead of a the full local minimum (Yokoo and Hirayama, 1996; Hirayama and Yokoo, 2005; Yokoo and Hirayama, 2000). The quasi-local minimum has the advantage that it can be recognized by an individual agent.

DISTRIBUTED BREAKOUT
QUASI-LOCAL MINIMUM

**Definition 2.6** (Quasi-local minimum)**.** *An agent $x_i$ is in a* quasi-local minimum *if it is violating some constraint and neither it nor any of its neighbors can make a change that results in lower total cost for the system.*

Unfortunately, the fact that an agent is in a quasi-local minimum does not necessarily imply that it is in a local minimum since it is possible that some non-neighboring agents could make changes that would lower the total cost. On the other hand, if the system is in a local minimum then at least one agent will find itself in a quasi-local minimum. Thus, to escape local minima the algorithm identifies quasi-local minima and increases the cost of the constraint violations.



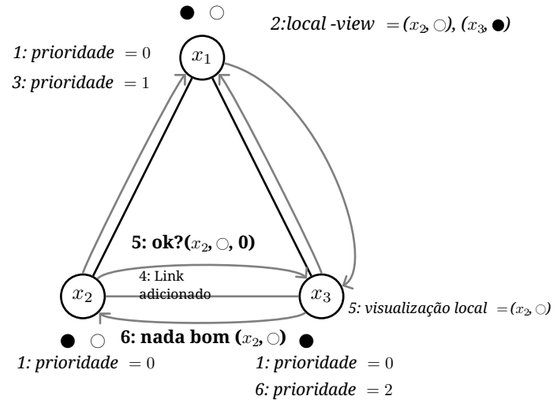



Figura 2.9: AWC exemplo. Os números indicam a hora em que a ação específica ocorreu. Apenas as primeiras seis etapas são mostradas. Observe como as prioridades dos agentes mudam.

mais rápido em gráficos gerados aleatoriamente (Yokoo e Hirayama, 2000). Nem ABT nemAWC nos fornecem orientação sobre quais bens não devem ser gerados. Ou seja, e m qualquer etapa, há muitos bens diferentes que podem ser gerados e esses bens i núteis aumentam à medida que o número de vizinhos aumenta.

### 2.1.5 Breakout Distribuído

Outra maneira de abordar o problema da satisfação de restrições distribuídas, diferente da ideia de retrocesso, é usar um método de escalada. Na escalada, começamos atribuindo a todas as variáveis valores escolhidos aleatoriamente, depois alteramos os valores de modo a minimizar as violações das restrições. Repetimos o processo até não conseguirmos melhorar a solução atual. Numa versão distribuída do hill-climbing cada agente é uma variável e todos os agentes mudam de cor em paralelo, enviando mensagens aos seus vizinhos que indicam a sua nova cor.

Todos os algoritmos de escalada sofrem com o problema dos mínimos locais. Ou seja, é possível que o sistema atinja um estado onde nenhum agente consiga encontrar um valor melhor para sua variável e ainda assim essa não seja a melhor atribuição de valor possível para o sistema. Ou seja, um mínimo local pode ser reconhecido pelo facto de ninguém

O agente pode reduzir o número total de violações de restrições alterando sua cor, mas esse número pode ser reduzido fazendo com que dois ou mais agentes mudem de cor simultaneamente. Como tal, identificar o facto de o sistema estar num mínimo local exige que todos os agentes transmitam o facto de estarem num mínimo local e estas transmissões devem ser anotadas posteriormente para garantir que as opiniões locais dos agentes sejam coerentes entre si – todos estavam no mesmo estado global quando enviaram as suas transmissões. A implementação de todas essas técnicas levaria a um algoritmo complexo co m grandes quantidades de comunicações e resultaria em longos tempos de execução.

Um mínimo local é semelhante ao equilíbrio de Nash, enquanto a solução ótima é semelhante à solução utilitária na teoria dos jogos.

O algoritmo de breakout distribuído tenta contornar esse problema reconhecendo nizando um mínimo quase local em vez de um mínimo local completo (Yokoo e Hirayama, 1996; Hirayama e Yokoo, 2005; Yokoo e Hirayama, 2000). O mínimo quase local tem a vantagem de poder ser reconhecido por um agente individual.

DISTRIBUTED BREAKOUT
QUASI-LOCAL MINIMUM

*Definição 2.6 (Mínimo quase local). Um agente $x_i$ está em um mínimo quase local se estiver violando alguma restrição e nem ele nem nenhum de seus vizinhos puderem fazer uma mudança que resulte em menor custo total para o sistema.*

Infelizmente, o facto de um agente estar num mínimo quase local não implica necessariamente que esteja num mínimo local, uma vez que é possível que alguns agentes não vizinhos possam fazer alterações que reduzam o custo total. Por outro lado, se o sistema estiver num mínimo local, pelo menos um agente se encontrará num mínimo quase local. Assim, para escapar dos mínimos locais o algoritmo identifica mínimos quase locais e aumenta o custo das violações das restrições.

HANDLE-OK?$(j, x_j)$
1 *received-ok*$[j] \leftarrow$ TRUE
2 *agent-view* $\leftarrow$ *agent-view* $+(j, x_j)$
3 **if** $\forall_{k \in neighbors}$ *received-ok*$[k]$ = TRUE
4 **then** SEND-IMPROVE()
5 $\forall_{k \in neighbors}$ *received-ok*$[k] \leftarrow$ FALSE

SEND-IMPROVE()
1 *new-value* $\leftarrow$ value that gives maximal improvement
2 *my-improve* $\leftarrow$ possible maximal improvement
3 $\forall_{k \in neighbors}$ $k$.HANDLE-IMPROVE$(i,$ *my-improve*, *cost*$)$

HANDLE-IMPROVE$(j,$ *improve*$)$
1 *received-improve*$[j] \leftarrow$ *improve*
2 **if** $\forall_{k \in neighbors}$ *received-improve*$[k] \neq$ NONE
3 **then** SEND-OK
4 *agent-view* $\leftarrow \emptyset$
5 $\forall_{k \in neighbors}$ *received-improve*$[k] \leftarrow$ NONE

SEND-OK()
1 **if** $\forall_{k \in neighbors}$ *my-improve* $\geq$ *received-improve*$[k]$
2 **then** $x_i \leftarrow$ *new-value*
3 **if** *cost* $> 0 \wedge \forall_{k \in neighbors}$ *received-improve*$[k] \leq 0 \triangleright$ quasi-local optimum
4 **then** increase weight of constraint violations
5 $\forall_{k \in neighbors}$ $k$.HANDLE-OK?$(i, x_i)$

**Figure 2.10:** The distributed breakout algorithm. Agents start by setting themselves to some color and then calling HANDLE-OK? on all their neighbors.

As such, the breakout algorithm maintains a weight associated with each constraint. These weights are initially set to one. Whenever an agent finds itself in a quasi-local-minimum it increases the weights of the constraints that are being violated. The weights in turn are used to calculate the cost of a constraint violation. That is, rather than simply counting the number of violated constraints, distributed breakout uses the weighted sum of the violated constraints. In this way, a constraint that was violated increases in weight so it becomes more important to satisfy that constraint in the next time step.

Distributed breakout implements two remote procedures.

- HANDLE-OK?$(i, x_i)$ where $i$ is the agent and $x_i$ is its current value,
- HANDLE-IMPROVE$(i,$ *improve*$)$ where *improve* is the maximum $i$ could gain by changing to some other color.

The full algorithm can be seen in figure 2.10. In it we can see that the agents spend their time either waiting to HANDLE-OK? or HANDLE-IMPROVE from all their neighbors. Once an agent gets all the HANDLE-OK? invocations it calculates its new cost and possible improvement and sends a HANDLE-IMPROVE to all its neighbors. Once an agent gets all the HANDLE-IMPROVE messages then the agent determines if it is in a quasi-local-minimum. If so then it increases the weights of the constraint violations and sends a HANDLE-OK? to its neighbors.

Figure 2.11 shows an example application of the distributed breakout algorithm. We start out with a random coloring for all the nodes as seen on the first picture on the left. Once the agents receive all the HANDLE-OK? from their neighbors they check to see how much they could improve by changing and send the HANDLE-IMPROVE messages. In this case, it turns out that for all agents *my-improve* $\leq 0$. Therefore, the agents increase weights to that shown on the second picture. At this point nodes 1, 2, 4 and 6 have a *my-improve* $= 1$ while 2, 5 are at $-3$. We break

DBgc



HANDLE-OK?*(j,* $x_j$*) 1 recebido -ok* $[j] \leftarrow$ TRUE*2 visualização do agente* $\leftarrow$ *visualização do agente* $+$ *(j,* $x_j$*) 3 se* $\forall_{k \in neighbors}$*recebido-ok* $[k]$ = TRUE*4 então* SEND-IMPROVE()5 $\forall_{k \in neighbors}$*recebido -ok* $[k] \leftarrow$ FALSE

SEND-IMPROVE()
1 novo valor $\leftarrow$ valor que proporciona melhoria máxima
2 my-improve $\leftarrow$ possível melhoria máxima
3 $\forall_{k \in neighbors}$ $k$.HANDLE-IMPROVE($i$, my-improve, custo)

HANDLE-IMPROVE*(j, melhorar)*
*1 recebeu -melhorar*$[j] \leftarrow$ *melhorar 2 se* $\forall_{k \in neighbors}$ *recebeu-melhorar*$[k] \neq$ NONE*3 então* SEND-OK*4 visualização do agente* $\leftarrow \emptyset$5 $\forall_{k \in neighbors}$ *recebeu -melhorar*$[k] \leftarrow$ NONE

SEND-OK()
1 se $\forall_{k \in neighbors}$ meu-melhorar $\geq$ recebeu -melhorar$[k]$2 então $x_i \leftarrow$ novo -valor 3 se custo $> 0 \wedge \forall_{k \in neighbors}$ recebeu-melhorar$[k] \leq 0 \triangleright$ ótimo quase local 4 então aumenta o peso das violações de restrição5 $\forall_{k \in neighbors}$ $k$.HANDLE-OK?$(i, x_i)$

Figura 2.10: O algoritmo de breakout distribuído. Os agentes começam definindo-se para alguma cor e então chamando HANDLE-OK? todos os seus vizinhos.

Como tal, o algoritmo de breakout mantém um peso associado a cada restrição. Esses pesos são inicialmente definidos como um. Sempre que um agente se encontra num mínimo quase local aumenta os pesos das restrições que estão a ser violadas. Os pesos, por sua vez, são usados para calcular o custo de uma violação de restrição. Ou seja, em vez de simplesmente contar o número de restrições violadas, o rompimento distribuído utiliza a soma ponderada das restrições violadas. Desta forma, uma restrição que foi violada aumenta de peso, tornando-se mais importante satisfazer essa restrição no próximo passo de tempo.

O breakout distribuído implementa dois procedimentos remotos.

- HANDLE-OK?$(i, x_i)$ onde $i$ é o agente e $x_i$ é seu valor atual,
- HANDLE-IMPROVE($i$,melhorar) onde melhorar é o máximo que $i$ poderia ganhar mudando para alguma outra cor.

O algoritmo completo pode ser visto na figura 2.10. Nele podemos ver que os agentes passam seu tempo esperando por HANDLE-OK? ou HANDLE-IMPROVE de todos os seus vizinhos. Depois que um agente recebe todas as HANDLE-OK? invocações, ele c alcula seu novo custo e possível melhoria e envia um HANDLE-IMPROVE a todos os seus vizinhos. Depois que um agente recebe todas as mensagens HANDLE-IMPROVE , o agente determina se está em um mínimo quase local. Se sim, então ele aumenta os pesos das violações de restrição e envia um HANDLE-OK? para seus vizinhos.

A Figura 2.11 mostra um exemplo de aplicação do algoritmo de breakout distribuído. Começamos com uma coloração aleatória para todos os nós, como pode ser visto na pri meira imagem à esquerda. Depois que os agentes recebem todas as HANDLE-OK? de seus vizinhos, eles verificam o quanto poderiam melhorar alterando e enviando as HANDLE-IMPROVE mensagens. Neste caso, verifica-se que para todos os agentes my-improve $\leq 0$. Portanto, os agentes aumentam os pesos até o valor mostrado na segunda figura. Neste ponto, os nós 1, 2, 4 e 6 têm my-improve $= 1$ enquanto 2, 5 estão em $-3$. Nós quebramos

DBgc

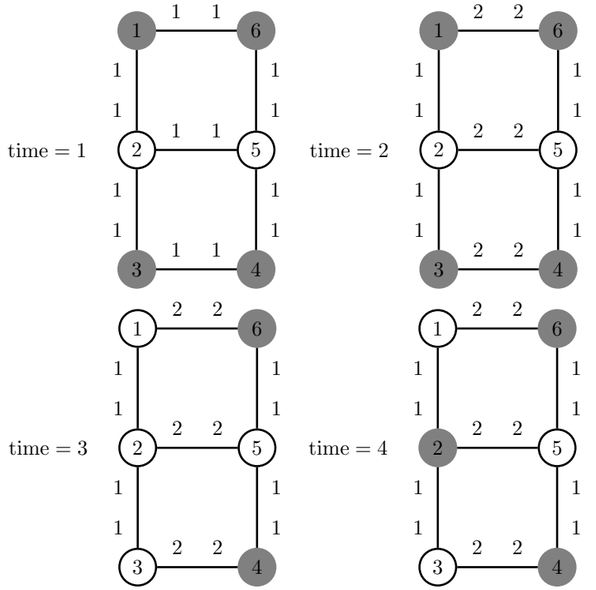



**Figure 2.11:** Example of distributed breakout. The only colors allowed are white and gray. The numbers represent the weights that the agents give to each of their constraints. We show four steps, starting at the top left.

ties using the agent number where smaller number wins. As such, agents 1 and 3 are the ones that get to change their color. They do so in the third picture. They then tell their neighbors their new color by calling HANDLE-OK? on them. At this point agent 2 has a *my-improve* $= 4$ (that is $1+1+2$) which is higher than anyone else's so it gets to change its color which moves us to the last picture.

The performance of distributed breakout is, on average, superior to AWC which is in turn superior to ABT. However, we should always remember that distributed breakout is not complete.

**Theorem 2.3** (Distributed Breakout is **not** Complete)**.** *Distributed breakout can get stuck in local minimum. Therefore, there are cases where a solution exists and it cannot find it.*

Distributed breakout will also not be able to identify that a particular problem lacks a solution. Specifically, the algorithm will either continue running or get stuck in a local minimum when given a problem that lacks a solution. As such, breakout cannot be used when there is any chance that the problem lacks a solution and you want to determine if the problem has or does not have a solution.

Tests have also shown that the load distribution of distributed breakout is uneven. That is, some agents end up handling several times more messages than other agents. There is likely a correlation between the number of edges an agent has and the number of messages it must process. Still, distributed breakout has been shown to, in practice, find the local optima a very large percentage of the time, and variations of the basic algorithm perform even better. Specifically, **Multi-DB$^{++}$** has been shown to, in practice, always find the solution for 3SAT problems (Hirayama and Yokoo, 2005).

MULTI-DB$^{++}$

## 2.2 Distributed Constraint Optimization

DISTRIBUTED CONSTRAINT OPTIMIZATION

The **distributed constraint optimization** problem is similar to the constraint satisfaction problem except that the constraints return a real number instead of a boolean value and the goal is to minimize the value of these constraint violations. The problem arises when multiple, perhaps all, solutions are valid but some are



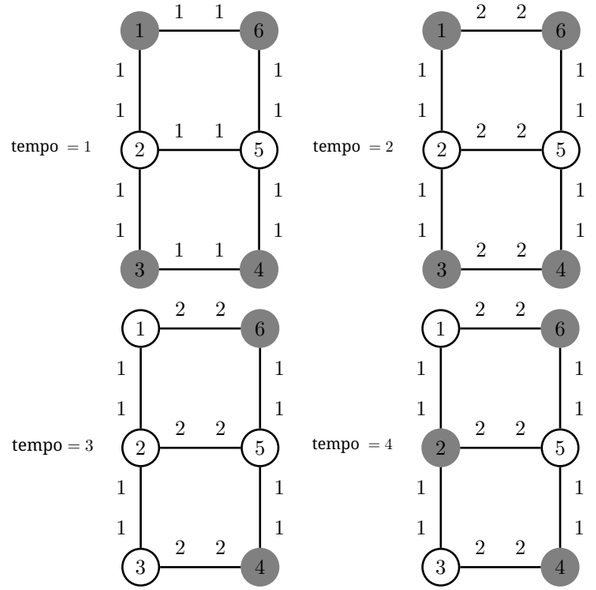



Figura 2.11: Exemplo de breakout distribuído. As únicas cores permitidas são branco e cinza. Os números r epresentam os pesos que os agentes atribuem a cada uma de suas restrições. Mostramos quatro passos, começando no canto superior esquerdo.

empata usando o número do agente onde o número menor vence. Assim, são os agentes 1 e 3 que mudam de cor. Eles fazem isso na terceira foto. Eles então contam aos vizinhos sua nova cor chamando-os HANDLE-OK? . Neste ponto, o agente 2 tem um my-improve $= 4$ (ou seja, $1+1+2$) que é maior do que o de qualquer outro, então ele muda de cor, o que nos leva para a última imagem.

O desempenho do breakout distribuído é, em média, superior ao AWC que por sua vez é superior ao ABT. No entanto, devemos sempre lembrar que o rompimento distribuído não é completo.

*Teorema 2.3 (O Breakout Distribuído não está Completo). O breakout distribuído pode ficar preso no mínimo local. Portanto, há casos em que existe uma solução e não é possível encontrá-la.*

O breakout distribuído também não será capaz de identificar que um problema específico não tem solução. Especificamente, o algoritmo continuará em execução ou ficará preso em um mínimo local quando for dado um problema que não tem solução. Como tal, o breakout não pode ser usado quando há alguma chance de o problema não ter solução e você deseja determinar se o problema tem ou não solução.

Os testes também mostraram que a distribuição de carga do breakout distribuído é desigual. Ou seja, alguns agentes acabam lidando com várias vezes mais mensagens do que outros agentes. Provavelmente existe uma correlação entre o número de arestas que um agente possui e o número de mensagens que ele deve processar. Ainda assim, foi demonstrado que o rompimento distribuído, na prática, encontra os ótimos locais em uma porcentagem muito grande do tempo, e variações do algoritmo básico têm desempenho ainda melhor. Especificamente, o Multi-DB$^{++}$ tem

demonstrou, na prática, sempre encontrar a solução para problemas 3SAT (Hirayama e Yokoo, 2005).

MULTI-DB$^{++}$

## 2.2 Otimização de Restrições Distribuídas

DISTRIBUTED CONSTRAINT OPTIMIZATION

O problema de otimização de restrição distribuída é semelhante ao problema de restrição problema de satisfação, exceto que as restrições retornam um número real em vez de um valor booleano e o objetivo é minimizar o valor dessas violações de restrição. O problema surge quando múltiplas soluções, talvez todas, são válidas, mas algumas são

```
BRANCH-AND-BOUND-COP()
1  c* ← ∞      ▷ Minimum cost found. Global variable.
2  g* ← ∅      ▷ Best solution found. Global variable.
3  BRANCH-AND-BOUND-COP-HELPER(1, ∅)
4  return g*

BRANCH-AND-BOUND-COP-HELPER(i, g)
1  if i = n
2     then if P(g) < c* ▷ Cost of violations in g is less than current bound.
3             then g* ← g
4                  c* ← P(g)
5          return
6  for v ∈ D_i
7       do g' ← g + {x_i ← v}
8          if P(g') < c*
9             then BRANCH-AND-BOUND-COP-HELPER(i + 1, g')
```

**Figure 2.12:** A centralized branch and bound algorithm for constraint optimization. It assumes there are $n$ variables, $x_1, \ldots, x_n$, and that $P(g)$ returns the total of any constraint violations given partial assignment $g$ and constraints $P$.

preferred to others. For example, say you are trying to schedule all your classes or weekly meetings and any schedule where no class overlaps another one is a valid schedule. However, you might prefer schedules with the least number of early morning classes.

We formally define a constraint optimization problem as follows:

**Definition 2.7** (Constraint Optimization Problem (COP))**.** *Given a set of variables $x_1, x_2, \ldots x_n$ with domains $D_1, D_2, \ldots D_n$ and a set of constraints $P$ of the form $pk(x_{k1}, x_{k2}, \ldots, x_{kj}) \rightarrow \Re$, find assignments for all the variables such that the sum of the constraint values is minimized.*

COP is an NP-Complete problem. That means that, at worse, all algorithms will take an exponential time (in the number of variables) to finish. Even parallel algorithms will, at worst, need exponential time. However, those are worst case scenarios. In practice we find some algorithms that do very well on most problems.

BRANCH AND BOUND

The most common approach to solving COPs is to use a **branch and bound** algorithm such as the one shown in figure 2.12. Branch and bound algorithms perform a depth first search of the variable assignments but also maintain a bound, $c^*$, which is the cost of the best solution found thus far. If a partial variable assignment already has a cost that is equal or higher than $c^*$ then we know that there is no need to find values for the rest of the variables as the cost will only increase. The algorithm is complete so it will find the best possible solution to the problem, but doing so might require a lot of time. Since it is based on a depth first search its memory requirements are linear on the number of variables.

The distributed constraint optimization problem distributes the variables to the agents.

**Definition 2.8** (Distributed Constraint Optimization Problem (DCOP))**.** *Give each agent one of the variables in a* COP*. Agents are responsible for finding a value for their variable and can find out the values of their neighbors' via communication*

We now look at some algorithms for solving the DCOP problem.

### 2.2.1 Adopt

ADOPT

The **Adopt** algorithm (Modi et al., 2005) is a recent addition to the family of DCOP algorithms. It is roughly a depth-first search on the set of possible value assignments, but with a lot of improvements on the basic search strategy. Namely, each agent in the depth-first tree keeps a lower and upper bound on the cost for the sub-problem below him (given assignments from above) and on the sub-problems



```
BRANCH-AND-BOUND-COP()
1  c* ← ∞      ▷ Custo mínimo encontrado. Variável gl
obal.2  g* ← ∅      ▷ Melhor solução encontrada. Variável
global.3  BRANCH-AND-BOUND-COP-HELPER(1, ∅) 4 retorno g*

BRANCH-AND-BOUND-COP-HELPER(i, g)
1 se  i = n2 então se  P(g)  < c* ▷ O custo das violações em  g é menor que o li
mite atual. 3 então  g* ← g4                    c* ← P(g) 5 retorne 6 para  v ∈ D_i7
faça  g' ← g + {x_i ← v}8 se  P(g')  < c*9 então  BRANCH-AND-BOUND-COP-HELPER(i + 1,
g')
```

Figura 2.12: Um algoritmo centralizado de ramificação e limite para otimização de restrições. Ele assume que existem $n$ variáveis, $x_1. .., x_n$, e que $P(g)$ retorna o total de quaisquer violações de restrição dada a atribuição parcial $g$ e as restrições $P$.

preferido a outros. Por exemplo, digamos que você esteja tentando agendar todas as suas aulas ou reuniões semanais e qualquer horário em que nenhuma aula se sobreponha a outra é um horário válido. No entanto, você pode preferir horários com o menor número de aulas matinais.

Definimos formalmente um problema de otimização de restrições da seguinte forma:

*Definição 2.7 (Problema de Otimização de Restrições (COP)). Dado um conjunto de variáveis$x_1, x_2, \ldots x_n$ com domínios $D_1, D_2, \ldots D_n$ e um conjunto de restrições $P$da forma$pk$ $(x_{k1}, x_{k2}, \ldots, x_{kj}) \rightarrow \Re$, encontre atribuições para todas as variáveis de modo que a soma dos valores de restrição seja minimizada.*

COP é um problema NP-Completo. Isso significa que, na pior das hipóteses, todos os algoritmos levarão um tempo exponencial (em número de variáveis) para terminar. Mesmo algoritmos paralelos precisarão, na pior das hipóteses, de tempo exponencial. No entanto, esses são os piores cenários. Na prática, encontramos alguns algoritmos que funcionam muito bem na maioria dos problemas.

BRANCH AND BOUND

A abordagem mais comum para resolver COPs é usar uma ramificação e limite algoritmo como o mostrado na figura 2.12. Algoritmos de ramificação e limite realizam uma primeira busca em profundidade nas atribuições de variáveis, mas também mantêm um limite, $c^*$, que é o custo da melhor solução encontrada até o momento. Se uma atribuição de variável parcial já tem um custo igual ou superior a $c^*$ então sabemos que não há necessidade de encontrar valores para o resto das variáveis, pois o custo só aumentará. O algoritmo está completo, portanto encontrará a melhor solução possível para o problema, mas isso pode exigir muito tempo. Como é baseado em uma primeira pesquisa em profundidade, seus requisitos de memória são lineares no número de variáveis.

O problema de otimização de restrições distribuídas distribui as variáveis aos agentes.

*Definição 2.8 (Problema de Otimização de Restrições Distribuídas (DCOP)). Dê a cada agente uma das variáveis em um* COP*. Os agentes são responsáveis por encontrar um valor para sua variável e podem descobrir os valores de seus vizinhos por meio de comunicação*

Agora veremos alguns algoritmos para resolver o problema DCOP .

### 2.2.1 Adotar

ADOPT

O algoritmo Adopt (Modi et al., 2005) é uma adição recente à família de DCOP algoritmos. É aproximadamente uma pesquisa em profundidade no conjunto de possíveis atribuições de valores, mas com muitas melhorias na estratégia de pesquisa básica. Ou seja, cada agente na árvore em profundidade mantém um limite inferior e superior no custo do subproblema abaixo dele (dadas as atribuições de cima) e nos subproblemas

```
RESET-VARIABLES(d, c)
1  lower-bound[d, c] ← 0
2  t[d, c] ← 0
3  upper-bound[d, c] ← ∞
4  context[d, c] ← {}

INITIALIZE()
1  threshold ← 0
2  received-terminate ← FALSE
3  current-context ← {}
4  ∀_{d∈D_i, c∈children} RESET-VARIABLES(d, c)
5  x_i ← d ∈ D_i which minimizes: my cost plus Σ_{c∈children} lower-bound[d, c]
6  BACKTRACK()

HANDLE-THRESHOLD(t, context)
1  if context is compatible with current-context
2     then threshold ← t
3          MAINTAIN-THRESHOLD-INVARIANT()
4          BACKTRACK()

HANDLE-TERMINATE(context)
1  received-terminate ← TRUE
2  current-context ← context
3  BACKTRACK()

HANDLE-VALUE(j, x_j)
1  if ¬ received-terminate
2     then current-context[j] ← x_j
3          for d ∈ D_i, c ∈ children such that context[d, c] is
                                        incompatible with current-context
4             do RESET-VARIABLES(d, c)
5          MAINTAIN-THRESHOLD-INVARIANT()
6          BACKTRACK()

HANDLE-COST(k, context, lb, ub)
 1  d ← context[i]
 2  delete context[i]
 3  if ¬ received-terminate
 4     then for (j, x_j) ∈ context and j is not my neighbor
 5             do current-context[j] ← x_j
 6          for d' ∈ D_i, c ∈ children such that context[d, c] is
                                        incompatible with current-context
 7             do RESET-VARIABLES(d', c)
 8          if context compatible with current-context
 9             then lower-bound[d, k] ← lb
10                  upper-bound[d, k] ← ub
11                  context[d, k] ← context
12                  MAINTAIN-CHILD-THRESHOLD-INVARIANT()
13                  MAINTAIN-THRESHOLD-INVARIANT()
14          BACKTRACK()
```

**Figure 2.13:** The Adopt algorithm. Before the algorithm is even called the agents must form a depth first search tree. Each agent has a *parent* variable which points to its parent. The root's parent is set as null.



```
RESET-VARIABLES(d, c)
1 limite inferior[d, c] ← 0
2  t[d, c] ← 0 3 limite
superior[d, c] ← ∞ 4
contexto[d, c] ← {}

INITIALIZE()
1 limite ← 0 2 recebimento-terminação ← FALSE 3 contexto atual ← {} 4 ∀_{d∈D_i,
c∈children} RESET-VARIABLES(d, c) 5 x_i ← d ∈ D_i que minimiza: meu custo mais
Σ_{c∈children} limite inferior[d, c] 6 BACKTRACK()

HANDLE-THRESHOLD(t, contexto)
1 se o contexto for compatível com o contexto atual
2 então limite ← t
3          MAINTAIN-THRESHOLD-INVARIANT()
4          BACKTRACK()

HANDLE-TERMINATE(contexto)
1 término recebido ← TRUE 2
contexto atual ←contexto
3  BACKTRACK()

HANDLE-VALUE(j, x_j)
1 se ¬ recebeu-terminação 2 então contexto atual[j] ← x_j 3 para d ∈ D_i, c ∈
filhos tais que o contexto[d, c] é incompatível com o contexto atual 4 faça
RESET-VARIABLES(d, c) 5          MAINTAIN-THRESHOLD-INVARIANT()
6          BACKTRACK()

HANDLE-COST(k , contexto , lb, ub)
 1  d ←contexto[i] 2 exclua o contexto[i] 3 se ¬ recebido-terminar 4 então
para (j, x_j) ∈ contexto e  j não é meu vizinho 5 faça o contexto atual[j] ← x_j 6
para d' ∈ D_i, c ∈ filhos de modo que o contexto[d, c] seja incompatível com o
contexto atual 7 faça RESET-VARIABLES(d', c) 8 se o contexto for compatível co
m o contexto atual 9 então limite inferior[d, k] ← lb 10 limite superior[d, k] ← ub
11 contexto[d, k] ← contexto
12                  MAINTAIN-CHILD-THRESHOLD-INVARIANT()
13                  MAINTAIN-THRESHOLD-INVARIANT() 14          BACKTRACK
()
```

Figura 2.13: O algoritmo Adopt. Antes mesmo de o algoritmo ser chamado, os agentes devem formar uma primeira árvore de pesquisa em profundidade. Cada agente possui uma variável *parent* que aponta para seu pai. O pai da raiz é definido como nulo.

```
BACKTRACK()
1  if threshold = min_{d∈D_i} cost(d) + Σ_{c∈children} upper-bound[d, c]
2     then x_i ← arg min_{d∈D_i} cost(d) + Σ_{c∈children} upper-bound[d, c]
3  elseif threshold < cost(x_i) + Σ_{c∈children} lower-bound[x_i, c]
4     then x_i ← arg min_{d∈D_i} cost(d) + Σ_{c∈children} lower-bound[d, c]
5  ∀_{k∈neighbors ∧k has lower priority} k.HANDLE-VALUE(i, x_i)
6  MAINTAIN-ALLOCATION-INVARIANT()
7  if threshold = min_{d∈D_i} cost(d) + Σ_{c∈children} upper-bound[d, c] and
            (received-terminate or I am root)
8     then current-context[i] ← x_i
9          ∀_{c∈children} c.HANDLE-TERMINATE(current-context)
10         exit
11 parent.HANDLE-COST(current-context,
            min_{d∈D_i} cost(d) + Σ_{c∈children} lower-bound[d, c],
            min_{d∈D_i} cost(d) + Σ_{c∈children} upper-bound[d, c])

MAINTAIN-THRESHOLD-INVARIANT()
1  if threshold < min_{d∈D_i} cost(d) + Σ_{c∈children} lower-bound[d, c]
2     then threshold ← min_{d∈D_i} cost(d) + Σ_{c∈children} lower-bound[d, c]
3  if threshold > min_{d∈D_i} cost(d) + Σ_{c∈children} upper-bound[d, c]
4     then threshold ← min_{d∈D_i} cost(d) + Σ_{c∈children} upper-bound[d, c]

MAINTAIN-ALLOCATION-INVARIANT()
1  while threshold > cost(x_i) + Σ_{c∈children} t[x_i, c]
2        do chosen ← c' ∈ children such that upper-bound[x_i, c'] > t[x_i, c']
3           t[x_i, chosen] ← t[x_i, chosen] + 1
4  while threshold < cost(x_i) + Σ_{c∈children} t[x_i, c]
5        do chosen ← c' ∈ children such that lower-bound[x_i, c'] < t[x_i, c']
6           t[x_i, chosen] ← t[x_i, chosen] − 1
7  ∀_{c∈children} c.HANDLE-THRESHOLD(t[x_i, chosen], current-context)

MAINTAIN-CHILD-THRESHOLD-INVARIANT()
1  for d ∈ D_i, c ∈ children
2        do if lower-bound[d, c] > t[d, c]
3              then t[d, c] ← lower-bound[d, c]
4  for d ∈ D_i, c ∈ children
5        do if upper-bound[d, c] < t[d, c]
6              then t[d, c] ← upper-bound[d, c]
```

**Figure 2.14:** The Adopt algorithm, continued.

for each one of his children. It then tells the children to look for a solution but ignore any partial solution whose cost is above the lower bound because it already knows that it can get that lower cost.

The general idea in Adopt is that each node in the tree calculates upper and lower bounds on the cost for all the constraints that involve all the nodes in the subtree rooted at that node, given an assignment for the nodes on the path to the root. Each node changes to the value (color) whose lower bound is smallest at each time and then asks its children to calculate new bounds given the new value. Upon receiving this message each node again changes its value to the one with the lowest lower bound and the process is repeated. Figures 2.13 and 2.14 show the full algorithm.

The basic workings of Adopt are best shown via an example. Table 2.2 shows the cost function we will be using in our example. Figure 2.15 shows a trace of an application of Adopt to a simple problem. As you can see, all the constraints are binary, as Adopt can only handle binary constraints. All the agents keep track of the



```
BACKTRACK()
1 se limite = min_{d∈D_i} custo(d) + Σ_{c∈children} limite superior[d, c]2 então
x_i ← arg min_{d∈D_i} custo(d) + Σ_{c∈children} limite superior[d, c]3 limite elseif <
custo(x_i) + Σ_{c∈children} limite inferior[x_i, c]4 então x_i ← arg min_{d∈D_i} custo(d)
+ Σ_{c∈children} limite inferior[d, c]5 ∀_{k∈neighbors ∧k} tem prioridade mais baixa
k.HANDLE-VALUE(i, x_i)6 MAINTAIN-ALLOCATION-INVARIANT() 7 se limite =
min_{d∈D_i} custo(d) + Σ_{c∈children} limite superior[d, c] e (recebido-terminação ou
sou root) 8 então contexto atual[i] ← x_i
9 ∀_{c∈children} c.HANDLE-TERMINATE(contexto atual) 10 saída 11 pai
.HANDLE-COST(contexto atual, custo mínimo_{d∈D_i}(d) + Σ_{c∈children} limite
inferior[d, c], custo mínimo_{d∈D_i}(d) + Σ_{c∈children} limite superior[d, c])

MAINTAIN-THRESHOLD-INVARIANT()
1 se limite < min_{d∈D_i} custo(d) + Σ_{c∈children} limite inferior[d, c]2 então limite ←
min_{d∈D_i} custo(d) + Σ_{c∈children} limite inferior[d, c]3 se limite > min_{d∈D_i} custo(d)
+ Σ_{c∈children} limite superior[d, c]4 então limite Custo ← min_{d∈D_i}(d) + Σ_{c∈children}
limite superior[d, c]

MAINTAIN-ALLOCATION-INVARIANT()
1 enquanto o limite > custo(x_i) + Σ_{c∈children} t[x_i, c]2 escolhe ← c' ∈ filhos de
modo que limite superior[x_i, c'] > t[x_i, c']3 t[x_i, escolhido] ← t[x_i,
escolhido] + 14 enquanto o limite < custo(x_i) + Σ_{c∈children} t[x_i, c]5 escolhe ← c' ∈
filhos de modo que inferior -bound[x_i, c'] < t[x_i, c']6 t[x_i, escolhido] ← t[x_i,
escolhido] − 17 ∀_{c∈children} c.HANDLE-THRESHOLD(t[x_i, escolhido], contexto atual)

MAINTAIN-CHILD-THRESHOLD-INVARIANT()
1 para d ∈ D_i, c ∈ filhos 2 fazem se limite inferior
[d, c] > t[d, c]3 então t[d, c] ← limite inferior[d, c]4
para d ∈ D_i, c ∈ filhos 5 fazem se limite superior
[d, c] < t[d, c]6 então t[d, c] ← limite superior[d, c]
```

Figura 2.14: O algoritmo Adopt, continuação.

para cada um de seus filhos. Em seguida, ele diz às crianças para procurarem uma solução, mas ignorarem qualquer solução parcial cujo custo esteja acima do limite inferior, porque já sabe que pode obter esse custo menor.

A ideia geral em Adopt é que cada nó na árvore calcule os limites superior e i nferior do custo para todas as restrições que envolvem todos os nós da subárvore enraizados naquele nó, dada uma atribuição para os nós no caminho para a raiz. Cada nó muda para o valor (cor) cujo limite inferior é o menor a cada vez e então pede a seus filhos que calculem novos limites dado o novo valor. Ao receber esta mensagem cada nó muda novamente seu valor para aquele com menor limite inferior e o processo é repetido. As Figuras 2.13 e 2.14 mostram o algoritmo completo.

O funcionamento básico do Adopt é melhor mostrado por meio de um exemplo. A Tabela 2.2 mostra a função de custo que usaremos em nosso exemplo. A Figura 2.15 mostra o traço de uma aplicação do Adopt a um problema simples. Como você pode ver, todas as restrições são binárias, pois o Adopt só pode lidar com restrições binárias. Todos os agentes acompanham o

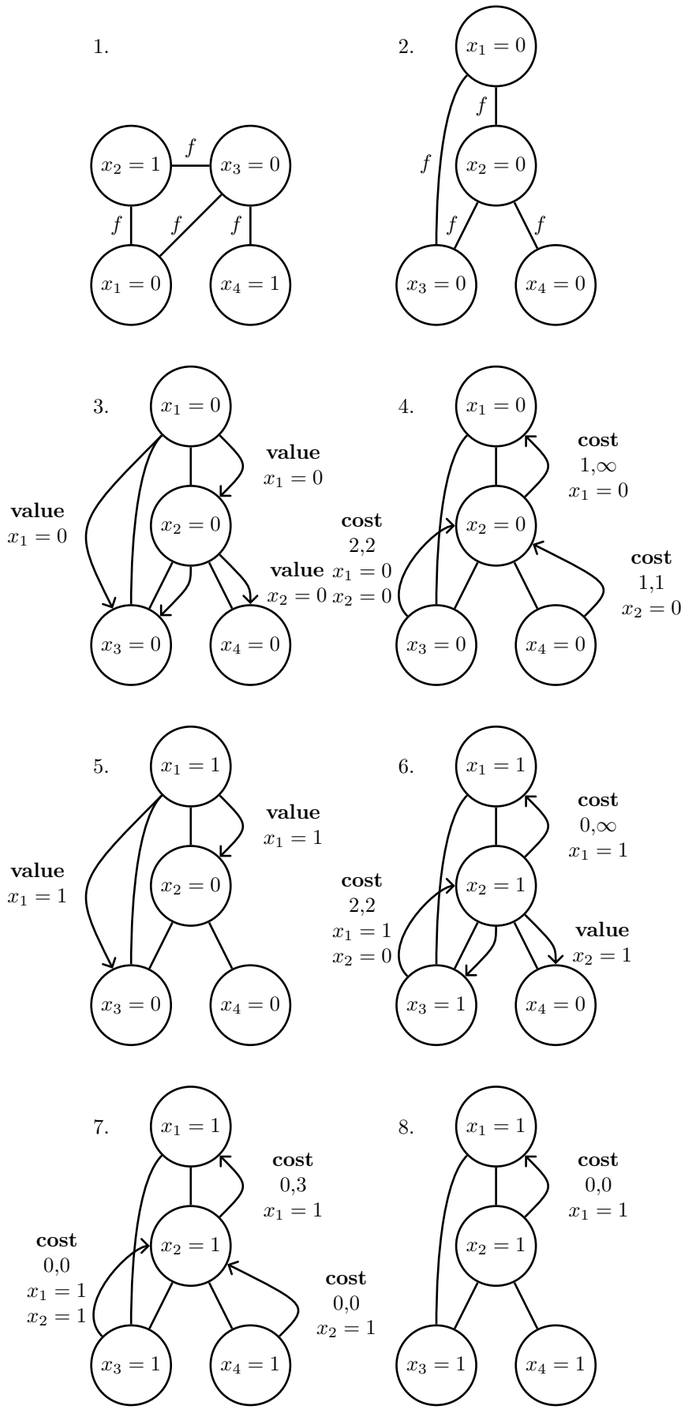



**Figure 2.15:** Adopt example. Start at the top left and read left to right. The domain for all nodes is the set $\{0, 1\}$.



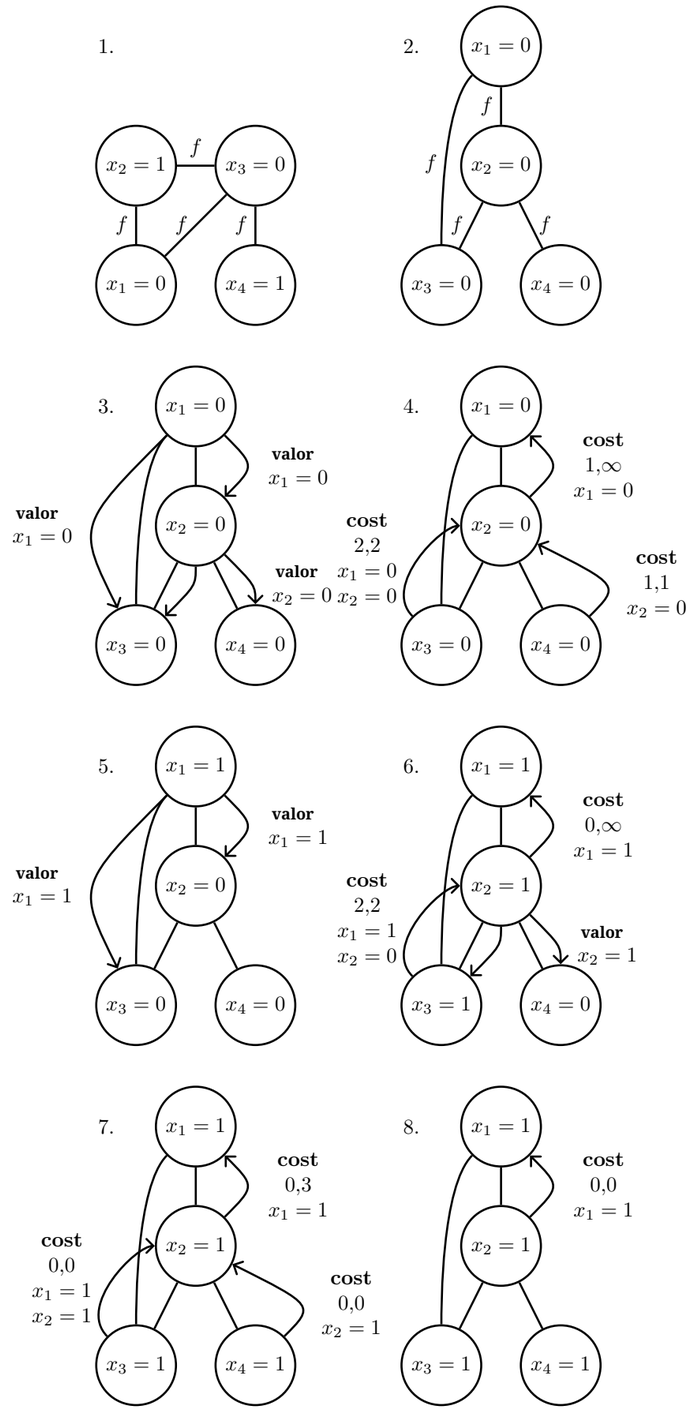



Figura 2.15: Exemplo de adoção. Comece no canto superior esquerdo e leia da esquerda para a direita. O domínio para todos os nós é o conjunto $\{0, 1\}$.

**Table 2.2:** Cost function for Adopt example in figure 2.15.

| $d_i$ | $d_j$ | $p(d_i, d_j)$ |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 2 |
| 1 | 0 | 2 |
| 1 | 1 | 0 |

upper and lower bounds sent by their children along with the associated context. They use these bounds to calculate bounds on their own costs. Initially, all lower bounds are set to 0 and all upper bounds to infinity.

The first diagram is the constraints graph itself. The first thing we must do before even starting Adopt is to form a depth-first search graph. That is, a graph such that all constraints emanating from a node go to either one of its descendants—the tree below it—or one of its ancestors—agents on the path from the node to the root. There exists several algorithms for finding such a tree and generally multiple trees can be formed from one constraints graph. The second diagram shows a possible depth-first search tree for our example.

The third diagram shows the first step in the algorithm. All the nodes invoke HANDLE-VALUE on all the descendants with whom they share a constraint. That is why $x_1$ sends a message to $x_3$ but not $x_4$. These HANDLE-VALUE are similar to the HANDLE-OK? functions we saw before. They simply inform the other agents of the new value, in this case the original values.



ADOPTgc

The agents use these values to calculate both upper and lower bounds on their cost and report these back up to their parents, as shown in the fourth diagram. For example, $x_3$ calculated that, given that $x_1 = 0$ and $x_2 = 0$ if it set itself to 0 the cost would be 2 and if set itself to 1 the cost would be 4, remember that we are using the cost function from Table 2.2, thus it stays at 0 and sends a HANDLE-COST to its parent saying that its lower and upper bounds, given $x_1 = 0$ and $x_2 = 0$, are 2. This message means that the cost of the constraints for the subtree rooted at $x_3$ (since $x_3$ is a leaf this then means just all the constraints in which $x_3$ is involved) given $x_1 = 0$ and $x_2 = 0$, are no lower or no higher than 2. $x_3$ can set both its upper and lower bounds because it has values for all its constraints. On the other hand, $x_2$ sends an upper bound of $\infty$ because it does not know the upper bounds of its children, so it does not know how much those costs could add up to.

In the fifth diagram the agents calculate new best values and so $x_1$ sends HANDLE-VALUE messages. In the sixth diagram $x_2$ receives the cost messages from before and sets himself to 1 because the value 1 has a lower lower bound (because all lower bounds are initially set to 0). $x_2$ then sends the appropriate HANDLE-VALUE messages.

In the seventh diagram both $x_3$ and $x_4$ calculate that both their upper and lower bounds are 0 given that $x_1 = 0$ and $x_2 = 0$ and send this message up to $x_2$. In the eighth diagram $x_2$ uses this message to calculate its new bounds, also 0 and 0, and sends these up to the root. Upon receiving this message the root knows it can stop the algorithm because it has identical upper and lower bounds. We note that the Adopt algorithm is actually a bit more complicated than this example shows as we ignored any HANDLE-THRESHOLD invocations.

Test results on Adopt show that there is a very wide discrepancy in the number of messages each agent must handle and, therefore, the individual workload. Namely, agents at the leafs of the tree end up doing all of the work while the root sits idle. This gets worse as the tree becomes thinner (like a line). Since the tree must be a depth-first search graph, this means that problems with a lot of constraints will be doubly bad because they will need a thinner tree and the extra constraints means more work for the agents.

Also, it must be noted that if each agent handles each message as it comes (and calls the BACKTRACK routine) then Adopt will never finish. For Adopt to work the agents need to accumulate a number of HANDLE-VALUE and HANDLE-COST



Tabela 2.2: Função de custo para o exemplo Adotar na figura 2.15.

| $d_i$ | $d_j$ | $p(d_i, d_j)$ |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 2 |
| 1 | 0 | 2 |
| 1 | 1 | 0 |

limites superior e inferior enviados por seus filhos junto com o contexto associado. Eles usam esses limites para calcular os limites de seus próprios custos. Inicialmente, todos os limites inferiores são definidos como 0 e todos os limites superiores como infinito.

O primeiro diagrama é o próprio gráfico de restrições. A primeira coisa que devemos fazer antes mesmo de iniciar o Adopt é formar um gráfico de pesquisa em profundidade. Ou seja, um grafo tal que todas as restrições emanadas de um nó vão para um de seus descendentes – a árvore abaixo dele – ou para um de seus ancestrais – agentes no caminho do nó até a raiz. Existem vários algoritmos para encontrar tal árvore e geralmente várias árvores podem ser formadas a partir de um gráfico de restrições. O segundo diagrama mostra uma possível árvore de pesquisa em profundidade para nosso exemplo.

O terceiro diagrama mostra a primeira etapa do algoritmo. Todos os nós invocam HANDLE-VALUE todos os descendentes com quem compartilham uma restrição. É por isso que $x_1$ envia uma mensagem para $x_3$ mas não para $x_4$. Essas HANDLE-VALUE são semelhantes àsHANDLE-OK? funções que vimos antes. Eles simplesmente informam aos demais agentes o novo valor, neste caso os valores originais.



ADOTARgc

Os agentes usam esses valores para calcular os limites superior e inferior de seus custos e reportá-los aos seus pais, conforme mostrado no quarto diagrama. Por exemplo, $x_3$ calculou que, dado que $x_1 = 0$ e $x_2 = 0$ se fosse definido como 0 o c usto seria 2 e se fosse definido como 1 o custo seria 4, lembre-se que estamos usando a função de custo da Tabela 2.2, portanto ela permanece em 0 e envia um HANDLE-COST para seu pai dizendo que seus limites inferior e superior, dados $x_1 = 0$ e $x_2 = 0$, são 2. Esta mensagem significa que o custo das restrições para a subárvore com raiz em $x_3$ (já que $x_3$ é uma folha, isso significa apenas todas as restrições nas quais $x_3$ está envolvido), dadas $x_1 = 0$ e $x_2 = 0$, não são inferiores nem superiores a 2. $x_3$ pode definir seus limites superior e inferior porque possui valores para todas as suas restrições. Por outro lado,$x_2$ envia um limite superior de $\infty$ porque não conhece os limites superiores de seus filhos, portanto, não sabe quanto esses custos poderiam atingir.

No quinto diagrama, os agentes calculam novos melhores valores e assim $x_1$ enviamHANDLE-VALUE mensagens. No sexto diagrama, $x_2$ recebe as mensagens de custo anteriores e se define como 1 porque o valor 1 tem um limite inferior (porque todos os limites inferiores são inicialmente definidos como 0). $x_2$ envia as mensagens HANDLE-VALUEapropriadas.

No sétimo diagrama, $x_3$ e $x_4$ calculam que seus limites superior e inferior são 0, dado que $x_1 = 0$ e $x_2 = 0$ , e enviam esta mensagem até $x_2$. No oitavo diagrama $x_2$ usa esta mensagem para calcular seus novos limites, também 0 e 0, e os envia para a raiz. Ao receber esta mensagem, a raiz sabe que pode parar o algoritmo porque possui limites superiores e inferiores idênticos. Observamos que o algoritmo Adopt é na verdade um pouco mais complicado do que este exemplo mostra, pois ignoramos quaisquer invocações HANDLE-THRESHOLD .

Os resultados dos testes no Adopt mostram que há uma discrepância muito grande no número de mensagens que cada agente deve tratar e, portanto, na carga de trabalho individual. Ou seja, os agentes nas folhas da árvore acabam fazendo todo o trabalho enquanto a raiz fica ociosa. Isso piora à medida que a árvore fica mais fina (como uma linha). Como a árvore deve ser um grafo de busca em profundidade, isso significa que problemas com muitas restrições serão duplamente ruins porque precisarão de uma árvore mais fina e as restrições extras significam mais trabalho para os agentes.

Além disso, deve-se observar que se cada agente tratar cada mensagem conforme ela chega (e chamar a rotina BACKTRACK), o Adopt nunca terminará. Para que o Adopt funcione, os agentes precisam acumular um número de HANDLE-VALUE e HANDLE-COST

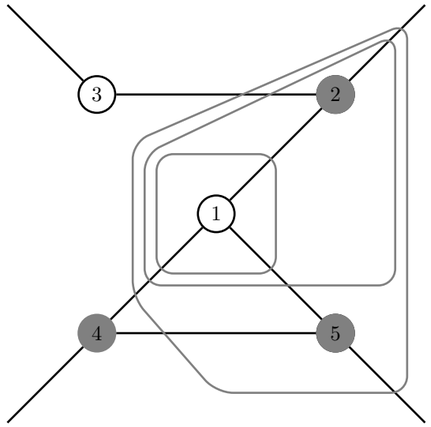



**Figure 2.16:** Trace of APO algorithm showing the growing size of node 1's *good-list*. Initially only 1 is in the *good-list*, then 2 joins, finally 5 joins.

invocations before running their BACKTRACK procedure.

### 2.2.2 OptAPO

A different approach to solving the distributed constraint optimization problem was taken by the Optimizing Asynchronous Partial Overlay (**optapo**) algorithm (Mailler and Lesser, 2004a), which is the cousin of the **apo** algorithm (Mailler and Lesser, 2004b) for distributed constraint satisfaction.

OPTAPO
APO

Both of these algorithms use the same basic idea. The agents start out doing individual hill-climbing, that is, each one changes its color so as to minimize constraints violations with its neighbors. When an agent realizes that it cannot find a value with no violations it tries to find a coloring for a (growing) set of agents which includes itself. More specifically, each agent maintains a variable called its *good-list* which initially contains only the agent. As the agent finds conflicts with other agents that it cannot resolve it adds these agents to its *good-list*. Then, the conflict is resolved by electing one of the agents in the conflict to be the *mediator* of the conflict. The mediator agent then finds a coloring for all the agents in its *good-list* and which minimizes violations with agents outside the *good-list* using any centralized algorithm, such as the branch and bound algorithm from figure 2.12. The agent then informs the other agents of their new color. The agents change to their new color as instructed by the mediator.

Figure 2.16 gives an idea of how APO works. Initially, node 1 has only itself in its *good-list*. If it finds that it cannot resolve a conflict with agent 2 then they enter into a negotiation phase to elect a mediator. In this case 1 becomes the mediator and then finds colors for both 1 and 2. Later on another conflict between 1 and 3 might arise which would lead to 1 becoming mediator for 1,2,3. Notice that if we give optAPO a graph coloring problem that has no solution, that is, there is always at least one conflict then it will eventually end up performing a complete depth first search of the complete problem in one node. This will be extremely time consuming.

In practice, APO and optAPO can be very fast, but their theoretical worst-case bound is still exponential.

**Theorem 2.4** (APO worst case is centralized search)**.** *In the worst case* APO *(opt*APO*) will make one agent do a completely centralized search of the complete problem space.*

optAPO requires fewer number of synchronous cycles (that is, steps) than Adopt but performs a lot more computations as measured by number of constraints checks (Davin and Modi, 2005). That is, optAPO centralizes the problem much more than Adopt. When an agent becomes a mediator it centrally solves a large part of the



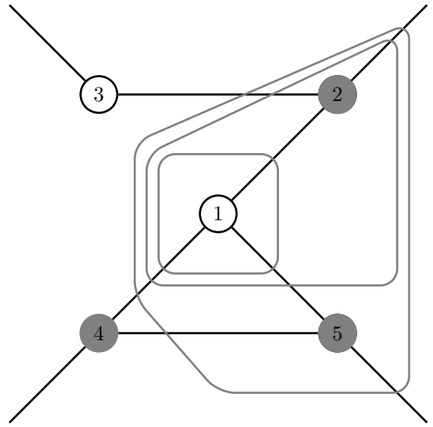



Figura 2.16: Traço do algoritmo APO mostrando o tamanho crescente do nó 1 *good-list*. Inicialmente apenas 1 está em *good-list*, depois 2 j unções, finalmente 5 junções.

invocações antes de executar seu BACKTRACK procedimento.

### 2.2.2 OpAPO

Uma abordagem diferente para resolver o problema de otimização de restrições distribuídas foi adotada pelo algoritmo Optimizing Asynchronous PartialOverlay (optapo)

OPTAPO
APO

(Mailler e Lesser, 2004a), que é primo do algoritmo apo (Mailler e Lesser, 2004b) para satisfação de restrições distribuídas.

Ambos os algoritmos usam a mesma ideia básica. Os agentes iniciam fazendo escaladas individuais, ou seja, cada um muda de cor para minimizar violações de restrições com seus vizinhos. Quando um agente percebe que não consegue encontrar um valor sem violações, ele tenta encontrar uma coloração para um conjunto (crescente) de agentes que o inclua. Mais especificamente, cada agente mantém uma variável chamada lista de produtos que inicialmente contém apenas o agente. À medida que o agente encontra conflitos com outros agentes que não consegue resolver, ele adiciona esses agentes à sua lista de bons. Então, o conflito é resolvido elegendo um dos agentes do conflito para ser o mediador do conflito. O agente mediador então encontra uma coloração para todos os agentes em sua lista de bens e que minimiza as violações com agentes fora da lista de bens usando qualquer algoritmo centralizado, como o algoritmo de ramificação e limite da figura 2.12. O agente então informa aos outros agentes sua nova cor. Os agentes mudam para a nova cor conforme instruções do mediador.

A Figura 2.16 dá uma ideia de como APO funciona. Inicialmente, o nó 1 possui apenas ele mesmo em sua lista de produtos bons. Se descobrir que não consegue resolver um conflito com o agente 2, entra numa fase de negociação para eleger um mediador. Neste caso, 1 torna-se o mediador e então encontra cores para 1 e 2. Mais tarde, pode surgir outro conflito entre 1 e 3, o que levaria 1 a tornar-se mediador de 1,2,3. Observe que se dermos optAPO um problema de coloração de gráfico que não tem solução, ou seja, sempre há pelo menos um conflito, então ele acabará realizando uma primeira pesquisa em profundidade completa do problema completo em um nó. Isso consumirá muito tempo.

Na prática, APO e optAPO podem ser muito rápidos, mas seu limite teórico de pior caso ainda é exponencial.

*Teorema 2.4 (*APO *o pior caso é a pesquisa centralizada). Na pior das hipóteses,* APO*(opt* APO*) fará com que um agente faça uma pesquisa completamente centralizada em todo o espaço do problema.*

optAPO requer menos ciclos síncronos (ou seja, etapas) do que Adopt, mas executa muito mais cálculos medidos pelo número de verificações de restrições (Davin e Modi, 2005). Ou seja, optAPO centraliza o problema muito mais do que Adopt. Quando um agente se torna u m mediador, ele resolve centralmente uma grande parte do problema.

problem and tells the other agents what to do. As such, that one agent ends up doing a lot of work while everyone else does nothing (generally there can be more than one mediator, each mediating over non-overlapping sets of agents, but this is rare). In this way optAPO can finish in a much smaller number of steps simply because some of those steps take a long time.

In general we find that Adopt is better when agent to agent communications are fast and optAPOis better when communications are slow in comparison to the agent's processing speed. Both of them have exponential worst-case running times but perform reasonable well for small problems.

## Exercises


DBsodoku

2.1 In both ABT and AWC the load distribution among the agents is very uneven, as demonstrated in our NetLogo implementations. Can you come up with an equation that predicts how many messages, proportionally, each agent receives given the initial problem structure and priorities?

2.2 The Sodoku puzzle is an instance of a constraint satisfaction problem. We can view it as a distributed constraint satisfaction problem by simply assuming that each empty square on the board corresponds to an agent. Implement any one of the distributed constraint satisfaction algorithms in this chapter to solve Sodoku puzzles.

2.3 Provide an example which proves Theorem 2.3.

2.4 Provide an example which proves Theorem 2.4.

2.5 All these algorithms assume a static constraint graph. Unfortunately, many real world applications are dynamic and can best be represented by a series of constraint satisfaction problems each of which is a little bit different from the previous one. In these applications it seems wasteful to re-start the algorithm from the beginning when the new problem is only slightly different from the old one.

Implement one of the constraint satisfactions algorithms for graph coloring and run tests to determine how it behaves as the graph changes. For example,

1. Start with a randomly generated graph and an initial coloring.
2. Run the algorithm starting with he current coloring.
3. Change the graph by adding or deleting a random node.
4. Goto step 2.

Does the algorithm take the same time in the first step as in all the other steps? What type of changes to the graph make the most difference in the run time?



problema e diz aos outros agentes o que fazer. Assim, um agente acaba fazendo muito trabalho enquanto todos os outros não fazem nada (geralmente pode haver mais de u m mediador, cada um mediando conjuntos de agentes não sobrepostos, mas isso é r aro). Dessa forma, optAPO pode terminar em um número muito menor de etapas simplesmente porque algumas dessas etapas demoram muito tempo.

Em geral, descobrimos que adotar é melhor quando as comunicações entre agentes são rápidas e optarAPOé melhor quando as comunicações são lentas em comparação com a velocidade de processamento do agente. Ambos têm tempos de execução exponenciais no pior caso, mas apresentam um desempenho razoável para pequenos problemas.

### Exercícios


DBsodoku

2.1 Tanto em ABT quanto em AWC a distribuição de carga entre os agentes é muito desigual, conforme demonstrado em nossas implementações NetLogo. Você consegue criar uma equação que preveja quantas mensagens, proporcionalmente, cada agente recebe, dada a estrutura e as prioridades iniciais do problema?

2.2 O quebra-cabeça Sodoku é um exemplo de problema de satisfação de restrições. Podemos vê-lo como um problema distribuído de satisfação de restrições simplesmente assumindo que cada quadrado vazio no tabuleiro corresponde a um agente. Implementar

qualquer um dos algoritmos distribuídos de satisfação de restrições neste capítulo para resolver quebra-cabeças Sodoku.

2.3 Forneça um exemplo que prove o Teorema 2.3.

2.4 Forneça um exemplo que prove o Teorema 2.4.

2.5 Todos esses algoritmos assumem um gráfico de restrição estático. Infelizmente, muitas aplicações do mundo real são dinâmicas e podem ser melhor representadas por uma série de problemas de satisfação de restrições, cada um dos quais é um pouco diferente do anterior. Nestas aplicações parece um desperdício reiniciar o algoritmo desde o início quando o novo problema é apenas ligeiramente diferente do antigo.

Implemente um dos algoritmos de satisfação de restrições para coloração de grafos e execute testes para determinar como ele se comporta conforme o grafo muda. Por exemplo,

1. Comece com um gráfico gerado aleatoriamente e uma coloração inicial. 2. Execute o algoritmo começando com a coloração atual. 3. Altere o gráfico adicionando ou excluindo um nó aleatório. 4. Vá para a etapa 2.

O algoritmo leva o mesmo tempo na primeira etapa e em todas as outras etapas? Que tipo de alterações no gráfico fazem mais diferença no tempo de execução?

Chapter 3

# Standard and Extended Form Games



John Von Neumann. 1903–1957. Pioneer of the digital computer, game theory and cellular automata.

GAME THEORY

In all multiagent systems we have a set of autonomous agents each performing its own actions using whatever information is has available. Since the other agents are also taking actions, each agent must also take these into account when deciding what to do. Thus, what one does depends on what the other one does and vice-versa. The agent must decide what to do when their choice of action depends on the others' choices. These types of problems are very common in different fields, from Economics to Biology, and their solution is sometimes of immense importance. As such, a set of mathematical tools has been developed over the years to model and solve these problems. It is known as **game theory** and is the subject of this and several other chapters in this book.

Game theory was first formally introduced in the book *"The Theory of Games and Economic Behavior"* (Neumann and Morgenstern, 1944). The book introduced a mathematical way of analyzing certain types of decisions where two or more parties take actions that impact all. In this chapter we present the two most basic forms of games: normal and extended form games. These games are known as **non-cooperative games** because the agents' preferred sets of actions can be in conflict with each other. That is, what is good for one agent might be bad for the others. Of course, the fact that they can be in conflict does not mean that they have to be, thus, non-cooperative games can lead to cooperation.

NON-COOPERATIVE GAMES

## 3.1 Games in Normal Form

In the simplest type of game we have two agents each of which must take one of two possible actions. The agents take their actions at the same time. They will then each receive a utility value, or payoff, based on their joint actions. Games such as this one can be represented using a **payoff matrix** which shows the utility the agents will receive given their actions. Figure 3.1 shows a sample game matrix in **normal form**, also known as **strategic form**, a phrase introduced by Shapley in 1965. In this game if Bob takes action $a$ and Alice takes action $c$ then Bob will receive a utility of 1 and Alice a utility of 2. We can extend the payoff matrix to any number of players and actions. In these games we always assume that the players take their actions simultaneously.

PAYOFF MATRIX

NORMAL FORM
STRATEGIC FORM

Normal form games also assume that the players have **common knowledge** of the utilities that all players can receive. That is, everybody knows that everybody knows that everybody knows, and so on, the values in the payoff matrix. This situation is different from having the agents know the values in the matrix but not know that the others know those values. For example, if you know that I am giving you a surprise party then you might go along and act surprised when we all jump

COMMON KNOWLEDGE

(Fagin et al., 1995) describes a logic for representing agents' knowledge about others' knowledge.

| | | Alice | |
|---|---|---|---|
| | | $c$ | $d$ |
| Bob | $a$ | 1,2 | 4,3 |
| | $b$ | 3,2 | 2,4 |

**Figure 3.1:** Sample game matrix in normal form.



Capítulo 3

# Jogos de formato padrão e estendido



John VonNeumann.1903–1957. Pioneiro do computador digital, teoria dos jogos e autômatos celulares.

GAME THEORY

Em todos os sistemas multiagentes temos um conjunto de agentes autônomos, cada um executando suas próprias ações usando qualquer informação disponível. Como os outros agentes também realizam ações, cada agente também deve levar isso em consideração ao decidir o que fazer. Assim, o que um faz depende do que o outro faz e vice-versa. O agente deve decidir o que fazer quando sua escolha de ação depende das escolhas dos outros. Este tipo de problemas são muito comuns em diversas áreas, da Economia à Biologia, e a sua solução é por vezes de imensa importância. Como tal, um conjunto de ferramentas matemáticas foi desenvolvido ao longo dos anos para modelar e resolver estes problemas. É conhecida como teoria dos jogos e é o tema deste e

vários outros capítulos deste livro.

A teoria dos jogos foi formalmente introduzida pela primeira vez no livro "A Teoria dos Jogos e Comportamento Econômico" (Neumann e Morgenstern, 1944). O livro introduziu uma forma matemática de analisar certos tipos de decisões em que duas ou mais partes tomam medidas que impactam a todos. Neste capítulo apresentamos as duas formas mais básicas de jogos: jogos de forma normal e estendida. Esses jogos são conhecidos como jogos não cooperativos porque os conjuntos de ações preferidos dos agentes podem estar em conflito.

NON-COOPERATIVE GAMES

um com o outro. Ou seja, o que é bom para um agente pode ser ruim para os outros. É claro que o fato de poderem estar em conflito não significa que devam estar, portanto, jogos não cooperativos podem levar à cooperação.

## 3.1 Jogos em Forma Normal

No tipo mais simples de jogo temos dois agentes, cada um dos quais deve realizar uma de duas ações possíveis. Os agentes realizam suas ações ao mesmo tempo. Cada um receberá um valor de utilidade, ou recompensa, com base em suas ações conjuntas. Jogos como este podem ser representados usando uma matriz de recompensas que mostra a utilidade que os agentes

PAYOFF MATRIX

receberão dadas as suas ações. A Figura 3.1 mostra um exemplo de matriz de jogo na forma normal, também conhecida como forma estratégica, frase introduzida por Shapley em 1965.

NORMAL FORM
STRATEGIC FORM

Neste jogo, se Bob agir $a$ e Alice agir $c$ então Bob receberá uma utilidade de 1 e Alice uma utilidade de 2. Podemos estender a matriz de recompensa para qualquer número de jogadores e ações. Nestes jogos assumimos sempre que os jogadores realizam as suas ações simultaneamente.

Os jogos de forma normal também assumem que os jogadores têm conhecimento comum os utilitários que todos os jogadores podem receber. Ou seja, todo mundo sabe que todo mundo sabe que todo mundo conhece, e assim por diante, os valores da matriz de retorno. Esse

A situação é diferente de fazer com que os agentes conheçam os valores da matriz, mas não saibam que os outros conhecem esses valores. Por exemplo, se você sabe que estou lhe dando uma festa surpresa, então você pode ir junto e agir como surpreso quando todos pularmos.

COMMON KNOWLEDGE

(Fagin et al., 1995) descreve u ma lógica para representar o conhecimento dos agentes sobre o conhecimento dos outros.

| | | Alice | |
|---|---|---|---|
| | | $c$ | $d$ |
| Bob | $a$ | 1,2 | 4,3 |
| | $b$ | 3,2 | 2,4 |

Figura 3.1: Exemplo de matriz de jogo na forma normal.

and yell "surprise!". However, if you know that I know that you know that I will be giving you a surprise party then the deception will no longer work.

It is also interesting to note that in message-passing multiagent systems where messages can be lost it is impossible for agents to ever achieve common knowledge about anything (Fagin et al., 1995). The problem is historically described as the **Byzantine generals problem** where two generals from the same army are poised on opposite sides of a valley which is occupied by the enemy. The generals must both attack at the same time in order to defeat the enemy. However, their only method of communication is by sending a messenger who could be captured by the enemy. We can see that, if one general sends a message to the other saying "We attack at dawn" it has no way of confirming whether the other general received this message. Similarly, if a general does receive the message and sends another messenger back acknowledging receipt then it has no way of confirming whether the other general received the acknowledgment. Thus, it is impossible to agree on a time. In practice, however, multiagent systems that need common knowledge either give their agents this common knowledge from the beginning or assume that communications are reliable enough that it is safe to assume that all messages are delivered.

BYZANTINE GENERALS PROBLEM

Getting back to the normal form game, we define a **strategy** $s$ to be the set of actions all players take. In this case a strategy of $s = (a, c)$ would give Bob a utility of 1 and Alice a utility of 2, in other words, $u_{\text{Bob}}(s) = 1$ and $u_{\text{Alice}}(s) = 2$. We also refer to Bob's strategy in $s$ as $s_{\text{Bob}}$, which is $a$ in this case. This strategy is also an example of a **pure strategy**: one where the agents take a specific action. In contrast, a **mixed strategy** is one where the agents take different actions, each with some fixed probabilities. For example, a mixed strategy for Bob is to take action $a$ with probability of .3 and action $b$ with a probability of .7. Note that in a mixed strategy the probabilities for all actions of each agent have to add up to 1. Typically, game theory further assumes that players are **rational**, which we use as a synonym for selfish. That is, a rational player always acts so as to maximize its utility.

STRATEGY

PURE STRATEGY
MIXED STRATEGY

RATIONAL

A special type of game are those in which the values in every box of the matrix add up to zero. These games are known as **zero-sum** games and represent scenarios where one agent's gain must come at a loss to he other agents. For example, in a zero-sum game with two players if for a particular strategy $s$ one player gets a utility of 5 then the other player must receive a utility of $-5$. In these games cooperation is unlikely. Note that every competitive sport is a zero-sum game as the fact that one team wins means the other must lose. Luckily, real-world problems are rarely zero-sum, even if our tendency is often to perceive them as such.

ZERO-SUM

### 3.1.1 Solution Concepts

Given a game matrix we can't help but ask: what strategy should they use? Which is best the best strategy in any given game? The problem, of course, is that there is no simple way to define what's best since what is best for one agent might not be good for another. As such, different solution concepts have been proposed.

The earliest solution concepts were proposed by Von Neumann. He realized that in any given game an agent could always take the action which maximized the worst possible utility it could get. This is known as the **maxmin strategy**, or minmax, if we are talking about losses instead of gains. Specifically, in a game with two agents, $i$ and $j$, agent $i$'s maxmin strategy is given by

MAXMIN STRATEGY

$$s_i^* = \max_{s_i} \min_{s_j} u_i(s_i, s_j). \tag{3.1}$$

That is, $i$ assumes that no matter what it does $j$ will take the action that is worst for $i$. Thus, $i$ takes its best possible action given this assumption. Unfortunately, the strategy where both players play their maxmin strategy might not be stable in the general case. In some payoff matrices it can happen that if $i$ knows that $j$ will play its maxmin strategy then $i$ will prefer a strategy different from its maxmin strategy. For example, in figure 3.1 the maxmin strategy is $(b, d)$ but if Alice plays $d$ then



e grite "surpresa!". No entanto, se você sabe que eu sei que você sabe que vou lhe dar uma festa surpresa, então o engano não funcionará mais.

Também é interessante notar que em sistemas multiagentes de passagem de mensagens, onde as mensagens podem ser perdidas, é impossível para os agentes alcançarem conhecimento comum sobre qualquer coisa (Fagin et al., 1995). O problema é historicamente descrito como o problema dos generais bizantinos, onde dois generais do mesmo exército estão posicionados

BYZANTINE GENERALS PROBLEM

em lados opostos de um vale ocupado pelo inimigo. Os generais devem atacar ao mesmo tempo para derrotar o inimigo. No entanto, o seu único método de comunicação é através do envio de um mensageiro que poderia ser capturado pelo inimigo. Podemos perceber que, se um general envia uma mensagem ao outro dizendo "Atacamos de madrugada" não tem co mo confirmar se o outro general recebeu esta mensagem. Da mesma forma, se um general receber a mensagem e enviar outro mensageiro de volta confirmando o recebimento, não haverá como confirmar se o outro general recebeu a confirmação. Assim, é impossível c hegar a um acordo sobre um horário. Na prática, contudo, os sistemas multiagentes que necessitam de conhecimento comum ou fornecem aos seus agentes esse conhecimento comum desde o início ou assumem que as comunicações são suficientemente fiáveis para que seja seguro assumir que todas as mensagens são entregues.

Voltando ao jogo de forma normal, definimos uma estratégia $s$ para ser o conjunto de ações que todos os jogadores realizam. Neste caso, uma estratégia de $s = (a, c)$ daria a Bob uma utilidade de 1 e a Alice uma utilidade de 2, em outras palavras, $u_{\text{Bob}}(s) = 1$ e $u_{\text{Alice}}(s) = 2$. Também nos referimos à estratégia de Bob em $s$ como $s_{\text{Bob}}$, que é $a$ neste caso. Esta estratégia é também um exemplo de estratégia pura: aquela em que os agentes realizam uma ação específica.

STRATEGY

PURE STRATEGY
MIXED STRATEGY

Em contraste, uma estratégia mista é aquela em que os agentes realizam ações diferentes, cada uma com algumas probabilidades fixas. Por exemplo, uma estratégia mista para Bob é realizar uma ação $a$ com probabilidade de 0,3 e uma ação $b$ com uma probabilidade de 0,7. Observe que em uma estratégia mista as probabilidades de todas as ações de cada agente devem somar 1. Normalmente, a teoria dos jogos assume ainda que os jogadores são racionais, o que usamos como

RATIONAL

sinônimo de egoísta. Ou seja, um jogador racional sempre age de forma a maximizar a sua utilidade.

Um tipo especial de jogo são aqueles em que os valores em cada caixa da matriz somam zero. Esses jogos são conhecidos como jogos de soma zero e representam cenários

ZERO-SUM

onde o ganho de um agente deve ser prejudicado pelos outros agentes. Por exemplo, em um jogo de soma zero com dois jogadores, se com uma estratégia específica $s$ um jogador obtiver uma utilidade de 5, então o outro jogador deverá receber uma utilidade de $-5$. Nestes jogos a cooperação é improvável. Observe que todo esporte competitivo é um jogo de soma zero, pois o fato de um time vencer significa que o outro deve perder. Felizmente, os problemas do mundo real raramente são de soma zero, mesmo que a nossa tendência seja frequentemente percebê-los como tal.

### 3.1.1 Conceitos de Solução

Dada uma matriz de jogo não podemos deixar de perguntar: qual estratégia eles deveriam usar? Qual é a melhor estratégia em qualquer jogo? O problema, claro, é que não existe uma forma simples de definir o que é melhor, uma vez que o que é melhor para um agente pode não ser bom para outro. Como tal, diferentes conceitos de solução foram propostos.

Os primeiros conceitos de solução foram propostos por Von Neumann. Ele percebeu que, em qualquer jogo, um agente sempre poderia realizar a ação que maximizasse a pior utilidade possível. Isso é conhecido como estratégia maxmin, ou minmax, se

MAXMIN STRATEGY

estamos falando de perdas em vez de ganhos. Especificamente, em um jogo com dois agentes,$i$ e $j$, a estratégia maxmin do agente $i$ é dada por

$$s_i^* = \max_{s_i} \min_{s_j} u_i(s_i, s_j). \tag{3.1}$$

Ou seja, $i$ assume que, não importa o que faça, $j$ tomará a ação que for pior para $i$. Assim, $i$ toma a melhor ação possível dada essa suposição. Infelizmente, a estratégia em que ambos os jogadores jogam a sua estratégia maxmin pode não ser estável no caso geral. Em algumas matrizes de payoff pode acontecer que se $i$ sabe que $j$ irá executar sua estratégia maxmin, então $i$ preferirá uma estratégia diferente de sua estratégia maxmin. Por exemplo, na figura 3.1 a estratégia maxmin é $(b, d)$ mas se Alice jogar $d$ então

Bob should play $a$. Thus, that strategy is not stable. This problem has prevented the maxmin strategy from becoming a popular solution concept.

The existence of a solution concept is also important to us since, if a solution concept does not exist for the game we are interested in then it is not of any use to us. For the maxmin strategy we have the **minimax theorem** which states that a strategy that minimizes the maximum loss, a minmax strategy, can always be found for all two-person zero-sum games (Neumann, 1928). Thus, we know it will work at least for this subset of games.

MINIMAX THEOREM

Another approach is to look for strategies that are clearly better. We say that $s$ is the **dominant** strategy for agent $i$ if the agent is better off doing $s$ regardless of which strategies the others use. Formally, we say that a pure strategy $s_i$ is dominant for agent $i$ if

DOMINANT

$$\forall_{s_{-i}} \forall_{r_i \neq s_i} u_i(s_{-i}, s_i) \geq u_i(s_{-i}, r_i), \tag{3.2}$$

where $s_{-i}$ represents the strategies of all agents except $i$. This idea can be expanded into the **iterated dominance** solution in which dominated strategies are eliminated in succession. First we eliminate strategies from one agent, then from another, and so on in a round-robin manner (and repeating agents) until we check all agents in successions and none has a dominated action. Unfortunately, the iterated dominance algorithm almost always ends before a solution is found. That is, in most games none of the players has a dominant strategy.

ITERATED DOMINANCE

We can also take a step back and look at the problem from a system designer's perspective. With our new enlightened outlook we might think that the right solution is to maximize overall welfare for all players. The **social welfare** strategy is the one that maximizes the sum of everyone's payoffs. That is

SOCIAL WELFARE

$$s^* = \arg\max_s \sum_i u_i(s). \tag{3.3}$$

However, once again we have the problem that a social welfare strategy might not be stable. Each selfish agent cares only about his own utility and will thus play a different strategy if it can get a higher utility, even if it means everyone else is much worse. Also, the social welfare strategy might not seem fair as it could be one where one agent gets an extremely high utility and everyone else gets almost nothing.

We can solve the unfairness problem by defining a solution concept that does not take into account the agents' absolute utility values. A strategy $s$ is said to be **Pareto optimal** if there is no other strategy $s'$ such that at least one agent is better off in $s'$ and no agent is worse off in $s'$ than in $s$. There can be more than one Pareto optimal solutions for a given problem. The set of all Pareto strategies for a given problem is formally defined to be the set



Vilfredo Pareto. 1848–1923.

PARETO OPTIMAL

$$\{s \mid \neg\exists_{s' \neq s}(\exists_i u_i(s') > u_i(s) \wedge \neg\exists_{j \in -i} u_j(s) > u_j(s'))\} \tag{3.4}$$

where $-i$ represents the set of all agents except $i$. Pareto solutions are highly desirable from a social welfare perspective as they ensure that no one agent can complain that it could get more utility without hurting someone else in the process. In Economics the Pareto solution of often referred to as Pareto efficient or, simply, the **efficient** solution. Unfortunately, Pareto solutions might also be unstable in that one player might have an incentive to play a different action because he gets higher utility, which means others get lower utility. In a dynamic multiagent system stability can be very important as designers often want the system to converge.

EFFICIENT

The problem of lack of stability was solved by John F. Nash. We say that a strategy $s$ is a **Nash equilibrium** if for all agents $i$, $s_i$ is $i$'s best strategy given that all the other players will play the strategies in $s$. That is, if everyone else is playing the Nash equilibrium then the best thing for everyone to do is to play the Nash equilibrium. As with the Pareto solution, a game can have more than one Nash equilibrium. Formally, the set of all Nash equilibrium strategies for a given game is given by

NASH EQUILIBRIUM

$$\{s \mid \forall_i \forall_{a_i \neq s_i} u_i(s_{-i}, s_i) \geq u_i(s_{-i}, a_i)\}. \tag{3.5}$$



Bob deveria jogar $a$. Assim, essa estratégia não é estável. Este problema impediu que a estratégia maxmin se tornasse um conceito de solução popular.

A existência de um conceito de solução também é importante para nós, pois, se não existir um conceito de solução para o jogo em que estamos interessados, ele não terá nenhuma utilidade para nós. Para a estratégia maxmin temos o teorema minimax que afirma que

MINIMAX THEOREM

Uma estratégia que minimiza a perda máxima, uma estratégia minmax, sempre pode ser encontrada para todos os jogos de soma zero para duas pessoas (Neumann, 1928). Assim, sabemos que funcionará pelo menos para este subconjunto de jogos.

Outra abordagem é procurar estratégias que sejam claramente melhores. Dizemos que $s$é a estratégia dominante para o agente $i$ se for melhor que o agente faça $s$ independentemente de

DOMINANT

quais estratégias os outros usam. Formalmente, dizemos que uma estratégia pura $s_i$ é dominante para o agente $i$ se

$$\forall_{s_{-i}} \forall_{r_i \neq s_i} u_i(s_{-i}, s_i) \geq u_i(s_{-i}, r_i), \tag{3.2}$$

onde $s_{-i}$ representa as estratégias de todos os agentes exceto $i$. Esta ideia pode ser expandida para a solução de dominância iterada, na qual as estratégias dominadas são eliminadas.

ITERATED DOMINANCE

em sucessão. Primeiro eliminamos estratégias de um agente, depois de outro, e assim por diante, de maneira round-robin (e agentes repetitivos), até verificarmos todos os agentes em sucessão e nenhum ter dominado a ação. Infelizmente, o algoritmo de dominância iterado quase sempre termina antes que uma solução seja encontrada. Ou seja, na maioria dos jogos nenhum dos jogadores possui uma estratégia dominante.

Também podemos dar um passo atrás e analisar o problema da perspectiva de um projetista de sistema. Com a nossa nova perspectiva esclarecida, poderíamos pensar que a solução certa é maximizar o bem-estar geral de todos os jogadores. A estratégia de bem-estar social é

SOCIAL WELFARE

aquele que maximiza a soma dos retornos de todos. Aquilo é

$$s^* = \arg\max_s \sum_i u_i(s). \tag{3.3}$$

No entanto, mais uma vez temos o problema de que uma estratégia de bem-estar social pode não ser estável. Cada agente egoísta preocupa-se apenas com a sua própria utilidade e, portanto, executará uma estratégia diferente se conseguir obter uma utilidade mais elevada, mesmo que isso signifique que todos os outros sejam muito piores. Além disso, a estratégia de bem-estar social pode não parecer justa, pois pode ser aquela em que um agente obtém uma utilidade extremamente elevada e todos os outros recebem quase nada.

Podemos resolver o problema da injustiça definindo um conceito de solução que não l eve em conta os valores de utilidade absoluta dos agentes. Diz-se que uma estratégia $s$ ser ótimo de Pareto se não houver outra estratégia $s'$ tal que pelo menos um agente seja em melhor situação em $s'$ e nenhum agente está em pior situação em $s'$ do que em $s$. Pode haver mais de uma solução ótima de Pareto para um determinado problema. O conjunto de todas as estratégias de Pareto para um determinado problema é formalmente definido como o conjunto



Vilfredo Pareto. 1848–1923.

PARETO OPTIMAL

$$\{s \mid \neg\exists_{s' \neq s}(\exists_i u_i(s') > u_i(s) \wedge \neg\exists_{j \in -i} u_j(s) > u_j(s'))\} \tag{3.4}$$

onde $-i$ representa o conjunto de todos os agentes exceto $i$. As soluções de Pareto são altamente desejáveis do ponto de vista do bem-estar social, pois asseguram que nenhum agente pode queixar-se de que poderia obter mais utilidade sem prejudicar alguém no processo. Em Economia, a solução de Pareto é muitas vezes referida como Pareto eficiente ou, simplesmente, a solução eficiente. Infelizmente, as soluções de Pareto também podem ser instáveis, pois

EFFICIENT

um jogador pode ter um incentivo para realizar uma ação diferente porque obtém maior utilidade, o que significa que outros obtêm menor utilidade. Em um sistema multiagente dinâmico, a estabilidade pode ser muito importante, pois os projetistas geralmente desejam que o sistema convirja.

O problema da falta de estabilidade foi resolvido por John F. Nash. Dizemos que uma estratégia $s$ é um equilíbrio de Nash se, para todos os agentes $i$, $s_i$ é a melhor estratégia de $i$ dada

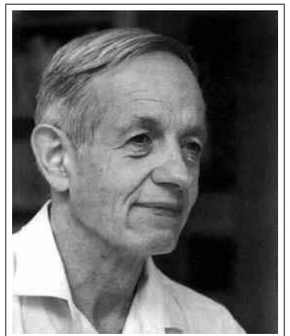
NASH EQUILIBRIUM

que todos os outros jogadores irão jogar as estratégias em $s$. Ou seja, se todos os outros estão jogando no equilíbrio de Nash, então a melhor coisa que todos podem fazer é j ogar no equilíbrio de Nash. Tal como acontece com a solução de Pareto, um jogo pode ter mais de um equilíbrio de Nash. Formalmente, o conjunto de todas as estratégias de equilíbrio de Nash para um determinado jogo é dado por

$$\{s \mid \forall_i \forall_{a_i \neq s_i} u_i(s_{-i}, s_i) \geq u_i(s_{-i}, a_i)\}. \tag{3.5}$$

| | | A | |
|---|---|---|---|
| | | Stays Silent | Betrays |
| B | Stays Silent | Both serve six months. | B serves 10 years; A goes free. |
| | Betrays | A serves 10 years; B goes free. | Both serve two years. |

**Figure 3.2:** Payoff matrix for original prisoner's dilemma problem.



John F. Nash. 1928–. Winner of 1994 Nobel prize on Economics.

Nash showed that all game matrices have at least one equilibrium strategy, but this strategy might be mixed. The only problem with the Nash equilibrium is the fact that in many games there are many such equilibria. Also, some of those Nash equilibria might be better for some players than other. Thus, the players might end up arguing about which specific one to adopt. Still, once the agents have agreed on one it is a very stable solution. Note that there is no general relation between the Nash equilibrium and the Pareto solution. That is, a strategy can be a Nash equilibrium but not be a Pareto solution. Similarly, a strategy can be Pareto optimal but not be a Nash equilibrium. On the other hand, the social welfare strategy must be a Pareto optimal.

All these solution concepts make it clear that it is not a simple matter to define what is the best answer. With multiple players, what is best depends on how much we are willing to respect the agents' selfishness, how we trade-off the various agents utilities, and how fair we wish to be.

The most common problem we face when designing multiagent systems is the existence of multiple equilibria. That is, we can generally design a system so that selfish agents will converge to an equilibrium solution if there is one. But, if there are more than one solution then we need to add extra coordination mechanisms to ensure that the agents converge to the same strategy.

### 3.1.2 Famous Games

Prisoner's Dilemma

The most famous game of all is the **Prisoner's Dilemma**. Its story typically goes something like the following.

> Two suspects A, B are arrested by the police. The police have insufficient evidence for a conviction, and having separated both prisoners, visit each of them and offer the same deal: if one testifies for the prosecution against the other and the other remains silent, the silent accomplice receives the full 10-year sentence and the betrayer goes free. If both stay silent, the police can only give both prisoners 6 months for a minor charge. If both betray each other, they receive a 2-year sentence each.

From this story we can generate a payoff matrix as shown in figure 3.2. We can replace the prison sentences with utility values and arrive at the standard prisoner's dilemma payoff matrix shown in figure 3.3, note that longer prison terms translate into lower utility. Thus, a 10 year sentence gets a utility of 0, a 2 year sentence has utility of 1, 6 months is 3, and no time served has utility of 5. The actions are labeled cooperate and defect because the suspects can either cooperate with each other and maintain their silence or defect from their coalition and tell on the other one.

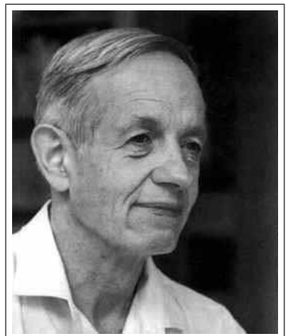
Analysis of this matrix reveals that the social welfare solution is (C,C), the set of Pareto optimal strategies is {(C,C) (D,C) (C,D)}, the dominant strategy for both players is Defection, and the Nash equilibrium strategy is (D,D).

The Prisoner's dilemma is interesting because the best choice is not the rational choice. That is, even thought the (C,C) solution Pareto dominates the (D,D)



| | | A | |
|---|---|---|---|
| | | Permanece em silêncio | Trai |
| B | Permanece em silêncio | Ambos servem seis meses. | B cumpre 10 anos; A fica livre. |
| | Trai | A cumpre 10 anos; B fica livre. | Ambos servem dois anos. |

Figura 3.2: Matriz de recompensas para o problema do dilema do prisioneiro original.



John F. Nash. 1928–. Vencedor do Prêmio Nobel de Economia de 1994.

Nash mostrou que todas as matrizes de jogos têm pelo menos uma estratégia de equilíbrio, mas esta estratégia pode ser mista. O único problema com o equilíbrio de Nash é o fato de que em muitos jogos existem muitos desses equilíbrios. Além disso, alguns desses equilíbrios de Nash podem ser melhores para alguns jogadores do que para outros. Assim, os jogadores podem acabar discutindo sobre qual deles adotar. Ainda assim, uma vez que os agentes cheguem a um acordo, trata-se de uma solução muito estável. Observe que não existe uma relação geral entre o equilíbrio de Nash e a solução de Pareto. Ou seja, uma estratégia pode ser um equilíbrio de Nash, mas não uma solução de Pareto. Da mesma forma, uma estratégia pode ser Paretoótima, mas não ser um equilíbrio de Nash. Por outro lado, a estratégia de bem-estar social deve ser um óptimo de Pareto.

Todos esses conceitos de solução deixam claro que não é simples definir qual é a melhor resposta. Com vários jogadores, o que é melhor depende de quanto estamos dispostos a respeitar o egoísmo dos agentes, de como compensamos as utilidades dos vários agentes e de quão justos desejamos ser.

O problema mais comum que enfrentamos ao projetar sistemas multiagentes é a existência de equilíbrios múltiplos. Ou seja, geralmente podemos projetar um sistema de modo que os agentes egoístas convirjam para uma solução de equilíbrio, se houver. Mas, se houver mais de uma solução, então precisamos adicionar mecanismos extras de coordenação para garantir que os agentes convirjam para a mesma estratégia.

### 3.1.2 Jogos Famosos

Prisoner's Dilemma

O jogo mais famoso de todos é o Dilema do Prisioneiro. Sua história normalmente vai algo como o seguinte.

> Dois suspeitos A, B são presos pela polícia. A polícia não tem provas suficientes para uma condenação e, depois de separar os dois prisioneiros, visita cada um deles e oferece o mesmo acordo: se um testemunhar a favor do processo contra o outro e o outro permanecer em silêncio, o cúmplice silencioso recebe a sentença completa de 10 anos e o traidor é libertado. Se ambos permanecerem em silêncio, a polícia só poderá conceder aos dois prisioneiros 6 meses por uma acusação menor. Se ambos se traírem, receberão pena de 2 anos cada.

A partir desta história podemos gerar uma matriz de payoff conforme mostrado na figura 3.2. Podemos substituir as penas de prisão por valores de utilidade e chegar à matriz padrão de recompensas do dilema do prisioneiro mostrada na figura 3.3, observando que penas de prisão mais longas se traduzem em menor utilidade. Assim, uma pena de 10 anos tem uma utilidade de 0, uma pena de 2 anos tem uma utilidade de 1, 6 meses tem uma utilidade de 3, e nenhum tempo cumprido tem uma utilidade de 5. As acções são rotuladas como cooperar e desertar porque os suspeitos podem cooperar entre si e manter o silêncio ou desertar da sua coligação e denunciar o outro.

A análise desta matriz revela que a solução de bem-estar social é (C,C), o conjunto de estratégias ótimas de Pareto é {(C,C)(D,C)(C,D)}, a estratégia dominante para ambos os jogadores é Deserção, e a estratégia de equilíbrio de Nash é (D,D).

O dilema do Prisioneiro é interessante porque a melhor escolha não é a escolha racional. Ou seja, mesmo que a solução (C,C) Pareto domine a (D,D)

| | | A | |
|---|---|---|---|
| | | Cooperate | Defect |
| B | Cooperate | 3,3 | 0,5 |
| | Defect | 5,0 | 1,1 |

**Figure 3.3:** Prisoner's dilemma with standard payoff values.

| | | Alice | |
|---|---|---|---|
| | | Ice Hockey | Football |
| Bob | Ice Hockey | 4,7 | 0,0 |
| | Football | 3,3 | 7,4 |

**Figure 3.4:** Battle of the sexes game.

solution, a rational player will only play D as that strategy is dominant. The game is widely studied because it applies to many real-world situations like nuclear disarmament and getting kids to clean up their room. There are some that object to the fact that defection is the equilibrium and claim that real people do not act like that.

BATTLE OF THE SEXES

The **battle of the sexes** is another popular game. It is shown in figure 3.4. In this game Alice and Bob like each other and would like to spend time together but must decide, without communicating with each other, where to go. Alice likes ice hockey while Bob likes football. As such, they have a coordination problem where each prefers to go to a different place but they would like to be together. This type of problem arises frequently in multiagent systems where we often have agents that want to cooperate with each other to achieve some larger goal but have conflicting priorities about how to achieve that goal. For example, the agents in a company want to get a new contract so that they can all get paid but each one wants to do as little work as possible while getting as much money as possible. Another example is agents that want to deliver a package but have different preferences on where the hand-off from one agent to the other should occur.

After some analysis of this game we can determine that the social welfare solutions are (I,I) (F,F), the Pareto optimal solutions are also (I,I) (F,F), there is no dominant strategy, and the Nash equilibrium solutions are (I,I) (F,F). As you can see, the problem here is that there are two strategies both of which are equally attractive. This is the type of problem that we could fix easily if we just had a little communication.

GAME OF CHICKEN

The **game of chicken**, shown in figure 3.5, is also common. In this story, two maladjusted teenagers drive their cars towards each other at high speed. The one who swerves first is a chicken and thus loses the game. But, if neither of them swerves then they both die in a horrible crash. After some analysis we can see that this game is very similar to the battle of the sexes. Its social welfare, Pareto optimal, and Nash strategies are all the same, namely (C,S) (S,C), and there is no dominant strategy. Once again there is a coordination problem: who will be the chicken?

One way out of the problem of multiple Nash equilibria is for one of the players to eliminate one of his choices, say by soldering the steering wheel of the car so that it does not turn. This would make it so that the other player's rational choice is to swerve. It might seem counter intuitive but in cases like this an agent can increase

| | | Alice | |
|---|---|---|---|
| | | Continue | Swerve |
| Bob | Continue | -1,-1 | 5,1 |
| | Swerve | 1,5 | 1,1 |

**Figure 3.5:** The game of chicken.



| | | A | |
|---|---|---|---|
| | | Cooperar | Defeito |
| B | Cooperar | 3,3 | 0,5 |
| | Defeito | 5,0 | 1,1 |

Figura 3.3: Dilema do prisioneiro com valores de recompensa padrão.

| | | Alice | |
|---|---|---|---|
| | | Hóquei no Gelo | Futebol |
| Bob | Hóquei no Gelo | 4,7 | 0,0 |
| | Futebol | 3,3 | 7,4 |

Figura 3.4: Jogo da Batalha dos Sexos.

solução, um jogador racional só jogará D porque essa estratégia é dominante. O jogo é amplamente estudado porque se aplica a muitas situações do mundo real, como o desarmamento nuclear e a necessidade de as crianças limparem o seu quarto. Há alguns que se opõem ao facto de a deserção ser o equilíbrio e afirmam que as pessoas reais não agem assim.

BATTLE OF THE SEXES

A batalha dos sexos é outro jogo popular. É mostrado na figura 3.4. Em neste jogo Alice e Bob gostam um do outro e gostariam de passar algum tempo juntos, mas devem decidir, sem se comunicarem, para onde ir. Alice gosta de hóquei no gelo enquanto Bob gosta de futebol. Como tal, têm um problema de coordenação onde cada um prefere ir para um local diferente, mas gostariam de estar juntos. Este tipo de problema surge frequentemente em sistemas multiagentes onde muitas vezes temos agentes que querem cooperar entre si para atingir algum objectivo maior, mas têm prioridades conflituantes sobre como atingir esse objectivo. Por exemplo, os agentes de uma empresa desejam obter um novo contrato para que todos possam ser pagos, mas cada um deseja trabalhar o mínimo possível e, ao mesmo tempo, obter o máximo de dinheiro possível. Outro exemplo são os agentes que desejam entregar um pacote, mas têm preferências diferentes sobre onde deve ocorrer a transferência de um agente para outro.

GAME OF CHICKEN

Após algumas análises deste jogo podemos determinar que as soluções de bem-estar social são (I,I) (F,F), as soluções ótimas de Pareto também são (I,I) (F,F), não existe estratégia dominante e as soluções de equilíbrio de Nash são (I,I) (F,F). Como você pode ver, o problema aqui é que existem duas estratégias, ambas igualmente atraentes. Este é o tipo de problema que poderíamos resolver facilmente se tivéssemos um pouco de comunicação.

A brincadeira do frango, mostrada na figura 3.5, também é comum. Nesta história, dois adolescentes desajustados dirigem seus carros uns em direção aos outros em alta velocidade. Quem desvia primeiro é uma galinha e perde o jogo. Mas, se nenhum deles desviar, ambos morrerão em um acidente horrível. Após algumas análises podemos perceber que este jogo é muito parecido com a batalha dos sexos. Suas estratégias de bem-estar social, Paretoótimo e Nash são todas iguais, ou seja, (C,S) (S,C), e não há estratégia dominante. Mais uma vez há u m problema de coordenação: quem será a galinha?

Uma maneira de resolver o problema dos equilíbrios múltiplos de Nash é um dos jogadores eliminar uma de suas escolhas, digamos, soldando o volante do carro para que ele não gire. Isto faria com que a escolha racional do outro jogador fosse desviar. Pode parecer contra-intuitivo, mas em casos como este um agente pode aumentar

| | | Alice | |
|---|---|---|---|
| | | Continuar | Desviar |
| Bob | Continuar | -1,-1 | 5,1 |
| | Desviar | 1,5 | 1,1 |

Figura 3.5: O jogo da galinha.

| | | Pig | |
|---|---|---|---|
| | | Nothing | Press Lever |
| Piglet | Nothing | 0,0 | 5,1 |
| | Press Lever | -1,6 | 1,5 |

**Figure 3.6:** The pig and the piglet.

its payoff by limiting its set of possible actions.

The final game is taken from the world of zoology. Imagine there is a big pen with a lever which, when pressed, delivers a small amount of food at the other end of the pen. The pen is occupied by one big, but slow, pig and one small and fast piglet. We then try to determine which one of them will press the lever and which one will eat. The situation can be summarized by the matrix in figure 3.6. Namely, if the big pig presses the lever then the piglet can stay standing by where the food comes out and eat some of it before the pig comes and shoves him out. If the piglet presses the lever then it will not get any food because the big pig will be standing by the food dispenser and will not let the piglet get close to the food. If both of them press it then the piglet will be able to eat a little bit since it can run faster than the pig.

An analysis of this game shows that the social welfare solutions are (N,P) (P,P), the Pareto optimal solutions are (N,P) (P,P) (P,N), the piglet has a dominant strategy of N, and the Nash equilibrium is (N,P). Since the piglet has a dominant strategy, it is clear that it will chose that, as such the big pig will have to press the lever. The end result has the piglet siting next to where the food comes out and the big pig running back and forth between the lever and the food.

Animal behaviorists have actually conducted experiments similar to this one and indeed found that the big pig usually ends up pushing the lever. Of course, the pigs do not actually think about their options. They simply try the different actions until they find the one that works best. This tells us that agents can arrive at an equilibrium solution without having much, if any, intelligence and use adaptation instead. We will learn about learning and how we can use it to solve coordination problems in a later chapter.

### 3.1.3 Repeated Games

We saw how in the prisoner's dilemma the dominant strategy was to defect but both players could have received a higher utility by cooperating. One way to try to get out of this conundrum, and to better simulate real-world interactions, is to let two players play the same game some number of times. This new game is known as the **iterated prisoner's dilemma** and, in general, we refer to these type of games as repeated games. Repeated games with a finite horizon are a special case of extended form games, which we will cover in the next section. They have, however, sometimes been studied independently.

ITERATED PRISONER'S DILEMMA

One way to analyze repeated games that last for a finite number of periods is to backtrack from the end. Let's say you are playing an iterated prisoner's dilemma which will last for 50 rounds. You know that at the last round you will defect because that is the dominant strategy and there is no need to be nice to the other player as that is the last game. Of course, since you realized this then you also know that the other player realized the same thing and so he will also defect at the last round. As such, you know that you have nothing to gain by cooperating at round 49 so you will defect at 49, and so will he. This backward induction can be repeated any number of times leading us to the conclusion that for any finite number of games the rational strategy is to always defect. However, people don't act like this.

We can also formally prove a cooperative equilibrium for the iterated prisoner's dilemma if instead of a fixed known number of interactions there is always a small probability that every interaction will be the last interaction. That is, the players



| | | Pig | |
|---|---|---|---|
| | | Nada | Pressione a alavanca |
| Leitão | Nada | 0,0 | 5,1 |
| | Pressione a alavanca | -1,6 | 1,5 |

Figura 3.6: O porco e o leitão.

sua recompensa, limitando seu conjunto de ações possíveis.

O jogo final é retirado do mundo da zoologia. Imagine que há um grande cercado com uma alavanca que, ao ser pressionada, entrega uma pequena quantidade de comida na outra extremidade do cercado. O curral é ocupado por um porco grande, mas lento, e um leitão pequeno e rápido. Tentamos então determinar qual deles irá pressionar a alavanca e qual deles irá comer. A situação pode ser resumida pela matriz da figura 3.6. Ou seja, se o porco grande pressionar a alavanca, o leitão poderá ficar parado onde a comida sai e comer um pouco antes que o porco chegue e o empurre para fora. Se o leitão pressionar a alavanca não conseguirá comida porque o porquinho ficará parado junto ao dispensador de comida e não deixará o leitão chegar perto da comida. Se ambos pressionarem, o leitão poderá comer um pouco, pois pode correr mais rápido que o porco.

Uma análise deste jogo mostra que as soluções de bem-estar social são (N,P)(P,P), as soluções ótimas de Pareto são (N,P) (P,P) (P,N), o leitão tem uma estratégia dominante de N, e o equilíbrio de Nash é (N,P). Como o leitão tem uma estratégia dominante, é claro que a escolherá, como tal o porco grande terá que pressionar a alavanca. O resultado final tem o leitão sentado próximo de onde a comida sai e o porco grande correndo para frente e para trás entre a alavanca e a comida.

Os behavioristas animais conduziram experimentos semelhantes a este e descobriram que o porco grande geralmente acaba empurrando a alavanca. É claro que os porcos não pensam realmente nas suas opções. Eles simplesmente tentam as diferentes ações até encontrarem aquela que funciona melhor. Isto diz-nos que os agentes podem chegar a uma solução de equilíbrio sem ter muita, ou nenhuma, inteligência e, em vez disso, usar a adaptação. Aprenderemos sobre aprendizagem e como podemos usá-la para resolver problemas de coordenação em um capítulo posterior.

### 3.1.3 Jogos Repetidos

Vimos como, no dilema do prisioneiro, a estratégia dominante era desertar, mas ambos os jogadores poderiam ter recebido uma utilidade maior cooperando. Uma maneira de tentar sair desse enigma e simular melhor as interações do mundo real é permitir que dois jogadores joguem o mesmo jogo algumas vezes. Este novo jogo é conhecido como o dilema do prisioneiro iterado e, em geral, referimo-nos a este tipo de jogos como

ITERATED PRISONER'S DILEMMA

jogos repetidos. Jogos repetidos com horizonte finito são um caso especial de jogos estendidos, que abordaremos na próxima seção. Eles foram, no entanto, algumas vezes estudados de forma independente.

Uma forma de analisar jogos repetidos que duram um número finito de períodos é retroceder desde o final. Digamos que você esteja jogando um dilema do prisioneiro iterativo que durará 50 rodadas. Você sabe que na última rodada você irá desertar porque essa é a estratégia dominante e não há necessidade de ser gentil com o outro jogador, pois esse é o último jogo. Claro, como você percebeu isso, você também sabe que o outro jogador percebeu a mesma coisa e então ele também desertará na última rodada. Como tal, você sabe que não tem nada a ganhar cooperando na rodada 49, então desertará na 49, e ele também. Esta indução retroativa pode ser repetida inúmeras vezes, levando-nos à conclusão de que, para qualquer número finito de jogos, a estratégia racional é sempre desertar. No entanto, as pessoas não agem assim.

Também podemos provar formalmente um equilíbrio cooperativo para o dilema do prisioneiro iterado se, em vez de um número fixo conhecido de interações, houver sempre uma pequena probabilidade de que cada interação seja a última interação. Ou seja, os jogadores

never know if this will be their last interaction or not. In this scenario we can show that the dominant strategy for the iterated prisoner's dilemma is to cooperate, with a certain probability.

FOLK THEOREM

More generally, however, the **folk theorem** tells us that in a repeated game any strategy that is not Pareto-dominated by another and where every agent gets a utility that is higher than his maxmin utility (3.1) is a feasible equilibrium strategy. In these cases, each player knows that he can get his maxmin utility by playing the appropriate action, regardless of what the others do. Thus, no player will be satisfied with less than his maxmin utility. On the other hand, if the agents agree to a strategy that gives everyone at least their maxmin and is not Pareto-dominated then any agent that diverges from it could be penalized by the other agents so that any gains he incurred from the deviation would be erased. In this way the agents can police any chosen strategy and thus ensure that it is a stable strategy.

Such analysis still leaves open the question of which, in practice, is the optimal strategy for the iterated prisoner's dilemma. In the early 1980's Robert Axelrod performed some experiments on the iterated prisoner's dilemma (Axelrod, 1984). He sent out an email asking people to submit fortran programs that played the prisoner's dilemma against each other for 200 rounds. The winner was the one that accumulated the most points. Many entries were submitted. They included the following strategies.

- *ALL-D*: always play defect.
- *RANDOM*: pick action randomly.
- *TIT-FOR-TAT*: cooperate in the first round, then do whatever the other player did last time.
- *TESTER*: defect on the first round. If other player defects then play tit-for-tat. If he cooperated then cooperate for two rounds then defect.
- *JOSS*: play tit-for-tat but 10% of the time defect instead of cooperating.

TIT-FOR-TAT

The **tit-for-tat** strategy won the tournament. It still made less than ALL-D when playing against it but, overall, it won more than any other strategy. It was successful because it had the opportunity to play against other programs that were inclined to cooperate. It cooperated with those that could cooperate and defected against the rest. This result, while intriguing, is not theoretically robust. For example a tit-for-tat strategy would lose against a strategy that plays tit-for-tat but defects on the last round. Still, the tit-for-tat strategy has been widely used and is considered to be a simple yet robust strategy.

Another method for analyzing repeated games is by assuming that the players use some learning algorithm and then trying to determine which strategy their learning will converge to. This research area is known as learning in games, which we present in section 5.3.

## 3.2 Games in Extended Form

EXTENDED FORM

In **extended form** games the players take sequential actions. These games are represented using a tree where the branches at each level correspond to a different player's actions and the payoffs to all agents are given at the leafs. Extended form games can also represent simultaneous actions by using dotted ellipses to group together nodes which are equivalent to the rest of the agents because, for example, they could not see the actions taken by the agent.

Figure 3.7 shows the extended form version of the payoff matrix from figure 3.1. In it, the first number inside the parentheses is the utility that Bob receives and the second number is Alice's utility. The dotted line groups together two states which are invisible to Bob: it shows that Bob does not know the action Alice took. If we



nunca se sabe se esta será a última interação deles ou não. Neste cenário podemos mostrar que a estratégia dominante para o dilema do prisioneiro iterado é cooperar, com uma certa probabilidade.

FOLK THEOREM

De forma mais geral, porém, o teorema folk nos diz que num jogo repetido qualquer estratégia que não seja dominada por Pareto por outra e onde cada agente obtenha uma utilidade superior à sua utilidade maxmin (3.1) é uma estratégia de equilíbrio viável. Nestes casos, cada jogador sabe que pode obter a sua utilidade máxima executando a acção apropriada, independentemente do que os outros façam. Assim, nenhum jogador ficará satisfeito com menos do que sua utilidade máxima. Por outro lado, se os agentes concordarem com uma estratégia que dê a todos pelo menos o seu maxmin e não seja dominada por Pareto, então qualquer agente que divirja dela poderá ser penalizado pelos outros agentes, de modo que quaisquer ganhos que ele incorra com o desvio seriam apagados. Desta forma os agentes podem policiar qualquer estratégia escolhida e assim garantir que seja uma estratégia estável.

Tal análise ainda deixa em aberto a questão de qual, na prática, é a estratégia ideal para o dilema do prisioneiro iterativo. No início da década de 1980, Robert Axelrod realizou algumas experiências sobre o dilema do prisioneiro iterativo (Axelrod, 1984). Ele enviou um e-mail pedindo às pessoas que enviassem programas Fortran que jogassem o dilema do prisioneiro uns contra os outros por 200 rodadas. O vencedor foi aquele que acumulou mais pontos. Muitas inscrições foram enviadas. Eles incluíram as seguintes estratégias.

- ALL-D: sempre reproduz defeito.
- RANDOM: escolha a ação aleatoriamente.
- TIT-FOR-TAT: coopere na primeira rodada e depois faça o que o outro jogador fez da última vez.
- TESTER: defeito na primeira rodada. Se outro jogador desertar, jogue na mesma moeda. Se ele cooperou, coopere por duas rodadas e depois deserte.
- JOSS: jogue olho por olho, mas 10% das vezes desista em vez de cooperar.

TIT-FOR-TAT

A estratégia de olho por olho venceu o torneio. Ainda fez menos que ALL-D ao jogar contra ele, mas, no geral, ganhou mais do que qualquer outra estratégia. Foi bem sucedido porque teve a oportunidade de jogar contra outros programas que estavam inclinados a cooperar. Cooperou com aqueles que podiam cooperar e desertou contra os demais. Este resultado, embora intrigante, não é teoricamente robusto. Por exemplo, uma estratégia de retaliação perderia contra uma estratégia que joga na mesma moeda, mas falha na última rodada. Ainda assim, a estratégia tit-for-tat tem si do amplamente utilizada e é considerada uma estratégia simples mas robusta.

Outro método para analisar jogos repetidos é assumir que os jogadores usam alg um algoritmo de aprendizagem e então tentar determinar para qual estratégia seu aprendizado irá convergir. Esta área de pesquisa é conhecida como aprendizagem em jogos, que apresentamos na seção 5.3.

## 3.2 Jogos em Formato Estendido

EXTENDED FORM

Em jogos estendidos, os jogadores realizam ações sequenciais. Esses jogos são representado por meio de uma árvore onde os ramos em cada nível correspondem às ações de um jogador diferente e os pagamentos a todos os agentes são dados nas folhas. Os jogos de forma estendida também podem representar ações simultâneas usando elipses pontilhadas para agrupar nós que são equivalentes ao resto dos agentes porque, por exemplo, eles não puderam ver as ações realizadas pelo agente.

A Figura 3.7 mostra a versão estendida da matriz de retorno da figura 3.1. Nele, o primeiro número entre parênteses é a utilidade que Bob recebe e o segundo número é a utilidade de Alice. A linha pontilhada agrupa dois estados que são invisíveis para Bob: mostra que Bob não sabe a ação que Alice realizou. Se nós

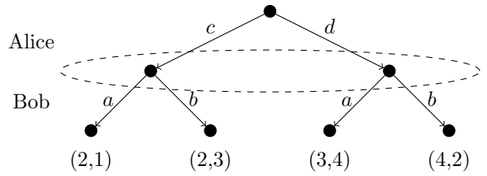



**Figure 3.7:** Game in extended form that corresponds to the normal form game from Figure 3.1.

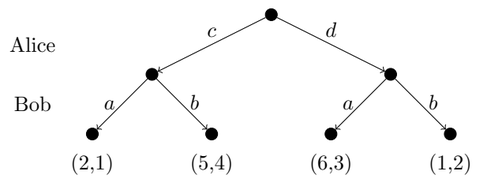



**Figure 3.8:** Game in extended form.

Extended form games, without the dotted ellipses, are nearly identical those built by the minimax algorithm (Russell and Norvig, 2003, Chapter 6).

eliminated the dotted line then in that new game Alice takes an action, which Bob can see, and then Bob takes his action.

In an extended game a player's strategy $s_i$ is no longer just an action but can now be a series of actions if the player gets to take more than one action in the tree, or if the player can have different actions depending on which node in the tree it is in. That is, if a player can see others' actions that come before him then his action can vary. For example, in figure 3.8 a rational Bob would choose $b$ if Alice has chosen $c$ but would choose $a$ if Alice has chosen $d$. As before, agent $i$'s utility from strategy $s$ is given by $u_i(s)$ and corresponds to the values on the leaf node reached when all players play $s$.

### 3.2.1 Solution Concepts

We can apply the Nash equilibrium idea from normal form games to extended games. Namely, we say that an extended game has a Nash equilibrium strategy $s^*$ if for all agents $i$ it is true that they can't gain any more utility by playing a strategy different from $s_i^*$ given that everyone else is playing $s_{-i}^*$. For example, for the game in figure 3.8 the Nash equilibrium is $(c, b)$—Alice plays $c$ and Bob plays $b$. While this strategy is in equilibrium notice how, if there was some problem or noise in the system and Alice ends up playing $d$ then Bob's strategy of playing $b$ is no longer his best response. That is, the basic Nash equilibrium solution might not be stable under noisy conditions.

SUBGAME PERFECT EQUILIBRIUM

A stronger solution concept is the **subgame perfect equilibrium** strategy $s^*$ which is defined as one where for all agents $i$ and all subgames it is true that $i$ can't gain any more utility by playing a strategy different from $s_i^*$. We define a subgame to be any subtree of the extended game. That is, a subgame perfect equilibrium gives all the agents their best response strategies for every node in the tree. For example, the subgame perfect equilibrium for figure 3.8 is for Alice to play $c$ and Bob to play $b$ if Alice plays $c$ and $a$ if Alice plays $d$.

Extended form games are a useful way to represent more complex multiagent interactions. Notice, however, that they are a special case of a multiagent Markov decision process. Most researchers have opted to use the more complete Markov model over the simpler extended form game for modeling multiagent systems.

These solution concepts provide us with solutions to shoot for when building multiagent systems, even when these solutions are impossible to find. For example, say you want to build a team of robot soccer players. Each robot will have a specific behavior that depends on its current perception of the world and its internal state. Each robot can also be assumed to receive a payoff whenever it, or someone in its team, scores a goal. If you consider the set of all possible agent behaviors for all agents in the team as the set of actions in a game then we can talk about the players



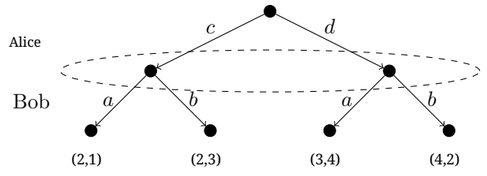



Figura 3.7: Jogo na f orma estendida que corresponde ao jogo na forma normal da Figura 3.1.

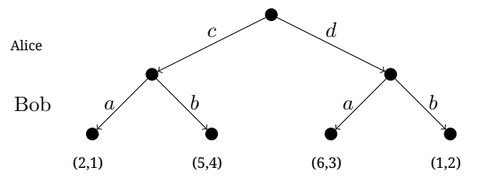



Figura 3.8: Jogo em formato estendido.

Os jogos de formato estendido, sem as elipses pontilhadas, são quase idênticos aos construídos pelo algoritmo minimax (Russell e Norvig, 2003, Capítulo 6).

eliminou a linha pontilhada naquele novo jogo, Alice realiza uma ação, que Bob pode ver, e então Bob realiza sua ação.

Em um jogo estendido, a estratégia de um jogador $s_i$ não é mais apenas uma ação, mas agora pode ser uma série de ações se o jogador puder realizar mais de uma ação na árvore, ou se o jogador puder ter ações diferentes dependendo de qual nó da árvore ele está. Por exemplo, na figura 3.8, um Bob racional escolheria $b$ se Alice tivesse escolhido $c$, mas escolheria $a$ se Alice tivesse escolhido $d$. Como antes, a utilidade do agente $i$ da estratégia $s$ é dada por $u_i(s)$ e corresponde aos valores no nó folha alcançados quando todos os jogadores jogam $s$.

### 3.2.1 Conceitos de Solução

Podemos aplicar a ideia do equilíbrio de Nash desde jogos normais até jogos estendidos. Ou seja, dizemos que um jogo estendido tem uma estratégia de equilíbrio de Nash $s^*$ se para todos os agentes $i$ for verdade que eles não podem ganhar mais utilidade j ogando uma estratégia diferente de $s_i^*$ dado que todos os outros estão jogando $s_{-i}^*$. Por exemplo, para o jogo da figura 3.8, o equilíbrio de Nash é $(c, b)$ – Alice joga $c$ e Bob joga $b$. Enquanto esta estratégia está em equilíbrio, observe como, se houve algum problema ou ruído no sistema e Alice acaba jogando $d$ então a estratégia de jogar de Bob $b$ não é mais sua melhor resposta. Ou seja, a solução básica de equilíbrio de Nash pode não ser estável sob condições ruidosas.

SUBGAME PERFECT EQUILIBRIUM

Um conceito de solução mais forte é a estratégia de equilíbrio perfeito do subjogo $s^*$ que é definido como aquele em que para todos os agentes $i$ e todos os subjogos é verdade que $i$ não podem ganhar mais utilidade ao jogar uma estratégia diferente de $s_i^*$. Definimos um subjogo como qualquer subárvore do jogo estendido. Ou seja, um equilíbrio perfeito de subjogo dá a todos os agentes suas melhores estratégias de resposta para cada nó da árvore. Por exemplo, o equilíbrio perfeito do subjogo para a figura 3.8 é que Alice jogue $c$ e Bob jogue $b$ se Alice jogar $c$ e $a$ se Alice jogar $d$.

Jogos estendidos são uma forma útil de representar interações multiagentes mais complexas. Observe, entretanto, que eles são um caso especial de processo de decisão multiagente de Markov. A maioria dos pesquisadores optou por usar o modelo de Markov mais completo em vez do jogo estendido mais simples para modelar sistemas multiagentes.

Esses conceitos de solução nos fornecem soluções para a construção de sistemas multiagentes, mesmo quando essas soluções são impossíveis de encontrar. Por exemplo, digamos que você queira construir um time de jogadores de futebol robôs. Cada robô terá um comportamento específico que depende da sua percepção atual do mundo e do seu estado interno. Também se pode presumir que cada robô receberá uma recompensa sempre que ele, ou alguém de sua equipe, marcar um gol. Se você considerar o conjunto de todos os comportamentos possíveis dos agentes para todos os agentes da equipe como o conjunto de ações em um jogo, então podemos falar sobre os jogadores.

being at a Nash equilibrium, or about the players in one team being in the Pareto optimum given the other team's behaviors. Of course, such equilibria are impossible to calculate but they do give us a precise goal, of the intellectual kind, to shoot for. Also, as many researchers have done, we can try to break up that immense problem into smaller problems for which the agents can find equilibrium strategies.

## 3.3 Finding a Solution

There is an extensive literature on centralized algorithms for finding the various equilibrium strategies for a given game they are, however, generally considered to be outside the purview of multiagent research. Generally, the algorithms involve a complete search using heuristics for pruning. The **Gambit** software program (McKelvey et al., 2006) is an open source implementation of several of these algorithms.

GAMBIT

However, note that some of these solutions can be found by more multiagent-friendly distributed algorithms. In chapter 5 we show multiagent learning algorithms which allow learning agents to converge to Nash and other equilibria.

### Exercises

3.1 Prove that a social welfare solution must be Pareto optimal.

3.2 Show a 2-person standard form game with 3 actions for each player in which iterated dominance leads to a unique equilibrium strategy.

3.3 Assume a 2-person standard form game with $x$ actions for each player.

1. What is the maximum number of Pareto optimal pure strategies that such a game could have?
2. Assuming that all payoff values are different, what is the maximum number of Pareto optimal pure strategies that such a game could have?

3.4 Find the pure Nash equilibria, and the Pareto optimal solutions of the following game:

| | | Alice | | |
|---|---|---|---|---|
| | | $d$ | $e$ | $f$ |
| Bob | $a$ | 1,2 | 2,3 | 2,3 |
| | $b$ | 4,5 | 6,7 | 3,4 |
| | $c$ | 5,4 | 6,5 | 5,6 |

3.5 The battle of the sexes game, seen in figure 3.4, is a classic coordination game in which the players must somehow coordinate to agree upon one of the Nash equilibria. These problems are solved by populations who adopt a social law after some trial and error adaptation phase. For example, in the US people drive on the right side of the rode while in England they drive on the left. Both solutions are equally valid.



coordination

Implement a NetLogo program where each patch repeatedly engages in a battle of the sexes game with one of its neighbors, chosen at random. Then try to come up with some simple adaptation strategy which the agents could use so that the population will quickly converge to an equilibrium.

Simple adaptation strategies for solving this problem exist (Shoham and Tennenholtz, 1997), as well as for more complex neighborhood definitions (Delgado, 2002).



estar em equilíbrio de Nash, ou sobre os jogadores de uma equipe estarem no ótimo de Pareto, dados os comportamentos da outra equipe. É claro que tais equilíbrios são i mpossíveis de calcular, mas dão-nos um objectivo preciso, de tipo intelectual, a atingir. Além disso, como muitos pesquisadores fizeram, podemos tentar dividir esse imenso problema e m problemas menores para os quais os agentes possam encontrar estratégias de equilíbrio.

## 3.3 Encontrando uma solução

Existe uma extensa literatura sobre algoritmos centralizados para encontrar as várias estratégias de equilíbrio para um determinado jogo; no entanto, são geralmente c onsiderados fora do alcance da pesquisa multiagente. Geralmente, os algoritmos envolvem uma busca completa usando heurísticas para poda. O programa de software Gambit (McKelvey et al., 2006) é uma implementação de código aberto de vários desses algoritmos.

GAMBIT

Entretanto, observe que algumas dessas soluções podem ser encontradas por algoritmos distribuídos mais amigáveis a multiagentes. No capítulo 5 mostramos algoritmos de aprendizagem multiagentes que permitem que os agentes de aprendizagem convirjam para Nash e outros equilíbrios.

### Exercícios

3.1 Prove que uma solução de bem-estar social deve ser ótima de Pareto.

3.2 Mostre um jogo padrão para 2 pessoas com 3 ações para cada jogador, no qual a dominância iterada leva a uma estratégia de equilíbrio única.

3.3 Suponha um jogo padrão para 2 pessoas com $x$ ações para cada jogador.

1. Qual é o número máximo de estratégias puras ótimas de Pareto que tal jogo poderia ter? 2. Supondo que todos os valores de payoff sejam diferentes, qual é o número máximo de estratégias puras ótimas de Pareto que tal jogo poderia ter?

3.4 Encontre o equilíbrio de Nash puro e as soluções ótimas de Pareto do seguinte jogo:

| | | Alice | | |
|---|---|---|---|---|
| | | $d$ | $e$ | $f$ |
| Bob | $a$ | 1,2 | 2,3 | 2,3 |
| | $b$ | 4,5 | 6,7 | 3,4 |
| | $c$ | 5,4 | 6,5 | 5,6 |

3.5 O jogo da batalha dos sexos, visto na figura 3.4, é um clássico jogo de coordenação no qual os jogadores devem de alguma forma coordenar-se para chegar a um acordo sobre um dos equilíbrios de Nash. Estes problemas são resolvidos pelas populações que adoptam uma lei social após alguma fase de adaptação por tentativa e erro. Por exemplo, nos EUA as pessoas dirigem do lado direito, enquanto na Inglaterra dirigem do lado esquerdo. Ambas as soluções são igualmente válidas.



coordenação

Implemente um programa NetLogo onde cada patch se envolva repetidamente em um jogo de batalha dos sexos com um de seus vizinhos, escolhido aleatoriamente. Em seguida, tente criar alguma estratégia de adaptação simples que os agentes possam usar para que a população convirja rapidamente para um equilíbrio.

Existem estratégias de adaptação simples para resolver este problema (Shoham e Tennenholtz, 1997), bem como para definições de vizinhança mais complexas (Delgado, 2002).

3.6 Lets say that in a repeated version of the game of chicken, figure 3.5, Alice and Bob decided to converge to the strategy (Swerve, Swerve). Since this strategy satisfies the maxmin criteria (check this) the Folk theorem tells us that is will be stable since both players could play an iterated strategy that penalizes the other so that any gains from defection are erased after some rounds. Write the short algorithm which describes the player's iterated strategy for this situation, that is, the iterated strategy they must use to guarantee that (Swerve, Swerve) is the equilibrium strategy on every step of the iterated game.

3.7 A generalization of the Tit-for-Tat strategy is to, instead of simply doing exactly what the other agent did last time, make a stochastic decision based on the other agent's action. That is, if the other agent defected last time then you will be very likely, with a given probability, to defect and vice versa. Write a NetLogo program where you add these type of agents to a tournament similar to Axelrod's. Which strategy triumphs in the end?


packages

This general strategy is known as reciprocity and often occurs in human interaction: you are more likely, but not entirely sure, to be nice to those that were nice to you in the past. Simulations of populations with various numbers of reciprocating agents have shown that populations of reciprocating agents tend to do better as a whole (since they help each other) but can be exploited by selfish agents. This exploitation can be curbed by having the reciprocating agents share their opinions of other (Sen, 2002).



3.6 Digamos que em uma versão repetida do jogo da galinha, figura 3.5, Alice e Bob decidiram convergir para a estratégia (Swerve, Swerve). Como esta estratégia satisfaz os critérios maxmin (verifique isto), o teorema Folk nos diz que será estável, pois ambos os jogadores poderiam jogar uma estratégia iterada que penalize o outro, de modo que quaisquer ganhos da deserção sejam apagados após algumas rodadas. Escreva o pequeno algoritmo que descreve a estratégia iterada do jogador para esta situação, ou seja, a estratégia iterada que ele deve usar para garantir que (Swerve, Swerve) seja a estratégia de equilíbrio em cada etapa do jogo iterado.

3.7 Uma generalização da estratégia Tit-for-Tat é, em vez de simplesmente fazer exatamente o que o outro agente fez da última vez, tomar uma decisão estocástica baseada na ação do outro agente. Ou seja, se o outro agente desertou da última vez, então é muito provável que você, com uma determinada probabilidade, deserte e vice-versa. Escreva um programa NetLogo onde você adiciona esse tipo de agente a um torneio semelhante ao de Axelrod. Qual estratégia triunfa no final?


pacotes

Esta estratégia geral é conhecida como reciprocidade e ocorre frequentemente na interação humana: é mais provável, mas não totalmente certo, que você seja gentil com aqueles que foram gentis com você no passado. Simulações de populações com vários números de agentes recíprocos mostraram que as populações de agentes recíprocos tendem a ter um desempenho melhor como um todo (uma vez que se ajudam mutuamente), mas podem ser exploradas por agentes egoístas. Esta exploração pode ser controlada fazendo com que os agentes recíprocos partilhem as suas opiniões sobre os outros (Sen, 2002).

## Chapter 4

# Characteristic Form Games and Coalition Formation

There is another type of game studied in game theory: the **characteristic form** game or **coalition** game (Osborne and Rubinstein, 1999). In these games the agents decide how to form coalitions among themselves and each coalition receives some utility. For example, a group of people, each with different skills, all want to start new companies. The problem they face is deciding how to divide themselves into subgroups such that each subgroup has the needed set of skills to succeed in the marketplace. Similarly, a group of agents with different skills must decide how to divide itself into subgroups so as handle as many tasks as possible in the most efficient manner. Because the agents must cooperate with each other in order to form coalitions and an agent cannot unilaterally decide that it will form a coalition with a second agent, these games are known as **cooperative games**. Multiagent researchers have also extended the basic characteristic form into to the more general coalition formation, which we also present in this chapter.

CHARACTERISTIC FORM
COALITION

COOPERATIVE GAMES

It is interesting to note that most game theory textbooks focus exclusively on non-cooperative games as these have found many applications in Economics and Business and have been the focus of most of the research. However, when building multiagent systems we find that cooperative games are much more useful since they clearly and immediately model the problem of which agents should perform which tasks.

## 4.1 Characteristic Form Games

Formally, a game in characteristic form includes a set $A = \{1, \ldots, |A|\}$ of agents. The agents are assumed to deliberate and the final result of the deliberation is an **outcome** $\vec{u} = (u_1, \ldots, u_{|A|}) \in \Re^{|A|}$ which is just a vector of utilities, one for each agent. There is also a rule $V(\cdot)$ that maps every coalition $S \subset A$ to a utility possibility set, that is $V(S) \subset \Re^{|S|}$. Notice that $V(S)$ returns a set of utility vectors, not a single utility vector. As such, $V(\cdot)$ provides us the set of payoffs that players in $S$ can achieve it they form a coalition. For example, for the players $\{1, 2, 3\}$ we might have that $V(\{1, 2\}) = \{(5, 4), (3, 6)\}$ meaning that if agents 1 and 2 formed a coalition they could either get 5 for agent 1 and 4 for agent 2, or they could get 3 for agent 1 and 6 for agent 2. The function $V$ must be defined for all subsets of $A$.

OUTCOME

A special case of the characteristic form game–the one nearly all multiagent research focuses on—is the **transferable utility** game in characteristic form. This game assumes that the players can exchange utilities among themselves as they see fit. For example, if the utility payments are in the form of money then we only need to specify the total amount of money the coalition will receive and decide later how this money will be distributed among the agents in the coalition. More formally, we define a transferable utility game

TRANSFERABLE UTILITY

**Definition 4.1** (Tranferable utility characteristic form game)**.** *These games consist of a set of agents* $A = \{1, \ldots, A\}$ *and a* characteristic function $v(S) \rightarrow \Re$ *defined for every* $S \subseteq A$*.*

The **characteristic function** $v(S)$ is also sometimes simply referred to as the **value function** for the coalitions. Characteristic form games with transferable

CHARACTERISTIC FUNCTION
VALUE FUNCTION



## Capítulo 4

# Jogos de Forma Característica e Formação de Coalizões

Existe outro tipo de jogo estudado na teoria dos jogos: a forma característica jogo ou jogo de coalizão (Osborne e Rubinstein, 1999). Nestes jogos os agentes decidem como formar coligações entre si e cada coligação recebe alguma utilidade. Por exemplo, um grupo de pessoas, cada uma com habilidades diferentes, deseja abrir novas empresas. O problema que enfrentam é decidir como se dividirem em subgrupos, de modo que cada subgrupo tenha o conjunto necessário de competências para ter sucesso no mercado. Da mesma forma, um grupo de agentes com competências diferentes deve decidir como se dividir em subgrupos para realizar o maior número possível de tarefas da maneira mais eficiente. Como os agentes devem cooperar entre si para formar coalizões e um agente não pode decidir unilateralmente que formará uma coalizão com um segundo agente, esses jogos são conhecidos como jogos cooperativos. Multiagente

CHARACTERISTIC FORM
COALITION

os investigadores também alargaram a forma característica básica à formação de coligação mais geral, que também apresentamos neste capítulo.

COOPERATIVE GAMES

É interessante notar que a maioria dos livros didáticos de teoria dos jogos concentra-se exclusivamente em jogos não cooperativos, uma vez que estes encontraram muitas aplicações em Economia e Negócios e têm sido o foco da maior parte da pesquisa. Contudo, ao construir sistemas multiagentes, descobrimos que os jogos cooperativos são muito mais úteis, uma vez que modelam clara e imediatamente o problema de quais agentes devem executar quais tarefas.

## 4.1 Forma CaracterísticaJogos

Formalmente, um jogo em forma característica inclui um conjunto $A = \{1, \ldots, |A|\}$ de agentes. Presume-se que os agentes deliberam e o resultado final da deliberação é um resultado $\vec{u} = (u_1, \ldots, u_{|A|}) \in \Re^{|A|}$ que é apenas um vetor de utilidades, um para cada agente. Existe também uma regra $V(\cdot)$ que mapeia cada coalizão $S \subset A$ para um conjunto de possibilidades de utilidade, que é $V(S) \subset \Re^{|S|}$. Observe que $V(S)$ retorna um conjunto de vetores de utilidade, não um único vetor de utilidade. Como tal, $V(\cdot)$ nos fornece o conjunto de recompensas que os jogadores em $S$ podem alcançar se formarem uma coalizão. Por exemplo, para os jogadores $\{1, 2, 3\}$ podemos ter que $V(\{1, 2\}) = \{(5,4), (3, 6)\}$ o que significa que se os agentes 1 e 2 formassem uma coalizão, eles poderiam obter 5 para o agente 1 e 4 para o agente 2, o u poderiam obter 3 para o agente 1 e 6 para o agente 2. A função $V$ deve ser definida para todos os subconjuntos de $A$.

OUTCOME

Um caso especial do jogo de forma característica – aquele em que quase todas as pesquisas multiagentes se concentram – é o jogo de utilidade transferível em forma característica. Esse

O jogo assume que os jogadores podem trocar utilidades entre si conforme acharem adequado. Por exemplo, se os pagamentos de serviços públicos forem na forma de dinheiro, então só precisamos de especificar o montante total de dinheiro que a coligação receberá e decidir mais tarde como esse dinheiro será distribuído entre os agentes da coligação. Mais formalmente, definimos um jogo utilitário transferível

TRANSFERABLE UTILITY

Definição 4.1 (Jogo de forma de característica de utilidade transferível). Esses jogos consistem em um conjunto de agentes $A = \{1, \ldots, A\}$ e uma função característica $v(S) \rightarrow \Re$ definida para cada $S \subseteq A$.

A função característica $v(S)$ às vezes também é chamada simplesmente de função de valor para as coalizões. Jogos de forma característica com transferíveis

CHARACTERISTIC FUNCTION
VALUE FUNCTION

| $S$ | $v(S)$ |
|---|---|
| (1) | 2 |
| (2) | 2 |
| (3) | 4 |
| (12) | 5 |
| (13) | 7 |
| (23) | 8 |
| (123) | 9 |

(1)(2)(3)
$2+2+4=8$

(1)(23)
$2+8=10$

(2)(13)
$2+7=9$

(3)(12)
$4+5=9$

(123)
9

**Figure 4.1:** Sample characteristic form game with transferable utility for three agents: 1, 2 and 3. The table on the left shows the values of each coalition. On the right are the coalition structures. Below each one we calculate its value.

utility can represent many multiagent scenarios. For example, they can represent a task allocation problem where a set of tasks has to be performed by a set of agents, subsets of whom can sometimes improve their performance by joining together to perform a task. They can represent a sensor network problems where the sensors must join together in subgroups to further refine their readings or relay important information, or they can represent workflow scheduling systems where agents must form groups to handle incoming workflows.

### 4.1.1 Solution Concepts

As is often the case in game theory, there is no clear best solution to all characteristic form games. Instead, various solutions concepts have been proposed each one having its own advantages and disadvantages.

Before defining the solution concepts, we must first notice that the outcome as we have defined it allows for impossible utility values in the transferable utility game. Specifically, there might not be a set of coalitions such that, given $v$, the agents can all get their utility as promised by $\vec{u}$. In order to rectify this problem we

FEASIBLE

first specify that we are only interested in **feasible** outcomes, that is, those that can be implemented given $v$.

**Definition 4.2** (Feasible)**.** *An outcome $\vec{u}$ is* feasible *if there exists a set of coalitions $T = S_1, \ldots, S_k$ where $\bigcup_{S \in T} S = A$ such that $\sum_{S \in T} v(S) \geq \sum_{i \in A} \vec{u}_i$.*

That is, an outcome $\vec{u}$ is feasible if we can find a disjoint set of coalitions whose values are as much as that in $\vec{u}$, so we can payoff $\vec{u}$ with $v$. The set of disjoint coalitions $T$ defined above is also often referred to as a **coalition structure**, also sometimes represented with the symbol $CS$.

COALITION STRUCTURE

Notice that, if the characteristic function is super-additive then we can check if an outcome $\vec{u}$ is feasible by simply ensuring that

$$\sum_{i \in A} \vec{u}_i = v(A). \tag{4.1}$$

We define a **super-additive** domain as one where, for all pairs of disjoint coalitions $S, T \subset A$, we have that $v(S \cup T) \geq v(S) + v(T)$. That is, there is nothing to be lost by merging into a bigger coalition. Unfortunately, multiagent systems are rarely supper-additive since agents have a habit of getting into each others' way, so that a team is not always better than letting each agent work on a task separately.

SUPER-ADDITIVE

The problem of finding feasible solutions can best be illustrated with an example. Figure 4.1 shows a sample transferable utility game for three agents along with the definition of the $v$ function and all possible coalition formations. In this game the outcome $\vec{u} = \{5, 5, 5\}$ is not feasible since there is no way to divide the agents into subsets such that they can all get their utility. If we tried the coalition (123) then we only have a value of 9 to distribute and we need a total of 15. On the other hand, the outcome $\vec{u} = \{2, 4, 3\}$, is feasible because the coalition structures (1)(23), (2)(13), and (123) can satisfy it (but not the other ones). However, $\vec{u} = \{2, 4, 3\}$ does have a problem in that in it agent 3 is getting an utility of 3 while we have that $v((3)) = 4$. That is, agent 3 could defect any one of the three coalition structures



| $S$ | $v(S)$ |
|---|---|
| (1) | 2 |
| (2) | 2 |
| (3) | 4 |
| (12) | 5 |
| (13) | 7 |
| (23) | 8 |
| (123) | 9 |

(1)(2)(3)
$2+2+4=8$

(1)(23)
$2+8=10$

(2)(13)
$2+7=9$

(3)(12)
$4+5=9$

(123)
9

Figura 4.1: Exemplo de jogo de forma característica com utilidade transferível para três agentes: 1, 2 e 3. A tabela à esquerda mostra os valores de cada coalizão. À direita estão as estruturas de coalizão. Abaixo de cada um calculamos seu valor.

utilitário pode representar muitos cenários multiagentes. Por exemplo, eles podem representar um problema de alocação de tarefas onde um conjunto de tarefas tem que ser executado por um conjunto de agentes, subconjuntos dos quais podem às vezes melhorar seu desempenho unindo-se para executar uma tarefa. Eles podem representar problemas de re de de sensores onde os sensores devem se unir em subgrupos para refinar ainda mais suas leituras ou retransmitir informações importantes, ou podem representar sistemas de agendamento de fluxo de trabalho onde os agentes devem formar grupos para lidar com fluxos de trabalho recebidos.

### 4.1.1 Conceitos de Solução

Como costuma acontecer na teoria dos jogos, não existe uma melhor solução clara para todas as formas características dos jogos. Em vez disso, foram propostos vários conceitos de soluções, cada um com as suas próprias vantagens e desvantagens.

Antes de definir os conceitos de solução, devemos primeiro notar que o resultado tal como o definimos permite valores de utilidade impossíveis no jogo da utilidade transferível. Especificamente, pode não haver um conjunto de coalizões tal que, dado $v$, todos os agentes possam obter sua utilidade conforme prometido por $\vec{u}$. Para corrigir este problema, primeiro especificamos que estamos interessados apenas em resultados viáveis, isto é, aqueles que

FEASIBLE

pode ser implementado dado $v$.

*Definição 4.2 (Viável). Um resultado $\vec{u}$ é viável se existir um conjunto de coalizões $T = S_1, \ldots, S_k$ onde $\bigcup_{S \in T} S = A$ tal que $\sum_{S \in T} v(S) \geq \sum_{i \in A} \vec{u}_i$.*

Ou seja, um resultado $\vec{u}$ é viável se pudermos encontrar um conjunto disjunto de coalizões cujos valores sejam iguais aos de $\vec{u}$, para que possamos compensar $\vec{u}$ com $v$. O conjunto de coalizões disjuntas $T$ definido acima também é frequentemente chamado de estrutura de coalizão, também

COALITION STRUCTURE

às vezes representado com o símbolo $CS$.

Observe que, se a função característica for superaditiva, então podemos verificar se um resultado $\vec{u}$ é viável simplesmente garantindo que

$$\sum_{i \in A} \vec{u}_i = v(A). \tag{4.1}$$

Definimos um domínio superaditivo como aquele onde, para todos os pares de coalizões disjuntas $S, T \subset A$, temos que $v(S \cup T) \geq v(S) + v(T)$. Ou seja, não há nada a perder com a fusão numa coligação maior. Infelizmente, os sistemas multiagentes raramente são aditivos, uma vez que os agentes têm o hábito de se atrapalhar uns com os outros, de modo que uma equipe nem sempre é melhor do que deixar cada agente trabalhar em uma tarefa separadamente.

SUPER-ADDITIVE

O problema de encontrar soluções viáveis pode ser melhor ilustrado com um exemplo. A Figura 4.1 mostra um exemplo de jogo de utilidade transferível para três agentes, juntamente com a definição da função $v$ e todas as possíveis formações de coalizão. Neste jogo o resultado $\vec{u} = \{5, 5, 5\}$ não é viável, pois não há como dividir os agentes em subconjuntos de modo que todos possam obter sua utilidade. Se tentarmos a coligação (123) então só temos um valor de 9 para distribuir e precisamos de um total de 15. Por outro lado, o resultado $\vec{u} = \{2, 4, 3\}$ é viável porque as estruturas de coligação (1)(23), (2)(13) e (123) podem satisfazê-lo (mas não as outras). No entanto, $\vec{u} = \{2, 4, 3\}$ tem um problema porque o agente 3 está obtendo uma utilidade de 3 enquanto temos isso $v((3)) = 4$. Ou seja, o agente 3 poderia desertar qualquer uma das três estruturas da coalizão

| $S$ | $v(S)$ |
|---|---|
| (1) | 1 |
| (2) | 2 |
| (3) | 2 |
| (12) | 4 |
| (13) | 3 |
| (23) | 4 |
| (123) | 6 |

(1)(2)(3)
$1+2+2=5$

(1)(23)
$1+4=5$

(2)(13)
$2+3=5$

(3)(12)
$2+4=6$

| $\vec{u}$ | in Core? |
|---|---|
| $\{2,2,2\}$ | *yes* |
| $\{2,2,3\}$ | *no* |
| $\{1,2,2\}$ | *no* |

(123)
6

**Figure 4.2:** Sample game showing some outcomes that are in the core and some that are not. The characteristic function $v$ is super-additive.

we found, join the coalition (3), and get a higher utility than he currently has. This outcome thus seems unstable.

### The Core

In general, we say that an outcome $\vec{u}$ is stable if no subset of agents gets paid more, as a whole, than what they get paid in $\vec{u}$. Stability is a nice property because it means that the agents do not have an incentive to go off into their own coalition. Our first solution concept, the **core**, refers to all the outcomes that are stable.

CORE

**Definition 4.3** (Core)**.** *An outcome $\vec{u}$ is in the* core *if*

1. $$\forall_{S \subset A} : \sum_{i \in S} \vec{u}_i \geq v(S)$$

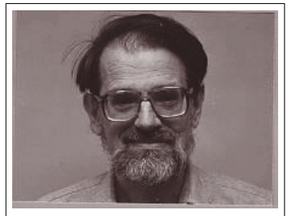
2. *it is feasible.*

The first condition in this definition tells us that the utility the agents receive in outcome $\vec{u}$ is bigger than those of any coalition, for the agents in the coalition. In other words, that there is no coalition $S$ whose $v(S)$ is bigger than the sum of payments the agents in $S$ get under $\vec{u}$. The second condition merely checks that the total utility we are giving out is not more than what is coming in via $v(\cdot)$.

Figure 4.2 shows a new game with different payments and a list of outcomes some of which are in the core and some which are not. $\{2,2,2\}$ is in the core because it is feasible and there is no subset of agents $S$ with a $v(S)$ that is bigger than what they could get in this outcome. $\{2,2,3\}$ is not in the core because it is not feasible. This outcome adds up to 7 and there is no coalition structure that adds up to 7. Note that, since the $v$ is super-additive, all we need to check is the grand coalition. Finally, $\{1,2,2\}$ is not in the core because agents 1 and 2 are getting a total of 3 while if they formed the coalition (12) they would get a utility of 4.

We like the core because we know that any solution that is in the core cannot be improved by having any of the $2^A$ subsets of agents form a coalition of higher value than they are getting now. It is a very stable solution. Unfortunately, there are many games with empty cores. Figure 4.3 shows one such example. Try to find an outcome in the core for this example. You will see that every outcome is blocked by some other outcome.

Another problem with the core solutions is that when it is not empty then there are often many outcomes in the core. For example, in figure 4.2 any outcome $\vec{u} = \{x, y, z\}$ where $x + y + z \leq 6$ and $x + y \geq 4$ and $x + z \geq 3$ and $y + z \geq 4$ is in the core. Thus, we still have a coordination or negotiation problem where we must choose between these coalitions. Finally, there is the fact that the outcome does not tell us which coalitions are formed. For example, if we choose the outcome $\vec{u} = \{2,2,2\}$ then we must still also choose a coalition structure as there are two which would work: (123) and (12)(3).



Lloyd F. Shapley. 1923–. Responsible for the core and Shapley value solution concepts.



| $S$ | $v(S)$ |
|---|---|
| (1) | 1 |
| (2) | 2 |
| (3) | 2 |
| (12) | 4 |
| (13) | 3 |
| (23) | 4 |
| (123) | 6 |

(1)(2)(3)
$1+2+2=5$

(1)(23)
$1+4=5$

(2)(13)
$2+3=5$

(3)(12)
$2+4=6$

| $\vec{u}$ | no Núcleo? |
|---|---|
| $\{2,2,2\}$ | *yes* |
| $\{2,2,3\}$ | *no* |
| $\{1,2,2\}$ | *no* |

(123)
6

Figura 4.2: Exemplo de jogo mostrando alguns resultados que estão no núcleo e outros que não estão. A função característica $v$ é superaditiva.

descobrimos, junte-se à coalizão (3) e obtenha uma utilidade maior do que a que tem atualmente. Este resultado parece, portanto, instável.

#### O Núcleo

Em geral, dizemos que um resultado $\vec{u}$ é estável se nenhum subconjunto de agentes recebe mais, como um todo, do que recebe $\vec{u}$. A estabilidade é uma boa propriedade porque sig nifica que os agentes não têm incentivo para aderir à sua própria coligação. Nosso primeiro conceito de solução, o núcleo, refere-se a todos os resultados que são estáveis.

CORE

*Definição 4.3 (Núcleo). Um resultado $\vec{u}$ está no núcleo se*

1. $$\forall_{S \subset A} : \sum_{i \in S} \vec{u}_i \geq v(S)$$

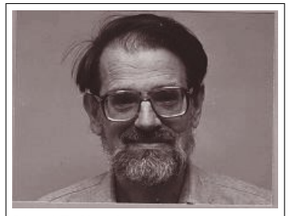
2. *é viável.*

A primeira condição nesta definição diz-nos que a utilidade que os agentes recebem no resultado $\vec{u}$ é maior do que a de qualquer coligação, para os agentes da coligação. Em outras palavras, que não existe nenhuma coalizão $S$ cuja $v(S)$ seja maior que a soma dos pagamentos que os agentes em $S$ recebem $\vec{u}$. A segunda condição apenas verifica se a utilidade total que estamos distribuindo não é maior do que a que chega via $v(\cdot)$.

A Figura 4.2 mostra um novo jogo com diferentes pagamentos e uma lista de resultados, alguns dos quais estão no núcleo e outros não. $\{2,2,2\}$ está no núcleo porque é viável e não há nenhum subconjunto de agentes $S$ com um $v(S)$ que seja maior do que o que eles poderiam obter neste resultado. $\{2,2,3\}$ não está no núcleo porque não é viável. Este resultado soma 7 e não há estrutura de coalizão que soma 7. Observe que, como o $v$ é superaditivo, tudo o que precisamos verificar é a grande coalizão. Finalmente, $\{1,2,2\}$ não está no núcleo porque os agentes 1 e 2 estão obtendo um total de 3, enquanto se formassem a coalizão (12) obteriam uma utilidade de 4.

Gostamos do núcleo porque sabemos que qualquer solução que esteja nele não pode ser melhorada fazendo com que qualquer um dos $2^A$ subconjuntos de agentes formem uma coalizão de valor mais alto do que o que estão obtendo agora. É uma solução muito estável. Infelizmente, existem muitos jogos com núcleos vazios. A Figura 4.3 mostra um exemplo. Tente encontrar um resultado no núcleo deste exemplo. Você verá que todo resultado é bloqueado por algum outro resultado.

Outro problema com as soluções principais é que, quando não está vazio, muitas vezes há muitos resultados no núcleo. Por exemplo, na figura 4.2, qualquer resultado $\vec{u} = \{x, y, z\}$ onde $x + y + z \leq 6$ e $x + y \geq 4$ e $x + z \geq 3$ e $y + z \geq 4$está no núcleo. Assim, ainda temos um problema de coordenação ou negociação onde devemos escolher entre estas coligações. Finalmente, há o facto de o resultado não nos dizer quais as coligações formadas. Por exemplo, se escolhermos o resultado$\vec{u} = \{2,2,2\}$ então também devemos escolher uma estrutura de coligação, pois há duas que funcionariam: (123) e (12)(3).



Lloyd F. Shapley. 1923–. Responsável pelos conceitos de solução de valor thecore e Shapley.

| $S$ | $v(S)$ |
|---|---|
| (1) | 0 |
| (2) | 0 |
| (3) | 0 |
| (12) | 10 |
| (13) | 10 |
| (23) | 10 |
| (123) | 10 |

**Figure 4.3:** Set of payments for a game with an empty core

| $S$ | $v(S)$ |
|---|---|
| () | 0 |
| (1) | 1 |
| (2) | 3 |
| (12) | 6 |

**Figure 4.4:** Example game. If the agents form coalition (12) then how much utility should each one get?

### The Shapley Value

While the core gives us one possible solution, it suffers from the fact that many games don't have any solutions in the core and from its lack of guidance in fairly distributing the payments from a coalition to its members. The Shapley value solves these problems by giving us one specific set of payments for coalition members, which are deemed fair.

The problem with identifying fairness in characteristic form games is best illustrated by an example. Figure 4.4 shows a game for two players. Clearly, we should choose the coalition (12) as it has the highest value. Now we must decide how much each agent should get. The simplest solution is to divide the total of 6 evenly amongst the coalition members, so that each agent gets 3. This seems unfair to agent 2 because agent 2 could have gotten 3 by simply staying on its own coalition (2). It seems like the fair thing to do is to give each agent a payment that is proportional to the value it contributes to the coalition, that is, the amount that value increases by having the agent in the coalition. But, how do we extend this idea to cases with more than 2 agents?

Shapley was able to extend this idea by realizing that each agent should get a payment that corresponds to its marginal contribution to the final value. An agent's marginal contribution to a coalition is the difference between the value before the agent joins the coalition and after he joined. For example, if before you join Initech their annual profits are $10M but after you are there for a year they increase to $11M then you can claim that your marginal contribution is to Initech is $1M assuming, of course, that everything else stays the same during that year.

The one remaining problem is that there are many different orderings in which $n$ agents could have joined the coalition, namely, there are $n!$ orderings of $n$ elements. The **Shapley value** simply averages over all possible orderings. That is, the Shapley value gives each agent a utility proportional to its average marginal contribution to every possible coalition, in every possible order it could have been formed. More formally, we define the Shapley value as:

Shapley value

**Definition 4.4** (Shapley Value)**.** *Let $B(\pi, i)$ be the set of agents in the agent ordering $\pi$ which appear before agent $i$. The* Shapley value *for agent $i$ given $A$ agents is given by*

$$\phi(A, i) = \frac{1}{A!} \sum_{\pi \in \Pi_A} v(B(\pi, i) \cup i) - v(B(\pi, i)),$$

*where $\Pi_A$ is the set of all possible orderings of the set $A$. Another way to express the same formula is*

$$\phi(A, i) = \sum_{S \subseteq A} \frac{(|A| - |S|)!\,(|S| - i)!}{|A|!} [v(S) - v(S - \{i\})].$$



| $S$ | $v(S)$ |
|---|---|
| (1) | 0 |
| (2) | 0 |
| (3) | 0 |
| (12) | 10 |
| (13) | 10 |
| (23) | 10 |
| (123) | 10 |

Figura 4.3: Conjunto de pagamentos para um jogo com núcleo vazio

| $S$ | $v(S)$ |
|---|---|
| () | 0 |
| (1) | 1 |
| (2) | 3 |
| (12) | 6 |

Figura 4.4: Exemplo de jogo. Se os agentes formarem uma coalizão (12), então quanta utilidade cada um deverá obter?

### O valor Shapley

Embora o núcleo nos dê uma solução possível, ele sofre com o fato de que muitos jogos não têm nenhuma solução no núcleo e com a falta de orientação na distribuição justa dos pagamentos de uma coalizão aos seus membros. O valor Shapley resolve estes problemas, dando-nos um conjunto específico de pagamentos para os membros da coligação, que são considerados justos.

O problema de identificar a justiça em jogos de formas características é melhor ilustrado por um exemplo. A Figura 4.4 mostra um jogo para dois jogadores. Claramente, devemos escolher a coligação (12) porque tem o valor mais elevado. Agora devemos decidir quanto cada agente deverá receber. A solução mais simples é dividir o total de 6 igualmente entre os membros da coligação, de modo que cada agente obtenha 3. Isto parece injusto para o agente 2 porque o agente 2 poderia ter obtido 3 simplesmente permanecendo na sua própria coligação (2). Parece que a coisa justa a fazer é dar a cada agente um pagamento que seja proporcional ao valor que contribui para a coligação, ou seja, o montante que o valor aumenta por ter o agente na coligação. Mas como estender esta ideia aos casos com mais de 2 agentes?

Shapley conseguiu ampliar esta ideia ao perceber que cada agente deveria receber um pagamento que correspondesse à sua contribuição marginal para o valor final. A contribuição marginal de um agente para uma coalizão é a diferença entre o valor antes de o agente ingressar na coalizão e depois de ele ingressar. Por exemplo, se antes de ingressar na Initech seus lucros anuais são de US$ 10 milhões, mas depois de você permanecer lá por um ano eles aumentam para US$ 11 milhões, então você pode alegar que sua contribuição marginal para a Initech é de US$ 1 milhão, assumindo, é claro, que todo o resto permanece o mesmo durante aquele ano.

O único problema que resta é que existem muitas ordens diferentes nas quais os agentes $n$ poderiam ter se juntado à coalizão, ou seja, existem $n!$ ordenações de $n$ elementos. Os valores de Shapley simplesmente calculam a média de todas as ordenações possíveis. Ou seja, o Shapley

Shapley value

o valor dá a cada agente uma utilidade proporcional à sua contribuição marginal média para todas as coligações possíveis, em todas as ordens possíveis em que poderiam ter sido formadas. Mais formalmente, definimos o valor de Shapley como:

*Definição 4.4 (Valor Shapley). Seja $B(\pi, i)$ o conjunto de agentes na ordem de agentes $\pi$ que aparecem antes do agente $i$. O valor Shapley para agente $i$ dado $A$ agentes é dado por*

$$\phi(A, i) = \frac{1}{A!} \sum_{\pi \in \Pi_A} v(B(\pi, i) \cup i) - v(B(\pi, i)),$$

*onde $\Pi_A$ é o conjunto de todas as ordenações possíveis do conjunto $A$. Outra maneira de expressar a mesma fórmula é*

$$\phi(A, i) = \sum_{S \subseteq A} \frac{(|A| - |S|)!\,(|S| - i)!}{|A|!} [v(S) - v(S - \{i\})].$$

Notice that the Shapley values are calculated for a particular coalition $A$ in the definition above. They are not meant as a way of determining which is the best coalition structure. They can only be used to distribute the payments of a coalition once it is formed.

Lets calculate the Shapley values for the game in figure 4.4 and the grand coalition (12). Since there are only two agents it means that there are only two possible orderings: (12) and (21). As such we have that

$$\begin{aligned}\phi(\{1,2\},1) &= \frac{1}{2}\cdot(v(1)-v()+v(21)-v(2))\\ &= \frac{1}{2}\cdot(1-0+6-3)=2\\ \phi(\{1,2\},2) &= \frac{1}{2}\cdot(v(12)-v(1)+v(2)-v())\\ &= \frac{1}{2}\cdot(6-1+3-0)=4\end{aligned}$$

A somewhat surprising and extremely useful characteristic of the Shapley value is that it is always feasible. In our example the payments of 4 and 2 add up to 6 which is the same value we get in the grand coalition (12). Another nice feature of the Shapley value is that it always exists and is unique. Thus, we do not have to worry about coordination mechanism to choose among different payments. A final interesting result is that the Shapley value might not be in the core, even for cases where the core exists. This is a potential problem as it means that the resulting payments might not be stable and some agents might choose to leave the coalition in order to receive a higher payment on a different coalition.

Unfortunately, while the Shapley value has some very attractive theoretical properties, it does have some serious drawbacks when we try to use it for building multiagent systems. The biggest problem is computational. The Shapley value requires us to calculate at least $2^{|A|}$ orderings, this is only possible for very small sets $A$. It also requires that we know the value of $v$ for every single subset $S$. In many real-world applications the calculation of $v$ is complex. For example, it might require simulating how a particular coalition of agents would work together. These complex calculations could dramatically increase the total time. Finally, the Shapley value does not give us the actual coalition structure. Thus, it only solves the second part of the coalition formation problem. We must still determine which coalition the agents will form and how they will do it.

### The Nucleolus

Since the core is often empty, researchers started looking for ways of relaxing it and find a new solution concept that would exist for every game. The problem with the core is that it says that there is no subset of agents that could get paid more than what they are currently getting paid in $\vec{u}$, because then they would be tempted to defect and form a new coalition. If it is impossible to find such an $\vec{u}$ then the next best thing would be to find the $\vec{u}$ that minimizes the total temptation felt by the agents. That is what the **nucleolus** aims to do.

NUCLEOLUS

We start by clarifying what we mean by temptation. Specifically, a coalition $S$ is more tempting the higher its value is over what the agents get in $\vec{u}$. This is known as the **excess**.

EXCESS

**Definition 4.5** (excess)**.** *The* excess *of coalition $S$ given outcome $\vec{u}$ is given by*

$$e(S,\vec{u}) = v(S) - \vec{u}(S),$$

*where*

$$\vec{u}(S) = \sum_{i\in S}\vec{u}_i.$$



Observe que os valores de Shapley são calculados para uma coalizão específica $A$ na definição acima. Não pretendem ser uma forma de determinar qual é a melhor estrutura de coligação. Só podem ser utilizados para distribuir os pagamentos de uma coligação depois de esta estar formada.

Vamos calcular os valores de Shapley para o jogo da figura 4.4 e a grande coalizão (12). Como existem apenas dois agentes, significa que existem apenas duas ordenações possíveis: (12) e (21). Como tal temos que

$$\begin{aligned}\phi(\{1,2\},1) &= \frac{1}{2}\cdot(v(1)-v()+v(21)-v(2))\\ &= \frac{1}{2}\cdot(1-0+6-3)=2\\ \phi(\{1,2\},2) &= \frac{1}{2}\cdot(v(12)-v(1)+v(2)-v())\\ &= \frac{1}{2}\cdot(6-1+3-0)=4\end{aligned}$$

Uma característica um tanto surpreendente e extremamente útil do valor de Shapley é que ele é sempre viável. No nosso exemplo, os pagamentos de 4 e 2 somam 6, que é o mesmo valor que obtemos na grande coligação (12). Outra característica interessante do valor Shapley é que ele sempre existe e é único. Assim, não precisamos nos preocupar com mecanismos de coordenação para escolher entre diferentes pagamentos. Um resultado final interessante é que o valor de Shapley pode não estar no núcleo, mesmo para os casos em que o núcleo existe. Isto é um problema potencial, pois significa que os pagamentos resultantes podem não ser estáveis e alguns agentes podem optar por deixar a coligação para receber u m pagamento mais elevado numa coligação diferente.

Infelizmente, embora o valor de Shapley tenha algumas propriedades teóricas muito atraentes, ele apresenta algumas sérias desvantagens quando tentamos usá-lo para construir sistemas multiagentes. O maior problema é computacional. O valor de Shapley exige que calculemos pelo menos $2^{|A|}$ ordenações, isso só é possível para conjuntos muito pequenos $A$. Também requer que saibamos o valor de $v$ para cada subconjunto $S$. Em muitas aplicações do mundo real, o cálculo de $v$ é complexo. Por exemplo, pode ser necessário simular como uma coligação específica de agentes trabalharia em conjunto. Esses cálculos complexos podem aumentar drasticamente o tempo total. Finalmente, o valor de Shapley não nos dá a estrutura real da coligação. Assim, resolve apenas a segunda parte do problema da formação de coligações. Ainda temos de determinar que coligação os agentes formarão e como o farão.

### O Nucléolo

Como o núcleo geralmente está vazio, os pesquisadores começaram a procurar maneiras de relaxá-lo e encontrar um novo conceito de solução que existisse para cada jogo. O problema com o núcleo é que ele diz que não existe nenhum subconjunto de agentes que poderia receber mais do que recebem atualmente $\vec{u}$, porque então seriam tentados a desertar e formar uma nova coalizão. Se for impossível encontrar tal $\vec{u}$ e ntão a próxima melhor coisa seria encontrar o $\vec{u}$ que minimize a tentação total sentida pelos agentes. É isso que o nucléolo pretende fazer.

NUCLEOLUS

Começamos esclarecendo o que queremos dizer com tentação. Especificamente, uma coalizão $S$ é mais tentadora quanto maior for o seu valor em relação ao que os agentes recebem $\vec{u}$. Isso é conhecido como excesso.

EXCESS

*Definição 4.5 (excesso). O excesso de coalizão $S$ resultado dado $\vec{u}$ é dado por*

$$e(S,\vec{u}) = v(S) - \vec{u}(S),$$

*onde*

$$\vec{u}(S) = \sum_{i\in S}\vec{u}_i.$$

That is, a coalition $S$ has a positive excess, given $\vec{u}$, if the agents in $S$ can get more from $v(S)$ than they can from $\vec{u}$. The more they can get from $S$ the higher the excess. Note that, by definition, if an outcome $\vec{u}$ is in the core then all coalitions have a excess that is less than or equal to 0 with respect to that outcome. But, since we are now concerned with outcomes that are not in the core we will instead look for those with minimal excess. Since excess is defined for all possible subsets $S$ we first need a way to compare the excesses of two outcomes. We do this by putting them in a sorted list and comparing the list. The one with the higher excess first is declared more excessive. More formally, for each $\vec{u}$ we find its excess for all subsets $S$ and order these in a list; the higher excesses come first, as such

$$\theta(\vec{u}) = \langle e(S_1^{\vec{u}}, \vec{u}), e(S_2^{\vec{u}}, \vec{u}), \ldots, e(S_{2^{|A|}}^{\vec{u}}, \vec{u})\rangle, \tag{4.2}$$

where $e(S_i^{\vec{u}}, \vec{u}) \geq e(S_j^{\vec{u}}, \vec{u})$ for all $i < j$. We then define a lexicographical ordering $\succ$ over these lists where $\theta(\vec{u}) \succ \theta(\vec{v})$ is true when there is some number $q \in 1 \ldots 2^{|A|}$ such for all $p < q$ we have that $e(S_p^{\vec{u}}, \vec{u}) = e(S_p^{\vec{v}}, \vec{v})$ and $e(S_q^{\vec{u}}, \vec{u}) > e(S_q^{\vec{v}}, \vec{v})$ where the $S_i$ have been sorted as per $\theta$. That is, if $\theta(\vec{u}) \succ \theta(\vec{v})$ then that means that when we sort their excesses for all subsets their first, and greatest, excesses are all the same and on the first set for which they have a discrepancy $\vec{u}$ has the highest excess. For example, if we had the lists $\{(2,2,2),(2,1,0),(3,2,2),(2,1,1)\}$ they would be ordered as $\{(3,2,2),(2,2,2),(2,1,1),(2,1,0)\}$.

NUCLEOLUS

We can now define the **nucleolus** as the $\vec{u}$ which is not lexicographically bigger than anyone else.

**Definition 4.6** (nucleolus)**.** *The* nucleolus *is the set*

$$\{\vec{u} \,|\, \theta(\vec{u}) \not\succ \theta(\vec{v}) \text{ for all } \vec{v}, \text{ given that } \vec{u} \text{ and } \vec{v} \text{ are feasible.}\}$$

In other words, the outcomes in the nucleolus are those where the excesses for all possible sets are lexicographically not greater than those of any other outcome. A nice feature of the nucleolus is that it is always unique for each coalition structure. That is, given a coalition structure there is only one nucleolus.

The nucleolus captures, to some degree, the idea of an outcome that minimizes the temptation the agents face. However, notice that the lexicographic order it defines only cares about the first coalition that has a higher excess, it does not care about the ones after that. This could lead to a situation where the sum of the excesses from the nucleolus is actually larger than that of some other outcome. For example, $(5.0, 0, 0)$ comes before $(4, 3, 3, 3)$. As such, the nucleolus does not seem to minimize the sum of temptations.

## Equal Excess

EQUAL EXCESS

Another technique for calculating the agents' payoff, besides the Shapley value, is called **equal excess**. It is an iterative algorithm where we adjust the payments that the agents expected they will receive from each coalition that includes them. At each time step $t$ we let $E^t(i, S)$ be agent $i$'s expected payoff for each coalition $S$ which includes him. Initially these are set to 0. We thus let

$$A^t(i, S) = \max_{T \neq S} E^t(i, T) \tag{4.3}$$

be agent $i$'s expected payment from not choosing $S$ and instead choosing the best alternative coalition. Then, at each time step we update the players' expected payments using

$$E^{t+1}(i, S) = A^t(i, S) + \frac{v(S) - \sum_{j \in S} A^t(j, S)}{|S|}. \tag{4.4}$$

For example, for the value function given in figure 4.4 we start with $E^0(1, *) = E^0(2, *) = 0$. Then, at time 1 agent 1 can update $E^1(1, (1)) = 1$ and $E^1(1, (12)) = 3$



Ou seja, uma coalizão $S$ tem um excesso positivo, dado $\vec{u}$, se os agentes em $S$ podem obter mais de $v(S)$ do que de $\vec{u}$. Quanto mais eles conseguirem, $S$ maior será o excesso. Observe que, por definição, se um resultado $\vec{u}$ está no núcleo, então todas as coalizões têm um excesso menor ou igual a 0 em relação a esse resultado. Mas, como agora estamos preocupados com resultados que não estão no núcleo, procuraremos aqueles com excesso mínimo. Como o excesso é definido para todos os subconjuntos possíveis, primeiro precisamos encontrar uma maneira de comparar os excessos de dois resultados. Fazemos isso colocando-os em uma lista ordenada e comparando a li sta. Aquele com maior excesso primeiro é declarado mais excessivo. Mais for malmente, para cada $\vec{u}$ encontramos seu excesso para todos os subconjuntos$S$ e os ordenamos em uma lista; os excessos maiores vêm primeiro, como tal

$$\theta(\vec{u}) = \langle e(S_1^{\vec{u}}, \vec{u}), e(S_2^{\vec{u}}, \vec{u}), \ldots, e(S_{2^{|A|}}^{\vec{u}}, \vec{u})\rangle, \tag{4.2}$$

onde $e(S_i^{\vec{u}}, \vec{u}) \geq e(S_j^{\vec{u}}, \vec{u})$ para todos $i < j$. Definimos então uma ordem lexicográfica $\succ$ sobre essas listas onde $\theta(\vec{u}) \succ \theta(\vec{v})$ é verdadeiro quando há algum número $q \in 1 \ldots 2^{|A|}$ tal para todos $p < q$temos que $e(S_p^{\vec{u}}, \vec{u}) = e(S_p^{\vec{v}}, \vec{v})$ e $e(S_q^{\vec{u}}, \vec{u}) > e(S_q^{\vec{v}}, \vec{v})$ onde $S_i$ foram classificados conforme $\theta$. Ou seja, se $\theta(\vec{u}) \succ \theta(\vec{v})$ então isso significa que quando classificamos seus excessos para todos os subconjuntos, seus primeiros e maiores excessos são todos iguais e no primeiro conjunto para o qual eles têm uma discrepância $\vec{u}$ tem o maior excesso. Por exemplo, se tivéssemos as listas {(2, 2, 2), (2, 1, 0), (3, 2, 2), (2, 1, 1)} elas seriam ordenadas como {(3, 2, 2), (2, 2, 2), (2, 1, 1), (2, 1, 0)}.

NUCLEOLUS

Podemos agora definir o nucléolo como $\vec{u}$ que não é lexicograficamente maior do que qualquer outra pessoa.

Definição 4.6 (nucléolo). O nucléolo é o conjunto

$$\{\vec{u} \,|\, \theta(\vec{u}) \not\succ \theta(\vec{v}) \text{ for all } \vec{v}, \text{ given that } \vec{u} \text{ and } \vec{v} \text{ are feasible.}\}$$

Em outras palavras, os resultados no nucléolo são aqueles em que os excessos para todos os conjuntos possíveis não são lexicograficamente maiores do que os de qualquer outro resultado. Uma característica interessante do nucléolo é que ele é sempre único para cada estrutura de coalizão. Isto é, dada uma estrutura de coligação existe apenas um nucléolo.

O nucléolo capta, até certo ponto, a ideia de um resultado que minimiza a tentação que os agentes enfrentam. Porém, observe que a ordem lexicográfica que ela define só se preocupa com a primeira coalizão que tem maior excesso, não se preocupa com as seguintes. Isto poderia levar a uma situação em que a soma dos excessos do nucléolo fosse realmente maior do que a de algum outro resultado. Por exemplo, (5,0, 0, 0) vem antes de (4, 3, 3, 3). Como tal, o nucléolo não parece minimizar a soma das tentações.

### Excesso igual

EQUAL EXCESS

Outra técnica para calcular o payoff dos agentes, além do valor de Shapley, é chamada de excesso igual. É um algoritmo iterativo onde ajustamos os pagamentos que os agentes esperavam receber de cada coligação que os inclui. Em cada passo de tempo $t$ deixamos $E^t(i, S)$ ser o retorno esperado do agente $i$ para cada coalizão $S$ que o inclui.

$$A^t(i, S) = \max_{T \neq S} E^t(i, T) \tag{4.3}$$

seja o pagamento esperado do agente $i$ por não escolher $S$ e, em vez disso, escolher a melhor coalizão alternativa. Então, a cada passo, atualizamos os pagamentos esperados dos jogadores usando

$$E^{t+1}(i, S) = A^t(i, S) + \frac{v(S) - \sum_{j \in S} A^t(j, S)}{|S|}. \tag{4.4}$$

Por exemplo, para a função de valor dada na figura 4.4, começamos com $E^0(1, *) = E^0(2, *) = 0$. Então, no momento 1, o agente 1 pode atualizar $E^1(1, (1)) = 1$ e $E^1(1, (12)) = 3$

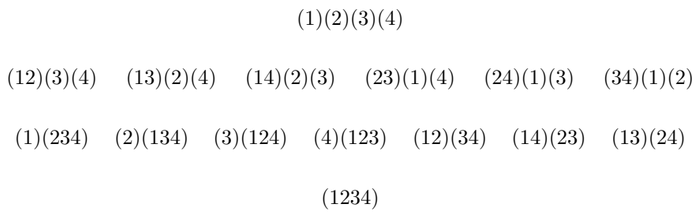



**Figure 4.5:** Coalition structure formation possibilities for four agents, organized by the number of coalitions.

| Level | Bound |
|---|---|
| $A$ | $A/2$ |
| $A-1$ | $A/2$ |
| $A-2$ | $A/3$ |
| $A-3$ | $A/3$ |
| $A-4$ | $A/4$ |
| $A-5$ | $A/4$ |
| : | : |
| 2 | $A$ |
| 1 | none |

**Figure 4.6:** Bounds on optimality after searching various levels.

and then $A^1(1,(1)) = 3$ and $A^1(1,(12)) = 1$, while agent 2 updates $E^1(2,(2)) = 3$ and $E^1(2,(12)) = 3$ and then $A^1(1,(1)) = 3$ and $A^1(1,(12)) = 3$.

It has been shown that this basic algorithm does not always converge to a fixed point, however, variations of it have been proposed which do converge, such as PACT (Goradia and Vidal, 2007). In PACT the agents calculate their own $E$ values and exchange them with others under the assumption that agents will not lie about these values. The algorithm ensures that the process will stop and a solution will be found.

Notice that equal excess is a procedural solution to the problem so we do not know which specific outcome the agents will converge to, other than to say that they will converge to the outcome that is found when we use equal excess.

### 4.1.2 Finding the Optimal Coalition Structure

As multiagent system designers we often simply want to find the outcome that maximizes the sum of values. That is, we want to find the **utilitarian** solution. When the characteristic function is super-additive then the grand coalition will have the highest value and thus finding the utilitarian solution is trivial. However, if the characteristic is not super-additive—as is often the case in multiagent systems—then we will want an algorithm for finding it. Notice that under this formulation we are no longer interested in the specific outcome (that is, individual payments to agents) we are now only interested in finding best coalition structure, ignoring the problem of dividing up the value of each coalition among its participants.

UTILITARIAN

#### Centralized Algorithm

One proposed approach is to a perform a complete search of the complete set of possible coalition structures, but in a specified order. Figure 4.5 shows all the possible coalition structures for four agents. Notice that the bottom two rows contain all possible coalitions. This means that after searching those two rows we have seen all possible coalitions. If we let $S^*$ be the value of the highest valued coalition (*not* coalition structure) found after searching those two rows then we know that the best coalition structure cannot be more than $A \cdot S^*$. As such, after searching the bottom two levels we can say that the optimal solution is no more than $A$ times better than the best solution we have found thus far.



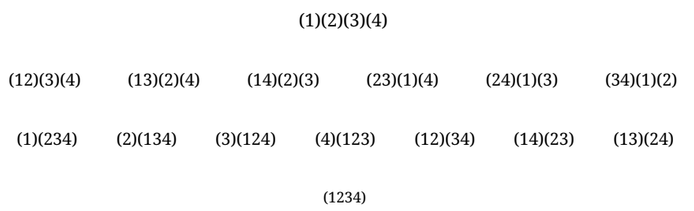



Figura 4.5: Possibilidades de formação de estrutura de coalizão para quatro agentes, organizadas pelo número de coalizões.

| Nível | Vinculado |
|---|---|
| $A$ | $A/2$ |
| $A-1$ | $A/2$ |
| $A-2$ | $A/3$ |
| $A-3$ | $A/3$ |
| $A-4$ | $A/4$ |
| $A-5$ | $A/4$ |
| : | : |
| 2 | $A$ |
| 1 | none |

Figura 4.6: Limites de otimização após pesquisa em vários níveis.

e então $A^1(1,(1)) = 3$ e $A^1(1,(12)) = 1$, enquanto o agente 2 atualiza $E^1(2,(2)) = 3$e $E^1(2,(12)) = 3$ e então $A^1(1,(1)) = 3$ e $A^1(1,(12)) = 3$.

Foi demonstrado que este algoritmo básico nem sempre converge para um ponto fixo, no entanto, foram propostas variações dele que convergem, como PACT (Goradia e Vidal, 2007). Em PACT os agentes calculam seus próprios valores $E$ e os trocam com outros sob a suposição de que os agentes não mentirão sobre esses valores. O algorit mo garante que o processo será interrompido e uma solução será encontrada.

Observe que o excesso igual é uma solução processual para o problema, portanto não sabemos para qual resultado específico os agentes irão convergir, a não ser para dizer que eles convergirão para o resultado encontrado quando usamos o excesso igual.

### 4.1.2 Encontrando a Estrutura de Coalizão Ideal

Como projetistas de sistemas multiagentes, muitas vezes queremos simplesmente encontrar o resultado que maximiza a soma dos valores. Isto é, queremos encontrar a solução utilitária.

Quando a função característica é superaditiva, então a grande coalizão terá o valor mais alto e, portanto, encontrar a solução utilitária é trivial. Contudo, se a característica não for superaditiva – como é frequentemente o caso em sistemas multiagentes – então precisare mos de um algoritmo para encontrá-la. Note-se que, sob esta formulação, já não estamos interessados no resultado específico (ou seja, pagamentos individuais aos agentes), estamos agora apenas interessados em encontrar a melhor estrutura de coligação, ignorando o problema de dividir o valor de cada coligação entre os seus participantes.

UTILITARIAN

#### Algoritmo Centralizado

Uma abordagem proposta é realizar uma busca completa do conjunto completo de possíveis estruturas de coalizão, mas em uma ordem específica. A Figura 4.5 mostra todas as estr uturas de coalizão possíveis para quatro agentes. Observe que as duas linhas inferiores contêm todas as coalizões possíveis. Isto significa que depois de pesquisar essas duas linhas vimos todas as coligações possíveis. Se deixarmos $S^*$ ser o valor da coalizão de maior valor (não da estrutura de coalizão) encontrada após pesquisar essas duas linhas, então sabere mos que a melhor estrutura de coalizão não pode ser maior que $A \cdot S^*$. Assim, depois de pesquisar os dois níveis inferiores, podemos dizer que a solução ótima não é mais do que $A$ vezes melhor do que a melhor solução que encontramos até agora.

FIND-COALITION($i$)
1 $L_i \leftarrow$ set of all coalitions that include $i$.
2 $S_i^* \leftarrow \arg\max_{S \in L_i} v_i(S)$
3 Broadcast $S_i^*$ and wait for all other broadcasts, put these into $S^*$ set.
4 $S_{max} \leftarrow \arg\max_{s \in S^*} v_i(s)$
5 **if** $i \in S_{\max}$
6 **then** join $S_{\max}$
7 **return**
8 **for** $j \in S_{\max}$
9 **do** Delete all coalitions in $L_i$ that contain $j$
10 **if** $L_i$ is not empty
11 **then** goto 2
12 **return**

**Figure 4.7:** Distributed algorithm for coalition formation. Each agent $i$ must execute this function. We let $v_i(S) = \frac{v(S)}{|S|}$

Figure 4.6 shows the bounds that can be calculated after examining each of the levels in the graph. One simple algorithm consists of first searching the bottom two levels then continue searching down from the top level (Sandholm et al., 1999). In this way, the bound from optimal is reduced as indicated in the figure. Note that searching the levels in some other order will not guarantee these bounds. Notice also that the number of coalition structures in the second level is given by its **Stirling** number

STIRLING

$$\text{Stirling}(A, 2) = \frac{1}{2}\sum_{i=0}^{1}(-1)^i\binom{2}{i}(2-i)^A = 2^{A-1} - 1. \quad (4.5)$$

So it takes exponential time just to search the second level. In general, the number of coalition structures for all levels is equal to the Stirling number for that level.

There also exists an algorithm for finding the optimal coalition structure which has slightly better bounds than the ones we just presented, but running time remains exponential and unusable for large number of agents (Dang and Jennings, 2004).

### Distributed Algorithm

find-coalition

While the previous algorithm found the optical coalition structure, it did so at the expense of a lot of computation and in a centralized manner. We now look at one possible way of finding a good, but possibly not optimal, coalition structure in a decentralized manner.

Figure 4.7 shows a distributed algorithm for coalition formation (Shehory and Kraus, 1998). The agents order all their possible coalitions based on how much each will get in that coalition, where each agent $i$ gets $v_i(S) = v(S)/|S|$ if it joins coalition $S$. The agents then broadcast the name of their best coalition. The coalition with maximal $v(S)/|S|$ is chosen by the agents in $S$ who join the coalition and drop out of the algorithm. The remaining agents take note of the missing agents by eliminating from consideration all coalitions that include them. The process is then repeated again with the new set of coalitions.

This is a classic example of a greedy or hill-climbing algorithm. As such, we know that it might get stuck on a local optima, which might or might not be the global optimum. Still, the algorithm should execute very fast as there are at most $A$ steps and each step involves having each agent examine all the possible coalitions that it can be participate in.

A slight modification of the algorithm would be to, instead of broadcasting at each time step we could let the agents meet randomly and form a coalition if $v_i(s)$ is maximal for all agents in $s$. Imagine the agents moving around in a space and forming sets whenever a group of them happen to be close to each other, then forming a coalition only if that set has a maximal value. This process is effectively the same as broadcasting except that it eliminates the need to broadcast at the expense of taking a longer time to converge (Sarne and Arponen, 2007).



FIND-COALITION(*i*)
1 $L_i \leftarrow$ conjunto de todas as coalizões que incluem $i$.2 $S_i^* \leftarrow$ arg max $_{S \in L_i} v_i(S)$ 3 Broadcast $S_i^*$ e espere por todas as outras transmissões, coloque-as em $S^*$ set.4 $S_{max} \leftarrow$ argmax$_{s \in S^*} v_i(s)$ 5 se $i \in S_{\max}$6 então junte $S_{\max}$7 return 8 para $j \in S_{\max}$9 do Exclua todas as coalizões em $L_i$ que contenham $j$10 se $L_i$ não estiver vazio 11 então vá para 2 12 retorne

Figura 4.7: Algoritmo distribuído para formação de coalizões. Cada agente $i$ deve executar esta função. Bem $v_i(S) = \frac{v(S)}{|S|}$

A Figura 4.6 mostra os limites que podem ser calculados após examinar cada um dos níveis do gráfico. Um algoritmo simples consiste em primeiro pesquisar nos dois níveis inferiores e depois continuar pesquisando a partir do nível superior (Sandholm et al., 1999). Desta forma, o limite do ideal é reduzido conforme indicado na figura. Observe que pesquisar os níveis em alguma outra ordem não garantirá esses limites. Observe também que o número de estruturas de coalizão no segundo nível é dado pelo seu Stirling

STIRLING

número

$$\text{Stirling}(A, 2) = \frac{1}{2}\sum_{i=0}^{1}(-1)^i\binom{2}{i}(2-i)^A = 2^{A-1} - 1. \quad (4.5)$$

Portanto, leva um tempo exponencial apenas para pesquisar o segundo nível. Em geral, o número de estruturas de coligação para todos os níveis é igual ao número de Stirling para esse nível.

Também existe um algoritmo para encontrar a estrutura de coalizão ideal que possui limites ligeiramente melhores do que os que acabamos de apresentar, mas o tempo de execução permanece exponencial e inutilizável para um grande número de agentes (Dang e Jennings, 2004).

### Algoritmo Distribuído

encontrar-coalizão

Embora o algoritmo anterior tenha encontrado a estrutura de coalizão óptica, ele o fez à custa de muita computação e de maneira centralizada. Analisamos agora uma forma possível de encontrar uma boa, mas possivelmente não óptima, estrutura de coligação de uma forma descentralizada.

A Figura 4.7 mostra um algoritmo distribuído para formação de coalizões (Shehory e Kraus, 1998). Os agentes ordenam todas as suas coalizões possíveis com base em quanto cada um receberá nessa coalizão, onde cada agente $i$ ganha $v_i(S) = v(S)/|S|$ se ingressar na coalizão$S$. Os agentes então transmitiram o nome da sua melhor coalizão. A coalizão com máximo $v(S)/|S|$ é escolhida pelos agentes em $S$ que se juntam à coalizão e abandonam o algoritmo. Os agentes restantes tomam nota dos agentes desaparecidos, eliminando da consideração todas as coalizões que os incluem. O processo é então repetido novamente com o novo conjunto de coligações.

Este é um exemplo clássico de algoritmo ganancioso ou de escalada. Como tal, sabemos que pode ficar preso num ótimo local, que pode ou não ser o ótimo global. Ainda assim, o algoritmo deve ser executado muito rápido, pois há no máximo$A$ etapas e cada etapa envolve fazer com que cada agente examine todas as possíveis coalizões das quais ele pode participar.

Uma ligeira modificação do algoritmo seria, em vez de transmitir a cada intervalo de tempo, poderíamos deixar os agentes se encontrarem aleatoriamente e formar uma coalizão se $v_i(s)$ for máximo para todos os agentes em $s$. Imagine os agentes se movimentando em um espaço e formando conjuntos sempre que um grupo deles estiver próximo um do outro, formando então uma coalizão apenas se esse conjunto tiver um valor máximo. Este processo é efectivamente o mesmo que a radiodifusão, excepto que elimina a necessidade de radiodifusão à custa de demorar mais tempo a convergir (Sarne e Arponen, 2007).

### Reduction to Constraint Optimization

We note that the problem of finding the optimal coalition structure can be reduced to a constraint optimization problem. The basic idea is that for $n$ agents there will be at most $n$ coalitions as, at worst, each agent will stay in the individual coalition. Thus, we can imagine the problem as consisting of $n$ agents each one deciding which of $n$ rooms to go into. The agents are the variables and the rooms are the domains. The agents in a room form a coalition and empty rooms are ignored. We set a constraint for each room equal to $-v(s)$ where $s$ is the set of agents that choose that room, or 0 if no agents choose it. We then have a constraint optimization problem where we are trying to the set of values which minimizes the sum of the constraint violations, thus maximizing the sum of the valuations.

Note that this is a degenerate case of the constraint optimization problem in that all the $n$ constraints are over all agents. Most constraint problems exhibit some degree of locality in that constraints are only over a small subset of the variables. Having all constraints be over all variables makes this problem harder than average. Thus this reduction is likely to be only of theoretical interest.

## 4.2 Coalition Formation



The **coalition formation** problem, as studied in multiagent systems, extends the basic characteristic form game in an effort to make it a better match for real world problems. A coalition formation problem consists of three steps.

1. Agents generate values for the $v(\cdot)$ function.
2. Agents solve the characteristic form game by finding a suitable set of coalitions.
3. Agents distribute the payments from these coalitions to themselves in a suitable manner.

Steps 2 and 3 can be thought of as part of the traditional characteristic form game. The coalition formation definition simply chooses to split the problem of finding a suitable outcome $\vec{u}$ into two parts: finding the coalitions and then dividing the payments. The split mirrors the requirements of many application domains. Step 1 is completely new. It is there because in many domains it is computationally expensive to determine the value of $v(S)$ for a given $S$. For example, if the agents are trying to form groups that solve particular tasks then calculating $v(S)$ requires them to determine out how well they can perform the task as a group, which requires considering how all their different skills can be brought to bear and, in some common scenarios, requires the development of a full plan—an exponential problem. The approaches at solving step 1 are thus generally dependent on the domain and do not generalize well.

### Exercises

4.1 Give an example problem in which agents using equal excess and reporting their own $E$ values will want to lie about their own $E$ values.

4.2 Find the set of core solutions and the Shapley value of the grand coalition for the following game:

| $S$ | $v(S)$ |
|---|---|
| (1) | 1 |
| (2) | 2 |
| (3) | 3 |
| (12) | 5 |
| (13) | 4 |
| (23) | 5 |
| (123) | 5 |



### Redução para otimização de restrições

Notamos que o problema de encontrar a estrutura de coalizão ótima pode ser reduzido a um problema de otimização de restrições. A idéia básica é que para $n$ agentes haverá no máximo $n$ coalizões, já que, na pior das hipóteses, cada agente permanecerá na coalizão individual. Assim, podemos imaginar o problema como consistindo em $n$ agentes, cada um decidindo em qual das $n$ salas entrar. Os agentes são as variáveis e as salas são os domínios. Os agentes numa sala formam uma coalizão e as salas vazias são ignoradas. Definimos uma restrição para cada sala igual a $-v(s)$ onde $s$ é o conjunto de agentes que escolhem aquela sala, ou 0 se nenhum agente a escolher. Temos então um problema de otimização de restrições onde tentamos encontrar o conjunto de valores que minimiza a soma das violações das restrições, maximizando assim a soma das avaliações.

Observe que este é um caso degenerado do problema de otimização de restrições, pois todas as $n$ restrições estão sobre todos os agentes. A maioria dos problemas de restrições exibe algum grau de localidade, pois as restrições se aplicam apenas a um pequeno subconjunto de variáveis. Ter todas as restrições sobre todas as variáveis torna esse problema mais difícil do que a média. Assim, é provável que esta redução tenha apenas interesse teórico.

## 4.2 Formação de Coalizão



O problema de formação de coalizões, conforme estudado em sistemas multiagentes, estende o características básicas do jogo em um esforço para torná-lo uma melhor combinação para problemas do mundo real. Um problema de formação de coalizão consiste em três etapas.

1. Os agentes geram valores para a função $v(\cdot)$. 2. Os agentes resolvem o jogo da forma característica encontrando um conjunto adequado de coalizões. 3. Os agentes distribuem entre si os pagamentos destas coligações de forma adequada.

As etapas 2 e 3 podem ser consideradas parte do tradicional jogo de formas características. A definição de formação de coligação opta simplesmente por dividir o problema de encontrar um resultado adequado $\vec{u}$ em duas partes: encontrar as coligações e depois dividir os pagamentos. A divisão reflete os requisitos de muitos domínios de aplicação. A etapa 1 é completamente nova. Existe porque em muitos domínios é computacionalmente caro determinar o valor de $v(S)$ para um deter minado $S$. Por exemplo, se os agentes estão tentando formar grupos que resolvem tarefas específicas, então o cálculo de $v(S)$ exige que eles determinem quão bem eles podem executar a tarefa como um grupo, o que requer considerar como todas as suas diferentes habilidades podem ser implementadas e, em alguns cenários comuns, requer o desenvolvimento de um plano completo – um problema exponencial. As abordagens para resolver a etapa 1 são, portanto, geralmente dependentes do domínio e não generalizam bem.

### Exercícios

4.1 Dê um exemplo de problema em que agentes que usam excesso igual e relatam seus próprios $E$ valores vão querer mentir sobre seus próprios valores $E$ .

4.2 Encontre o conjunto de soluções centrais e o valor de Shapley da grande coalizão para o seguinte jogo:

| $S$ | $v(S)$ |
|---|---|
| (1) | 1 |
| (2) | 2 |
| (3) | 3 |
| (12) | 5 |
| (13) | 4 |
| (23) | 5 |
| (123) | 5 |

4.3 Say you have robots which live in a 2-dimensional grid and each one has a strength given by a number in the set $\{1, 2, 3, 4, 5\}$. There are boxes in this world, each one of which must be moved to a specified destination. The speed with which a set of robots $S$ can move a box is given by $1 - 1/\sum_{i \in S} i$.strength.

- Formulate this problem as a characteristic form game and provide a $v(S)$ definition.
- Find a good algorithm for calculating the optimal coalition structure.

4.4 Modify the FIND-COALITION algorithm from figure 4.7 so that instead of the agents broadcasting their values they move around randomly in a two-dimensional space. After each time step the set of agents in a tile checks if they form a maximal coalition, that is, there is no other coalitions that gives one of the agents a higher $v_i(S)$ value. If so, they form that coalition and leave the game while the rest keep moving. Implement this algorithm in NetLogo and check how long it takes to find a solution.

4.5 We can extend the problem of coalition formation and make it more realistic by defining the value function over a set of possible agent abilities and then giving the agents sets of abilities (Yokoo et al., 2005). We then face the possibility of an agent pretending to be multiple agents, each with a different ability. Why would an agent do this? Give an example when an agent benefits from this technique.

Another problem might be agents that fail to mention to others that they have certain skills. Why would an agent do this? Give an example when an agent benefits from this technique.



4.3 Digamos que você tenha robôs que vivem em uma grade bidimensional e cada um t enha uma força dada por um número no conjunto $\{1, 2, 3, 4, 5\}$. Existem caixas neste mundo, cada uma das quais deve ser movida para um destino específico. A velocidade co m que um conjunto de robôs $S$ pode mover uma caixa é dada pela $1 - 1/\sum_{i \in S} i$.força.

- Formule este problema como uma forma característica de jogo e forneça uma definição $v(S)$.
- Encontre um bom algoritmo para calcular a estrutura de coalizão ideal.

4.4 Modifique o algoritmo FIND-COALITION da figura 4.7 para que, em vez de os agentes transmitirem seus valores, eles se movam aleatoriamente em um espaço bidimensional. Após cada passo de tempo o conjunto de agentes em um bloco verifica se eles formam uma coalizão máxima, ou seja, não há outras coalizões que dêem a u m dos agentes um valor $v_i(S)$ maior. Nesse caso, eles formam essa coalizão e abandonam o jogo enquanto os demais continuam em movimento. Implemente este algoritmo no NetLogo e verifique quanto tempo leva para encontrar uma solução.

4.5 Podemos ampliar o problema da formação de coalizões e torná-lo mais realista definindo a função de valor sobre um conjunto de possíveis habilidades dos agentes e, em seguida, dando aos agentes conjuntos de habilidades (Yokoo et al., 2005). Enfrentamos então a possibilidade de um agente fingir ser múltiplos agentes, cada u m com uma habilidade diferente. Por que um agente faria isso? Dê um exemplo de quando um agente se beneficia desta técnica.

Outro problema pode ser os agentes que não mencionam aos outros que possue m certas habilidades. Por que um agente faria isso? Dê um exemplo de quando um agente se beneficia desta técnica.

Chapter 5

# Learning in Multiagent Systems

Machine learning algorithms have achieved impressive results. We can write software that processes larger amounts of data than any human can and which can learn to find patterns that escape even the best experts in the field. As such, it is only reasonable that at some point we will want to add learning agents to our multiagent system. There are several scenarios in which one might want to add these learning agents.

Many multiagent systems have as their goal the exploration or monitoring of a given space, where each agent has only a local view of its own area. In these scenarios we can envision that each agent learns a map of its world and the agents further share their maps in order to aggregate a global view of the field and cooperatively decide which areas need further exploration. This is a form of cooperative learning.

Another scenario is in competitive environments each selfish agent tries to maximize its own utility by learning the other agents' behaviors and weaknesses. In these environments we are interested in the dynamics of the system and in determining if the agents will reach a stable equilibrium. At their simplest these scenarios are repeated games with learning agents.

To summarize, agents might learn because they don't know everything about their environment or because they don't know how the other agents behave. Furthermore, the learning can happen in a cooperative environment where we also want the agents to share their learned knowledge, or in a competitive environment where we want them to best each other. We present analysis and algorithms for learning agents in these various environment.

## 5.1 The Machine Learning Problem

Before delving into multiagent learning we first present a high level view of what we mean by **machine learning**. The word "learning" as used casually can have many different meanings, from remembering to deduction, but machine learning researchers have a very specific definition of the machine learning problem.

The goal of machine learning research is the development of algorithms that increase the ability of an agent to match a set of inputs to their corresponding outputs (Mitchell, 1997). That is, we assume the existence of a large set of examples $E$. Each example $e \in E$ is a pair $e = \{a, b\}$ where $a \in A$ represents the input the agent receives and $b \in B$ is the output the agent should produce when receiving this input. The agent must find a function $f$ which maps $A \to B$ for as many examples of $A$ as possible. For example, $A$ could be a set of photo portraits, $B$ could be the set $\{\text{male}, \text{female}\}$, and each element $e$ tells the program if a particular photo is of a man or of a woman. The machine learning algorithm would have to learn to differentiate between a photo of a man and that of a woman.

In a controlled test the set $E$ is usually first divided into a training set which is used for training the agent, and a testing set which is used for testing the performance of the agent. However, in some scenarios it is impossible to first train the agent and then test it. In these cases the training and testing examples are interleaved. The agent's performance is assessed on an ongoing manner.

Figure 5.1 shows a graphical representation of the machine learning problem. The





Capítulo 5

# Aprendizagem em Sistemas Multiagentes

Algoritmos de aprendizado de máquina alcançaram resultados impressionantes. Podemos escrever software que processe maiores quantidades de dados do que qualquer ser humano e que possa aprender a encontrar padrões que escapam até mesmo aos melhores especialistas na área. Como tal, é razoável que em algum momento desejemos adicionar agentes de aprendizagem ao nosso sistema multiagente. Existem vários cenários em que se pode querer adicionar estes agentes de aprendizagem.

Muitos sistemas multiagentes têm como objetivo a exploração ou monitoramento de um determinado espaço, onde cada agente possui apenas uma visão local de sua própria área. Nestes cenários podemos imaginar que cada agente aprende um mapa do seu mundo e os agentes partilham ainda mais os seus mapas, a fim de agregar uma visão global do campo e decidir cooperativamente quais as áreas que necessitam de maior exploração. Esta é uma forma de aprendizagem cooperativa.

Outro cenário é que em ambientes competitivos cada agente egoísta tenta maximizar a sua própria utilidade aprendendo os comportamentos e fraquezas dos outros agentes. Nestes ambientes estamos interessados na dinâmica do sistema e em determinar se os agentes atingirão um equilíbrio estável. Na sua forma mais simples, estes cenários são jogos repetidos com agentes de aprendizagem.

Resumindo, os agentes podem aprender porque não sabem tudo sobre o seu ambiente ou porque não sabem como os outros agentes se comportam. Além disso, a aprendizagem pode acontecer num ambiente cooperativo onde também queremos que os agentes partilhem o conhecimento aprendido, ou num ambiente competitivo onde queremos que eles superem uns aos outros. Apresentamos análises e algoritmos para agentes de aprendizagem nesses diversos ambientes.

## 5.1 O problema do aprendizado de máquina

Antes de nos aprofundarmos no aprendizado multiagente, primeiro apresentamos uma visão de alto nível do que entendemos por aprendizado de máquina. A palavra "aprendizado", quando usada casualmente, pode ter muitos significados diferentes, desde a lembrança até a dedução, mas os pesquisadores de aprendizado de máquina têm uma definição muito específica do problema do aprendizado de máquina.

O objetivo da pesquisa em aprendizado de máquina é o desenvolvimento de algoritmos que aumentem a capacidade de um agente de combinar um conjunto de entradas com suas saídas correspondentes (Mitchell, 1997). Ou seja, assumimos a existência de um grande conjunto de exemplos$E$. Cada exemplo $e \in E$ é um par $e = \{a, b\}$ onde $a \in A$ representa a entrada que o agente recebe e $b \in B$ é a saída que o agente deve produzir ao receber esta entrada. O agente deve encontrar uma função $f$ que mapeie $A \to B$ para tantos exemplos de $A$ quanto possível. Por exemplo, $A$ poderia ser um conjunto de retratos fotográficos, $B$ poderia ser o conjunto $\{\text{masculino,feminino}\}$, e cada elemento $e$ informa ao programa se uma determinada foto é de um homem ou de uma mulher. O algoritmo de aprendizado de máquina teria que aprender a diferenciar entre a foto de um homem e a de uma mulher.

Em um teste controlado, o conjunto $E$ geralmente é primeiro dividido em um conjunto de treinamento, que é usado para treinar o agente, e um conjunto de testes, que é usado para testar o desempenho do agente. Porém, em alguns cenários é impossível primeiro treinar o agente e depois testá-lo. Nestes casos, os exemplos de treinamento e teste são intercalados. O desempenho do agente é avaliado de forma contínua.

A Figura 5.1 mostra uma representação gráfica do problema de aprendizado de máquina. O

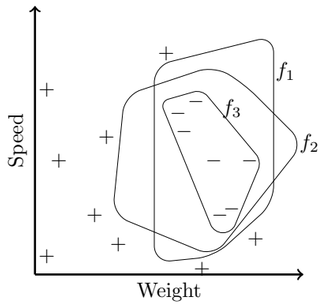



**Figure 5.1:** The machine learning problem. The input set $A$ corresponds to the two axis: Weight and Speed. The outputs $B$ are the set $\{+, -\}$. The lines represent three possible functions $f$ which, in this case, map anything within the lines as a $-$ and anything outside as a $+$. Note that all functions have correctly solved the learning problem but in different ways.

learning problem is coming up with a function that maps all the points in the space to either $+$ or $-$ such that it will also correctly categorize any new examples that we have not seen. Our function must, therefore, extrapolate from the examples it has seen and generalize to all possible instances. That is, machine learning performs **induction** over the set of examples it has seen in order to categorize all future examples.

INDUCTION

Figure 5.1 shows three different functions, $f_1$, $f_2$, and $f_3$, each one of which correctly solves the learning problem. That is, they are all correct inductions since they correctly categorize all the examples. In fact, there are an infinite number of such functions. One might wonder which one is the best one to use. We don't have a general answer to that question. Each learning algorithm, from reinforcement learning to support vector machines, arrives at one learned function but the choice is arbitrary. That is, given the same examples two learning algorithms can learn to perfectly classify them but still have learned different functions. This effect is known as the **induction bias** of a learning algorithm. It is because of this induction bias that some learning algorithms appear to perform better than others in certain domains—their bias coincides with the implicit structure of the problem. However, we know that in general, that is, averages over all possible learning problems there is no learning algorithm that outperforms all others, a fact that has been formalized by the **no free lunch theorem** (Wolpert and Macready, 1995). Still, in practice we, as designers, do know a lot about any specific problem to be learned. You should always try to integrate this knowledge into the learning algorithm you are using.

INDUCTION BIAS

NO FREE LUNCH THEOREM

When a learning agent is placed in a multiagent scenario some of the fundamental assumptions of machine learning are violated. The agent is no longer learning to extrapolate from the examples it has seen of fixed set $E$, instead it's target concept keeps changing (the points in figure 5.1 keep moving), leading to a **moving target function** problem (Vidal and Durfee, 1998b). In general, however, the target concept does not change randomly; it changes based on the learning dynamics of the other agents in the system. Since these agents also learn using machine learning algorithms we are left with some hope that we might someday be able to understand the complex dynamics of these type of systems.

MOVING TARGET FUNCTION

Learning agents are most often selfish utility maximizers. These agents often face each other in encounters where the simultaneous actions of a set of agents leads to different utility payoffs for all the participants. For example, in a market-based setting a set of agents might submit their bids to a first-price sealed-bid auction. The outcome of this auction will result in a utility gain or loss for all the agents. In a robotic setting two agents headed in a collision course towards each other have to decide whether to stay the course or to swerve. The results of their combined actions have direct results in the utilities the agents receive from their actions. However, even agents that are trying to learn for themselves might find utility in sharing this knowledge.



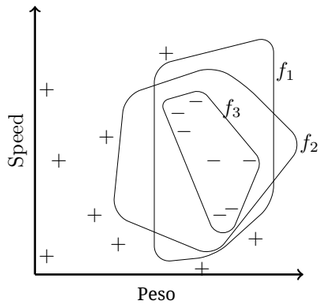



Figura 5.1: O problema de aprendizado de máquina. O conjunto de entrada $A$corresponde aos dois eixos: Peso e Velocidade. As saídas $B$ são o conjunto $\{+, -\}$. As linhas representam três funções possíveis $f$ que, neste caso, mapeiam qualquer coisa dentro das linhas como $-$ e qualquer coisa fora das linhas como $+$. Observe que todas as funções resolveram corretamente o problema de aprendizagem, mas de maneiras diferentes.

O problema de aprendizagem é criar uma função que mapeie todos os pontos no espaço para $+$ ou $-$ de modo que também categorizará corretamente quaisquer novos exemplos que não vimos. A nossa função deve, portanto, extrapolar a partir dos exemplos que viu e generalizar para todas as instâncias possíveis. Ou seja, o aprendizado de máquina realiza indução sobre o conjunto de exemplos que viu para categorizar todos os futuros

INDUCTION

exemplos.

A Figura 5.1 mostra três funções diferentes, $f_1$, $f_2$ e $f_3$, cada uma das quais resolve corretamente o problema de aprendizagem. Ou seja, todas são induções corretas, pois categorizam corretamente todos os exemplos. Na verdade, existe um número infinito de tais funções. Alguém pode estar se perguntando qual é o melhor para usar. Não temos uma resposta geral para essa pergunta. Cada algoritmo de aprendizagem, desde a aprendizagem por reforço até às máquinas vetoriais de suporte, chega a uma função aprendida, mas a escolha é arbitrária. Ou seja, dados os mesmos exemplos, dois algoritmos de aprendizagem podem aprender a classificá-los perfeitamente, mas ainda assim aprenderam funções diferentes. Este efeito é conhecido como viés de indução de um algoritmo de aprendizagem. É por causa dessa indução

INDUCTION BIAS

viés de que alguns algoritmos de aprendizagem parecem ter um desempenho melhor do que outros em determinados domínios – seu viés coincide com a estrutura implícita do problema. Porém, sabemos que em geral, ou seja, médias sobre todos os possíveis problemas de aprendizagem, não existe algoritmo de aprendizagem que supere todos os outros, fato que foi formalizado pelo t eorema do almoço grátis (Wolpert e Macready, 1995). Ainda assim, na prática

NO FREE LUNCH THEOREM

nós, como designers, sabemos muito sobre qualquer problema específico a ser aprendido. Você deve sempre tentar integrar esse conhecimento ao algoritmo de aprendizagem que está usando.

Quando um agente de aprendizagem é colocado em um cenário multiagentes, alguns dos pressupostos fundamentais do aprendizado de máquina são violados. O agente não está mais aprendendo a extrapolar a partir dos exemplos que viu de conjunto fixo $E$; em vez disso, seu conceito de alvo continua mudando (os pontos na figura 5.1 continuam se movendo), levando a um problema de função alvo móvel (Vidal e Durfee, 1998b). Em geral, porém, o alvo

MOVING TARGET FUNCTION

o conceito não muda aleatoriamente; ele muda com base na dinâmica de aprendizage m dos demais agentes do sistema. Como esses agentes também aprendem usando algoritmos de aprendizado de máquina, temos alguma esperança de que algum dia possamos compreender a dinâmica complexa desse tipo de sistema.

Os agentes de aprendizagem são, na maioria das vezes, maximizadores de utilidade egoístas. Esses agentes frequentemente se enfrentam em encontros onde as ações simultâneas de um conjunto de agentes levam a diferentes recompensas de utilidade para todos os participantes. Por exemplo, num cenário baseado no mercado, um conjunto de agentes pode submeter as suas propostas a um leilão de lance selado de primeiro preço. O resultado deste leilão resultará em ganho ou perda de utilidade para todos os agentes. Em um cenário robótico, dois agentes em rota de colisão um contra o outro precisam decidir se mantêm o curso ou desviam. Os resultados de suas ações combinadas têm resultados diretos nas utilidades que os agentes recebem de suas ações. No entanto, mesmo os agentes que estão a tentar aprender por si próprios podem encontrar utilidade na partilha deste conhecimento.

| | | | $j$ | |
|---|---|---|---|---|
| | | | $c$ | $d$ |
| $i$ | | $a$ | 0,0 | 5,1 |
| | | $b$ | -1,6 | 1,5 |

**Figure 5.2:** Sample two-player game matrix. Agent $i$ chooses from the rows and agent $j$ chooses from the columns.

## 5.2 Cooperative Learning

Imagine two robots equipped with wireless communication capabilities and trying to map an unknown environment. One of the robots could learn that the red rocks can be moved but the black rocks are too heavy to move. The robot could communicate this information to the other one so that it does not have to re-learn it. Similarly, once one robot has built a map of one area it could send this map to the other robot. Of course, this scenario assumes that the two robots are cooperating with each other in order to build the map.

This type of problem is easy to solve when the robots are identical. In this case they can simply tell each other everything that they learn knowing that it will be equally applicable to the other one. One challenge is trying to prevent the robots from telling each other things the other already knows. The problem gets much harder when the robots are heterogeneous. For example, one robot might have learned that the black rocks can be moved using its large arm but the other robot might not have an arm that large so this knowledge is useless to him. To solve this problem we need to somehow model the agents' capabilities so as to allow one agent to determine which parts of his learned knowledge will be useful to an agent with a different set of capabilities. To date, there is scant research on general approaches to the problem of sharing learned knowledge. Most systems that share learned knowledge among agents, such as (Stone, 2000), simply assume that all agents have the same capabilities.

## 5.3 Repeated Games

REPEATED GAMES

We now focus on the problem of learning in **repeated games** (Fudenberg and Levine, 1998). In these problems we have two players that face each other repeatedly on the same game matrix, like the one shown in figure 5.2, and each one tries to maximize the sum of its payoffs over time. You will remember that we already saw a specific version of this problem called the iterated prisoner's dilemma.

The theory of learning in games studies the equilibrium concepts dictated by various simple learning mechanisms. That is, while the Nash equilibrium is based on the assumption of perfectly rational players, in learning in games the assumption is that the agents use some kind of algorithm. The theory determines the equilibrium strategy that will be arrived at by the various learning mechanisms and maps these equilibria to the standard solution concepts, if possible. Many learning mechanisms have been studied. The most common of them are explained in the next few subsections.

### 5.3.1 Fictitious Play

FICTITIOUS PLAY

A widely studied model of learning in games is the process of **fictitious play**. In it agents assume that their opponents are playing a fixed strategy. The agents use their past experiences to build a model of the opponent's strategy and use this model to choose their own action. Given that all agents are using fictitious play we try to determine if their learning will converge and, if so, to which strategy.

Fictitious play uses a simple form of learning where an agent remembers everything the other agents have done and uses this information to build a probability distribution for the other agents' expected strategy. Formally, for the two agent case we say that agent $i$ maintains a weight function $k_i : S_j \rightarrow \mathcal{R}^+$. The weight



| | | | $j$ | |
|---|---|---|---|---|
| | | | $c$ | $d$ |
| $i$ | | $a$ | 0,0 | 5,1 |
| | | $b$ | -1,6 | 1,5 |

Figura 5.2: Exemplo de matriz de jogo para dois jogadores. O agente $i$ escolhe nas linhas e o agente $j$ escolhe nas colunas.

## 5.2 Aprendizagem Cooperativa

Imagine dois robôs equipados com recursos de comunicação sem fio e tentando mapear um ambiente desconhecido. Um dos robôs poderia aprender que as pedras vermelhas podem ser movidas, mas as pedras pretas são pesadas demais para serem movidas. O robô poderia comunicar esta informação ao outro para que este não tenha que reaprende-la. Da mesma forma, uma vez que um robô tenha construído um mapa de uma área, ele poderá enviar esse mapa para o outro robô. É claro que este cenário pressupõe que os dois robôs estão cooperando entre si para construir o mapa.

Este tipo de problema é fácil de resolver quando os robôs são idênticos. Neste caso, eles podem simplesmente contar um ao outro tudo o que aprenderam, sabendo que será igualmente aplicável ao outro. Um desafio é tentar evitar que os robôs contem uns aos outros coisas que o outro já sabe. O problema fica muito mais difícil quando os robôs são heterogêneos. Por exemplo, um robô pode ter aprendido que as pedras pretas podem ser movidas usando o seu braço grande, mas o outro robô pode não ter um braço tão grande, então este conhecimento é inútil para ele. Para resolver este problema, precisamos de alguma forma modelar as capacidades dos agentes de modo a permitir que um agente determine quais partes do seu conhecimento aprendido serão úteis para um agente com um conjunto diferente de capacidades. Até à data, há pouca investigação sobre abordagens gerais ao problema da partilha do conhecimento aprendido. A maioria dos sistemas que partilham conhecimento aprendido entre agentes, como (Stone, 2000), simplesmente assumem que todos os agentes têm as mesmas capacidades.

## 5.3 Jogos Repetidos

REPEATED GAMES

Agora nos concentramos no problema da aprendizagem em jogos repetidos (Fudenberg e Levine, 1998). Nestes problemas temos dois jogadores que se enfrentam repetidamente na mesma matriz de jogo, como a mostrada na figura 5.2, e cada um tenta maximizar a soma dos seus payoffs ao longo do tempo. Você deve se lembrar que já vimos uma versão específica desse problema chamada dilema do prisioneiro iterado.

A teoria da aprendizagem em jogos estuda os conceitos de equilíbrio ditados por vários mecanismos simples de aprendizagem. Ou seja, enquanto o equilíbrio de Nash se baseia na suposição de jogadores perfeitamente racionais, na aprendizagem em jogos a suposição é que os agentes utilizem algum tipo de algoritmo. A teoria determina a estratégia de equilíbrio que será alcançada pelos vários mecanismos de aprendizagem e mapeia esses equilíbrios para os conceitos de solução padrão, se possível. Muitos mecanismos de aprendizagem foram estudados. Os mais comuns deles são explicados nas próximas subseções.

### 5.3.1 Jogo Fictício

FICTITIOUS PLAY

Um modelo de aprendizagem em jogos amplamente estudado é o processo de jogo fictício. Em os agentes assumem que seus oponentes estão jogando uma estratégia fixa. Os agentes usam suas experiências passadas para construir um modelo da estratégia do oponente e usam esse modelo para escolher suas próprias ações. Dado que todos os agentes estão a utilizar jogos fictícios, tentamos determinar se a sua aprendizagem irá convergir e, em caso afirmativo, para que estratégia.

O jogo fictício utiliza uma forma simples de aprendizagem onde um agente se lembra de tudo o que os outros agentes fizeram e utiliza esta informação para construir uma distribuição de probabilidade para a estratégia esperada dos outros agentes. Formalmente, para o caso de dois agentes dizemos que o agente $i$ mantém uma função de peso $k_i : S_j \rightarrow \mathcal{R}^+$. O peso

**Figure 5.3:** Example of fictitious play. The matrix is shown above and the values at successive times, each on a different row, are shown on the table below. The first row corresponds to time 0. Note that only $i$ is using fictitious play, $j$ plays the values as in the $s_j$ column. $i$'s first two actions are stochastically chosen.

| | | $j$ | |
|---|---|---|---|
| | | $c$ | $d$ |
| $i$ | $a$ | 0,0 | 1,2 |
| | $b$ | 1,2 | 0,0 |

| $s_i$ | $s_j$ | $k_i(c)$ | $k_i(d)$ | $\Pr_i[c]$ | $\Pr_i[d]$ |
|---|---|---|---|---|---|
| $a$ | $c$ | 1 | 0 | 1 | 0 |
| $b$ | $d$ | 1 | 1 | .5 | .5 |
| $a$ | $d$ | 1 | 2 | 1/3 | 2/3 |
| $a$ | $d$ | 1 | 3 | 1/4 | 3/4 |
| $a$ | $d$ | 1 | 4 | 1/5 | 4/5 |

function changes over time as the agent learns. The weight function at time $t$ is represented by $k_i^t$. It maintains a count of how many times each strategy has been played by each other player $j$. When at time $t-1$ opponent $j$ plays strategy $s_j^{t-1}$ then $i$ updates its weight function with

$$k_i^t(s_j) = k_i^{t-1}(s_j) + \begin{cases} 1 & \text{if } s_j^{t-1} = s_j, \\ 0 & \text{if } s_j^{t-1} \neq s_j. \end{cases} \tag{5.1}$$

Using this weight function, agent $i$ can assign a probability to $j$ playing any of its $s_j \in S_j$ strategies with

$$\mathbf{Pr}_i^t[s_j] = \frac{k_i^t(s_j)}{\sum_{\tilde{s}_j \in S_j} k_i^t(\tilde{s}_j)}. \tag{5.2}$$

That is, $i$ assumes $j$ will pick its action **stochastically** given the values in $k_i(s_j)$. Player $i$ then determines the strategy that will give it the highest expected utility given that $j$ will play each of its $s_j \in S_j$ with probability $\mathbf{Pr}_i^t[s_j]$. In other words, $i$ determines its best response to a probability distribution over $j$'s possible strategies. In effect, $i$ is assuming that $j$'s strategy at each time is taken from some fixed but unknown probability distribution.

An example of the best response dynamic at work is shown in figure 5.3. Here we see the values for agent $i$ and its best responses to agent $j$'s action. Note that in this example agent $j$ is not using best response. Agent $i$ first notices that $j$ played $c$ and thus sets $k_i^1(c) = 1$. It therefore predicts that $j$ will play $c$ with probability 1 so its best response at time 2 is to play $b$, as seen in the second row. Agent $j$ then plays $d$ which makes $i$ have $\mathbf{Pr}_i^1[c] = \mathbf{Pr}_i^1[d] = .5$. Both of $i$'s actions have the same expected payoff (1) so it randomly chooses to play $b$. After that, when $j$ plays $d$ again then $i$'s best response is unequivocally $b$.

Several interesting results have been derived by research in repeated games. These results assume that all players are using fictitious play. For example, we know that the Nash equilibrium remains a powerful attractor.

**Theorem 5.1** (Nash Equilibrium is Attractor to Fictitious Play)**.** *If $s$ is a strict Nash equilibrium and it is played at time $t$ then it will be played at all times greater than $t$ (Fudenberg and Kreps, 1990).*

Intuitively, we can see that if the fictitious play algorithm leads all players to play the same Nash equilibrium then, afterward, they will all increase the probability that all others are playing the equilibrium because they just saw them play it. Since, by definition, the best response of a player when everyone else is playing a strict Nash equilibrium is to play the same equilibrium, then all players will play the same strategy and the next time. The same holds true for every time after that. More importantly, Nash is also where we will converge to.

**Theorem 5.2** (Fictitious Play Converges to Nash)**.** *If fictitious play converges to a pure strategy then that strategy must be a Nash equilibrium (Fudenberg and Kreps, 1990).*

STOCHASTICALLY



Figura 5.3: Exemplo de brincadeira fictícia. A matriz é mostrada acima e os valores em momentos sucessivos, cada um em uma linha diferente, são mostrados na tabela abaixo. A primeira linha corresponde ao tempo 0. Observe que apenas $i$ est á usando jogo fictício, $j$ reproduz os valores como na coluna $s_j$ . As duas primeiras ações de $i$ são escolhidas estocasticamente.

| | | $j$ | |
|---|---|---|---|
| | | $c$ | $d$ |
| $i$ | $a$ | 0,0 | 1,2 |
| | $b$ | 1,2 | 0,0 |

| $s_i$ | $s_j$ | $k_i(c)$ | $k_i(d)$ | $\Pr_i[c]$ | $\Pr_i[d]$ |
|---|---|---|---|---|---|
| $a$ | $c$ | 1 | 0 | 1 | 0 |
| $b$ | $d$ | 1 | 1 | .5 | .5 |
| $a$ | $d$ | 1 | 2 | 1/3 | 2/3 |
| $a$ | $d$ | 1 | 3 | 1/4 | 3/4 |
| $a$ | $d$ | 1 | 4 | 1/5 | 4/5 |

a função muda com o tempo à medida que o agente aprende. A função de peso no te mpo $t$ é representada por $k_i^t$. Ele mantém uma contagem de quantas vezes cada estratégia foi jogada por cada outro jogador $j$. Quando em algum momento o $t-1$ oponente $j$ usa estratégia $s_j^{t-1}$ então $i$ atualiza sua função de peso com

$$k_i^t(s_j) = k_i^{t-1}(s_j) + \begin{cases} 1 & \text{if } s_j^{t-1} = s_j, \\ 0 & \text{if } s_j^{t-1} \neq s_j. \end{cases} \tag{5.1}$$

Usando esta função de peso, o agente $i$ pode atribuir uma probabilidade de $j$ executar qualquer uma de suas $s_j \in S_j$ estratégias com

$$\mathbf{Pr}_i^t[s_j] = \frac{k_i^t(s_j)}{\sum_{\tilde{s}_j \in S_j} k_i^t(\tilde{s}_j)}. \tag{5.2}$$

Ou seja, $i$ assume que $j$ escolherá sua ação estocasticamente dados os valores em $k_i(s_j)$. O jogador $i$determina então a estratégia que lhe dará a maior utilidade esperada, dado que $j$ jogará cada um de seus $s_j \in S_j$ com probabilidade $\Pr_i^t[s_j]$. Em outras palavras, $i$ determina sua melhor resposta a uma distribuição de probabilidade sobre as estratégias possíveis de $j$. Na verdade, $i$ está assumindo que a estratégia de $j$ em cada momento é obtida de alguma distribuição de probabilidade fixa, mas desconhecida.

Um exemplo da dinâmica de melhor resposta no trabalho é mostrado na figura 5.3. Aqui vemos os valores do agente $i$ e suas melhores respostas à ação do agente $j$. Observe que neste exemplo o agente $j$ não está usando a melhor resposta. O agente $i$ primeiro percebe que $j$ jogou$c$ e, portanto, ativa $k_i^1(c) = 1$. Portanto, prevê que $j$ jogará $c$ com probabilidade 1, portanto sua melhor resposta no tempo 2 é jogar $b$, como visto na segunda linha. O agente $j$ então toca $d$ o que faz com que $i$ tenha $\mathbf{Pr}_i^1[c] = \mathbf{Pr}_i^1[d] = .5$. Ambas as ações de $i$ têm o mesmo resultado esperado (1), então ele escolhe aleatoriamente jogar $b$. Depois disso, quando $j$ tocar $d$novamente, a melhor resposta de $i$ será inequivocamente $b$.

Vários resultados interessantes foram obtidos por pesquisas em jogos repetidos. Estes resultados assumem que todos os jogadores estão usando jogos fictícios. Por exemplo, sabemos que o equilíbrio de Nash continua a ser um atrator poderoso.

*Teorema 5.1 (Equilíbrio de Nash é Atrator para Jogo Fictício). Se $s$ é um equilíbrio de Nash estrito e é jogado no tempo $t$ então será jogado em todos os tempos maiores que $t$ (Fudenberg e Kreps, 1990).*

Intuitivamente, podemos ver que se o algoritmo de jogo fictício leva todos os jogadores a jogar o mesmo equilíbrio de Nash, então, posteriormente, todos eles aumentarão a probabilidade de todos os outros estarem jogando o equilíbrio porque acabaram de vê-los jogá-lo. Visto que, por definição, a melhor resposta de um jogador quando todos os outros estão jogando um equilíbrio de Nash estrito é jogar o mesmo equilíbrio, então todos os jogadores jogarão a mesma estratégia e na próxima vez. O mesmo vale para todas as vezes depois disso. Mais importante ainda, Nash também é para onde convergiremos.

*Teorema 5.2 (Jogo Fictício Converge para Nash). Se o jogo fictício converge para uma estratégia pura, então essa estratégia deve ser um equilíbrio de Nash (Fudenberg e Kreps, 1990).*

STOCHASTICALLY

| | | $j$ | |
|---|---|---|---|
| | | $c$ | $d$ |
| $i$ | $a$ | 0,0 | 1,1 |
| | $b$ | 1,1 | 0,0 |

| $s_i$ | $s_j$ | $k_i(c)$ | $k_i(d)$ | $k_j(a)$ | $k_j(b)$ |
|---|---|---|---|---|---|
| | | 1 | 1.5 | 1 | 1.5 |
| $a$ | $c$ | 2 | 1.5 | 2 | 1.5 |
| $b$ | $d$ | 2 | 2.5 | 2 | 2.5 |
| $a$ | $c$ | 3 | 2.5 | 3 | 2.5 |
| $b$ | $d$ | 3 | 3.5 | 3 | 3.5 |

**Figure 5.4:** A game matrix with an infinite cycle.

We can show this by contradiction. If fictitious play converges to a strategy that is not a Nash equilibrium then this means that the best response for at least one of the players is not the same as the convergent strategy. Therefore, that player will take that action at the next time, taking the system away from the strategy profile it was supposed to have converged to.

An obvious problem with the solutions provided by fictitious play can be seen in the existence of infinite cycles of actions. An example is illustrated by the game matrix in figure 5.4. If the players start with initial weights of $k_i^1(c) = 1$, $k_i^1(d) = 1.5$, $k_j^1(a) = 1$, and $k_j^1(b) = 1.5$ they will both believe that the other will play $b$ or $d$ and will, therefore, play $a$ or $c$ respectively. The weights will then be updated to $k_i^2(c) = 2$, $k_i^2(d) = 1.5$, $k_j^2(a) = 2$, and $k_j^2(b) = 1.5$. Next time, both agents will believe that the other will play $a$ or $c$ so both will play $b$ or $d$. The agents will engage in an endless cycle where they alternatively play $(a, c)$ and $(b, d)$. The agents end up receiving the worst possible payoff.

This example illustrates the type of problems we encounter when adding learning to multiagent systems. Most learning algorithms can easily fall into cycles such as this one. One common strategy for avoiding this problem is the use of randomness. Agents will sometimes take a random action in an effort to exit possible loops and to explore the search space. It is interesting to note that, as in the example from figure 5.4, it is often the case that the loops the agents fall in often reflect one of the mixed strategy Nash equilibria for the game. That is, $(.5, .5)$ is a Nash equilibrium for this game. Unfortunately, if the agents are synchronized, as in this case, the implementation of a mixed strategy could lead to a lower payoff.

Games with more than two players require that we decide whether the agent should learn individual models of each of the other agents independently or a joint probability distribution over their combined strategies. Individual models assume that each agent operates independently while the joint distributions capture the possibility that the others agents' strategies are correlated. Unfortunately, for any interesting system the set of all possible strategy profiles is too large to explore—it grows exponentially with the number of agents. Therefore, most learning systems assume that all agents operate independently so they need to maintain only one model per agent.

### 5.3.2 Replicator Dynamics

REPLICATOR DYNAMICS

EVOLUTIONARY GAME THEORY

Another widely studied learning model in repeated games is **replicator dynamics**. This model assumes that the fraction of agents playing a particular strategy will grow in proportion to how well that strategy performs in the population. A homogeneous population of agents is assumed. The agents are randomly paired in order to play a symmetric game, that is, a game where both agents have the same set of possible strategies and receive the same payoffs for the same actions. The replicator dynamics model is meant to capture situations where agents reproduce in proportion to how well they are doing and is inspired by biological evolution. In fact, the field that studies these type of solution concepts is known as **evolutionary game theory** (Weibull, 1997).

Formally, we let $\phi^t(s)$ be the number of agents using strategy $s$ at time $t$. We



| | | $j$ | |
|---|---|---|---|
| | | $c$ | $d$ |
| $i$ | $a$ | 0,0 | 1,1 |
| | $b$ | 1,1 | 0,0 |

| $s_i$ | $s_j$ | $k_i(c)$ | $k_i(d)$ | $k_j(a)$ | $k_j(b)$ |
|---|---|---|---|---|---|
| | | 1 | 1.5 | 1 | 1.5 |
| $a$ | $c$ | 2 | 1.5 | 2 | 1.5 |
| $b$ | $d$ | 2 | 2.5 | 2 | 2.5 |
| $a$ | $c$ | 3 | 2.5 | 3 | 2.5 |
| $b$ | $d$ | 3 | 3.5 | 3 | 3.5 |

Figura 5.4: Uma matriz de jogo com ciclo infinito.

Podemos mostrar isso por contradição. Se o jogo fictício converge para uma estratégia que não é um equilíbrio de Nash, isso significa que a melhor resposta para pelo menos um dos jogadores não é a mesma da estratégia convergente. Portanto, esse jogador realizará aquela ação na próxima vez, afastando o sistema do perfil de estratégia para o qual deveria ter convergido.

Um problema óbvio com as soluções fornecidas pelo jogo fictício pode ser visto na existência de ciclos infinitos de ações. Um exemplo é ilustrado pela matriz do jogo na figura 5.4. Se os jogadores começarem com pesos iniciais de $k_i^1(c) = 1$, $k_i^1(d) = 1.5$,$k_j^1(a) = 1$ e $k_j^1(b) = 1.5$ambos acreditarão que o outro jogará $b$ou $d$e, portanto, jogará $a$ ou $c$ respectivamente. Os pesos serão então atualizados para$k_i^2(c) = 2$, $k_i^2(d) = 1.5$, $k_j^2(a) = 2$ e $k_j^2(b) = 1.5$. Da próxima vez, ambos os agentes acreditarão que o outro jogará $a$ ou $c$ então ambos jogarão $b$ ou $d$. Os agentes se envolverão em um ciclo interminável onde jogarão alternativamente $(a, c)$ e $(b, d)$. Os agentes acabam recebendo a pior recompensa possível.

Este exemplo ilustra o tipo de problemas que encontramos ao adicionar aprendizagem a sistemas multiagentes. A maioria dos algoritmos de aprendizagem pode facilmente cair em ciclos como este. Uma estratégia comum para evitar esse problema é o uso da aleatoriedade. Às vezes, os agentes realizam ações aleatórias em um esforço para sair de possíveis loops e explorar o espaço de busca. É interessante notar que, como no exemplo da figura 5.4, acontece frequentemente que os ciclos em que os agentes se enquadram refletem frequentemente um dos equilíbrios de Nash da estratégia mista para o jogo. Ou seja, (0,5,0,5) é um equilíbrio de Nash para este jogo. Infelizmente, se os agentes estiverem sincronizados, como neste caso, a implementação de uma estratégia mista poderá levar a um retorno menor.

Jogos com mais de dois jogadores exigem que decidamos se o agente deve aprender modelos individuais de cada um dos outros agentes de forma independente ou uma distribuição de probabilidade conjunta sobre as suas estratégias combinadas. Os modelos individuais assumem que cada agente opera de forma independente enquanto as distribuições conjuntas capturam a possibilidade de que as estratégias dos outros agentes estejam correlacionadas. Infelizmente, para qualquer sistema interessante, o conjunto de todos os perfis estratégicos possíveis é demasiado grande para ser explorado – cresce exponencialmente com o número de agentes. Portanto, a maioria dos sistemas de aprendizagem assume que todos os agentes operam de forma independente, pelo que necessitam de manter apenas um modelo por agente.

### 5.3.2 Dinâmica do Replicador

REPLICATOR DYNAMICS

EVOLUTIONARY GAME THEORY

Outro modelo de aprendizagem amplamente estudado em jogos repetidos é a dinâmica do replicador. Este modelo assume que a fração de agentes que jogam uma estratégia específica crescerá proporcionalmente ao desempenho dessa estratégia na população. Assume-se uma população homogênea de agentes. Os agentes são emparelhados aleatoriamente para jogar um jogo simétrico, ou seja, um jogo onde ambos os agentes têm o mesmo conjunto de estratégias possíveis e recebem os mesmos pagamentos pelas mesmas ações. O modelo de dinâmica do replicador destina-se a capturar situações em que os agentes se reproduzem proporcionalmente ao seu desempenho e é inspirado na evolução biológica. Na verdade, o campo que estuda este tipo de conceitos de solução é conhecido como teoria evolutiva dos jogos (Weibull, 1997).

Formalmente, consideramos $\phi^t(s)$ o número de agentes que utilizam a estratégia $s$ no tempo $t$. Nós




can then define

$$\theta^t(s) = \frac{\phi^t(s)}{\sum_{s' \in S} \phi^t(s')} \tag{5.3}$$

to be the fraction of agents playing $s$ at time $t$. The expected utility for an agent playing strategy $s$ at time $t$ is defined as

$$u^t(s) = \sum_{s' \in S} \theta^t(s') u(s, s'), \tag{5.4}$$

where $u(s, s')$ is the utility than an agent playing $s$ receives against an agent playing $s'$. Notice that this expected utility assumes that the agents face each other in pairs and choose their opponents randomly. In the replicator dynamics the reproduction rate for each agent is proportional to how well it did on the previous step. Thus, the number of agents playing $s$ at the next time step is given by

$$\phi^{t+1}(s) = \phi^t(s)(1 + u^t(s)). \tag{5.5}$$

Notice that the number of agents playing a particular strategy will continue to increase as long as the expected utility for that strategy is greater than zero. Only strategies whose expected utility is negative will decrease in population. As such, the size of a population will constantly fluctuate. However, when studying replicator dynamics we ignore the absolute size of the population and focus on the fraction of the population playing a particular strategy. We are also interested in determining if the system's dynamics will converge to some strategy and, if so, which one.

In order to study these systems using the standard solution concepts we view the fraction of agents playing each strategy as a mixed strategy for the game. Since the game is symmetric we can use that strategy as the strategy for both players, so it becomes a strategy profile. We say that the system is in a mixed Nash equilibrium if the fraction of players playing each pure strategy is the same as the probability of the corresponding strategy in the mixed Nash equilibrium. For a pure equilibrium all players must play that strategy. For example, if half the agents are playing $a$ and half $b$ then we can consider this a mixed Nash equilibrium where $a$ and $b$ are each played with .5 probability.

An examination of these systems quickly leads to the conclusion that

**Theorem 5.3** (Nash equilibrium is a Steady State)**.** *Every Nash equilibrium is a steady state for the replicator dynamics (Fudenberg and Levine, 1998).*

We can prove this theorem by contradiction. If an agent had a pure strategy that would return a higher utility than any other strategy then this strategy would be a best response to the Nash equilibrium. If this strategy was different from the Nash equilibrium then we would have a best response to the equilibrium which is not the equilibrium, so the system could not be at a Nash equilibrium.

The reverse has also been shown to be true.

**Theorem 5.4** (Stable Steady State is a Nash Equilibrium)**.** *A* stable steady state *of the replicator dynamics is a Nash equilibrium. A* stable steady state *is one that, after suffering from a small perturbation, is pushed back to the same steady state by the system's dynamics (Fudenberg and Levine, 1998) .*

These states are necessarily Nash equilibria because if they were not then there would exist some particular small perturbation which would take the system away from the steady state. This correspondence was further refined to show that

**Theorem 5.5** (Asymptotically Stable is Trembling-Hand Nash)**.** *An asymptotically stable steady state corresponds to a Nash equilibrium that is trembling-hand perfect and isolated. That is, the stable steady states are a refinement on Nash equilibria—only a few Nash equilibria are stable steady states (Bomze, 1986).*






pode então definir

$$\theta^t(s) = \frac{\phi^t(s)}{\sum_{s' \in S} \phi^t(s')} \tag{5.3}$$

será a fração de agentes jogando $s$ no horário $t$. A utilidade esperada para uma estratégia de jogo do agente $s$ no momento $t$ é definida como

$$u^t(s) = \sum_{s' \in S} \theta^t(s') u(s, s'), \tag{5.4}$$

onde $u(s, s')$ é a utilidade que um agente jogando $s$ recebe contra um agente jogando $s'$. Observe que esta utilidade esperada pressupõe que os agentes se enfrenta m em pares e escolhem seus oponentes aleatoriamente. Na dinâmica do replicador, a taxa de reprodução de cada agente é proporcional ao seu desempenho na etapa anterior. Assim, o número de agentes jogando $s$ na próxima etapa de tempo é dado por

$$\phi^{t+1}(s) = \phi^t(s)(1 + u^t(s)). \tag{5.5}$$

Observe que o número de agentes que jogam uma determinada estratégia continuará a aumentar enquanto a utilidade esperada para essa estratégia for maior que zero. Somente as estratégias cuja utilidade esperada é negativa diminuirão em população. Como tal, o ta manho de uma população flutuará constantemente. No entanto, quando estudamos a dinâmica dos replicadores, ignoramos o tamanho absoluto da população e concentramo-nos na fração da população que joga uma estratégia específica. Estamos também interessados e m determinar se a dinâmica do sistema irá convergir para alguma estratégia e, em caso afirmativo, qual.

Para estudar estes sistemas utilizando os conceitos de solução padrão, vemos a fração de agentes que jogam cada estratégia como uma estratégia mista para o jogo. Como o jogo é simétrico podemos usar essa estratégia como estratégia para ambos os jogadores, tornando-se assim um perfil de estratégia. Dizemos que o sistema está em equilíbrio de Nash misto se a fração de jogadores que jogam cada estratégia pura for igual à probabilidade da estratégia correspondente no equilíbrio de Nash misto. Para um equilíbrio puro, todos os jogadores devem jogar essa estratégia. Por exemplo, se metade dos agentes estão jogando $a$ e metade $b$ , então podemos considerar isso um equilíbrio de Nash misto, onde $a$ e $b$ são jogados cada um com probabilidade de 0,5.

Um exame destes sistemas leva rapidamente à conclusão de que

*Teorema 5.3 (O equilíbrio de Nash é um estado estacionário). Todo equilíbrio de Nash é um estado estacionário para a dinâmica do replicador (Fudenberg e Levine, 1998).*

Podemos provar este teorema por contradição. Se um agente tivesse uma estratégia pura que retornasse uma utilidade mais elevada do que qualquer outra estratégia, então esta estratégia seria a melhor resposta ao equilíbrio de Nash. Se esta estratégia fosse diferente do equilíbrio de Nash, então teríamos uma melhor resposta ao equilíbrio que não é o equilíbrio, portanto o sistema não poderia estar em equilíbrio de Nash.

O inverso também se mostrou verdadeiro.

*Teorema 5.4 (Estado Estável é um Equilíbrio de Nash). Um estado estacionário estável da dinâmica do replicador é um equilíbrio de Nash. Um estado estacionário estável é aquele que, após sofrer uma pequena perturbação, é empurrado de volta ao mesmo estado estacionário pela dinâmica do sistema (Fudenberg e Levine, 1998).*

Esses estados são necessariamente equilíbrios de Nash porque, se não fossem, existiria alguma pequena perturbação específica que afastaria o sistema do estado estacionário. Esta correspondência foi ainda mais refinada para mostrar que

*Teorema 5.5 (Assintoticamente estável é Trembling-Hand Nash). Um estado estacionário assintoticamente estável corresponde a um equilíbrio de Nash que é perfeito e isolado. Ou seja, os estados estacionários estáveis são um refinamento dos equilíbrios de Nash – apenas alguns equilíbrios de Nash são estados estacionários estáveis (Bomze, 1986).*

| | | $j$ | | |
|---|---|---|---|---|
| | | $a$ | $b$ | $c$ |
| $i$ | $a$ | 1,1 | 2,0 | 0,2 |
| | $b$ | 0,2 | 1,1 | 2,0 |
| | $c$ | 2,0 | 0,2 | 1,1 |

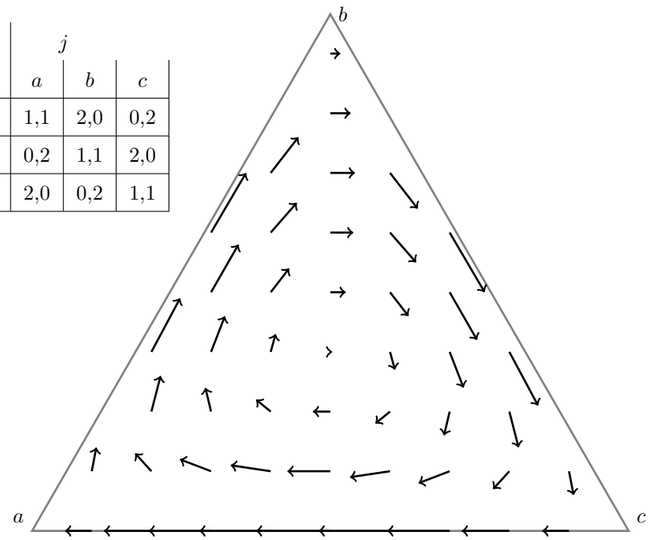



**Figure 5.5:** Visualization of the evolution of populations in replicator dynamics. The game is shown in the top left.

On the other hand, it is also possible that a replicator dynamics system will never converge. In fact, there are many examples of simple games with no asymptotically stable steady states.

While replicator dynamics reflect some of the most troublesome aspects of learning in multiagent systems some differences are evident. These differences are mainly due to the replication assumption. Agents are not usually expected to replicate, instead they acquire the strategies of others. For example, in a real multiagent system all the agents might choose to play the strategy that performed best in the last round instead of choosing their next strategy in proportion to how well it did last time. As such, we cannot directly apply the results from replicator dynamics to multiagent systems. However, the convergence of the systems' dynamics to a Nash equilibrium does illustrate the importance of this solution concept as an attractor of learning agent's dynamics.

Within replicator dynamics we can define a new solution concept inspired by the "survival of the fittest" idea from evolution. An **evolutionary stable strategy** (ESS) is one which, as a population, defeats any small number of invading mutants. That is, if everyone in the population plays that strategy then there is no way a small number of mutants can invade the population and receive greater utility than the agents there. More formally, we define it as

EVOLUTIONARY STABLE STRATEGY

**Definition 5.1** (Evolutionary Stable Strategy)**.** *An* ESS *is an equilibrium strategy that can overcome the presence of a small number of invaders. That is, if the equilibrium strategy profile is $\omega$ and small number $\epsilon$ of invaders start playing $\omega'$ then ESS states that the existing population should get a higher payoff against the new mixture ($\epsilon\omega' + (1-\epsilon)\omega$) than the invaders.*

It has further been shown that

**Theorem 5.6** (ESS is Steady State of Replicator Dynamics)**.** ESS *is an asymptotically stable steady state of the replicator dynamics. However, the converse need not be true—a stable state in the replicator dynamics does not need to be an* ESS *(Taylor and Jonker, 1978).*

This means that ESS is a further refinement of the solution concept provided by the replicator dynamics. ESS can be used when we need a very stable equilibrium concept.



| | | $j$ | | |
|---|---|---|---|---|
| | | $a$ | $b$ | $c$ |
| $i$ | $a$ | 1,1 | 2,0 | 0,2 |
| | $b$ | 0,2 | 1,1 | 2,0 |
| | $c$ | 2,0 | 0,2 | 1,1 |

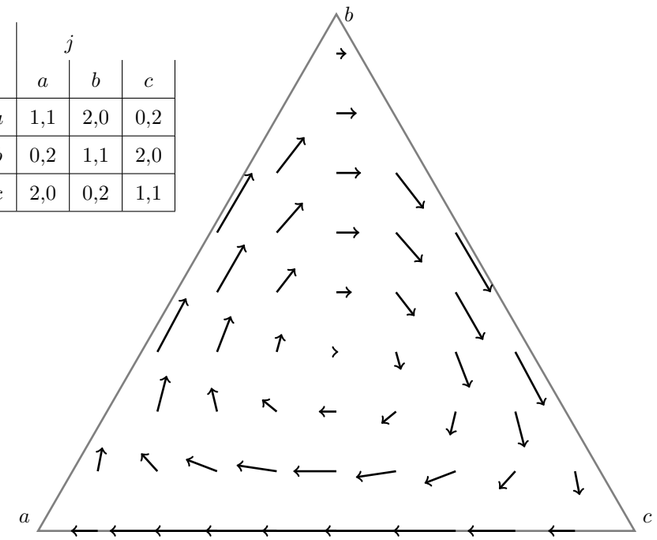



Figura 5.5: Visualização da evolução das populações na dinâmica do replicador. O jogo é mostrado no c anto superior esquerdo.

Por outro lado, também é possível que um sistema dinâmico replicador nunca convirja. Na verdade, existem muitos exemplos de jogos simples sem estados estacionários assintoticamente estáveis.

Embora a dinâmica do replicador reflita alguns dos aspectos mais problemáticos da aprendizagem em sistemas multiagentes, algumas diferenças são evidentes. Essas diferenças se devem principalmente à suposição de replicação. Geralmente não se espera que os age ntes repliquem; em vez disso, adquiram as estratégias de outros. Por exemplo, num sistema multiagente real, todos os agentes podem escolher jogar a estratégia que teve melhor desempenho na última rodada, em vez de escolher a próxima estratégia proporcionalmente ao desempenho da última vez. Como tal, não podemos aplicar diretamente os resultados da dinâmica do replicador a sistemas multiagentes. No entanto, a convergência da dinâmica dos sistemas para um equilíbrio de Nash ilustra a importância deste conceito de solução como um atrator da dinâmica do agente de aprendizagem.

Dentro da dinâmica do replicador, podemos definir um novo conceito de solução inspirado na ideia de "sobrevivência do mais apto" da evolução. Uma estratégia evolutivamente estável (ESS) é aquele que, como população, derrota qualquer pequeno número de mutantes invasores. Isto é, se todos na população adotarem essa estratégia, então não haverá como um pequeno número de mutantes invadir a população e receber maior utilidade do que os agentes ali presentes. Mais formalmente, definimo-lo como

EVOLUTIONARY STABLE STRATEGY

*Definição 5.1 (Estratégia Evolucionária Estável). Um* ESS *é uma estratégia de equilíbrio que pode superar a presença de um pequeno número de invasores. Isto é, se o perfil da estratégia de equilíbrio for $\omega$ e um pequeno número $\epsilon$ de invasores começar a jogar $\omega'$ então a ESS afirma que a população existente deve obter um retorno maior contra a nova mistura ($\epsilon\omega' + (1-\epsilon)\omega$) do que os invasores.*

Foi ainda demonstrado que

*Teorema 5.6 (*ESS *é o estado estacionário da dinâmica do replicador).* ESS *é um estado estacionário assintoticamente estável da dinâmica do replicador. No entanto, o inverso não precisa ser verdadeiro – um estado estável na dinâmica do replicador não precisa ser bean* ESS *(Taylor e Jonker, 1978).*

Isto significa que ESS é um refinamento adicional do conceito de solução fornecido pela dinâmica do replicador. ESS pode ser usado quando precisamos de um conceito de equilíbrio muito estável.

While these convergence theorems are very usefull, it is also the case that most systems never converge to a fixed point. In these cases we need a way to visualize the system dynamics. Since replicator dynamics is deterministic—we know exactly how each population will evolve—we can plot a map showing how the population will vary over time. Figure 5.5 shows a sample game, at the top left, and its visualization. The triangle is known as a **simplex plot** and is used for games with three actions. Every point inside the triangle represents a possible population. The point at the bottom left is the population where all agents play $a$. Similarly, the top point is where all agents play $b$. The point in the exact middle of the triangle represents the population where $1/3$ of the population play $a$, $1/3$ play $b$, and $1/3$ play $c$. Each arrow starts at some population and ends at the next population that would evolve from that one at the next time step. As can be seen, in this specific game the population can get into cycles and never converge to a fixed point.

SIMPLEX PLOT

### 5.3.3 The AWESOME Algorithm

While fictitious play and replicator dynamics do not always converge to an equilibrium, one way we can guarantee that agents will converge to an equilibrium is by having them calculate and play the same equilibrium strategy upon starting. Then, if an agent notices that the others are not playing the agreed upon equilibrium strategy it can play fictitious play instead. This is the basic idea of the **AWESOME** (Adapt When Everyone is Stationary, Otherwise Move to Equilibrium) algorithm (Conitzer and Sandholm, 2003; Conitzer and Sandholm, 2006). That is, if the other agents appear to be stationary the an AWESOME agent plays the best response to their strategy. On the other hand, if they appear to be adapting then the agents retreats to the equilibrium strategy.

AWESOME

The AWESOME algorithm starts by playing the equilibrium strategy $\pi_i$ and keeping track of the actions each other player $j$ has played. Every $N$ rounds (an epoch) it uses these actions to build a, possibly mixed, strategy $s_j$ for each player $j$. If $s_j$ is the equilibrium strategy $\pi_j$ for all players $j$ then the algorithm keeps playing the equilibrium strategy. Otherwise, the algorithm plays the best response to $s_j$. It is easy to see that if all the other players are AWESOME players then they will play their equilibrium strategies and will never diverge from it. If, on the other hand, an AWESOME player is playing against some players who are, eventually, playing some other fixed strategy then it will play a best response strategy against those fixed strategies. Notice how the algorithm implements the reasoning behind the proof of the folk theorem.

While the basic idea is simple, in order to prove that the algorithm will converge some extra complexity had to be added to it. The algorithm, shown in figure 5.6, has two Boolean state variables *playing-equilibrium* which is true when all other agents played the equilibrium strategy during the last epoch and *playing-stationary* which is true when all the other agents played a stationary strategy during the last epoch. Also, *playing-equilibrium-just-rejected* is true when *playing-equilibrium* has just been set to false during the last check. The algorithm plays the same strategy $\phi$ for a whole epoch and then assesses the situation. If it turns out that either the players are not stationary or not in equilibrium the algorithm makes a note of this and changes its state. If the stationarity hypothesis is rejected then the whole algorithm is re-started again (back to line 2).

In order for the algorithm to always converge, $\epsilon_e$ and $\epsilon_s$ must be decreased and $N$ must be increased over time using a schedule where

1. $\epsilon_s$ and $\epsilon_e$ decrease monotonically to 0,
2. $N$ increases to infinity,
3. $\prod_{t \leftarrow 1,\ldots,\infty} 1 - \frac{\sum_i |A_i|}{N^t (\epsilon_s^t)^2} > 0$
4. $\prod_{t \leftarrow 1,\ldots,\infty} 1 - \frac{\sum_i |A_i|}{N^t (\epsilon_e^t)^2} > 0.$



Embora estes teoremas de convergência sejam muito úteis, também acontece que a maioria dos sistemas nunca converge para um ponto fixo. Nestes casos precisamos de uma forma de visualizar a dinâmica do sistema. Como a dinâmica do replicador é determinística – sabemos exatamente como cada população irá evoluir – podemos traçar um mapa mostrando como a população irá variar ao longo do tempo. A Figura 5.5 mostra um exemplo de jogo, no canto superior esquerdo, e sua visualização. O triângulo é conhecido como enredo simplex e é usado para jogos com três ações.

SIMPLEX PLOT

Cada ponto dentro do triângulo representa uma população possível. O ponto inferior esquerdo é a população onde todos os agentes jogam $a$. Da mesma forma, o ponto superior é onde todos os agentes jogam $b$. O ponto exatamente no meio do triângulo representa a população onde $1/3$ da população joga $a$, $1/3$ joga $b$ e $1/3$ joga $c$. Cada seta começa em alguma população e termina na próxima população que evoluiria daquela no próximo passo de tempo. Como pode ser visto, neste jogo específico a população pode entrar em ciclos e nunca convergir para um ponto fixo.

### 5.3.3 O algoritmo INCRÍVEL

Embora o jogo fictício e a dinâmica do replicador nem sempre convirjam para um equilíbrio, uma forma de garantir que os agentes convergirão para um equilíbrio é fazê-los calcular e executar a mesma estratégia de equilíbrio no início. Então, se um agente perceber que os outros não estão jogando a estratégia de equilíbrio acordada, ele poderá jogar um jogo fictício. Esta é a ideia básica do INCRÍVEL

AWESOME

Algoritmo (Adaptar quando todos estão parados, caso contrário, mover para o equilíbrio) (Conitzer e Sandholm, 2003; Conitzer e Sandholm, 2006). Ou seja, se os outros agentes parecem estar estacionários, um AWESOME agente desempenha a melhor resposta à sua estratégia. Por outro lado, se parecerem estar a adaptar-se, então os agentes recuam para a estratégia de equilíbrio.

O algoritmo AWESOME começa executando a estratégia de equilíbrio $\pi_i$ e registrando as ações que cada outro jogador $j$ executou. A cada $N$ rodadas (uma época) ele usa essas ações para construir uma estratégia, possivelmente mista, $s_j$ para cada jogador $j$. Se $s_j$é a estratégia de equilíbrio $\pi_j$ para todos os jogadores $j$ então o algoritmo continua jogando a estratégia de equilíbrio. Caso contrário, o algoritmo reproduz a melhor resposta para $s_j$. É fácil ver que se todos os outros jogadores forem jogadores, então eles usarão suas estratégias de equilíbrio e nunca divergirão delas. Se, por outro lado, umAWESOME jogador estiver jogando contra alguns jogadores que, eventualmente, estão jogando alguma outra estratégia fixa, então ele jogará uma estratégia de melhor resposta contra essas estratégias fixas. Observe como o algoritmo implementa o raciocínio por trás da prova do teorema popular.

Embora a ideia básica seja simples, para provar que o algoritmo irá convergir, alguma complexidade extra teve que ser adicionada a ele. O algoritmo, mostrado na figura 5.6, tem duas variáveis de estado booleanas jogando-equilíbrio que é verdadeiro quando todos os outros agentes jogaram a estratégia de equilíbrio durante a última época e jogando-estacionário que é verdadeiro quando todos os outros agentes jogaram uma estratégia estacionária durante a última época. Além disso, o equilíbrio de jogo acabou de ser rejeitado é verdadeiro quando o equilíbrio de jogo acabou de ser definido como falso durante a última verificação. O algoritmo usa a mesma estratégia$\phi$ durante uma época inteira e depois avalia a situação. Se se verificar que os jogadores não estão estacionários ou não estão em equilíbrio, o algoritmo toma nota disso e altera o seu estado. Se a hipótese de estacionariedade for rejeitada, todo o algoritmo será reiniciado novamente (de volta à linha 2).

Para que o algoritmo sempre convirja, $\epsilon_e$ e $\epsilon_s$ devem ser diminuídos e$N$ devem ser aumentados ao longo do tempo usando um cronograma onde

1. $\epsilon_s$ e $\epsilon_e$ diminuem monotonicamente para 0,
2. $N$ aumenta até o infinito,
3. $\prod_{t \leftarrow 1,\ldots,\infty} 1 - \frac{\sum_i |A_i|}{N^t (\epsilon_s^t)^2} > 0$ 4. $\prod_{t \leftarrow 1,\ldots,\infty} 1 - \frac{\sum_i |A_i|}{N^t (\epsilon_e^t)^2} > 0$.

AWESOME

```
1  π ← equilibrium strategy
2  repeat
3          playing-equilibrium ← TRUE
4          playing-stationary ← TRUE
5          playing-equilibrium-just-rejected ← FALSE
6          φ ← π_i
7          t ← 0
8          while playing-stationary
9                do play φ for N^t times in a row (an epoch)
10                  ∀_j update s_j given what they played in these N^t rounds.
11                  if playing-equilibrium
12                     then if some player j has max_a(s_j(a), π_j(a)) > ε_e
13                                then playing-equilibrium-just-rejected ← TRUE
14                                     φ ← random action
15                     else  if playing-equilibrium-just-rejected = FALSE
                                 and some j has max_a(s_j^old(a), s_j(a)) > ε_s
16                                then playing-stationary ← FALSE
17                           playing-equilibrium-just-rejected ← FALSE
18                           b ← arg max_a u_i(a, s_{-i})
19                           if u_i(b, s_{-i}) > u_i(φ, s_{-i}) + n|A_i|ε_s^{t+1}μ
20                              then φ ← b
21                  ∀_j s_j^old ← s_j
22                  t ← t + 1
```

**Figure 5.6:** The AWESOME algorithm. Here $\pi$ is the equilibrium strategy which has been calculated before the algorithm starts, $n$ is the number of agents, $|A_i|$ is the number of actions the agent can take, $\mu$ is the difference between the player's best and worst possible utility values, and $s_j(a)$ gives the probability with which $j$ will play action $a$ under strategy $s_j$. Also, $\epsilon_e$ and $\epsilon_s$ must be decreased and $N$ must be increased over time using a valid schedule.

Given such a valid schedule, it can be shown that

**Theorem 5.7** (AWESOME converges)**.** *With a valid schedule, the* AWESOME *algorithm converges to best response if all the other players play fixed strategies and to a Nash equilibrium if all the other players are* AWESOME *players.*

Notice that the theorem says that, in self play, it converges to a Nash equilibrium which might be different from the originally agreed upon equilibrium strategy $\pi$. For example, say the agents agree on a mixed equilibrium strategy $\pi$ but some of the actions played in $\pi$ constitute a pure Nash equilibrium. In this case it could happen that the players end up, by chance, repeatedly playing the actions in the pure Nash equilibrium during the first epoch. They might then decide that everybody else is playing a stationary strategy which constitutes the pure Nash equilibrium. They will then play the best response to that pure Nash which, we know, is the same pure Nash equilibrium. As such, they will get stuck in that equilibrium.

The AWESOME algorithm is a simple way to force agents to converge to a Nash equilibrium while not letting them be exploited by other agents that are not using the same algorithm. In those cases where all agents are AWESOME agents then it converges from the first step. However, it remains to be seen exactly how much it can be exploited by clever opponents who know the equilibrium it wants to play but would rather play something else.

## 5.4 Stochastic Games

In many multiagent applications the agents do not know the payoffs they will receive for their actions. Instead, they must take random actions in order to first explore the world so that they may then determine which actions lead them to the best payoffs. That is, the agents inhabit a multiagent Markov decision problem.



AWESOME

```
1  π ← estratégia de equilíbrio 2 repetir 3 jogando em equilíbrio ← TRUE4
jogando parado ← TRUE5 jogando em equilíbrio recém-rejeitado ← FALSE
6          φ ← π_i7          t ← 08 enquanto jogando parado 9 jogam φ por N^t
vezes seguidas (uma época)10          ∀_j atualizam s_j de acordo com o que
jogaram nessas N^t rodadas. 11 se o equilíbrio de jogo 12 então se algum jogador j
tem max_a(s_j(a)π_j(a)) > ε_e13 então o equilíbrio de jogo acabou de ser rejeitado ← TRUE
14                    φ ← ação aleatória 15 senão se o equilíbrio de
jogo acabou de ser rejeitado = FALSEe alguns j tem max_a(s_j^old(a) s_j(a)) > ε_s16 então
jogando parado ← FALSE17 jogando-equilíbrio-rejeitado ← FALSE18 19 se u_i(b, s_{-i})
> u_i(φ, s_{-i}) + n|A_i|ε_s^{t+1}μ20 então φ ← b21          ∀_j s_j^old ← s_j
22          t ← t + 1
```

Figura 5.6: O algoritmo AWESOME . Aqui $\pi$ é a estratégia de equilíbrio que foi calculada antes do início do algoritmo, $n$ é o número de agentes, $|A_i|$ é o número de ações que o agente pode realizar, $\mu$ é a diferença entre os melhores e os piores valores de utilidade possíveis do jogador, e $s_j(a)$ fornece a probabilidade com que $j$ realizará uma ação $a$ sob a estratégia $s_j$. Além disso, $\epsilon_e$e $\epsilon_s$ devem ser diminuídos e$N$ devem ser aumentados ao longo do tempo usando uma programação válida.

Dado um cronograma tão válido, pode-se mostrar que

*Teorema 5.7 (*AWESOME*converge). Com um escalonamento válido, o algoritmo* AWESOME *converge para a melhor resposta se todos os outros jogadores jogarem estratégias fixas e para um equilíbrio de Nash se todos os outros jogadores forem jogadores* AWESOME .

Observe que o teorema diz que, no jogo próprio, ele converge para um equilíbrio de Nash que pode ser diferente da estratégia de equilíbrio originalmente acordada $\pi$. Por exemplo, digamos que os agentes concordam com uma estratégia de equilíbrio misto $\pi$ mas algumas das ações executadas em $\pi$ constituem um equilíbrio de Nash puro. Neste caso pode acontecer que os jogadores acabem, por acaso, executando repetidamente as ações no puro equilíbrio de Nash durante a primeira época. Eles poderiam então decidir que todos os outros estão jogando uma estratégia estacionária que constitui o puro equilíbrio de Nash. Eles então reproduzirão a melhor resposta àquele Nash puro que, sabemos, é o mesmo equilíbrio de Nash puro. Como tal, eles ficarão presos nesse equilíbrio.

O algoritmo AWESOME é uma maneira simples de forçar os agentes a convergir para um equilíbrio de Nash, sem permitir que sejam explorados por outros agentes que não estejam usando o mesmo algoritmo. Nos casos em que todos os agentes são AWESOME agentes, então ele converge desde a primeira etapa. No entanto, resta saber exatamente até que ponto pode ser explorado por adversários inteligentes que conhecem o equilíbrio que pretendem jogar, mas preferem jogar outra coisa.

## 5.4 Jogos Estocásticos

Em muitas aplicações multiagentes, os agentes não sabem os retornos que receberão por s uas ações. Em vez disso, eles devem realizar ações aleatórias para primeiro explorar o mundo, para que possam então determinar quais ações os levam aos melhores resultados. Ou seja, os agentes habitam um problema de decisão de Markov multiagente.

Q-LEARNING

1 $\forall_s \forall_a Q(s,a) \leftarrow 0$; $\lambda \leftarrow 1$; $\epsilon \leftarrow 1$
2 $s \leftarrow$ current state
3 **if** RAND() $< \epsilon$ ▷ Exploration rate
4 **then** $a \leftarrow$ random action
5 **else** $a \leftarrow \arg\max_a Q(s,a)$
6 Take action $a$
7 Receive reward $r$
8 $s' \leftarrow$ current state
9 $Q(s,a) \leftarrow \lambda(r + \gamma \max_{a'} Q(s',a')) + (1-\lambda)Q(s,a)$
10 $\lambda \leftarrow .99\lambda$
11 $\epsilon \leftarrow .98\epsilon$
12 **goto** 2

**Figure 5.7:** Q-LEARNING algorithm. Note that the .99 and .98 numbers are domain-dependent and need to be changed for each problem to ensure that the algorithm works. With $\epsilon \leftarrow 0$ the algorithm is still guaranteed to work, but in practice it might take longer to converge.

### 5.4.1 Reinforcement Learning

REINFORCEMENT LEARNING
Q-LEARNING

A very popular machine learning technique for solving these types of problems is called **reinforcement learning** (Sutton and Barto, 1998), a specific instance of it is known as **Q-learning** (Watkins and Dayan, 1992). Reinforcement learning assumes that the agent lives in a Markov process and receives a reward in certain states. The goal is to find the right action to take in each state so as to maximize the agent's discounted future reward. That is, find the optimal policy.

More formally, we define the reinforcement learning problem as given by an MDP (section 1.2) where the rewards are given on the edges instead of in the states. That is, the reward function is $r(s_t, a_t) \rightarrow \Re$. A reinforcement learning agent must find the policy $\pi^*$ which maximizes his future discounted rewards (1.6).

The reinforcement learning problem can be solved using the Q-LEARNING algorithm shown in figure 5.7. Here $\lambda$ is the **learning rate** and $\epsilon$ is the **exploration rate**. Both are always between 0 and 1. The learning rate guides how heavily we consider new rewards versus old values we have learned. When $\lambda = 1$ the algorithm completely re-writes the old $Q(s,a)$ value while when $\lambda = 0$ is completely ignores any new reward and instead uses the old $Q(s,a)$. The exploration rate ensures that we do not converge too quickly to a solution. When $\epsilon = 1$ all the actions taken by the agent are chosen randomly while when $\epsilon = 0$ all the actions taken maximize the $Q$ values.

LEARNING RATE
EXPLORATION RATE



qlearning

It has been shown that Q-LEARNING will converge.

**Theorem 5.8** (Q-LEARNING Converges)**.** *Given that the learning and exploration rates decrease slowly enough,* Q-LEARNING *is guaranteed to converge to the optimal policy (Watkins and Dayan, 1992) .*

Q-LEARNING differs from the value-iteration algorithm, from figure 1.2, in several respects. In Q-LEARNING the agent takes actions and learns at the same time. Of course, the agent's initial actions will be completely random as it has not learned anything about its expected rewards but, as it takes more actions it learns to choose better actions. Also, the value-iteration algorithm requires knowledge of the complete transition and reward functions of the MDP while the Q-LEARNING algorithm explores the parts of the MDP that it needs. However, to be sure that it has found the optimal policy, a Q-LEARNING agent will need to visit every state and take every action.

The convergence results of Q-LEARNING are impressive, but they assume that only one agent is learning. We are interested in multiagent systems where multiple agents are learning. In these games the reward function is no longer a function of the state and the agent's actions, instead it is a function of the state and all the agents' actions. That is, one agent's reward depends on the actions that other agents take, as captured by the multiagent MDP we discussed in section 1.2.1. In



Q-LEARNING

1 $\forall_s \forall_a Q(s,a) \leftarrow 0$; $\lambda \leftarrow 1$; $\epsilon \leftarrow 12$ $s \leftarrow$ estado atual 3 se RAND() $< \epsilon$ ▷ Taxa de exploração 4 então $a \leftarrow$ ação aleatória 5 senão $a \leftarrow \arg\max_a Q(s,a)$ 6 Realizar ação $a$7 Receber recompensa $r8$ $s' \leftarrow$ estado atual9 $Q(s,a) \leftarrow \lambda(r + \gamma$ máx$_{a'} Q(s',a')) + (1-\lambda)Q(s,a)10$ $\lambda \leftarrow .99\lambda$ 11 $\epsilon \leftarrow .98\epsilon12$ vá para 2

Figura 5.7: Q-LEARNING algoritmo. Observe que os números 0,99 e 0,98 dependem do domínio e precisam ser alterados para cada problema para garantir que o algoritmo funcione. Com $\epsilon \leftarrow 0$ ainda é garantido que o algoritmo funcione, mas na prática pode levar mais tempo para c onvergir.

### 5.4.1 Aprendizado por Reforço

REINFORCEMENT LEARNING
Q-LEARNING

Uma técnica de aprendizado de máquina muito popular para resolver esses tipos de problemas é chamada de aprendizado por reforço (Sutton e Barto, 1998), um exemplo específico de

é conhecido como Q-learning (Watkins e Dayan, 1992). Aprendizagem por reforço assume que o agente vive em um processo de Markov e recebe uma recompensa em determinados estados. O objetivo é encontrar a ação correta a ser tomada em cada estado, de modo a maximizar a recompensa futura com desconto do agente. Isto é, encontre a política ideal.

Mais formalmente, definimos os problemas de aprendizagem por reforço dados por MDP (seção 1.2), onde as recompensas são dadas nas bordas, e não nos estados. Ou seja, a função de recompensa é $r(s_t, a_t) \rightarrow \Re$. Um agente de aprendizagem por reforço deve encontrar a política $\pi^*$ que maximiza suas futuras recompensas com desconto (1.6).

O problema de aprendizagem por reforço pode ser resolvido usando o Q-LEARNING algoritmo mostrado na figura 5.7. Aqui $\lambda$ está a taxa de aprendizagem e $\epsilon$ é a exploração avaliar. Ambos estão sempre entre 0 e 1. A taxa de aprendizagem orienta o quanto considere novas recompensas versus antigos valores que aprendemos. Quando $\lambda = 1$ o algoritmo reescreve completamente o valor antigo $Q(s,a)$ enquanto quando $\lambda = 0$ é ignora completamente qualquer nova recompensa e em vez disso usa o antigo $Q(s,a)$. A taxa de exploração garante que não convirjamos demasiado rapidamente para uma solução. Quando $\epsilon = 1$ todas as ações realizadas pelo agente são escolhidas aleatoriamente, enquanto $\epsilon = 0$ todas as ações realizadas maximizam os$Q$ valores.

LEARNING RATE
EXPLORATION RATE



aprendizage
m

Foi demonstrado que Q-LEARNING convergirá.

*Teorema 5.8 (*Q-LEARNING*Converge). Dado que as taxas de aprendizagem e exploração diminuem lentamente, é garantido que convergirão para a política ideal (Watkins e Dayan, 1992).*

Q-LEARNING difere do algoritmo de iteração de valor, da figura 1.2, em vários aspectos. No Q-LEARNING o agente executa ações e aprende ao mesmo tempo. É claro que as ações iniciais do agente serão completamente aleatórias, pois ele não aprendeu nada sobre as recompensas esperadas, mas, à medida que realiza mais ações, aprende a escolher ações melhores. Além disso, o algoritmo de iteração de valor requer conhecimento das funções completas de transição e recompensa do MDP enquanto o algoritmo Q-LEARNING explora as partes do MDP que ele precisa. No entanto, para ter certeza de que encontrou a política ideal, um Q-LEARNING agente precisará visitar t odos os estados e realizar todas as ações.

Os resultados de convergência de Q-LEARNING são impressionantes, mas assumem que apenas um agente está aprendendo. Estamos interessados em sistemas multiagentes onde múltiplos agentes estão aprendendo. Nestes jogos, a função de recompensa não é mais uma função do estado e das ações do agente, mas sim uma função do estado e de todas as ações dos agentes. Ou seja, a recompensa de um agente depende das ações que outros agentes realizam, conforme capturado pelo multiagente MDP que discutimos na seção 1.2.1. Em

NashQ-learning

1 $t \leftarrow 0$
2 $s \leftarrow$ current state
3 $\forall_{s \in S} \forall_{j \leftarrow 1,\ldots,n} \forall_{a_j \in A_j} Q_j^t(s, a_1, \ldots, a_n) \leftarrow 0$
4 Choose action $a_i^t$
5 $s \leftarrow s'$
6 Observe $r_1^t, \ldots, r_n^t$; $a_1^t, \ldots, a_n^t$; $s'$
7 **for** $j \leftarrow 1, \ldots, n$
8 **do** $Q_j^{t+1}(s, a_1, \ldots, a_n) \leftarrow$
$(1 - \lambda^t) Q_j^t(s, a_1, \ldots, a_n) + \lambda^t (r_j^t + \gamma\, NashQ_j^t(s'))$
where $NashQ_j^t(s') = Q_j^t(s', \pi_1(s') \cdots \pi_n(s'))$
and $\pi_1(s') \cdots \pi_n(s')$ are Nash EP calculated from $Q$ values
9 $t \leftarrow t + 1$
10 **goto** 4

**Figure 5.8:** NashQ-learning algorithm

these multiagent MDPs it might be impossible for a Q-learning agent to converge to an optimal policy.

As we set out to study these multiagent learning problems, the first thing we need to do is choose a new equilibrium. Under the single agent problem definition we were simply looking for the policy that maximizes the agent's discounted rewards. However, when we have multiple agents we might want to maximize the sum of all agents' discounted future rewards (social welfare), or we might choose a more amenable (for convergence) equilibrium such as the **Nash equilibrium point**.

Nash equilibrium point

**Definition 5.2** (Nash Equilibrium Point)**.** *A tuple of $n$ policies $(\pi_1^*, \ldots, \pi_n^*)$ such that for all $s \in S$ and $i = 1, \ldots, n$,*

$$\forall_{\pi_i \in \Pi_i} v_i(s, \pi_1^*, \ldots \pi_n^*) \geq v_i(s, \pi_1^*, \ldots \pi_{i-1}^*, \pi_i, \pi_{i+1}^*, \ldots, \pi_n^*),$$

*where $v_i(s, \pi_1^*, \ldots \pi_n^*)$ is the total rewards (value) that agent $i$ can expect to receive starting from state $s$ and assuming that agents use policies $\pi_1^*, \ldots \pi_n^*$.*

The Nash equilibrium point is a set of policies such that no one agent $i$ will gain anything by changing its policy from its Nash equilibrium point strategies to something else. As with the regular Nash equilibrium, it has been shown that the Nash equilibrium point always exists.

**Theorem 5.9** (Nash Equilibrium Point Exists)**.** *Every $n$-player discounted stochastic game possesses at least one Nash equilibrium point in stationary strategies (Hu and Wellman, 2003).*

We can find the Nash equilibrium point in a system where all the agents use the NashQ-learning algorithm shown in figure 5.8. In this algorithm each agent must keep $n$ $Q$-tables, one for each agent in the population. These tables are updated in line 7 using a formula similar to the standard Q-learning update formula but instead of using the Q values to determine future rewards it uses the *NashQ* tables. These tables hold the Q value for every agent given that all agents play the same Nash equilibrium policy on the multiagent MDP game induced by the current Q values. That is, the algorithm assumes that the MDP is defined by the Q tables it has and then calculates a Nash equilibrium point for this problem. We thus note that at each step the each agent must calculate the Nash equilibrium point given the current $Q$ functions. This can be an extremely expensive computation—harder than finding the Nash equilibrium for a game matrix.

NashQ-learning

Still, the NashQ-learning algorithm is guaranteed to converge as long as enough time is given so that all state/action pairs are explored sufficiently, and the following assumptions hold.

**Assumption 5.1.** *There exists an adversarial equilibrium for the entire game and for every game defined by the Q functions encountered during learning.*



NashQ-learning

1 $t \leftarrow 0$
2 $s \leftarrow$ current state
3 $\forall_{s \in S} \forall_{j \leftarrow 1,\ldots,n} \forall_{a_j \in A_j} Q_j^t(s, a_1, \ldots, a_n) \leftarrow 0$
4 Choose action $a_i^t$
5 $s \leftarrow s'$
6 Observe $r_1^t, \ldots, r_n^t$; $a_1^t, \ldots, a_n^t$; $s'$
7 **for** $j \leftarrow 1, \ldots, n$
8 **do** $Q_j^{t+1}(s, a_1, \ldots, a_n) \leftarrow$
$(1 - \lambda^t) Q_j^t(s, a_1, \ldots, a_n) + \lambda^t (r_j^t + \gamma\, NashQ_j^t(s'))$
where $NashQ_j^t(s') = Q_j^t(s', \pi_1(s') \cdots \pi_n(s'))$
and $\pi_1(s') \cdots \pi_n(s')$ are Nash EP calculated from $Q$ values
9 $t \leftarrow t + 1$
10 **goto** 4

**Figura 5.8:** NashQ-learning **algoritmo**

Para esses MDP multiagentes, pode ser impossível para um Q-learning agente convergir para uma política ideal.

Ao estudarmos esses problemas de aprendizagem multiagente, a primeira coisa que precisamos fazer é escolher um novo equilíbrio. Na definição do problema de agente único, estávamos simplesmente procurando a política que maximizasse as recompensas com desconto do agente. No entanto, quando temos múltiplos agentes, podemos querer maximizar a soma das recompensas futuras descontadas de todos os agentes (bem-estar social), ou podemos escolher um equilíbrio mais favorável (para convergência), tal como o ponto de equilíbrio de Nash.

Nash equilibrium point

***Definição 5.2 (Ponto de Equilíbrio de Nash). Uma tupla de $n$ políticas $(\pi_1^*, \ldots, \pi_n^*)$ tal que para todos $s \in S$ e $i = 1, \ldots, n$,***

$$\forall_{\pi_i \in \Pi_i} v_i(s, \pi_1^*, \ldots \pi_n^*) \geq v_i(s, \pi_1^*, \ldots \pi_{i-1}^*, \pi_i, \pi_{i+1}^*, \ldots, \pi_n^*),$$

***onde $v_i(s, \pi_1^*, \ldots \pi_n^*)$ é o total de recompensas (valor) que o agente $i$ pode esperar receber a partir do estado $s$ e assumindo que os agentes usam políticas $\pi_1^*, \ldots \pi_n^*$.***

O ponto de equilíbrio de Nash é um conjunto de políticas tais que nenhum agente $i$ ganhará nada mudando sua política de suas estratégias de ponto de equilíbrio de Nash para outra coisa. Tal como acontece com o equilíbrio de Nash regular, foi demonstrado que o ponto de equilíbrio de Nash sempre existe.

***Teorema 5.9 (Existe Ponto de Equilíbrio de Nash). Todo jogo estocástico com desconto de $n$ jogadores possui pelo menos um ponto de equilíbrio de Nash em estratégias estacionárias (Hu e Wellman, 2003).***

Podemos encontrar o ponto de equilíbrio de Nash em um sistema onde todos os agentes usam oNashQ-learning algoritmo mostrado na figura 5.8. Neste algoritmo cada agente deve mantenha tabelas $n$ $Q$, uma para cada agente na população. Essas tabelas são atualizadas na linha 7 usando uma fórmula semelhante à fórmula de atualização Q-learning padrão, mas em vez de usar os valores Q para determinar recompensas futuras, ela usa as tabelas NashQ. Essas tabelas contêm o valor Q para cada agente, dado que todos os agentes jogam a mesma política de equilíbrio de Nash no jogo multiagente MDP induzido pelos valores Q atuais. Ou seja, o algoritmo assume que MDP é definido pelas Q tabelas que possui e então calcula um ponto de equilíbrio de Nash para este problema. Notamos assim que a cada passo cada agente deve calcular o ponto de equilíbrio de Nash dadas as funções $Q$ atuais. Este pode ser um cálculo extremamente caro – mais difícil do que encontrar o equilíbrio de Nash para u ma matriz de jogo.

NashQ-learning

Ainda assim, é garantido que o algoritmo converja, desde que seja dado tempo suficiente para que todos os pares estado/ação sejam explorados suficientemente, e as seguintes suposições sejam válidas.

***Suposição 5.1. Existe um equilíbrio adversário para todo o jogo e para cada jogo definido pelas $Q$ funções encontradas durante o aprendizado.***

FRIEND-OR-FOE

1 $t \leftarrow 0$
2 $s_0 \leftarrow$ current state
3 $\forall_{s \in S} \forall_{a_j \in A_j} Q_i^t(s, a_1, \ldots, a_n) \leftarrow 0$
4 Choose action $a_i^t$
5 $s \leftarrow s'$
6 Observe $r_1^t, \ldots, r_n^t$; $a_1^t, \ldots, a_n^t$; $s'$
7 $Q_i^{t+1}(s, a_1, \ldots, a_n) \leftarrow$
$(1 - \lambda^t) Q_i^t(s, a_1, \ldots, a_n) + \lambda^t (r_i^t + \gamma \, NashQ_i^t(s'))$
where $NashQ_i^t(s') = \max_{\pi \in \Pi(X_1 \times \cdots \times X_k)} \min_{y_i, \ldots, y_l \in Y_1 \times \cdots \times Y_l}$
$\sum_{x_1, \ldots, x_k \in X_1 \times \cdots \times X_k} \pi(x_1) \cdots \pi(x_k) Q_i(s, x_1, \ldots, x_k, y_1, \ldots y_l)$
and $X$ are actions for $i$'s friends and $Y$ are for the foes.
8 $t \leftarrow t + 1$
9 **goto** 4

**Figure 5.9:** FRIEND-OR-FOE algorithm. There are $k$ friends with actions taken from $X_1, \ldots, X_k$, and $l$ foes with actions taken from $Y_1, \ldots, Y_l$.

Where an adversarial equilibrium is one where no agent has anything to lose if the other agents change their policies. That is, if the other agents change their policies from equilibrium then the agent's expected reward will either stay the same or increase.

**Assumption 5.2.** *There exists a coordination equilibrium for the entire game and for every game defined by the Q functions encountered during learning.*

Where a coordination equilibrium is one where all the agents receive the highest possible value. That is, the social welfare solution.

Under these assumptions it can be shown that NASHQ-LEARNING converges.

**Theorem 5.10** (NASHQ-LEARNING Converges)**.** *Under assumptions 5.1 and 5.2* NASHQ-LEARNING *converges to a Nash equilibrium as long as all the equilibria encountered during the game are unique (Hu and Wellman, 2003).*

These assumptions can be further relaxed by assuming that we can tell the agent whether the opponent is a "friend", in which case we are looking for a coordination equilibrium, or a "foe", in which case we are looking for an adversarial equilibrium (Littman, 2001). With this additional information we no longer need to maintain a $Q$ table for each opponent and can achieve convergence with only one, expanded, $Q$ table. The algorithm is thus called the FRIEND-OR-FOE algorithm and is shown in figure 5.9.

FRIEND-OR-FOE

In the FRIEND-OR-FOE algorithm the agent $i$ has $k$ friends with action sets represented by $X_1, \ldots, X_k$ and $l$ foes with action sets represented by $Y_1, \ldots, Y_l$. The algorithm implements the idea that $i$'s friends are assumed to work together to maximize $i$'s value and $i$'s foes are working to minimize it. We can show that FRIEND-OR-FOE converges to a stable policy. However, in general these do not correspond to a Nash equilibrium point. Still, we can show that it often converges to a Nash equilibrium.

**Theorem 5.11.** FOE-Q *learns values for a Nash equilibrium policy if the game has an adversarial equilibrium and* FRIEND-Q *learns values for a Nash equilibrium policy if the game has a coordination equilibrium. This is true regardless of opponent behavior (Littman, 2001).*

That is, if the game has one of the equilibria and we correctly classify all the other agents as either friends or foes then the FRIEND-OR-FOE algorithm is guaranteed to converge.

FRIEND-OR-FOE has several advantages over NASHQ-LEARNING. It is does not require the learning of $Q$ functions for each one of the other agents and it is easy to implement as it does not require the calculation of a Nash equilibrium point at each



FRIEND-OR-FOE

1 $t \leftarrow 0$
2 $s_0 \leftarrow$ current state
3 $\forall_{s \in S} \forall_{a_j \in A_j} Q_i^t(s, a_1, \ldots, a_n) \leftarrow 0$
4 Choose action $a_i^t$
5 $s \leftarrow s'$
6 Observe $r_1^t, \ldots, r_n^t$; $a_1^t, \ldots, a_n^t$; $s'$
7 $Q_i^{t+1}(s, a_1, \ldots, a_n) \leftarrow$
$(1 - \lambda^t) Q_i^t(s, a_1, \ldots, a_n) + \lambda^t (r_i^t + \gamma \, NashQ_i^t(s'))$
where $NashQ_i^t(s') = \max_{\pi \in \Pi(X_1 \times \cdots \times X_k)} \min_{y_i, \ldots, y_l \in Y_1 \times \cdots \times Y_l}$
$\sum_{x_1, \ldots, x_k \in X_1 \times \cdots \times X_k} \pi(x_1) \cdots \pi(x_k) Q_i(s, x_1, \ldots, x_k, y_1, \ldots y_l)$
and $X$ are actions for $i$'s friends and $Y$ are for the foes.
8 $t \leftarrow t + 1$
9 **goto** 4

Figura 5.9: FRIEND-OR-FOE algoritmo. Existem $k$ amigos com ações realizadas em $X_1, \ldots, X_k$ e $l$ inimigos com ações tomadas de $Y_1, \ldots, Y_l$.

Onde um equilíbrio adversário é aquele em que nenhum agente tem nada a perder se os outros agentes mudarem as suas políticas. Isto é, se os outros agentes alterarem as suas políticas a partir do equilíbrio, então a recompensa esperada do agente permanecerá a mesma ou aumentará.

**A***suposição 5.2. Existe um equilíbrio de coordenação para todo o jogo e fou cada jogo definido pelas* $Q$ *funções encontradas durante o aprendizado.*

Já o equilíbrio de coordenação é aquele em que todos os agentes recebem o maior valor possível. Ou seja, a solução de bem-estar social.

Sob essas suposições, pode-se mostrar que NASHQ-LEARNING converge.

*Teorema 5.10 (*NASHQ-LEARNING*Converge). Sob as suposições 5.1 e 5.2,*NASHQ-LEARNING *converge para um equilíbrio de Nash, desde que todos os equilíbrios encontrados durante o jogo sejam únicos (Hu e Wellman, 2003).*

Estas suposições podem ser ainda mais relaxadas assumindo que podemos dizer ao agente se o oponente é um "amigo", caso em que procuramos um equilíbrio de coordenação, ou um "inimigo", caso em que procuramos um equilíbrio adversário (Littman, 2001). Com essas informações adicionais, não precisamos mais manter uma $Q$ tabela para cada oponente e podemos alcançar a convergência com apenas uma mesa $Q$ expandida. O algoritmo é então chamado de algoritmo FRIEND-OR-FOE e é mostrado na figura 5.9.

FRIEND-OR-FOE

No FRIEND-OR-FOE algoritmo, o agente $i$ tem $k$ amigos com conjuntos de ações representado por $X_1, \ldots, X_k$ e $l$ inimigos com conjuntos de ações representados por $Y_1, \ldots, Y_l$. O algoritmo implementa a ideia de que os amigos de $i$ trabalham juntos para maximizar o valor de $i$ e os inimigos de $i$ estão trabalhando para minimizá-lo. Podemos mostrar que FRIEND-OR-FOE converge para uma política estável. No entanto, em geral estes não correspondem a um ponto de equilíbrio de Nash. Ainda assim, podemos mostrar que frequentemente converge para um equilíbrio de Nash.

*Teorema 5.11.* FOE-Q *aprende valores para uma política de equilíbrio de Nash se o jogo tiver um equilíbrio adversário e* FRIEND-Q *aprende valores para uma política de equilíbrio de Nash se o jogo tiver um equilíbrio de coordenação. Isto é verdade independentemente do comportamento do oponente (Littman, 2001).*

Ou seja, se o jogo tiver um dos equilíbrios e classificarmos corretamente todos os outros agentes como amigos ou inimigos, então o algoritmo FRIEND-OR-FOE tem garantia de convergência.

FRIEND-OR-FOE tem diversas vantagens sobre NASHQ-LEARNING. Não requer o aprendizado de funções $Q$ para cada um dos outros agentes e é fácil de implementar, pois não requer o cálculo de um ponto de equilíbrio de Nash em cada

step. On the other hand, it does require us to know if the opponents are friends or foes, that is, whether there exists a coordination or an adversarial equilibrium.

Neither algorithm deals with the problem of finding equilibria in cases without either coordination or adversarial equilibria. Such cases are the most common and most interesting as they require some degree of cooperation among otherwise selfish agents.

## 5.5 General Theories for Learning Agents

The theory of learning in games provides the designer of multiagent systems with many useful tools for determining the possible equilibrium points of a system. Unfortunately, most multiagent systems with learning agents do not converge to an equilibrium. Designers often use learning agents because they do not know, at design time, the specific circumstances that the agents will face at run time. If a designer knew the best strategy, that is, the Nash equilibrium strategy, for his agent then he would simply implement this strategy and avoid the complexities of implementing a learning algorithm. Therefore, we will see a multiagent system with learning agents when the designer cannot predict that an equilibrium solution will emerge.

The two main reasons for this inability to predict the equilibrium solution of a system are the existence of unpredictable environmental changes that affect the agents' payoffs and the fact that on many systems an agent only has access to its own set of payoffs—it does not know the payoffs of other agents. These two reasons make it impossible for a designer to predict which equilibria, if any, the system would converge to. However, the agents in the system are still playing a game for which an equilibrium exists, even if the designer cannot predict it at design-time. But, since the actual payoffs keep changing it is often the case that the agents are constantly changing their strategy in order to accommodate the new payoffs.

As mentioned earlier, learning agents in a multiagent system are faced with a moving target function problem (Vidal and Durfee, 1998b). That is, as the agents change their behavior in an effort to maximize their utility their payoffs for those actions change, changing the expected utility of their behavior. The system will likely have non-stationary dynamics—always changing in order to match the new goal. While game theory tells us where the equilibrium points are, given that the payoffs stay fixed, multiagent systems often never get to those points. A system designer needs to know how changes in the design of the system and learning algorithms will affect the time to convergence. This type of information can be determined by using CLRI theory.

### 5.5.1 CLRI Model

CLRI MODEL

The **CLRI model** (Vidal and Durfee, 2003) provides a method for analyzing a system composed of learning agents and determining how an agent's learning is expected to affect the learning of other agents in the system. It assumes a system where each agent has a decision function that governs its behavior as well as a target function that describes the agent's best possible behavior. The target function is unknown to the agent. The goal of the agent's learning is to have its decision function be an exact duplicate of its target function. Of course, the target function keeps changing as a result of other agents' learning.

Formally, the CLRI model assumes that there is a fixed set of autonomous agents in the system. The system can be described by a set of discrete states $s \in S$ which are presented to the agent with a probability dictated by the fixed probability distribution $\mathcal{D}(S)$. Each agent $i$ has a set of possible actions $A_i$ where $|A_i| \geq 2$. Time is discrete and indexed by a variable $t$. At each time $t$ all agents are presented with a new $s \in \mathcal{D}(S)$, take a simultaneous action, and receive some payoff. The scenario is similar to the one used by fictitious play except for the addition of state $s$.



etapa. Por outro lado, exige que saibamos se os adversários são amigos ou inimigos, isto é, se existe uma coordenação ou um equilíbrio adversário.

Nenhum dos algoritmos lida com o problema de encontrar equilíbrio em casos se m coordenação ou equilíbrio adversário. Tais casos são os mais comuns e mais interessantes, pois requerem algum grau de cooperação entre agentes que de outra forma seriam egoístas.

## 5.5 Teorias Gerais para Agentes de Aprendizagem

A teoria da aprendizagem em jogos fornece ao projetista de sistemas multiagentes muitas ferramentas úteis para determinar os possíveis pontos de equilíbrio de um sistema. Infelizmente, a maioria dos sistemas multiagentes com agentes de aprendizagem não convergem para um equilíbrio. Os designers geralmente usam agentes de aprendizagem porque não sabem, em tempo de design, as circunstâncias específicas que os agentes enfrentarão em tempo de execução. Se um designer conhecesse a melhor estratégia, isto é, a estratégia de equilíbrio de Nash, para o seu agente, então ele simplesmente implementaria esta estratégia e evitaria as complexidades de implementar um algoritmo de aprendizagem. Portanto, veremos um sistema multiagente com agentes de aprendizagem quando o projetista não puder prever que surgirá uma solução de equilíbrio.

As duas principais razões para esta incapacidade de prever a solução de equilíbrio de u m sistema são a existência de alterações ambientais imprevisíveis que afectam os ganhos dos agentes e o facto de em muitos sistemas um agente apenas ter acesso ao seu próprio conjunto de resultados – não conhece os resultados de outros agentes. Estas duas razões tornam impossível para um projetista prever para quais equilíbrios, se houver, o sistema co nvergiria. No entanto, os agentes no sistema ainda estão jogando um jogo para o qual existe um equilíbrio, mesmo que o projetista não possa prevê-lo no momento do projeto. Mas, uma vez que os resultados reais continuam a mudar, acontece frequentemente que os agentes mudam constantemente a sua estratégia, a fim de acomodar os novos resultados.

Conforme mencionado anteriormente, os agentes de aprendizagem em um sistema multiagente enfrentam um problema de função alvo móvel (Vidal e Durfee, 1998b). Isto é, à medida que os agentes mudam o seu comportamento num esforço para maximizar a sua utilidade, os seus retornos para essas ações mudam, alterando a utilidade esperada do seu comportamento. O sistema provavelmente terá uma dinâmica não estacionária – sempre mudando para corresponder ao novo objetivo. Embora a teoria dos jogos nos diga onde estão os pontos de equilíbrio, dado que os resultados permanecem fixos, os sistemas multiagentes muitas vezes nunca chegam a esses pontos. Um projetista de sistema precisa saber como as mudanças no projeto do siste ma e nos algoritmos de aprendizagem afetarão o tempo de convergência. Esse tipo de informação pode ser determinado usandoCLRI a teoria.

### 5.5.1 Modelo CLRI

O modelo CLRI (Vidal e Durfee, 2003) fornece um método para analisar um sistema composto por agentes de aprendizagem e determinar como se espera que a aprendizagem de um agente afete a aprendizagem de outros agentes no sistema. Assume um sistema onde cada agente tem uma função de decisão que governa o seu comportamento, bem como uma função alvo que descreve o melhor comportamento possível do agente. A função alvo é desconhecida para o agente. O objetivo do aprendizado do agente é fazer com que sua função de decisão seja uma duplicata exata de sua função alvo. É claro que a função alvo continua mudando como resultado do aprendizado de outros agentes.

CLRI MODEL

Formalmente, o modelo CLRI assume que existe um conjunto fixo de agentes autônomos no sistema. O sistema pode ser descrito por um conjunto de estados discretos $s \in S$ que são apresentados ao agente com uma probabilidade ditada pela distribuição de probabilidade fixa $\mathcal{D}(S)$. Cada agente $i$ tem um conjunto de ações possíveis $A_i$ onde $|A_i| \geq 2$. O tempo é discreto e indexado por uma variável $t$. A cada vez, $t$ todos os agentes recebem um novo $s \in \mathcal{D}(S)$, realizam uma ação simultânea e recebem alguma recompensa. O cenário é semelhante ao usado no jogo fictício, exceto pela adição do estados.

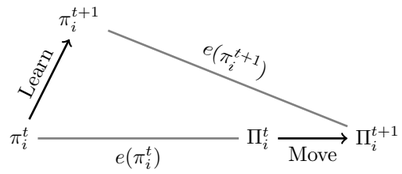



**Figure 5.10:** The moving target function problem.

Each agent $i$'s behavior is defined by a policy $\pi_i^t(s) : S \to A$. When $i$ learns at time $t$ that it is in state $s$ it will take action $\pi_i^t(s)$. At any time there is an optimal policy for each agent $i$, also known as its target function, which we represent as $\Pi_i^t(s)$. Agent $i$'s learning algorithm will try to reduce the discrepancy between $\pi_i$ and $\Pi_i$ by using the payoffs it receives for each action as clues as to what it should do given that it does not have direct access to $\Pi_i$. The probability that an agent will take a wrong action is given by its error $e(\pi_i^t) = \mathbf{Pr}[\pi_i^t(s) \neq \Pi_i^t(s) \,|\, s \in \mathcal{D}(S)]$. As other agents learn and change their decision function, $i$'s target function will also change, leading to the moving target function problem, as depicted in figure 5.10.

An agent's error is based on a fixed probability distribution over world states and a Boolean matching between the decision and target functions. These constraints are often too restrictive to properly model many multiagent systems. However, even if the system being modeled does not completely obey these two constraints, the use of the CLRI model in these cases still gives the designer valuable insight into how changes in the design will affect the dynamics of the system. This practice is akin to the use of Q-learning in non-Markovian games—while Q-learning is only guaranteed to converge if the environment is Markovian, it can still perform well on other domains.

The CLRI model allows a designer to understand the expected dynamics of the system, regardless of what learning algorithm is used, by modeling the system using four parameters: Change rate, Learning rate, Retention rate, and Impact (CLRI). A designer can determine values for these parameters and then use the CLRI difference equation to determine the expected behavior of the system.

The change rate ($c$) is the probability that an agent will change at least one of its incorrect mappings in $\delta^t(w)$ for the new $\delta^{t+1}(w)$. It captures the rate at which the agent's learning algorithm tries to change its erroneous mappings. The learning rate ($l$) is the probability that the agent changes an incorrect mapping to the correct one. That is, the probability that $\delta^{t+1}(w) = \Delta^t(w)$, for all $w$. By definition, the learning rate must be less than or equal to the change rate, i.e. $l \leq c$. The retention rate ($r$) represents the probability that the agent will retain its correct mapping. That is, the probability that $\delta^{t+1}(w) = \delta^t(w)$ given that $\delta^t(w) = \Delta^t(w)$.

CLRI defines a volatility term ($v$) to be the probability that the target function $\Delta$ changes from time $t$ to $t+1$. That is, the probability that $\Delta^t(w) \neq \Delta^{t+1}(w)$. As one would expect, volatility captures the amount of change that the agent must deal with. It can also be viewed as the speed of the target function in the moving target function problem, with the learning and retention rates representing the speed of the decision function. Since the volatility is a dynamic property of the system (usually it can only be calculated by running the system) CLRI provides an impact ($I_{ij}$) measure. $I_{ij}$ represents the impact that $i$'s learning has on $j$'s target function. Specifically, it is the probability that $\Delta_j^t(w)$ will change given that $\delta_i^{t+1}(w) \neq \delta_i^t(w)$.

Someone trying to build a multiagent system with learning agents would determine the appropriate values for $c$, $l$, $r$, and either $v$ or $I$ and then use

$$E[e(\delta_i^{t+1})] = 1 - r_i + v_i \left( \frac{|A_i| r_i - 1}{|A_i| - 1} \right) + e(\delta_i^t) \left( r_i - l_i + v_i \left( \frac{|A_i|(l_i - r_i) + l_i - c_i}{|A_i| - 1} \right) \right) \quad (5.6)$$



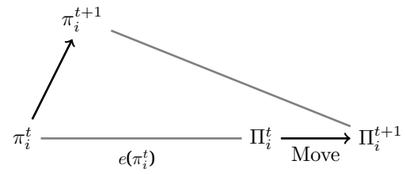



Figura 5.10: O problema da função alvo móvel.

O comportamento de cada agente $i$ é definido por uma política $\pi_i^t(s)$: $S \to A$. Quando $i$ fica sabendo $t$ que está no estado $s$, ele tomará medidas $\pi_i^t(s)$. A qualquer momento existe uma política ótima para cada agente $i$, também conhecida como sua função alvo, que representamos como$\Pi_i^t(s)$. O algoritmo de aprendizagem do agente $i$ tentará reduzir a discrepância entre $\pi_i$ e$\Pi_i$ usando as recompensas que recebe para cada ação como pistas sobre o que deve fazer, uma vez que não tem acesso direto a $\Pi_i$. A probabilidade de um agente realizar uma ação errada é dada por seu erro $e(\pi_i^t) = \mathbf{Pr}[\pi_i^t(s) \neq \Pi_i^t(s) \,|\, s \in \mathcal{D}(S)]$. À medida que outros agentes aprendem e mudam a sua função de decisão, a função alvo de $i$ também mudará, levando ao problema da função alvo móvel, conforme ilustrado na figura 5.10.

O erro de um agente é baseado em uma distribuição de probabilidade fixa sobre os estados mundiais e em uma correspondência booleana entre as funções de decisão e de destino. Essas restrições costumam ser muito restritivas para modelar adequadamente muitos sistemas multiagentes. No entanto, mesmo que o sistema que está sendo modelado não obedeça completamente a essas duas restrições, o uso do modelo CLRI nestes casos ainda dá ao projetista informações valiosas sobre como as mudanças no projeto afetarão a dinâmica do sistema. Esta prática é semelhante ao uso de Q-learning em jogos não Markovianos - embora o Q-learning só tenha garantia de convergência se o ambiente for Markoviano, ele ainda pode ter um bom desempenho em outros domínios.

O modelo CLRI permite que um designer entenda a dinâmica esperada do sistema, independentemente de qual algoritmo de aprendizagem é usado, modelando o sistema usando quatro parâmetros: taxa de mudança, taxa de aprendizagem, taxa de retenção e impacto (CLRI). Um projetista pode determinar valores para esses parâmetros e então usar a equação de diferenças CLRI para determinar o comportamento esperado do sistema.

A taxa de mudança ($c$) é a probabilidade de um agente alterar pelo menos um de seus mapeamentos incorretos em $\delta^t(w)$ para o novo $\delta^{t+1}(w)$. Ele captura a taxa na qual o algoritmo de aprendizagem do agente tenta alterar seus mapeamentos errôneos. A taxa de aprendizagem ($l$) é a probabilidade de o agente alterar um mapeamento incorreto para o correto. Ou seja, a probabilidade de que $\delta^{t+1}(w) = \Delta^t(w)$, para todo $w$. Por definição, a taxa de aprendizagem deve ser menor ou igual à taxa de mudança, ou seja, $l \leq c$. A taxa de retenção ($r$) representa a probabilidade de o agente reter seu mapeamento correto. Ou seja, a probabilidade de que $\delta^{t+1}(w) = \delta^t(w)$ dado que $\delta^t(w) = \Delta^t(w)$.

CLRI define um termo de volatilidade ($v$) como a probabilidade de a função alvo$\Delta$ mudar do tempo $t$ para $t+1$. Ou seja, a probabilidade de que $\Delta^t(w) \neq \Delta^{t+1}(w)$. Como seria de esperar, a volatilidade captura a quantidade de mudança com a qual o agente deve lidar. Também pode ser vista como a velocidade da função alvo no problema da função alvo móvel, com as taxas de aprendizagem e retenção representando a velocidade da função de decisão. Como a volatilidade é uma propriedade dinâmica do sistema (geralmente só pode ser calculada executando o sistema), CLRI fornece uma medida de impacto ($I_{ij}$). $I_{ij}$ representa o impacto que o aprendizado de $i$ tem na função alvo de $j$. Especificamente, é a probabilidade de que $\Delta_j^t(w)$ mude dado que $\delta_i^{t+1}(w) \neq \delta_i^t(w)$.

Alguém tentando construir um sistema multiagente com agentes de aprendizagem determinaria os valores apropriados para $c$, $l$, $r$ e $v$ ou $I$ e então usaria

$$E[e(\delta_i^{t+1})] = 1 - r_i + v_i \left( \frac{|A_i| r_i - 1}{|A_i| - 1} \right) + e(\delta_i^t) \left( r_i - l_i + v_i \left( \frac{|A_i|(l_i - r_i) + l_i - c_i}{|A_i| - 1} \right) \right) \quad (5.6)$$

in order to determine the successive expected errors for a typical agent $i$. This equation relies on a definition of volatility in terms of impact given by

$$\begin{aligned}\forall_{w \in W}\ v_i^t &= \mathbf{Pr}[\Delta_i^{t+1}(w) \neq \Delta_i^t(w)] \\ &= 1 - \prod_{j \in N_{-i}} (1 - I_{ji}\mathbf{Pr}[\delta_j^{t+1}(w) \neq \delta_j^t(w)]),\end{aligned} \tag{5.7}$$

which makes the simplifying assumption that changes in agents' decision functions will not cancel each other out when calculating their impact on other agents. The difference equation (5.6) cannot, under most circumstances, be collapsed into a function of $t$ so it must still be iterated over. On the other hand, a careful study of the function and the reasoning behind the choice of the CLRI parameter leads to an intuitive understanding of how changes in these parameters will be reflected in the function and, therefore, the system. A knowledgeable designer can simply use this added understanding to determine the expected behavior of his system under various assumptions. An example of this approach is shown in (Brooks and Durfee, 2003).

For example, it is easy to see that an agent's learning rate and the system's volatility together help to determine how fast, if ever, the agent will reach its target function. A large learning rate means that an agent will change its decision function to almost match the target function. Meanwhile, a low volatility means that the target function will not move much, so it will be easy for the agent to match it. Thus, if the agents have no impact on each other, that is, $I_{ij} = 0$ for all $i, j$, then the agents will learn their target function and the system will converge. As the impact increases then it becomes more likely that the system will never converge.

Of course, this type of simple analysis ignores a common situation where the agent's high learning rate is coupled with a high impact on other agents' target function making their volatility much higher. These agents might then have to increase their learning rate and thereby increase the original agent's volatility. Equation (5.6) is most helpful in these type of feedback situations.

### 5.5.2 N-Level Agents

N-LEVEL

Another issue that arises when building learning agents is the choice of a modeling level. A designer must decide whether his agent will learn to correlate actions with rewards, or will try to learn to predict the expected actions of others and use these predictions along with knowledge of the problem domain to determine its actions, or will try to learn how other agents build models of other agents, etc. These choices are usually referred to as **n-level** modeling agents—an idea first presented in the recursive modeling method (Gmytrasiewicz and Durfee, 1995) (Gmytrasiewicz and Durfee, 2001).

0-LEVEL

A **0-level** agent is one that does not recognize the existence of other agents in the world. It learns which action to take in each possible state of the world because it receives a reward after its actions. The state is usually defined as a static snapshot of the observable aspects of the agent's environment. A **1-level** agent recognizes that there are other agents in the world whose actions affect its payoff. It also has some knowledge that tells it the utility it will receive given any set of joint actions. This knowledge usually takes the form of a game matrix that only has utility values for the agent. The 1-level agent observes the other agents' actions and builds probabilistic models of the other agents. It then uses these models to predict their action probability distribution and uses these distributions to determine its best possible action. A **2-level** agent believes that all other agents are 1-level agents. It, therefore, builds models of their models of other agents based on the actions it thinks they have seen others take. In essence, the 2-level agent applies the 1-level algorithm to all other agents in an effort to predict their action probability distribution and uses these distributions to determine its best possible actions. A 3-level agent believes that all other agents are 2-level, an so on. Using these guidelines

1-LEVEL

2-LEVEL



para determinar os erros sucessivos esperados para um agente típico $i$. Esta equação baseia-se numa definição de volatilidade em termos de impacto dada por

$$\begin{aligned}\forall_{w \in W}\ v_i^t &= \mathbf{Pr}[\Delta_i^{t+1}(w) \neq \Delta_i^t(w)] \\ &= 1 - \prod_{j \in N_{-i}} (1 - I_{ji}\mathbf{Pr}[\delta_j^{t+1}(w) \neq \delta_j^t(w)]),\end{aligned} \tag{5.7}$$

o que faz a suposição simplificada de que as mudanças nas funções de decisão dos agentes não se anularão mutuamente no cálculo do seu impacto sobre outros agentes. A equação de diferença (5.6) não pode, na maioria das circunstâncias, ser reduzida a uma função de $t$ portanto, ela ainda deve ser iterada. Por outro lado, um estudo cuidadoso da função e do raciocínio por trás da escolha do parâmetro CLRI leva a uma compreensão intuitiva de como as alterações nesses parâmetros serão refletidas na função e, portanto, no sistema. Um projetista experiente pode simplesmente usar esse conhecimento adicional para determinar o comportamento esperado de seu sistema s ob diversas suposições. Um exemplo desta abordagem é mostrado em (Brooksand Durfee, 2003).

Por exemplo, é fácil ver que a taxa de aprendizagem de um agente e a volatilidade do sistema em conjunto ajudam a determinar a rapidez com que, se alguma vez, o agente atingirá a sua função alvo. Uma grande taxa de aprendizado significa que um agente mudará sua função de decisão para quase coincidir com a função alvo. Enquanto isso, uma volatilidade baixa significa que a função alvo não se moverá muito, portanto será fácil para o agente igualá-la. Assim, se os agentes não tiverem impacto entre si, ou seja, $I_{ij} = 0$ para todos $i$, $j$, então os agentes aprenderão sua função alvo e o sistema convergirá. À medida que o impacto aumenta, torna-se mais provável que o sistema nunca convirja.

É claro que este tipo de análise simples ignora uma situação comum em que a elevada taxa de aprendizagem do agente está associada a um elevado impacto na função alvo de outros agentes, tornando a sua volatilidade muito maior. Esses agentes poderão então ter q ue aumentar a sua taxa de aprendizagem e, assim, aumentar a volatilidade do agente original. A equação (5.6) é mais útil neste tipo de situação de feedback.

### 5.5.2 Agentes de Nível N

Outra questão que surge na construção de agentes de aprendizagem é a escolha do nível de modelagem. Um designer deve decidir se seu agente aprenderá a correlacionar ações com recompensas, ou tentará aprender a prever as ações esperadas de outros e usar essas previsões junto com o conhecimento do domínio do problema para determinar suas ações, ou tentará aprender como outros agentes constroem modelos de outros agentes, etc. Essas escolhas são geralmente chamadas de agentes de modelagem de nível n – uma ideia apresentada pela primeira vez no

N-LEVEL

método de modelagem recursiva (Gmytrasiewicz e Durfee, 1995) (Gmytrasiewicz e Durfee, 2001).

Um agente de nível 0 é aquele que não reconhece a existência de outros agentes

0-LEVEL

no mundo. Ele aprende qual ação tomar em cada estado possível do mundo porque recebe uma recompensa após suas ações. O estado é geralmente definido como um instantâneo estático dos aspectos observáveis do ambiente do agente. Um agente de 1 nível

1-LEVEL

reconhece que existem outros agentes no mundo cujas ações afetam o seu retorno. Ele também possui algum conhecimento que lhe indica a utilidade que receberá em qualquer conjunto de ações conjuntas. Esse conhecimento geralmente assume a forma de uma matriz de jogo que possui apenas valores de utilidade para o agente. O agente de 1 nível observa as ações dos demais agentes e constrói modelos probabilísticos dos demais agentes. Em seguida, ele usa esses modelos para prever sua distribuição de probabilidade de ação e usa essas distribuições para determinar sua melhor ação possível. Um agente de 2 níveis acredita que todos os outros agentes são agentes de 1 nível.

2-LEVEL

Portanto, constrói modelos dos seus modelos de outros agentes com base nas ações que pensa ter visto outros realizarem. Em essência, o agente de 2 níveis aplica o algoritmo de 1 nível a todos os outros agentes em um esforço para prever sua distribuição de probabilidade de ação e usa essas distribuições para determinar suas melhores ações possíveis. Um agente de 3 níveis acredita que todos os outros agentes são de 2 níveis e assim por diante. Usando essas diretrizes

we can determine that fictitious play (section 5.3.1) uses 1-level agents while the replicator dynamics (section 5.3.2) uses 0-level agents.

These categorizations help us to determine the relative computational costs of each approach and the machine-learning algorithms that are best suited for that learning problem. 0-level is usually the easiest to implement since it only requires the learning of one function and no additional knowledge. 1-level learning requires us to build a model of every agent and can only be implemented if the agent has the knowledge that tells it which action to take given the set of actions that others have taken. This knowledge must be integrated into the agents. However, recent studies in layered learning (Stone, 2000) have shown how some knowledge could be learned in a "training" situation and then fixed into the agent so that other knowledge that uses the first one can be learned, either at runtime or in another training situation. In general, a change in the level that an agent operates on implies a change on the learning problem and the knowledge built into the agent.

Studies with n-level agents have shown (Vidal and Durfee, 1998a) that an n-level agent will perform better in a society full of (n-1)-level agents, and that the computational costs of increasing a level grow exponentially. Meanwhile, the utility gains to the agent grow smaller as the agents in the system increase their level, within an economic scenario. The reason is that an n-level agent is able to exploit the non-equilibrium dynamics of a system composed of (n-1)-level agents. However, as the agents increase their level the system reaches equilibrium faster so the advantages of strategic thinking are reduced—it is best to play the equilibrium strategy and not worry about what others might do. On the other hand, if all agents stopped learning then it would be very easy for a new learning agent to take advantage of them. As such, the research concludes that some of the agents should do some learning some of the time in order to preserve the robustness of the system, even if this learning does not have any direct results. That is, there appear to be decreasing marginal returns for strategic thinking.

## 5.6 Collective Intelligence

We can also try to use machine learning as a way to automatically build multiagent systems where each one of the agents learns to do what we want. For example, imagine a factory floor with boxes that need to be moved and robots of different abilities. Depending on the size and weight of the boxes, different robots, or combinations of robots, are able to move them. We know that our goal is for all boxes to be placed against the South wall. Then, rather than trying to come up with specific plans or rules of behavior for each robot, we instead install a reinforcement learning algorithm in each one of them and set them off to learn how best to coordinate. The question we must then ask is: what is the reward function for each agent? As we saw in the CLRI model, one agent's actions can have an impact on another agent's target function thus making it hard, or even impossible, for the other agent to converge in its learning. For example, if one robot keeps changing its behavior and moving around randomly then it will be impossible for another agent to learn to predict when they will both be in front of a large box so that they can move it together.

COIN

Collective intelligence (**COIN**) aims to formalize these ideas and determine what kind of rewards we should provide reinforcement-learning agents in order to achieve a desired global behavior (Wolpert and Tumer, 1999). Specifically, we are given a global utility function $U(s, \vec{a}) \rightarrow \Re$ which tells us the value for every vector of actions $\vec{a} = \{a_1, a_2, \ldots, a_n\}$ of the agents and state of the world $s$. The agents are assumed to use some reinforcement learning algorithm, such as Q-learning. Our problem is then to define the agents' reward functions $u_i(s, \vec{a})$ such that the agents will end up with policies $\pi_i^*$ that maximize $U$. Notice that simply setting $u_i(s, \vec{a}) = U(s, \vec{a})$ for all $i$ can lead to agents receiving an uninformative reward. For example, if a confused agents throws itself against a wall while, at the same time, all the other agents cooperate to move the box correctly then the confused agent will receive a high reward and will thus likely continue to throw itself against the wall.



podemos determinar que o jogo fictício (seção 5.3.1) usa agentes de nível 1, enquanto a dinâmica do replicador (seção 5.3.2) usa agentes de nível 0.

Essas categorizações nos ajudam a determinar os custos computacionais relativos de cada abordagem e os algoritmos de aprendizado de máquina mais adequados para esse problema de aprendizado. O nível 0 é geralmente o mais fácil de implementar, pois requer apenas o aprendizado de uma função e nenhum conhecimento adicional. O aprendizado de 1 nível exige que construamos um modelo de cada agente e só pode ser implementado se o agente tiver o conhecimento que lhe diga qual ação tomar, dado o conjunto de ações que outros realizaram. Esse conhecimento deve ser integrado aos agentes. No entanto, estudos recentes sobre aprendizagem em camadas (Stone, 2000) mostraram como algum conhecimento pode ser aprendido numa situação de "treinamento" e depois fixado no agente para que outro conhecimento que utilize o primeiro possa ser aprendido, seja em tempo de execução ou em outra situação de treinamento. Em geral, uma mudança no nível em que um agente opera implica uma mudança no problema de aprendizagem e no conhecimento incorporado ao agente.

Estudos com agentes de nível n mostraram (Vidal e Durfee, 1998a) que um agente de nível n terá melhor desempenho numa sociedade cheia de agentes de nível (n-1), e que os custos computacionais para aumentar um nível crescem exponencialmente. Enquanto isso, os ganhos de utilidade para o agente diminuem à medida que os agentes do sistema aumentam de nível, dentro de um cenário econômico. A razão é que um agente de nível n é capaz de explorar a dinâmica de não equilíbrio de um sistema composto por agentes de nível (n-1). No entanto, à medida que os agentes aumentam o seu nível, o sistema atinge o equilíbrio mais rapidamente, pelo que as vantagens do pensamento estratégico são reduzidas – é melhor seguir a estratégia de equilíbrio e não se preocupar com o que os outros poderão fazer. Por outro lado, se todos os agentes parassem de aprender, seria muito fácil para um novo agente de aprendizagem tirar vantagem deles. Como tal, a investigação conclui que alguns dos agentes devem realizar alguma aprendizagem durante algum tempo, a fim de preservar a robustez do sistema, mesmo que essa aprendizagem não tenha quaisquer resultados diretos. Ou seja, parece haver retornos marginais decrescentes para o pensamento estratégico.

## 5.6 Inteligência Coletiva

Também podemos tentar usar o aprendizado de máquina como forma de construir automaticamente sistemas multiagentes onde cada um dos agentes aprende a fazer o que quiser. Por exemplo, imagine um chão de fábrica com caixas que precisam ser movidas e robôs de diferentes habilidades. Dependendo do tamanho e peso das caixas, diferentes robôs, ou combinações de robôs, são capazes de movê-las. Sabemos que o nosso objetivo é que todas as caixas sejam colocadas contra a parede Sul. Então, em vez de tentar elaborar planos ou regras de comportamento específicos para cada robô, instalamos um algoritmo de aprendizagem por reforço em cada um deles e os i niciamos para aprender a melhor forma de coordenação. A questão que devemos então fazer é: qual é a função de recompensa para cada agente? Como vimos no modelo CLRI , as ações de um agente podem ter impacto na função alvo de outro agente, tornando assim difícil, ou mesmo impossível, que o outro agente convirja em seu aprendizado. Por exemplo, se um robô continuar mudando seu comportamento e se movendo aleatoriamente, será impossível para outro agente aprender a prever quando ambos estarão na frente de uma caixa grande para que possam movê-la juntos.

COIN

A inteligência coletiva (COIN) visa formalizar essas ideias e determinar o que tipo de recompensas que devemos fornecer aos agentes de aprendizagem por reforço, a fim de alcançar um comportamento global desejado (Wolpert e Tumer, 1999). Especificamente, recebemos uma função de utilidade global $U(s, \vec{a}) \rightarrow \Re$ que nos informa o valor de cada vetor de ações $\vec{a} = \{a_1, a_2, \ldots, a_n\}$ dos agentes e do estado do mundo $s$. Supõe-se que os agentes usem algum algoritmo de aprendizagem por reforço, como Q-learning. Nosso problema é então definir as funções de recompensa dos agentes $u_i(s, \vec{a})$ de modo que os agentes terminem com políticas $\pi_i^*$ que maximizem $U$. Observe que simplesmente definir $u_i(s, \vec{a}) = U(s, \vec{a})$ para todos $i$ pode fazer com que os agentes recebam uma recompensa não informativa. Por exemplo, se um agente confuso se atira contra uma parede enquanto, ao mesmo tempo, todos os outros agentes cooperam para mover a caixa corretamente, então o agente confuso receberá uma recompensa elevada e, portanto, provavelmente continuará a atirar-se contra a parede.

Still, in general we do want to align each agent's rewards with the global utility. We can quantify this alignment by defining agent $i$'s preference over $s, \vec{a}$ as

$$P_i(s,\vec{a}) = \frac{\sum_{\vec{a}' \in \vec{A}} \Theta[u_i(s,\vec{a}) - u_i(s,\vec{a}')]}{|\vec{A}|}, \tag{5.8}$$

where $\Theta(x)$ is the Heaviside function which is 1 if $x$ is greater than or equal to 0, otherwise it is 0. Similarly, we can define the global preference function as

$$P(s,\vec{a}) = \frac{\sum_{\vec{a}' \in \vec{A}} \Theta[U(s,\vec{a}) - U(s,\vec{a}')]}{|\vec{A}|}. \tag{5.9}$$

Both of these functions serve to rank the agents', or the global, preferences over the set of all possible states and actions vectors. For example, if a particular $(s,\vec{a})$ is preferred over all others then $P(s,\vec{a}) = 1$.

In general, we might want an agent's preferences to be similar to the system's preferences so that the agent's learning mechanism will try to converge towards the desired system's preferences. A system where, for all agents $i$ it is true that $P_i(s,\vec{a}) = P(s,\vec{a})$ is called **factored**. These systems are nice in that they provide the correct incentives to the agents in all situations. Thus, factored systems are more likely to converge to the set of policies that maximizes $U$.

FACTORED

It is possible that factored systems might not converge because agents' action might have an impact on each other thus changing each other's target function. This idea is captured within the COIN framework as the **opacity** of $u_i(s,\vec{a})$. Specifically, we define the opacity $\Omega_i$ for agent $i$ as

OPACITY

$$\Omega_i(s,\vec{a}) = \sum_{\vec{a}' \in \vec{A}} \mathbf{Pr}[\vec{a}'] \frac{|u_i(s,\vec{a}) - u_i(s,\vec{a}'_{-i},\vec{a}_i)|}{|u_i(s,\vec{a}) - u_i(s,\vec{a}_{-i},\vec{a}'_i)|}. \tag{5.10}$$

The opacity is zero when the agent's rewards are the same regardless of the actions taken by other, that is, when the other agents' actions have no impact on what the agent should do. On the other hand, the opacity gets larger as the other agents' actions change the reward the agent will get. Just like with the CLRI impact, if the opacity is zero for all states and action vectors then the multiagent learning problem is reduced to the single-agent learning problem as the agents have no effect on each other's target function. More generally, a decrease in the opacity amounts to an increase in the signal-to-noise ratio that the agent receives via its reward function.

Systems that are both factored and have zero opacity for all agents are extremely rare, but would be easy to solve. Systems where the opacity is zero for all agents amount to multiple learning problems that do not interfere with each other and are easy to solve, but they are also very rare. Our goal can now be re-stated as that of finding reward functions for each agent that have as low opacity as possible while also being as highly factored as possible.

One solution COIN proposes is the use of the **wonderful life** reward function which gives each agent a reward proportional to its own contribution to the global utility. It implements the same idea as the VCG payments (Chapter 8.4). Formally, agent $i$'s wonderful life reward is given by

WONDERFUL LIFE

$$u_i(s,\vec{a}) = U(s,\vec{a}) - U(s,\vec{a}_{-i}), \tag{5.11}$$

where the $U(s,\vec{a}_{-i})$ represents the utility in state $s$ when agent $i$ does not exist and all other agents take actions as in $\vec{a}_{-i}$. Notice that this equation can be derived directly from the global utility function. Thus, it is applicable to any system where we have a pre-defined utility function $U$. It has been shown that this reward function performs better than using $u_i = U$ and other seemingly appropriate reward functions (Wolpert et al., 1999; Tumer and Wolpert, 2004).

Another solution is the **aristocrat utility**, which is defined as

ARISTOCRAT UTILITY

$$u_i(s,\vec{a}) = U(s,\vec{a}) - \sum_{\vec{a}' \in \vec{A}} \mathbf{Pr}[\vec{a}'] U(s,\vec{a}_{-i},\vec{a}'_i), \tag{5.12}$$



Ainda assim, em geral queremos alinhar as recompensas de cada agente com a utilidade gl obal. Podemos quantificar esse alinhamento definindo a preferência do agente $i$ sobre $s, \vec{a}$ como

$$P_i(s,\vec{a}) = \frac{\sum_{\vec{a}' \in \vec{A}} \Theta[u_i(s,\vec{a}) - u_i(s,\vec{a}')]}{|\vec{A}|}, \tag{5.8}$$

onde $\Theta(x)$ é a função de Heaviside que é 1 se $x$ for maior ou igual a 0, caso contrário é 0. Da mesma forma, podemos definir a função de preferência global como

$$P(s,\vec{a}) = \frac{\sum_{\vec{a}' \in \vec{A}} \Theta[U(s,\vec{a}) - U(s,\vec{a}')]}{|\vec{A}|}. \tag{5.9}$$

Ambas as funções servem para classificar as preferências dos agentes, ou globais, sobre o conjunto de todos os estados e vetores de ações possíveis. Por exemplo, se um determinado $(s,\vec{a})$ for preferido a todos os outros, então $P(s,\vec{a}) = 1$.

Em geral, podemos querer que as preferências de um agente sejam semelhantes às preferências do sistema, de modo que o mecanismo de aprendizagem do agente tente convergir para as preferências do sistema desejado. Um sistema onde, para todos os agentes $i$ é verdade que $P_i(s,\vec{a}) = P(s,\vec{a})$ é chamado fatorado. Esses sistemas são bons porque fornecem

FACTORED

os incentivos corretos aos agentes em todas as situações. Assim, os sistemas fatorados têm maior probabilidade de convergir para o conjunto de políticas que maximiza $U$.

É possível que os sistemas fatorados não convirjam porque a ação dos agentes pode ter um impacto um sobre o outro, alterando assim a função alvo de cada um. Essa ideia é capturada na estrutura COIN como a opacidade de $u_i(s,\vec{a})$. Especificamente, definimos a opacidade $\Omega_i$ para o agente $i$ como

OPACITY

$$\Omega_i(s,\vec{a}) = \sum_{\vec{a}' \in \vec{A}} \mathbf{Pr}[\vec{a}'] \frac{|u_i(s,\vec{a}) - u_i(s,\vec{a}'_{-i},\vec{a}_i)|}{|u_i(s,\vec{a}) - u_i(s,\vec{a}_{-i},\vec{a}'_i)|}. \tag{5.10}$$

A opacidade é zero quando as recompensas do agente são as mesmas independentemente das ações tomadas pelos outros, ou seja, quando as ações dos outros agentes não têm i mpacto no que o agente deve fazer. Por outro lado, a opacidade aumenta à medida que as ações dos outros agentes alteram a recompensa que o agente receberá. Assim como com o CLRI impacto, se a opacidade for zero para todos os estados e vetores de ação, o problema de aprendizagem multiagente será reduzido ao problema de aprendizagem de agente único, pois os agentes não têm efeito na função alvo um do outro. De forma mais geral, uma diminuição na opacidade equivale a um aumento na relação sinal-ruído que o agente recebe através da sua função de recompensa.

Sistemas que são fatorados e têm opacidade zero para todos os agentes são extremamente raros, mas seriam fáceis de resolver. Sistemas onde a opacidade é zero para todos os agentes representam múltiplos problemas de aprendizagem que não interferem entre si e são fáceis de resolver, mas também são muito raros. Nosso objetivo agora pode ser reafirmado como encontrar funções de recompensa para cada agente que tenham a menor opacidade possível e, ao mesmo tempo, sejam tão altamente fatoradas quanto possível.

Uma solução proposta COIN é o uso da maravilhosa função de recompensa de vida que dá a cada agente uma recompensa proporcional à sua própria contribuição para a utilidade global. Ele implementa a mesma ideia dos pagamentos VCG (Capítulo 8.4). Formalmente, a maravilhosa recompensa de vida do agente $i$ é dada por

WONDERFUL LIFE

$$u_i(s,\vec{a}) = U(s,\vec{a}) - U(s,\vec{a}_{-i}), \tag{5.11}$$

onde $U(s,\vec{a}_{-i})$ representa o utilitário no estado $s$ quando o agente $i$ não existe e todos os outros agentes executam ações como em $\vec{a}_{-i}$. Observe que esta equação pode ser derivada diretamente da função de utilidade global. Assim, é aplicável a qualquer sistema onde tenhamos uma função de utilidade pré-definida $U$. Foi demonstrado que esta função de recompensa tem melhor desempenho do que usar $u_i = U$ e outras funções de recompensa aparentemente apropriadas (Wolpert et al., 1999; Tumer e Wolpert, 2004).

Outra solução é a utilidade aristocrata, que é definida como

ARISTOCRAT UTILITY

$$u_i(s,\vec{a}) = U(s,\vec{a}) - \sum_{\vec{a}' \in \vec{A}} \mathbf{Pr}[\vec{a}'] U(s,\vec{a}_{-i},\vec{a}'_i), \tag{5.12}$$

where $\mathbf{Pr}[\vec{a}']$ is the probability that $\vec{a}'$ happens. The aristocrat utility measures the difference in the global utility between the agent's action and its average or expected action. It has been shown that this reward function also performs well, sometimes better than the wonderful life utility (Wolpert and Tumer, 2001) .

## 5.7 Summary

We have seen how the theory of learning in games provides us with various equilibrium solution concepts and often tells us when some of them will be reached by simple learning models. On the other hand, we have argued that the reason learning is used in a multiagent system is often because there is no known equilibrium or the equilibrium point keeps changing due to outside forces. We have also shown how the CLRI theory and n-level agents are attempts to characterize and predict, to a limited degree, the dynamics of a system given some basic learning parameters.

## 5.8 Recent Advances

The GAMUT (Nudelman et al., 2004) software can produce game matrices from thirty-five base classes of games. These matrices can be used to determine how well learning algorithms perform under all possible game types and sizes.

## Exercises

5.1 In general, which game matrices have cycles for fictitious play?



onde Pr$[\vec{a}']$ é a probabilidade de $\vec{a}'$ acontecer. A utilidade aristocrata mede a diferença na utilidade global entre a ação do agente e sua ação média ou esperada. Foi demonstrado que esta função de recompensa também funciona bem, às vezes melhor do que a maravilhosa utilidade de vida (Wolpert e Tumer, 2001).

## 5.7 Resumo

Vimos como a teoria da aprendizagem em jogos nos fornece vários conceitos de solução de equilíbrio e muitas vezes nos diz quando alguns deles serão alcançados por modelos simples de aprendizagem. Por outro lado, argumentamos que a razão pela qual a aprendizagem é usada em um sistema multiagente é muitas vezes porque não há equilíbrio conhecido ou porque o ponto de equilíbrio continua mudando devido a forças externas. Também mostramos como a teoria CLRI e os agentes de nível n são tentativas de caracterizar e prever, até um grau limitado, a dinâmica de um sistema, dados alguns parâmetros básicos de aprendizagem.

## 5.8 Avanços Recentes

O software GAMUT (Nudelman et al., 2004) pode produzir matrizes de jogos a partir de trinta e cinco classes base de jogos. Essas matrizes podem ser usadas para determinar o desempenho dos algoritmos de aprendizagem em todos os tipos e tamanhos de jogos possíveis.

## Exercícios

5.1 Em geral, quais matrizes de jogos possuem ciclos para jogos fictícios?

# Chapter 6

# Negotiation

A negotiation problem is one where multiple agents try to come to an agreement or **deal**. Each agent is assumed to have a preference over all possible deals. The agents send messages to each other in the hope of finding a deal that all agents can agree on. These agents face an interesting problem. They want to maximize their own utility but they also face the risk of a break-down in negotiation, or expiration of a deadline for agreement. As such, each agent must negotiate carefully, trading off any utility it gains from a tentative against a possibly better deal or the risk of a breakdown in negotiation.

decide whether the current deal is good enough or whether it should ask for more and risk agreement failure.

Automated negotiation can be very useful in multiagent systems as it provides a distributed method of aggregating distributed knowledge. That is, in a problem where each agent has different local knowledge negotiation can be an effective method for finding the one global course of action which maximizes utility without having to aggregate all local knowledge in a central location. In fact, the metaphor of autonomous agents cooperating in this manner to solve a problem that cannot be solved by any one agent, due to limited abilities or knowledge, was the central metaphor from which the field of distributed artificial intelligence, later known as multiagent systems, emerged (Davis and Smith, 1983). The metaphor is based on the observation that teams of scientists, businesses, citizens, and others regularly negotiate over future courses of action and the result of these negotiations can incorporate more knowledge than any one individual possesses (Surowiecki, 2005). For example, in a NASA rover mission to Mars the various engineering teams and science teams negotiate over what feature to include in the rover. The scientists are concerned with having the proper equipment in Mars so that they can do good science while the engineers try to ensure that everything will work as expected. Often it is the case that one side does not understand exactly why the other wants or rejects a particular feature but by negotiating with each other they arrive at a rover that is engineered solidly enough to survive the trip to Mars and has enough equipment to do useful science while there. Thus, negotiation results in the aggregation of knowledge from multiple individuals in order to make decisions which are better, for the whole, than if they were made by any one individual.

DEAL

See (Squyres, 2005) for the full story on the MER mission to Mars. An exciting read.

## 6.1 The Bargaining Problem

A specific version of the negotiation problem has been studied in game theory. It is known as the **bargaining problem** (Nash, 1950). In the bargaining problem, we say that each agent $i$ has a utility function $u_i$ defined over the set of all possible deals $\Delta$. That is, $u_i : \Delta \rightarrow \Re$. We also assume that there is a special deal $\delta^-$ which is the no-deal deal. Without loss of generality we will assume that for all agents $u_i(\delta^-) = 0$ so that the agents will prefer no deal than accepting any deal with negative utility. The problem then is finding a protocol $f$ which will lead the agents to the best deal. But, as with all the game theory we have studied, it is not obvious which deal is the best one. Many solutions concepts have been proposed. We provide an overview of them in the next sections.

BARGAINING PROBLEM

See (Osborne and Rubinstein, 1999, Chapter 7) or (Osborne and Rubinstein, 1990) for a more extended introduction to bargaining.



**Capítulo 6**

# Negociação

Um problema de negociação é aquele em que vários agentes tentam chegar a um acordo ou acordo. Presume-se que cada agente tenha preferência sobre todos os negócios possíveis. O os agentes enviam mensagens uns aos outros na esperança de encontrar um acordo com o qual todos os agentes possam concordar. Esses agentes enfrentam um problema interessante. Querem maximizar a sua própria utilidade, mas também enfrentam o risco de uma negociação fracassada ou de expiração do prazo para um acordo. Como tal, cada agente deve negociar cuidadosamente, trocando qualquer utilidade que obtenha com uma tentativa contra um acordo possivelmente melhor ou o risco de uma falha na negociação.

decidir se o acordo atual é bom o suficiente ou se deve pedir mais e arriscar o fracasso do acordo.

A negociação automatizada pode ser muito útil em sistemas multiagentes, pois fornece um método distribuído de agregação de conhecimento distribuído. Isto é, num problema em que cada agente tem um conhecimento local diferente, a negociação pode ser um método eficaz para encontrar o curso de acção global que maximiza a utilidade sem ter de agregar todo o conhecimento local numa localização central. Na verdade, a metáfora de agentes autónomos cooperando desta forma para resolver um problema que não pode ser resolvido por nenhum agente, devido a capacidades ou conhecimentos limitados, foi a metáfora central a partir da qual surgiu o campo da inteligência artificial distribuída, mais tarde conhecida como sistemas multiagentes (Davis e Smith, 1983). A metáfora baseia-se na observação de que equipas de cientistas, empresas, cidadãos e outros negociam regularmente sobre cursos de acção futuros e o resultado destas negociações pode incorporar mais conhecimento do que qualquer indivíduo possui (Surowiecki, 2005). Por exemplo, numa missão rover da NASA a Marte, as várias equipas de engenharia e as equipes científicas negociam qual recurso incluir no rover. Os cientistas estão preocupados em ter os equipamentos adequados em Marte para que possam fazer boa ciência enquanto os engenheiros tentam garantir que tudo funcionará conforme o esperado. Muitas vezes acontece que um lado não entende exatamente por que o outro quer ou rejeita uma característica específica, mas, ao negociar entre si, eles chegam a um veículo espacial que é projetado de forma sólida o suficiente para sobreviver à viagem a Marte e tem equipamento suficiente para fazer ciência útil enquanto estiver lá. Assim, a negociação resulta na agregação de conhecimentos de múltiplos indivíduos, a fim de tomar decisões que sejam melhores, para o todo, do que se fossem tomadas por qualquer indivíduo.

DEAL

Veja (Squyres, 2005) a história completa sobre a missão MER a Marte. Uma leitura emocionante.

## 6.1 O problema da negociação

Uma versão específica do problema de negociação foi estudada na teoria dos jogos. É conhecido como o problema da barganha (Nash, 1950). No problema da negociação, digamos que cada agente $i$ tem uma função de utilidade $u_i$ definida sobre o conjunto de todos os negócios possíveis $\Delta$. Isto é, $u_i : \Delta \rightarrow \Re$. Também assumimos que existe um acordo especial $\delta^-$ que é o acordo sem acordo. Sem perda de generalidade, assumiremos que para todos os agentes $u_i(\delta^-) = 0$ de modo que os agentes preferirão nenhum acordo a aceitar qualquer acordo com utilidade negativa. O problema então é encontrar um protocolo $f$ que leve os agentes ao melhor negócio. Mas, como acontece com toda a teoria dos jogos que estudamos, não é óbvio qual negócio é o melhor. Muitos conceitos de soluções foram propostos. Forneceremos uma visão geral deles nas próximas seções.

BARGAINING PROBLEM

Ver (Osborne e Rubinstein, 1999, Capítulo 7) ou (Osborne e Rubinstein, 1990) para uma introdução mais extensa à negociação.

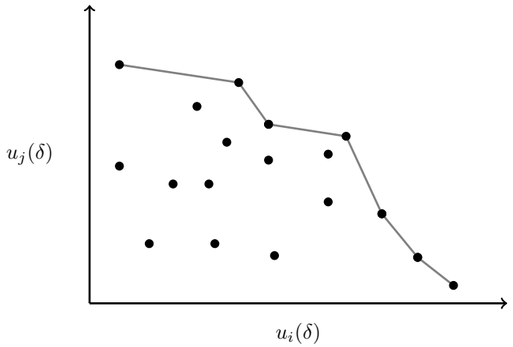



**Figure 6.1:** The dots represent possible deals. The deals on the gray line are Pareto optimal. The line is the Pareto frontier.

### 6.1.1 Axiomatic Solution Concepts

AXIOMATIC

An **axiomatic** solution concept is one which we can formally describe and then simply declare it to be our most desirable solution concept simply because we believe it satisfies all the important requirements. There are many possible requirements we might want to impose on a possible solution deal. The most obvious one is Pareto optimality.

**Definition 6.1** (Pareto optimal)**.** *A deal $\delta$ is Pareto optimal if there is no other deal such that everyone prefers it over $\delta$. That is, there is no $\delta'$ such that*

$$\forall_i u_i(\delta') > u_i(\delta).$$

This is an obvious requirement because if a deal is not Pareto optimal then that means that there is some other deal which all of the agents like better. As such, it does not make sense to use the non-Pareto deal when we could find this other deal which everyone prefers.

PARETO FRONTIER

With only two agents, $i$ and $j$, we can visualize the set of Pareto-optimal deals by using an $xy$-plot where the $x$ and $y$ coordinates are the utilities each agent receives for a deal. Each deal then becomes a point in the graph, as seen in figure 6.1. The **pareto frontier** is represented by the line shown in the upper-right which connects all the Pareto deals.

We might also want a solution that remains the same regardless of the magnitude of an agent's utility values.

**Definition 6.2** (Independence of utility units)**.** *A negotiation protocol is independent of utility units if when given $U$ it chooses $\delta$ and when given $U' = \{(\beta_1 u_1, \ldots, \beta_I u_I) : u \in U\}$ it chooses $\delta'$ where*

$$\forall_i\ u_i(\delta') = \beta_i u_i(\delta).$$

That is, if an agent in a protocol that is independent of utility units used to get a utility of 10 from the result deal and now has multiplied its utility function by 5 then that agent will now get a utility of 50 from the new resulting deal. So, the utilities the agents receive remain proportional under multiplication.

**Definition 6.3** (Symmetry)**.** *A negotiation protocol is symmetric if the solution remains the same as long as the set of utility functions $U$ is the same, regardless of which agent has which utility.*

That is, if two agents swapped utility function then they also end up swapping the utilities they get from the resulting deal. In other words, the specific agents do not matter, the only thing that matters are the utility functions.

**Definition 6.4** (Individual rationality)**.** *A deal $\delta$ is individually rational if*

$$\forall_i\ u_i(\delta) \geq u_i(\delta^-).$$



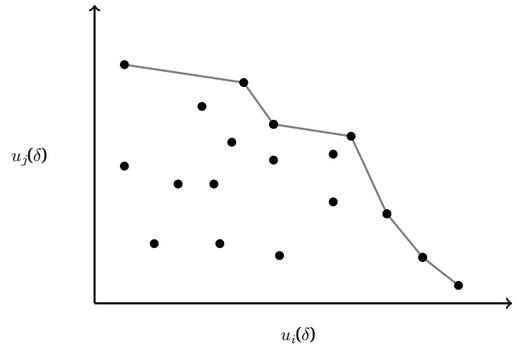



Figura 6.1: Os pontos representam possíveis negócios. As negociações na linha cinza são ótimas de Pareto. A linha é a fronteira de Pareto.

### 6.1.1 Conceitos de Solução Axiomática

AXIOMATIC

Um conceito de solução axiomática é aquele que podemos descrever formalmente e depois simplesmente declaramos que ele é o nosso conceito de solução mais desejável, simplesmente porque acreditamos que ele satisfaz todos os requisitos importantes. Existem muitos requisitos possíveis que podemos querer impor a um possível acordo de solução. A mais óbvia é a otimalidade de Pareto.

*Definição 6.1 (óptimo de Pareto). Um acordo $\delta$ é ótimo de Pareto se não houver outro acordo tal que todos o prefiram a $\delta$. Ou seja, não existe $\delta'$ tal que*

$$\forall_i u_i(\delta') > u_i(\delta).$$

Este é um requisito óbvio porque se um negócio não for ótimo de Pareto, isso significa que existe algum outro negócio que todos os agentes gostam mais. Como tal, não faz sentido utilizar o acordo não-Pareto quando poderíamos encontrar este outro acordo que todos preferem.

PARETO FRONTIER

Com apenas dois agentes, $i$ e $j$, podemos visualizar o conjunto de negócios ideais de Pareto usando um gráfico $xy$ onde as coordenadas $x$ e $y$ são as utilidades que cada agente recebe por um negócio. Cada negócio torna-se então um ponto no gráfico, como pode ser visto na figura 6.1. A fronteira de Pareto é representada pela linha mostrada no canto superior direito que conecta todas as negociações de Pareto.

Poderíamos também querer uma solução que permaneça a mesma independentemente da magnitude dos valores de utilidade de um agente.

*Definição 6.2 (Independência das unidades de utilidade). Um protocolo de negociação é independente de unidades de utilidade se, quando fornecido $U$ele escolhe $\delta$e quando fornecido $U' = \{(\beta_1 u_1, \ldots, \beta_I u_I): u \in U\}$ ele escolhe $\delta'$ onde*

$$\forall_i\ u_i(\delta') = \beta_i u_i(\delta).$$

Ou seja, se um agente num protocolo que é independente de unidades de utilidade costumava obter uma utilidade de 10 do acordo resultante e agora multiplicou a sua função de utilidade por 5, então esse agente obterá agora uma utilidade de 50 do novo acordo resultante. Assim, as utilidades que os agentes recebem permanecem proporcionais na multiplicação.

*Definição 6.3 (Simetria). Um protocolo de negociação é simétrico se a solução permanecer a mesma, desde que o conjunto de funções de utilidade $U$ seja o mesmo, independentemente de qual agente tenha qual utilidade.*

Ou seja, se dois agentes trocaram de função de utilidade, eles também acabam trocando as utilidades que obtêm do acordo resultante. Em outras palavras, os agentes específicos não importam, a única coisa que importa são as funções de utilidade.

*Definição 6.4 (Racionalidade individual). Um acordo $\delta$ é individualmente racional se*

$$\forall_i\ u_i(\delta) \geq u_i(\delta^-).$$

So, a deal is individual rational if $u_i(\delta) \geq 0$ since we will be assuming that $u_i(\delta^-) = 0$. That is, a deal is individually rational if all the agents prefer it over not reaching an agreement.

**Definition 6.5** (Independece of irrelavant alternatives)**.** *A negotiation protocol is independent of irrelevant alternatives if it is true that when given the set of possible deals $\Delta$ it chooses $\delta$ and when given $\Delta' \subset \Delta$ where $\delta \in \Delta'$ it again chooses $\delta$, assuming $U$ stays constant.*

That is, a protocol is independent of irrelevant alternative if the deal it chooses does not change after we remove a deal that lost. Only removal of the winning deal changes the deal the protocol chooses.

Given these requirements we can now consider various possible solutions to a negotiation problem. In the **egalitarian solution** the gains from cooperation are split equally among the agents. That is, the egalitarian deal is the one where all the agents receive the same utility and the sum of their utilities is maximal, that is

EGALITARIAN SOLUTION

$$\delta = \arg\max_{\delta' \in E} \sum_i u_i(\delta') \tag{6.1}$$

where $E$ is the set of all deals where all agents receive the same utility, namely

$$E = \{\delta \,|\, \forall_{i,j} u_i(\delta) = u_j(\delta)\}.$$

**egalitarian** n: one who believes in the equality of all people.

We can find the egalitarian solution visually for the two agent case by simply drawing the line $y = x$ and finding the deal on this line that is farther away from the origin, as seen in figure 6.2. Note that the egalitarian deal in this case is not Pareto optimal. However, if we allowed all possible deals (the graph would be solid black as every pair of $x, y$ coordinates would represent a possible deal) then the egalitarian deal would also be a Pareto deal. Specifically, it would be the point on the $y = x$ line which marks the intersection with the Pareto frontier. Also note that the egalitarian deal does not satisfy the independence of utility units requirement since it assumes the utility units for the agents are comparable, in fact, it assumes that all agents' utility functions use the same units.

A variation on the pure egalitarian deal is the **egalitarian social welfare** solution which is the deal that maximizes the utility received by the agent with the lowest utility. That is, it is the deal $\delta$ that satisfies

EGALITARIAN SOCIAL WELFARE

$$\delta = \arg\max_{\delta} \min_i u_i(\delta). \tag{6.2}$$

The egalitarian social welfare solution is especially useful in scenarios where no deal exists which provides all agents with the same utility since every problem is guaranteed to have an egalitarian social welfare solution. However, the solution itself can in some cases seem very un-egalitarian. For example, a deal where two agents receive utilities of 10 and 100 respectively is the egalitarian social welfare solution even when another deal exists which gives the agents 9 and 11 respectively.

The **utilitarian solution** is the deal that maximizes the sum of the utilities, that is

UTILITARIAN SOLUTION

$$\delta = \arg\max \sum_i u_i(\delta). \tag{6.3}$$

**utilitarian** n : someone who believes that the value of a thing depends on its utility.

The utilitarian deal is, by definition, a Pareto optimal deal. There might be more than one utilitarian deals in the case of a tie. The utilitarian deal violates the independence of utility units assumption as it also assumes utilities are comparable.

We can find the utilitarian deal visually for the two agent case by drawing a line with slope of $-1$ and, starting at the top right, moving it perpendicular to $y = x$ until it intersects a deal. The first deals intersected by the line are utilitarian deals, as shown in figure 6.3.

The utilitarian and egalitarian solutions both seem like fairly reasonable solutions. Unfortunately, both violate the independence of utility units assumption. It



Portanto, um acordo é racional individual se $u_i(\delta) \geq 0$ já que assumiremos que$u_i(\delta^-) = 0$. Ou seja, um acordo é individualmente racional se todos os agentes o preferirem a não chegar a um acordo.

*Definição 6.5 (Independência de alternativas irrelevantes). Um protocolo de negociação é independente de alternativas irrelevantes se for verdade que quando dado o conjunto de acordos possíveis $\Delta$ ele escolhe $\delta$ e quando dado $\Delta' \subset \Delta$ onde $\delta \in \Delta'$ ele nova mente escolhe $\delta$, assumindo que $U$ permanece constante.*

Ou seja, um protocolo é independente de alternativas irrelevantes se o acordo que ele escolhe não muda depois de removermos um acordo que foi perdido. Somente a remoção da negociação vencedora altera a negociação escolhida pelo protocolo.

Tendo em conta estes requisitos, podemos agora considerar várias soluções possíveis para um problema de negociação. Na solução igualitária, os ganhos da cooperação são divididos igualmente entre os agentes. Ou seja, o acordo igualitário é aquele onde todos os agentes recebem a mesma utilidade e a soma de suas utilidades é máxima, ou seja

EGALITARIAN SOLUTION

$$\delta = \arg\max_{\delta' \in E} \sum_i u_i(\delta') \tag{6.1}$$

onde $E$ é o conjunto de todos os negócios onde todos os agentes recebem a mesma utilidade, ou seja,

$$E = \{\delta \,|\, \forall_{i,j} u_i(\delta) = u_j(\delta)\}.$$

igualitário s: aquele que acredita na igualdade de todas as pessoas.

Podemos encontrar visualmente a solução igualitária para o caso de dois agentes simplesmente traçando a linha $y = x$ e encontrando o acordo nesta linha que está mais distante da origem, como visto na figura 6.2. Observe que o acordo igualitário neste caso não é o ótimo de Pareto. No entanto, se permitíssemos todos os acordos possíveis (o gráfico seria preto sólido, pois cada par de coordenadas $x, y$ representaria um acordo possível), então o acordo igualitário também seria um acordo de Pareto. Especificamente, seria o ponto da linha $y = x$ que marca a intersecção com a fronteira de Pareto. Observe também que o acordo igualitário não satisfaz o requisito de independência das unidades de utilidade, uma vez que assume que as unidades de utilidade para os agentes são comparáveis; na verdade, assume que todas as funções de utilidade dos agentes utilizam as mesmas unidades.

Uma variação do acordo igualitário puro é o acordo igualitário de bem-estar social. solução que é o negócio que maximiza a utilidade recebida pelo agente com menor utilidade. Ou seja, é o acordo $\delta$ que satisfaz

EGALITARIAN SOCIAL WELFARE

$$\delta = \arg\max_{\delta} \min_i u_i(\delta). \tag{6.2}$$

A solução igualitária de bem-estar social é especialmente útil em cenários onde não existe um acordo que proporcione a todos os agentes a mesma utilidade, uma vez que é garantido que cada problema tenha uma solução igualitária de bem-estar social. No entanto, a solução em si pode, em alguns casos, parecer muito pouco igualitária. Por exemplo, um acordo em que dois agentes recebem utilidades de 10 e 100, respectivamente, é a solução igualitária de bem-estar social, mesmo quando existe outro acordo que dá aos agentes 9 e 11, respectivamente.

A solução utilitária é o acordo que maximiza a soma das utilidades,

UTILITARIAN SOLUTION

aquilo é

$$\delta = \arg\max \sum_i u_i(\delta). \tag{6.3}$$

utilitário n: alguém que acredita que o valor de uma coisa depende de sua utilidade.

O acordo utilitário é, por definição, um acordo ótimo de Pareto. Pode haver mais de um acordo utilitário em caso de empate. O acordo utilitário viola a suposição de independência das unidades de serviços públicos, pois também pressupõe que os serviços públicos são comparáveis.

Podemos encontrar visualmente o acordo utilitário para o caso de dois agentes desenhando uma linha com inclinação de $-1$ e, começando no canto superior direito, movendo-a perpendicularmente a $y = x$ até cruzar um acordo. Os primeiros negócios intersectados pela linha são negócios utilitários, como mostra a figura 6.3.

As soluções utilitárias e igualitárias parecem ambas soluções bastante razoáveis. Infelizmente, ambos violam a suposição de independência das unidades de utilidade.

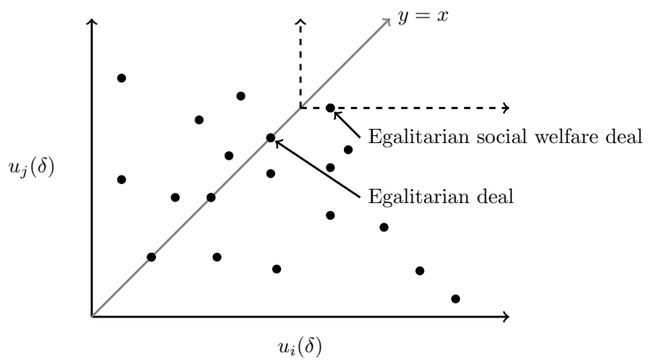



**Figure 6.2:** Egalitarian and egalitarian social welfare deals. The egalitarian social welfare deal can be found by moving the perpendicular dashed lines from a point high in the $y = x$ line down to $0, 0$ until they touch the first deal.

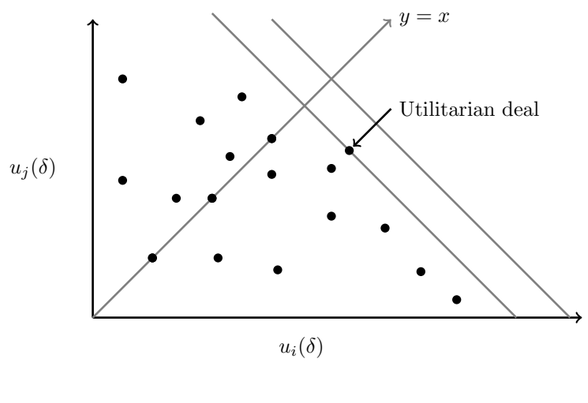



**Figure 6.3:** Utilitarian deal. The line $y = b - x$ starts with a very large $b$ which is reduced until the line intersects a deal.

Nash bargaining solution

was Nash who proposed a solution which does not violate this assumption. The **Nash bargaining solution** is the deal that maximizes the product of the utilities. That is,

$$\delta = \arg\max_{\delta'} \prod u_i(\delta'). \tag{6.4}$$

The Nash solution is also Pareto efficient (Definition 6.1), independent of utility units (Definition 6.2), symmetric (Definition 6.3), independent of irrelevant alternatives (Definition 6.5). In fact, it is the *only* solution that satisfies these four requirements (Nash, 1950). This means that if we want a solution that satisfies these four requirements then our only choice is the Nash bargaining solution.

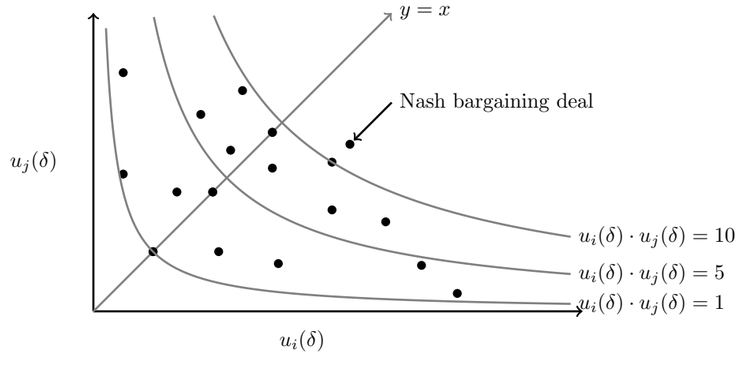



**Figure 6.4:** Nash bargaining deal.



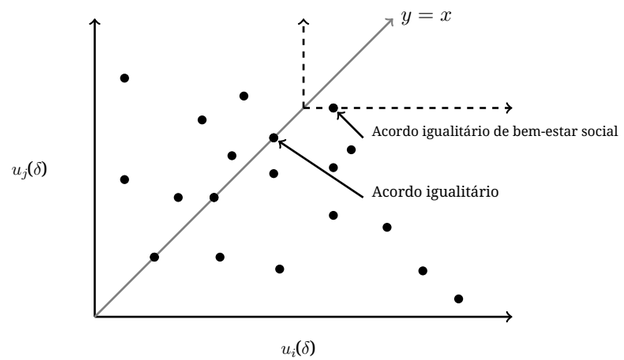



Figura 6.2: Acordos de bem-estar social igualitários e igualitários. O acordo igualitário de bem-estar social pode ser encontrado movendo as linhas tracejadas perpendiculares de um ponto superior na$y = x$ linha até 0,0 até tocarem o primeiro acordo.

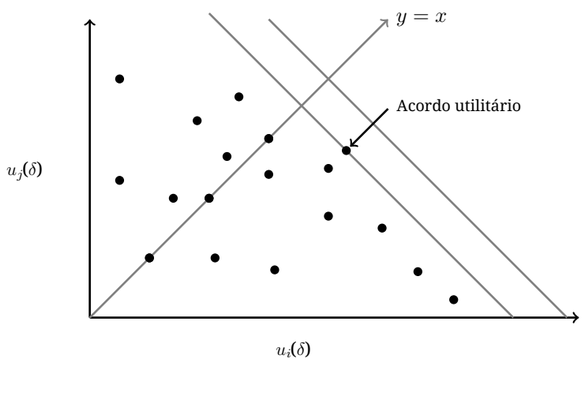



Figura 6.3: Acordo utilitário. A linha $y = b - x$ começa com um $b$ muito grande que é reduzido até que a linha cruze um acordo.

Nash bargaining solution

foi Nash quem propôs uma solução que não viola esta suposição. A solução de barganha de Nash é o acordo que maximiza o produto das concessionárias.
Aquilo é,

$$\delta = \arg\max_{\delta'} \prod u_i(\delta'). \tag{6.4}$$

A solução de Nash também é Pareto eficiente (Definição 6.1), independente de unidades de utilidade (Definição 6.2), simétrica (Definição 6.3), independente de alternativas irrelevantes (Definição 6.5). Na verdade, é a única solução que satisfaz estes quatro requisitos (Nash, 1950). Isto significa que se quisermos uma solução que satisfaça estes quatro requisitos então a nossa única escolha é a solução de negociação Nash.

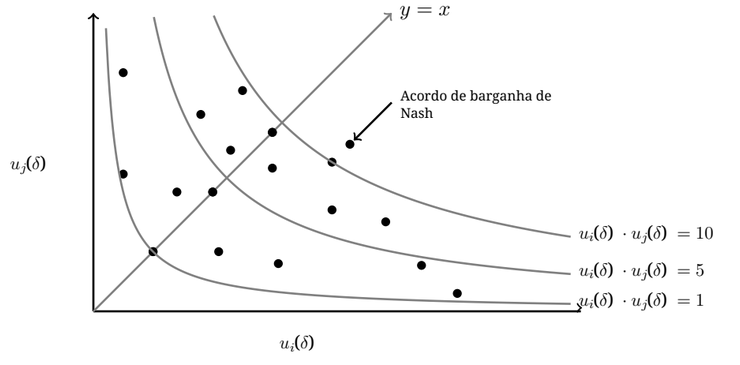



Figura 6.4: Negociação de Nash.

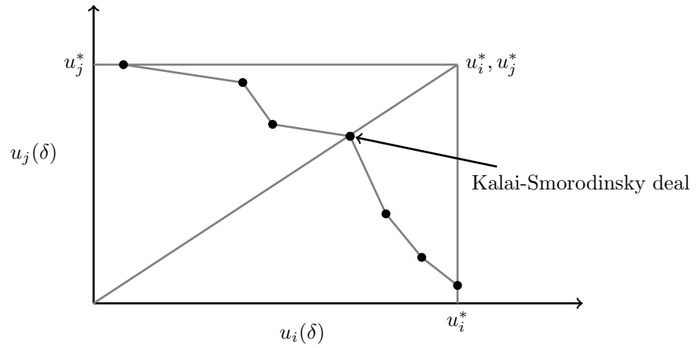



**Figure 6.5:** Kalai-Smorodinsky bargaining deal. We only show deals on the Pareto frontier.

Figure 6.4 shows a visualization of the Nash bargaining deal. Each curve represents all pairs of utilities that have the same product, that is, the line $y = c/x$. As we move northeast, following the line $y = x$, we cut across indifference curves of monotonically higher products. As such, the last deal to intersect a curve is the one which maximizes the product of the utilities, so it is the Nash bargaining solution.

Yet another possible solution is to find the deal that distributes the utility in proportion to the maximum that the agent could get (Kalai and Smorodinsky, 1975). Specifically, let $u_i^*$ be the maximum utility that $i$ could get from the set of all deals in the Pareto frontier. That is, the utility that $i$ would receive if he got his best possible deal from the Pareto frontier. Then, find the deal that lies in the line between the point $\delta^-$ and the point $(u_i^*, u_j^*)$. This solution is known as the **Kalai-Smorodinsky solution**. If all deals are possible then this solution is Pareto optimal. However, if there are only a finite set of possible deals then this solution might not exist. That is, there might not be a point in the specified line. This is a serious drawback to the use of the Kalai-Smorodinsky solution in a discrete setting.

KALAI-SMORODINSKY SOLUTION

The Kalai-Smorodinsky solution, like the Nash bargaining solution, is also independent of utility units requirement. On the other hand, it is not independent of irrelevant alternatives.

There is no general agreement as to which one of these axiomatic solutions is better. The social sciences try to develop experiments that will tell us which one of the solutions, if any, is arrived at by humans engaged in negotiation. Others try to justify some of these solutions as more fair than others. We, as multiagent designers, will probably find that all of them will be useful at some point, depending on the requirements of the system we are building.

### 6.1.2 Strategic Solution Concepts

Another way to think about what solution will be arrived at in a bargaining problem is to formalize the bargaining process, assume rational agents, and then determine their equilibrium strategies for their bargaining process. That is, define the solution concept to be the solution that is reached by automated rational agents in a bargaining problem. This method does raise one large obstacle: we need to first formally define a bargaining process that allows all the same "moves" as real-life bargaining. If you have ever haggled over the price of an item then you know that it is impossible to formalize all the possible moves of market-vendor. We need a negotiation protocol that is simple enough to be formally analyzed but still allows the agents to use most of their moves.

One such model is the **Rubinstein's alternating offers** model (Rubinstein, 1982). In this model two agents try to reach agreement on a deal. The agents can take actions only at discrete time steps. In each time step one of the agents proposes a deal $\delta$ to the other who either accepts it or rejects it. If the offer is rejected then

RUBINSTEIN'S ALTERNATING OFFERS



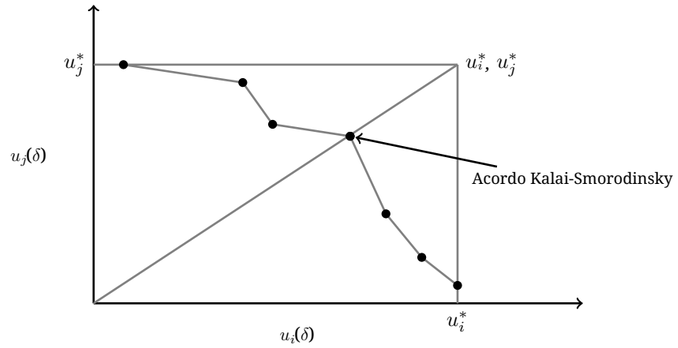



Figura 6.5: Acordo Kalai-Smorodinsky. Mostramos apenas negócios na fronteira de Pareto.

A Figura 6.4 mostra uma visualização do acordo de Nash. Cada curva representa todos os pares de utilidades que possuem o mesmo produto, ou seja, a reta $y = c/x$. À medida que avançamos para nordeste, seguindo a linha $y = x$, atravessamos curvas de indiferença de produtos monotonicamente mais elevados. Como tal, a última transação a cruzar uma curva é aquela que maximiza o produto das utilidades, portanto é a solução de barganha de Nash.

Ainda outra solução possível é encontrar o acordo que distribui a utilidade proporcionalmente ao máximo que o agente poderia obter (Kalai e Smorodinsky, 1975). Especificamente, seja $u_i^*$ a utilidade máxima que $i$ poderia obter do conjunto de todas as negociações na fronteira de Pareto. Ou seja, a utilidade que $i$ receberia se conseguisse o melhor negócio possível na fronteira de Pareto. Em seguida, encontre o acordo que está na linha entre o ponto $\delta^-$ e o ponto $(u_i^*, u_j^*)$. Esta solução é conhecida como solução Kalai-Smorodinsky. Se todos os negócios forem possíveis, então esta solução é Pareto ideal. No entanto, se houver apenas um conjunto finito de negócios possíveis, esta solução poderá não existir. Ou seja, pode não haver um ponto na linha especificada. Esta é uma séria desvantagem para o uso da solução Kalai-Smorodinsky em um ambiente discreto.

KALAI-SMORODINSKY SOLUTION

A solução Kalai-Smorodinsky, tal como a solução de negociação de Nash, também é independente da necessidade de unidades de utilidade. Por outro lado, não é independente de alternativas irrelevantes.

Não existe um acordo geral sobre qual destas soluções axiomáticas é melhor. As ciências sociais tentam desenvolver experiências que nos dirão qual das soluções, se houver, é alcançada pelos humanos envolvidos na negociação. Outros tentam justificar algumas destas soluções como mais justas do que outras. Nós, como projetistas multiagentes, provavelmente descobriremos que todos eles serão úteis em algum momento, dependendo dos requisitos do sistema que estamos construindo.

#### 6.1.2 Conceitos de Solução Estratégica

Outra forma de pensar sobre a solução a que se chegará num problema de negociação é formalizar o processo de negociação, assumir agentes racionais e depois determinar as suas estratégias de equilíbrio para o seu processo de negociação. Ou seja, defina o conceito de solução como sendo a solução alcançada por agentes racionais automatizados num proble ma de negociação. Este método levanta um grande obstáculo: precisamos primeiro definir formalmente um processo de negociação que permita todos os mesmos "movimentos" que a negociação na vida real. Se você já pechinchou o preço de um item, sabe que é impossível formalizar todos os movimentos possíveis do vendedor no mercado. Precisamos de um protocolo de negociação que seja simples o suficiente para ser analisado formalmente, mas que ainda permita que os agentes utilizem a maior parte de seus movimentos.

Um desses modelos é o modelo de ofertas alternadas de Rubinstein (Rubinstein, 1982). Neste modelo, dois agentes tentam chegar a um acordo sobre um acordo. Os agentes podem realizar ações apenas em etapas discretas. Em cada etapa de tempo, um dos agentes propõe um acordo $\delta$ ao outro, que o aceita ou rejeita. Se a oferta for rejeitada então

RUBINSTEIN'S ALTERNATING OFFERS

we move to the next time step where the other agent gets to propose a deal. Once a deal has been rejected it is considered void and cannot be accepted at a later time. The agents, however, are always free to propose any deal and to accept or reject any deal as they wish. We further assume that the agents know each other's utility functions.

The alternating offers models, as it stands, does not have a dominant strategy. For example, imagine that the two agents are bargaining over how to divide a dollar. Each agent wants to keep the whole dollar to itself and leave the other agent with nothing. Under the basic alternating offers protocol the agents' best strategy is to keep proposing this deal to the other agent. That is, each agent keeps telling the other one "I propose that I keep the whole dollar" and the other agent keeps rejecting this proposal. This scenario is not very interesting.

In order to make it more interesting, we further assume that time is valuable to the agents. That is, the agents' utility for all possible deals is reduced as time passes. For example, imagine that instead of haggling over a dollar the agents are haggling over how to split an ice cream sundae which is slowly melting and the agents hate melted ice cream. Formally, we say that agent $i$'s utility at time $t$ for deal $\delta$ is given by $\lambda_i^t u_i(\delta)$ where $\lambda_i$ is $i$'s discount factor, similarly the utility for $j$ is $\lambda_j^t u_j(\delta)$. Thus, the agents' utility for every possible deal decreases monotonically as a function of time with a discount factor given by $\lambda$. Note that if $\lambda_i = 0$ then the agent must agree to a deal at time 0 since after that it will receive a utility of 0 for any deal. Conversely, if $\lambda_i = 1$ then the agent can wait forever without any utility loss.

Furthermore, lets assume that the agents' utilities are linear and complementary. Specifically, imagine that the deal $\delta$ is simply a number between 0 and 1 and represents the amount of utility an agent receives so that $u_i(\delta) = \delta$ and $u_j(\delta) = 1 - \delta$. In this scenario, it can be shown that a unique subgame perfect equilibrium strategy exists.

**Theorem 6.1** (Alternating Offers Bargaining Strategy)**.** *The Rubinstein's alternating offers game where the agents have complementary linear utilities has a unique subgame perfect equilibrium strategy where*

- *agent $i$ proposes a deal*
$$\delta_i^* = \frac{1 - \lambda_j}{1 - \lambda_i \lambda_j}$$
*and accepts the offer $\delta_j$ from $j$ only if $u_i(\delta_j) \leq u_i(\delta_j^*)$,*
- *agent $j$ proposes a deal*
$$\delta_j^* = \frac{1 - \lambda_i}{1 - \lambda_i \lambda_j}$$
*and accepts the offer $\delta_i$ from $i$ only if $u_j(\delta_i) \leq u_j(\delta_i^*)$.*

*(Rubinstein, 1982; Muthoo, 1999).*

We can understand how these deals are derived by noting that since both agents have utilities that decrease with time the best deal will be reached in the first step. That means that each agent must propose a deal that the other will accept. Specifically, agent $i$ must propose a deal $\delta_i^*$ such that $u_j(\delta_i^*) = \lambda_j u_j(\delta_j^*)$ because if it proposes a deal that gives $j$ lower utility then $j$ will reject it and if it proposes a deal that gives $j$ higher utility then $i$ is needlessly giving up some of its own utility. Conversely, $j$ must propose a deal $\delta_j^*$ such that $u_i(\delta_j^*) = \lambda_i u_i(\delta_i^*)$. We thus have two equations.

Since $u_i(\delta) = \delta$ and $u_j(\delta) = 1 - \delta$, we can replace these definitions into the above equations to get

$$1 - \delta_i^* = \lambda_j (1 - \delta_j^*) \tag{6.5}$$
$$\delta_j^* = \lambda_i \delta_i^*. \tag{6.6}$$



passamos para a próxima etapa de tempo em que o outro agente propõe um acordo. Uma vez que um acordo tenha sido rejeitado, ele será considerado nulo e não poderá ser aceito posteriormente. Os agentes, no entanto, são sempre livres para propor qualquer acordo e aceitar ou rejeitar qualquer acordo que desejarem. Assumimos ainda que os agentes conhecem as funções de utilidade uns dos outros.

Os modelos de ofertas alternadas, tal como estão, não possuem uma estratégia dominante. Por exemplo, imagine que os dois agentes estejam negociando como dividir um dólar. Cada agente quer ficar com todo o dólar para si e deixar o outro agente sem nada. No protocolo básico de ofertas alternadas, a melhor estratégia dos agentes é continuar propondo esse negócio ao outro agente. Ou seja, cada agente fica dizendo ao outro "Proponho que fique com o dólar inteiro" e o outro agente continua rejeitando essa proposta. Este cenário não é muito interessante.

Para torná-lo mais interessante, assumimos ainda que o tempo é valioso para os agentes. Ou seja, a utilidade dos agentes para todos os negócios possíveis é reduzida com o passar do tempo. Por exemplo, imagine que em vez de regatear um dólar, os agentes estão a discutir como dividir um sundae de gelado que está a derreter lentamente e os agentes odeiam gel ado derretido. Formalmente, dizemos que a utilidade do agente $i$ no momento $t$ do negócio $\delta$ é dada por $\lambda_i^t u_i(\delta)$ onde $\lambda_i$ é o fator de desconto de $i$, da mesma forma que a utilidade para $j$ é $\lambda_j^t u_j(\delta)$. Assim, a utilidade dos agentes para cada negócio possível diminui monotonicamente em função do tempo com um fator de desconto dado por $\lambda$. Observe que se $\lambda_i = 0$ então o agente deve concordar com um acordo no tempo 0, pois depois disso ele receberá a utilidade de 0 para qualquer acordo. Por outro lado, se $\lambda_i = 1$ então o agente pode esperar para sempre sem qualquer perda de utilidade.

Além disso, vamos supor que as utilidades dos agentes sejam lineares e complementares. Especificamente, imagine que o acordo $\delta$ é simplesmente um nú mero entre 0 e 1 e representa a quantidade de utilidade que um agente recebe de modo que $u_i(\delta) = \delta$ e $u_j(\delta) = 1 - \delta$. Neste cenário, pode-se mostrar que existe uma estratégia única de equilíbrio perfeito em subjogos.

*Teorema 6.1 (Estratégia de Negociação de Ofertas Alternadas). O jogo de ofertas alternadas de Rubinstein, onde os agentes têm utilidades lineares complementares, possui um único subjogo de estratégia de equilíbrio perfeito, onde*

- *agente $i$ propõe um acordo*
$$\delta_i^* = \frac{1 - \lambda_j}{1 - \lambda_i \lambda_j}$$
*e aceita a oferta $\delta_j$ de $j$ somente se $u_i(\delta_j) \leq u_i(\delta_j^*)$,*
- *agente $j$ propõe um acordo*
$$\delta_j^* = \frac{1 - \lambda_i}{1 - \lambda_i \lambda_j}$$
*e aceita a oferta $\delta_i$ de $i$ somente se $u_j(\delta_i) \leq u_j(\delta_i^*)$.*

*(Rubinstein, 1982; Muthoo, 1999).*

Podemos entender como esses negócios são derivados observando que, como ambos os agentes têm utilidades que diminuem com o tempo, o melhor negócio será alcançado na primeira etapa. Isso significa que cada agente deve propor um acordo que o outro aceite. Especificamente, o agente $i$ deve propor um acordo $\delta_i^*$ tal que $u_j(\delta_i^*) = \lambda_j u_j(\delta_j^*)$ porque se ele propor um acordo que dê $j$ menor utilidade então $j$ irá rejeitá-lo e se ele propor um acordo que dê $j$ maior utilidade então $i$ está abrindo mão desnecessariamente de parte de sua própria utilidade. Por outro lado, $j$ deve propor um acordo $\delta_j^*$ tal que $u_i(\delta_j^*) = \lambda_i u_i(\delta_i^*)$. Temos assim duas equações.

Como $u_i(\delta) = \delta$ e $u_j(\delta) = 1 - \delta$, podemos substituir essas definições nas equações acima para obter

$$1 - \delta_i^* = \lambda_j (1 - \delta_j^*) \tag{6.5}$$
$$\delta_j^* = \lambda_i \delta_i^*. \tag{6.6}$$

MONOTONIC-CONCESSION

1 $\delta_i \leftarrow \arg\max_\delta u_i(\delta)$
2 Propose $\delta_i$
3 Receive $\delta_j$ proposal
4 **if** $u_i(\delta_j) \geq u_i(\delta_i)$
5 **then** Accept $\delta_j$
6 **else** $\delta_i \leftarrow \delta_i'$ such that $u_j(\delta_i') \geq \epsilon + u_j(\delta_i)$ and $u_i(\delta_i') \geq u_i(\delta^-)$
7 **goto** 2

**Figure 6.6:** The monotonic concession protocol.

Upon solving these two equations for $\delta_i^*$ and $\delta_j^*$ we get the equilibrium values from Theorem 6.1. More generally, it has been shown that a unique subgame perfect equilibrium exists even when the utility functions are not linear but are simply monotonic in $\delta$.

The theorem tells us that the best strategy for these agents is propose a bid on the first time step which will be accepted by the other agent. This action makes sense because we know that utilities only decrease with time so the best possible deal will be had on the first time step. The value of the proposed deal, on the other hand, is very interesting. Notice that the deal $i$ proposes depends only on the agents' discount factor, not on their utility values. The important thing is how fast each agent loses utility over time. For example, notice that if $\lambda_j = 0$ then $i$ will propose $\delta_i^* = 1$. That is, $i$ will propose to keep all the utility to himself. Agent $j$ will accept this proposal since he knows that given that his $\lambda_j = 0$ if he waits for the next time step then he will receive utility of 0 in every possible deal.

Similar techniques have been used to show the existence of equilibrium in other types of bargaining games (Kraus, 2001). These games are all alternating offer games but assume different utility discounts such as a fixed loss utility function that is reduced by a constant amount each time and an interest rate utility which models the opportunity loss, among others. Depending on the type of utility discount and the type of game, some of these games can be proven to have a unique perfect equilibrium strategy that involves only one actions, others have multiple equilibria and thus would require some other coordination method, and yet others can be shown to be NP-complete in the time it takes to find the unique equilibrium. In general, however, the technique used to solve these bargaining games is the same.

1. Formalize bargaining as a Rubinstein's alternating offers games.
2. Generate extended-form game.
3. Determine equilibrium strategy for this game.

In practice, step 3 is often computationally intractable, at least for the general case of the problem.

## 6.2 Monotonic Concession Protocol

The simplest negotiation protocol following the Rubinstein's model of alternating offers is the **monotonic concession protocol**. In this protocol the agents agree to always make a counter offer that is slightly better for the other agent than its previous offer. Specifically, in monotonic concession the agents follow the algorithm shown in figure 6.6.

Each agent starts by proposing the deal that is best for itself. The agent then receives a similar proposal from the other agent. If the utility the agent receives from the other's proposal is bigger than the utility it gets from its own proposal then it accepts it and negotiation ends. If no agreement was reached then the agent must propose a deal that is at least an increase of $\epsilon$ in the other agent's utility. If neither agent makes a new proposal then there are no more messages sent and





MONOTONIC-CONCESSION

1 $\delta_i \leftarrow$ arg max$_\delta$ $u_i$(δ) 2 Proponha $\delta_i$3 Receba $\delta_j$ a proposta 4 se $u_i(\delta_j) \geq u_i$ ($\delta_i$) 5 então aceite $\delta_j$6 else $\delta_i \leftarrow \delta_i'$ tal que $u_j(\delta_i') \geq \epsilon + u_j(\delta_i)$ e $u_i(\delta_i') \geq u_i(\delta^-)$ 7 vá para 2

Figura 6.6: O protocolo de concessão monotônico.

Ao resolver estas duas equações para $\delta_i^*$ e $\delta_j^*$ obtemos os valores de equilíbrio do Teorema 6.1. De forma mais geral, foi demonstrado que existe um equilíbrio perfeito de subjogo único mesmo quando as funções de utilidade não são lineares, mas são simplesmente monotônicas em $\delta$.

O teorema nos diz que a melhor estratégia para esses agentes é propor uma oferta no primeiro intervalo de tempo que será aceita pelo outro agente. Esta ação faz sentido porque sabemos que as utilidades só diminuem com o tempo, portanto o melhor negócio possível será obtido no primeiro passo de tempo. O valor do negócio proposto, por outro lado, é muito interessante. Observe que o acordo $i$ proposto depende apenas do fator de desconto dos agentes, e não de seus valores de utilidade. O importante é a rapidez com que cada agente perde utilidade ao longo do tempo. Por exemplo, observe que se $\lambda_j = 0$ então $i$ irá propor$\delta_i^* = 1$. Ou seja, $i$ proporá manter toda a utilidade para si. O agente $j$ aceitará esta proposta porque sabe que, dado que $\lambda_j = 0$ se esperar pelo próximo passo de tempo, receberá utilidade de 0 em todas as negociações possíveis.

Técnicas semelhantes têm sido utilizadas para mostrar a existência de equilíbrio em outros tipos de jogos de barganha (Kraus, 2001). Esses jogos são todos jogos de oferta alternada, mas assumem diferentes descontos de utilidade, como uma função de utilidade de perda fixa que é reduzida a um valor constante a cada vez e uma utilidade de taxa de juros que modela a perda de oportunidade, entre outros. Dependendo do ti po de desconto de utilidade e do tipo de jogo, pode-se provar que alguns desses jogos têm uma estratégia única de equilíbrio perfeito que envolve apenas uma ação, outros têm equilíbrios múltiplos e, portanto, exigiriam algum outro método de coordenação, e ainda outros podem ser mostrados como NP-completos no tempo que leva para encontrar o equilíbrio único. Em geral, porém, a técnica utilizada para resolver estes jogos de barganha é a mesma.

1. Formalize a negociação como um jogo de ofertas alternadas de Rubinstein.
2. Gere um jogo de formato estendido.
3. Determine a estratégia de equilíbrio para este jogo.

Na prática, o passo 3 é frequentemente intratável computacionalmente, pelo menos para o caso geral do problema.

### 6.2 Protocolo de Concessão Monotônica

O protocolo de negociação mais simples seguindo o modelo de ofertas alternadas de Rubinstein é o protocolo de concessão monotônico. Neste protocolo os agentes concordam fazer sempre uma contraproposta que seja ligeiramente melhor para o outro agente do que a oferta anterior. Especificamente, na concessão monotônica os agentes seguem o algoritmo mostrado na figura 6.6.

Cada agente começa propondo o negócio que é melhor para si. O agente então recebe uma proposta semelhante do outro agente. Se a utilidade que o agente recebe da proposta do outro for maior do que a utilidade que ele obtém da sua própria proposta, ele a aceita e a negociação termina. Se nenhum acordo for alcançado, o agente deverá propor um acordo que represente pelo menos um aumento de $\epsilon$ na utilidade do outro agente. Se nenhum dos agentes fizer uma nova proposta, não haverá mais mensagens enviadas e

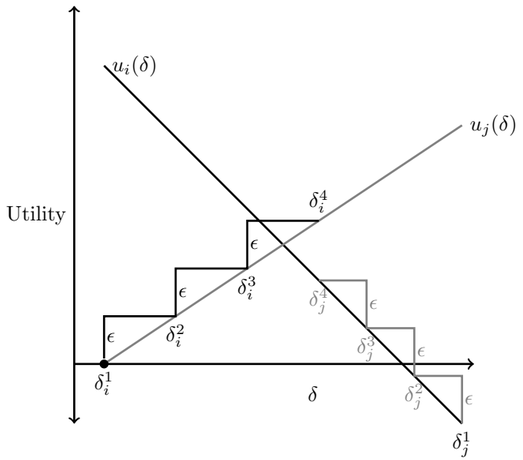



**Figure 6.7:** Monotonic concession protocol visualization. Both agents have linear utility functions over the set of deals. The superscripts indicate the time so $\delta_i^1$ is $i$'s proposal at time 1. At time 4 we have that $\delta_i^4 = \delta_j^4$ so $u_i(\delta_j^4) = u_i(\delta_i^4)$ and the agents agree on this deal.

negotiations fail, so they implicitly agree on the no-deal deal $\delta^-$. The monotonic concession protocol makes it easy for the agents to verify that the other agent is also obeying the protocol. Namely, if $i$ ever receives a proposal whose utility is less than a previous proposal then it knows that $j$ is not following the protocol.

The protocol can be visualized as in figure 6.7. Here we see a simple example with linear utility functions where the agents reach an agreement ($\delta^4$) after four time steps. Notice that the agreement reached is *not* the point at which the lines intersect. The monotonic concession protocol is not guaranteed to arrive at any particular axiomatic solution concept. All that it guarantees is that it will stop.

A common workaround is to assume its a zero-sum game and use the agent's own utility function.

Monotonic concession has several drawbacks. It can be very slow to converge. Convergence time is dictated by the number of possible deals, which is usually very large, and the value of $\epsilon$. It is also impossible to implement this algorithm if the agents do not know the other agents' utility function. In practice, it is rare for an agent to know its opponent's utility function. Finally, the monotonic concession protocol has a tricky last step. Namely, the two agents could make simultaneous offers where each one ends up preferring the other agent's offer to the one it just sent. This is a problem as both of them now want to accept different offers. This can be solved by forcing the agents to take turns. But, if we did that then neither will want to go first as the agent that goes first will end up with a slightly worse deal.

### 6.2.1 The Zeuthen Strategy

The monotonic concession protocol we presented implements a simplistic strategy for the agent: always concede at least $\epsilon$ to the other agent. However, if an agent knows that the other one will always concede then it might choose to not concede at all. Smarter agents will examine their opponent's behavior and concede in proportion to how much they are conceding.

Zeuthen strategy

This idea is formalized in the **Zeuthen strategy** for the monotonic concession protocol. We start by defining the willingness to risk a breakdown in negotiations to be

$$\text{risk}_i = \frac{u_i(\delta_i) - u_i(\delta_j)}{u_i(\delta_i)}. \tag{6.7}$$

The agent can then calculate the risks for both agents. The Zeuthen strategy tells us that the agent with the smallest risk should concede just enough so that it does not have to concede again in the next time step. That is, the agent that has the least



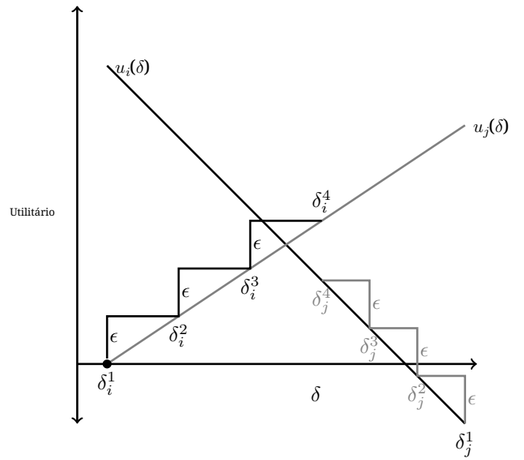



Figura 6.7: Visualização do protocolo de concessão monotônica. Ambos os agentes possuem funções de utilidade li near sobre o conjunto de negócios. Os sobrescritos indicam o tempo, então $\delta_i^1$ é a proposta de $i$ no te mpo 1. No tempo 4 temos isso $\delta_i^4 = \delta_j^4$ então $u_i(\delta_j^4) = u_i(\delta_i^4)$ e os agentes concordam com este acordo.

as negociações falham, então eles concordam implicitamente com o acordo sem acordo $\delta^-$. O protocolo de concessão monotônico torna mais fácil para os agentes verificarem se o outro agente também está obedecendo ao protocolo. Ou seja, se $i$ alguma vez receber uma proposta cuja utilidade é menor que u ma proposta anterior, então ele saberá que $j$ não está seguindo o protocolo.

O protocolo pode ser visualizado conforme a figura 6.7. Aqui vemos um exemplo si mples com funções de utilidade linear onde os agentes chegam a um acordo ($\delta^4$) após quatro intervalos de tempo. Observe que o acordo alcançado não é o ponto em que as linhas se cruzam. Não é garantido que o protocolo de concessão monotônico chegue a qualquer conceito de solução axiomática particular. Tudo o que garante é que irá parar.

Uma solução alternativa comum é assumir que se trata de um jogo de soma zero e usar a função de utilidade do próprio agente.

A concessão monotônica tem várias desvantagens. A convergência pode ser muito lenta. O tempo de convergência é ditado pelo número de negócios possíveis, que geralmente é muito grande, e pelo valor de $\epsilon$. Também é impossível implementar este algoritmo se os agentes não conhecerem a função de utilidade dos outros agentes. Na prática, é raro que um agente conheça a função de utilidade do seu oponente. Finalmente, o protocolo de concessão monotônico tem uma última etapa complicada. Ou seja, os dois agentes poderiam fazer ofertas simultâneas onde cada um acaba por preferir a oferta do outro agente à que acabou de enviar. Isto é um problema porque ambos agora querem aceitar ofertas diferentes. Isto pode ser resolvido forçando os agentes a se revezarem. Mas, se fizermos isso, nenhum dos dois vai querer ir primeiro, pois o agente que for primeiro acabará com um negócio um pouco pior.

### 6.2.1 A Estratégia Zeuthen

O protocolo de concessão monotônico que apresentamos implementa uma estratégia simplista para o agente: sempre conceder pelo menos $\epsilon$ ao outro agente. Contudo, se um agente sabe que o outro sempre concederá, então poderá optar por não conceder nada. Os agentes mais inteligentes examinarão o comportamento do seu oponente e concederão proporcionalmente ao quanto estão concedendo.

Zeuthen strategy

Esta ideia está formalizada na estratégia de Zeuthen para a concessão monotónica protocolo. Começamos por definir a vontade de arriscar um colapso nas negociações a serem

$$\text{risk}_i = \frac{u_i(\delta_i) - u_i(\delta_j)}{u_i(\delta_i)}. \tag{6.7}$$

O agente pode então calcular os riscos para ambos os agentes. A estratégia de Zeuthen diz-nos que o agente com o menor risco deve conceder apenas o suficiente para não ter de sofrer novamente no próximo intervalo de tempo. Ou seja, o agente que tiver menos

ZEUTHEN-MONOTONIC-CONCESSION

1 $\delta_i \leftarrow \arg\max_\delta u_i(\delta)$
2 Propose $\delta_i$
3 Receive $\delta_j$ proposal
4 **if** $u_i(\delta_j) \geq u_i(\delta_i)$
5 **then** Accept $\delta_j$
6 $\text{risk}_i \leftarrow \frac{u_i(\delta_i)-u_i(\delta_j)}{u_i(\delta_i)}$
7 $\text{risk}_j \leftarrow \frac{u_j(\delta_j)-u_j(\delta_i)}{u_j(\delta_j)}$
8 **if** $\text{risk}_i < \text{risk}_j$
9 **then** $\delta_i \leftarrow \delta_i'$ such that $\text{risk}_i(\delta_i') > \text{risk}_j(\delta_j')$
10 **goto** 2
11 **goto** 3

**Figure 6.8:** The monotonic concession protocol using the Zeuthen strategy.

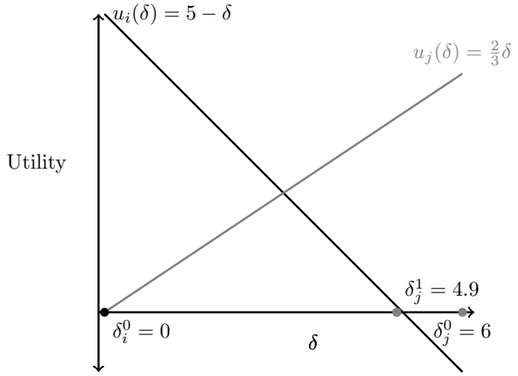



**Figure 6.9:** Visualization of the Zeuthen strategy at work. The initial proposals are $\delta_i^0 = 0$ and $\delta_j^0 = 6$.

to lose by conceding should concede. More formally, we define the new protocol as shown in figure 6.8.

Figure 6.9 shows a graphical representation of the first step in a Zeuthen negotiation. After the initial proposal the agents calculate their risks to be

$$\text{risk}_i^0 = \frac{5-(-1)}{5} = \frac{6}{5}$$

and

$$\text{risk}_j^0 = \frac{4-0}{4} = 1.$$

Since $j$ has a lower risk it must concede. The new deal must be such that $j$ will not be forced to concede again. That is, it must insure that

$$\text{risk}_i = \frac{5-(5-\delta_j)}{5} < \frac{\frac{2}{3}\delta_j - 0}{\frac{2}{3}\delta_j} = \text{risk}_j$$

which simplifies to $\delta_j < 5$. As such, $j$ can pick any deal $\delta$ less than 5. In the figure the agent chooses $\delta_j^1 = 4.9$.

The Zeuthen strategy is guaranteed to terminate and the agreement it reaches upon termination is guaranteed to be individually rational and Pareto optimal. Furthermore, it has been shown that two agents using the Zeuthen strategy will converge to a Nash bargaining solution.

**Theorem 6.2** (Zeuthen converges to Nash solution)**.** *If both agents use the Zeuthen strategy they will converge to a Nash bargaining solution deal, that is, a deal that maximizes the product of the utilities (Harsanyi, 1965).*



ZEUTHEN-MONOTONIC-CONCESSION

1 $\delta_i \leftarrow$ arg max$_\delta$ $u_i(\delta)$ 2 Proponha $\delta_i$3 Receba $\delta_j$ a proposta 4 se $u_i(\delta_j) \geq u_i(\delta_i)$ 5 então aceite $\delta_j$6 risco $_i \leftarrow$ $^{u_i}(\delta_i)-u_i(\delta_j)$ $_{u_i}(\delta_i)$ 7 risco$_j \leftarrow$ $^{u_j}(\delta_j)-u_j(\delta_i)$ $_{u_j}(\delta_j)$ 8 se risco$_i$ <risco$_j$9 então $\delta_i \leftarrow \delta_i'$ tal risco$_i(\delta_i') >$ risco$_j(\delta_j')$ 10 ir para 2 11 ir para 3

Figura 6.8: O protocolo de concessão monotônico utilizando a estratégia de Zeuthen.

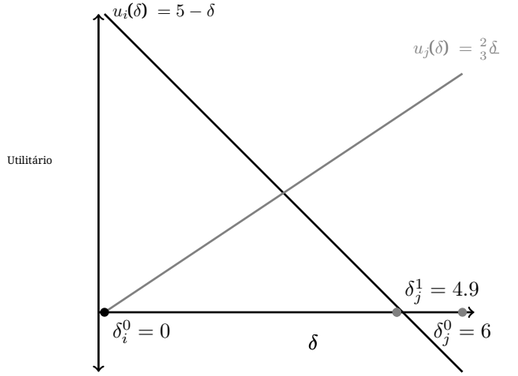



Figura 6.9: Visualização da estratégia Zeuthen em ação. As propostas iniciais são $\delta_i^0 =$ 0 e $\delta_j^0 = 6$.

perder por sofrer deve conceder. Mais formalmente, definimos o novo protocolo conforme mostrado na figura 6.8.

A Figura 6.9 mostra uma representação gráfica do primeiro passo de uma negociação de Zeuthen. Após a proposta inicial os agentes calculam seus riscos a serem

$$\text{risk}_i^0 = \frac{5-(-1)}{5} = \frac{6}{5}$$

and

$$\text{risk}_j^0 = \frac{4-0}{4} = 1.$$

Dado que $j$ tem um risco menor, deve conceder. O novo acordo deve ser tal que $j$ não seja forçado a ceder novamente. Ou seja, deve garantir que

$$\text{risk}_i = \frac{5-(5-\delta_j)}{5} < \frac{\frac{2}{3}\delta_j - 0}{\frac{2}{3}\delta_j} = \text{risk}_j$$

que simplifica para $\delta_j < 5$. Assim, $j$ pode escolher qualquer negócio $\delta$ menor que 5. Na figura, o agente escolhe $\delta_j^1 = 4.9$.

A estratégia de Zeuthen tem garantia de término e o acordo alcançado após a rescisão é garantido como individualmente racional e ótimo de Pareto. Além disso, foi demonstrado que dois agentes que utilizam a estratégia de Zeuthen convergirão para uma solução de negociação de Nash.

*Teorema 6.2 (Zeuthen converge para solução de Nash). Se ambos os agentes utilizarem a estratégia de Zeuthen, convergirão para um acordo de solução de barganha de Nash, ou seja, um acordo que maximiza o produto das utilidades (Harsanyi, 1965).*

ONE-STEP-NEGOTIATION
1 $E \leftarrow \{\delta \mid \forall_{\delta'} u_i(\delta)u_j(\delta) \geq u_i(\delta')u_j(\delta')\}$
2 $\delta_i \leftarrow \arg\max_{\delta \in E} u_i(\delta)$
3 Propose $\delta_i$
4 Receive $\delta_j$
5 **if** $u_i(\delta_j)u_j(\delta_j) < u_i(\delta_i)u_j(\delta_i)$
6 **then** Report error, $j$ is not following strategy.
7 Coordinate with $j$ to choose randomly between $\delta_i$ and $\delta_j$.

**Figure 6.10:** The one step negotiation protocol (Rosenschein and Zlotkin, 1994).

The proof of the theorem shows how the maximum of both agents' utilities monotonically increases until the protocol reaches an agreement. As such, if there is any other agreement that has higher utility than the one they agreed upon then that agreement would have been proposed in the negotiation. Therefore, there can be no deal with higher utility for any agent than the one they agreed on.

A problem with the Zeuthen strategy might arise in the last step if the agents' risks are exactly the same. Specifically, if both agents realize that their risks are the same and that the next proposal that either of them makes will be accepted by their opponent then both agents will want to wait for the other agent to concede. This is an example of the game of chicken. As such, selfish agents should play the Nash equilibrium strategy which, in this case, is a mixed strategy. Since it is a mixed strategy it means that there remains a possibility for both of them to decide to wait for the other one, thereby breaking down the negotiations when a solution was possible.

The Zeuthen strategy is also attractive because, when combined with the mixed Nash strategy we described, it is in a Nash equilibrium. That is, if one agent declares that it will be using the Zeuthen strategy then the other agent has nothing to gain by using any other negotiation strategy. As such, and agent that publicly states that it will be using the Zeuthen strategy can expect that every agent it negotiates with will also use the Zeuthen strategy.

### 6.2.2 One-Step Protocol

Once we have a multi-step protocol that (usually) reaches a particular solution, we immediately think about the possibility of skipping all those intermediate steps and jumping right to the final agreement. Specifically, we could define a protocol that asks both agents to send a proposal. Each agent then has two proposals: the one it made and the one it received. The agents must then accept the proposal that maximizes the product of the agents' utilities. If there is a tie then they coordinate and choose one of them at random.

Given the rules established by this protocol, an agent's best strategy is to find of the deals that maximize the product of the utilities and then, from these deals, propose the one that is best for the agent. The agent's algorithm for this **one-step negotiation** protocol is shown in figure 6.10. The strategy implemented by this algorithm is a Nash equilibrium. That is, if one agent decides to implement this algorithm then the other agent's best strategy is to also implement the same algorithm. Thus, when using perfectly rational agents it is always more efficient to use one-step negotiation rather than the more long winded monotonic concession protocol.



## 6.3 Negotiation as Distributed Search

If we take a step back and look at the bargaining problem, we realize that the problem of finding a chosen solution deal is in effect a distributed search problem. That is, via their negotiations the agents are effectively searching for a specific solution. A simple and natural way to carry out this search is by starting with some



ONE-STEP-NEGOTIATION
1 $E \leftarrow \{\delta \mid \forall_{\delta'} u_i(\delta)u_j(\delta) \geq u_i(\delta')u_j(\delta')\}$2 $\delta_i \leftarrow$ argmax$_{\delta \in E}\, u_i(\delta)$ 3 Proponha $\delta_i$4 Receba $\delta_j$5 se $u_i(\delta_j)u_j(\delta_j) < u_i(\delta_i)u_j(\delta_i)$ 6 então Reportar erro, $j$ não está seguindo a estratégia. 7 Coordene com $j$ para escolher aleatoriamente entre $\delta_i$ e $\delta_j$.

Figura 6.10: Protocolo de negociação em uma etapa (Rosenschein e Zlotkin, 1994).

A prova do teorema mostra como o máximo das utilidades de ambos os agentes aumenta monotonicamente até que o protocolo chegue a um acordo. Como tal, se houver qualquer outro acordo que tenha maior utilidade do que aquele acordado, então esse acordo terá sido proposto na negociação. Portanto, não pode haver acordo com maior utilidade para qualquer agente do que aquele com o qual eles concordaram.

Um problema com a estratégia de Zeuthen poderá surgir na última etapa se os riscos dos agentes forem exatamente os mesmos. Especificamente, se ambos os agentes perceberem que os seus riscos são os mesmos e que a próxima proposta que qualquer um deles fizer será aceite pelo seu oponente, então ambos os agentes quererão esperar que o outro agente conceda. Este é um exemplo do jogo da galinha. Como tal, os agentes egoístas deveriam utilizar a estratégia de equilíbrio de Nash que, neste caso, é uma estratégia mista. Por se tratar de uma estratégia mista, significa que permanece a possibilidade de ambos decidirem esperar pelo outro, interrompendo assim as negociações quando uma solução era possível.

A estratégia de Zeuthen também é atraente porque, quando combinada com a estratégia mista de Nash que descrevemos, resulta num equilíbrio de Nash. Ou seja, se um agente declarar que usará a estratégia de Zeuthen, então o outro agente não terá nada a ganhar usando qualquer outra estratégia de negociação. Como tal, um agente que declara publicamente que utilizará a estratégia de Zeuthen pode esperar que todos os agentes com quem negocia também utilizem a estratégia de Zeuthen.

### 6.2.2 Protocolo de Uma Etapa

Uma vez que temos um protocolo de múltiplas etapas que (geralmente) atinge uma solução específica, pensamos imediatamente na possibilidade de pular todas essas etapas intermediárias e ir direto para o acordo final. Especificamente, poderíamos definir um protocolo que solicitasse a ambos os agentes o envio de uma proposta. Cada agente tem então duas propostas: a que fez e a que recebeu. Os agentes devem então aceitar a proposta que maximiza o produto das utilidades dos agentes. Se houver empate, eles se coordenam e escolhem um deles aleatoriamente.

Dadas as regras estabelecidas por este protocolo, a melhor estratégia de um agente é encontrar os negócios que maximizam o produto das concessionárias e então, a partir desses negócios, propor aquele que é melhor para o agente. O algoritmo do agente para este protocolo de negociação de uma etapa é mostrado na figura 6.10. A estratégia implementada por



este algoritmo é um equilíbrio de Nash. Ou seja, se um agente decidir implementar este algoritmo então a melhor estratégia do outro agente é implementar também o mesmo algoritmo. Assim, ao usar agentes perfeitamente racionais, é sempre mais eficiente usar a negociação em uma etapa, em vez do protocolo de concessão monotônico, mais prolixo.

## 6.3 Negociação como Pesquisa Distribuída

Se dermos um passo atrás e olharmos para o problema da negociação, perceberemos que o problema de encontrar uma solução escolhida é, na verdade, um problema de busca distribuída. Ou seja, através das suas negociações os agentes procuram efectivamente uma solução específica. Uma forma simples e natural de realizar esta busca é começar com alguns

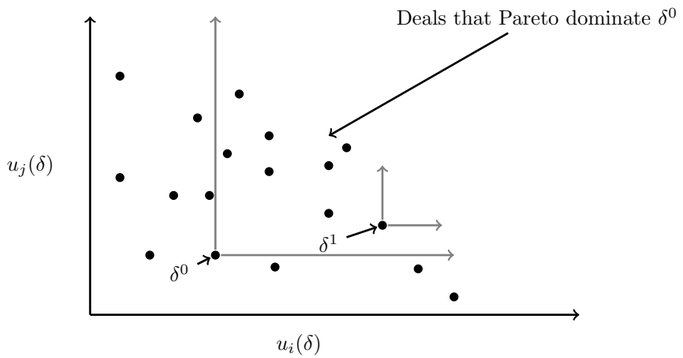



**Figure 6.11:** Hill-climbing on the Pareto landscape. The quadrant to the top and right of $\delta^0$ contains all the deals that dominate $\delta^0$, similarly for $\delta^1$. There are no deals that dominate $\delta^1$ and yet it is not the social welfare or Nash bargaining solution.

initial deal and at each time step moving to another deal. The process is repeated until there is no other deal which the agents can agree on.

For example, figure 6.11 shows an initial deal $\delta^0$ along with all the other possible deals for a negotiation problem. Any deal that is above and to the right of $\delta^0$, as delineated by the gray lines, dominates $\delta^0$. This means that any deal in this quadrant is preferred by all the agents. Thus, we can expect selfish agents to cheerfully accept any one of those deals. A simple hill-climbing search process will continue moving in this up-and-right direction.

However, the search might get stuck in a local optima. For example, suppose that we go directly from $\delta^0$ to $\delta^1$ as shown in the figure. This new deal is not dominated by any other deal, thus our simple hill-climbing algorithm is now stuck at that deal. Notice also that this deal is not the social welfare maximizing deal nor is it the Nash bargaining deal. Thus, simple hill-climbing among selfish agents can lead us to sub-optimal solutions.

In real applications this search problem is further compounded by the fact that it is often impossible to move directly from any one deal to any other deal because the number of possible deals is simply too large to consider or there are other external limitations imposed on the agents. That is, most applications overlay an undirected graph to the bargaining problem. In this graph the nodes represent the possible deals and an edge between two nodes denotes the fact that it is possible to move between those two deals. Given such a graph, it is possible that the agents would find themselves in one deal that is pareto dominated by a lot of other deals but which are unreachable in one step because there is no edge between those deals and the current deal. Thus, we have even more local optima.

Researchers have tried to circumvent this problem of local optima in bargaining via a number of different methods, as we shall see in the next few sections.

## 6.4 Ad-hoc Negotiation Strategies

Deployed multiagent negotiation systems, such as **ADEPT** (Faratin et al., 1998; Binmore and Vulkan, 1999), have implemented agents with ad-hoc negotiation strategies. In ADEPT, a handful of selected negotiation **tactics** were tested against each other to determine how they fared against each other. The basic negotiation model used was an alternating offers model with time discounts. The tactics the agents could use included: a *linear* tactic that concedes a fixed amount each time, a *conceder* tactic that concedes large amounts of utility on earlier time steps, and an *impatient* tactic which conceded very little at first and requested a lot. We can think of each one of these tactics as stylized versions of the negotiation strategies people might use in a negotiation.

This ad-hoc approach is often used by multiagent developers who need to build a negotiating multiagent system which achieves a good-enough solution and where

ADEPT

TACTICS



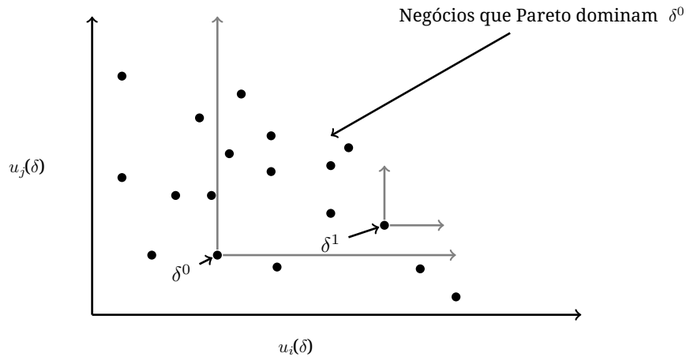



Figura 6.11: Subida de colina na paisagem de Pareto. O quadrante superior e direito de $\delta^0$ contém todas as negociações que dominam $\delta^0$, da mesma forma para $\delta^1$. Há nodeals que dominam $\delta^1$ e ainda assim não é a solução de bem-estar social ou de negociação de Nash.

negócio inicial e a cada passo passando para outro negócio. O processo é repetido até que não haja outro acordo com o qual os agentes possam concordar.

Por exemplo, a figura 6.11 mostra um acordo inicial $\delta^0$ juntamente com todos os outros acordos possíveis para um problema de negociação. Qualquer negócio que esteja acima e à direita de $\delta^0$, conforme delineado pelas linhas cinzas, domina $\delta^0$. Isso significa que qualquer negócio neste quadrante é preferido por todos os agentes. Assim, podemos esperar que agentes egoístas aceitem alegremente qualquer um desses acordos. Um simples processo de busca ascendente continuará se movendo nessa direção para cima e para a direita.

No entanto, a busca pode ficar presa em um ótimo local. Por exemplo, suponha que vamos diretamente de $\delta^0$ para $\delta^1$ conforme mostrado na figura. Este novo acordo não é dominado por nenhum outro acordo, portanto o nosso simples algoritmo de escalada está agora preso nesse acordo. Observe também que este acordo não é o acordo de maximização do bem-estar social nem é o acordo de negociação de Nash. Assim, a simples subida de colinas entre agentes egoístas pode levar-nos a soluções subótimas.

Em aplicações reais, esse problema de busca é ainda agravado pelo fato de que muitas vezes é impossível passar diretamente de qualquer negócio para qualquer outro porque o número de negócios possíveis é simplesmente grande demais para ser considerado ou porque há outras limitações externas impostas aos agentes. Ou seja, a maioria das aplicações sobrepõe um gráfico não direcionado ao problema de negociação. Neste gráfico os nós representam os negócios possíveis e uma aresta entre dois nós denota o fato de que é possível mover-se entre esses dois negócios. Dado esse gráfico, é possível que os agentes se encontrem em um negócio que é dominado por muitos outros negócios, mas que são inacessíveis em uma única etapa porque não há vantagem entre esses negócios e o negócio atual. Assim, temos ainda mais ótimos locais.

Os investigadores tentaram contornar este problema dos óptimos locais na negociação através de vários métodos diferentes, como veremos nas próximas secções.

### 6.4 Estratégias de Negociação Ad-hoc

Sistemas de negociação multiagentes implantados, como ADEPT (Faratin et al., 1998; Binmore e Vulkan, 1999) implementaram agentes com estratégias de negociação ad-hoc. Em ADEPT, algumas táticas de negociação selecionadas foram testadas entre si para determinar como eles se saíram um contra o outro. O modelo básico de negociação utilizado foi um modelo de ofertas alternadas com descontos por prazo. As táticas que os agentes poderiam usar incluíam: uma tática linear que concede uma quantia fixa a cada vez, uma tática de concessão que concede grandes quantidades de utilidade em intervalos de tempo anteriores e uma tática impaciente que cedeu muito pouco no início e solicitou muito. Podemos pensar em cada uma destas tácticas como versões estilizadas das estratégias de negociação que as pessoas podem utilizar numa negociação.

ADEPT

TACTICS

Esta abordagem ad-hoc é frequentemente usada por desenvolvedores multiagentes que precisam construir um sistema multiagente de negociação que alcance uma solução suficientemente boa e onde

all the agents are owned by the same party and, thus, do not need to behave truly selfishly. It allows us to use somewhat selfish agents that encapsulate some of the system's requirements and then use the negotiation protocol to aggregate these individual selfish interests into a global solution which, although not guaranteed to be the utilitarian solutions, can nonetheless be shown via experimentation to be a good-enough solutions. Nonetheless, we must always remember that these systems are not to be opened up to outside competition. Once we know the other agents' ad-hoc tactics, it is generally very easy to implement an agent that can defeat those tactics, usually at the cost of a much lower system utility. That is, one renegade agent can easily bring down a system of ad hoc negotiating agents, unless the system has been shown to be at an evolutionary stable equilibrium.

## 6.5 The Task Allocation Problem

TASK ALLOCATION PROBLEM

A common problem in multiagent systems is deciding how to re-allocate a set of tasks among a set of agents. This is known as the **task allocation problem**. In this problem there is a set of tasks $T$, a set of agents, and a cost function $c_i : s \rightarrow \Re$ which tells us the cost that agent $i$ incurs in carrying out tasks $s \subseteq T$. In some simplified versions of the problem we assume that all agents share the same cost function. Each agent starts out with a set of tasks such that all tasks are distributed amongst all agents. We can think of this initial allocation as $\delta^-$ since, if negotiations break down then each agent is responsible for the tasks it was originally assigned. Similarly, every allocation of tasks to agents is a possible deal $\delta$ where $s_i(\delta)$ is the set of tasks allocated to $i$ under deal $\delta$. The problem we then face is how to design a negotiation protocol such that the agents can negotiate task re-allocations and arrive at a final re-allocation of the tasks that is one of the axiomatic solution concepts, such as the utilitarian deal.

POSTMAN PROBLEM

An example of this type of problem is the **postman problem** in which a set of postmen are placed around a city each with a bag of letters that must be delivered. A postman prefers to have all his letters with delivery addresses that are very close to each other so as to minimize his delivery costs. The postmen can negotiate with each other in order to make this happen. That is, a postman can trade some of its letters with another postman. In this example the tasks are the letters to be delivered and the cost function is the distance traveled to deliver a set of letters. Different cost functions arise when the postmen have different final destinations, for example, if they are required to go to their respective homes after delivering all the letters, or if they prefer certain areas of town, etc.

Once we are given task allocation problem we must then decide how the agents will exchange tasks. The simplest method is for agents to pair up and exchange a single task. This means that not all deals are directly accessible from other deals since, for example, we can't go from the deal where $j$ does 2 tasks and $i$ does nothing to the deal where $i$ does 2 tasks and $j$ does nothing in one step. We can represent this constraint graphically by drawing an edge between every pair of deals that are reachable from one another by the exchange of a single task. An example of such a graph is shown in figure 6.12. The table in this figure shows the tasks assignments that each deal represents and the costs that each one of the agents incurs for carrying out various subsets of tasks. In order to have positive utility numbers, we let the utility to the agent be 8 minus the cost of carrying out the tasks. Utility is often simply the negative of the cost so these two are used interchangeably. The graph shows all the possible deals as points and the edges connect deals that can be reached from one another by moving a single task between agents.

If the agents are only willing to accept deals that are better for them than their current deal, that is, the accept only Pareto-dominant deals, then every deal in figure 6.12 forms a local maximum. For example, if the agents are in $\delta^1$ then, while $j$ would prefer any one of $\delta^2$, $\delta^3$, or $\delta^4$, agent $i$ would not accept any one of these because $u_i$ is lower for all of them than for $\delta^1$. Thus, if the agents find themselves in $\delta^1$ and they are not willing to lower their utility they will remain there. That is,



todos os agentes são propriedade da mesma parte e, portanto, não precisam se comportar de forma verdadeiramente egoísta. Permite-nos utilizar agentes algo egoístas que encapsulam alguns dos requisitos do sistema e depois utilizar o protocolo de negociação para agregar estes interesses egoístas individuais numa solução global que, embora não seja garantida a solução utilitária, pode, no entanto, ser demonstrada através da experimentação como sendo uma solução suficientemente boa. No entanto, devemos sempre lembrar que estes sistemas não devem ser abertos à concorrência externa. Uma vez que conhecemos as táticas ad-hoc dos outros agentes, geralmente é muito fácil implementar um agente que possa derrotar essas táticas, geralmente ao custo de uma utilidade de sistema muito menor. Isto é, um agente renegado pode facilmente derrubar um sistema de agentes negociadores ad hoc, a menos que se tenha demonstrado que o sistema está num equilíbrio evolutivamente estável.

## 6.5 O Problema TaskAllocation

TASK ALLOCATION PROBLEM

Um problema comum em sistemas multiagentes é decidir como realocar um conjunto de tarefas entre um conjunto de agentes. Isso é conhecido como problema de alocação de tarefas. Em

Neste problema existe um conjunto de tarefas $T$, um conjunto de agentes e uma função de custo $c_i : s \rightarrow \Re$que nos informa o custo que o agente $i$ incorre na execução das tarefas $s \subseteq T$. Em algumas versões simplificadas do problema assumimos que todos os agentes partilham a mesma função de custo. Cada agente começa com um conjunto de tarefas de forma que todas as tarefas sejam distribuídas entre todos os agentes. Podemos pensar nesta alocação inicial como $\delta^-$, já que, se as negociações fracassarem, cada agente será responsável pelas tarefas que lhe foram originalmente atribuídas. Da mesma forma, toda alocação de tarefas aos agentes é um acordo possível $\delta$ onde $s_i(\delta)$ é o conjunto de tarefas alocadas para $i$ no acordo $\delta$. O problema que enfrentamos então é como projetar um protocolo de negociação tal que os agentes possam negociar realocações de tarefas e chegar a uma realocação final das tarefas que é um dos conceitos de solução axiomática, como o acordo utilitário.

POSTMAN PROBLEM

Um exemplo deste tipo de problema é o problema do carteiro, no qual um conjunto de carteiros são colocados em torno de uma cidade, cada um com uma sacola de cartas que deve ser entregue. O carteiro prefere que todas as suas cartas tenham endereços de entrega muito próximos entre si para minimizar os custos de entrega. Os carteiros podem negociar entre si para que isso aconteça. Ou seja, um carteiro pode trocar algumas de suas cartas com outro carteiro. Neste exemplo as tarefas são as cartas a serem entregues e a função de custo é a distância percorrida para entregar um conjunto de cartas. Diferentes funções de custo surgem quando os carteiros têm destinos finais diferentes, por exemplo, se forem obrigados a deslocar-se às suas respectivas casas depois de entregarem todas as cartas, ou se preferirem determinadas zonas da cidade, etc.

Assim que tivermos o problema de alocação de tarefas, devemos então decidir como os agentes irão trocar tarefas. O método mais simples é os agentes formarem pares e trocarem uma única tarefa. Isso significa que nem todos os negócios são diretamente acessíveis a partir de outros negócios, pois, por exemplo, não podemos ir do negócio em que $j$ faz 2 tarefas e $i$ não faz nada para o negócio em que $i$ faz 2 tarefas e $j$ não faz nada em uma única etapa. Podemos representar essa restrição graficamente desenhando uma aresta entre cada par de negócios que podem ser alcançados entre si pela troca de uma única tarefa. Um exemplo de tal gráfico é mostrado na figura 6.12. A tabela nesta figura mostra as atribuições de tarefas que cada negócio representa e os custos que cada um dos agentes incorre para realizar vários subconjuntos de tarefas. Para termos números de utilidade positivos, deixamos a utilidade para o agente ser 8 menos o custo de execução das tarefas. A utilidade muitas vezes é simplesmente o negativo do custo, portanto esses dois são usados de forma intercambiável. O gráfico mostra todos os negócios possíveis como pontos e as arestas conectam negócios que podem ser alcançados entre si movendo uma única tarefa entre agentes.

Se os agentes estiverem dispostos a aceitar apenas negócios que sejam melhores para eles do que o seu acordo atual, ou seja, aceitar apenas negócios com dominância de Pareto, então cada negócio na figura 6.12 forma um máximo local. Por exemplo, se os agentes estiverem em $\delta^1$ então, enquanto$j$ preferiria qualquer um de $\delta^2$, $\delta^3$ ou $\delta^4$, o agente $i$ não aceitaria nenhum deles porque $u_i$ é menor para todos eles do que para $\delta^1$. Assim, se os agentes se encontrarem $\delta^1$ e não estiverem dispostos a diminuir a sua utilidade, permanecerão lá. Aquilo é,

| $\delta$ | $s_i(\delta)$ | $s_j(\delta)$ | $c_i(\delta)$ | $c_j(\delta)$ | $u_i(\delta) = 8 - c_i(\delta)$ | $u_j(\delta) = 8 - c_j(\delta)$ |
|---|---|---|---|---|---|---|
| $\delta^1$ | $\emptyset$ | $\{t_1, t_2, t_3\}$ | 0 | 8 | 8 | 0 |
| $\delta^2$ | $\{t_1\}$ | $\{t_2, t_3\}$ | 1 | 4 | 7 | 4 |
| $\delta^3$ | $\{t_2\}$ | $\{t_1, t_3\}$ | 2 | 5 | 6 | 3 |
| $\delta^4$ | $\{t_3\}$ | $\{t_2, t_3\}$ | 4 | 7 | 4 | 1 |
| $\delta^5$ | $\{t_2, t_3\}$ | $\{t_1\}$ | 6 | 4 | 2 | 4 |
| $\delta^6$ | $\{t_1, t_3\}$ | $\{t_2\}$ | 5 | 3 | 3 | 5 |
| $\delta^7$ | $\{t_1, t_2\}$ | $\{t_3\}$ | 3 | 1 | 5 | 7 |
| $\delta^8$ | $\{t_1, t_2, t_3\}$ | $\emptyset$ | 7 | 0 | 1 | 8 |

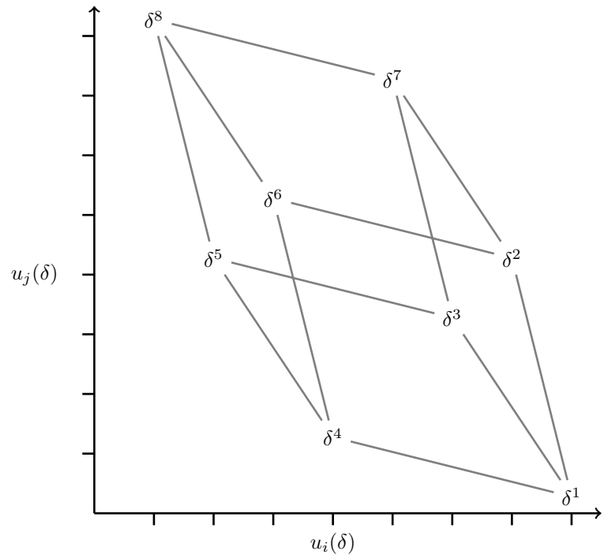



**Figure 6.12:** An example task allocation problem is shown in the table and its graphical representation as a bargaining problem is shown in the graph. The edges on the graph connect deals that can be reached by moving a single task between the agents. $s_i(\delta)$ is the set of tasks $i$ has under deal $\delta$.

$\delta^1$ is a local maximum, as are all the other deals in this example.

### 6.5.1 Payments

One possible way to allow the agents to find deals that are closer to the utilitarian deal is by allowing them to use **monetary payments**. For example, agent $i$ might convince $j$ to take a deal that gives $j$ less utility if $i$ can sweeten the deal by giving $j$ some money to take that deal. This basic idea was implemented in the **contract net protocol** (Smith, 1981; Davis and Smith, 1983). In contract net each agent takes the roles of either a contractor or contractee. The contractor has a task that it wants someone else to perform. The contractor is also willing to pay to get that task done. In the protocol, the contractor first announces that it has a task available by broadcasting a call for bids. All agents receive this call for bids and, if they want, reply to the contractor telling him how much they charge for performing the task. The contractor then chooses one of these bids, assigns the task to the appropriate agent and pays him the requested amount.

MONETARY PAYMENTS

CONTRACT NET PROTOCOL

For example, given the current task allocation $\delta$ agent $i$ might find that one of his current tasks $t \in s_i(\delta)$ costs him a lot. That is, $c_i(s_i(\delta)) - c_i(s_i(\delta) - t)$ is a large number. In this case $i$ will be willing to pay up to $c_i(s_i(\delta)) - c_i(s_i(\delta) - t)$ in order to have some other agent perform $t$—the agent will pay up to the utility he will gain from losing the task, any more than that does not make sense as the agent can simply do the task himself. Similarly, any other agent $j$ will be willing



| $\delta$ | $s_i(\delta)$ | $s_j(\delta)$ | $c_i(\delta)$ | $c_j(\delta)$ | $u_i(\delta) = 8 - c_i(\delta)$ | $u_j(\delta) = 8 - c_j(\delta)$ |
|---|---|---|---|---|---|---|
| $\delta^1$ | $\emptyset$ | $\{t_1, t_2, t_3\}$ | 0 | 8 | 8 | 0 |
| $\delta^2$ | $\{t_1\}$ | $\{t_2, t_3\}$ | 1 | 4 | 7 | 4 |
| $\delta^3$ | $\{t_2\}$ | $\{t_1, t_3\}$ | 2 | 5 | 6 | 3 |
| $\delta^4$ | $\{t_3\}$ | $\{t_2, t_3\}$ | 4 | 7 | 4 | 1 |
| $\delta^5$ | $\{t_2, t_3\}$ | $\{t_1\}$ | 6 | 4 | 2 | 4 |
| $\delta^6$ | $\{t_1, t_3\}$ | $\{t_2\}$ | 5 | 3 | 3 | 5 |
| $\delta^7$ | $\{t_1, t_2\}$ | $\{t_3\}$ | 3 | 1 | 5 | 7 |
| $\delta^8$ | $\{t_1, t_2, t_3\}$ | $\emptyset$ | 7 | 0 | 1 | 8 |

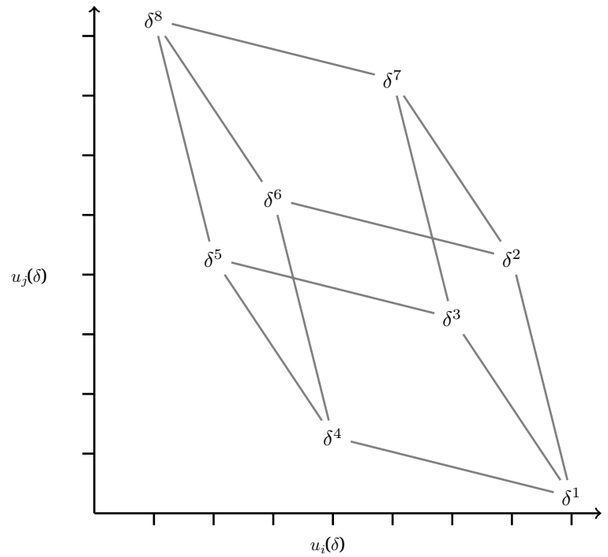



Figura 6.12: Um exemplo de problema de alocação de tarefas é mostrado na tabela e sua representação gráfica como um problema de negociação é mostrada no gráfico. As arestas do gráfico conectam negócios que podem ser alcançados movendo uma única tarefa entre os agentes. $s_i(\delta)$ é o conjunto de tarefas que $i$ tem sob acordo $\delta$.

$\delta^1$ é um máximo local, assim como todas as outras transações neste exemplo.

### 6.5.1 Pagamentos

Uma forma possível de permitir que os agentes encontrem negócios mais próximos do negócio utilitário é permitir-lhes utilizar pagamentos monetários. Por exemplo, o agente $i$ pode

convencer $j$ a aceitar um acordo que ofereça $j$ menos utilidade se $i$ poder melhorar o acordo dando$j$ algum dinheiro para aceitá-lo. Esta ideia básica foi implementada no protocolo de rede contratual (Smith, 1981; Davis e Smith, 1983). No contrato líquido cada agente

MONETARY PAYMENTS

CONTRACT NET PROTOCOL

assume o papel de contratante ou contratado. O empreiteiro tem uma tarefa que deseja que outra pessoa execute. O empreiteiro também está disposto a pagar para realizar essa tarefa. No protocolo, o contratante primeiro anuncia que tem uma tarefa disponível por meio da transmissão de um edital. Todos os agentes recebem esse edital e, se quiserem, respondem ao contratante informando quanto cobram pela execução da tarefa. O contratante escolhe então uma dessas propostas, atribui a tarefa ao agente adequado e paga-lhe o valor solicitado.

Por exemplo, dada a alocação de tarefas atual, o $\delta$ agente $i$ pode descobrir que uma de suas tarefas atuais $t \in s_i(\delta)$ lhe custa muito caro. Ou seja, $c_i(s_i(\delta)) - c_i(s_i(\delta) - t)$ é um número grande. Neste caso, $i$ estará disposto a pagar até $c_i(s_i(\delta)) - c_i(s_i(\delta) - t)$ para que algum outro agente execute $t$ - o agente pagará até a utilidade que ganhará ao perder a tarefa, mais do que isso não faz sentido, pois o agente pode simplesmente f azer a tarefa sozinho. Da mesma forma, qualquer outro agente $j$ estará disposto

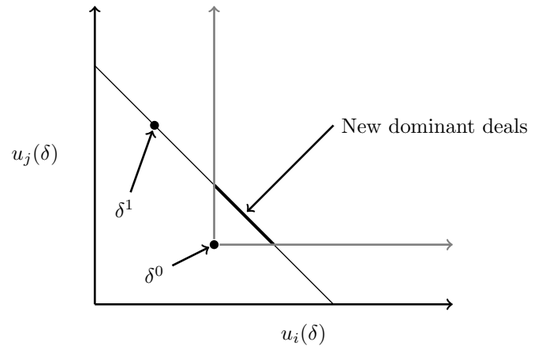



**Figure 6.13:** Deal $\delta^1$ is turned into an infinite number of deals, represented by the line that intersects $\delta^1$, with the use of payments. Some of those new deals Pareto dominate $\delta^0$, as shown by the thicker part of the line.

to perform $t$ as long as it gets paid an amount that is at least equal to any cost increase he will endure by performing the task, that is, the payment must be at least $c_j(s_j(\delta)) - c_j(s_j(\delta) + t)$. Note that in the general case some of these numbers could be negative, for example, an agent could actually have lower costs from performing an extra task. However, these sub-additive cost functions are rare in practice.

The use of monetary payments has the effect of turning one deal into an infinite number of deals: the original deal and the infinite number of payments, from $-\infty$ to $\infty$ that can be made between the agents. Figure 6.13 shows a graphical representation of this process. Here we see deal $\delta^1$ transformed into the set of deals represented by the line going thru $\delta^1$. The figure also shows us how this transformation creates new deals that dominate another existing deal $\delta^0$. Thus, if the system was in $\delta^0$ and we were doing hill-climbing then we would be stuck there as $\delta^1$ does not dominate $\delta^0$. But, if we used the contract net then agent $j$ could make a payment to $i$, along with the task transfer, which would give $i$ a higher total utility. This new task allocation plus payment is a deal that lies within the thick area of the line that intersects $\delta^1$. Thus, contract net allows us to reach a task allocation that is the utilitarian solution. Unfortunately, the addition of payments does not guarantee that we will always reach the utilitarian solution, this depends on the particular characteristics of the cost function.

One type of cost functions that have been found to be easy to search over are additive cost functions where the cost of doing a set of tasks is always equal to the sum of the costs of doing each task individually.

ADDITIVE COST FUNCTION

**Definition 6.6.** *A function $c(s)$ is an **additive cost function** if for all $s \subseteq T$ it is true that*

$$c(s) = \sum_{t \in s} c(t).$$

In these scenarios it has been shown that the contract net protocol, or any other protocol that uses payments and always moves to dominant deals, will eventually converge to the utilitarian social welfare solution.

**Theorem 6.3.** *In a task allocation problem with an additive cost function where we only allow exchange of one task at a time, any protocol that allows payments and always moves to dominant deals will eventually converge to the utilitarian solution (Endriss et al., 2006).*

For example, figure 6.14 reproduces the previous figure 6.12 but now we draw some lines to represent all the possible deals that are possible from the various deals using payments. The new deals are represented by the parallel lines which intersect each of the original deals. It is easy to confirm that all deals except for the utilitarian deal $\delta^7$ are dominated by an adjacent deal. Thus, if we continue moving to successively dominant deals we are guaranteed to end up at $\delta^7$. The proof of Theorem 6.3 follows the same line of thought; it shows that any deal (with



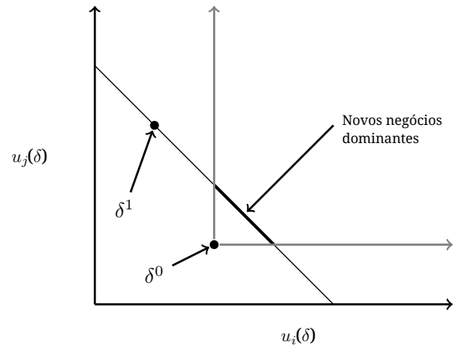



Figura 6.13: O negócio $\delta^1$ é transformado em um número infinito de negócios, representado pela linha que cruza $\delta^1$, com o uso de pagamentos. Alguns desses novos acordos Pareto dominam $\delta^0$, como mostra a parte mais gr ossa da linha.

executar $t$ desde que receba um valor pelo menos igual a qualquer aumento de custo que ele suportará ao executar a tarefa, ou seja, o pagamento deve ser de pelo menos$c_j(s_j(\delta)) - c_j(s_j(\delta) + t)$. Observe que, no caso geral, alguns desses números podem ser negativos, por exemplo, um agente pode realmente ter custos mais baixos ao realizar uma tarefa extra. No entanto, estas funções de custo subaditivo são raras na prática.

A utilização de pagamentos monetários tem o efeito de transformar um negócio num número infinito de negócios: o negócio original e o número infinito de pagamentos, de$-\infty$ a $\infty$ que podem ser feitos entre os agentes. A Figura 6.13 mostra uma representação gráfica deste processo. Aqui vemos o negócio $\delta^1$ transformado no conjunto de negócios representado pela linha que passa por $\delta^1$. A figura também nos mostra como esta transformação cria novos acordos que dominam outro acordo existente $\delta^0$. Assim, se o sistema estivesse em $\delta^0$ e estivéssemos subindo uma colina, ficaríamos presos lá, pois $\delta^1$ não domina $\delta^0$. Mas, se usássemos a rede contratual, então o agente $j$ poderia fazer um pagamento a $i$, junto com a transferência da tarefa, o que daria a $i$ uma utilidade total maior. Esta nova alocação de tarefas mais pagamento é um acordo que fica dentro da área espessa da linha que cruza $\delta^1$. Assim, a rede contratual permite-nos chegar a uma alocação de tarefas que é a solução utilitária. Infelizmente, a adição de pagamentos não garante que chegaremos sempre à solução utilitária, isto depende das características particulares da função custo.

Um tipo de função de custo que se descobriu ser fácil de pesquisar são as funções de custo aditivas, em que o custo de realizar um conjunto de tarefas é sempre igual à soma dos custos de realizar cada tarefa individualmente.

ADDITIVE COST FUNCTION

*Definição 6.6. Uma função $c(s)$ é uma função de custo aditivo se para todos $s \subseteq T$ ela é verdade que*

$$c(s) = \sum_{t \in s} c(t).$$

Nestes cenários foi demonstrado que o protocolo de rede contratual, ou qualquer outro protocolo que utilize pagamentos e sempre se mova para acordos dominantes, acabará por convergir para a solução utilitária de bem-estar social.

*Teorema 6.3. Num problema de alocação de tarefas com uma função de custo aditivo onde apenas permitimos a troca de uma tarefa de cada vez, qualquer protocolo que permita pagamentos e sempre se mova para negócios dominantes acabará por convergir para a solução utilitária (Endriss et al., 2006).*

Por exemplo, a figura 6.14 reproduz a figura 6.12 anterior, mas agora traçamos algumas linhas para representar todas as transações possíveis a partir das várias transações que utilizam pagamentos. Os novos negócios são representados pelas linhas paralelas que cr uzam cada um dos negócios originais. É fácil confirmar que todos os negócios, exceto o acordo utilitário, $\delta^7$ são dominados por um acordo adjacente. Assim, se continuarmos a avançar para negócios sucessivamente dominantes, teremos a garantia de terminar em $\delta^7$. A prova do Teorema 6.3 segue a mesma linha de pensamento; mostra que qualquer acordo (com

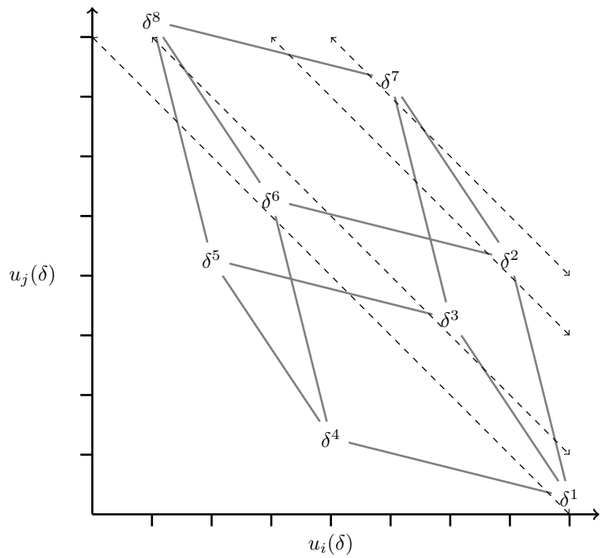



**Figure 6.14:** Monetary payments with an additive cost function. The deals expand to an infinite set of deals, represented by dashed lines intersecting the original deals. We show lines for deals $\delta^7$, $\delta^2$, $\delta^8$, and $\delta^1$ only. Note that the utilitarian deal $\delta^7$ is the only one not dominated by any other deal.

payments) that is not dominated by some other deal must be the utilitarian social welfare solution.

If we allow arbitrary cost functions then there is very little we can say about which solution a hill-climbing protocol will reach, except for one scenario. In the case where every pair of deals is connected by an edge, we can get to any deal from every other deal in one step, imagine that the graph in figure 6.12 fully connected, then using payments guarantees we will converge to the utilitarian social welfare (Sandholm, 1997; Sandholm, 1998; Andersson and Sandholm, 1998). We can easily verify this by looking again at figure 6.14. Here we can see how each deal is converted into a line of slope $-1$ and the utilitarian deal will always be the one with the line that is furthest to the top right. Thus, any protocol that successively moves via dominant deals (hill-climbing towards the top right) will always arrive at the utilitarian solution because, as we assumed, every deal is always reachable from every other deal. We can directly connect every deal to every other deal by using a very flexible contracting language, such as **OCSM contracts** which allows all possible task transfer combinations.

Unfortunately, being able to move from every deal to every other deal in one step means that the agents will need to consider a vast number of possible deals at each time step. Thus, this approach merely replaces one intractable problem with another. In general, what we want is to limit the deal accessibility—the number of edges in the graph—such that there is a manageable number of deals accessible from every deal and there are few local optima where a hill-climbing negotiation could get stuck. We currently lack any general tools for achieving this result but, for any specific problem it is likely that a good multiagent designer can engineer a viable solution.

### 6.5.2 Lying About Tasks

In some cases it might be worthwhile for an agent to hide from other agents some of the tasks it has to perform, or to make up tasks and tell others that it has perform them, or to make up these tasks and, if someone else offers to perform the phony tasks then actually create the new tasks. All these methods of cheating result in the modification of the set of existing deals. For example, if an agent creates a new





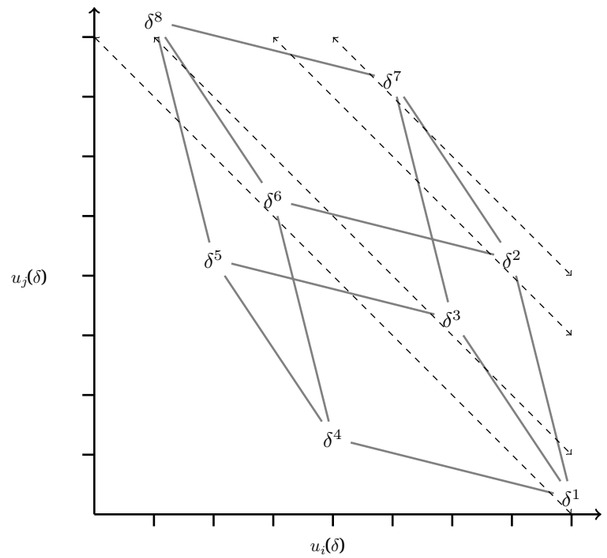



Figura 6.14: Pagamentos monetários com função de custo aditivo. Os negócios se expande m para um conjunto infinito de negócios, representados por linhas tracejadas que cruzam os negócios originais. Mostramos linhas para negócios $\delta^7$, $\delta^2$ $\delta^8$ $\delta^1$ , e somente. Observe que o acordo utilitário $\delta^7$ é o único que não é dominado por nenhum outro acordo.

pagamentos) que não seja dominado por algum outro acordo deve ser a solução utilitária de bem-estar social.

Se permitirmos funções de custo arbitrárias, então há muito pouco que podemos dizer sobre qual solução um protocolo de escalada alcançará, exceto para um cenário. No caso em que cada par de negócios está conectado por uma aresta, podemos chegar a qualquer negócio de todos os outros negócios em uma única etapa, imagine que o gráfico da figura 6.12 esteja totalmente conectado, então usando garantias de pagamentos convergiremos para o bem-estar social utilitário (Sandholm, 1997; Sandholm, 1998; Andersson e Sandholm, 1998). Podemos facilmente verificar isso olhando novamente para a figura 6.14. Aqui podemos ver como cada acordo é convertido em uma linha de inclinação $-1$ e o acordo utilitário será sempre aquele com a linha mais para o canto superior direito. Assim, qualquer protocolo que se mova sucessivamente através de acordos dominantes (subindo em direcção ao canto superior direito) chegará sempre à solução utilitária porque, como assumimos, cada acordo é sempre acessível a partir de qualquer outro acordo. Podemos conectar diretamente cada negócio a todos os outros usando uma linguagem de contratação muito flexível, como os contratos OCSM, que permitem que todos

possíveis combinações de transferência de tarefas.

Infelizmente, ser capaz de passar de qualquer negócio para outro em uma única etapa significa que os agentes precisarão considerar um grande número de negócios possíveis em cada etapa de tempo. Assim, esta abordagem apenas substitui um problema intratável por outro. Em geral, o que queremos é limitar a acessibilidade do negócio – o número de arestas no gráfico – de modo que haja um número gerenciável de negócios acessíveis a partir de cada negócio e haja poucos ótimos locais onde uma negociação ascendente possa travar. Atualmente não temos ferramentas gerais para alcançar esse resultado, mas, para qualquer problema específico, é provável que um bom projetista multiagente possa projetar uma solução viável.

#### 6.5.2 Tarefas de mentir sobre

Em alguns casos, pode valer a pena para um agente esconder de outros agentes algumas das tarefas que tem de executar, ou inventar tarefas e dizer aos outros que as executou, ou inventar essas tarefas e, se alguém se oferecer para executar as tarefas falsas, então, na verdade, criar as novas tarefas. Todos esses métodos de trapaça resultam na modificação do conjunto de acordos existentes. Por exemplo, se um agente criar um novo

| $\delta$ | $s_i(\delta)$ | $s_j(\delta)$ | $u_i(\delta)$ | $u_j(\delta)$ |
|---|---|---|---|---|
| $\delta^1$ | $\emptyset$ | $\{t_1\}$ | 1 | 3 |
| $\delta^2$ | $\{t_1\}$ | $\emptyset$ | 2 | 1 |

| $\delta$ | $s_i(\delta)$ | $s_j(\delta)$ | $u_i(\delta)$ | $u_j(\delta)$ |
|---|---|---|---|---|
| $\delta^1$ | $\emptyset$ | $\{t_1, t_2\}$ | 1 | 5 |
| $\delta^2$ | $\{t_1\}$ | $\{t_2\}$ | 2 | 3 |
| $\delta^3$ | $\{t_2\}$ | $\{t_1\}$ | 2 | 3 |
| $\delta^4$ | $\{t_1, t_2\}$ | $\emptyset$ | 8 | 1 |

**Figure 6.15:** Lying by task creation example. The top table shows the original utility values, where $\delta^1$ is the Nash bargaining solution. The bottom table shows the values after agent $i$ creates phony task $t_2$. Here, $\delta^4$ is the Nash bargaining solution.

task then we now have to consider all the possible ways in which the new bigger set of tasks can be distributed among the agents. If an agent hides a task from the agents then the system has fewer possible allocations to consider. We can then ask the question, when is it in an agent's best interest to lie?

That is the question asked in *Rules of Encounter* (Rosenschein and Zlotkin, 1994). In it, the authors assume that the agents will use a bargaining protocol which will arrive at the Nash bargaining solution, presumably by either using alternating offers with the Zeuthen strategy or the ONE-STEP-NEGOTIATION protocol from figure 6.10. Once an agent knows that the final agreement deal will be the Nash bargaining solution then all it has to do is check each possible lie to see if the solution reached when it tells that lie gives him a higher utility than he would have received by telling the truth. Fortunately for the agent, and unfortunately for us, it has been shown that such lies do generally exist and will benefit the agent.

Figure 6.15 shows an example where an agent has an incentive to create a phony task. The table on the left shows the initial utility values for a simple game with only one task. In this game the Nash bargaining solution is $\delta^1$ in which $i$ does not perform any task and receives a utility of 1. Noticing that it would get a utility of 2 if it did perform $t_1$, $i$ decides to create a phony task $t_2$ whose utility values are given on the table. In this new game the Nash bargaining solution is $\delta^4$ which gives both tasks to agent $i$. Thus, by creating this task $i$ was able to get a utility of 2 instead of the original 1 by getting $t_1$ allocated to itself. That is, since $t_2$ was assigned to $i$, the agent does not have to worry about perform this task nor does he have to worry about other agents trying to perform a task he made up. Thus, it can lie and get away with it.

### 6.5.3 Contracts

In our discussion of payments we have thus far assumed that if one agent says that he will pay another agent to perform some tasks then both of them will abide by that contract. The tasks will be performed and the money will be paid. Of course, things are not so simple for real applications. Specifically, in a dynamic environment an agent that is given a set of tasks to perform might find that right after he agrees to perform the tasks a new deal appears which will give him much higher utility. In this case the agent will be tempted to de-commit on his current contract and establish a new one. Similarly, it is possible that right after an agent agrees to pay a certain amount to get some tasks done that a new agent appears which is willing to perform those tasks for less. Again, the agent will be tempted to de-commit on his payment.

For example, in figure 6.16 the agents start out with $\delta^0$. Lets say that in this scenario there is only one task to perform and, under $\delta^0$, agent $j$ is going to perform it. Both agents get a utility of 2 under this initial deal. However, $j$ notices that if he can give the task to $i$ then they will be at $\delta^1$ which gives $j$ a utility of 4 and $i$ a utility of 1. Thus, $j$ reasons that he could pay $i$ the amount of \$2 to perform the task. This new deal is $\delta^2$ which gives both agents a utility of 3. $\delta^2$ dominates $\delta^0$ and will thus be accepted by both. The only problem in moving from $\delta^0$ to $\delta^2$ is



| $\delta$ | $s_i(\delta)$ | $s_j(\delta)$ | $u_i(\delta)$ | $u_j(\delta)$ |
|---|---|---|---|---|
| $\delta^1$ | $\emptyset$ | $\{t_1\}$ | 1 | 3 |
| $\delta^2$ | $\{t_1\}$ | $\emptyset$ | 2 | 1 |

| $\delta$ | $s_i(\delta)$ | $s_j(\delta)$ | $u_i(\delta)$ | $u_j(\delta)$ |
|---|---|---|---|---|
| $\delta^1$ | $\emptyset$ | $\{t_1, t_2\}$ | 1 | 5 |
| $\delta^2$ | $\{t_1\}$ | $\{t_2\}$ | 2 | 3 |
| $\delta^3$ | $\{t_2\}$ | $\{t_1\}$ | 2 | 3 |
| $\delta^4$ | $\{t_1, t_2\}$ | $\emptyset$ | 8 | 1 |

Figura 6.15: Exemplo de criação de mentira por tarefa. A tabela superior mostra os valores de utilidade originais, onde $\delta^1$ é a solução de barganha de Nash. A tabela inferior mostra os valores após o agente $i$ criar a tarefa falsa $t_2$. Aqui, $\delta^4$ está a solução de negociação de Nash.

tarefa, então temos agora que considerar todas as maneiras possíveis pelas quais o novo conjunto maior de tarefas pode ser distribuído entre os agentes. Se um agente ocultar uma tarefa dos agentes, o sistema terá menos alocações possíveis a considerar. Podemos então fazer a pergunta: quando é do interesse de um agente mentir?

Essa é a questão colocada em Rules of Encounter (Rosenschein e Zlotkin, 1994). Nele, os autores assumem que os agentes usarão um protocolo de negociação que chegará à solução de negociação de Nash, presumivelmente usando ofertas alternadas com a estratégia de Zeuthen ou o protocolo ONE-STEP-NEGOTIATION da figura 6.10. Uma vez que um agente saiba que o acordo final será a solução de barganha de Nash, tudo o que ele precisa fazer é verificar cada mentira possível para ver se a solução alcançada ao contar essa mentira lhe dá uma utilidade maior do que a que ele teria recebido se dissesse a verdade. Felizmente para o agente, e infelizmente para nós, ficou demonstrado que tais mentiras geralmente existem e irão beneficiar o agente.

A Figura 6.15 mostra um exemplo em que um agente tem um incentivo para criar uma tarefa falsa. A tabela à esquerda mostra os valores iniciais de utilidade para um jogo simples com apenas uma tarefa. Neste jogo, a solução de barganha de Nash é $\delta^1$ em que $i$ não executa nenhuma tarefa e recebe uma utilidade de 1. Percebendo que obteria uma utilidade de 2 se executasse $t_1$, $i$ decide criar uma tarefa falsa $t_2$ cujos valores de utilidade são dados na tabela. Neste novo jogo, a solução de barganha de Nash é $\delta^4$ , que atribui ambas as tarefas ao agente $i$. Assim, ao criar esta tarefa, $i$ foi capaz de obter uma utilidade de 2 em vez do 1 original, obtendo $t_1$ alocado para si mesmo. Ou seja, como $t_2$ foi atribuído a$i$, o agente não precisa se preocupar em realizar esta tarefa nem se preocupar com outros agentes tentando realizar uma tarefa que ele inventou. Assim, ele pode mentir e escapar impune.

### 6.5.3 Contratos

Na nossa discussão sobre pagamentos, assumimos até agora que se um agente disser que pagará outro agente para realizar algumas tarefas, então ambos cumprirão esse contrato. As tarefas serão executadas e o dinheiro será pago. É claro que as coisas não são tão simples para aplicações reais. Especificamente, num ambiente dinâmico, um agente que recebe um conjunto de tarefas para executar pode descobrir que, logo após concordar em executar as tarefas, surge um novo acordo que lhe dará uma utilidade muito maior. Neste caso, o agente será tentado a cancelar o seu contrato atual e estabelecer um novo. Da mesma forma, é possível que logo após um agente concordar em pagar uma determinada quantia para realizar algumas tarefas, apareça um novo agente disposto a realizar essas tarefas por menos. Mais uma vez, o agente ficará tentado a cancelar o seu pagamento.

Por exemplo, na figura 6.16 os agentes começam com $\delta^0$. Digamos que neste cenário haja apenas uma tarefa a ser executada e, em $\delta^0$, o agente $j$ irá executá-la. Ambos os agentes obtêm uma utilidade de 2 neste acordo inicial. No entanto, $j$ observa que se ele puder atribuir a tarefa a $i$ então eles estarão em $\delta^1$ o que dá $j$ uma utilidade de 4 e$i$ uma utilidade de 1. Assim, $j$ razões pelas quais ele poderia pagar $i$ a quantia de \$2 para realizar a tarefa. Este novo acordo $\delta^2$ dá a ambos os agentes uma utilidade de 3. $\delta^2$ domina$\delta^0$ e, portanto, será aceito por ambos. O único problema em mudar de $\delta^0$ para $\delta^2$ é

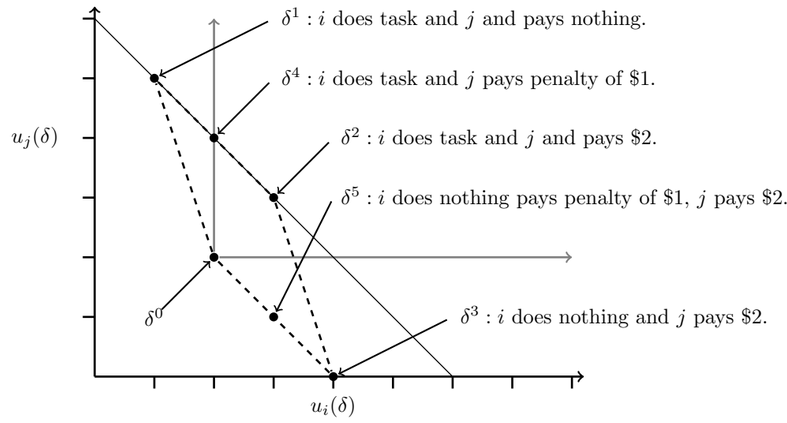



**Figure 6.16:** Leveled commitment contracts illustration. The agents start at $\delta^0$ where $j$ is performing the one available task and $i$ is idle. $\delta^1$ and $\delta^3$ are the two de-commitment contracts. When we add a de-commitment penalty of 1 then these two become $\delta^4$ and $\delta^5$ respectively.

the risks involved with this deal. If $j$ fails to pay $i$ then we are at $\delta^1$ where $i$ loses a utility of 1 as compared to $\delta^0$. Similarly, if $j$ pays but $i$ fails to perform the task then $j$ has to perform the task and we end up at $\delta^3$ where $j$ has lost 2 in utility.

A simple way to fix this problem is to enforce all contracts. If the agents agree to a deal then each one must deliver on that promise. However, there are cases when this might be impossible or undesirable. Specifically, in dynamic environments where new opportunities can appear at any time allowing agents to de-commit might increase social welfare, in other applications the cost of policing the systems might be higher than the gains from having enforcable rules. In these cases we might wish to use more sophisticated contracts.

One solution is to use **leveled commitment contracts** (Sandholm and Lesser, 2002; Sandholm and Zhou, 2002) which have the added stipulation that if an agent breaks the contract then it must pay a penalty. For example, we could add a penalty of \$1 to anyone who breaks the contract in figure 6.16. By adding this penalty we have that, instead of the de-commitments leading us to $\delta^1$ and $\delta^3$ they will lead us to $\delta^4$ and $\delta^5$. Note how these new deals are closer to $\delta^0$ thus reducing the risks of loss for both agents. Specifically, if $j$ were to de-commit and fail to pay the \$2, he must still pay the \$1 penalty to $i$ so $i$'s final utility is 2 even if $j$ de-commits, the same as $u_i(\delta^0)$. This means that there is no risk for $i$ to take this contract. Similarly, if $i$ de-commits and fails to do the task then we end up at $\delta^4$ because $i$ pays a penalty of \$1 to $j$ who must perform the task. In this case, $j$'s utility is 1 which is greater than $u_j(\delta^3) = 0$ so $j$ has reduced his risk. Such risk reduction means that in dynamic situations where the agents are constantly trying to tradeoff current income against future possibilities—deciding whether or not to wait for a better deal to come along—the agents will be able to accept more contracts and, hopefully, the system will perform better.

Note that the de-commitment decision itself is not obvious, even when the agent receives a better offer from somewhere else. For example, say that after agreeing to $\delta^2$ agent $i$ receives another offer that will give him a utility of 10 but in order to perform that task he must de-commit on the task from $j$. Clearly $i$ wants to de-commit but, a strategic agent $i$ would also realize that there is a change that $j$ will de-commit first. If $j$ de-commits then $i$ gets the penalty payment from $j$ and still gets to perform to other task. The agents thus face a version of the game of chicken (figure 3.5) with uncertainty about the other agent's payoff. Namely, $i$ does not know if $j$ wants to de-commit or not. If $i$ has some probabilistic data on $j$'s offers then these could be used to generated an extended-form game. Agent $i$ can then find the Nash equilibrium of this game and play that strategy.

Another solution is to extend the negotiation protocol to include tentative con-





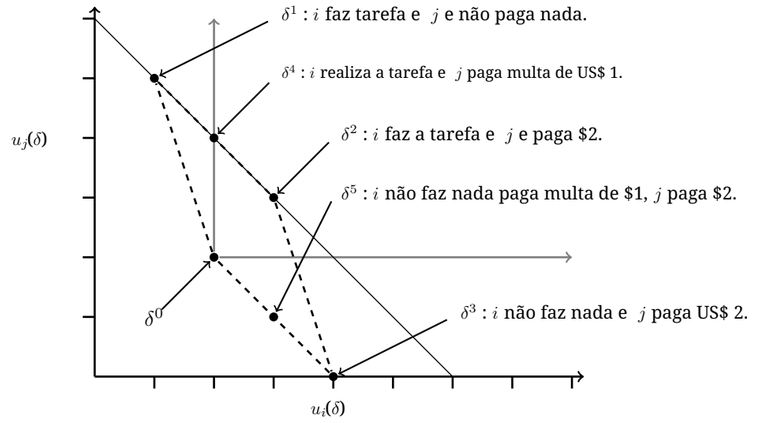



Figura 6.16: Ilustração de contratos de compromisso nivelado. Os agentes começam em $\delta^0$ onde$j$ está executando a tarefa disponível e $i$ está ocioso. $\delta^1$ e$\delta^3$ são os dois contratos de cancelamento de compromisso. Quando adicionamos uma penalidade de descomprometimento de 1, esses dois se tornam $\delta^4$ e $\delta^5$ respectivamente.

os riscos envolvidos neste negócio. Se $j$ não pagar $i$ então estaremos em $\delta^1$ onde $i$ perde uma utilidade de 1 em comparação com $\delta^0$. Da mesma forma, se $j$ paga, mas $i$ não consegue executar a tarefa, então $j$ tem que executar a tarefa e terminamos em $\delta^3$ onde $j$ perdeu 2 inutilidades.

Uma maneira simples de resolver esse problema é fazer cumprir todos os contratos. Se os agentes concordarem com um acordo, cada um deverá cumprir essa promessa. No entanto, há casos em que isso pode ser impossível ou indesejável. Especificamente, em ambientes dinâmicos onde novas oportunidades podem surgir a qualquer momento, permitindo que os agentes se descomprometam possa aumentar o bem-estar social, noutras aplicações o custo do policiamento dos sistemas pode ser superior aos ganhos decorrentes da existência de regras aplicáveis. Nestes casos, podemos desejar utilizar contratos mais sofisticados.

Uma solução é usar contratos de compromisso nivelado (Sandholm e Lesser, 2002; Sandholm e Zhou, 2002) que têm a estipulação adicional de que, se um agente quebrar o contrato, deverá pagar uma multa. Por exemplo, poderíamos adicionar uma multa de US\$ 1 a quem quebrar o contrato da figura 6.16. Ao adicionar esta penalidade temos que, em vez dos descompromissos nos levarem a $\delta^1$ e $\delta^3$ eles nos levarão a $\delta^4$ e $\delta^5$. Observe como esses novos negócios estão mais próximos de $\delta^0$ , reduzindo assim os riscos de perda para ambos os agentes. Especificamente, se $j$ cancelar o compromisso e deixar de pagar os \$2, ele ainda deverá pagar a penalidade de \$1 para $i$ então a utilidade final de $i$ é 2 mesmo se $j$ descomprometer, o mesmo que $u_i(\delta^0)$. Isso significa que não há risco de $i$ aceitar este contrato. Da mesma forma, se $i$ cancelar a confirmação e falhar na execução da tarefa, terminaremos em $\delta^4$ porque$i$ paga uma multa de US\$ 1 para $j$ quem deve executar a tarefa. Neste caso, a utilidade de $j$ é 1, que é maior que $u_j(\delta^3) = 0$ então $j$ reduziu seu risco. Esta redução do risco significa que em situações dinâmicas em que os agentes estão constantemente a tentar equilibrar o rendimento actual com as possibilidades futuras – decidindo se devem ou não esperar que surja um acordo melhor – os agentes serão capazes de aceitar mais contratos e, esperançosamente, o sistema terá um melhor desempenho.

Observe que a decisão de cancelamento de compromisso em si não é óbvia, mesmo quando o agente recebe uma oferta melhor de outro lugar. Por exemplo, digamos que depois de concordar com o $\delta^2$ agente $i$ ele recebe outra oferta que lhe dará uma utilidade de 10, mas para executar essa tarefa ele deve cancelar o compromisso da tarefa de $j$. Claramente $i$ quer cancelar o compromisso, mas um agente estratégico $i$ também perceberia que há uma mudança que $j$descomprometerá primeiro. Se $j$ descomprometer-se, então $i$ receberá o pagamento da multa de $j$ e ainda poderá executar outra tarefa. Os agentes enfrentam assim uma versão do jogo da galinha (figura 3.5) com incerteza quanto ao retorno do outro agente. Ou seja, $i$ não sabe se $j$ quer cancelar a confirmação ou não. Se $i$ tiver alguns dados probabilísticos sobre as ofertas de $j$, então eles poderão ser usados para gerar um jogo de formato estendido. O agente $i$ pode então encontrar o equilíbrio de Nash deste jogo e jogar essa estratégia.

Outra solução é estender o protocolo de negociação para incluir tentativas de negociação.

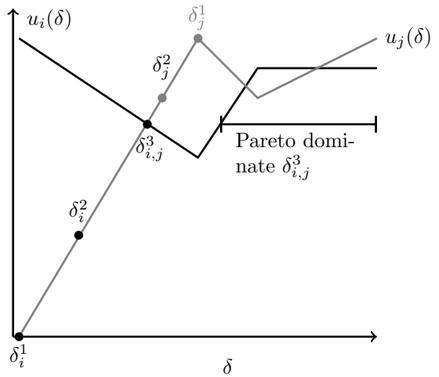



**Figure 6.17:** Example of convergence to non-Pareto deal. The agents converge to deal $\delta_{i,j}^3$ which is Pareto-dominated by all the deals indicated on the right.

tracts. For example, we could extend the contract net protocol to include pre-bid and pre-assignment steps (Aknine et al., 2004). This allows agents to make tentative commitments which they can make binding at a later time when they are more certain that no new offers will arrive from other agents.

## 6.6 Complex Deals

Thus far we have considered deals to be atomic units that cannot be broken down into smaller pieces. In contrast, real world deals are known to be composed of many different items such as price, warranty, delivery time, color, etc. For example, two agents negotiating over the purchase of a car will need to agree on the price to be paid, the color of the car, the number of years in the warranty, the value of the trade-in car, the type of financing and interest rate provided, and many other possible parameters. We describe these complex preference languages in Section 6.7. These dimensions inevitably lead to an explosion in the space of possible deals.

MULTI-DIMENSIONAL DEAL

More formally we define a **multi-dimensional deal** as one that is composed of a set of variables $x_1, x_2, \ldots, x_n$ with domains $D_1, D_2, \ldots D_n$, respectively. For example, one variable could correspond to price and its domain could be all integer numbers in the range 0 to 100. We can also re-define the agent's utility function so that it explicitly represents each variable. For example, instead of an opaque $u_i(\delta)$, we could have a more expressive $u_i(\delta) = c_1 u_i^1(x_1) + c_2 u_i^2(x_2) + \cdots + c_n u_i^n(x_n)$ or some other combination of the individual variables. The negotiation problem remains that of finding a deal that corresponds to a chosen solution concept, such as the utilitarian deal or the Nash bargaining deal.

The astute reader will notice that this is nearly the exact same problem as the distributed constraint optimization problem from Chapter 2. In fact, the only difference between multi-dimensional negotiation and the constraint optimization problem is that they assume different data distribution requirements. In distributed constraint optimization it is assumed that each agent owns one or more variables whose value it can change and the agents generally only care about the value of their variable and a few other ones. In negotiation the variables are not owned by any agent and the agents generally have preferences over all the variables. Finally, in constraint optimization there is a clear solution concept—minimize constraint valuations—while in negotiation each agent is assumed to have a utility function and there is no one obviously better solution concept.

When the problem uses multi-dimensional deals we often find that the total number of deals is extremely large—the number of possible deals grows exponentially with the number of variables. Thus, it becomes even more likely that a hill-climbing negotiation algorithm will get stuck in a local optima. For example, figure 6.17 shows how we could converge to a non-Pareto deal via protocol similar to mono-



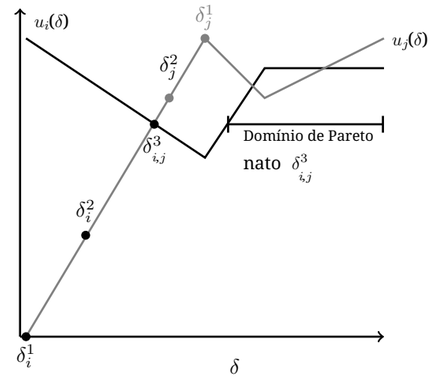



Figura 6.17: Exemplo de convergência para acordo não Pareto. Os agentes convergem para o negócio$\delta_{i,j}^3$ que é dominado por Pareto por todos os negócios indicados à direita.

folhetos. Por exemplo, poderíamos estender o protocolo da rede contratual para incluir etapas de pré-licitação e pré-atribuição (Aknine et al., 2004). Isto permite que os agentes assumam compromissos provisórios que poderão tornar vinculativos num momento posterior, quando tiverem mais certeza de que não chegarão novas ofertas de outros agentes.

### 6.6 Negócios Complexos

Até agora consideramos os negócios como unidades atômicas que não podem ser divididas em pedaços menores. Em contraste, sabe-se que os negócios do mundo real são compostos por muitos itens diferentes, como preço, garantia, prazo de entrega, cor, etc. Por exemplo, dois agentes negociando a compra de um carro precisarão concordar sobre o preço a ser pago, a cor do carro, o número de anos de garantia, o valor do carro em troca, o tipo de financiamento e a taxa de juros fornecida, e muitos outros parâmetros possíveis. Descrevemos essas linguagens de preferência complexas na Seção 6.7. Estas dimensões conduzem inevitavelmente a uma explosão no espaço de negócios possíveis.

MULTI-DIMENSIONAL DEAL

Mais formalmente, definimos um acordo multidimensional como aquele que é composto de um conjunto de variáveis $x_1, x_2, \ldots, x_n$ com domínios $D_1, D_2, \ldots D_n$, respectivamente. Por exemplo, uma variável pode corresponder ao preço e o seu domínio pode ser composto por todos os números inteiros no intervalo de 0 a 100. Também podemos redefinir a função de utilidade do agente para que represente explicitamente cada variável. Por exemplo, em vez de um opaco$u_i(\delta)$, poderíamos ter um $u_i(\delta) = c_1 u_i^1(x_1) + c_2 u_i^2(x_2) + \cdots + c_n u_i^n(x_n)$ mais expressivo ou alguma outra combinação de variáveis individuais. O problema da negociação continua a ser o de encontrar um acordo que corresponda a um conceito de solução escolhido, como o acordo utilitário ou o acordo de negociação de Nash.

O leitor astuto notará que este é quase exatamente o mesmo problema que o problema de otimização de restrições distribuídas do Capítulo 2. Na verdade, a única diferença entre a negociação multidimensional e o problema de otimização de restrições é que eles assumem diferentes requisitos de distribuição de dados. Na otimização de restrições distribuídas assume-se que cada agente possui uma ou mais variáveis cujo valor pode alterar e os agentes geralmente só se preocupam com o valor de sua variável e de algumas outras. Na negociação as variáveis não pertencem a nenhum agente e os agentes geralmente têm preferências sobre todas as variáveis. Finalmente, na otimização de restrições existe um conceito de solução claro – minimizar as avaliações de restrições – enquanto na negociação se presume que cada agente tem uma função de utilidade e não existe um conceito de solução obviamente melhor.

Quando o problema utiliza negócios multidimensionais, muitas vezes descobrimos que o número total de negócios é extremamente grande – o número de negócios possíveis cresce exponencialmente com o número de variáveis. Assim, torna-se ainda mais provável que um algoritmo de negociação ascendente fique preso em um ótimo local. Por exemplo, a figura 6.17 mostra como poderíamos convergir para um acordo não-Pareto via protocolo semelhante ao mono-

tonic concession. The agents start out by proposing deals that are the best possible for them and then concede to the other agent. However, the agents only consider deals that differ from the last proposed deal by changing only a few variables—they are "near" the last proposed deal if we define the distance to be the number of variables that have changed. Since they only consider a subset of the deals they might end up ignoring better deals. Specifically, the agents in the figure ignore the deals on the right part of the chart. As such, they end up converging on a deal which is Pareto-dominated by many other possible deals.

The Hamming distance, a similar concept from information theory, between two binary strings is the number of positions which are occupied by different values. Thus the distance between 101 and 110 is 2.

The reason for ignoring the deals is often the simple fact that the space of possible deals is so large that there is not enough time for the agent to sample it well enough to find other deals which are better than the ones it has proposed. In these cases agents choose a few attributes and change their values monotonically. For example, when negotiating a complex car deal you might wish to keep all other attributes at fixed values and negotiate only over price. In that case you might end up at a non-Pareto deal if your chosen values for the other attributes are not the right ones, such as the case where the dealer would be willing to accept a much lower price for a car with a one-year warranty rather than the three-year warranty you are asking for.

We can formalize this strategy by having the agents agree on one dimension at a time. For example, we could declare that the agents first agree on the price at which the car will be sold, then go on to haggle on the warranty, then on the color, and so on until all the dimensions are covered. The order in which these negotiations are carried out is called an **agenda**. As you might expect, ordering the negotiation in this way can sometimes lead to sub-optimal outcomes but does make the negotiation problem tractable for a lot more cases. Comparison of the results from agenda-based negotiations versus full negotiation, using specific negotiation strategies have shown that a social welfare deal is reached under certain combination of strategies (Fatima et al., 2004) .

AGENDA

### 6.6.1 Annealing Over Complex Deals

A common solution to the problem of searching for an optimal solution over a very large space of possible answers is to use an **annealing** method. Simulated annealing algorithms start out with a randomly chosen deal which becomes the best deal found thus far. New possible deals are generated by mutating this deal and are accepted as the new best deal if they are better than the current best deal or, with a certain probability, they are accepted even if they are worse than the current best. The probability of accepting a worse deal and the severity of the mutations both decrease with time. In this way the algorithms is guaranteed to converge to a locally optimal solution. In practice it has been found that this solution is often also the global optimum.

ANNEALING

Simulated annealing has been used to implement several simple negotiation protocols (Klein et al., 2003). The protocol uses a **mediator** agent instead of having agents negotiate with each other. At each step the mediator presents a deal to both agents. The agents can either accept the deal or reject it. If both of them accept the deal the mediator mutates the deal slightly and offers the new deal to both agents, who can once again either accept it or reject it. If one or more of the agents rejected the proposed deal then a mutation of the most recently accepted deal is used instead. The protocol implemented by the mediator is shown in figure 6.18.

MEDIATOR

We can then implemented two types of agents.

**Hill Climber** Accepts a deal only if it gives him a utility higher than its reservation price $u_i(\delta^-)$ and higher than that of the last deal it accepted. That is, it monotonically increases it reservation price as it accepts deals with higher utility.

**Annealer** Use a simulated annealing algorithm. That is, it maintains a temperature $T$ and accepts deals worse than the last accepted deal with probability



concessão tônica. Os agentes começam propondo negócios que sejam os melhores possíveis para eles e depois concedem ao outro agente. No entanto, os agentes apenas consideram negócios que diferem do último negócio proposto alterando apenas algumas variáveis – eles estão "próximos" do último negócio proposto se definirmos a distância como o número de variáveis que foram alteradas. Como eles consideram apenas um subconjunto dos negócios, eles podem

A distância de Hamming, um conceito semelhante à teoria da informação, entre duas cadeias binárias é o número de posições ocupadas por valores diferentes. Portanto, a distância entre 101 e 110 é 2.

acabam ignorando melhores negócios. Especificamente, os agentes na figura ignoram as negociações na parte direita do gráfico. Como tal, acabam convergindo para um acordo que é dominado por Pareto por muitos outros acordos possíveis.

A razão para ignorar os negócios é muitas vezes o simples fato de que o espaço de negócios possíveis é tão grande que não há tempo suficiente para o agente amostrá-lo bem o suficiente para encontrar outros negócios que sejam melhores do que aqueles que ele propôs. Nestes casos os agentes escolhem alguns atributos e alteram seus valores monotonicamente. Por exemplo, ao negociar um contrato complexo de carro, você pode querer manter todos os outros atributos em valores fixos e negociar apenas o preço. Nesse caso, você pode acabar em um acordo não Pareto se os valores escolhidos para os outros atributos não forem os corretos, como no caso em que o revendedor estaria disposto a aceitar um preço muito mais baixo por um carro com garantia de um ano em vez da garantia de três anos que você está solicitando.

Podemos formalizar esta estratégia fazendo com que os agentes cheguem a acordo sobre uma dimensão de cada vez. Por exemplo, poderíamos declarar que os agentes primeiro concordam com o preço pelo qual o carro será vendido, depois negociam a garantia, depois a cor e assim por diante, até que todas as dimensões sejam cobertas. A ordem pela qual essas negociações são realizadas é chamada de agenda. Como seria de esperar, ordenar a negociação em

AGENDA

dessa forma, às vezes, pode levar a resultados abaixo do ideal, mas torna o problema de negociação tratável em muito mais casos. A comparação dos resultados das negociações baseadas na agenda versus a negociação plena, utilizando estratégias de negociação específicas, mostrou que um acordo de bem-estar social é alcançado sob certas combinações de estratégias (Fatima et al., 2004).

### 6.6.1 Recozimento de negociações supercomplexas

Uma solução comum para o problema de busca de uma solução ótima em um espaço muito grande de respostas possíveis é usar um método de recozimento. Simulado

ANNEALING

algoritmos de recozimento começam com um acordo escolhido aleatoriamente que se torna o melhor acordo encontrado até o momento. Novos negócios possíveis são gerados pela mutação deste acordo e são aceitos como o novo melhor negócio se forem melhores que o melhor negócio atual ou, com uma certa probabilidade, são aceitos mesmo que sejam piores que o melhor negócio atual. A probabilidade de aceitar um acordo pior e a gravidade das mutações diminuem com o tempo. Desta forma, é garantido que os algoritmos convirjam para uma solução localmente ótima. Na prática verificou-se que esta solução é muitas vezes também o óptimo global.

O recozimento simulado tem sido usado para implementar vários protocolos de negociação simples (Klein et al., 2003). O protocolo usa um agente mediador em vez de ter

MEDIATOR

agentes negociam entre si. A cada passo, o mediador apresenta um acordo a ambos os agentes. Os agentes podem aceitar o acordo ou rejeitá-lo. Se ambos aceitarem o acordo, o mediador altera ligeiramente o acordo e oferece o novo acordo a ambos os agentes, que podem mais uma vez aceitá-lo ou rejeitá-lo. Se um ou mais agentes rejeitarem o acordo proposto, então será usada uma mutação do acordo aceito mais recentemente. O protocolo implementado pelo mediador é mostrado na figura 6.18.

Podemos então implementar dois tipos de agentes.

Hill Climber Aceita um acordo apenas se ele lhe der uma utilidade superior ao seu preço de reserva $u_i(\delta^-)$ e superior ao do último acordo que aceitou. Ou seja, aumenta monotonicamente seu preço de reserva à medida que aceita negócios de maior utilidade.

Recozimento Use um algoritmo de recozimento simulado. Ou seja, mantém a temperatura $T$ e aceita negócios piores que o último negócio aceito com probabilidade

ANNEALING-MEDIATOR

```
1  Generate random deal δ.
2  δ_accepted ← δ
3  Present δ to agents.
4  if both accept
5      then δ_accepted ← δ
6           δ ← mutate(δ)
7           goto 3
8  if one or more reject
9      then δ ← mutate(δ_accepted)
10 goto 3
```

**Figure 6.18:** Procedure used by annealing mediator.

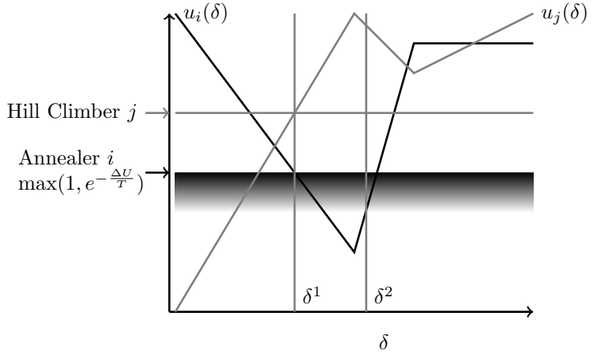



**Figure 6.19:** Two steps in the annealing algorithm with one hill climber $j$ and one annealing agent $i$. After the mediator presents $\delta^1$ both agents set up their new reservation prices shown by the line, for $j$, and the fuzzy bar for $i$.

$\max(1, e^{-\frac{\Delta U}{T}})$, where $\Delta U$ is the utility change between the deals.

Annealer agents, along with the annealing mediator, effectively implement a simulated annealing search over the set of possible deals. Meanwhile, hill climbing agents implement a hill climbing search over the set of possible deals.

For example, in figure 6.19 agent $j$ is a hill climber and $i$ is an annealer. After the mediator presents $\delta_1$ both of them accept since its utility is higher than their reservation prices. The hill climber sets a new reservation price below which he will not accept any deal. The annealer sets up a probability of accepting deals that are below his new reservation price. When $\delta_2$ appears, $j$ will accept it because it is above its reservation price and $i$ might also accept it, but with a small probability as $u_i(\delta_2)$ is slightly below its reservation price.

This type of annealing algorithm is a good example of how we can take a standard search technique like annealing and apply it to a distributed negotiation problem. Annealing works because there is some randomness in the sequence of deals that are explored. The annealing negotiation protocol places this randomness within a negotiation. Unfortunately, generating proposal deals purely at random means ignoring any knowledge that the agents have about the shape of their utility function. Also, an annealing agent is acting irrationally when it accepts a deal that is worse for it than a previous deal. We would need some way to justify why this agent occasionally acts irrationally.

## 6.7 Argumentation-Based Negotiation

We can further relax the idea of an agenda, where agents negotiate sequentially over each dimension of the deal, and instead define a more complex negotiation language which agents can use for negotiation. That is, thus far we have only considered the use of a **1-sided proposal** where a deal is proposes and it is either accepted





ANNEALING-MEDIATOR

1 Gere acordo aleatório $\delta$.2 $\delta$ aceito $\leftarrow \delta$3 Apresente $\delta$ aos agentes. 4 se ambos aceitam 5 então $\delta$aceitado $\leftarrow \delta$6 $\delta \leftarrow$mutação($\delta$) 7 vai para 3 8 se um ou mais rejeitam 9 então $\delta \leftarrow$mutação($\delta$aceita) 10 vai para 3

Figura 6.18: Procedimento utilizado pelo recozimento do mediador.

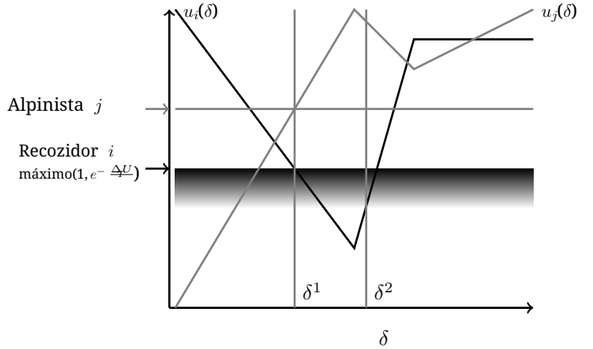



Figura 6.19: Duas etapas no algoritmo de recozimento com u m escalador $j$ e um agente de recozimento $i$. Após a apresentação do mediador $\delta^1$ ambos os agentes configuram seus novos preços de reserva mostrados pela linha, por $j$ e a barra difusa para $i$ .

$\max(1, e^{-\frac{\Delta U}{T}})$, onde $\Delta U$ é a mudança de utilidade entre as negociações.

Os agentes de recozimento, juntamente com o mediador de recozimento, implementam efetivamente uma busca de recozimento simulada sobre o conjunto de negócios possíveis. Enquanto isso, os agentes de escalada implementam uma pesquisa de escalada sobre o conjunto de negócios possíveis.

Por exemplo, na figura 6.19, o agente $j$ é um alpinista e $i$ é um recozimento. Depois que o mediador apresenta $\delta_1$ ambos aceitam, já que sua utilidade é maior que seus preços de reserva. O alpinista estabelece um novo preço de reserva abaixo do qual não aceitará nenhum acordo. O recozedor estabelece uma probabilidade de aceitar negócios abaixo do seu novo preço de reserva. Quando $\delta_2$ aparece, $j$ irá aceitá-lo porque está acima do seu preço de reserva e $i$ também poderá aceitá-lo, mas com uma pequena probabilidade, pois $u_i(\delta_2)$ está ligeiramente abaixo do seu preço de reserva.

Este tipo de algoritmo de recozimento é um bom exemplo de como podemos pegar uma técnica de busca padrão como o recozimento e aplicá-la a um problema de negociação distribuída. O recozimento funciona porque há alguma aleatoriedade na sequência de negócios explorados. O protocolo de negociação de recozimento coloca essa aleatoriedade dentro de uma negociação. Infelizmente, gerar ofertas de propostas puramente aleatórias significa ignorar qualquer conhecimento que os agentes tenham sobre a forma da sua função de utilidade. Além disso, um agente de recozimento está agindo irracionalmente quando aceita um acordo que é pior para ele do que um acordo anterior. Precisaríamos de alguma forma de justificar por que esse agente ocasionalmente age irracionalmente.

### 6.7 Negociação Baseada em Argumentação

Podemos relaxar ainda mais a ideia de uma agenda, onde os agentes negociam sequencialmente sobre cada dimensão do acordo e, em vez disso, definir uma linguagem de negociação mais complexa que os agentes possam usar para negociação. Isto é, até agora consideramos apenas o uso de uma proposta unilateral onde um acordo é proposto e é aceito

or rejected. In **argument-based protocols** the agents use a more sophisticated language for communications. There are currently no standard languages for argumentation although there is a lot of work being done on preference languages, as we will see in Chapter 7. Still, we can categorize the various types of utterances that an argumentation language might support (Jennings et al., 2001).

ARGUMENT-BASED PROTOCOLS

Specifically, an agent might be able to **critique** the proposal submitted by the other agent. Critiques provide the agents with information about others' utility functions, specifically with respect to the previous proposal. This information can be used to rule out some deals from consideration. For example, agent $i$ and $j$ might engage in the following negotiation:

CRITIQUE

$i$**:** I propose that you provide me with $x_1 = 10$ under conditions $x_2 < 20$ and delivery $x_3 < 20061025$ .

$j$**:** I am happy with the price of $x_2 < 20$ but the delivery date $x_3$ is too late.

$i$**:** I propose that I will provide you with service $x_1 = 9$ if you provide me with $x_1 = 10$.

$j$**:** I don't want $x_1 = 9$.

An agent might come back with a **counter-proposal** which are new deals, just like in the alternating offers models, but which are generally assumed to be related to the last offer. For example,

COUNTER-PROPOSAL

$i$**:** I propose that you provide me with service $x_1 = 10$ under conditions $x_2 < 20$ and delivery $x_3 < 20061025$ .

$j$**:** I propose that you provide me with service $x_1 = 10$ under conditions $x_2 < 30$ and delivery $x_3 \geq 20061025$ .

An agent might be able to **justify** his negotiation stance with other statements. Statements it hopes will convince the other agent to accept his proposal. These statements are just ways of giving more knowledge to the other agent in the hopes that it will use that knowledge to eliminate certain deals from consideration. For example,

JUSTIFY

$i$**:** My warehouses is being renovated and it will be impossible to deliver anything before the end of the month, that is $x_3 > 20061031$ .

An agent might try to **persuade** the other agent to change its negotiation stance. Persuasion is just another way of giving more knowledge tot the other agent. For example,

PERSUADE

$i$**:** Service $x_1 = 9$ is much better than you think, look at this report.

Finally, an agent might also employ **threats**, **rewards**, and **appeals** in order to convince the others to accept his proposal. These techniques help agents build better models of the others' utility functions, eliminate sets of deals from consideration, and change the agents utility functions in order to make them better reflect the reality of the situation.

THREATS
REWARDS
APPEALS

Argument-based negotiation closely matches human negotiation. Unfortunately, that also means that it is very hard to build agents that can understand these complex negotiation languages. A common approach is to build the agent using Prolog or some other logic-based language. The agent can then keep a database of the messages (facts) sent by the opponent and try to infer the set of possible contracts. Even when exploiting the inference powers of a logic-based programming language the problem of implementing correct argument-based negotiators is still very hard. No one has been able to implement a general argument-based negotiator. Even when limiting the problem to a specific domain, there are very few examples of successful programs that do argument-based negotiation. Still, there is a growing



ou rejeitado. Em protocolos baseados em argumentos, os agentes usam uma forma mais sofisticada linguagem para comunicações. Actualmente não existem linguagens padrão para argumentação, embora haja muito trabalho a ser feito sobre linguagens de preferência, como veremos no Capítulo 7. Ainda assim, podemos categorizar os vários tipos de enunciados que uma linguagem de argumentação pode suportar (Jennings et al., 2001).

ARGUMENT-BASED PROTOCOLS

Especificamente, um agente pode ser capaz de criticar a proposta apresentada pelo outro agente. As críticas fornecem aos agentes informações sobre as funções de utilidade dos outros, especificamente no que diz respeito à proposta anterior. Essas informações podem ser usadas para descartar a consideração de alguns negócios. Por exemplo, os agentes $i$ e $j$ podem participar da seguinte negociação:

CRITIQUE

$i$: proponho que você me forneça $x_1 = 10$ sob condições $x_2 < 20$ e entrega $x_3 < 20061025$ .

$j$: estou satisfeito com o preço de $x_2 < 20$ mas a data de entrega $x_3$ é muito tarde.

$i$: Proponho que lhe fornecerei um serviço $x_1 = 9$ se você me fornecer$x_1 = 10$.

$j$: não quero $x_1 = 9$.

Um agente pode voltar com uma contraproposta que são novos negócios, apenas como nos modelos de ofertas alternadas, mas que geralmente se presume estarem relacionadas com a última oferta. Por exemplo,

COUNTER-PROPOSAL

$i$: Proponho que você me forneça um serviço $x_1 = 10$ sob condições $x_2 < 20$e entrega $x_3 < 20061025$ .

$j$: Proponho que você me forneça um serviço $x_1 = 10$ sob condições $x_2 < 30$e entrega $x_3 \geq 20061025$ .

Um agente pode justificar sua posição de negociação com outras declarações. Declarações que espera convencerão o outro agente a aceitar a sua proposta. Essas declarações são apenas maneiras de dar mais conhecimento ao outro agente, na esperança de que ele use esse conhecimento para eliminar certos negócios da consideração. Por exemplo,

JUSTIFY

$i$: Meus armazéns estão sendo reformados e será impossível entregar qualquer coisa antes do final do mês, ou seja, $x_3 > 20061031$ .

Um agente pode tentar persuadir o outro agente a mudar a sua posição de negociação. A persuasão é apenas mais uma forma de dar mais conhecimento ao outro agente. Por exemplo,

PERSUADE

$i$: O serviço $x_1 = 9$ é muito melhor do que você pensa, veja este relatório.

Finalmente, um agente também pode empregar ameaças, recompensas e apelos para convencer os outros a aceitarem sua proposta. Essas técnicas ajudam os agentes a construir melhores modelos das funções de utilidade dos outros, a eliminar conjuntos de negócios da consideração e a alterar as funções de utilidade dos agentes para que reflitam melhor a realidade da situação.

THREATS
REWARDS
APPEALS

A negociação baseada em argumentos se assemelha muito à negociação humana. Infelizmente, isso também significa que é muito difícil construir agentes que possam compreender estas complexas linguagens de negociação. Uma abordagem comum é construir o agente usando Prolog ou alguma outra linguagem baseada em lógica. O agente pode então manter um banco de dados das mensagens (fatos) enviadas pelo adversário e tentar inferir o conjunto de contratos possíveis. Mesmo quando se exploram os poderes de inferência de uma linguagem de programação baseada em lógica, o problema de implementar negociadores corretos baseados em argumentos ainda é muito difícil. Ninguém foi capaz de implementar um negociador geral baseado em argumentos. Mesmo quando se limita o problema a um domínio específico, existem muito poucos exemplos de programas bem-sucedidos que realizam negociações baseadas em argumentos. Ainda assim, há u m crescente

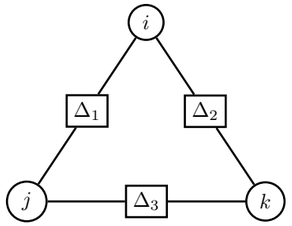



**Figure 6.20:** Graphical representation of a negotiation network with agents $i$, $j$, and $k$, where $\Delta_1$ is a set of deals that $i$ and $j$ can agree upon.

research community developing new preference description languages to be used by auctions, as we will see in Chapter 7. These languages could just as easily be used by an argumentation-based agent as by a centralized auctioneer.

There is a community of agent-based argumentation researchers working on developing a standardized language for arguments (nevar et al., 2006; Rahwan et al., 2004). They distinguish between the communication and domain languages used for representing the arguments and the negotiation protocol which constraints which type of messages can be sent at which time and what they mean. For example, a protocol could have rules on when a negotiation must end such as when one of the agents says "that is my final offer". Similarly, a protocol could have commitement rules which specify whether or not an agent can withdraw from a previous commitment it made.

## 6.8 Negotiation Networks

CONCURRENT NEGOTIATIONS

It is possible that an agent might be involved in **concurrent negotiations** with several other agents where the resulting deals might not be compatible with each other. For example, you might be negotiating with several car dealers at the same time, as well as negotiating a car loan with several lenders and car insurance with several insurance agencies. Once you make a purchasing deal with a specific dealer then you can stop negotiating with the other dealers. Furthermore, if the deal you made includes a loan agreement you can also stop negotiating with the lenders. Thus, as you negotiate with the various parties their last offers will tend to have an effect on how you negotiate with the others. For example, if a lender offers you a very low interest rate then you might be able to afford a more expensive car and you will not be swayed by the dealer's offer that includes a higher interest rate loan.

More formally, we can define a negotiation network problem as one were a set of agents negotiate over a set of deals such that all agents end up agreeing to a set of deals that are compatible with each other.

NEGOTIATION NETWORK

**Definition 6.7.** *A **negotiation network** problem involves a set of agents $A$ and set of sets of deals. Each set of deals $\Delta_i$ involves only a subset of agents $\Delta_i^a \subseteq A$ and always includes the no-deal deal $\delta^-$. A solution $\vec{\delta}$ to the problem is a set of deals, one from each $\Delta_i$ set, such that all the deals that each agent is involved in are compatible with each other. Compatibility is captures by the c function, where*

$$c_i(\delta, \delta') = \begin{cases} 1 & \text{if } \delta \text{ and } \delta' \text{ are compatible} \\ 0 & \text{otherwise.} \end{cases}$$

Figure 6.20 shows a graphical representation of a negotiation network problem with three agents: $i$, $j$ and $k$, and where $i$ and $j$ can enter into any of the deals in $\Delta_1$, similarly $i$ and $k$ can enter into a deal from $\Delta_2$ and $j$ and $k$ can enter into a deal from $\Delta_3$. We also need to define the boolean functions $c_i$, $c_j$, and $c_k$ which tell us which pair of deals are compatible.

In the negotiation networks the standard negotiation problem is further exacerbated by the fact that each agent must maintain simultaneous negotiations with several other agents and these negotiations impact each other. The approaches at solving these type of problem has thus far consisted of using ad-hoc negotiation



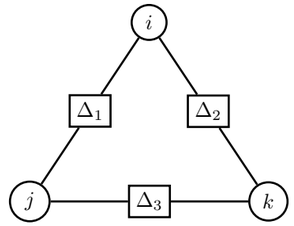



Figura 6.20: Representação gráfica de uma rede de negociação com agentes $i$, $j$ e $k$, onde $\Delta_1$ é um conjunto de negócios que $i$ e $j$ podem concordar.

comunidade de pesquisa desenvolvendo novas linguagens de descrição de preferências a serem usadas em leilões, como veremos no Capítulo 7. Essas linguagens poderiam ser usadas com a mesma facilidade por um agente baseado em argumentação ou por um leiloeiro centralizado.

Existe uma comunidade de pesquisadores de argumentação baseada em agentes trabalhando no desenvolvimento de uma linguagem padronizada para argumentos (nevar et al., 2006; Rahwan et al., 2004). Eles distinguem entre as linguagens de comunicação e de domínio usadas para representar os argumentos e o protocolo de negociação que restringe que tipo de mensagens podem ser enviadas em que momento e o que elas significam. Por exemplo, um protocolo poderia ter regras sobre quando uma negociação deve terminar, como quando um dos agentes diz "esta é a minha oferta final". Da mesma forma, um protocolo poderia ter regras de compromisso que especificassem se um agente pode ou não rescindir um compromisso anterior que assumiu.

## 6.8 Redes de Negociação

CONCURRENT NEGOTIATIONS

É possível que um agente esteja envolvido em negociações simultâneas com vários outros agentes onde os negócios resultantes podem não ser compatíveis entre si. Por exemplo, você pode estar negociando com vários revendedores de automóveis ao mesmo tempo, bem como negociando um empréstimo de carro com vários credores e um seguro automóvel com várias seguradoras. Depois de fazer um acordo de compra com um revendedor específico, você poderá interromper a negociação com os outros revendedores. Além disso, se o negócio que você fez incluir um contrato de empréstimo, você também poderá interromper a negociação com os credores. Assim, à medida que você negocia com as diversas partes, suas últimas ofertas tenderão a afetar a forma como você negocia com as outras. Por exemplo, se um credor lhe oferecer uma taxa de juros muito baixa, você poderá comprar um carro mais caro e não será influenciado pela oferta do revendedor que inclui um empréstimo com taxa de juros mais alta.

Mais formalmente, podemos definir um problema de rede de negociação como aquele em que um conjunto de agentes negocia um conjunto de acordos de modo que todos os agentes acabem concordando com um conjunto de acordos compatíveis entre si.

NEGOTIATION NETWORK

*Definição 6.7. Um problema de rede de negociação envolve um conjunto de agentes $A$ e conjunto de conjuntos de negócios. Cada conjunto de acordos $\Delta_i$ envolve apenas um subconjunto de agentes $\Delta_i^a \subseteq A$ e sempre inclui o acordo sem acordo $\delta^-$. Uma solução $\delta$ para o problema é um conjunto de negócios, um de cada conjunto $\Delta_i$ , de modo que todos os negócios em que cada agente está envolvido sejam compatíveis entre si. A compatibilidade é capturada pela função $c$ , onde*

$$c_i(\delta, \delta') = \begin{cases} 1 & \text{if } \delta \text{ and } \delta' \text{ are compatible} \\ 0 & \text{otherwise.} \end{cases}$$

A Figura 6.20 mostra uma representação gráfica de um problema de rede de negociação com três agentes: $i$, $j$ e $k$, e onde $i$ e $j$ podem entrar em qualquer um dos negócios em$\Delta_1$, da mesma forma $i$ e $k$ podem entrar em um acordo de $\Delta_2$ e $j$ e $k$ podem entrar em um acordo de $\Delta_3$. Também precisamos definir as funções booleanas $c_i$, $c_j$ e $c_k$ que nos informam quais pares de negócios são compatíveis.

Nas redes de negociação, o problema da negociação padrão é ainda agravado pelo fato de que cada agente deve manter negociações simultâneas com vários outros agentes e essas negociações impactam umas às outras. As abordagens para resolver este tipo de problema têm consistido até agora na utilização de negociações ad-hoc

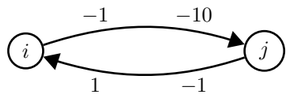



**Figure 6.21:** The coercion network.

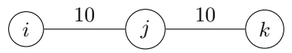



**Figure 6.22:** A network with three agents. Agent $j$ can only split 10 dollars with either $i$ or $k$, but not both.

strategies and running tests to determine how these fare against each other (Nguyen and Jennings, 2004; Zhang et al., 2005a; Zhang et al., 2005b).

### 6.8.1 Network Exchange Theory

DESCRIPTIVE APPROACH

NETWORK EXCHANGE THEORY

Another way to approach the problem of negotiation networks is to look at what humans do and try to imitate it. This is known as the **descriptive approach**. Luckily for us, researchers in Sociology have developed and studied a model which they call **network exchange theory** (NET) that is very similar to the negotiation networks problem (Willer, 1999). In this model humans are represented by nodes in a graph. The annotated edges between nodes represent the possibility of a negotiation between those two humans. Sociologists have run tests with human subjects in these networks in order to record how they behave. They then found equations that can be used to predict the deals that people eventually agree on, on average, based on the initial structure of the problem.

Figure 6.21 shows a simple NET network. The nodes represent agents $i$ and $j$. The edges are directional and represent interaction possibilities from the source agent to the destination agent where only the source agent can decide whether to enact the interaction. Specifically, if $i$ decides to enact its edge it would give $i$ a utility of $-1$ and would give $j$ a utility of $-10$. Imagine that $i$ is threatening to harm $j$ if $i$ does not relinquish his wallet. Similarly $j$ could decide to enact its edge which would give it a utility of $-1$ and give $i$ a utility of 1. This particular network is known as a coercion scenario because $i$ can threaten $j$ to give him the 1 otherwise $i$ will punish $j$ by giving it $-10$. Even thought it is not rational for $i$ to want go punish $j$ for it is punishing itself at the same time, the threat works in the real world. Another type of edge used is the resource pool edge where an amount of utility is to be divided among the agents but the agents must decide how it is to be distributed. A sample is shown in figure 6.23. This network shows two agents, $i$ and $j$, who are negotiating over how to divide 10 dollars.

NET can also represent various deal compatibility requirements. One such example is exclusive connections where one agent can exchange with any one of a number of other agents but exchange with one of them precludes exchange with any of the other ones. This is typically simply represented by just adding more nodes and edges to the graph, as shown in figure 6.22.

RESISTANCE

Based on test results, NET tells us that each person has a **resistance** to each particular payment $p$ given by a resistance equation. $i$'s resistance to payment $p$ is given by

$$r_i = \frac{p_i^{\max} - p_i}{p_i - p_i^{\text{con},}} \tag{6.8}$$

where $p_i^{\max}$ is the maximum $i$ could get and $p_i^{\text{con}}$ is the no-deal deal. $j$'s resistance is similarly defined. If we further know that $i$ and $j$ are splitting 10 dollars then we know that $p_i + p_j = 10$.

The resistance equation is meant to capture the person's resistance to agreeing to a deal at any particular price. The higher the resistance the less willing the person is to agree to the deal. Note how this equation is not linear, as might be expected if people were rational, but has an exponential shape. This shape tells us a lot about our irrational behavior.



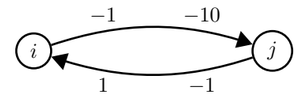



Figura 6.21: A rede de coerção.

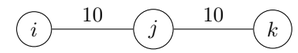



Figura 6.22: Uma rede com três agentes. O agente $j$ só pode dividir 10 dólares com $i$ ou $k$, mas não com ambos.

estratégias e execução de testes para determinar como eles se comparam (Nguyen e Jennings, 2004; Zhang et al., 2005a; Zhang et al., 2005b).

### 6.8.1 Teoria do NetworkExchange

DESCRIPTIVE APPROACH

NETWORK EXCHANGE THEORY

Outra forma de abordar o problema das redes de negociação é observar o que os humanos fazem e tentar imitá-lo. Isso é conhecido como abordagem descritiva.

Felizmente para nós, pesquisadores em Sociologia desenvolveram e estudaram um modelo que chamam de teoria da troca em rede (NET) que é muito semelhante à negociação problema de redes (Willer, 1999). Neste modelo, os humanos são representados por nós em um gráfico. As arestas anotadas entre os nós representam a possibilidade de uma negociação entre esses dois humanos. Os sociólogos realizaram testes com seres humanos nessas redes para registrar como eles se comportam. Eles então encontraram equações que podem ser usadas para prever os acordos com os quais as pessoas eventualmente concordarão, em média, com base na estrutura inicial do problema.

A Figura 6.21 mostra uma rede NET simples. Os nós representam agentes $i$ e $j$. As arestas são direcionais e representam possibilidades de interação do agente de origem para o agente de destino, onde apenas o agente de origem pode decidir se deseja realizar a interação. Especificamente, se $i$ decidir implementar sua vantagem, isso daria a $i$ uma utilidade de $-1$ e a $j$ uma utilidade de $-10$. Imagine que $i$ está ameaçando prejudicar $j$ se $i$ não abrir mão da carteira. Da mesma forma, $j$ poderia decidir implementar sua vantagem, o que lhe daria uma utilidade de $-1$ e daria a $i$ uma utilidade de 1. Esta rede específica é conhecida como cenário de coerção porque $i$ pode ameaçar $j$ dar-lhe o 1, caso contrário$i$ punirá $j$ dando-lhe $-10$. Mesmo que não seja racional $i$ querer punir $j$ pois está se punindo ao mesmo tempo, a ameaça funciona no mundo real. Outro tipo de borda usada é a borda do pool de recursos, onde uma quantidade de utilidade deve ser dividida entre os agentes, mas os agentes devem decidir como ela será distribuída. Uma amostra é mostrada na figura 6.23. Esta rede mostra dois agentes, $i$e $j$, que estão negociando como dividir 10 dólares.

NET também pode representar vários requisitos de compatibilidade de negócios. Um exemplo são as conexões exclusivas em que um agente pode trocar com qualquer um de vários outros agentes, mas a troca com um deles impede a troca com qualquer um dos outros. Isso normalmente é representado simplesmente pela adição de mais nós e arestas ao gráfico, conforme mostrado na figura 6.22.

RESISTANCE

Com base nos resultados dos testes, NET nos diz que cada pessoa tem uma resistência a cada pagamento específico $p$ dado por uma equação de resistência. A resistência de $i$ ao pagamento $p$ é dada por

$$r_i = \frac{p_i^{\max} - p_i}{p_i - p_i^{\text{con},}} \tag{6.8}$$

onde $p_i^{\max}$é o máximo que $i$ poderia obter e $p_i^{\text{con}}$é o acordo sem acordo. A resistência de $j$ é definida de forma semelhante. Se soubermos ainda que $i$ e $j$ estão dividindo 10 dólares, então saberemos que $p_i + p_j = 10$.

A equação de resistência destina-se a capturar a resistência da pessoa em c oncordar com um acordo a qualquer preço específico. Quanto maior a resistência, menos disposta a pessoa estará a concordar com o acordo. Observe como esta equação não é linear, como seria de esperar se as pessoas fossem racionais, mas tem uma forma exponencial. Essa forma nos diz muito sobre nosso comportamento irracional.

**Figure 6.23:** A sample exchange network with 10 units to be distributed among two agents.

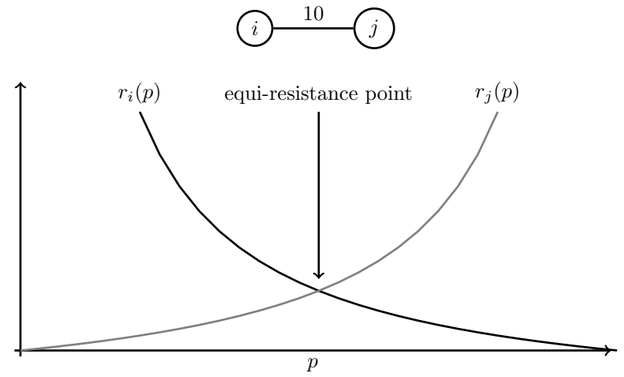



**Figure 6.24:** Resistance of two agents to each possible deal $p$.

EQUI-RESISTANCE POINT

NET tells us that exchange happens at the **equi-resistance point** where both agents have equal resistance, that is where

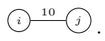
$$r_i = \frac{p_i^{max} - p_i}{p_i - p_i^{con}} = \frac{p_j^{max} - p_j}{p_j - p_j^{con}} = r_j. \tag{6.9}$$

Notice that the equi-resistance equation tells us the agreement that people will eventually reach via negotiation, but it does not tell us how the negotiation took place, that is, it does not tell us their negotiation tactics.

We can represent the equi-resistance point graphically by simply replacing $p_j$ with $10 - p_i$ in $j$'s resistance equation $r_j$ and plotting the two curves $r_i$ and $r_j$. The point at which the curves cross is the point of exchange, as shown in figure 6.24. Of course, the exchange does not always happen at the midpoint. For example, if $i$ had an offer from some other agent for 6 dollars if it refused to negotiate with $j$, this would mean that $p_i^{con} = 6$ which would change the equi-resistance point.

ITERATED EQUI-RESISTANCE ALGORITHM

We can solve complex NETs using the **iterated equi-resistance algorithm**. The algorithm simply uses the equi-resistance equation repeatedly on each edge of the graph in order to calculate the payments that these agents can expect to receiver. This is repeated for all edges until payments stop changing. For example, for the graph in figure 6.22 we would follow these steps.

1. Apply Equi-resistance to 

. Gives us $p_j = 5$.
2. Apply Equi-resistance to 

. Let $p_j^{con} = 5$ and apply equi-resistance again.
3. Repeat until quiescence.

The iterated equi-resistance algorithm is not guaranteed to converge and it might converge to different solutions when the edges are sorted in differently. Even when it does converge the deal it reaches is not guaranteed to be the one that humans would reach. However, many experiments have been performed with humans in *small* networks (less than 12 nodes) which have shown that the iterated equi-resistance algorithm correctly predicts the resulting deal. This algorithm gives a new solution for the negotiation problem which, unlike the solutions in Section 6.1.1, is not based on some desirable properties of the solution but is based on evidence from human negotiation, that is, it gives us a descriptive solution to the negotiation problem. If you want to implement agents that reach the same solution as humans then you probably want to use the iterated equi-resistance solution.



Figura 6.23: Uma rede de troca de amostras com 10 unidades a serem distribuídas entre dois agentes.

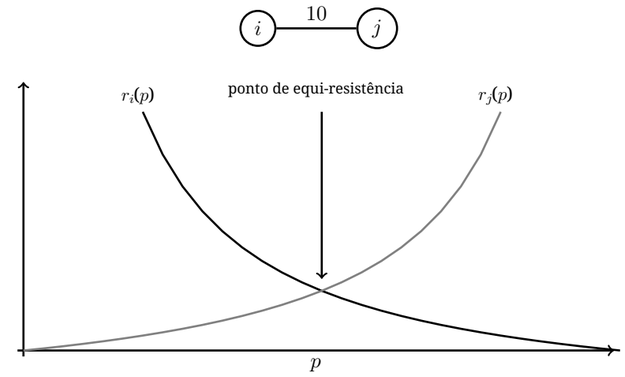



Figura 6.24: Resistência de dois agentes a cada acordo possível $p$.

EQUI-RESISTANCE POINT

NET nos diz que a troca acontece no ponto de equi-resistência onde ambos agentes têm resistência igual, é aí que

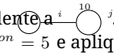
$$r_i = \frac{p_i^{max} - p_i}{p_i - p_i^{con}} = \frac{p_j^{max} - p_j}{p_j - p_j^{con}} = r_j. \tag{6.9}$$

Observe que a equação da equi-resistência nos diz o acordo que as pessoas eventualmente alcançarão por meio da negociação, mas não nos diz como a negociação ocorreu, ou seja, não nos diz suas táticas de negociação.

Podemos representar graficamente o ponto de equi-resistência simplesmente substituindo $p_j$ por $10 - p_i$ na equação de resistência de $j$ $r_j$ e traçando as duas curvas $r_i$ e $r_j$. O ponto em que as curvas se cruzam é o ponto de troca, conforme mostrado na figura 6.24. É claro que a troca nem sempre acontece no ponto médio. Por exemplo, se $i$ tivesse uma oferta de algum outro agente de 6 dólares se ele se recusasse a negociar com $j$, isso significaria que $p_i^{con} = 6$ o que mudaria o ponto de equi-resistência.

ITERATED EQUI-RESISTANCE ALGORITHM

**Podemos resolver NETs complexos usando o algoritmo de equi-resistência iterado.** O algoritmo simplesmente usa a equação de equi-resistência repetidamente em cada borda do gráfico para calcular os pagamentos que esses agentes podem esperar receber. Isso se repete para todas as arestas até que os pagamentos parem de mudar. Por exemplo, para o gráfico da figura 6.22 seguiríamos estes passos.

1. Aplique resistência equivalente a 

. Nos dá $p_j = 5$. 2. Aplique resistência equivalente a 

. Seja $p_j^{con} = 5$ e aplique a equi-resistência novamente. 3. Repita até a quiescência.

Não é garantido que o algoritmo de equi-resistência iterado convirja e pode convergir para soluções diferentes quando as arestas são classificadas de forma diferente. Mesmo quando converge, não há garantia de que o acordo alcançado seja aquele que os humanos alcançariam. No entanto, muitos experimentos foram realizados com humanos em redes pequenas (menos de 12 nós) que mostraram que o algoritmo iterado de equi-resistência prevê corretamente o acordo resultante. Este algoritmo fornece uma nova solução para o problema de negociação que, ao contrário das soluções da Secção 6.1.1, não se baseia em algumas propriedades desejáveis da solução, mas sim em evidências da negociação humana, ou seja, dá-nos uma solução descritiva para o problema de negociação. Se você deseja implementar agentes que alcancem a mesma solução que os humanos, provavelmente desejará usar a solução iterada de equi-resistência.

## Exercises

6.1 Given the following utility values for agents $i$ and $j$ over a set of possible deals $\delta$:

| $\delta$ | $u_i(\delta)$ | $u_j(\delta)$ |
|---|---|---|
| $\delta^1$ | 1 | 0 |
| $\delta^2$ | 0 | 1 |
| $\delta^3$ | 1 | 2 |
| $\delta^4$ | 3 | 1 |
| $\delta^5$ | 2 | 2 |
| $\delta^6$ | 1 | 1 |
| $\delta^7$ | 8 | 1 |

1. Which deals are on the Pareto frontier?
2. Which one is the egalitarian social welfare deal?
3. Which one is the utilitarian deal?
4. Which one is the Nash bargaining deal?
5. Which one is the Kalai-Smorodinsky deal?

6.2 In a distributed workflow enactment application, a workflow is defined as a set of tasks all of which must be performed in order to complete the workflow and collect the payment. The following table lists the available workflow instances, their tasks and their payments:

| Workflow | Tasks | Payment |
|---|---|---|
| $w_1$ | $t_1, t_1, t_1$ | 6 |
| $w_2$ | $t_1, t_1, t_2$ | 5 |
| $w_3$ | $t_1, t_1, t_3$ | 3 |
| $w_4$ | $t_1, t_2, t_2$ | 1 |
| $w_5$ | $t_2, t_2, t_3, t_3$ | 8 |
| $w_6$ | $t_2, t_2$ | 1 |

There are three agents. An agent can perform at most two tasks at a time, as long as those tasks are **different**. Also, a task is performed for only a single workflow. That, if an agent performs $t_1$ for $w_1$ then this $t_1$ cannot be used for any other workflow. The goal is for the agents to find the set of workflows that maximizes the total payment.

1. Re-state this problem as a negotiation problem, show the set of possible deals. (Hint: note that the agents have identical capabilities, so you do not need to worry about which one does which task).
2. We further constrain this negotiation by only allowing agents to either drop one workflow, add one workflow, or exchange one workflow for another. But, they can only do so if this increases the total payment (thus, you quickly conclude that they will never drop a workflow). Which deals become local optima under these constraints?



**Exercícios**

6.1 Dados os seguintes valores de utilidade para agentes $i$ e $j$ em um conjunto de negócios possíveis$\delta$:

| $\delta$ | $u_i(\delta)$ | $u_j(\delta)$ |
|---|---|---|
| $\delta^1$ | 1 | 0 |
| $\delta^2$ | 0 | 1 |
| $\delta^3$ | 1 | 2 |
| $\delta^4$ | 3 | 1 |
| $\delta^5$ | 2 | 2 |
| $\delta^6$ | 1 | 1 |
| $\delta^7$ | 8 | 1 |

1. Quais negócios estão na fronteira de Pareto? 2. Qual é o acordo igualitário de bem-estar social? 3. Qual é o acordo utilitário? 4. Qual é o acordo de barganha de Nash? 5. Qual é o acordo Kalai-Smorodinsky?

6.2 Em um aplicativo de implementação de fluxo de trabalho distribuído, um fluxo de trabalho é definido como um conjunto de tarefas que devem ser executadas para concluir o fluxo de trabalho e receber o pagamento. A tabela a seguir lista as instâncias de fluxo de trabalho disponíveis, suas tarefas e seus pagamentos:

| Fluxo de trabalho | Tarefas | Pagamento |
|---|---|---|
| $w_1$ | $t_1, t_1, t_1$ | 6 |
| $w_2$ | $t_1, t_1, t_2$ | 5 |
| $w_3$ | $t_1, t_1, t_3$ | 3 |
| $w_4$ | $t_1, t_2, t_2$ | 1 |
| $w_5$ | $t_2, t_2, t_3, t_3$ | 8 |
| $w_6$ | $t_2, t_2$ | 1 |

Existem três agentes. Um agente pode executar no máximo duas tarefas por vez, desde que essas tarefas sejam diferentes. Além disso, uma tarefa é executada apenas para um único fluxo de trabalho. Ou seja, se um agente executar $t_1$ para $w_1$ então isso $t_1$ não poderá ser usado para nenhum outro fluxo de trabalho. O objetivo é que os agentes encontrem o conjunto de fluxos de trabalho que maximize o pagamento total.

1. Reafirme este problema como um problema de negociação, mostre o conjunto de acordos possíveis. (Dica: observe que os agentes têm capacidades idênticas, portanto você não precisa se preocupar com qual deles executa qual tarefa). 2. Restringimos ainda mais essa negociação, permitindo apenas que os agentes abandonem um fluxo de trabalho, adicionem um fluxo de trabalho ou troquem um fluxo de trabalho por outro. Porém, eles só poderão fazer isso se isso aumentar o pagamento total (assim, você conclui rapidamente que eles nunca abandonarão um fluxo de trabalho). Quais negócios se tornam ótimos locais sob essas restrições?

Chapter 7

# Auctions

Auctions are a common and simple way of performing resource allocation in a multiagent system. In an auction, agents can express how much they want a particular item via their bid and a central auctioneer can make the allocation based on these bids. Of course, this generally requires the use of a centralized auctioneer but there are techniques for reducing this bottleneck. Still, even centralized auctions can be very complex and produce unexpected results if one does not understand all the details.

## 7.1 Valuations

Before we begin to talk about the various types of auctions we must first clarify how people value the items being sold. We have used the notation $u_i(s)$ to refer to the utility that agent $i$ derives from state $s$. Similarly, if $s$ is instead an item, or set of items, for sale we can say that $v_i(s)$ is the **valuation** that $i$ assigns to $s$. We furthermore assume that this valuation is expressed in a common currency, thus $v_i(s)$ then becomes the maximum amount of money that agent $i$ is willing to pay for $s$. When studying auctions we generally assume that all agents have a valuation function over all the items being sold.

VALUATION

In the simplest case this valuation function reflects the agent's utility of owning the given items. For example, if you plan to eat a meal then the amount you are willing to pay for each item in the menu depends solely on how hungry you are and how much you like each item. In these cases we say that the agent has a **private value** function.

PRIVATE VALUE

On the other hand, there are items which you cannot consume and gain no direct utility from but which might still have a resale value. The classic example are stocks. When you buy a share in some company you cannot do anything with that share, except sell it. As such, your valuation on that share depends completely on the value that others attribute, and will attribute, to that share. These are known as **common value** functions.

COMMON VALUE

Most cases, however, lie somewhere in the middle. When you buy a house you take into consideration the value that you will derive from living in that house as well as its appreciation prospects: the price you think others will pay when you finally sell it. This is an example of a **correlated value** function and is very common in the real world with durable high priced items.

CORRELATED VALUE

The type of valuation function that the agents use changes their optimal behavior in an auction. For example, if an agent has a common value function then it will likely pay more attention to what the other agents are bidding. Most multiagent implementations use agents with private value functions as most systems do not want to waste the time required to implement secondary markets for the items being sold. Still, in open multiagent systems it might be impossible to prevent secondary markets from appearing.



Capítulo 7

# Leilões

Os leilões são uma forma comum e simples de realizar a alocação de recursos em um sistema multiagente. Em um leilão, os agentes podem expressar o quanto desejam de determinado item por meio de seu lance e um leiloeiro central pode fazer a alocação com base nesses lances. É claro que isto geralmente requer o uso de um leiloeiro centralizado, mas existem técnicas para reduzir este gargalo. Ainda assim, mesmo os leilões centralizados podem ser muito complexos e produzir resultados inesperados se não compreendermos todos os detalhes.

## 7.1 Avaliações

Antes de começarmos a falar sobre os vários tipos de leilões devemos primeiro esclarecer como as pessoas valorizam os itens vendidos. Usamos a notação $u_i(s)$ para nos referir à utilidade que o agente $i$ deriva do estado $s$. Da mesma forma, se $s$ for um item, ou c onjunto de itens, à venda, podemos dizer que $v_i(s)$ é a avaliação que $i$ atribui a $s$. Nós além disso, suponha que essa avaliação seja expressa em uma moeda comum, então$v_i(s)$ torna-se então a quantia máxima de dinheiro que o agente $i$ está disposto a pagar por $s$. Ao estudar leilões geralmente assumimos que todos os agentes têm uma função de avaliação de todos os itens vendidos.

VALUATION

No caso mais simples, esta função de avaliação reflete a utilidade do agente em possuir determinados itens. Por exemplo, se você planeja fazer uma refeição, o valor que está disposto a pagar por cada item do cardápio depende apenas de sua fome e do quanto você gosta de cada item. Nestes casos dizemos que o agente possui uma função de valor privado.

PRIVATE VALUE

Por outro lado, existem itens que você não pode consumir e dos quais não obtém utilidade direta, mas que ainda podem ter valor de revenda. O exemplo clássico são as ações. Quando você compra uma ação de alguma empresa, você não pode fazer nada com essa ação, a não ser vendê-la. Como tal, a sua avaliação dessa ação depende completamente do valor que outros atribuem, e atribuirão, a essa ação. Estas são conhecidas como funções de valor comum.

COMMON VALUE

A maioria dos casos, entretanto, fica em algum ponto intermediário. Quando você compra u ma casa, você leva em consideração o valor que obterá por morar naquela casa, bem como suas perspectivas de valorização: o preço que você acha que os outros pagarão quando você finalmente vendê-la. Este é um exemplo de função de valor correlacionado e é muito comum em o mundo real com itens duráveis e de alto preço.

CORRELATED VALUE

O tipo de função de avaliação que os agentes utilizam altera seu comportamento ótimo em um leilão. Por exemplo, se um agente tiver uma função de valor comum, provavelmente prestará mais atenção ao que os outros agentes estão oferecendo. A maioria das implementações multiagentes utiliza agentes com funções de valor privado, pois a maioria dos sistemas não quer perder o tempo necessário para implementar mercados secundários para os itens vendidos. Ainda assim, em sistemas multiagentes abertos pode ser impossível impedir o aparecimento de mercados secundários.

## 7.2 Simple Auctions

There are times when there are many agents and the only thing they need to negotiate over is price. In these occasions it makes sense to use an auction since they are fast and require little agent communication. However, auctions are not as simple as they might appear at first and there are many ways in which things can go wrong when using them.

ENGLISH AUCTION
FIRST-PRICE OPEN-CRY ASCENDING

RESERVATION PRICE

The actual mechanisms used for carrying out an auction are varied. The most common of all is the **English auction**. This is a **first-price open-cry ascending** auction. It is the standard one used in most auction houses. In it, the auctioneer raises the price as people yell higher bids. Once no one is willing to bid higher, the person with the highest bid gets the item and pays his bid price. These auctions sometimes have an initial or **reservation price** below which the seller is not willing to sell. The dominant strategy in an English auction, with private value, is to bid the current price plus some small amount until either the auction is won or one's reservation price is reached.

WINNER'S CURSE

If an English auction is common or correlated value then it suffers from the **winner's curse**. For example, when you buy a stock in an English auction it means that you paid more than anyone else was willing to pay. As such, your valuation of that share must now be less than what you paid for it. In the real world we gamble that at some point in the future the others will realize that this stock really is worth that much more.

FIRST-PRICE SEALED-BID

A similar auction type is the **First-price sealed-bid** auction. In this auction each person places his bid in a sealed envelope. These are given to the auctioneer who then picks the highest bid. The winner must pay his bid amount. These auctions have no dominant strategy. The buyer's best strategy is to spy on the other bidders in order to determine what they are going to bid and then bid slightly higher than that, as long as that is less than one's reservation price. If spying is impossible then the agent has no clearly superior strategy. Because of the incentive for spying, these auctions lead to a lot of inefficiencies when paired with intelligent agents.



Ontario Flower Growers Co-op, an example of a Dutch auction at work. The two large circles in the back are used to show the descending price.

DUTCH
OPEN-CRY DESCENDING PRICE

The **Dutch** auction is an **open-cry descending price** auction. In it the seller continuously lowers the selling price until a buyer hits a buzzer, agreeing to buy at the current price. The auction's name comes from its use by the Dutch to sell flowers. The Dutch flower markets have been around for centuries and are still thriving. Every morning carts of flowers are paraded before eager flower shop owners who are equipped with a buzzer. Each cart stops before the buyers and a clock starts ticking backwards from a high number. When the first buyer hits his buzzer the flowers are sold to him at the current price. Analysis of the Dutch auction shows that it is equivalent to a first-price sealed-bid auction in terms of strategy. That is, it has no dominant strategy. However, it has the nice property of being real-time efficient. The auction closes quickly and the auctioneer can make it move even faster by lowering the price faster. This real-time efficiency makes it a very attractive auction for selling cut flowers as these lose their value quickly after being harvested.

VICKREY
SECOND-PRICE SEALED-BID

The **Vickrey** auction is a more recent addition and has some very interesting properties. It is a **second-price sealed-bid** auction. All agents place their bids and the agent with the highest bid wins the auction but the price he pays is the price of the second highest bid. Analysis of this auction has shown that bidding one's true valuation, in a private value auction, is the dominant strategy. For example, let your valuation for the item being sold be $v$. If you bid less than $v$ then you are risking the possibility that some other agent will bid $w < v$ and get the item even though you could have won it. Moreover, since $w$ is less than $v$ you could have bid $v$ and paid only $w$. As such, you have nothing to gain by bidding less than $v$ but risk the possibility of losing an auction that you could have won at an acceptable price. On the other hand, if you bid $v' > v$ then you are risking that some other agent will bid $w$, where $v' > w > v$, and thereby cause you to pay more than your reservation price $v$. At the same time, you do not gain anything by bidding higher



## 7.2 Leilões Simples

Há momentos em que há muitos agentes e a única coisa que precisam negociar é o preço. Nessas ocasiões faz sentido utilizar um leilão, pois são rápidos e exigem pouca comunicação do agente. No entanto, os leilões não são tão simples como podem parecer à primeira vista e há muitas maneiras pelas quais as coisas podem dar errado ao usá-los.

ENGLISH AUCTION
FIRST-PRICE OPEN-CRY ASCENDING

RESERVATION PRICE

Os mecanismos reais utilizados para a realização de um leilão são variados. O mais comum de todos é o leilão inglês. Este é um ascendente de grito aberto de primeiro preço leilão. É o padrão usado na maioria das casas de leilões. Nele, o leiloeiro aumenta o preço enquanto as pessoas gritam lances mais altos. Uma vez que ninguém está disposto a dar um lance mais alto, a pessoa com o lance mais alto recebe o item e paga o preço do lance. Esses leilões às vezes têm um preço inicial ou de reserva abaixo do qual o vendedor não está disposto a para vender. A estratégia dominante num leilão inglês, com valor privado, é licitar o preço atual mais uma pequena quantia até que o leilão seja ganho ou o preço de reserva seja alcançado.

WINNER'S CURSE

Se um leilão inglês tiver valor comum ou correlacionado, então ele sofre a maldição do vencedor. Por exemplo, quando você compra uma ação em um leilão inglês, isso significa que você pagou mais do que qualquer outra pessoa estava disposta a pagar. Como tal, a avaliação dessa ação deve agora ser inferior ao valor que você pagou por ela. No mundo real, apostamos que em algum momento no futuro os outros perceberão que essa ação realmente vale muito mais.

FIRST-PRICE SEALED-BID

Um tipo de leilão semelhante é o leilão de lance selado de primeiro preço. Neste leilão cada pessoa coloca sua oferta em um envelope lacrado. Estes são entregues ao leiloeiro, que então escolhe o lance mais alto. O vencedor deve pagar o valor do lance. Esses leilões não tê m estratégia dominante. A melhor estratégia do comprador é espiar os outros licitantes para determinar o que vão licitar e depois licitar um pouco mais do que isso, desde que seja inferior ao preço de reserva. Se a espionagem for impossível, então o agente não possui uma estratégia claramente superior. Devido ao incentivo à espionagem, esses leilões levam a muitas ineficiências quando combinados com agentes inteligentes.



Ontario Flower Growers Co-op, um exemplo de leilão holandês em ação. Os dois grandes círculos na parte de trás são usados para mostrar o preço descendente.

DUTCH
OPEN-CRY DESCENDING PRICE

O leilão holandês é um leilão aberto de preços descendentes. Nele o vendedor reduz continuamente o preço de venda até que um comprador toque a campainha, concordando em comprar pelo preço atual. O nome do leilão vem da sua utilização pelos holandeses para vender flores. Os mercados de flores holandeses existem há séculos e ainda prosperam. Todas as manhãs, carrinhos de flores desfilam diante dos ansiosos proprietários de floriculturas, equipados com uma campainha. Cada carrinho para diante dos c ompradores e um relógio começa a andar para trás a partir de um número alto. Quando o primeiro comprador toca a campainha, as flores são vendidas a ele pelo preço atual. A análise do leilão holandês mostra que é equivalente a um leilão de lance selado de primeiro preço em termos de estratégia. Ou seja, não tem estratégia dominante. No entanto, tem a boa propriedade de ser eficiente em tempo real. O leilão fecha rapidamente e o leiloeiro pode fazê-lo avançar ainda mais rápido, baixando o preço mais rapidamente. Esta eficiência em tempo real torna-o um leilão muito atrativo para a venda de flores cortadas, uma vez que estas perdem o seu valor rapidamente após serem colhidas.

VICKREY
SECOND-PRICE SEALED-BID

O leilão de Vickrey é uma adição mais recente e tem algumas coisas muito interessantes propriedades. É um leilão de lance selado de segundo preço. Todos os agentes fazem suas ofertas e o agente com o lance mais alto vence o leilão, mas o preço que ele paga é o preço do segundo lance mais alto. A análise deste leilão mostrou que licitar a verdadeira avaliação de alguém, num leilão de valor privado, é a estratégia dominante. Por exemplo, deixe que sua avaliação para o item que está sendo vendido seja $v$. Se você der um lance menor que $v$, estará arriscando a possibilidade de que algum outro agente faça um lance $w < v$ e receba o item, mesmo que você pudesse ter ganho. Além disso, como $w$ é menor que $v$ você poderia ter feito um lance$v$ e pago apenas $w$. Como tal, você não tem nada a ganhar com lances inferiores a $v$, mas corre o risco de perder um leilão que poderia ter vencido por um preço aceitável. Por outro lado, se você fizer um lance $v' > v$, então você corre o risco de que algum outro agente faça um lance $w$, onde $v' > w > v$, e assim faça com que você pague mais do que seu preço de reserva $v$. Ao mesmo tempo, você não ganha nada licitando mais alto

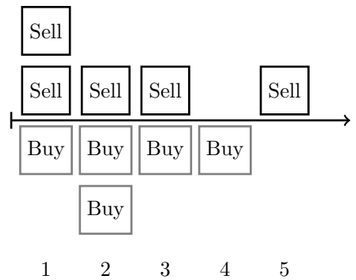



**Figure 7.1:** Graphical representation of a double auction. The $x$-axis represents prices. Each box represents one buy or sell order at the given price.

than $v$ because the only auction that you might win by bidding $v'$ instead of $v$ are those where you have to pay more than $v$.

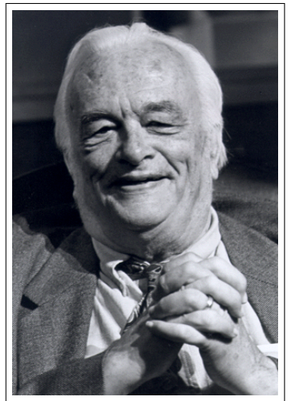
As such, the Vickrey auction eliminates the need for strategizing. Since there is an easy dominant strategy the agents do not have to think about what they should do. They just play their dominant strategy and bid their true valuation. Thus makes it a very attractive auction in terms of its efficiency but it is also for this reason that most people don't like Vickrey auctions. People are often hesitant about revealing their true valuations for an item because we know that the world is an iterated game and this information could be used against us in some future auction. As such, Vickrey auctions are seldom used in the real world.

Finally, the **double auction** is a way of selling multiple units of the same item. It is the auction used in stock markets. Each buyer places either a buy or a sell order at a particular price for a number of instances of the item (number of shares in the stock-market). The buy and sell bids can be visualized in a simple graph such as the one shown in Figure 7.1. Here, the $x$-axis represents a price and each box represents an offer to buy or sell a share at the given price.

Once we have all the bids then it is time to clear the auction. There are many different ways to clear a double auction. For example, if figure 7.1 we could match the sell order for 1 with the buy for 5 then pocket the difference of $5 - 1 = 4$ for ourselves, or we could clear it at 3 and thus give both bidders a deal, or we could match the seller for 1 with the buy for 1, and so on. As can be seen, there are many different ways to match up these pairs and it is not clear which one is better.

One metric we might wish to maximize is the amount of surplus, known as the spread by traders. That is, the sum of the differences between the buy bids and the sell bids. In some auctions this surplus is kept by the auctioneer who then has an incentive to maximize it. Another option is to use it to enable more bids to clear. In the example above the total supply and demands are 12, therefore all bids should clear. One way to do this is to pair up all the small sell bids and give these sellers exactly what they asked for then give the surplus to the sell bid of 5 in order to clear it with the remaining buy bid.

Another metric we could use is a uniform price. That is, rather than giving each seller and buyer the exact price they asked for, we give them all one uniform clearing price. Clearly, the only buy bids that will clear are those that are above the clearing price and the only sell bids that clear are those below the clearing price.



William Vickrey. 1914–1996. Nobel Prize in Economics.

### 7.2.1 Analysis

Now that we know the various auction types, there is an obvious question that we must ask ourselves. On which auction do sellers make more money? This question is answered by the following theorem.

**Theorem 7.1** (Revenue Equivalence)**.** *All four single-item auctions produce the same expected revenue in private value auctions with bidders that are risk-neutral.*





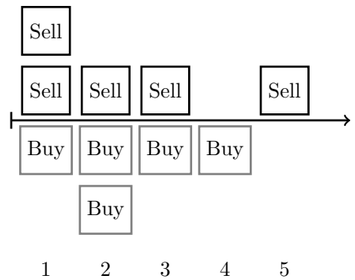



Figura 7.1: Representação gráfica de um leilão duplo. O eixo $x$ representa os preços. Cada caixa representa uma ordem de compra ou venda ao preço determinado.

do que $v$ porque os únicos leilões que você pode ganhar licitando $v'$ em vez de $v$ são aqueles em que você tem que pagar mais de $v$.

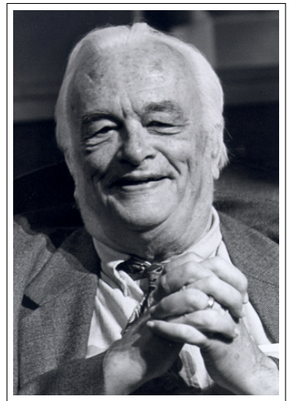
Como tal, o leilão de Vickrey elimina a necessidade de elaboração de estratégias. Como existe uma estratégia dominante fácil, os agentes não precisam pensar no que devem fazer. Eles apenas usam sua estratégia dominante e oferecem seu verdadeiro valor. Torna-se assim um leilão muito atractivo em termos de eficiência, mas é também por esta razão que a maioria das pessoas não gosta dos leilões Vickrey. As pessoas muitas vezes hesitam em revelar as suas verdadeiras avaliações de um item porque sabemos que o mundo é um jogo iterativo e esta informação pode ser usada contra nós em algum leilão futuro. Como tal, os leilões Vickrey raramente são usados no mundo real.

Por fim, o leilão duplo é uma forma de vender múltiplas unidades do mesmo item. É o leilão utilizado nas bolsas de valores. Cada comprador coloca uma ordem de compra ou de venda a um preço específico para um número de instâncias do item (número de ações no mercado de ações). As ofertas de compra e venda podem ser visualizadas em um gráfico simples como o mostrado na Figura 7.1. Aqui, o eixo $x$ representa um preço e cada caixa representa uma oferta de compra ou venda de uma ação a um determinado preço.

Assim que tivermos todos os lances, é hora de encerrar o leilão. Existem muitas maneiras diferentes de concluir um leilão duplo. Por exemplo, na figura 7.1, poderíamos igualar a ordem de venda de 1 com a compra de 5 e então embolsar a diferença de $5 - 1 = 4$ para nós mesmos, ou poderíamos liquidá-la em 3 e, assim, dar um acordo a ambos os licitantes, ou poderíamos igualar o vendedor por 1 com a compra de 1, e assim por diante. Como pode ser visto, existem muitas maneiras diferentes de combinar esses pares e não está claro qual é a melhor.

Uma métrica que podemos querer maximizar é a quantidade de excedente, conhecida como spread pelos traders. Ou seja, a soma das diferenças entre as ofertas de compra e as ofertas de venda. Em alguns leilões, esse excedente fica com o leiloeiro, que então tem um incentivo para maximizá-lo. Outra opção é usá-lo para permitir a compensação de mais lances. No exemplo acima, a oferta e a procura totais são 12, portanto todas as propostas devem ser eliminadas. Uma maneira de fazer isso é emparelhar todas as pequenas ofertas de venda e dar a esses vendedores exatamente o que eles pediram e, em seguida, dar o excedente à oferta de venda de 5, a fim de compensá-lo com a oferta de compra restante.

Outra métrica que poderíamos usar é um preço uniforme. Isto é, em vez de dar a cada vendedor e comprador o preço exato que pediram, damos a todos um preço de compensação uniforme. Claramente, as únicas ofertas de compra que serão compensadas são aquelas que estão acima do preço de compensação e as únicas ofertas de venda que serão compensadas serão aquelas abaixo do preço de compensação.



William Vickrey. 1914–1996. Prêmio Nobel de Economia.

### 7.2.1 Análise

Agora que conhecemos os vários tipos de leilão, há uma questão óbvia que devemos colocar-nos. Em qual leilão os vendedores ganham mais dinheiro? Esta questão é respondida pelo seguinte teorema.

*Teorema 7.1 (Equivalência de Receitas). Todos os quatro leilões de item único produzem a mesma receita esperada em leilões de valor privado com licitantes neutros ao risco.*

**Table 7.1:** Example of inefficient allocation.

| | tasks | Agent 1 | Agent 2 |
|---|---|---|---|
| Costs of Doing Tasks | $t_1$ | 2 | 1.5 |
| | $t_2$ | 1 | 1.5 |
| | $t_1$,$t_2$ | 2 | 2.5 |

We also know that if the bidders are risk-averse then the Dutch and first-price are better. A risk-averse bidder is willing to pay a bit more than their private valuation in order to get the item. In a Dutch or First-price auction a risk-averse agent can insure himself by bidding more than would be required.

In common or correlated value cases the English auction gives a higher revenue to the seller. The increasing price causes others to increase valuation. That is, once the agent sees others bidding very high for the item the agent realizes that the item is really worth more to the other agents so it also raises its valuation of the item.[1]

BIDDER COLLUSION

As it is often the case when money is involved, we have to be on the look out for ways in which the agents might cheat. The problem of **bidder collusion** affects all 4 auctions. In bidder collusion the bidders come to an a-priory agreement about what they will bid. They determine which one of them has the higher valuation and then all the other bidders refrain from bidding their true valuation so that the one agent can get it for a much lower price. The winner will then need to payback the others. The English and Vickrey auctions are especially vulnerable to bidder collusion as they **self-enforce collusion** agreements. For example, say there are 10 buyers and they realize that one of them has a valuation of 100 while the others have a valuation of 50 for the item. They agree to let him buy it for 1. In an English auction one of the 99 agents could defect and bid 2. However, this would only make the high-valuation agent bid 3, and so on until they get to 51. The high-valuation agent will get the item for 51 so the other agent gets nothing by defecting. The same logic applies in a Vickrey auction.

SELF-ENFORCE COLLUSION

LYING AUCTIONEER

Another problem might be that a **lying auctioneer** can make money from a Vickrey auction. All he has to do is to report a higher second-price than the one that was announced. The winner then pays this higher second price to the auctioneer who gives the buyer the real second price and pockets the difference. This requires that the bids are not revealed and that the buyer does not pay the seller directly. If the buyer paid the seller directly then a lying auctioneer and the seller could collude to charge the buyer a higher price. A lying auctioneer can also place a shill in an English auction. That is, assuming that the auctioneer gets paid a percentage of the sales price. If the auctioneer gets paid a fixed amount then there is no incentive for him to increase the sales price.

**shill**: a decoy who acts as an enthusiastic customer in order to stimulate the participation of others.

When auctioning items in a series when their valuations are interrelated, such as chairs in a dining set or bandwidth rights in adjacent counties, it is possible to arrive at inefficient allocations. For example, the problem in Table 7.1 leads to an inefficient allocation if we first auction $t_1$ and then $t_2$. Specifically, if we auction $t_1$ first then Agent 2 will get it as it has the lower cost. When we then auction $t_2$ both agents have the same cost (1) but, no matter who gets it the total cost is 2.5. If, on the other hand agent 1 had won both tasks then the total cost would be 2. matter who gets it the total cost is 2.5. This is the problem of **inefficient allocation**.

INEFFICIENT ALLOCATION

We could solve this problem if we made the agents use full lookahead effectively building an extended-form game from their possible interactions. With full lookahead the agents can build a tree where each level corresponds to a task being auctioned. In this way agent 1 can see that it will only cost him 2 to do $t_1$ and $t_2$ so it can reduce its cost for $t_1$ from 2 to 1. Of course, this puts agent 1 at risk of not getting $t_2$ since agent 1 generally will not know agent 2's cost for t2 so it does not know if it will win that auction. Another much better way of solving the problem of

[1] An interesting example of this was a British auction for 3G bandwidth licenses. The standard English auction was modified so that everyone must agree to buy at the current price or leave the room. This led to the licenses selling for 1000 times the expected amount (Harford, 2005).



Tabela 7.1: Exemplo de alocação ineficiente.

| | tarefas | Agente 1 | Agente 2 |
|---|---|---|---|
| Custos de realização de tarefas | $t_1$ | 2 | 1.5 |
| | $t_2$ | 1 | 1.5 |
| | $t_1$,$t_2$ | 2 | 2.5 |

Também sabemos que se os licitantes forem avessos ao risco, então os holandeses e o primeiro preço serão melhores. Um licitante avesso ao risco está disposto a pagar um pouco mais do que sua avaliação pessoal para obter o item. Num leilão holandês ou de primeiro preço, um agente avesso ao risco pode assegurar-se licitando mais do que seria necessário.

Nos casos de valor comum ou correlacionado o leilão inglês proporciona uma receita maior ao vendedor. O aumento do preço faz com que outros aumentem a avaliação. Ou seja, uma vez que o agente vê outros dando lances muito altos pelo item, o agente percebe que o item realmente vale mais para os outros agentes, então também aumenta a avaliação do item.[1]

BIDDER COLLUSION

Como acontece frequentemente quando há dinheiro envolvido, temos de estar atentos às formas pelas quais os agentes podem trapacear. O problema do conluio entre licitantes afeta todos os 4 leilões. No conluio entre licitantes, os licitantes chegam a um acordo prévio sobre o que irão licitar. Eles determinam qual deles tem a avaliação mais alta e então todos os outros licitantes se abstêm de licitar sua avaliação verdadeira para que o único agente possa obtê-la por um preço muito mais baixo. O vencedor precisará então pagar os outros. Os leilões de English e Vickrey são especialmente vulneráveis ao conluio de licitantes, uma vez que auto-aplicam acordos de conluio. Por exemplo, digamos que há

SELF-ENFORCE COLLUSION

10 compradores e eles percebem que um deles tem uma avaliação de 100 enquanto os outros têm uma avaliação de 50 para o item. Eles concordam em deixá-lo comprar por 1. Num leilão inglês, um dos 99 agentes poderia desertar e dar um lance de 2. No entanto, isso só faria com que o agente de maior valor oferecesse 3, e assim por diante até chegar a 51. O agente de maior valor receberá o item por 51, de modo que o outro agente não ganha nada ao desertar. A mesma lógica se aplica a um leilão de Vickrey.

LYING AUCTIONEER

Outro problema pode ser que um leiloeiro mentiroso possa ganhar dinheiro com uma Leilão de Vickrey. Tudo o que ele precisa fazer é reportar um segundo preço mais alto do que o anunciado. O vencedor então paga esse segundo preço mais alto ao leiloeiro, que dá ao comprador o segundo preço real e embolsa a diferença. Isto exige que as propostas não sejam reveladas e que o comprador não pague diretamente ao vendedor. Se o comprador pagasse diretamente ao vendedor, um leiloeiro mentiroso e o vendedor poderiam conspirar para cobrar do comprador um preço mais alto. Um leiloeiro mentiroso também pode colocar um dinheiro em uma

shill: um chamariz que atua como um cliente entusiasmado para estimular a participação de outros.

Leilão inglês. Isto é, supondo que o leiloeiro receba uma porcentagem do preço de venda. Se o leiloeiro receber um valor fixo, não haverá incentivo para ele aumentar o preço de venda.

Ao leiloar itens de uma série quando as suas avaliações estão inter-relacionadas, como cadeiras num conjunto de jantar ou direitos de largura de banda em condados adjacentes, é possível chegar a alocações ineficientes. Por exemplo, o problema da Tabela 7.1 leva a uma alocação ineficiente se leiloarmos primeiro $t_1$ e depois $t_2$. Especificamente, se leiloarmos $t_1$ primeiro, o Agente 2 receberá o item, pois tem o custo mais baixo. Quando leiloamos $t_2$ ambos os agentes têm o mesmo custo (1), mas, independentemente de quem recebe, o custo total é 2,5. Se, por outro lado, o agente 1 tivesse vencido ambas as tarefas então o custo total seria 2. Não importa quem ganha o custo total é 2,5. Este é o problema da alocação i neficiente.

INEFFICIENT ALLOCATION

Poderíamos resolver esse problema se fizéssemos com que os agentes usassem o full lookahead construindo efetivamente um jogo de formato estendido a partir de suas possíveis interações. Com full lookahead os agentes podem construir uma árvore onde cada nível corresponde a uma tarefa que está sendo leiloada. Dessa forma, o agente 1 pode ver que custará apenas 2 para fazer $t_1$ e $t_2$, portanto, pode reduzir seu custo para $t_1$ de 2 para 1. É claro que isso coloca o agente 1 em risco de não conseguir $t_2$, já que o agente 1 geralmente não saberá o custo do agente 2 para t2, portanto, ele não sabe se vencerá aquele leilão. Outra maneira muito melhor de resolver o problema de

[1] Um exemplo interessante disso foi um leilão britânico de licenças de largura de banda 3G. O leilão padrão inglês foi modificado para que todos concordem em comprar pelo preço atual ou sair da sala. Isto fez com que as licenças fossem vendidas por 1.000 vezes o valor esperado (Harford, 2005).

inefficient allocations is to use a combinatorial auction, which we will learn about in Section 7.3.

### 7.2.2 Auction Design

When designing an auction for a multiagent system you must make many decisions. You must first determine what kind of control you have over the system. It is possible that you control only the bidding agent and the auction is already implemented, as when building agents to bid on Ebay. It is possible that you control only the auction mechanism, as when building an auction website. Finally, it is possible that you might control both agents and mechanism, as when building a closed multiagent system.

The case where you control the mechanism is especially interesting. You must then decide what bidding rules you will use: when bids are to be placed, when they can no longer be placed, what form can these bids takes, and what rules they must follow. For example, you might set a rule that a new bid has to always be for a higher value. You also set up clearing rules which determine when the items are sold. We explained some of the problems with various clearing rules in the double auction. The four standard auction types already have clearing rules but you might want to modify these. Finally, you must decide on information rules: how much information the agents are to know about what the other agents bid, whether to reveal information during the bidding process itself or after clearing (Wurman et al., 2002).

Currently all online auctions are implemented as centralized web applications but it is not hard to imagine a future where the auctions are freed from the constraints of a central hub and become a protocol enacted by buying and selling agents.

## 7.3 Combinatorial Auctions

COMBINATORIAL AUCTION

Arguably, the **combinatorial auction** has been the most widely used auction in multiagent systems. In it agents can place bids for sets of items instead of just placing one bid for each item for sale. In many systems we have the problem that there is a set of tasks or jobs that needs to be distributed among the agents but the agents have complex preferences over the set of tasks. For example, in a workflow application we have a set of workflows, each composed of a set of web services, which must be performed by certain deadlines. Each agent can perform a subset of the services but each agent has different costs which might depend on the agent's type, its current load, the services it has done before, etc. Our problem as system designers is to allocate the workflows to agents so that we maximize the total number of workflows completed. Another example of combinatorial auctions is the selling of broadcasting rights by the federal government where cellular companies prefer to own the same frequencies in nearby locations, or at least to own some bandwidth in all the neighborhoods of a city. A final example is the buying of parts to put together a PC which requires a motherboard, CPU, ram, etc. Each part can be bought independently but only some bundles work together. These problems, and all problems of this type, can be solved by a combinatorial auction.

Formally, we define a combinatorial auction over a set of items $M$ as being composed of a set of bids, where each agent can supply many different bids for different subset of items. Each bid $b \in B$ is composed of $b^{\text{items}}$, which is the set of items the bid is over, $b^{\text{value}}$ the value or price of the bid, and $b^{\text{agent}}$ which is the agent that placed the bid.

For example, say you had a set of 5 figurines, one each of a different Teen Titan and you received 6 different combinatorial bids, as shown in Figure 7.2. The question you then face is how to determine which are the winning bids so as to maximize the amount of revenue you receive. Note that you can sell each item only once since you



alocações ineficientes é usar um leilão combinatório, sobre o qual aprenderemos na Seção 7.3.

### 7.2.2 Design do Leilão

Ao projetar um leilão para um sistema multiagente você deve tomar muitas decisões. Você deve primeiro determinar que tipo de controle você tem sobre o sistema. É possível que você controle apenas o agente licitante e o leilão já esteja implementado, como quando se constrói agentes para licitar no Ebay. É possível que você controle apenas o mecanismo de leilão, como ao construir um site de leilões. Finalmente, é possível que você controle tanto os agentes quanto o mecanismo, como ao construir u m sistema multiagente fechado.

O caso em que você controla o mecanismo é especialmente interessante. Você deve então decidir quais regras de licitação você usará: quando as licitações serão feitas, quando elas não poderão mais ser feitas, que forma essas licitações podem assumir e quais regras elas devem seguir. Por exemplo, você pode definir uma regra segundo a qual um novo lance deve ser sempre de um valor mais alto. Você também configura regras de compensação que determinam quando os itens são vendidos. Explicamos alguns dos problemas com diversas regras de compensação no leilão duplo. Os quatro tipos de leilão padrão já possuem regras de compensação, mas você pode querer modificá-las. Por fim, deve-se decidir sobre as regras de informação: quanta informação os agentes devem saber sobre o que os outros agentes licitam, se devem revelar informações durante o próprio processo de licitação ou após a compensação (Wurman et al., 2002).

Actualmente, todos os leilões online são implementados como aplicações web centralizadas, mas não é difícil imaginar um futuro onde os leilões sejam libertados das restrições de um hub central e se tornem num protocolo promulgado por agentes de compra e venda.

## 7.3 Leilões Combinatórios

COMBINATORIAL AUCTION

Indiscutivelmente, o leilão combinatório tem sido o leilão mais utilizado no mundo. sistemas multiagentes. Nele os agentes podem fazer lances para conjuntos de itens em vez de apenas fazer um lance para cada item à venda. Em muitos sistemas temos o problema de existir um conjunto de tarefas ou trabalhos que precisam ser distribuídos entre os agentes, mas os agentes têm preferências complexas sobre o conjunto de tarefas. Por exemplo, em uma aplicação de fluxo de trabalho temos um conjunto de fluxos de trabalho, cada um composto por um conjunto de serviços web, que devem ser executados dentro de determinados prazos. Cada agente pode executar um subconjunto de serviços, mas cada agente tem custos diferentes que podem depender do tipo do agente, da sua carga atual, dos serviços que realizou antes, etc. Nosso problema como projetistas de sistemas é alocar os fluxos de trabalho aos agentes para que maximizemos o número total de fluxos de trabalho concluídos. Outro exemplo de leilões combinatórios é a venda de direitos de transmissão pelo governo federal, onde as empresas de telefonia celular preferem possuir as mesmas frequências em locais próximos, ou pelo menos possuir alguma largura de banda em todos os bairros de uma cidade. Um último exemplo é a compra de peças para montar um PC que requer placa-mãe, CPU, memória RAM, etc. Cada peça pode ser comprada de forma independente, mas apenas alguns pacotes funcionam juntos. Estes problemas, e todos os problemas deste tipo, podem ser resolvidos através de um leilão combinatório.

Formalmente, definimos um leilão combinatório sobre um conjunto de itens $M$ como sendo composto por um conjunto de lances, onde cada agente pode fornecer muitos lances diferentes para diferentes subconjuntos de itens. Cada lance $b \in B$ é composto por $b^{\text{items}}$, que é o conjunto de itens sobre os quais o lance está, $b^{\text{value}}$ o valor ou preço do lance, e $b^{\text{agent}}$ que é o agente que fez o lance.

Por exemplo, digamos que você tenha um conjunto de 5 estatuetas, cada uma de um Teen Titan diferente, e receba 6 lances combinatórios diferentes, conforme mostrado na Figura 7.2. A questão que você enfrenta é como determinar quais são os lances vencedores para maximizar o valor da receita recebida. Observe que você pode vender cada item apenas uma vez, já que você



| Price | Bid items |
|---|---|
| $1 | Beast Boy |
| $3 | Robin |
| $5 | Raven, Starfire |
| $6 | Cyborg, Robin |
| $7 | Cyborg, Beast Boy |
| $8 | Raven, Beast Boy |

**Figure 7.2:** Teen Titans figurines: (from top left) Beast Boy, Cyborg, Robin, Raven, and Starfire. The set of combinatorial bids received for them in on the table at the right.

only have one of each. This is the problem of winner determination. In the figure, the correct solution would be to accept the $3, the $5 and the $7 bids.

### 7.3.1 Centralized Winner Determination

WINNER DETERMINATION

The **winner determination** problem is finding the set of bids that maximizes the seller's revenue. Or, more formally, find

$$X^* = \arg\max_{X \subseteq C} \sum_{b \in X} b^{\text{value}} \tag{7.1}$$

where $C$ is a set of all bid sets in which none of the bids share an item, that is

$$C = \{Y \subseteq B \,|\, \forall_{a,b' \in Y} a^{\text{items}} \cap b^{\text{items}} = \emptyset\}. \tag{7.2}$$

A variation on this problem is when agents can submit XOR bids. That is, when an agent can say that it wants only one of his bids to win. Computationally, both problems are similar.

Unfortunately, this is not a simple problem as there are, in the worst case, many possible bidsets. Specifically, if bids exists for all subsets of items then $X$ is a way of partitioning the set of items $S$ into non-overlapping subset. That is, take the set of items $S$ and figure out how many ways it can be split into smaller sets. We can calculate this number by remembering that the **Stirling number of the second kind** gives us the number of ways to partition a set of $n$ elements into $k$ non-empty sets. Namely,

STIRLING NUMBER OF THE SECOND KIND

$$S(n,k) = \frac{1}{k!} \sum_{i=0}^{k-1} (-1)^i \binom{k}{i} (k-i)^n. \tag{7.3}$$

Using this formula we can easily determine that the total number of allocations of $m$ items is given by

$$\sum_{i=1}^{m} S(m,i), \tag{7.4}$$

which is bounded by

$$O(m^m) \text{ and } \omega(m^{m/2}).$$

This means that a brute force search of all possible allocations of items to agents is computationally intractable. In fact, no approach will work in polynomial time.

**Theorem 7.2.** *Winner Determination in Combinatorial Auction is NP-hard. That is, finding the $X^*$ that satisfies* (7.1) *is NP-hard (Rothkopf et al., 1998).*

Even simplifying the problem does not make it easier to solve. For example, say that instead of trying to find the best allocation we simply want to check if there exists an allocation with total revenue of at least $w$. We call this the **decision version** of the winner determination problem. Lets also further restrict the types of bids the agents can submit. Even under these circumstances the problem remains hard.

DECISION VERSION

**Theorem 7.3.** *The decision version of the winner determination problem in combinatorial auctions is NP-complete, even if we restrict it to instances where every bid has a value equal to 1, every bidder submits only one bid, and every item is contained in exactly two bids (Cramton et al., 2006, Chapter 12).*





| Preço | Itens de l ance |
|---|---|
| $1 | Mutano |
| $3 | Robin |
| $5 | Corvo , Estelar |
| $6 | Ciborgue, Robin |
| $7 | Ciborgue, Mutano |
| $8 | Ravena, Mutano |

Figura 7.2: Figuras dos Jovens Titãs: (no canto superior esquerdo) Mutano, Ciborgue, Robin, Ravena e Estelar. O conjunto de propostas combinatórias recebidas por eles está na mesa à direita.

só tem um de cada. Este é o problema da determinação do vencedor. Na figura, a solução correta seria aceitar as ofertas de US$ 3, de US$ 5 e de US$ 7.

### 7.3.1 Determinação Centralizada do Vencedor

WINNER DETERMINATION

O problema de determinação do vencedor é encontrar o conjunto de lances que maximiza o receita do vendedor. Ou, mais formalmente, encontre

$$X^* = \arg\max_{X \subseteq C} \sum_{b \in X} b^{\text{value}} \tag{7.1}$$

Uma variação desse problema é quando os agentes podem enviar XORlances. Isto é, quando um agente pode dizer que deseja que apenas uma de suas propostas ganhe. Computacionalmente, ambos os problemas são semelhantes.

onde $C$ é um conjunto de todos os conjuntos de lances em que nenhum dos lances compartilha um item, ou seja

$$C = \{Y \subseteq B \,|\, \forall_{a,b' \in Y} a^{\text{items}} \cap b^{\text{items}} = \emptyset\}. \tag{7.2}$$

Infelizmente, este não é um problema simples; existem, no pior dos casos, muitos conjuntos de lances possíveis. Especificamente, se existirem lances para todos os subconjuntos de itens, então $X$ é uma forma de particionar o conjunto de itens $S$ em subconjuntos não sobrepostos. Ou seja, pegue o conjunto de itens $S$ e descubra de quantas maneiras ele pode ser dividido em conjuntos menores. Podemos calcular esse número lembrando que o número de Stirling de segundo tipo nos dá o número de maneiras de particionar um conjunto de $n$ elementos em $k$ não vazios

STIRLING NUMBER OF THE SECOND KIND

conjuntos.
No
meadamente,

$$S(n,k) = \frac{1}{k!} \sum_{i=0}^{k-1} (-1)^i \binom{k}{i} (k-i)^n. \tag{7.3}$$

Usando esta fórmula, podemos determinar facilmente que o número total de alocações de $m$ itens é dado por

$$\sum_{i=1}^{m} S(m,i), \tag{7.4}$$

que é delimitado por

$$O(m^m) \text{ and } \omega(m^{m/2}).$$

Isso significa que uma busca por força bruta de todas as possíveis alocações de itens aos agentes é computacionalmente intratável. Na verdade, nenhuma abordagem funcionará em tempo polinomial.

*Teorema 7.2. A determinação do vencedor no leilão combinatório é NP-difícil. Ou seja, encontrar o $X^*$ que satisfaz (7.1) é NP-difícil (Rothkopf et al., 1998).*

Mesmo simplificar o problema não o torna mais fácil de resolver. Por exemplo, digamos que em vez de tentar encontrar a melhor alocação queremos simplesmente verificar se existe uma alocação com receita total de pelo menos $w$. Chamamos isso de versão de decisão do problema de determinação do vencedor. Vamos também restringir ainda mais os tipos

DECISION VERSION

de propostas que os agentes podem apresentar. Mesmo nestas circunstâncias, o problema continua difícil.

*Teorema 7.3. A versão de decisão do problema de determinação do vencedor em leilões combinatórios é NP-completa, mesmo que a restrinjamos aos casos em que cada oferta tem um valor igual a 1, cada licitante submete apenas uma oferta e cada item está contido em exatamente duas ofertas (Cramton et al., 2006, Capítulo 12).*

```
BUILD-BRANCH-ON-ITEMS-SEARCH-TREE
1  Create a singleton bid for any item that does not have one
2  Number items from 1 to m
3  Create empty root node
4  for n ∈ M in order
5       do Add as its children all bids that
6            include the smallest item that is not an ancestor of n but
7            that do not include any item that is an ancestor of n.
```

**Figure 7.3:** Algorithm for building a branch on items search tree. This algorithms does not find a solution, it only builds a tree for the purpose of illustration.

Thus, the problem is very hard, even when we try to limit its complexity. But, there is some hope. The winner determination problem in combinatorial auctions can be reduced to a **linear programming** problem and, therefore, solved in polynomial time with well-known algorithms but only if prices can be attached to single items in the auction (Nisan, 2000). That is, there needs to be a singleton bid for every item. In many cases we can satisfy this requirement by simply adding the missing singleton bids, each with a value of 0. Specifically, the linear program which models the winner determination problem is to find the $x$ that satisfies the following:

LINEAR PROGRAMMING

Simplex is the most widely used linear programming algorithm. It has worst-case exponential time, but in practice it is much faster. Other algorithms exist that are guaranteed polynomial.

**Maximize:**

$$\sum_{b \in B} x[b] b^{\text{value}}$$

**Subject to:**

$$\sum_{b \mid j \in b^{\text{items}}} x[b] \leq 1, \forall j \in M$$

$$x[b] \in \{0, 1\}, \forall b \in B,$$

where $x[b]$ is a bit which denotes whether bid $b$ is a winning bid. That is, maximize the sum of the bid values given that each item can be in, at most, one winning bid. It has also been shown that the linear programming problem will solve a combinatorial auction when the bids satisfy any one of the following criteria (Nisan, 2000):

1. All bids are for consecutive sub-ranges of the items.
2. The bids are hierarchical.
3. The bids are only OR-of-XORs of singleton bids.
4. The bids are all singleton bids.
5. The bids are downward sloping symmetric.

A different approach to solving the winner determination problem is to conduct one of the standard AI-searches over all possible allocations, given the bids submitted. The advantage over using a linear programming solver is that we can tweak our AI search algorithms and optimize them to solve our specific problem. That is, we can put some of our domain knowledge into the algorithm to make it run faster, as we shall see.

Before we can do search we need to define our search tree. One way we can build a search tree is by having each node be a bid and each path from the root to a leaf correspond to a set of bids where no two bids share an item. The algorithm for building this tree is shown in figure 7.3. We refer to this tree as a **branch on items** search tree. Figure 7.4 shows an example tree built in this fashion. In this case we have five items for sale, numbered 1–5. The column on the left lists all the bids received. We omit the bid amount and only show the set of items for each bid. The search algorithm uses these bids to build the tree shown on the right of the figure. We start at the first level from the top. All the children of the root are bids that have item 1 in them. Then, we proceed to add children to each node. The

BRANCH ON ITEMS



BUILD-BRANCH-ON-ITEMS-SEARCH-TREE

1 Crie um lance único para qualquer item que não tenha um 2 Numere os itens de 1 a $m$3 Crie o nó raiz vazio 4 para $n \in M$ na ordem 5 doAdicione como filhos todos os lances que 6 incluam o menor item que não seja um ancestral de $n$ , mas 7 que não incluam nenhum item que seja um ancestral de $n$.

Figura 7.3: Algoritmo para construção de um ramo na árvore de busca de itens. Este algoritmo não encontra uma solução, apenas constrói uma árvore para fins de ilustração.

Assim, o problema é muito difícil, mesmo quando tentamos limitar a sua complexidade. Mas há alguma esperança. O problema de determinação do vencedor em leilões combinatórios pode ser reduzido a um problema de programação linear e, portanto, resolvido em poli-

tempo nominal com algoritmos bem conhecidos, mas apenas se os preços puderem ser atribuídos a itens individuais no leilão (Nisan, 2000). Ou seja, é necessário haver um lance único para cada ite m. Em muitos casos, podemos satisfazer esse requisito simplesmente adicionando os lances únicos ausentes, cada um com valor 0. Especificamente, o programa linear que modela o problema de determinação do vencedor é encontrar o $x$ que satisfaça o seguinte:

LINEAR PROGRAMMING

Simplex é o algoritmo de programação linear mais amplamente utilizado. Tem tempo exponencial no pior caso, mas na prática é muito mais rápido. Existem outros algoritmos que são polinomiais garantidos.

**Maximizar:**

$$\sum_{b \in B} x[b] b^{\text{value}}$$

**Sujeito a:**

$$\sum_{b \mid j \in b^{\text{items}}} x[b] \leq 1, \forall j \in M$$

$$x[b] \in \{0, 1\}, \forall b \in B,$$

onde $x[b]$ é um bit que indica se o lance $b$ é um lance vencedor. Ou seja,

maximizar a soma dos valores do lance desde que cada item possa estar em, no máximo, um lance vencedor. Também foi demonstrado que o problema de programação linear resolverá um leilão combinatório quando as propostas satisfizere m qualquer um dos seguintes critérios (Nisan, 2000):

1. Todas as licitações são para subfaixas consecutivas dos itens.

2. As licitações são hierárquicas.

3. Os lances são apenas OR-de-XORs de lances únicos. 4. Os lances são todos lances únicos. 5. As propostas são simétricas com inclinação descendente.

Uma abordagem diferente para resolver o problema de determinação do vencedor é realizar uma das pesquisas padrão AI sobre todas as alocações possíveis, dadas as propostas apresentadas. A vantagem de usar um solucionador de programação linear é que podemos ajustar nossos AI algoritmos de pesquisa e otimizá-los para resolver nosso problema específico. Ou seja, podemos colocar parte do nosso conhecimento de domínio no algoritmo para fazê-lo funcionar mais rapidamente, como veremos.

Antes de podermos fazer pesquisas, precisamos definir nossa árvore de pesquisa. Uma maneira de construir uma árvore de busca é fazer com que cada nó seja um lance e cada caminho da raiz até uma folha corresponda a um conjunto de lances onde não há dois lances que compartilhem um item. O algoritmo para construção desta árvore é mostrado na figura 7.3. Referimo-nos a esta árvore como um ramo na árvore de pesquisa de itens. A Figura 7.4 mostra um exemplo de árvore construída desta forma. Nesta

caso tenhamos cinco itens à venda, numerados de 1 a 5. A coluna da esquerda lista todas as propostas recebidas. Omitimos o valor do lance e mostramos apenas o conjunto de itens de cada lance. O algoritmo de busca utiliza esses lances para construir a árvore mostrada à direita da figura. Começamos no primeiro nível do topo. Todos os filhos da raiz são lances que possuem o item 1. Em seguida, procedemos à adição de filhos a cada nó. O

BRANCH ON ITEMS

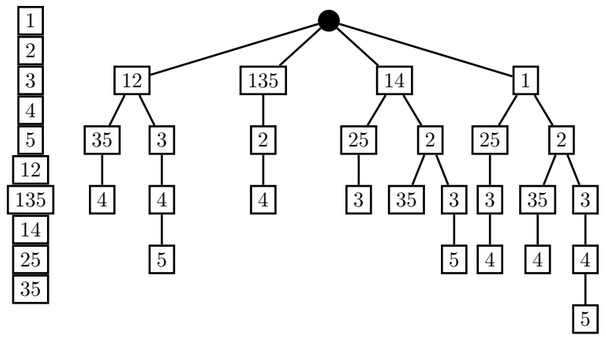



**Figure 7.4:** Branch on items search tree for winner determination in combinatorial auctions. Note that this tree has 9 leafs (9 possible ways of selling all items given the bids) while the total number of dividing 5 items into subsets is 52.

children of every node will be all the bids that contain the smallest number that is **not** on the path from the root to the node. Since the algorithm has the provision of adding a singleton bid with value 0 for every item, we are guaranteed to find a suitable bid as a children of every node. The only time we cannot find such a bid is when the path from the root to the node contains all items. In this case the node is a leaf and the set of bids from root to leaf constitutes a possible bid set.

The speedup of this search over the brute force method of considering all possible ways of breaking up 5 items into subsets can be confirmed by the fact that this tree has 9 leafs, therefore only 9 working bid sets exists. Meanwhile, the application of the Stirling formula gives us

$$\sum_{i=1\ldots5} S(5,i) = 52,$$

which means that there are 52 ways to break up 5 items into subsets. Clearly, fewer bids means faster run time which is the central idea of the search algorithm. In general, we know that the number of leafs in the tree is bounded.

**Theorem 7.4.** *The number of leaves in the tree produced by* BUILD-BRANCH-ON-ITEMS-SEARCH-TREE *is no greater than* $(|B|/|M|)^{|M|}$. *The number of nodes is no greater than* $|M|$ *times the number of leaves plus 1 (Sandholm, 2002).*

We can also build a binary tree where each node is a bid and reach edge represents whether or not that particular bid is in the solution. We refer to this tree as a **branch on bids** search tree, an example is shown in Figure 7.5. Each edge on the tree indicates whether the parent node (bid) is to be considered as part of the final bidset. For example, the rightmost branch of the tree consists of all "In" branches so the rightmost leaf node corresponds to the bidset (35)(14)(2) which forms a complete allocation. In practice, the branch on bids search tree is often faster than the previous tree because it gives us more control over the order of bids so we can better optimize the bid order. Also, the branch on bids search does not require us to add dummy singleton bids.

BRANCH ON BIDS

We now have to decide how to search our chosen tree. Since both trees have a number of nodes that is exponential on the number of bids a breadth first search would require too much memory. However, a depth first search should be possible, but time consuming. A branch and bound algorithm like the one we used for DCOP in Chapter 2.2 further helps reduce the search space and speed up computation. In order to implement it we first need a function $h$ which gives us an upper bound on the value of allocating all the items that have yet to be allocated. One such function is $h$

$$h(g) = \sum_{j \in M - \bigcup_{b \in g} b^{\text{items}}} \max_{b | j \in b^{\text{items}}} \frac{b^{\text{value}}}{|b^{\text{items}}|}, \tag{7.5}$$

where $g$ is the set of bids that have been cleared. The function $h$ simply adds up the maximum possible revenue that each item not in $g$ could contribute by using the



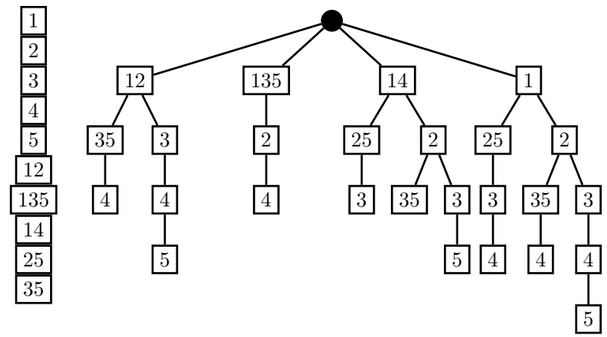



Figura 7.4: Ramificação na árvore de busca de itens para determinação do vencedor em leilões combinatórios. Observe que esta árvore tem 9 folhas (9 maneiras possíveis de vender todos os itens de acordo com os lances), enquanto o número total de divisão de 5 itens em subconjuntos é 52.

filhos de cada nó serão todos os lances que contêm o menor número que não está no caminho da raiz até o nó. Como o algoritmo permite adicionar um lance único com valor 0 para cada item, temos a garantia de encontrar um lance adequado como filho de cada nó. A única vez que não conseguimos encontrar tal lance é quando o caminho da raiz até o nó contém todos os itens. Neste caso o nó é uma folha e o conjunto de lances da raiz à folha constitui um possível conjunto de lances.

A aceleração desta busca sobre o método da força bruta de considerar todas as formas possíveis de dividir 5 itens em subconjuntos pode ser confirmada pelo fato de que esta árvore tem 9 folhas, portanto existem apenas 9 conjuntos de ofertas de trabalho. Enquanto isso, a aplicação da fórmula de Stirling nos dá

$$\sum_{i=1\ldots5} S(5,i) = 52,$$

o que significa que existem 52 maneiras de dividir 5 itens em subconjuntos. Claramente, menos lances significa tempo de execução mais rápido, que é a ideia central do algoritmo de busca. Em geral, sabemos que o número de folhas da árvore é limitado.

*Teorema 7.4. O número de folhas na árvore produzida por* BUILD-BRANCH-ON-ITEMS-SEARCH-TREE *não é maior que* $(|B|/|M|)^{|M|}$. *O número de nós não é maior que* $|M|$ *vezes o número de folhas mais 1 (Sandholm, 2002).*

Também podemos construir uma árvore binária onde cada nó é uma oferta e o limite de alcance representa se aquela oferta específica está ou não na solução. Referimo-nos a esta árvore como um ramo na árvore de busca de lances, um exemplo é mostrado na Figura 7.5. Cada borda

BRANCH ON BIDS

na árvore indica se o nó pai (bid) deve ser considerado como parte do conjunto de lances final. Por exemplo, o ramo mais à direita da árvore consiste em todos os ramos " In", de modo que o nó folha mais à direita corresponde ao bidset (35)(14)(2) que forma uma alocação completa. Na prática, a ramificação da árvore de busca de lances cost uma ser mais rápida que a árvore anterior porque nos dá mais controle sobre a ordem dos lances para que possamos otimizar melhor a ordem dos lances. Além disso, o ramo de pesquisa de lances não exige que adicionemos lances únicos fictícios.

Agora temos que decidir como pesquisar a árvore escolhida. Como ambas as árvores têm um número de nós que é exponencial em relação ao número de lances, uma primeira pesquisa ampla exigiria muita memória. No entanto, uma primeira pesquisa profunda deve ser possível, mas demorada. Um algoritmo branch and bind como aquele que usamos para DCOPno Capítulo 2.2 ajuda ainda mais a reduzir o es paço de busca e a acelerar a computação. Para implementá-lo, primeiro precisamos de uma função $h$ que nos dê um limite superior para o valor de alocação de todos os itens que ainda não foram alocados. Uma dessas funções é $h$

$$h(g) = \sum_{j \in M - \bigcup_{b \in g} b^{\text{items}}} \max_{b | j \in b^{\text{items}}} \frac{b^{\text{value}}}{|b^{\text{items}}|}, \tag{7.5}$$

onde $g$ é o conjunto de lances que foram limpos. A função $h$ simplesmente soma a receita máxima possível com a qual cada item que não está em $g$ poderia contribuir usando o

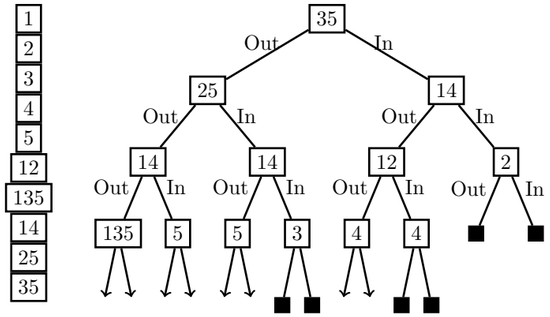



**Figure 7.5:** Branch on bids partial tree. The black boxes indicate search paths that have terminated because they denote a complete set of bids, that is, no more bids can be added because they contain items already sold.

```
BRANCH-ON-BIDS-CA()
1  r* ← 0          ▷ Max revenue found. Global variable.
2  g* ← ∅          ▷ Best solution found. Global variable.
3  BRANCH-ON-BIDS-CA-HELPER(∅, B)
4  return g*

BRANCH-ON-BIDS-CA-HELPER(g, available-bids)
1  if available-bids = ∅
2     then return
3  if ⋃_{b∈g} b^items = M                    ▷ g covers all items
4     then if Σ_{b∈g} b^value > r*           ▷ g has higher revenue than r*
5             then g* ← g
6                  r* ← Σ_{b∈g} b^value
7          return
8  next ← FIRST(available-bids)
9  if next^items ∩ ⋃_{b1∈g} b1^items = ∅}    ▷ next's items do not overlap g
10    then g' ← g + next
11         if Σ_{b1∈g'} b1^value + h(g') > r*
12            then BRANCH-ON-BIDS-CA-HELPER(g', REST(available-bids))
13 BRANCH-ON-BIDS-CA-HELPER(g, REST(available-bids))
```

**Figure 7.6:** A centralized branch and bound algorithm that searchers a branch on bids tree and finds the revenue maximizing solution given a set $B$ of combinatorial bids over items $M$.

bid that pays the most for each item, divided by the number of items on the bid. This function provides an upper bound since no feasible bidset with higher revenue can exist.

Given the upper bound $h(g)$ we can then implement the branch and bound algorithm shown in figure 7.6. This algorithm searches the branch on bids tree. It maintains a partial solution $g$ to which it adds one bid on each recursive call. Whenever it realizes that partial solution will never be able to achieve revenue that is higher than the best revenue it has already found then it gives up on that subtree, see line 8 of BRANCH-ON-BIDS-CA-HELPER. This algorithm is complete and thus guaranteed to find the revenue maximizing bidset.

We can also use the same heuristic function to do an $A^*$ search. Unfortunately, since $A^*$ acts much like a breadth first search it generally consumes too much memory. A viable solution is to use iterative deepening $A^*$. $IDA^*$ guesses how much revenue we can expect and runs a depth-first search that prunes nodes that have used more than that. If a solution is not found then the guess is reduced and we try again. $IDA^*$, with some optimizations, was implemented by the **Bidtree** algorithm (Sandholm, 1999) on the branch on items search tree. In practice, this approach was found to often be slower than a branch and bound search.

The BRANCH-ON-BIDS-CA algorithm is the basic framework for the Combinatorial Auction Branch on Bids (**CABOB**) algorithm (Sandholm et al., 2005). CABOB

BIDTREE

CABOB



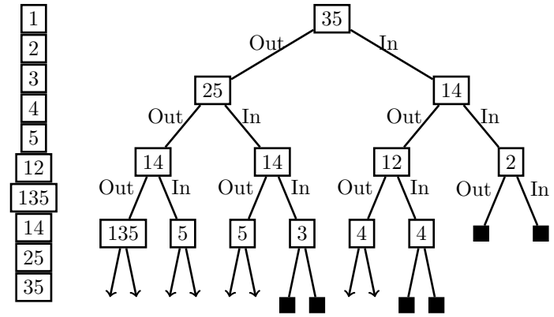



Figura 7.5: Ramificação na árvore parcial de ofertas. As caixas pretas indicam caminhos de busca que foram encerrados porque denotam um conjunto completo de lances, ou seja, não podem ser adicionados mais lances porque contêm itens já vendidos.

BRANCH-ON-BIDS-CA()
1 $r* \leftarrow 0$ ▷ Receita máxima encontrada. Variável global.2 $g* \leftarrow \emptyset$ ▷ Melhor solução encontrada. Variável global.3 BRANCH-ON-BIDS-CA-HELPER($\emptyset, B$) 4 retorno $g^*$

BRANCH-ON-BIDS-CA-HELPER(*g, lances disponíveis)*

1 se lances disponíveis $= \emptyset$2 então retorna 3 se $\bigcup_{b\in g} b^{\text{items}} = M$ ▷ $g$ cobre todos os itens 4 então se $\sum_{b\in g} b^{\text{value}} > r^*$6 ▷ $g$ tem receita maior que $r^*$5 então $g^* \leftarrow g$ $r^* \leftarrow \sum_{b\in g} b^{\text{value}}$7 retorna 8 próximos $\leftarrow$ FIRST(lances disponíveis) 9 se próximo$^{\text{items}} \cap \bigcup_{b_1\in g} b_1^{\text{items}} = \emptyset\}$ ▷ os itens do próximo não se s obrepõem $g$10 então $g' \leftarrow g +$próximos 11 se $\sum_{b_1\in g'} b_1^{\text{value}} + h(g') > r^*$12 então BRANCH-ON-BIDS-CA-HELPER($g'$, REST(lances disponíveis))13 BRANCH-ON-BIDS-CA-HELPER ($g$, REST(lances disponíveis))

Figura 7.6: Um algoritmo centralizado de ramificação e limite que pesquisa uma ramificação na árvore de lances e encontra a solução de maximização de receita dado um conjunto $B$ de lances combinatórios sobre itens $M$.

lance que paga mais por cada item, dividido pelo número de itens no lance. Esta função fornece um limite superior, uma vez que não pode existir nenhum conjunto de lances viável com receita mais alta.

Dado o limite superior $h(g)$, podemos então implementar o algoritmo de ramificação e limite mostrado na figura 7.6. Este algoritmo pesquisa o ramo da árvore de lances. Mantém uma solução parcial $g$ à qual adiciona um lance em cada chamada recursiva. Sempre que percebe que a solução parcial nunca será capaz de obter uma receita superior à melhor receita já encontrada, então ele desiste dessa subárvore, veja a linha 8 de BRANCH-ON-BIDS-CA-HELPER. Este algoritmo é completo e, portanto, garantido para encontrar o conjunto de lances que maximiza a receita.

Também podemos usar a mesma função heurística para fazer uma pesquisa $A^*$. Infelizmente, como $A^*$ age como uma primeira pesquisa ampla, geralmente consome muita memória. Uma solução viável é usar o aprofundamento iterativo $A^*$. $IDA^*$ adivinha quanta receita podemos esperar e executa uma pesquisa profunda que remove os nós que usaram mais do que isso. Se uma solução não for encontrada, a estimativa é reduzida e tentamos novamente. $IDA^*$, com algumas otimizações, foi implementado pelo Bidtree algoritmo (Sandholm, 1999) no ramo da árvore de busca de itens. Na prática, descobriu-se que essa abordagem costuma ser mais lenta do que uma pesquisa ramificada.

O algoritmo BRANCH-ON-BIDS-CA é a estrutura básica para o algoritmo Combinatorial Auction Branch on Bids (CABOB) (Sandholm etal., 2005). CABOB

BIDTREE

CABOB

```
BRANCH-ON-ITEMS-CA()
1  r* ← 0        ▷ Max revenue found. Global variable.
2  g* ← ∅        ▷ Best solution found. Global variable.
3  BRANCH-ON-ITEMS-CA-HELPER(1, ∅)
4  return g*

BRANCH-ON-ITEMS-CA-HELPER(i, g)
1  if i = m                                          ▷ g covers all items
2     then if Σ_{b∈g} b^value > r*                   ▷ g has higher revenue than r*
3             then g* ← g
4                  r* ← Σ_{b∈g} b^value
5          return
6  for b ∈ {b ∈ B | i ∈ b^items ∧ b^items ∩ ⋃_{b1∈g} b1^items = ∅}  ▷ b's items do not overlap g
7       do g' ← g + b
8          if Σ_{b1∈g'} b1^value + h(g') > r*
9             then BRANCH-ON-ITEMS-CA-HELPER(i + 1, g')
```

**Figure 7.7:** A centralized branch and bound algorithm that searchers a branch on items tree and finds the revenue maximizing solution given a set $B$ of combinatorial bids over items $M$.

improves the performance of the basic algorithm in several ways, one of which is by improving the search for new bids to add to the partial solution. Specifically, we note that a naive implementation of line 6 of BRANCH-ON-BIDS-CA-HELPER would mean that we would build this set on each recursive call to the function. That would be very time consuming as there are an exponential number of bids in $B$. CABOB handles this problem by maintaining graph data structure which has all the bids that can still be used given $g$. The nodes in the graph are the bids that are still available and the edges connect every pair of bids that share an item. In this way when a new bid is added to $g$ it is removed from the graph as well as all the other bids that are connected to it.

ca

CASS

We can also perform the branch and bound search on the branch on items search tree, as shown in Figure 7.7. This algorithm is the basis for the **CASS** (Combinatorial Auction Structured Search) algorithm which also implements further refinements on the basic algorithm (Fujishima et al., 1999).

Most algorithms for centralized winner determination in combinatorial auction expand on the basic branch and bound search by using specialized data structures to speed up access to the information need—the viable bids given the current partial solution—and implement heuristics which have been shown to reduce the size of the search space, especially for certain popular bid distributions. Some heuristics that have been found useful include the following:

- Keep only the highest bid for any set. That is, if there is a bid of $10 for items 1,2 and another bid of $20 for 1,2 then we get rid of the $10 bid.
- Remove provably noncompetitive bids, that is, those that are dominated by another bid or sets of bids. For example, if there is a bid for $10 for item 1 and another bid for $5 for items 1,2 then the $10 bid dominates the $5 bid—any situation in which we choose the $5 bid would be made better if we changed that bid for the $10 bid.
- Decompose bids into connected sets, each solved independently. If we can separate the set of bids into two or more sets of bids where all bids for any item are to be found in only one of the sets then this set of bids becomes a smaller, and independent, winner determination problem.
- Mark noncompetitive tuple of bids. For example, if there are bids $1:(1,2), $1:(3,4), $10:(1,3), $10:(2,4) then the pair of $10 bids dominates the pair of $1 bids, so we can get rid of them.



```
BRANCH-ON-ITEMS-CA()
1  r* ← 0        ▷ Receita máxima encontrada. Variável
global.2  g* ← ∅        ▷ Melhor solução encontrada.
Variável global.3  BRANCH-ON-ITEMS-CA-HELPER(1, ∅) 4
retorno g*

BRANCH-ON-ITEMS-CA-HELPER(i, g)
1 se  i = m                                          ▷ g cobre todos os itens 2 então se
4 Σ_{b∈g} b^value > r*          ▷ g tem receita maior que  r*3 então  g* ← g
                  r* ← Σ_{b∈g} b^value 5 retorna 6 para os itens de
 b ∈ {b ∈ B | i ∈ b^items ∧ b^items ∩ ⋃_{b1∈g} b1^items = ∅} ▷ b não se sobrepõem  g7 faz  g' ← g + b8 se
Σ_{b1∈g'} b1^value + h(g')  > r*9 então  BRANCH-ON-ITEMS-CA-HELPER(i + 1, g')
```

Figura 7.7: Um algoritmo centralizado de ramificação e limite em que os pesquisadores ramificam-se na árvore de itens e encontram a solução de maximização de receita dado u m conjunto $B$ de lances combinatórios sobre itens $M$.

melhora o desempenho do algoritmo básico de diversas maneiras, uma delas é melhorando a busca por novos lances para adicionar à solução parcial. Especificamente, notamos que uma implementação ingênua da linha 6 de BRANCH-ON-BIDS-CA-HELPER significaria que construiríamos esse conjunto em cada chamada recursiva para a função. Isso consumiria muito tempo, pois há um número exponencial de lances em $B$. CABOBlida com esse problema mantendo a estrutura de dados do gráfico que contém todos os lances que ainda podem ser usados $g$. Os nós no gráfico são os lances que ainda estão disponíveis e as arestas conectam cada par de lances que compartilham um item. Desta forma quando um novo lance é adicionado a $g$ ele é removido do gráfico assim como todos os outros lances que estão conectados a ele.

ca

CASS

Também podemos realizar a pesquisa ramificada e vinculada na árvore de pesquisa de ramificação de itens, conforme mostrado na Figura 7.7. Este algoritmo é a base para o CASS (Combina-

(Fujishima etal., 1999).

A maioria dos algoritmos para determinação centralizada de vencedores em leilões combinatórios ampliam a pesquisa básica branch andbound, usando estruturas de dados especializadas para acelerar o acesso à necessidade de informação – as ofertas viáveis, dada a solução parcial atual – e implementam heurísticas que demonstraram reduzir o tamanho do espaço de pesquisa, especialmente para certas distribuições de ofertas populares. Algumas heurísticas consideradas úteis incluem o seguinte:

- Mantenha apenas o lance mais alto para qualquer conjunto. Ou seja, se houver um lance de R$ 10 para os itens 1,2 e outro lance de R$ 20 para 1,2 então nos livramos do lance de R$ 10.
- Remova lances comprovadamente não competitivos, ou seja, aqueles que são dominados por outro lance ou conjuntos de lances. Por exemplo, se houver um l ance de US$ 10 para o item 1 e outro lance de US$ 5 para os itens 1,2, então o lance de US$ 10 domina o lance de US$ 5 – qualquer situação em que escolhêssemos o lance de US$ 5 seria melhor se mudássemos esse lance para o lance de US$ 10.
- Decomponha os lances em conjuntos conectados, cada um resolvido de forma independente. Se pudermos separar o conjunto de lances em dois ou mais conjuntos de lances, onde todos os lances para qualquer item podem ser encontrados em apenas um dos conjuntos, então esse conjunto de lances se torna um problema menor e independente de determinação do ve ncedor.
- Marcar tupla não competitiva de lances. Por exemplo, se houver lances $1:(1,2), $ 1:(3,4), $10:(1,3), $10:(2,4) então o par de lances de $10 domina o par de lances de $1, então podemos nos livrar deles.

- In the branch-on-items tree place the items with the most bids first on the tree. This way the most constrained items are tried first thereby creating fewer leafs.
- If the remaining search subtree is the same for different nodes in the search tree, as can happen when different items are cleared but by different bids, then these subtrees can be cached. The subtree is solved once and the answer, that is, the best set of bids found in it, is saved for possible future use.

In general the best speed attainable by the best algorithms varies greatly depending on the type of bids submitted. For example, if the number of items in each bid is chosen from a flat probability distribution then we can solve problems with thousands of items and tens of thousands of bids in seconds. On the other hand, if each bid contains exactly five randomly chosen items and a price of 1 then we can only solve problems with tens of items and hundreds of bids in a minute. The Combinatorial Auction Test Suite (**CATS**) can generate realistic types of bid distributions so new algorithms can be compared to existing ones using realistic bid sets (Leyton-Brown et al., 2000). It generates these bids by using several sample scenarios. In one scenario there is a graph where the nodes represent cities and the edges are railroad tracks that connect these cities. The items for sale are the tracks between cities, that is, the edges. The agents are given pairs of host/destination cities and are told that they need to establish a train route between their city pairs. Thus, each agent determines all the possible sets of edges which connect his city pairs and submits XOR combinatorial bids on them. The value of each path depends on the total distance; shorter routes are preferred.

CATS

### 7.3.2 Distributed Winner Determination

One problem with the centralized winner determination algorithms, aside from the bottleneck, is that they require the use of a trusted auctioneer who will perform the needed computations. Another option is to build a peer-to-peer combinatorial auction protocol which lets the sellers themselves determine the winning set of bids and discourages them from cheating.

#### Incremental Auctions: Distribute over Bidders

One way to distribute the winner determination calculation is by offloading it on the bidding agents. We can do this by using an increasing price auction and making the bidders figure out which bids would be better than the current standing bid. This is the approach taken by the Progressive Adaptive User Selection Environment or **PAUSE** combinatorial auction (Kelly and Stenberg, 2000) (Cramton et al., 2006, Chapter 6).

A PAUSE auction for $m$ items has $m$ stages. Stage 1 consists of having simultaneous ascending price open-cry auctions for each individual item. During this stage the bidders can only place individual bids on items. At the end of this state we will know what is the highest bid for each individual item and who placed that bid. In each successive stage $k = 2, 3, \ldots, m$ we hold an ascending price auction where the bidders must submit sets of bids that cover all items but each one of the bids must be for $k$ items or less. The bidders are allowed to use bids that other agents have placed in previous rounds. Also, any new bid set has to have a sum of bid prices which is bigger than the currently winning bid set.

At the end of each stage $k$ all agents know the best bid for every subset of size $k$ or less. Also, at any point in time after stage 1 has ended there is a standing bid set whose value increases monotonically as new bid sets are submitted. Since in the final round all agents consider all possible bid sets, we know that the final winning bid set will be one such that no agent can propose a better bidset. Note, however, that this bid set will generally not be $X^*$ since we are using ascending price auction so the winning bid will be only slightly bigger than the second highest bid for the particular set of items.

PAUSE



- Na árvore ramificada em itens, coloque os itens com mais lances primeiro na árvore. Dessa forma, os itens mais restritos são testados primeiro, criando assim menos folhas.
- Se a subárvore de pesquisa restante for a mesma para nós diferentes na árvore de pesquisa, como pode acontecer quando itens diferentes são limpos, mas por lances diferentes, essas subárvores poderão ser armazenadas em cache. A subárvore é resolvida uma vez e a resposta, ou seja, o melhor conjunto de lances nela encontrado, é salva para possível uso futuro.

Em geral, a melhor velocidade alcançada pelos melhores algoritmos varia muito dependendo do tipo de propostas apresentadas. Por exemplo, se o número de itens em cada oferta for escolhido a partir de uma distribuição de probabilidade plana, poderemos resolver problemas com milhares de itens e dezenas de milhares de ofertas em segundos. Por outro lado, se cada lance contiver exatamente cinco itens escolhidos aleatoriamente e um preço de 1, então só poderemos resolver problemas com dezenas de itens e centenas de lances em um minuto. O Combinatorial Auction Test Suite (CATS) pode gerar tipos realistas de distribuição de lances.

CATS

contribuições para que novos algoritmos possam ser comparados aos existentes usando conjuntos de lances realistas (Leyton-Brown et al., 2000). Ele gera esses lances usando vários cenários de amostra. Em um cenário existe um grafo onde os nós representam cidades e as arestas são trilhos de trem que conectam essas cidades. Os itens à venda são as trilhas entre as cidades, ou seja, as bordas. Os agentes recebem pares de cidades anfitriãs/destino e são informados de que precisam estabelecer uma rota de trem entre seus pares de cidades. Assim, cada agente determina todos os possíveis conjuntos de arestas que conectam seus pares de cidades e submete XOR ofertas combinatórias sobre eles. O valor de cada caminho depende da distância total; rotas mais curtas são preferidas.

### 7.3.2 Determinação do DistributedWinner

Um problema com os algoritmos centralizados de determinação do vencedor, além do gargalo, é que eles exigem o uso de um leiloeiro confiável que realizará os cálculos necessários. Outra opção é construir um protocolo de leilão combinatório ponto a po nto que permita aos próprios vendedores determinar o conjunto vencedor de lances e os desencoraje de trapacear.

#### Leilões incrementais: distribuir pelos licitantes

Uma forma de distribuir o cálculo da determinação do vencedor é descarregá-lo nos agentes licitantes. Podemos fazer isso usando um leilão de preços crescentes e fazendo com que os licitantes descubram quais lances seriam melhores do que o lance atual. Esta é a abordagem adotada pelo Progressive Adaptive User Selection Environment ou leilão combinatório PAUSE (Kelly e Stenberg, 2000) (Cramton et al., 2006,

Capítulo 6).

PAUSE

Um PAUSE leilão de $m$ itens tem $m$ estágios. A Fase 1 consiste em realizar leilões simultâneos de preços ascendentes para cada item individual. Durante esta fase os licitantes só poderão fazer licitações individuais para itens. No final deste estado saberemos qual é o lance mais alto para cada item individual e quem fez esse lance. Em cada estágio sucessivo $k = 2, 3, \ldots, m$ realizamos um leilão de preço ascendente em que os licitantes devem enviar conjuntos de lances que cubram todos os itens, mas cada um dos lances deve ser para $k$ ite ns ou menos. Os licitantes podem usar lances que outros agentes tenham feito em rodadas anteriores. Além disso, qualquer novo conjunto de licitações deve ter uma soma de preços de licitação maior do que o conjunto de licitações atualmente vencedor.

No final de cada etapa, $k$ todos os agentes sabem o melhor lance para cada subconjunto de tamanho$k$ ou menor. Além disso, a qualquer momento após o término da fase 1, há um conjunto de propostas permanente cujo valor aumenta monotonicamente à medida que novos conjuntos de propostas são enviados. Como na rodada final todos os agentes consideram todos os conjuntos de lances possíveis, sabemos que o conjunto de lances vencedor final será tal que nenhum agente poderá propor um conjunto de lances melhor. Observe, no entanto, que esse conjunto de lances geralmente não será $X^*$, já que estamos usando um leilão de preço ascendente, portanto o lance vencedor será apenas um pouco maior que o segundo lance mais alto para o conjunto específico de itens.

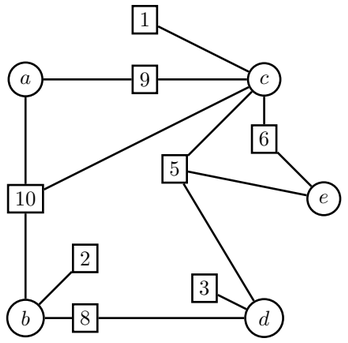



**Figure 7.8:** Graphical representation of a distributed winner determination problem. The circles represent agents/items while the squares represent combinatorial bids.

ENVY-FREE

In the general case, the PAUSE auction has been shown to be **envy-free** in that at the conclusion of the auction no bidder would prefer to exchange his allocation with that of any other bidder. However, it is not guaranteed to find the utilitarian solution (7.1).

The PAUSE auction makes the job of the auctioneer very easy. All it has to do is make sure each new bidset adds up to a number that is bigger than the current best as well as make sure that any bids an agent places that are not his do indeed correspond to other agents' bids. The computational problem shits from one of winner determination by the auctioneer to one of bid generation by the prospective buyer agents. Each agent must search over the space of all bid sets which contain at least one of its bids. The search is made easier by the fact that the agent need only consider the current best bids and that in stage $k$ all bid sets must contain at least one bid of size $k$ since they would have otherwise been bid in a previous stage. The **pausebid** algorithm uses the same branch and bound techniques used in centralized winner determination but expands them to include the added constraints an agent faces. As such, the pausebid algorithm can be used to find the myopically optimal bid for agents in the PAUSE auction (Mendoza and Vidal, 2007). The research also reveals that agents using pausebid reach the same allocation as the utilitarian solution, assuming all the agents bid their true valuations, at least 95% of the time.

PAUSEBID

Another way of distributing the winner determination problem among the bidders is provided by the Virtual Simultaneous Auction (**VSA**) (Fujishima et al., 1999) which is based on market-oriented programming ideas (Wellman, 1996). The VSA is an iterative algorithm where successive auctions for the items are held and the bidders change their bids based on the last auction's outcome. The auction is guaranteed to find the optimal solution when the bidding terminates. Unfortunately, there is no guarantee that bidding will terminate and experimental results show that in most cases bidding appears to go on forever.

VSA

### Distribute over Sellers

Another way to distribute the problem of winner determination is to distribute the actual search for the winning bid set among the sellers. Imagine a distributed system where each person that wants to sell an item runs an agent. The agent advertises that the item is for sale. Every person who wants to place a, possibly combinatorial, bid does so by telling all the agents present in the bid about it. After some time the agents have gathered some bids and begin the clearing process. The set of agents and bids can be visualized in a graph such as Figure 7.8.

Here we see that agent $b$ has received three bids. One of them is a singleton bid worth $2, the other two are combinatorial bids one of them for $8 and including agent $d$ and the other for $10 and including agents $a$ and $c$. The problem we face is how can the agents $a$–$e$ determine the set of winning bids.

The simplest solution is to do a complete search via sequentialized ordering. In



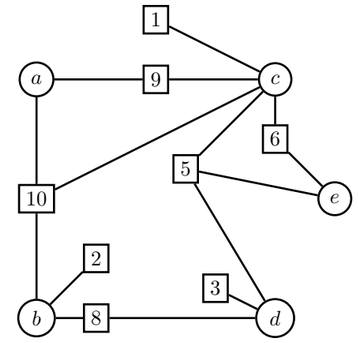



Figura 7.8: Representação gráfica de um problema distribuído de determinação de vencedor. Os círculos representam agentes/itens enquanto os quadrados representam lances co mbinatórios.

ENVY-FREE

No caso geral, o PAUSE leilão demonstrou ser isento de inveja porque na conclusão do leilão, nenhum licitante preferiria trocar a sua atribuição com a de qualquer outro licitante. Contudo, não é garantido encontrar a solução utilitária (7.1).

O PAUSE leilão facilita muito o trabalho do leiloeiro. Tudo o que tem a fazer é garantir que cada novo conjunto de lances totalize um número maior do que o melhor atual, bem como garantir que quaisquer lances que um agente faça que não sejam os dele corres pondam de fato aos lances de outros agentes. O problema computacional vai desde a determinação do vencedor pelo leiloeiro até o da geração de lances pelos potenciais agentes compradores. Cada agente deve pesquisar no espaço de todos os conjuntos de lances que contenham pelo menos um de seus lances. A busca é facilitada pelo fato de que o agente precisa considerar apenas os melhores lances atuais e que no estágio $k$ todos os conjuntos de lances devem conter pelo menos um lance de tamanho $k$ já que, de outra forma, teriam sido ofertados em um estágio anterior. O algoritmo pausebid usa as mesmas técnicas de ramificação e vinculação usadas em centralizado

PAUSEBID

determinação do vencedor, mas as expande para incluir as restrições adicionais que um agente enfrenta. Dessa forma, o algoritmo pausebid pode ser usado para encontrar o lance míope ideal para os agentes no leilão PAUSE (Mendoza e Vidal, 2007). A pesquisa também revela que os agentes que utilizam o pausebid alcançam a mesma alocação que a solução utilitária, assumindo que todos os agentes oferecem suas verdadeiras avaliações, pelo menos 95% das vezes.

Outra forma de distribuição do problema de determinação do vencedor entre os licitantes é fornecida pelo Leilão Virtual Simultâneo (VSA) (Fujishima et al.,

VSA

1999) que se baseia em ideias de programação orientada para o mercado (Wellman, 1996). O VSA é um algoritmo iterativo onde leilões sucessivos para os itens são realizados e os l icitantes alteram seus lances com base no resultado do último leilão. O leilão tem a garantia de encontrar a solução ótima quando a licitação terminar. Infelizmente, não há garantia de que a licitação terminará e os resultados experimentais mostram que, na maioria dos casos, a licitação parece durar para sempre.

#### Distribuir entre vendedores

Outra forma de distribuir o problema da determinação do vencedor é distribuir a busca real pelo lance vencedor definido entre os vendedores. Imagine um sistema distribuído onde ca da pessoa que deseja vender um item dirige um agente. O agente anuncia que o item está à venda. Cada pessoa que deseja fazer uma oferta, possivelmente combinatória, o faz informando a todos os agentes presentes na oferta. Depois de algum tempo os agentes reúnem algumas propostas e iniciam o processo de compensação. O conjunto de agentes e ofertas pode ser visualizado em um gráfico como a Figura 7.8.

Aqui vemos que o agente $b$ recebeu três lances. Um deles é um lance único no valor de US$ 2, os outros dois são lances combinatórios, um deles de US$ 8 e incluindo o agente $d$ e o outro de US$ 10 e incluindo os agentes $a$ e $c$. O problema que enfrentamos é como os agentes podem $a$–$e$ determinar o conjunto de lances vencedores.

A solução mais simples é fazer uma busca completa por meio de ordenação sequencial. Em

this method we use the same search tree as before but instead of implementing it centrally we pass messages among the agents. Agent 1 handles the top level of the tree. It tentatively sets one of its bids as cleared and sends a message to agent 2. In this way, each agent (roughly) handles one level of the tree. Note the agents are sequentialized so there is no parallelism, even though it is distributed.

Another option is to partition the problem into smaller pieces then have sets of agents to a complete search via sequentialized ordering on each of the parts. That is, we first partition the set of agents then do a complete search on each subset. This method means that each subset works in parallel. However, if there is any bid that contains agents from more than one subset then the solution found is no longer guaranteed to be optimal.

Another option is to maximize the available parallelism by having the agents do **individual hill-climbing**. Each agent starts by ordering all its bids based on their price divided by the number of agents in the bid under the assumption that the agent gets $1/n$ of a bid that includes $n$ agents. The agent picks the first bid in the list (the highest valued) and tells all the other agents in the bid that it wants to clear it. If the agent receives similar messages from all the agents in the bid this means that they all wanted to clear it so the bid is considered cleared and all the agents in it are removed from the protocol. The remaining agents repeat the same process with their next highest bid and so on until they run out of bids. This algorithm is guaranteed to terminate but will only find, at best, a local optima.

INDIVIDUAL HILL-CLIMBING

### Winner Determination as Constraint Optimization

It is interesting to note that we can reduce the winner determination problem to a constraint optimization problem as described in Chapter 2.2 in two different ways. One way is to let the variables $x_1, \ldots, x_m$ be the items to be sold and their domains be the set of bids which include the particular item. That is, each item $k$ is represented by a variable $x_k$ with domain $D_k$ which contains all the bids that involve $k$. The constraints are given by the bids. Every bid is replaced with a constraint which returns the value of the bid if all the items in the bid have a value equal to that bid. That is, if all the items are cleared for that bid/constraint then that constraint returns its value, otherwise the constraints returns a value of zero. In this problem we now want to maximize the value returned by the constraints.

We can also reduce the winner determination problem by letting the variables be the bids themselves with binary domains which indicate whether the bid has been cleared or not. We then need two sets of constraints. One set of constraints has to be generated such that they give a very large penalty if any two bids with items in common have been cleared, so as to make sure that never happens. The other set simply gives a value for every cleared bid equal to the value of that bid.

Since both of these are standard constraint optimization problems we can use the algorithms from Chapter 2.2 to solve them in a distributed manner. However, as those algorithms are generic, it seems likely that they will not perform as well as the specialized algorithms.

## 7.3.3 Bidding Languages

We have thus far assumed that each buyer can submit a set of bids, where each bid $b$ specifies that he is willing to pay $b^{\text{value}}$ for the set of items $b^{\text{items}}$. Implicit in the set of bids is the fact that the agent is also willing to accept winning two or more of his bids. That is, if $b$ and $b'$ are two bids for non-overlapping sets of items then any agent that places them should also be happy to win both bids. This bidding language is known as **or bids**, because agents can place multiple **atomic bids** and they can win any feasible combination of the bids they submit. That is, the agent expresses his valuation as $b_1$ OR $b_2$ OR... OR $b_k$.

A limitation of OR bids is that they cannot represent sub-additive valuations. For example, a sub-additive valuation arises if you have a value of 10 for a red hat and 10 for a blue hat but together value them at 11 because you really only need

OR BIDS
ATOMIC BIDS



neste método usamos a mesma árvore de busca de antes, mas em vez de implementá-la centralmente, passamos mensagens entre os agentes. O Agente 1 lida com o nível superior da árvore. Ele provisoriamente define um de seus lances como liberado e envia uma mensagem ao agente 2. Dessa forma, cada agente (aproximadamente) lida com um nível da árvore. Observe que os agentes são sequenciais para que não haja paralelismo, mesmo sendo distribuído.

Outra opção é particionar o problema em partes menores e então ter conjuntos de agentes para uma busca completa por meio de ordenação sequencial em cada uma das partes. Ou seja, primeiro particionamos o conjunto de agentes e depois fazemos uma busca completa em cada subconjunto. Este método significa que cada subconjunto funciona em paralelo. Entretanto, se houver alguma oferta que contenha agentes de mais de um subconjunto, a solução encontrada não será mais garantida como ótima.

Outra opção é maximizar o paralelismo disponível fazendo com que os agentes façam escaladas individuais. Cada agente começa ordenando todos os seus lances com base em seu preço dividido pelo número de agentes na oferta sob a suposição de que o agente recebe $1/n$ de uma oferta que inclui $n$ agentes. O agente escolhe a primeira oferta da lista (a de maior valor) e informa a todos os outros agentes da oferta que deseja limpá-la. Se o agente receber mensagens semelhantes de todos os agentes da oferta, isso significa que todos queriam liberá-la, portanto a oferta é considerada limpa e todos os agentes nela contidos são removidos do protocolo. Os demais agentes repetem o mesmo processo com o próximo lance mais alto e assim por diante até ficarem sem lances. É garantido que este algoritmo termine, mas apenas encontrará, na melhor das hipóteses, um ótimo local.

INDIVIDUAL HILL-CLIMBING

### Determinação do vencedor como otimização de restrições

É interessante notar que podemos reduzir o problema de determinação do vencedor a um problema de otimização de restrições, conforme descrito no Capítulo 2.2, de duas maneiras diferentes. Uma maneira é deixar as variáveis $x_1, \ldots, x_m$ são os itens a serem vendidos e seus domínios são o conjunto de lances que incluem o item específico. Ou seja, cada item $k$ é representado por uma variável $x_k$ com domínio $D_k$ que contém todos os lances que envolvem $k$. As restrições são dadas pelas propostas. Cada lance é substituído por uma restrição que retorna o valor do lance se todos os itens do lance tiverem um valor igual a esse lance. Ou seja, se todos os itens forem compensados para essa oferta/restrição, essa restrição retornará seu valor, caso contrário, as restrições retornarão um valor zero. Neste problema queremos agora maximizar o valor retornado pelas restrições.

Também podemos reduzir o problema de determinação do vencedor, deixando as variáveis serem as próprias propostas com domínios binários que indicam se a oferta foi compensada ou não. Precisamos então de dois conjuntos de restrições. Um conjunto de restrições deve ser gerado de tal forma que dê uma penalidade muito grande se quaisquer duas propostas com itens em comum forem compensadas, de modo a garantir que isso nunca aconteça. O outro conjunto simplesmente fornece um valor para cada oferta compensada igual ao valor dessa oferta.

Como ambos são problemas padrão de otimização de restrições, podemos usar os algoritmos do Capítulo 2.2 para resolvê-los de maneira distribuída. No entanto, como esses algoritmos são genéricos, parece provável que não tenham um desempenho tão bom quanto os algoritmos especializados.

### 7.3.3 Idiomas de Licitação

Até agora, assumimos que cada comprador pode enviar um conjunto de lances, onde cada lance $b$ especifica que ele está disposto a pagar $b^{\text{value}}$ pelo conjunto de itens $b^{\text{items}}$. Implícito no conjunto de lances está o fato de o agente também estar disposto a aceitar vencer dois ou mais de seus lances. Ou seja, se $b$ e $b'$ são dois lances para conjuntos de itens não sobrepostos, então qualquer agente que os fizer também ficará feliz em ganhar ambos os lances. Esta linguagem de licitação é conhecida como ou lances, porque os agentes podem fazer vários lances atômicos e eles podem ganhar qualquer combinação viável das propostas que apresentarem. Ou seja, o agente expressa sua avaliação como $b_1$ OR $b_2$ OR... OR $b_k$.

Uma limitação dos lances OR é que eles não podem representar avaliações subaditivas. Por exemplo, uma avaliação subaditiva surge se você tiver um valor de 10 para um chapéu vermelho e 10 para um chapéu azul, mas juntos os avaliarem como 11 porque você realmente só precisa

OR BIDS
ATOMIC BIDS

one hat. In this scenario if you placed the individual bids as OR bids it could be that you end up paying 10 for both hats. We thus say that OR bids are not a complete bidding language since they cannot represent all possible valuations.

XOR BIDS

**Xor bids**, on the other hand, can represent all possible valuations. An XOR bid takes the form of a series of atomic bids joined together by exclusive-or operations: $b_1$ XOR $b_2$ XOR... XOR $b_k$. This bid represents the fact that the agent is willing to win any *one* of the bids, but not more than one. Thus, you can use it to place a bid that says you are willing to pay 10 for a red hat or 10 for a blue hat or 11 for both but want to buy at most one of them.

One problem with XOR bids is that they can get very long for seemingly common valuations that can be more succinctly expressed using the OR bids. For example, say an agent has a purely additive valuation function over a set of items, that is if $s = s' \cup s''$ then $v(s) = v(s') + v(s'')$. This agent could have expressed this valuation by placing an OR bid where each atomic bid was just for one item. Implicit in this bid is the fact that the agent would be willing to accept any subset of the items as long as he gets paid the sum of the individual valuation. If the same agent had to place an XOR bid it would have to place an atomic bid for every subset of items, and there are $2^{|s|}$ such subsets.

OR* BIDS

Another practical problem with XOR bids is that most of the winner determination algorithms are designed to work with OR bids. The problem can be solved by adding dummy items to OR bids, these bids are known as **or* bids**. For example, if an agent wants to place a bid for item $a$ and $b$, but not both, it could generate a dummy item $d$ and place an OR bid for items $\{a, d\}$ and $\{b, d\}$. This way the agent guarantees that it will not win both $a$ and $b$. OR* combines the completeness of XOR bids with the succinctness of OR bids without adding too many new dummy items. In fact, any bid that can be expressed by OR or XOR using $x$ atomic bids can be represent using an OR* bids with at most $x^2$ dummy items (Nisan, 2000). Thus, all the winner determination algorithms we have studied can also be used with XOR bids as long as you first convert them to OR* bids.

### 7.3.4 Preference Elicitation

FREE DISPOSAL

We can also try to reduce the amount of information the bidders must supply by trying to only ask them about those valuations that are important in finding the utilitarian solution (Hudson and Sandholm, 2004). This can best be achieved in the common case of **free disposal** where there is no cost associated with the disposal of an item, that is, if $S \subseteq T$ then $v(S) \leq v(T)$. For example, if we know that an agent values item $a$ at 10 then we know that it must also value the set $(a, b)$ at *least* at 10. Similarly, if the agent values items $(a, b)$ at 5 then we know that its value for item $a$ is at *most* 5.

Given free disposal, an auctioneer can incrementally explore a network like the one in Figure 7.9 which shows all the possible subsets of items and associates with each one a upper bound (UB) and a lower bound (LB) on the valuation that the agent has for that particular set of items. The directed edges indicate which sets are preferred over other ones. For example, the agent will always prefer the set $\{a, b, c\}$ over the set $\{a, b\}$ and even, transitively, over the set $\{a\}$. The graph also makes it easy to see how the auctioneer can propagate the bounds he learns on one set to the others. Namely, the lower bounds can be propagated upstream and the upper bounds can be propagated downstream. For example, in the figure there is a lower bound of 5 the set $\{b\}$, knowing this the auctioneer can immediately set the lower bounds of $\{b, c\}$ and $\{a, b, c\}$ to 5. Similarly, the upper bound of 9 for the set $\{a, c\}$ can be propagated down to $\{a\}$, $\{c\}$, and $\emptyset$. Note also that if the agent tells the auctioneer its exact valuation for a particular set then the auctioneer will set both the upper and lower bounds of that set to the given value.

The goal of an elicitation auctioneer is to minimize the amount of questions that it asks the bidders while still finding the best allocation. If we limit the auctioneer to only asking questions of the type *"What is your $n^{th}$ most preferred set?"* then



um chapéu. Nesse cenário, se você fizer lances individuais como OR lances, poderá acabar pagando 10 por ambos os chapéus. Dizemos, portanto, que OR bids não são uma linguagem de licitação completa, uma vez que não podem representar todas as avaliações possíveis.

XOR BIDS

Os lances Xor, por outro lado, podem representar todas as avaliações possíveis. Um lance XOR assume a forma de uma série de lances atômicos unidos por operações ou exclusivas: $b_1$ XOR $b_2$ XOR... XOR $b_k$. Esta licitação representa o fato de que o agente está disposto a vencer qualquer uma das licitações, mas não mais do que uma. Assim, você pode usá-lo para fazer um lance que diga que você está disposto a pagar 10 por um chapéu vermelho ou 10 por um chapéu azul ou 11 por ambos, mas deseja comprar no máximo um deles.

Um problema com os lances XORé que eles podem ficar muito longos para avaliações aparentemente comuns que podem ser expressas de forma mais sucinta usando os lances OR . Por exemplo, digamos que um agente tenha uma função de avaliação puramente aditiva sobre um conjunto de itens, ou seja, se$s = s' \cup s''$então $v(s) = v(s') + v(s'')$. Este agente poderia ter expressado essa avaliação fazendo um OR lance em que cada lance atômico fosse apenas para um item. Implícito nesta proposta está o fato de que o agente estaria disposto a aceitar qualquer subconjunto de itens, desde que recebesse a soma da avaliação individual. Se o mesmo agente tivesse que fazer um XOR lance, ele teria que fazer um lance atômico para cada subconjunto de itens, e existem $2^{|s|}$ esses subconjuntos.

OR* BIDS

Outro problema prático com lances XOR é que a maioria dos algoritmos de determinação de vencedores são projetados para funcionar com lances OR . O problema pode ser resolvido adicionando itens fictícios aos OR lances. Esses lances são conhecidos como ou* lances. Por exemplo,

se um agente quiser fazer um lance para o item $a$ e $b$, mas não para ambos, ele poderá gerar um item fictício $d$ e fazer um lance OR para os itens $\{a, d\}$ e $\{b, d\}$. Desta forma o agente garante que não vencerá $a$ e $b$. OR* combina a integridade dos XORlances com a sucinta dos OR lances sem adicionar muitos novos itens fictícios. Na verdade, qualquer lance que possa ser expresso por OR ou XOR usando $x$ lances atômicos pode ser representado usando um OR* lances com no máximo $x^2$ itens fictícios (Nisan, 2000). Assim, todos os algoritmos de determinação de vencedores que estudamos também podem ser usados com XORlances, desde que você primeiro os converta em OR* lances.

### 7.3.4 Elicitação de Preferências

FREE DISPOSAL

Também podemos tentar reduzir a quantidade de informação que os licitantes devem fornecer, tentando perguntar-lhes apenas sobre as avaliações que são importantes para encontrar a solução utilitária (Hudson e Sandholm, 2004). Isto pode ser melhor conseguido no caso comum de eliminação gratuita, onde não há custos associados à eliminação.

de um item, ou seja, se $S \subseteq T$ então $v(S) \leq v(T)$. Por exemplo, se sabemos que um agente avalia o item $a$ como 10, então sabemos que ele também deve avaliar o conjunto $(a, b)$ pelo menos como 10. Da mesma forma, se o agente avalia os itens $(a, b)$ como 5, então sabemos que seu valor para o item $a$ é no máximo 5.

Dada a livre disposição, um leiloeiro pode explorar incrementalmente uma rede como a da Figura 7.9, que mostra todos os possíveis subconjuntos de itens e associa a cada um deles um limite superior (UB) e um limite inferior (LB) na avaliação que o agente tem para esse conjunto específico de itens. As arestas direcionadas indicam quais conjuntos são preferidos em relação a outros. Por exemplo, o agente sempre preferirá o conjunto$\{a, b, c\}$ ao conjunto $\{a, b\}$ e até, transitivamente, ao conjunto $\{a\}$. O gráfico também facilita ver como o leiloeiro pode propagar os limites que aprende em um conjunto para os outros. Ou seja, os limites inferiores podem ser propagados a montante e os limites superiores podem ser propagados a jusante. Por exemplo, na figura há um limite inferior de 5 para o conjunto $\{b\}$, sabendo disso o leiloeiro pode definir imediatamente os limites inferiores de $\{b, c\}$ e $\{a, b, c\}$ para 5. Da mesma forma, o limite superior de 9 para o conjunto$\{a, c\}$ pode ser propagado para $\{a\}$, $\{c\}$ e $\emptyset$. Observe também que se o agente informar ao leiloeiro sua avaliação exata para um determinado conjunto, o leiloeiro definirá os limites superior e inferior desse conjunto para o valor determinado.

O objetivo de um leiloeiro de licitação é minimizar a quantidade de perguntas que ele faz aos licitantes e, ao mesmo tempo, encontrar a melhor alocação. Se limitarmos o leiloeiro a fazer apenas perguntas do tipo "Qual é o seu $n^{th}$ conjunto preferido?" então

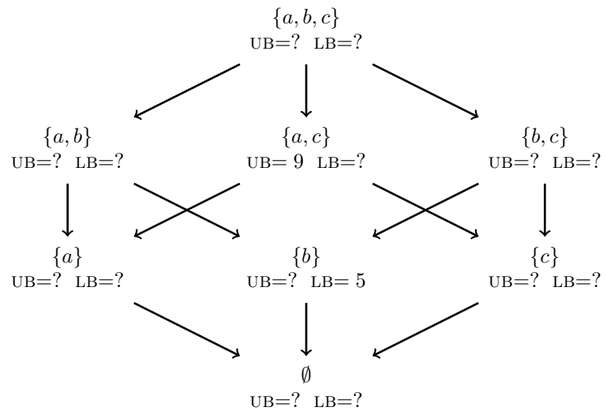



**Figure 7.9:** Constraint network for determining an agent's valuation. Directed edges indicate preferred sets.

PAR()
1 $fringe \leftarrow \{\{1,\ldots,1\}\}$
2 **while** $fringe \neq \emptyset$
3 **do** $c \leftarrow$ first($fringe$)
4 $fringe \leftarrow$ rest($fringe$)
5 $successors \leftarrow$ CHILDREN($c$)
6 **if** FEASIBLE($c$)
7 **then** $pareto\text{-}solutions \leftarrow pareto\text{-}solutions \cup c$
8 **else for** $n \in successors$
9 **do if** $n \notin fringe \wedge$UN-DOMINATED($n, pareto\text{-}solutions$)
10 **then** $fringe \leftarrow fringe \cup n$

CHILDREN($\{k_1,\ldots,k_n\}$)
1 **for** $i \in 1\ldots n$
2 **do** $c \leftarrow \{k_1,\ldots,k_n\}$
3 $c[i] \leftarrow c[i]+1$
4 $result \leftarrow result \cup c$
5 **return** $result$

**Figure 7.10:** The PAR algorithm. The procedure FEASIBLE($\{k_1,\ldots,k_n\}$) asks each bidder $i$ for its $k_i$ most valued set, if we haven't asked before. It uses these sets to determine if, together, they form a feasible allocation. The CHILDREN procedure returns a set of possible solutions, by adding 1 to each position in $\{k_1,\ldots,k_n\}$.

we will be unable to find the revenue maximizing allocation. However, we can still find the Pareto optimal allocations.

The **PAR algorithm**, shown in figure 7.10, allows an elicitation auctioneer to find a Pareto optimal solution by only using rank questions (Conen and Sandholm, 2002). It does this by incrementally building a solution lattice for the bidders. Figure 7.11 shows an example of a complete lattice for two bidders. The PAR algorithm maintains a set variable, called the *fringe*, of possible Pareto optimal allocations. At the first time step the auctioneer adds the solution {1,1} to the *fringe*, where {1,1} represents the solution formed by using both agents' most preferred solution. In every succeeding step the auctioneer picks some solution from the *fringe* and asks the agents for those sets, if it does not already know them. This communication with the bidders occurs within the FEASIBLE procedure. In the example figure both agents prefer the set $\{a,b\}$ the most so they will both respond with this set. Since the set of bids ($\{a,b\},\{a,b\}$) is not feasible the algorithm checks to make sure that each one of the CHILDREN of {1,1}, in this case {1,2} and {2,1}, is not dominated by any other allocation in the set *pareto-solutions* and adds it to the *fringe* if its not already there. The algorithm continues in this way until the *fringe* is empty, at that point the variable *pareto-solutions* contains all the Pareto allocations for the

PAR ALGORITHM



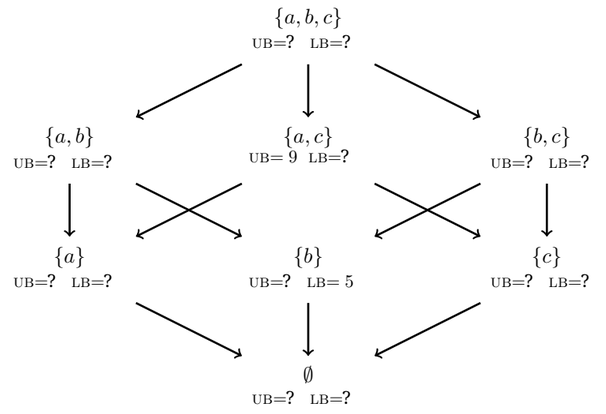



Figura 7.9: Rede de restrições para determinar a avaliação de um agente. As arestas direcionadas indicam conjuntos preferenciais.

PAR()
*1 franja* $\leftarrow \{\{1,\ldots,1\}$*2 enquanto franja* $\neq \emptyset$*3 faça* $c \leftarrow$ *primeiro(franja) 4 franja* $\leftarrow$ *descanso(franja) 5 sucessores* $\leftarrow$ CHILDREN*(c) 6 se* FEASIBLE*(c) 7 então soluções de pareto* $\leftarrow$ *soluções de pareto* $\cup c$*8 senão para* $n \in$ *sucessores 9 faça se* $n \notin$ *franja* $\wedge$UN-DOMINATED*(n,pareto-solutions) 10 então franja* $\leftarrow$ *franja* $\cup n$

CHILDREN($\{k_1,\ldots,k_n\}$)
*1 para* $i \in 1\ldots n$*2 faz*
$c \leftarrow \{k_1,\ldots,k_n\}$
3 $c[i] \leftarrow c[i]+1$*4*
*resultado* $\leftarrow$ *resultado* $\cup c$*5*
*returnresult*

Figura 7.10: O algoritmo PAR. O procedimento FEASIBLE($\{k_1,\ldots,k_n\}$) solicita a cada licitante $i$ seu $k_i$ conjunto mais valioso, caso ainda não t enhamos perguntado. Utiliza estes conjuntos para determinar se, juntos, formam uma alocação viável. O procedimentoCHILDREN retorna um conjunto de soluções possíveis, adicionando 1 a cada posição em $\{k_1,\ldots,k_n\}$.

não conseguiremos encontrar a alocação que maximiza a receita. No entanto, ainda podemos encontrar as alocações ótimas de Pareto.

O algoritmo PAR, mostrado na figura 7.10, permite que um leiloeiro de elicitação encontre uma solução ótima de Pareto usando apenas questões de classificação (Conen e Sandholm, 2002). Isso é feito através da construção incremental de uma rede de soluções para os licitantes. A Figura 7.11 mostra um exemplo de rede completa para dois licitantes. O algoritmo PAR mantém uma variável definida, chamada franja, de possíveis alocações ótimas de Pareto. Na primeira etapa, o leiloeiro adiciona a solução {1,1} à franja, onde {1,1}representa a solução formada usando a solução preferida de ambos os agentes. Em cada passo seguinte, o leiloeiro escolhe alguma solução da margem e pede esses conjuntos aos agentes, caso ainda não os conheça. Essa comunicação com os licitantes ocorre dentro do procedimento FEASIBLE . Na figura de exemplo, ambos os agentes preferem o conjunto $\{a,b\}$ mais, então ambos responderão com este conjunto. Como o conjunto de lances ($\{a,b\},\{a,b\}$) não é viável, o algoritmo verifica se cada um dos CHILDREN de {1,1}, neste caso {1,2} e {2,1}, não é dominado por nenhuma outra alocação no conjunto de soluções de pareto e adiciona-o à margem se ainda não estiver lá. O algoritmo continua desta forma até que a franja esteja vazia, nesse ponto a variável soluções de Pareto contém todas as alocações de Pareto para o

PAR ALGORITHM

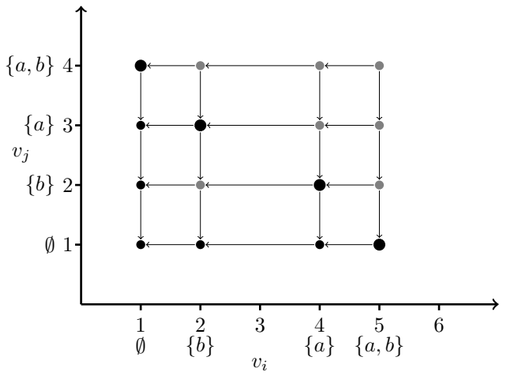



**Figure 7.11:** Rank lattice. The dots represent all possible allocations. Directed edges represent Pareto dominance. Grey dots are infeasible while black dots are feasible allocations. The PAR search starts at the top rightmost point and stops when it has identified the complete Pareto frontier of feasible allocations—the larger black dots.

EBF()

```
1  fringe ← {{1, ..., 1}}
2  if |fringe| = 1
3      then c ← first(fringe)
4      else M ← {k ∈ fringe | v(k) = max_{d∈fringe} v(d)}
5           if |M| ≥ 1 ∧ ∃_{d∈M} FEASIBLE(d)
6              then return d
7              else c ← PARETO-SOLUTION(M)
8  if FEASIBLE(c)
9      then return c
10 successors ← CHILDREN(c)
11 for n ∈ successors
12       do if n ∉ successors
13            then fringe ← fringe ∪ {n}
14 goto 2
```

**Figure 7.12:** The EBF algorithm. FEASIBLE($d$) returns true if $d$ is a feasible allocation. PARETO-SOLUTION($M$) returns one allocation from $M$ which is not Pareto-dominated by any other allocation in $M$.

problem.

Since the PAR algorithm does not ask the agents for their valuation values it cannot determine which one of the Pareto allocations is the utilitarian allocation. Of course, once we start asking for valuations we have a better idea of which bids will likely be part of the utilitarian allocation, namely those sets that have the highest value per item.

EFFICIENT BEST FIRST

The **efficient best first** (EBF) algorithm performs a search similar to the one that PAR implements but it also asks for the values of the sets and always expands the allocation in the *fringe* which has the highest value (Conen and Sandholm, 2002). Figure 7.12 shows the algorithm. This algorithm will find the utilitarian allocation.

Unfortunately, both PAR and EBF have worst case running times that are exponential in the number of items. They also perform rather poorly in practice. PAR does not make any effort at trying to pick a item set to ask about, it simply chooses randomly, so its bad performance is not surprising. EBF's elicitation strategy has also been shown to perform poorly in experiments—it asks too many questions. Its performance also degrades as more agents are added.

A more general framework for elicitation is to maintain a set of allocations which are potentially optimal. Initially, this set would contain all allocations in the general case. In cases where we can assume some structure for the value function, such as free disposal, then it contains all those allocations that are not dominated by another. The elicitation algorithm can then choose one allocation from this set and ask the agents their values for the sets they receive in that allocation. These



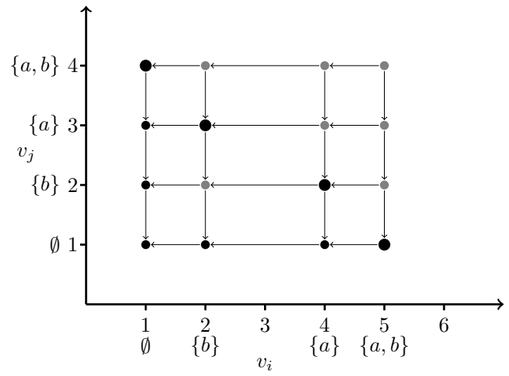



Figura 7.11: Rede de classificação. Os pontos representam todas as alocações possíveis. As bordas direcionadas representam a dominância de Pareto. Os pontos cinzentos são inviáveis, enquanto os pontos pretos são alocações viáveis. A busca PAR começa no ponto superior direito e termina quando é identificada a fronteira Paret completa de alocações viáveis – os pontos pretos maiores.

EBF*() 1 franja* ← {{1, ..., 1}}*2 se* |*franja* | = 1*3 então* $c \leftarrow$ *primeiro(franja) 4 senão* $M \leftarrow \{k \in$ *franja* $| v(k) = \max_{d \in fringe} v(d)\}$*5 se* $|M| \geq 1 \wedge \exists_{d \in M}$FEASIBLE*(d) 6 então retorne* $d$ *7 senão* $c \leftarrow$ PARETO-SOLUTION*(M) 8 if* FEASIBLE*(c) 9 thenreturn* $c$*10 sucessores* $\leftarrow$ CHILDREN*(c) 11 para* $n \in$ *sucessores 12 faça se* $n \notin$*sucessores 13 então franja* $\leftarrow$ *franja* $\cup \{n\}$*14 vá para 2*

Figura 7.12: O algoritmo EBF. FEASIBLE($d$) retorna verdadeiro se $d$ for uma alocação viável. PARETO-SOLUTION($M$) retorna uma alocação de $M$que não é dominada por Pareto por nenhuma outra alocação em $M$.

problema.

Como o PAR algoritmo não solicita aos agentes seus valores de avaliação, ele não pode determinar qual das alocações de Pareto é a alocação utilitária. É claro que, assim que começarmos a pedir avaliações, teremos uma ideia melhor de quais as propostas que provavelmente farão parte da alocação utilitária, nomeadamente os conjuntos que têm o valor mais elevado por item.

EFFICIENT BEST FIRST

O algoritmo eficiente melhor primeiro (EBF) realiza uma pesquisa semelhante à que PAR implementa, mas também solicita os valores dos conjuntos e sempre expande a alocação na franja que possui o maior valor (ConenandSandholm, 2002). A Figura 7.12 mostra o algoritmo. Este algoritmo encontrará a alocação utilitária.

Infelizmente, tanto PAR quanto EBF têm tempos de execução de pior caso que são exponenciais no número de itens. Eles também têm um desempenho bastante ruim na prática. PARnão faz nenhum esforço ao tentar escolher um conjunto de itens para perguntar, simplesmente escolhe aleatoriamente, portanto seu mau desempenho não é surpreendente. A estratégia de elicitação de EBF também demonstrou ter um desempenho insatisfatório em experimentos – ela faz muitas perguntas. Seu desempenho também diminui à medida que mais agentes são adicionados.

Uma estrutura mais geral para a elicitação é manter um conjunto de alocações que seja m potencialmente ótimas. Inicialmente, este conjunto conteria todas as alocações no caso geral. Nos casos em que podemos assumir alguma estrutura para a função de valor, como a livre disposição, então ela contém todas as alocações que não são dominadas por outra. O algoritmo de elicitação pode então escolher uma alocação desse conjunto e perguntar aos agentes seus valores para os conjuntos que recebem nessa alocação. Esses

values can then be propagated to other sets and a new allocation chosen (Conen and Sandholm, 2001b; Conen and Sandholm, 2001a).

Within this general framework there are various elicitation strategies we could try. The simplest one is to randomly choose an allocation from the set. This technique has been shown to require a number of elicitations that is proportional to $n2^m$ where $n$ is the number of agents and $m$ is the number of items. Another strategy is to choose the candidate with the highest lower bound value on the expectation that a candidate with a higher value is more likely to be the optimal choice, or at least will need to be eliminated from competition. Experiments have shown that this strategy performs better than random elicitations.

### 7.3.5 VCG Payments

VCG payments, which we will see in Chapter 8.2, can be applied to combinatorial auctions (MacKie-Mason and Varian, 1994). In a VCG combinatorial auction the bid set with maximum payment is chosen as the winner but the bidders do not have to pay the amounts they bid. Instead, each bidder pays the difference in the total value that the *other* bidders would have received if he had not bid (and the best set of bids was chosen from their bids) minus the total value the other bidders receive given his bid. Each bidder thus get a payment that is proportional to his contribution to everyone else's valuation. This has the desirable effect of aligning the bidders' incentives with the utilitarian allocation and thus eliminating the incentive to lie about their true valuation. However, it increases the computational requirements as we now have to also solve a winner determination problem for every subset of $n-1$ agents in order to calculate the payments. That is, VCG payments increase the work by a factor of $n$, where $n$ is the number of agents.

### Exercises

7.1 The branch and bound algorithm for the branch on bids search tree, seen in figure 7.6, does not specify in which order the bids should be searched. Give a item heuristic for this ordering and explain why it should, on average, help find a solution quicker than expanding bids in a random order.

7.2 What is the set of winning bids given the following bids?

| Price | Bid items |
|---|---|
| $1 | Beast Boy |
| $3 | Robin |
| $5 | Raven, Starfire |
| $6 | Cyborg, Robin |
| $4 | Cyborg, Beast Boy |
| $3 | Raven, Beast Boy |

7.3 Show how we can reduce the problem of winner determination in a combinatorial auction to a constraint optimization problem.

7.4 Can the problem of winner determination in a combinatorial auction be reduced to a constraint satisfaction problem? Show your proof.

7.5 You are given a painting to sell at an auction and wish to maximize its sale price. What type of auction should you use? Explain.

7.6 In Chapter 6.5 we mentioned the postman problem. How can the postmen use a combinatorial auction to solve their problem? Explain how the bids are to be generated.

7.7 Why is a branch on bids search faster than a branch on items search for winner determination in combinatorial auctions?



os valores podem então ser propagados para outros conjuntos e uma nova alocação escolhida (Conen e Sandholm, 2001b; ConenandSandholm, 2001a).

Dentro deste quadro geral, existem várias estratégias de elicitação que poderíamos tentar. A mais simples é escolher aleatoriamente uma alocação do conjunto. Foi demonstrado que esta técnica requer um número de elicitações que é proporcional a $n2^m$ onde $n$ é o número de agentes e $m$ é o número de itens. Outra estratégia é escolher o candidato com o valor do limite inferior mais alto, na expectativa de que um candidato com um valor mais alto tenha maior probabilidade de ser a escolha ideal ou, pelo menos, precise ser eliminado da competição. Experimentos mostraram que esta estratégia tem melhor desempenho do que elicitações aleatórias.

### 7.3.5 Pagamentos VCG

VCG pagamentos, que veremos no Capítulo 8.2, podem ser aplicados a leilões combinatórios (MacKie-Mason e Varian, 1994). Num leilão combinatório VCG, o lance definido com pagamento máximo é escolhido como vencedor, mas os licitantes não precisam pagar os valores que licitaram. Em vez disso, cada licitante paga a diferença no valor total que os outros licitantes teriam recebido se ele não tivesse licitado (e o melhor conjunto de propostas foi escolhido a partir das suas propostas) menos o valor total que os outros licitantes receberiam dada a sua proposta. Cada licitante recebe assim um pagamento que é proporcional à sua contribuição para a avaliação de todos os outros. Isto tem o efeito desejável de alinhar os incentivos dos licitantes com a alocação utilitária e, assim, eliminar o incentivo para mentir sobre a sua verdadeira avaliação. No entanto, isso aumenta os requisitos computacionais, pois agora temos que resolver também um problema de determinação de vencedor para cada subconjunto de $n-1$ agentes para calcular os pagamentos. Ou seja, os pagamentos VCG aumentam o trabalho por um fator de $n$, onde $n$ é o número de agentes.

### Exercícios

7.1 O algoritmo de ramificação e limite para a árvore de busca de licitações, visto na figura 7.6, não especifica em que ordem as licitações devem ser pesquisadas. Forneça uma heurística de item para essa ordenação e explique por que ela deveria, em média, ajudar a encontrar uma solução mais rapidamente do que expandir os lances em ordem aleatória.

7.2 Qual é o conjunto de propostas vencedoras dadas as seguintes propostas?

| Preço | Itens de l ance |
|---|---|
| $1 | Mutano |
| $3 | Robin |
| $5 | Corvo , Estelar |
| $6 | Ciborgue, Robin |
| $4 | Ciborgue, Mutano |
| $3 | Ravena, Mutano |

7.3 Mostre como podemos reduzir o problema de determinação do vencedor em um leilão combinatório a um problema de otimização de restrições.

7.4 O problema da determinação do vencedor num leilão combinatório pode ser reduzido a um problema de satisfação de restrições? Mostre sua prova.

7.5 Você recebeu uma pintura para vender em leilão e deseja maximizar seu preço de venda. Que tipo de leilão você deve usar? Explicar.

7.6 No Capítulo 6.5 mencionamos o problema do carteiro. Como os carteiros podem usar um leilão combinatório para resolver o seu problema? Explique como os lances serão gerados.

7.7 Por que uma ramificação na busca de lances é mais rápida do que uma ramificação na busca de itens para determinação do vencedor em leilões combinatórios?

7.8 In a combinatorial auction with 50 items and using a computer that takes 1 millisecond to explore each bidset, what is an upper bound on the amount of time it would take to find a solution to the winner determination problem?

7.9 The winner determination problem in combinatorial auction seeks to maximize revenue, that is, maximize the amount paid by the buyers. Provide three reasons or situations in which this solution might not be the most desirable on.



7.8 Em um leilão combinatório com 50 itens e usando um computador que leva 1 milissegundo para explorar cada conjunto de lances, qual é o limite superior para o tempo que levaria para encontrar uma solução para o problema de determinação do vencedor?

7.9 O problema de determinação do vencedor em leilões combinatórios busca maximizar a receita, ou seja, maximizar o valor pago pelos compradores. Forneça tr ês razões ou situações em que esta solução pode não ser a mais desejável.

Chapter 8

# Voting and Mechanism Design

Once you have a multiagent system composed of autonomous locally-aware agents, you will often desire a way to aggregate their knowledge for making a system-wide decision. That is, you will want to ask them to vote on some issue. Unfortunately, there are many different voting mechanisms, each one of which might lead to different answers. More importantly, it could be that selfish agents do not want to vote their true preferences. They might prefer to lie, in the same way you might not vote for your favorite political candidate if you think that she does not have any hope of winning and, instead, you vote for your second most favorite candidate. In this chapter we examine the problems with voting and mechanism design.

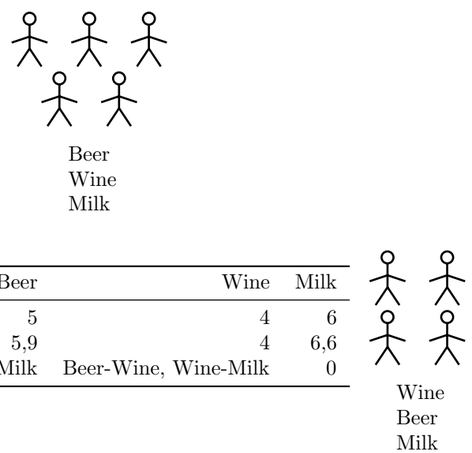
## 8.1 The Voting Problem

At first glance, the voting problem seems very simple. We ask all agents to proclaim their preferences over a set of candidates and then tally these votes to find the most preferred candidate. The problem comes if we want this result to match, in some way, the agents' preferences. It turns out that the common voting mechanisms all fail to aggregate the voters' preferences in some cases.

For example, figure 8.1 shows 15 mathematicians who are planning to throw a party. They must first decide which beverage the math department will serve at this party. There are three choices available to them: beer, wine, and milk. As the figure shows, 6 mathematicians prefer milk over wine and wine over beer, 5 prefer beer over wine and wine over milk and 4 prefer wine over beer and beer over milk. We then need to determine which drink they will choose.

One option is to have a **plurality vote** where each one votes for their favorite drink, the votes are tallied and the drink with the most votes is the winner. Under this voting scheme beer would get 5 votes, wine 4, and milk 6. Therefore, the






| | Beer | Wine | Milk |
|---|---|---|---|
| Plurality | 5 | 4 | 6 |
| Runoff | 5,9 | 4 | 6,6 |
| Pairwise | Beer-Milk | Beer-Wine, Wine-Milk | 0 |

**Figure 8.1:** Fifteen mathematicians trying to decide whether they should buy beer, wine, or milk.



Capítulo 8

# Votação e Design de Mecanismo

Depois de ter um sistema multiagente composto por agentes autônomos com reconhecimento local, muitas vezes você desejará uma maneira de agregar seu conhecimento para tomar uma decisão em todo o sistema. Ou seja, você vai querer pedir que votem em algum assunto. Infelizmente, existem muitos mecanismos de votação diferentes, cada um dos quais pode levar a respostas diferentes. Mais importante ainda, pode acontecer que os agentes egoístas não queiram votar nas suas verdadeiras preferências. Eles podem preferir mentir, da mesma forma que você pode não votar na sua candidata política favorita se achar que ela não tem qualquer esperança de vencer e, em vez disso, votar no seu segundo candidato favorito. Neste capítulo examinamos os problemas com a votação e o design dos mecanismos.

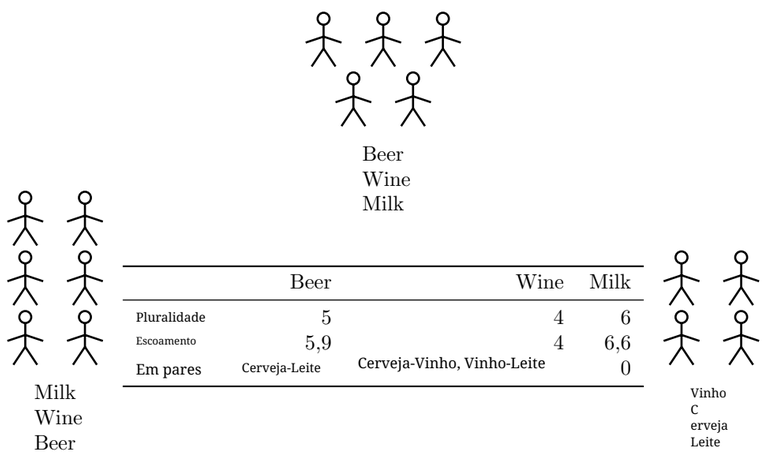
## 8.1 O problema da votação

À primeira vista, o problema da votação parece muito simples. Pedimos aos agentes que declarem suas preferências sobre um conjunto de candidatos e então contem esses votos para encontrar o candidato preferido. O problema surge se quisermos que este resultado corresponda, de alguma forma, às preferências dos agentes. Acontece que todos os mecanismos de votação comuns não conseguem agregar as preferências dos eleitores em alguns casos.

Por exemplo, a figura 8.1 mostra 15 matemáticos que estão planejando dar uma festa. Eles devem primeiro decidir qual bebida o departamento de matemática servirá nesta festa. Existem três opções disponíveis para eles: cerveja, vinho e leite. Como mostra a figura, 6 matemáticos preferem leite a vinho e vinho a cerveja, 5 preferem cerveja a vinho e vinho a leite e 4 preferem vinho a cerveja e cerveja a leite. Precisa mos então determinar qual bebida eles escolherão.

Uma opção é ter uma votação plural onde cada um vota no seu favorito bebida, os votos são contabilizados e a bebida com mais votos é a vencedora. Sob este esquema de votação, a cerveja receberia 5 votos, o vinho 4 e o leite 6. Portanto, o






| | Beer | Wine | Milk |
|---|---|---|---|
| Pluralidade | 5 | 4 | 6 |
| Escoamento | 5,9 | 4 | 6,6 |
| Em pares | Cerveja-Leite | Cerveja-Vinho, Vinho-Leite | 0 |

Figura 8.1: Quinze matemáticos tentando decidir se deveriam comprar cerveja, vinho ou leite.

mathematicians should clearly choose milk as the drink for the party.

RUNOFF

Another option is to have a **runoff** election (primaries) then pick the two winners and have another election with only those two (these technique can be easily extended to any number of runoff elections). Under this scheme the first election would lead to the same votes as before so the second election would consist of beer and milk. With only these two candidates beer would get 9 votes while milk would get 6 votes. Therefore, the mathematicians should clearly choose beer as the drink for the party.

PAIRWISE

Yet another option is to hold three elections each one with only two candidates (that is, implement all **pairwise** elections) and the candidate that wins the most elections is the chosen one. Under this scheme we would see that if beer and wine are paired then wine wins, if beer and milk are paired then beer wins, and if wine and milk are paired then wine wins. Wine wins two elections so it is the chosen one. Therefore, the mathematicians should clearly choose wine as the drink for the party. After realizing the complexity of this problem the mathematicians wisely decide to give up and have everyone bring their own drink.

### 8.1.1 Possible Solutions

SYMMETRY

We, on the other hand, will not give up that easily. We want to clearly define the best solution. Specifically, we want a fair solution. But, it is not clear what fairness means in this situation. One way to approach the fairness problem is to require **symmetry**. There are two different types of symmetry that we can identify in this problem.

- *Reflectional symmetry*: If one agent prefers A to B and another one prefers B to A then their votes should cancel each other out.
- *Rotational symmetry*: If one agent's preferences are A,B,C and another one's are B,C,A and a third one prefers C,A,B then their votes should cancel out.

If we look back at the three types of schemes presented in the previous section we notice that the plurality votes violates reflectional symmetry since, in the example from figure 8.1, there are 8 agents that prefer beer over milk and wine over milk, but only 6 that have the opposite preferences, and yet milk wins. Similarly, since the runoff election is just a series of plurality votes it also violates reflectional symmetry. It has also been shown that pairwise comparison violates rotational symmetry. In the example we can take one agent from each of the three groups and these three agents cancel each other's votes, so we can eliminate them. We can do this four times and end up with two agents with preferences milk, wine, beer and one agent with preference of beer, wine, milk. A plurality vote over these would lead to milk as the winner while the pairwise vote led to wine being the winner.



Jean-Charles de Borda.
1733–1799.

BORDA COUNT

Thus, none of the previous voting schemes satisfy both forms of symmetry. But there exists one voting mechanism which does satisfy them. It is known as the **Borda count** and it works as follows:

1. With $x$ choices, each agent awards $x$ to points to his first choice, $x-1$ points to his second choice, and so on.
2. The candidate with the most points wins.

The Borda count satisfies both reflectional and rotational symmetry. It is most useful when there are many candidates and we want to choose the best one by taking into account all agents' knowledge, equally. With the Borda count we do not have to worry about a minority winning the election because the majority is divided among a small number of choices.

Now that you understand the intuitions behind the voting problem, we give a formal presentation of the problem.



os matemáticos deveriam escolher claramente o leite como bebida para a festa.

RUNOFF

Outra opção é realizar um segundo turno (primárias) e depois escolher os dois vencedores.

e realizar outra eleição apenas com esses dois (esta técnica pode ser facilmente estendida a qualquer número de eleições de segundo turno). Sob este esquema, a primeira eleição levaria aos mesmos votos de antes, de modo que a segunda eleição consistiria em cerveja e leite. Com apenas esses dois candidatos, a cerveja receberia 9 votos, enquanto o leite receberia 6 votos. Portanto, os matemáticos deveriam escolher claramente a cerveja como bebida da festa.

PAIRWISE

Ainda outra opção é realizar três eleições cada uma com apenas dois candidatos ( ou seja, implementar todas as eleições aos pares) e o candidato que ganhar mais eleições é o escolhido. Neste esquema, veríamos que se a cerveja e o vinho forem emparelhados, então o vinho vence; se a cerveja e o leite forem emparelhados, a cerveja vence, e se o vinho e o leite forem emparelhados, o vinho vence. Wine vence duas eleições então é o escolhido. Portanto, os matemáticos deveriam escolher claramente o vinho como bebida da festa. Depois de perceberem a complexidade deste problema, os matemáticos decidem sabiamente desistir e fazer com que cada um traga a sua própria bebida.

### 8.1.1 Possíveis Soluções

SYMMETRY

Nós, por outro lado, não desistiremos tão facilmente. Queremos definir claramente a melhor solução. Especificamente, queremos uma solução justa. Mas não está claro o que significa justiça nesta situação. Uma maneira de abordar o problema da justiça é exigir simetria. Existem dois tipos diferentes de simetria que podemos identificar neste problema.

- Simetria reflexiva: se um agente prefere A a Banda, outro prefere B a A, então s eus votos devem se anular.
- Simetria rotacional: se as preferências de um agente forem A,B,C e as de outro forem B,C,A e um terceiro preferir C,A,B, então seus votos deverão ser cancelados.

Se olharmos para trás, para os três tipos de esquemas apresentados na secção anterior, notamos que os votos por pluralidade violam a simetria reflexiva, uma vez que, no exemplo da figura 8.1, existem 8 agentes que preferem cerveja a leite e vinho a leite, mas apenas 6 que têm preferências opostas, e ainda assim o leite vence. Da mesma forma, uma vez que a segunda volta das eleições é apenas uma série de votações por pluralidade, também viola a simetria reflexiva. Também foi demonstrado que a comparação aos pares viola a simetria rotacional. No exemplo podemos pegar um agente de cada um dos três grupos e esses três agentes anulam os votos uns dos outros, para que possamos eliminá-los. Podemos fazer isso quatro vezes e acabar com dois agentes com preferências leite, vinho, cerveja e um agente com preferência cerveja, vinho, leite. Uma votação pluralista sobre estes levaria ao leite c omo vencedor, enquanto a votação em pares levaria ao vinho a ser o vencedor.



Jean-Charles de Borda.
1733–1799.

BORDA COUNT

Assim, nenhum dos esquemas de votação anteriores satisfaz ambas as formas de simetria. Mas existe um mecanismo de votação que os satisfaz. É conhecida como contagem de Borda e funciona da seguinte forma:

1. Com $x$ escolhas, cada agente concede $x$ pontos para sua primeira escolha, $x-1$ pontos para sua segunda escolha e assim por diante.
2. Vence o candidato que obtiver mais pontos.

A contagem de Borda satisfaz a simetria reflexiva e rotacional. É mais útil quando há muitos candidatos e queremos escolher o melhor tendo em conta o conhecimento de todos os agentes, igualmente. Com a contagem de Borda não temos de nos preocupar com a possibilidade de uma minoria ganhar as eleições porque a maioria está dividida entre um pequeno número de escolhas.

Agora que você entende as intuições por trás do problema da votação, faremos uma apresentação formal do problema.

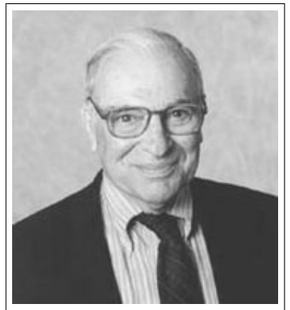
**Definition 8.1** (Voting Problem)**.** *We are given set of agents $A$ and a set of outcomes $O$. Each agent $i \in A$ has a preference function $>_i$ over the set of outcomes. Let $>^*$ be the global set of social preferences – what we think the final vote should reflect.*

Using this notation we can clearly specify the kind of $>^*$ that we would like. Namely, we are probably interested in a $> *$ that is efficient, can be calculated, and is fair to everyone. After thinking about it for some time, you would probably come up with a set of voting conditions similar to the following.

**Definition 8.2** (Desirable voting outcome conditions)**.** *A voting protocol is desirable if it obeys the following conditions:*

1. *$>^*$ exists for all possible inputs $>_i$,*
2. *$>^*$ exists for every pair of outcomes,*
3. *$>^*$ is asymmetric and transitive over the set of outcomes,*
4. *$>^*$ should be Pareto efficient. That is, if all agents prefer Beer over Milk then $>^*$ should also prefer Beer over Milk.*
5. *The scheme used to arrive at $>^*$ should be independent of irrelevant alternatives. That is, if in one world all agents prefer Beer to Milk and in another world all agents again prefer Beer to Milk then in both cases the rankings of Beer and Milk should be the same, regardless of how they feel about Wine.*
6. *No agent should be a dictator in the sense that $>^*$ is always the same as $>_i$, regardless of the other $>_j$.*

Unfortunately, Arrow showed that no voting mechanism exists which satisfies all these conditions.



Kenneth Joseph Arrow. 1921–. Nobel prize in Economics.

**Theorem 8.1** (Arrow's Impossibility)**.** *There is no social choice rule that satisfies the six conditions (Arrow, 1951).*

Specifically, we can show that plurality voting violates conditions 3 and 5 when there are three or more candidates. Similarly, since runoff elections are just several plurality votes they also violate conditions 3 and 5. Pairwise voting can violate condition 5 as does the Borda count. We can show that the Borda count violates condition 5 with a simple example. Say there are seven agents whose preferences over choices $a, b, c, d$ are as follows:

1. $a > b > c > d$
2. $b > c > d > a$
3. $c > d > a > b$
4. $a > b > c > d$
5. $b > c > d > a$
6. $c > d > a > b$
7. $a > b > c > d$

If we applied the Borda count to these agents we would find that $c$ gets 20 points, $b$ gets 19, $a$ gets 18, and $d$ 13. As such, $c$ wins and $d$ comes out last. If we then eliminate $d$ we would then have the following preferences.

1. $a > b > c$
2. $b > c > a$

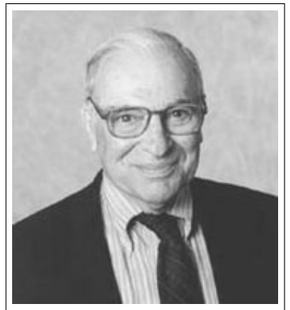


*Definição 8.1 (Problema de Votação). Recebemos um conjunto de agentes $A$ e um conjunto de resultados $O$. Cada agente $i \in A$ tem uma função de preferência $>_i$ sobre o conjunto de resultados. Seja $>^*$ o conjunto global de preferências sociais – o que achamos que a votação final deveria refletir.*

Usando esta notação podemos especificar claramente o tipo de $>^*$ que gostaríamos. Ou seja, provavelmente estamos interessados em um $> *$ que seja eficiente, possa ser calculado e seja justo para todos. Depois de pensar sobre isso por algum tempo, você provavelmente chegaria a um conjunto de condições de votação semelhantes às seguintes.

*Definição 8.2 (Condições desejáveis para o resultado da votação). Um protocolo de votação é desejável se obedecer às seguintes condições:*

1. *$>^*$ existe para todas as entradas possíveis $>_i$,*
2. *$>^*$ existe para cada par de resultados,*
3. *$>^*$ é assimétrico e transitivo em relação ao conjunto de resultados,*
4. *$>^*$ deve ser eficiente de Pareto. Ou seja, se todos os agentes preferirem Cerveja a Leite, então$>^*$ também deverão preferir Cerveja a Leite.*
5. *O esquema usado para chegar a $>^*$ deve ser independente de alternativas irrelevantes. Ou seja, se num mundo todos os agentes preferem a Cerveja ao Leite e noutro mundo todos os agentes voltam a preferir a Cerveja ao Leite, então em ambos os casos as classificações da Cerveja e do Leite deveriam ser as mesmas, independentemente da forma como se sentem em relação ao Vinho. 6. Nenhum agente deve ser ditador no sentido de que $>^*$ é sempre igual a $>_i$, independentemente do outro $>_j$.*



Kenneth Joseph Arrow. 1921–. Prêmio Nobel de Economia.

Infelizmente, Arrow mostrou que não existe nenhum mecanismo de votação que satisfaça todas estas condições.

*Teorema 8.1 (Impossibilidade da Flecha). Não existe uma regra de escolha social que satisfaça as seis condições (Arrow, 1951).*

Especificamente, podemos mostrar que a votação por pluralidade viola as condições 3 e 5 quando há três ou mais candidatos. Da mesma forma, como as eleições de segundo turno são apenas várias votações por pluralidade, elas também violam as condições 3 e 5. A votação e m pares pode violar a condição 5, assim como a contagem de Borda. Podemos mostrar que a contagem de Borda viola a condição 5 com um exemplo simples. Digamos que existam sete agentes cujas preferências sobre as escolhas $a, b, c, d$ são as seguintes:

1. $a > b > c > d$
2. $b > c > d > a$
3. $c > d > a > b$
4. $a > b > c > d$
5. $b > c > d > a$
6. $c > d > a > b$
7. $a > b > c > d$

Se aplicássemos a contagem de Borda a esses agentes, descobriríamos que $c$ obtém 20 pontos,$b$ obtém 19, $a$ obtém 18 e $d$ 13. Sendo assim, $c$ vence e $d$ sai por último. Se eliminarmos $d$ teremos as seguintes preferências.

1. $a > b > c$
2. $b > c > a$

**Table 8.1:** List of individual desires for painting the house.

| Name | Wants house painted? |
|---|---|
| Alice | Yes |
| Bob | No |
| Caroline | Yes |
| Donald | Yes |
| Emily | Yes |

3. $c > a > b$
4. $a > b > c$
5. $b > c > a$
6. $c > a > b$
7. $a > b > c$

If we ran Borda on this scenario we would find that $a$ gets 15 votes, $b$ gets 14, and $c$ gets 13, so $a$ wins. So, originally $c$ wins and $d$ comes out last but by eliminating $d$ we then get that $a$ wins! This is a violation of condition 5.

### 8.1.2 Voting Summary

In practice we find voting mechanism often used in multiagent systems but without much thought given to which one might be best. Thus, plurality vote is most often used. In general, the Borda count should be the preferred voting mechanism as it can effectively aggregate multiple disparate opinions, but it does have some drawbacks. Namely, the Borda count requires the agents to generate a complete preference ordering over all items, which could be computationally expensive. For example, if each choice is an allocation of packages that the agent must deliver then the agent must solve a traveling salesman problem for each choice in order to determine how much it would cost him to deliver all those packages. One could try to reduce these costs by implementing a limited version of the Borda count where instead of voting for all choices the agents limit themselves to voting for only their best $k$ options, for some small number $k$.

## 8.2 Mechanism Design

Mechanism design asks how we can provide the proper incentives to agents so that we can aggregate their preferences correctly. The mechanism design problem has been studied in Economics for some time. It is and interesting to us because it maps very well to open multiagent systems with selfish agents. In this chapter we present the standard mechanism design problem as studied in Economics as well as the distributed mechanism design extension which is an even better model for many multiagent system design problems.

### 8.2.1 Problem Description

Alice lives in a house with four other housemates. They are thinking about paying someone to paint the exterior of their house and have decided to hold a vote where everyone will vote either Yes, if they want the house painted, or No if they don't. The votes will be public and the set of people who vote for painting will share equally in the cost of the painters, as long as two or more people vote Yes. The people who voted against painting will pay nothing. We note that, since the paint covers the whole outside of the house everyone will be able to enjoy the new cleaner house. Each person knows whether or not they want the house to be painted. Their desires are shown in Table 8.1.



Tabela 8.1: Lista de desejos individuais para pintar a casa.

| Name | Quer casa pintada? |
|---|---|
| Alice | Yes |
| Bob | No |
| Carolina | Yes |
| Donald | Yes |
| Emilly | Yes |

3. $c > a > b$
4. $a > b > c$
5. $b > c > a$
6. $c > a > b$
7. $a > b > c$

Se executarmos Borda nesse cenário, descobriremos que $a$ obtém 15 votos, $b$obté m 14 e$c$ obtém 13, então $a$ vence. Então, originalmente $c$ ganha e $d$ sai por último, mas ao eliminar$d$ obtemos essas $a$ vitórias! Isto é uma violação da condição5.

#### 8.1.2 Resumo da votação

Na prática, encontramos mecanismos de votação frequentemente usados em sistemas multiagentes, mas sem pensar muito sobre qual seria o melhor. Assim, o voto por pluralidade é mais frequentemente usado. Em geral, a contagem de Borda pode ser o mecanismo de votação preferido, pois pode efetivamente agregar múltiplas opiniões díspares, mas tem algumas desvantagens. Ou seja, a contagem de Borda exige que os agentes gerem uma ordem de preferência completa sobre todos os itens, o que pode ser computacionalmente caro. Por exemplo, se cada escolha é uma alocação de pacotes que o agente deve entregar, então o agente deve resolver o problema do caixeiro viajante para cada escolha, a fim de determinar quanto lhe custaria entregar todos esses pacotes. Poderíamos tentar reduzir esses custos implementando uma versão limitada da contagem de Borda onde, em vez de votar em todas as escolhas, os agentes se li mitam a votar apenas nas suas melhores $k$ opções, para algum pequeno número $k$.

## 8.2 Projeto do Mecanismo

O desenho do mecanismo pergunta como podemos fornecer os incentivos adequados aos agentes para que possamos agregar corretamente as suas preferências. O problema de projeto de mecanismos tem sido estudado em Economia há algum tempo. É interessante para nós porque mapeia muito bem sistemas multiagentes abertos com agentes egoístas. Neste capítulo apresentamos o problema de projeto de mecanismo padrão estudado em Economia, bem como a extensão de projeto de mecanismo distribuído, que é um modelo ainda melhor para muitos problemas de projeto de sistemas multiagentes.

### 8.2.1 Descrição do Problema

Alice mora em uma casa com outras quatro colegas. Eles estão pensando em pagar alguém para pintar o exterior de sua casa e decidiram realizar uma votação onde todos votarão Sim, se quiserem que a casa seja pintada, ou Não, se não quiserem. As votações serão públicas e o conjunto de pessoas que votarem pela pintura dividirão igualmente o custo dos pintores, desde que duas ou mais pessoas votem Sim. As pessoas que votara m contra a pintura não pagarão nada. Ressaltamos que, como a tinta cobre toda a parte externa da casa, todos poderão aproveitar a nova casa mais limpa. Cada pessoa sabe se quer ou não que a casa seja pintada. Os seus desejos são mostrados na Tabela 8.1.

| $i$ | $\theta_i$ | $v_i(\text{Paint}, \theta_i)$ | $v_i(\text{NoPaint}, \theta_i)$ |
|---|---|---|---|
| Alice | WantPaint | 10 | 0 |
| Bob | DontWantPaint | 0 | 0 |
| Caroline | WantPaint | 10 | 0 |
| Donald | WantPaint | 10 | 0 |
| Emily | WantPaint | 10 | 0 |

**Table 8.2:** Values for the house painting problem where $O = \{\text{Paint}, \text{NoPaint}\}$ and the agents are either of type WantPaint or type DontWantPaint.

Alice wants the house painted, but lets assume that she does not want to pay for it. She realizes that if only two people voted yes that the house will be painted. As such, she has an incentive to vote against painting – that is, lie about her true preferences – in the hope that some other two agents will vote for it and the house will get painted anyway. This means that Alice's strategy will be to try to determine what the others are likely to vote and see if there are enough Yes votes so that she can safely lie. Unfortunately, all that scheming is very inefficient from a system's perspective. It would be much easier if everyone wanted to tell the truth.

We would like to create a protocol where these types of incentives do not exist. That is, we would like for all agents to want to vote truthfully rather than lie or try to find out how the other agents are going to vote. If we could do that then the agents would not waste their resources trying to beat the system and instead use them to work for the system.

Mechanism design (Mas-Colell et al., 1995, Chapter 23) studies how private information can be elicited from individuals. It tells us how to build the proper incentives into our protocols such that the agents will want to tell the truth about their preferences. It also tells us about some circumstances when this is impossible.

More formally, we define a **mechanism design** problem as consisting of a set of agents with the following properties.

MECHANISM DESIGN

- Each agent $i$ has a **type** $\theta_i \in \Theta_i$ which is private. That is, only the agent knows its type, no one else does.

TYPE

- We let $\theta = \{\theta_1, \theta_2, \ldots, \theta_A\}$ be the set of types.
- The **mechanism** $g$ we are to implement will map from the set of agents' actions to a particular **outcome** $o \in O$.

MECHANISM
OUTCOME

- Each agent $i$ receives a value $v_i(o, \theta_i)$ for outcome $o$.
- The **social choice function** $f : \theta \to O$ tells us the outcome we want to achieve.

SOCIAL CHOICE FUNCTION

For example, the social choice function

$$f(\theta) = \arg\max_{o \in O} \sum_{i=1}^{n} v_i(o, \theta_i) \tag{8.1}$$

is the social welfare solution. It tries to maximize the sum of everyone's utility. You can, however, choose to implement a different social choice function. Other popular choices include minimizing the difference in the agents' utility, maximizing the utility of the agent that receives the highest utility, and the **paretian** social choice function $f$ such that for all $\theta$ there is no $o' \neq o = f(\theta)$ such that some agent $i$ gets a higher utility from $o'$ than it would have received under $f(\theta)$. That is, in the paretian social choice function $f$ there does not exist an outcome $o'$ such that there is some agent $i$ for which $v_i(o', \theta_i) > v_i(f(\theta), \theta_i)$ and for all $i$ $v_i(o', \theta_i) \geq v_i(f(\theta), \theta_i)$.

PARETIAN

Note also that since the agent's type are usually fixed – an agent cannot change its true type, only lie about it – then the $v_i$ usually only needs to be defined for the agent's particular $\theta_i$.

If we apply this notation to the example from Table 8.1 we get the values shown in Table 8.2. We have that $\Theta = \{WantPaint, DontNeedPaint\}$ since there are



| $i$ | $\theta_i$ | $v_i$(pintar $\theta_i$) | $v_i$(Sem pintura $\theta_i$) |
|---|---|---|---|
| Alice | QuerPaint | 10 | 0 |
| Bob | Não quero pintar | 0 | 0 |
| Carolina | QuerPaint | 10 | 0 |
| Donald | QuerPaint | 10 | 0 |
| Emilly | QuerPaint | 10 | 0 |

Tabela 8.2: Valores para o problema de pintura de casa onde$O = \{$Paint,NoPaint$\}$e os agentes são do tipo WantPaint ou do tipo DontWantPaint.

Alice quer a casa pintada, mas vamos supor que ela não queira pagar por isso. Ela percebe que se apenas duas pessoas votassem sim a casa seria pintada. Como tal, ela tem um incentivo para votar contra a pintura – isto é, mentir sobre as suas verdadeiras preferências – na esperança de que outros dois agentes votem a favor e a casa seja pintada de qualquer maneira. Isto significa que a estratégia de Alice será tentar determinar o que os outros provavelmente votarão e ver se há votos Sim suficientes para que ela possa mentir com segurança. Infelizmente, todo esse esquema é muito ineficiente do ponto de vista do sistema. Seria muito mais fácil se todos quisessem dizer a verdade.

Gostaríamos de criar um protocolo onde não existissem estes tipos de incentivos. Ou seja, gostaríamos que todos os agentes votassem com sinceridade, em vez de mentir ou tentar descobrir como os outros agentes irão votar. Se pudéssemos fazer isso, os agentes não desperdiçariam seus recursos tentando vencer o sistema e, em vez disso, os usariam para trabalhar para o sistema.

O projeto do mecanismo (Mas-Colell et al., 1995, Capítulo 23) estuda como informações privadas podem ser extraídas dos indivíduos. Diz-nos como incorporar os incentivos adequados nos nossos protocolos, de modo a que os agentes queiram dizer a verdade sobre as suas preferências. Também nos fala sobre algumas circunstâncias em que isso é impossível.

Mais formalmente, definimos um problema de projeto de mecanismo como consistindo em um conjunto de agentes com as seguintes propriedades.

MECHANISM DESIGN

- Cada agente $i$ tem um tipo $\theta_i \in \Theta_i$ que é privado. Ou seja, apenas o agente conhece seu tipo, ninguém mais conhece.

TYPE

- Seremos $\theta = \{\theta_1, \theta_2, \ldots, \theta_A\}$ o conjunto de tipos.
- O mecanismo $g$ que devemos implementar será mapeado a partir do conjunto de agentes ações para um resultado específico $o \in O$.

MECHANISM
OUTCOME

- Cada agente $i$ recebe um valor $v_i(o, \theta_i)$ para o resultado $o$.
- A função de escolha social $f : \theta \to O$ nos diz o resultado que queremos alcançar.

SOCIAL CHOICE FUNCTION

Por exemplo, a função de escolha social

$$f(\theta) = \arg\max_{o \in O} \sum_{i=1}^{n} v_i(o, \theta_i) \tag{8.1}$$

é a solução de bem-estar social. Tenta maximizar a soma da utilidade de todos. Você pode, no entanto, optar por implementar uma função de escolha social diferente. Outras escolhas populares incluem minimizar a diferença na utilidade dos agentes, maximizar a utilidade do agente que recebe a maior utilidade e o equilíbrio social paretiano.

PARETIAN

função de escolha $f$ tal que para todo $\theta$ não há $o' \neq o = f(\theta)$ tal que algum agente $i$ obtenha uma utilidade maior de $o'$ do que teria recebido em $f(\theta)$. Ou seja, na função de escolha social paretiana $f$ não existe um resultado $o'$ tal que exista algum agente $i$ para o qual $v_i(o', \theta_i) > v_i(f(\theta) \ \theta_i)$ e para todos $i$ $v_i(o', \theta_i) \geq v_i(f(\theta) \ \theta_i)$.

Observe também que, como o tipo do agente geralmente é fixo – um agente não pode alterar seu tipo verdadeiro, apenas mentir sobre ele – então o $v_i$ geralmente só precisa ser definido para o $\theta_i$ específico do agente.

Se aplicarmos esta notação ao exemplo da Tabela 8.1, obteremos os valores mostrados na Tabela 8.2. Temos isso $\Theta = \{WantPaint, DontNeedPaint\}$ já que existem

only two types of agents: those that want the house painted and those that think it does not need paint. Also, $O = \{Paint, NoPaint\}$ since either the house gets painted or it doesn't. Notice that we had to add some arbitrary number for the agents' utilities for all possible actions. We decided that the agents that want the house painted would get a value of 10 from seeing it painted and 0 if it does not get painted while those who think the house is fine as it is get a value of 0 either way. Lets further assume that we want to maximize social welfare. That is, our social choice function is (8.1). Finally, we assume that the cost of painting the house is 20. We now face the problem of designing a protocol that will decide whether or not to paint the house and how to pay for it.

One possible way to solve this problem is to let all the agents vote Yes or No. We then count the votes and if a majority voted for painting the house then the house will be painted and the cost (20) will be divided evenly among the 5 agents. That is, each agent will have to pay 4 no matter what. This scheme works fine for all agents except for Bob who did not want the house painted and must now pay 4. We are imposing a tax on Bob for something he does not want. This might not be a desirable solution.

Another way is to let everyone vote Yes or No and then split the cost of painting the house among those who voted Yes, as we discussed earlier. This seems fairer but it has the problem that it gives all the agents, except Bob, and incentive to lie, as we explained before. They would want to lie in the hopes that someone else would vote Yes and spare them having to pay for it. In general, we are looking for a mechanism which implements the social choice function. This idea can be formalized as follows:

IMPLEMENTS

**Definition 8.3** ($g$ Implements $f$)**.** *A mechanism $g : S_1 \times \cdots \times S_A \to O$ **implements** social choice function $f(\cdot)$ if there is an equilibrium strategy profile $(s_1^*(\cdot), \ldots, s_A^*(\cdot))$ of the game induced by $g$ such that*

$$\forall_\theta \; g(s_1^*(\theta_1), \ldots, s_A^*(\theta_A)) = f(\theta_1, \ldots, \theta_A)$$

*where $s_i(\theta_i)$ is agent $i$'s strategy given that it is of type $\theta_i$.*

The definition might sound a little bit circular but it isn't. Say you start out with a set of agents each one with a type – which you don't know about – and you tell them that you are going to use $g(\cdot)$ to calculate the final outcome. That is, you tell them how the function $g$ works. The agents will use their knowledge of $g$ to determine their best action and will take that action. You then input this set of actions into $g$ to come up with the outcome. If the outcome is the same as $f(\theta_1, \ldots, \theta_A)$ then you just implemented $f$. As you can see, the tricky part is that you have to pick $g$ such that it will encourage the agents to take the appropriate actions.

Another point of confusion might be that we have changed from types to actions. In the previous examples the agents actions – their votes – where merely the revelation of their types. That is, there was a one-to-one mapping from types to actions. However, in general this need not be the case. We could, for example, have a system with 20 different types but only 2 possible actions.

We have also not defined what we mean by an "equilibrium strategy" as used in Definition 8.3. As you will remember from Chapter 3, there are many equilibrium concepts that can be applied to a game. The equilibrium we will concern ourselves with is the dominant strategy equilibrium. A player has a dominant strategy (action) if the agent prefers to use this strategy regardless of what anyone else will do. In our mechanism design problem we formally define a dominant strategy as follows:

DOMINANT STRATEGY EQUILIBRIUM

**Definition 8.4** (Dominant Strategy Equilibrium)**.** *We say that a strategy profile $(s_1^*(\cdot), \ldots, s_A^*(\cdot))$ of the game induced by $g$ is a **dominant strategy equilibrium** if for all $i$ and all $\theta_i$,*

$$v_i(g(s_i^*(\theta_i), s_{-i}), \theta_i) \geq v_i(g(s_i', s_{-i}), \theta_i)$$

*for all $s_i' \in S_i$ and all $s_{-i} \in S_{-i}$.*



apenas dois tipos de agentes: os que querem a casa pintada e os que acham que ela não precisa de pintura. Além disso, $O = \{Paint, NoPaint\}$ já que ou a casa é pintada ou não. Observe que tivemos que adicionar algum número arbitrário para os utilitários dos agentes para todas as ações possíveis. Decidimos que os agentes que desejam que a casa seja pintada receberão o valor 10 por vê-la pintada e 0 se ela não for pintada, enquanto aqueles que acham que a casa está bem do jeito que está recebem o valor 0 de qualquer maneira. Vamos supor ainda que queremos maximizar o bem-estar social. Ou seja, a nossa função de escolha social é (8.1). Finalmente, assumimos que o custo de pintar a casa é 20. Enfrentamos agora o problema de elaborar um protocolo que decidirá se pintaremos ou não a casa e como pagar por isso.

Uma forma possível de resolver este problema é deixar todos os agentes votarem Sim ou Não. Contamos então os votos e se a maioria votar pela pintura da casa então a casa será pintada e o custo (20) será dividido igualmente entre os 5 agentes. Ou seja, cada agente terá que pagar 4, aconteça o que acontecer. Este esquema funciona bem para todos os agentes, exceto para Bob, que não queria que a casa fosse pintada e agora deve pagar4. Estamos cobrando um imposto sobre Bob por algo que ele não deseja. Esta pode não ser uma solução desejável.

Outra forma é deixar todos votarem Sim ou Não e depois dividir o custo da pintura da casa entre aqueles que votaram Sim, como discutimos anteriormente. Isto parece mais justo, mas tem o problema de dar a todos os agentes, exceto Bob, um incentivo para mentir, como explicamos antes. Eles gostariam de mentir na esperança de que alguém votasse sim e os poupasse de ter que pagar por isso. Em geral, procuramos um mecanismo que implemente a função de escolha social. Essa ideia pode ser formalizada da seguinte forma:

IMPLEMENTS

**Definição 8.3 (*g* Implementos *f*). Um mecanismo** $g$**:** $S_1 \times \cdots \times S_A \to O$ **implementa** *função de escolha social $f(\cdot)$ se houver um perfil de estratégia de equilíbrio $(s_1^*(\cdot) \ldots, s_A^*(\cdot))$ do jogo induzido por $g$ tal que*

$$\forall_\theta \; g(s_1^*(\theta_1), \ldots, s_A^*(\theta_A)) = f(\theta_1, \ldots, \theta_A)$$

*onde $s_i(\theta_i)$ é a estratégia do agente $i$ dado que é do tipo $\theta_i$.*

A definição pode parecer um pouco circular, mas não é. Digamos que você comece com um conjunto de agentes, cada um com um tipo – que você não conhece – e diga a eles que usará $g(\cdot)$ para calcular o resultado final. Ou seja, você conta a eles como a função $g$ funciona. Os agentes usarão seu conhecimento de $g$ para determinar a melhor ação e executarão essa ação. Em seguida, você insere esse conjunto de ações em $g$ para chegar ao resultado. Se o resultado for igual a $f(\theta_1, \ldots, \theta_A)$ então você acabou de implementar $f$. Como você pode ver, a parte complicada é que você precisa escolher $g$ de forma que incentive os agentes a tomar as ações apropriadas.

Outro ponto de confusão pode ser que mudamos de tipos para ações. Nos exemplos anteriores as ações dos agentes – os seus votos – foram apenas a revelação dos seus tipos. Ou seja, houve um mapeamento um a um dos tipos para as ações. Contudo, em geral, isto não precisa ser o caso. Poderíamos, por exemplo, ter um sistema com 20 tipos diferentes mas apenas 2 ações possíveis.

Também não definimos o que entendemos por "estratégia de equilíbrio" conforme usada na Definição 8.3. Como você se lembrará do Capítulo 3, existem muitos conceitos de equilíbrio que podem ser aplicados a um jogo. O equilíbrio com o qual nos preocuparemos é o equilíbrio da estratégia dominante. Um jogador tem uma estratégia (ação) dominante se o agente preferir usar essa estratégia independentemente do que os outros farão. Em nosso problema de projeto de mecanismo, definimos formalmente uma estratégia dominante da seguinte forma:

DOMINANT STRATEGY EQUILIBRIUM

*Definição 8.4 (Equilíbrio da Estratégia Dominante). Dizemos que um perfil de estratégia $(s_1^*(\cdot) \ldots, s_A^*(\cdot))$ do jogo induzido por $g$ é um equilíbrio de estratégia dominante se para todos $i$ e todos $\theta_i$,*

$$v_i(g(s_i^*(\theta_i), s_{-i}), \theta_i) \geq v_i(g(s_i', s_{-i}), \theta_i)$$

*para todos $s_i' \in S_i$ e todos $s_{-i} \in S_{-i}$.*

We can now specialize Definition 8.3 for dominant equilibria.

**Definition 8.5** ($g$ Implements $f$ in Dominant Strategies)**.** *A mechanism $g : S_1 \times \cdots \times S_A \to O$* ***implements social choice function $f(\cdot)$ in dominant strategies*** *if there is a dominant strategy equilibrium strategy profile $(s_1^*(\cdot), \ldots, s_A^*(\cdot))$ of the game induced by $g$ such that $g(s_1^*(\theta_1), \ldots, s_A^*(\theta_A)) = f(\theta_1, \ldots, \theta_A)$ for all $\theta \in \Theta$.*

IMPLEMENTS SOCIAL CHOICE FUNCTION $f(\cdot)$ IN DOMINANT STRATEGIES

Before we go into how to find this magical $g$ lets explore some simplifications of the problem. The first simplification is one we made in our first example. Namely, that the agents' strategies correspond to the revelation of their types. That is, lets assume that the agents' actions are simply type revelations. The only thing an agent can do is say "I am of type $\theta_x$". Of course, he could be lying when he makes this statement. We call this a **direct revelation mechanism** because the agents directly reveal their types rather than taking an action that might or might not be correlated to their type.

DIRECT REVELATION MECHANISM

In these cases we would want to design a mechanism $g$ which implements a social choice function $f$ and encourages all agents to tell their true type. This might or might not be possible for a particular $f$. If it is possible then we say that $f$ is strategy-proof.

**Definition 8.6** (Strategy-Proof)**.** *The social choice function $f(\cdot)$ is* ***truthfully implementable in dominant strategies*** *(or* ***strategy-proof****) if for all $i$ and $\theta_i$ we have that $s_i^*(\theta_i) = \theta_i$ is a dominant strategy equilibrium of the direct revelation mechanism $f(\cdot)$. That is, if for all $i$ and all $\theta_i \in \Theta_i$,*

STRATEGY-PROOF

$$v_i(f(\theta_i, \theta_{-i}), \theta_i) \geq v_i(f(\hat{\theta}_i, \theta_{-i}), \theta_i)$$

*for all $\hat{\theta}_i \in \Theta_i$ and all $\theta_{-i} \in \Theta_{-i}$.*

That is, the value that each agent receives under the outcome prescribed by the social choice function when all tell the truth is bigger than or equal to the value it gets if it lied about its type. Notice how we have plugged in $f$ directly as the mechanism instead of using $g$, in other words $g = f$. As you might guess, this would make $g$ trivial to implement because we are given $f$. For example, if I ask you to find a mechanism that implements social function $f$ and you look at $f$ and realize that it is strategy-proof then all you have to do is directly use $f$. That is, you would ask the agents for their types, they would all tell the truth because telling the truth is their dominant strategy, you would then plug these values into $f$ and out would come the desired outcome.

These strategy-proof social choice functions make it trivial to find a $g$ that implements them, namely $g = f$. Still, we might worry that the particular $f$ we have been given to implement is not strategy-proof but there might exist some mechanism $g$ which implements $f$ in dominant strategies. That is, $g$ lets the agents take some action, which might be different from revealing their type, and uses these actions to come up with the same outcome that $f$ would have resolved using the agents true types. Furthermore, the actions the agents take are dominant given that they know about $g$.

Fortunately, there is no need to worry about finding such $g$ as it has been proven that no such $g$ exists. This is known as the **revelation principle**.

REVELATION PRINCIPLE

**Theorem 8.2** (Revelation Principle)**.** *If there exists a mechanism $g$ that implements the social choice function $f$ in dominant strategies then $f$ is truthfully implementable in dominant strategies.*

That is, if there exists a complicated mechanism that implements a given social function then there is also a much simpler mechanism which just asks the agents to reveal their types. Of course, this simpler mechanism might have other problems, as we will see.



Podemos agora especializar a Definição 8.3 para equilíbrios dominantes.

Definição 8.5 ($g$ Implementos $f$ em estratégias dominantes). Um mecanismo $g : S_1 \times \cdots \times S_A \to O$ ***implementa a função de escolha social $f(\cdot)$ em estratégias dominantes*** *se houver um perfil de estratégia de equilíbrio de estratégia dominante $(s_1^*(\cdot) \ldots, s_A^*(\cdot))$ do jogo induzido por $g$ tal que $g(s_1^*(\theta_1) \ldots, s_A^*(\theta_A)) = f(\theta_1, \ldots, \theta_A)$ para todos $\theta \in \Theta$.*

IMPLEMENTS SOCIAL CHOICE FUNCTION $f(\cdot)$ IN DOMINANT STRATEGIES

Antes de entrarmos em como encontrar essa mágica $g$ vamos explorar algumas simplificações do problema. A primeira simplificação é a que fizemos no nosso primeiro exemplo. Ou seja, que as estratégias dos agentes correspondem à revelação dos seus tipos. Ou seja, vamos supor que as ações dos agentes sejam simplesmente revelações de tipos. A única coisa que um agente pode fazer é dizer "Eu sou do tipo $\theta_x$". Claro, ele pode estar mentindo ao fazer essa afirmação. Chamamos isso de mecanismo de revelação direta porque os agentes revelam diretamente seus tipos, em vez de realizar uma ação que pode ou não estar correlacionada ao seu tipo.

DIRECT REVELATION MECHANISM

Nestes casos, gostaríamos de projetar um mecanismo $g$ que implemente uma função de escolha social $f$ e encoraje todos os agentes a dizerem seu verdadeiro tipo. Isso pode ou não ser possível para um $f$ específico. Se for possível, então dizemos que $f$ é à prova de estratégia.

***Definição 8.6 (Prova de Estratégia). A função de escolha social $f(\cdot)$ é verdadeiramente implementável em estratégias dominantes (ou à prova de estratégia) se for para todos $i$ e $\theta_i$ temos que $s_i^*(\theta_i) = \theta_i$ é um equilíbrio estratégico dominante do mecanismo de revelação direta $f(\cdot)$. Isto é, se para todos $i$ e todos $\theta_i \in \Theta_i$,***

STRATEGY-PROOF

$$v_i(f(\theta_i, \theta_{-i}), \theta_i) \geq v_i(f(\hat{\theta}_i, \theta_{-i}), \theta_i)$$

*ˆpara todos $\theta_i \in \Theta_i$ e todos $\theta_{-i} \in \Theta_{-i}$.*

Ou seja, o valor que cada agente recebe sob o resultado prescrito pela função de escolha social quando todos dizem a verdade é maior ou igual ao valor que obtém se mentiu sobre o seu tipo. Observe como conectamos $f$ diretamente como mecanismo em vez de usar $g$, em outras palavras, $g = f$. Como você pode imaginar, isso tornaria a implementação de $g$ trivial porque recebemos $f$. Por exemplo, se eu pedir para você encontrar um mecanismo que implemente a função social $f$ e você olhar para $f$ e perceber que ele é à prova de estratégia, então tudo o que você precisa fazer é usar diretamente $f$. Ou seja, você perguntaria aos agentes seus tipos, todos eles diriam a verdade porque dizer a verdade é sua estratégia dominante, você então inseriria esses valores $f$ e obteria o resultado desejado.

Essas funções de escolha social à prova de estratégia tornam trivial encontrar um $g$ que as implemente, ou seja, $g = f$. Ainda assim, podemos temer que o $f$ específico que nos foi dado implementar não seja à prova de estratégia, mas pode existir algum mecanismo $g$ que implemente $f$ em estratégias dominantes. Ou seja, $g$ permite que os agentes realizem alguma ação, que pode ser diferente de revelar seu tipo, e usa essas ações para chegar ao mesmo resultado que $f$ teria resolvido usando os truetypes dos agentes. Além disso, as ações que os agentes realizam são dominantes, visto que eles conhecem $g$.

Felizmente, não há necessidade de se preocupar em encontrá-los, $g$ já que foi provado que tais $g$ não existem. Isso é conhecido como princípio da revelação.

REVELATION PRINCIPLE

*Teorema 8.2 (Princípio da Revelação). Se existe um mecanismo $g$ que implementa a função de escolha social $f$ nas estratégias dominantes, então $f$ é verdadeiramente implementável nas estratégias dominantes.*

Isto é, se existe um mecanismo complicado que implementa uma determinada função social, então existe também um mecanismo muito mais simples que apenas pede aos agentes que revelem os seus tipos. É claro que este mecanismo mais simples pode ter outros problemas, como veremos.

### An Example Problem and Solution

Let's now use all this notation in an example. Imagine that you want to sell an item. There are a bunch of prospective buyer agents. You want to sell it to the agent that wants it the most, but you can't trust them to tell you the truth. More formally, we can describe this problem as consisting of the following variables.

- $\theta_i \in \Re$: types are the valuations.
- $o \in \{1, \ldots, n\}$: index of agent who gets the item.
- $v_i(o, \theta_i) = \theta_i$ if $o = i$, and 0 otherwise.
- $f(\theta) = \arg\max_i(\theta_i)$
- Each agent gets a $p_i(o)$ so that $u_i(o, \theta_i) = v_i(o, \theta_i) + p_i(o)$.

Given this problem we must now try to figure out how to implement the payments $p$ as well as how to determine the outcome given the agents' reported types, both of these together constitute the desired mechanism $g$. That is, as with most research in mechanism design, we are only interested in mechanism that involve paying or taxing the agents some amount of money in order to change their utility valuation.

After thinking about this problem for a while, you suddenly realize that this is a problem you have already seen. The solution is to use a Vickrey auction. Set $p(o)$ such that the agent who wins must pay a tax equal to the second highest valuation. No one else pays or receives anything. This mechanism results in the agents having final utilities as follows.

$$u_i(o, \theta_i) = \begin{cases} \theta_i - \max_{j \neq i} \theta_j & \text{if } o = i \\ 0 & \text{otherwise.} \end{cases}$$

That is, we define $g$ such that it returns an outcome $o$ which contains the index of the agent that sent you the highest bid. The $g$ also charges this winning agent an amount equal to the second highest bid and charges everyone else 0.

In fact, we can use the notation we have set up to prove that telling the truth is the dominant strategy in this scenario which, along with the fact that we implement the social choice function, makes the Vickrey auction strategy-proof for this social choice function.

*Truth-Telling is Dominant in Vickrey Payments Example.* We can prove that telling the truth is the dominant strategy by following these steps.

1. Let $b_i(\theta_i)$ be $i$'s bid given that his true valuation is $\theta_i$.
2. Let $b' = \max_{j \neq i} b_j(\theta_j)$ be the highest bid amongst the rest.
3. If $b' < \theta_i$ then any bid $b_i(\theta_i) > b'$ is optimal since
$$u_i(i, \theta_i) = \theta_i - b' > 0$$
4. If $b' > \theta_i$ then any bid $b_i(\theta_i) < b'$ is optimal since
$$u_i(i, \theta_i) = 0$$
5. Since we have that if $b' < \theta_i$ then $i$ should bid $> b'$ and if $b' > \theta_i$ then $i$ should bid $< b'$, and we don't know $b'$ then $i$ should bid $\theta_i$.

□



### Um exemplo de problema e solução

Vamos agora usar toda esta notação num exemplo. Imagine que você deseja vender um ite m. Existem vários agentes compradores em potencial. Você quer vendê-lo para o agente que mais deseja, mas não pode confiar nele para lhe dizer a verdade. Mais formalmente, podemos descrever este problema como consistindo nas seguintes variáveis.

- $\theta_i \in \Re$: os tipos são as avaliações.
- $o \in \{1, \ldots, n\}$: índice do agente que recebe o item.
- $v_i(o, \theta_i) = \theta_i$ se $o = i$ e 0 caso contrário.
- $f(\theta) =$ arg máximo$_i(\theta_i)$
- Cada agente recebe um $p_i(o)$ para que $u_i(o, \theta_i) = v_i(o, \theta_i) + p_i(o)$.

Dado este problema, devemos agora tentar descobrir como implementar os pagamentos $p$ bem como determinar o resultado dados os tipos relatados pelos agentes, ambos juntos constituem o mecanismo desejado $g$. Isto é, como acontece com a maioria das pesquisas em design de mecanismos, estamos interessados apenas em mecanismos que envolvam pagar ou tributar os agentes de alguma quantia de dinheiro, a fim de alterar a sua avaliação de utilidade.

Depois de pensar um pouco sobre esse problema, você de repente percebe que esse é um problema que você já viu. A solução é usar um leilão Vickrey. Defina $p(o)$ de modo que o agente vencedor deva pagar um imposto igual à segunda avaliação mais alta. Ninguém mais paga ou recebe nada. Este mecanismo faz com que os agentes tenham utilidades finais como segue.

$$u_i(o, \theta_i) = \begin{cases} \theta_i - \max_{j \neq i} \theta_j & \text{if } o = i \\ 0 & \text{otherwise.} \end{cases}$$

Ou seja, definimos $g$ de forma que retorne um resultado $o$ que contém o índice do agente que lhe enviou o lance mais alto. O $g$ também cobra desse agente vencedor um valor igual ao segundo lance mais alto e cobra 0 de todos os demais.

Na verdade, podemos utilizar a notação que estabelecemos para provar que dizer a verdade é a estratégia dominante neste cenário, o que, juntamente com o facto de implementarmos a função de escolha social, torna a estratégia de leilão de Vickrey à prova desta função de escolha social.

Dizer a verdade é dominante no exemplo de pagamentos Vickrey. Podemos provar que dizer a verdade é a estratégia dominante seguindo estes passos.

1. Seja $b_i(\theta_i)$ o lance de $i$, dado que sua verdadeira avaliação é $\theta_i$.
2. Seja $b' = \max_{j \neq i} b_j(\theta_j)$ o lance mais alto entre os demais.
3. Se $b' < \theta_i$ então qualquer lance $b_i(\theta_i) > b'$ é ideal, pois
$$u_i(i, \theta_i) = \theta_i - b' > 0$$
4. Se $b' > \theta_i$ então qualquer lance $b_i(\theta_i) < b'$ é ideal, pois
$$u_i(i, \theta_i) = 0$$
5. Como temos que se $b' < \theta_i$ então $i$ deveríamos dar um lance $> b'$ e se $b' > \theta_i$ então $i$ deveríamos dar um lance $< b'$, e não sabemos $b'$ então $i$ deveríamos dar um lance $\theta_i$.

□

| Name | $v_i(o,\tilde{\theta})$ | $v_i(o,\theta)+\sum_{j\neq i} v_j(\tilde{\theta})$ |
|---|---|---|
| Alice | $10-\frac{20}{4}=5$ | $5+15=20$ |
| Bob | $0-0=0$ | $0+20=20$ |
| Caroline | $10-\frac{20}{4}=5$ | $5+15=20$ |
| Donald | $10-\frac{20}{4}=5$ | $5+15=20$ |
| Emily | $10-\frac{20}{4}=5$ | $5+15=20$ |

**Table 8.3:** Groves-Clarke payments for house painting assuming that all agents tell the truth.



Theodore Groves.

GROVES-CLARKE MECHANISM

### The Groves-Clarke Mechanism

We have just shown how to check that a mechanism implements a particular social choice function can be truthfully implemented in dominant strategies. However, we did not show how we came up with the mechanism itself or how we decided to use Vickrey payments. We would like a general formula that can be used to calculate the agents' payments no matter what social choice function is given to us. Unfortunately, such a formula does not appear to exist.

However, if we instead assume that the social choice function is the social welfare solution and further assume that the agents have quasilinear preferences then we can use the **Groves-Clarke mechanism** to calculate the desired payments. Agents with quasilinear preferences are those with utilities in the form $u_i(o,\theta_i) = v_i(o,\theta_i)+p_i(o)$. Formally, the Groves-Clarke mechanism is defined as follows:

**Theorem 8.3** (Groves-Clarke Mechanism)**.** *If we have a social choice function*

$$f(\theta) = \arg\max_{o\in O} \sum_{i=1}^{n} v_i(o,\theta_i)$$

*then calculating the outcome using*

$$f(\tilde{\theta}) = \arg\max_{o\in O} \sum_{i=1}^{n} v_i(o,\tilde{\theta}_i),$$

*where $\tilde{\theta}$ are reported types, and giving the agents payments of*

$$p_i(\tilde{\theta}) = \sum_{j\neq i} v_j(f(\tilde{\theta}),\tilde{\theta}_j) - h_i(\tilde{\theta}_{-i}), \tag{8.2}$$

*where $h_i(\theta_{-i})$ is an arbitrary function, results in a strategy-proof mechanism (Groves, 1973; Clarke, 1971)..*



Edward H. Clarke. 1939–.

Notice that the payments that $i$ receives are directly proportional to the sum of everybody else's value. This is the key insight of the Groves-Clarke mechanism. In order to get the agents to tell the truth so that we may improve the social welfare we must pay the agents in proportion to this social welfare. Another way to look at it, perhaps a bit cynically, is to say that the way to get individuals to care about how everyone else is doing is to pay them in proportion to how everyone else is doing. For example, companies give shares of their company to employees in the hope that this will make them want the company as a whole to increase its profits, even if it means they have to work longer or take a pay-cut. In effect, the Groves-Clarke mechanism places the social welfare directly into the agent's utility function.

Lets apply the Groves-Clarke Mechanism to the house painting example from Table 8.2. Remember that the second solution we tried, where the cost of painting was divided among those who voted to paint, was not strategy-proof. Perhaps we can add Groves-Clarke payments to make it strategy-proof. To do this we must first re-evaluate the agents' value which will be decreased from 10 since they might have



| Name | $v_i(o,\tilde{\theta})$ | $v_i(o,\theta)+\sum_{j\neq i} v_j(\tilde{\theta})$ |
|---|---|---|
| Alice | $10-\frac{20}{4}=5$ | $5+15=20$ |
| Bob | $0-0=0$ | $0+20=20$ |
| Carolina | $10-\frac{20}{4}=5$ | $5+15=20$ |
| Donald | $10-\frac{20}{4}=5$ | $5+15=20$ |
| Emilly | $10-\frac{20}{4}=5$ | $5+15=20$ |

Tabela 8.3: Pagamentos Groves-Clarke para pintura de casas, assumindo que todos os agentes dizem a verdade.



Theodoro Groves.

GROVES-CLARKE MECHANISM

### O Mecanismo Groves-Clarke

Acabamos de mostrar como verificar se um mecanismo que implementa uma determinada função de escolha social pode ser verdadeiramente implementado em estratégias dominantes. No entanto, não mostramos como criamos o mecanismo em si ou como decidimos usar os pagamentos Vickrey. Gostaríamos de ter uma fórmula geral que pudesse ser usada para calcular os pagamentos dos agentes, independentemente da função de esc olha social que nos for dada. Infelizmente, tal fórmula parece não existir.

Contudo, se, em vez disso, assumirmos que a função de escolha social é a solução de bem-estar social e assumirmos ainda que os agentes têm preferências quase lineares, então poderemos utilizar o mecanismo de Groves-Clarke para calcular os pagamentos desejados. Agentes

com preferências quase lineares são aqueles com utilidades na forma $u_i(o,\theta_i) = v_i(o,\theta_i)+p_i(o)$. Formalmente, o mecanismo Groves-Clarke é definido da seguinte forma:

*Teorema 8.3 (Mecanismo de Groves-Clarke). Se tivermos uma função de escolha social*

$$f(\theta) = \arg\max_{o\in O} \sum_{i=1}^{n} v_i(o,\theta_i)$$

*em seguida, calculando o resultado usando*

$$f(\tilde{\theta}) = \arg\max_{o\in O} \sum_{i=1}^{n} v_i(o,\tilde{\theta}_i),$$

*˜onde $\theta$ estão os tipos relatados e dando aos agentes pagamentos de*

$$p_i(\tilde{\theta}) = \sum_{j\neq i} v_j(f(\tilde{\theta}),\tilde{\theta}_j) - h_i(\tilde{\theta}_{-i}), \tag{8.2}$$

*onde $h_i(\theta_{-i})$ é uma função arbitrária, resulta em um mecanismo à prova de estratégia (Groves, 1973;Clarke, 1971)..*



Edward H. Clarke. 1939–.

Observe que os pagamentos que $i$ recebe são diretamente proporcionais à soma do valor de todos os outros. Esta é a principal visão do mecanismo Groves-Clarke. Para que os agentes digam a verdade e possamos melhorar o bem-estar social, devemos pagar aos agentes proporcionalmente a esse bem-estar social. Outra maneira de encarar a questão, talvez com um pouco de cinismo, é dizer que a forma de fazer com que os indivíduos se preocupem com o desempenho de todos os outros é pagá-los proporcionalmente ao desempenho de todos os outros. Por exemplo, as empresas dão ações da sua empresa aos empregados na esperança de que isso os faça querer que a empresa como um todo aumente os seus lucros, mesmo que isso signifique que tenham de trabalhar mais tempo ou sofrer uma redução salarial. Com efeito, o mecanismo Groves-Clarke coloca o bem-estar social directamente na função de utilidade do agente.

Vamos aplicar o Mecanismo Groves-Clarke ao exemplo de pintura de casa da Tabela 8.2. Lembre-se que a segunda solução que tentámos, onde o custo da pintura foi dividido entre aqueles que votaram pela pintura, não era à prova de estratégia. Talvez possamos acrescentar pagamentos de Groves-Clarke para torná-lo à prova de estratégia. Para fazer isso, devemos primeiro reavaliar o valor dos agentes, que será diminuído de 10, uma vez que eles podem ter

**Table 8.4:** Groves-Clarke payments for house painting assuming that Alice lies and all others tell the truth.

| Name | $v_i(o,\tilde{\theta})$ | $v_i(o,\theta)+\sum_{j\neq i} v_j(\tilde{\theta})$ |
|---|---|---|
| Alice | $0-0=0$ | $10+(\frac{10}{3}\cdot 3)=20$ |
| Bob | $0-0=0$ | $0+(\frac{10}{3}\cdot 3)=10$ |
| Caroline | $10-\frac{20}{3}=\frac{10}{3}$ | $\frac{10}{3}+(\frac{10}{3}\cdot 2)=10$ |
| Donald | $10-\frac{20}{3}=\frac{10}{3}$ | $\frac{10}{3}+(\frac{10}{3}\cdot 2)=10$ |
| Emily | $10-\frac{20}{3}=\frac{10}{3}$ | $\frac{10}{3}+(\frac{10}{3}\cdot 2)=10$ |

to pay for part of the painting, if they voted yes, and then calculate the agents' payments using Theorem 8.3. The set of payments the agents would receive if they all told the truth is shown in Table 8.3. As you can see, all the agents get the same utility (20) from telling the truth.

Now, what if Alice lied? Would she get a higher utility? Table 8.4 shows the payments and utility values for the case where Alice lies and the rest tell the truth. As you can see, Alice still gets the same 20 of utility. As such, she has nothing to gain by lying (you should repeat these calculations for Bob to make sure you understand how the equations are used). It is interesting to note how everyone else's utilities have dropped due to Alice's lie. Of course Alice, being purely selfish, does not care about this.

Finally, notice how the payments add up to 80 on the first example and 40 in the second example. Where does this money come from? Note that we must give the agents real money otherwise the mechanism does not work. We cannot simply tell them to imagine we are giving them $20. What would be really nice is if some of the agents payed us money and we payed some back to the other agents such that the total amount we pay equals the total amount we receive. We would thus

REVENUE EQUIVALENCE

achieve **revenue equivalence**. Unfortunately, as you have seen, the Groves-Clarke mechanism is not revenue equivalent.

### The Vickrey-Clarke-Groves Mechanism

VICKREY-CLARKE-GROVES

Another well-known payment mechanism is **Vickrey-Clarke-Groves**, which is just a small variation on Groves-Clarke but is closer to achieving revenue equivalence.

**Theorem 8.4** (Vickrey-Clarke-Groves (VCG) Mechanism)**.** *If*

$$f(\theta)=\arg\max_{o\in O}\sum_{i=1}^{n} v_i(o,\theta_i)$$

*then calculating the outcome using*

$$f(\tilde{\theta})=\arg\max_{o\in O}\sum_{i=1}^{n} v_i(o,\tilde{\theta}_i)$$

*(where $\tilde{\theta}$ are reported types) and giving the agents payments of*

$$p_i(\tilde{\theta})=\sum_{j\neq i} v_j(f(\tilde{\theta}),\tilde{\theta}_j)-\sum_{j\neq i} v_j(f(\tilde{\theta}_{-i}),\tilde{\theta}_j) \tag{8.3}$$

*results in a strategy-proof mechanism.*

VCG is almost identical to Groves-Clarke except that the payments are slightly different. The payments in VCG measure the agent's contribution to the whole, what some call the "wonderful life" utility, which we saw in Chapter 5.6. You might have seen that old Frank Capra film "It's a Wonderful life". In it, George Bailey gets a chance to see how the world would have been had he never existed. He notices that



Tabela 8.4: Pagamentos de Groves-Clarke para pintura de casas, assumindo que Alicelies e todas as outras dizem a verdade.

| Name | $v_i(o,\tilde{\theta})$ | $v_i(o,\theta)+\sum_{j\neq i} v_j(\tilde{\theta})$ |
|---|---|---|
| Alice | $0-0=0$ | $10+(\frac{10}{3}\cdot 3)=20$ |
| Bob | $0-0=0$ | $0+(\frac{10}{3}\cdot 3)=10$ |
| Carolina | $10-\frac{20}{3}=\frac{10}{3}$ | $\frac{10}{3}+(\frac{10}{3}\cdot 2)=10$ |
| Donald | $10-\frac{20}{3}=\frac{10}{3}$ | $\frac{10}{3}+(\frac{10}{3}\cdot 2)=10$ |
| Emilly | $10-\frac{20}{3}=\frac{10}{3}$ | $\frac{10}{3}+(\frac{10}{3}\cdot 2)=10$ |

pagar parte da pintura, caso votassem sim, e então calcular os pagamentos dos agentes usando o Teorema 8.3. O conjunto de pagamentos que os agentes receberiam se todos dissessem a verdade é mostrado na Tabela 8.3. Como você pode ver, todos os agentes obtêm a mesma utilidade (20) ao dizer a verdade.

Agora, e se Alice mentiu? Ela teria uma utilidade maior? A Tabela 8.4 mostra os pagamentos e os valores de utilidade para o caso em que Alice mente e os restantes dizem a verdade. Como você pode ver, Alice ainda obtém os mesmos 20 de utilidade. Como tal, ela não tem nada a ganhar mentindo (você deve repetir esses cálculos para Bob para ter certeza de que entendeu como as equações são usadas). É interessante notar como as utilidades de todos os outros caíram devido à mentira de Alice. É claro que Alice, sendo puramente egoísta, não se importa com isso.

Por fim, observe como os pagamentos somam 80 no primeiro exemplo e 40 no segundo exemplo. De onde vem esse dinheiro? Observe que devemos dar dinheiro real aos agentes, caso contrário o mecanismo não funciona. Não podemos simplesmente dizer-lhes que imaginem que lhes estamos a dar 20 dólares. O que seria muito bom é se alguns dos agentes nos pagassem dinheiro e nós reembolsássemos alguns aos outros agentes, de modo que o valor total que pagamos fosse igual ao valor total que recebemos. Alcançaríamos assim a equivalência de receitas. Infelizmente, como você viu, o grupo Groves-Clarke

REVENUE EQUIVALENCE

mecanismo não é equivalente a receitas.

### O Mecanismo Vickrey-Clarke-Groves

VICKREY-CLARKE-GROVES

Outro mecanismo de pagamento bem conhecido é o Vickrey-Clarke-Groves, que é apenas uma pequena variação em relação a Groves-Clarke, mas está mais perto de alcançar a equivalência de receitas.

Teorema 8.4 (Mecanismo Vickrey-Clarke-Groves (VCG)). Se

$$f(\theta)=\arg\max_{o\in O}\sum_{i=1}^{n} v_i(o,\theta_i)$$

*em seguida, calculando o resultado usando*

$$f(\tilde{\theta})=\arg\max_{o\in O}\sum_{i=1}^{n} v_i(o,\tilde{\theta}_i)$$

*˜(onde $\theta$ são os tipos relatados) e dando aos agentes pagamentos de*

$$p_i(\tilde{\theta})=\sum_{j\neq i} v_j(f(\tilde{\theta}),\tilde{\theta}_j)-\sum_{j\neq i} v_j(f(\tilde{\theta}_{-i}),\tilde{\theta}_j) \tag{8.3}$$

*resulta em um mecanismo à prova de estratégia.*

O VCG é quase idêntico ao Groves-Clarke, exceto que os pagamentos são ligeiramente diferentes. Os pagamentos em VCG medem a contribuição do agente para o todo, o que alguns chamam de utilidade da "vida maravilhosa", que vimos no Capítulo 5.6. Você deve ter visto aquele filme antigo de Frank Capra "It's a Wonderful life". Nele, George Bailey tem a chance de ver como seria o mundo se ele nunca tivesse existido. Ele percebe que

| Name | $v_i(o, \tilde{\theta})$ | $\sum_{j \neq i} v_j(f(\tilde{\theta}_{-i}), \tilde{\theta}_j)$ | $\sum_{j \neq i} v_j(f(\tilde{\theta}), \tilde{\theta}_j) - \sum_{j \neq i} v_j(f(\tilde{\theta}_{-i}), \tilde{\theta}_j)$ |
|---|---|---|---|
| Alice | $10 - \frac{20}{4} = 5$ | $(10 - \frac{20}{3}) \cdot 3 = 10$ | $15 - 10 = 5$ |
| Bob | $0 - 0 = 0$ | $(10 - \frac{20}{4}) \cdot 4 = 20$ | $20 - 20 = 0$ |
| Caroline | $10 - \frac{20}{4} = 5$ | $(10 - \frac{20}{3}) \cdot 3 = 10$ | $15 - 10 = 5$ |
| Donald | $10 - \frac{20}{4} = 5$ | $(10 - \frac{20}{3}) \cdot 3 = 10$ | $15 - 10 = 5$ |
| Emily | $10 - \frac{20}{4} = 5$ | $(10 - \frac{20}{3}) \cdot 3 = 10$ | $15 - 10 = 5$ |

**Table 8.5:** VCG payments for house painting assuming Alice tells the truth.

the social welfare in the world without him is lower than in the world with him in it. As such, he calculates that his existence has had a positive effect on social welfare and thus decides not to end his life, which makes us conclude that he wanted to increase social welfare. Nice guy that George.

That is exactly what the VCG payments calculate. In the first term of (8.3) the $f(\tilde{\theta})$ captures the outcome had agent $i$ not existed. The first term thus captures the social welfare had $i$ not existed while the second term captures the social welfare with $i$ in the picture. We point out that, in practice, $f(\tilde{\theta})$ is very hard to compute. For many multiagent systems it is not clear what would happen if we took out one of the agents – imagine a soccer team without a player, or an assembly line without a worker, or workflow without an agent.

The advantage of VCG over Groves-Clarke is that it results in a lower net revenue gain or loss. We can see this if we re-calculate the payments for the house painting example, as seen in Table 8.5. The payments in this case add up to 20, as compared with 40 and 80 for the Groves-Clarke payments. Of course, 20 is not 0 so we have not yet achieved total revenue equivalence. The research literature tells us that revenue equivalence for many specific cases is impossible to achieve.

The main problem we face when trying to use both Groves-Clarke and VCG payments is that calculating these payments takes exponential time on the number of agents. This means that they can only be used when we have a small number of agents. Still, you might consider approximations of these payments which can be calculated quickly for your specific problem domain.

### 8.2.2 Distributed Mechanism Design

ALGORITHMIC MECHANISM DESIGN

DISTRIBUTED ALGORITHMIC MECHANISM DESIGN

Computer scientists have taken the results from mechanism design and tried to apply them to multiagent systems problems, this required further specification of the problem. The first extension to mechanism design that appeared was **algorithmic mechanism design** which proposes that the mechanism should run in polynomial time (Nisan and Ronen, 2001). These types of mechanism are sorely needed because the calculation of VCG and Groves-Clarke payments takes exponential time. The second extension was **distributed algorithmic mechanism design** (DAMD) (Feigenbaum et al., 2001; Dash et al., 2003) which proposes that the computations carried out by the mechanism should be performed by the agents themselves and in polynomial time, with added limitations on the number of messages that can be sent. Note that if the agents themselves are performing the calculations for the mechanism then we must also guard against their interference with the calculation of the outcome. This can be done either by design – for example, by encrypting partial results – or by providing the agents with the proper incentives so they will not want to cheat.

Thus far, DAMD algorithms have been almost exclusively developed for computer network applications. Unlike traditional network research, these algorithms treat the individual routers as selfish agents that try to maximize their given utility function. That is, the Internet is no longer viewed as composed of dumb and obedient routers but is instead viewed as composed of utility-maximizing autonomous entities who control specific routers or whole sub-networks. It is interesting to not that while



| Name | $v_i(o, \tilde{\theta})$ | $\sum_{j \neq i} v_j(f(\tilde{\theta}_{-i}), \tilde{\theta}_j)$ | $\sum_{j \neq i} v_j(f(\tilde{\theta}), \tilde{\theta}_j) - \sum_{j \neq i} v_j(f(\tilde{\theta}_{-i}), \tilde{\theta}_j)$ |
|---|---|---|---|
| Alice | $10 - \frac{20}{4} = 5$ | $(10 - \frac{20}{3}) \cdot 3 = 10$ | $15 - 10 = 5$ |
| Bob | $0 - 0 = 0$ | $(10 - \frac{20}{4}) \cdot 4 = 20$ | $20 - 20 = 0$ |
| Carolina | $10 - \frac{20}{4} = 5$ | $(10 - \frac{20}{3}) \cdot 3 = 10$ | $15 - 10 = 5$ |
| Donald | $10 - \frac{20}{4} = 5$ | $(10 - \frac{20}{3}) \cdot 3 = 10$ | $15 - 10 = 5$ |
| Emilly | $10 - \frac{20}{4} = 5$ | $(10 - \frac{20}{3}) \cdot 3 = 10$ | $15 - 10 = 5$ |

**Tabela 8.5:** VCG **pagamentos para** pintura de casa assumindo Alice diz a verdade.

o bem-estar social no mundo sem ele é menor do que no mundo com ele nele. Como tal, calcula que a sua existência teve um efeito positivo no bem-estar social e por isso decide não acabar com a sua vida, o que nos leva a concluir que pretendia aumentar o bem-estar social. Cara legal esse George.

É exatamente isso que os VCG pagamentos calculam. No primeiro termo de (8.3) o $f(\tilde{\theta})$ captura o resultado se o agente $i$ não existisse. O primeiro termo captura, portanto, o bem-estar social $i$não existia, enquanto o segundo termo captura o bem-estar social com $i$ na imagem. Ressaltamos que, na prática, $f(\theta)$ é muito difícil de calcular. Para muitos sistemas multiagentes não está claro o que aconteceria se retirássemos um dos agentes – imagine um time de futebol sem jogador, ou uma linha de montagem sem trabalhador, ou um fluxo de trabalho sem agente.

A vantagem de VCG sobre Groves-Clarke é que ele resulta em menor ganho ou perda de receita líquida. Podemos ver isso se recalcularmos os pagamentos do exemplo da pintura da casa, como visto na Tabela 8.5. Os pagamentos neste caso somam 20, em comparação com 40 e 80 para os pagamentos Groves-Clarke. É claro que 20 não é 0, por isso ainda não atingimos a equivalência total de receitas. A literatura de investigação diz-nos que é impossível alcançar a equivalência de receitas para muitos casos específicos.

O principal problema que enfrentamos ao tentar usar pagamentos Groves-Clarke e VCG é que o cálculo desses pagamentos leva um tempo exponencial em relação ao número de agentes. Isso significa que eles só podem ser usados quando temos um pequeno número de agentes. Ainda assim, você pode considerar aproximações desses pagamentos que podem ser calculadas rapidamente para o domínio do seu problema específico.

### 8.2.2 Projeto de Mecanismo Distribuído

ALGORITHMIC MECHANISM DESIGN

DISTRIBUTED ALGORITHMIC MECHANISM DESIGN

Os cientistas da computação pegaram os resultados do projeto de mecanismos e tentaram aplicá-los a problemas de sistemas multiagentes, o que exigiu especificações adicionais do problema. A primeira extensão do projeto de mecanismo que apareceu foi o projeto de mecanismo algorítmico, que propõe que o mecanismo possa ser executado em polinômio tempo (Nisan e Ronen, 2001). Esses tipos de mecanismos são extremamente necessários porque o cálculo dos pagamentos VCG e Groves-Clarke leva um tempo exponencial. A segunda extensão foi o design de mecanismo algorítmico distribuído(DAMD) (Feigenbaum etal., 2001; Dashet al., 2003) que propõe que os cálculos realizados pelo mecanismo sejam realizados pelos próprios agentes e em tempo polinomial, com limitações adicionais no número de mensagens que podem ser enviadas. Note-se que se os próprios agentes estão a realizar os cálculos para o mecanismo, então devemos também precaver-nos contra a sua interferência no cálculo do resultado. Isto pode ser feito intencionalmente – por exemplo, criptografando resultados parciais – ou fornecendo aos agentes os incentivos adequados para que não queiram trapacear.

Até agora, DAMDalgoritmos foram desenvolvidos quase exclusivamente para aplicações de redes de computadores. Ao contrário da pesquisa tradicional de redes, esses algoritmos tratam os roteadores individuais como agentes egoístas que tentam maximizar sua função de utilidade. Ou seja, a Internet já não é vista como composta por routers burros e obedientes, mas sim como composta por entidades autónomas que maximizam a utilidade e controlam routers específicos ou sub-redes inteiras. É interessante não fazer isso enquanto

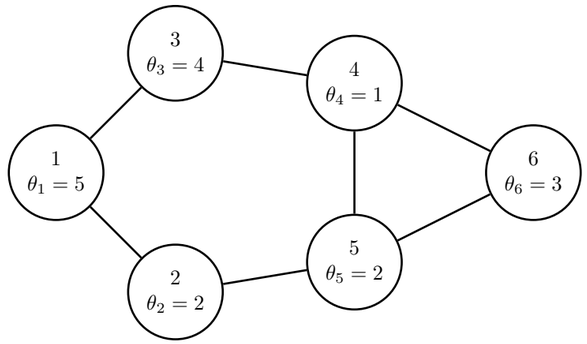



**Figure 8.2:** Example Inter-domain routing problem. The nodes represent networks. Each one shows the cost it would incur in transporting a packet through its network. The agents' types are their costs.

this model is a much more faithful representation of the real Internet, network researchers have largely ignored the fact that private interests exist in the Internet and instead assume that all parties want to maximize the social welfare.

Figure 8.2 shows a sample inter-domain routing problem. In this problem each node represents a computer network. The edges represent how these networks are connected to each other. Each network is willing to handle packets whose source or destination address is within the network. However, they are reticent to carry packets that are just passing thru. Each network in the figure also shows the cost it incurs in routing one of these passing-thru packets. These costs correspond to the agents' types. That is, only the agent knows how much it costs to pass a packet across its network. We further assume that we know how much traffic will travel from every start node to every destination node. The problem then is to find the routes for each source-destination pair such that the costs are minimized. This is a mechanism design problem which can be solved in a distributed manner (Feigenbaum et al., 2005), as we now show.

We can formally define the problem as consisting of $n$ agents. Each agent $i$ knows its cost to be $\theta_i$. An outcome $o$ of our mechanism design problem corresponds to a set of routing rules for all source-destination pairs. That is, the outcome $o$ should tell us, for all $i, j$, which path a packet traveling from $i$ to $j$ should follow. The value $v_i(o)$ that each agent receives for a particular outcome $o$ is the negative of the total amount of traffic that $i$ must endure given the paths specified in $o$ and the traffic amounts between every pair.

The solution we want is then simply

$$f(\theta) = \arg\max_o \sum_i v_i(o). \tag{8.4}$$

Since this is the social-welfare maximizing solution we are free to use VCG payments to solve the problem. Specifically, we can ask all agents to report their costs $\tilde{\theta}$ and then use (8.3) to calculate everyone's payments thus ensuring that telling the truth is the dominant strategy for all agents. Note that, in this case, calculating the outcome given $\tilde{\theta}$ amounts to calculating a minimum spanning tree for each destination node $j$. The problem of finding the routes that minimize the sum of all costs has already been solved, in a distributed manner, by the standard Internet **Border Gateway Protocol** (BGP). The protocol works by propagating back the minimum costs to a given destination. All agents tell each other their costs to destination. Each agent then updates its cost to a destination by adding its own transportation costs. The process then repeats until quiescence. The algorithm is, in fact, a variation on Dijkstra's algorithm for finding shortest paths in a graph (Dijkstra, 1959).

Border Gateway Protocol

Note, however, that the payment calculation (8.3) requires us to find the sum of everyone else's valuation for the case where $i$ does not exist, the second term in (8.3). For this problem, eliminating $i$ simply means eliminating all links to $i$. But,



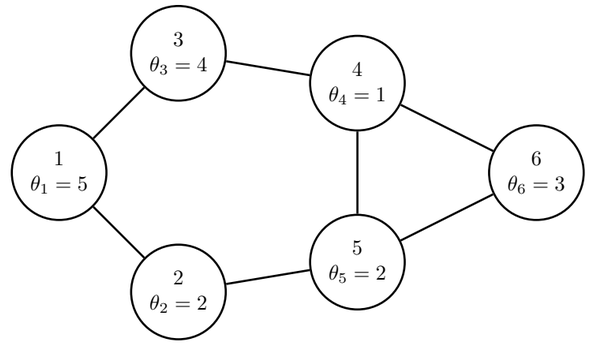



Figura 8.2: Exemplo de problema de roteamento entre domínios. Os nós representam redes. Cada um mostra o custo que incorreria no transporte de um pacote através de sua rede. Os tipos dos agentes são os seus custos.

este modelo é uma representação muito mais fiel da Internet real, os investigadores de redes têm ignorado largamente o facto de que existem interesses privados na Internet e, em vez disso, assumem que todas as partes querem maximizar o bem-estar social.

A Figura 8.2 mostra um exemplo de problema de roteamento entre domínios. Neste problema cada nó representa uma rede de computadores. As arestas r epresentam como essas redes estão conectadas entre si. Cada rede está disposta a lidar com pacotes cujo endereço de origem ou destino esteja dentro da rede. No entanto, eles são reticentes em transportar pacotes que estão apenas de passagem. Cada rede na figura também mostra o custo incorrido no roteamento de um desses pacotes de passagem. Estes custos correspondem aos tipos de agentes. Ou seja, apenas o agente sabe quanto custa passar um pacote pela sua rede. Assumimos ainda que sabemos quanto tráfego viajará de cada nó inicial para cada nó destino. O problema então é encontrar as rotas para cada par origem-destino de modo que os custos sejam minimizados. Este é um problema de projeto de mecanismo que pode ser resolvido de maneira distribuída (Feigenbaum et al., 2005), como mostramos agora.

Podemos definir formalmente os problemas que consistem em $n$ agentes. Cada agente $i$ sabe que seu custo é $\theta_i$. Um resultado $o$ do nosso problema de design de mecanismo corresponde a um conjunto de regras de roteamento para todos os pares origem-destino. Ou seja, o resultado $o$ deveria nos dizer, para todo $i, j$, qual caminho um pacote viajando de $i$ para $j$ deveria seguir. O valor$v_i(o)$ que cada agente recebe para um resultado específico $o$ é o negativo da quantidade total de tráfego que $i$ deve suportar dados os caminhos especificados em $o$ e os valores de tráfego entre cada par.

A solução que queremos é então simplesmente

$$f(\theta) = \arg\max_o \sum_i v_i(o). \tag{8.4}$$

Como esta é a solução que maximiza o bem-estar social, somos livres para usar VCG pagamentos para resolver o problema. Especificamente, podemos pedir a todos os agentes que relatem seus custos $\theta$ e então usar (8.3) para calcular os pagamentos de todos, gara ntindo assim que dizer a verdade seja a estratégia dominante para todos os agentes. Observe que, neste caso, calcular o resultado dado $\theta$ equivale ao cálculo de uma árvore geradora mínima para cada nó de destino$j$. O problema de encontrar as rotas que minimizem a soma de todos os custos já foi resolvido, de forma distribuída, pelo padrão Internet Border Gateway Protocol (BGP). O protocolo funciona propagando de volta os custos mínimos para um

Border Gateway Protocol

determinado destino. Todos os agentes informam uns aos outros seus custos até o destino. Cada agente então atualiza seu custo para um destino adicionando seus próprios custos de transporte. O processo então se repete até a quiescência. O algoritmo é, na verdade, uma variação do algoritmo de Dijkstra para encontrar caminhos mais curtos em um gráfico (Dijkstra, 1959).

Observe, entretanto, que o cálculo do pagamento (8.3) exige que encontremos a soma da avaliação de todos os outros para o caso em que $i$ não existe, o segundo termo em (8.3). Para este problema, eliminar $i$ significa simplesmente eliminar todos os links para $i$. Mas,

it could be that the elimination of these links partitions the graph into two or more pieces thereby making it impossible for traffic from some nodes to reach other nodes. In such a case then it would be impossible to calculate the VCG payments. Thus, we must make the further assumption that the network is **bi-connected**: any one node can be eliminated and still every node can be reached from every other node.

BI-CONNECTED

We also note that the VCG payments can be broken down into a sum

$$p_i(\tilde{\theta}) = \sum_{a,b} p_i(a, b, \tilde{\theta}), \tag{8.5}$$

where $p_i(a, b, \tilde{\theta})$ is the payment that $i$ receives for the traffic that goes from node $a$ to node $b$. This payment can be further broken down into its constituent VCG parts: the value everyone else gets and the value everyone else would get if $i$ did not exist. We can find these by first defining the total cost incurred in sending a packet from $a$ to $b$. We let *lowest-cost-path*$(a, b, \tilde{\theta})$ be the lowest cost path – a list of nodes – for sending a packet from $a$ to $b$ given $\tilde{\theta}$. The total cost associated with that path is given by

$$cost(a, b, \tilde{\theta}) = \sum_{i \in lowest\text{-}cost\text{-}path(a,b,\tilde{\theta})} \tilde{\theta}_i, \tag{8.6}$$

which is simply the sum of the costs in all the intervening nodes. We can then determine that the payment that $i$ gets is given by

$$p_i(a, b, \tilde{(\theta)}) = \tilde{\theta}_i - cost(a, b, \tilde{\theta}_i) + cost_{-i}(a, b, \tilde{\theta}_i), \tag{8.7}$$

where $cost_{-i}(a, b, \tilde{\theta}_i)$ is defined in the same as *cost* but for a graph without agent $i$. Note how this payment equation is really just the VCG payments once again. The first two terms capture the value that everyone else receives while the third term capture the value that everyone else receives if $i$ did not exist; since $i$ does not exist he does not contribute cost. The main difference is that the signs are reversed since we are dealing with costs and not value.

We now note that since the payments depend solely on the costs of the lowest cost path from $i$ to $j$ then some of these costs could be found by adding other costs. For example, if the lowest cost path from $i$ to $j$ involves first going from $i$ to $a$ directly and then taking the lowest cost path from $a$ to $j$ then the $cost(i, j, \tilde{\theta}) = \tilde{\theta}_i + cost(a, j, \tilde{\theta})$. A careful analysis of these type of constraints leads us to deduce that for all $k \in lowest\text{-}cost\text{-}path(i, j, \tilde{\theta})$ where $a$ is a neighbor of $i$, we have that:

1. If $a$ is $i$'s parent in *lowest-cost-path*$(i, j, \tilde{\theta})$ then $p_k(i, j) = p_k(a, j)$.
2. If $a$ is $i$'s child in *lowest-cost-path*$(i, j, \tilde{\theta})$ then $p_k(i, j) = p_k(a, j) + \tilde{\theta}_i + \tilde{\theta}_a$.
3. If $i$ is neither a parent or a child of $i$ and $k \in lowest\text{-}cost\text{-}path(a, j, \tilde{\theta})$ then $p_i(i, j) = \tilde{\theta}_a + cost(a, j) = cost(i, j)$.
4. If $i$ is neither a parent or a child of $i$ and $k \notin lowest\text{-}cost\text{-}path(a, j, \tilde{\theta})$ then $p_k(i, j) = \tilde{\theta}_k + \tilde{\theta}_a + cost(a, j) - cost(i, j)$.

We can then build an algorithm where the agents send each other payment tables and then update their own payment tables using the ones they received along with the above equations. Figure 8.3 shows such an algorithm. All agents start by executing the procedure INITIALIZE and then send HANDLE-UPDATE to their neighbors in order to get the process going. The algorithm has been shown to converge to the true prices for agents.

Note that the calculation of *lowest-cost-path* is itself a distributed calculation that is being carried out simultaneously by the BGP algorithm. This means that the values of *lowest-cost-path* are also changing and, thus, might be incorrect. However, it has been shown that the payment calculations will converge even in the presence of such changes.



pode ser que a eliminação desses links particione o gráfico em duas ou mais partes, tornando impossível que o tráfego de alguns nós chegue a outros nós. Nesse caso, seria impossível calcular os VCGpagamentos. Assim, devemos assumir ainda que a rede é biconectada: qualquer uma

o nó pode ser eliminado e ainda assim cada nó pode ser alcançado a partir de qualquer outro nó.

BI-CONNECTED

Observamos também que os VCG pagamentos podem ser divididos em uma soma

$$p_i(\tilde{\theta}) = \sum_{a,b} p_i(a, b, \tilde{\theta}), \tag{8.5}$$

˜onde $p_i(a, b, \theta)$ é o pagamento que $i$ recebe pelo tráfego que vai do nó $a$ ao nó $b$. Esse pagamento pode ser subdividido em suas VCG partes constituintes: o valor que todos os outros recebem e o valor que todos os outros receberiam se $i$ não existisse. Podemos encontrá-los definindo primeiro o custo total incorrido no envio de um pacote de$_a$ para $b$. Deixamos que o caminho de custo mais baixo$(a, b, \theta)$ seja o caminho de menor custo – uma lista de nós – para ˜enviar um pacote de $a$ para $b$ dado $\theta$. O custo total associado a esse caminho é dado por

$$cost(a, b, \tilde{\theta}) = \sum_{i \in lowest\text{-}cost\text{-}path(a,b,\tilde{\theta})} \tilde{\theta}_i, \tag{8.6}$$

que é simplesmente a soma dos custos em todos os nós intervenientes. Podemos então determinar que o pagamento $i$ recebido é dado por

$$p_i(a, b, \tilde{(\theta)}) = \tilde{\theta}_i - cost(a, b, \tilde{\theta}_i) + cost_{-i}(a, b, \tilde{\theta}_i), \tag{8.7}$$

˜onde custo$_{-i}(a, b, \theta_i)$ é definido da mesma forma que custo, mas para um gráfico sem agente $i$. Observe como essa equação de pagamento é, na verdade, apenas VCG pagamentos mais uma vez. Os dois primeiros termos capturam o valor que todos os outros recebem, enquanto o terceiro termo captura o valor que todos os outros r ecebem se $i$ não existisse; como $i$ não existe, ele não contribui com custos. A principal diferença é que os sinais se invertem, pois estamos tratando de custos e não de valor.

Observamos agora que, uma vez que os pagamentos dependem exclusivamente dos custos do caminho de menor custo de $i$ a $j$ então alguns desses custos poderiam ser encontrados adicionando outros custos. Por exemplo, se o caminho de menor custo de $i$ para $j$ envolve primeiro ir de $i$para $a$˜diretamente e depois seguir o caminho de menor custo de ˜ $a$ =para $j$então o custo$(i, j, \theta)$ $\theta_i$ + custo$(a, j, \theta)$. Uma análise cuidadosa deste tipo de restrições nos leva a deduzir ˜ que para todo $k \in$caminho de menor custo$(i, j, \theta)$ onde $a$é vizinho de $i$, temos que:

1. ˜Se $a$é o pai de $i$no caminho de menor custo$(i, j, \theta)$ então $p_k(i, j) = p_k(a, j)$.
2. ˜Se $a$é filho de $i$no caminho de menor custo$(i, j, \theta)$ então $p_k(i, j) = p_k(a, j) + \theta_i + \theta_a$.˜3. Se $i$ não for pai nem filho de $i$e $k \in$caminho de custo mais baixo$(a, j, \theta)$, então $p_i(i, j) = \theta_a +$custo$(a, j) =$custo$(i, j)$.
3. ˜Se $i$ não for pai nem filho de $i$e $k \notin$caminho de custo mais baixo$(a, j, \theta)$, então $p_k(i, j) = \theta_k + \theta_a +$custo$(a, j) -$custo$(i, j)$.

Podemos então construir um algoritmo onde os agentes enviam tabelas de pagamento uns aos outros e então atualizam suas próprias tabelas de pagamento usando as que receberam junto com as equações acima. A Figura 8.3 mostra esse algoritmo. Todos os agentes começam executando o procedimento INITIALIZE e depois enviam HANDLE-UPDATE aos seus vizinhos para iniciar o processo. Foi demonstrado que o algoritmo converge para os preços reais dos agentes.

Observe que o cálculo do caminho de menor custo é em si um cálculo distribuído que está sendo realizado simultaneamente pelo algoritmo BGP . Isto significa que os valores do caminho de menor custo também estão mudando e, portanto, podem estar incorretos. No entanto, foi demonstrado que os cálculos dos pagamentos convergirão mesmo na presença de tais alterações.

```
INITIALIZE()
1  for j ← 1 . . . n ▷ for each destination j
2       do calculate lowest-cost-path(i, j) and cost(i, j)
3          for k ∈ lowest-cost-path(i, j)
4               do p_k[i, j] ← ∞

HANDLE-UPDATE(a, j, cost-aj, path, payments)
    ▷ a is the agent sending the message (invoking this procedure).
    ▷ j is the destination node.
    ▷ cost-aj is the reported cost of sending a packet from a to j.
    ▷ path is a list of nodes on the lowest cost path from a to j.
    ▷ payments are the payments for each node on path.
 1  modified ← FALSE
 2  if a ∈ lowest-cost-path(i, j) ▷ Parent
 3     then for k ∈ path
 4               do if p_k[i, j] > p_k[a, j]
 5                     then p_k[i, j] ← p_k[a, j]
 6                          modified ← TRUE
 7  elseif i ∈ lowest-cost-path(a, j) ▷ Child
 8     then for k ∈ {All but last item in path}
 9               do if p_k[i, j] > p_k[a, j] + θ̃_a + θ̃_i
10                     then p_k[i, j] ← p_k[a, j] + θ̃_a + θ̃_i
11                          modified ← TRUE
12  else
13          t ← position of last node for which path equals lowest-cost-path(i, j)
14          for k ∈ lowest-cost-path(i, j)[1 . . t]
15               do if p_k[i, j] > p_k[a, j] + θ̃_a + cost-aj − cost(i, j)
16                     then p_k[i, j] ← p_k[a, j] + θ̃_a + cost-aj − cost(i, j)
17                          modified ← TRUE
18          for k ∈ lowest-cost-path(i, j)[t + 1 . .]
19               do if p_k[i, j] > θ̃_k + θ̃_a + cost-aj − cost(i, j)
20                     then p_k[i, j] ← θ̃_k + θ̃_a + cost-aj − cost(i, j)
21                          modified ← TRUE
22  if modified
23     then my-payments ← p_k[i, j] for k ∈ lowest-cost-path(i, j)
24          for b is my neighbor
25               do b.HANDLE-UPDATE(i, j, cost(i, j),
                                    lowest-cost-path[i, j], my-payments)
```

**Figure 8.3:** Algorithm for calculating VCG payments for the inter-domain routing problem. $i$ refers to the agent running the algorithm. All agents start by executing INITIALIZE and then send a HANDLE-UPDATE to all their neighbors.

### 8.2.3 Mechanism Design Summary

Mechanism design provides us with a solid mathematical framework for expressing many multiagent design problems. Specifically, we can represent problems involving selfish agents who are intelligent enough to lie if it will increase their utility. Furthermore, the Groves-Clarke and VCG payment equations give us a ready-made solution for these problems. Unfortunately, this solution only works if we can implements payments and, even then, the payments are not revenue-neutral so we need an infinite amount of cash to give the agents. Also, these solutions are centralized in that all types are reported to a central agent who is then in charge of calculating and distributing the payments.

Distributed mechanism design is a new research area: few effective algorithms exist and many roadblocks are visible. Different approaches are being taken to overcome these roadblocks. One approach is to extend the mechanism design problem by defining trust-based mechanisms in which agents take into account the degree of trust they have on each other when determining their allocations (Dash et al.,



INITIALIZE() 1 para $j \leftarrow 1 \ldots n$ ▷ para cada destino $j$2 c
alcule o caminho de menor custo($i, j$) e custo($i, j$) 3 para $k \in$
caminho de menor custo($i, j$) 4 faça $p_k[i, j] \leftarrow \infty$

HANDLE-UPDATE*(a, j, custo-aj , caminho, pagadores* $ts$)

▷ $a$ é o agente que envia a mensagem (invocando este procedimento).▷ $j$ é o nó de destino.▷ cost-aj é o custo relatado de envio de um pacote de $a$ para $j$.▷ path é uma lista de nós no caminho de menor custo de $a$ a $j$.▷ pagamentos são os pagamentos para cada nó no caminho. 1 modificado $\leftarrow$ FALSE2 se $a \in$ caminho de custo mais baixo($i, j$) ▷ Pai 3 então para $k \in$caminho 4 faça se $p_k[i, j] > p_k[a, j]$5 então $p_k[i, j] \leftarrow p_k[a, j]$6 modificado $\leftarrow$ TRUE7 elseif $i \in$ caminho de custo mais baixo($a, j$) ▷ Filho 8 então para $k \in$ {Todos, exceto o último item no caminho}˜ ˜9 faça se $p_k[i, j] > p_k[a, j] + \theta_a + \tilde{\theta}_i$ ˜10 então $p_k[i, j] \leftarrow p_k[a, j] + \theta_a + \theta_i$11 modificado $\leftarrow$ TRUE 12 else13 $t \leftarrow$ posição do último nó para o qual o caminho é igual ao caminho de menor custo($i, j$) 14 para $k \in$caminho de custo mais baixo($i, j$)$[1 \ldots t]$˜15 faça se $p_k[i, j] > p_k[a, j] + \theta_a +$custo-aj $-$custo($i, j$)˜16 então $p_k[i, j] \leftarrow p_k[a, j] + \theta_a +$custo-aj $-$ custo($i, j$) 17 modificado $\leftarrow$ TRUE18 para $k \in$caminho de custo mais baixo($i, j$)$[t + 1 \ldots]$ ˜ ˜19 faça se $p_k[i, j] > \theta_k + \theta_a +$custo-aj $-$custo($i, j$)˜ ˜20 então $p_k[i, j] \leftarrow \theta_k + \theta_a +$ custo-aj $-$custo($i, j$) 21 modificado $\leftarrow$ TRUE22 se modificado 23 então meus pagamentos $\leftarrow p_k[i, j]$ para $k \in$caminho de custo mais baixo($i, j$) 24 para $b$ é meu vizinho 25 do $b$.HANDLE-UPDATE($i, j$, custo($i, j$), caminho de custo mais baixo$[i, j]$, meus pagamentos)

Figura 8.3: Algoritmo para calcular VCG pagamentos para o i inter- domaini^n derrota i^profissional b - lema. $i$ refere-se ao agente que executa o algoritmo. Todos os agentes comece executando INITIAL-IZE e depois envie um HANDLE-UPDATE para todos os seus vizinhos.

### 8.2.3 Resumo do Projeto do Mecanismo

O projeto do mecanismo nos fornece uma estrutura matemática sólida para expressar muitos problemas de projeto multiagente. Especificamente, podemos representar problemas que envolvem agentes egoístas que são inteligentes o suficiente para mentir se isso au mentar a sua utilidade. Além disso, as equações de Groves-Clarke e de pagamento nos fornecem uma solução pronta para esses problemas. Infelizmente, essa solução só funciona se pudermos implementar pagamentos e, mesmo assim, os pagamentos não são neutros em termos de receita, por isso precisamos de uma quantidade infinita de dinheiro para dar aos agentes. Além disso, estas soluções são centralizadas na medida em que todos os tipos são reportados a um agente central que se encarrega de calcular e distribuir os pagamentos.

O projeto de mecanismos distribuídos é uma nova área de pesquisa: existem poucos algoritmos eficazes e muitos obstáculos são visíveis. Diferentes abordagens estão sendo adotadas para superar esses obstáculos. Uma abordagem é estender o problema de design de mecanismo, definindo mecanismos baseados em confiança, nos quais os agentes levam em conta o grau de confiança que têm uns nos outros ao determinar suas alocações (Dash et al.,

2004). Another approach is to distribute the calculation of VCG payments by using redundantly asking agents to perform sub-problems (Parkes and Shneidman, 2004) and use encryption (Monderer and Tennenholtz, 1999).



2004). Outra abordagem é distribuir o cálculo de VCG pagamentos usando agentes redundantes que solicitam a execução de subproblemas (ParkesandShneidman, 2004) e usam criptografia (Monderer e Tennenholtz, 1999).

Chapter 9

# Coordination Using Goal and Plan Hierarchies

Goal, plan, and policy hierarchies have proven to be very successful methods for coordinating agents. In these approaches we assume the existance of one of these hierarchies and the problem then becomes that of determining which parts of the hierarchy are to be done by which agents. In this setting we assume that the agents are coopertive, that is, they will do exactly what we tell them to. The problem is one of finding an good-enough answer.

Hierarchies like the ones we show here are a way to solve the problem of finding an optimal policy in multiagent MDPs. That is, since the traditional mathematical tools for solving those problems are computationally intractable for real-world problems, researchers hoping to build real-world systems have had to find ways to add more of their domain knowledge into the problem description so as to make it easier to find a solution. In general, the more we tell the agent about the world the easier it is to solve the problems it faces. Unfortunately, if we want to tell the agent more information about its domain we need more sophisticated data structures. In this chapter we describe one of the most popular such data structures: TÆMS. Once we have these new data structures we will generally need new algorithms which take advantage of the new information. The algorithms used for TÆMS are captured in the GPGP framework which we also discuss.

## 9.1 tæms

**tæms** is a language for representing large task hierarchies that contain complex constraints among tasks. You can think of it as a data structure for representing very complicated constraint optimization problems (Lesser et al., 2004). At its simplest, TÆMS represents a goal hierarchy. As such, TÆMS structures are roughly tree-shaped. The root is the top-level goals that we want the system to achieve. The children are the sub-goals that need to be achieved in order for the top goal to be achieved—a form of divide and conquer. The leafs of the tree are either goals that can be achieved by a single agent or tasks that can be done by an agent. These goals and tasks, however, might require the use of some resources or data. TÆMS represents these requirements with an arrow from the data or resource to the goal.

TÆMS

Figure 9.1 shows a simple example TÆMS structure. It tells us that in order to achieve goal $G_0$ we must do $G_1$ and $G_2$ and $G_3$. But, in order to achieve $G_3$ we must achieve either $G_{23}$ or $G_{31}$. Note also how $G_{23}$ is a subgoal of both $G_2$ and $G_3$, which makes this a graph instead of a tree. The bottom row shows two data elements and two resources. Data elements are pieces of data that are needed in order to achieve the goals to which they are connected. Resources are consumable resources that are needed by their particular goals. The difference between data and resource is that a piece of data can be re-used as many times as needed while resources are consumed every time they are used and must therefore be replenished at some point. TÆMS also allows one to annotate links to resources to show how much of the resource is needed in order to achieve a particular goal. We can see that $G_1$ requires $data_1$ while $G_{22}$ requires $data_1$ and $resource_1$.

Figure 9.1 also shows **non local effects**, namely the *enables* constraint. The directed link (in red) from $G_1$ to $G_{22}$ indicates that $G_1$ enables $G_{22}$ which means

NON LOCAL EFFECTS



Capítulo 9

# Coordenação usando hierarquias de metas e planos

As hierarquias de objetivos, planos e políticas provaram ser métodos muito bem-sucedidos para coordenar agentes. Nessas abordagens, assumimos a existência de uma dessas hierarquias e o problema passa a ser determinar quais partes da hierarquia devem ser executadas por quais agentes. Neste cenário assumimos que os agentes são cooperativos, ou seja, farão exatamente o que lhes mandamos. O problema é encontrar uma resposta suficientemente boa.

Hierarquias como as que mostramos aqui são uma forma de resolver o problema de encontrar uma política ótima em MDPs multiagentes. Isto é, uma vez que as ferramentas matemáticas tradicionais para resolver esses problemas são computacionalmente intratáveis para problemas do mundo real, os investigadores que esperam construir sistemas do mundo real tiveram de encontrar formas de adicionar mais do seu conhecimento de domínio à descrição do problema, de modo a tornar mais fácil encontrar uma solução. Em geral, quanto mais contamos ao agente sobre o mundo, mais fácil é resolver os problemas que ele enfrenta. Infelizmente, se quisermos fornecer ao agente mais informações sobre o seu domínio, precisaremos de estruturas de dados mais sofisticadas. Neste capítulo descrevemos uma das estruturas de dados mais populares: TÆMS. Assim que tivermos essas novas estruturas de dados, geralmente precisaremos de novos algoritmos que aproveitem as novas informações. Os algoritmos usados para TÆMS são capturados na estrutura GPGP que também discuti mos.

## 9,1 minutos

tæms é uma linguagem para representar grandes hierarquias de tarefas que contêm tarefas complexas restrições entre tarefas. Você pode pensar nisso como uma estrutura de dados para representar problemas de otimização de restrições muito complicados (Lesser et al., 2004). Na sua forma mais simples,TÆMS representa uma hierarquia de metas. Como tal, as estruturas TÆMS têm aproximadamente o formato de uma árvore. A raiz são as metas de nível superior que queremos que o sistema alcance. Os filhos são os subobjetivos que precisam ser alcançados para que o objetivo principal seja alcançado – uma forma de dividir para conquistar. As folhas da árvore são metas que podem ser alcançadas por um único agente ou tarefas que podem ser realizadas por um agente. Estes objectivos e tarefas, no entanto, podem exigir a utilização de alguns recursos ou dados. TÆMS representa esses requisitos com uma seta que vai dos dados ou recursos até a meta.

TÆMS

A Figura 9.1 mostra um exemplo simples de estrutura TÆMS . Diz-nos que para atingir o objetivo $G_0$ devemos fazer $G_1$ e $G_2$ e $G_3$. Mas, para alcançar $G_3$ , devemos alcançar $G_{23}$ ou $G_{31}$. Observe também como $G_{23}$ é um subobjetivo de $G_2$ e de $G_3$, o que o torna um gráfico em vez de uma árvore. A linha inferior mostra dois elementos de dados e dois recursos. Elementos de dados são pedaços de dados necessários para atingir os objetivos aos quais estão conectados. Recursos são recursos consumíveis necessários para seus objetivos específicos. A diferença entre dados e recursos é que um dado pode ser reutilizado quantas vezes forem necessárias, enquanto os recursos são consumidos sempre que são usados e, portanto, devem ser reabastecidos em algum momento. TÆMStambém permite anotar links para recursos para mostrar quanto do recurso é necessário para atingir um objetivo específico. Podemos ver que $G_1$ requer $data_1$enquanto $G_{22}$ requer $data_1$ e $resource_1$.

A Figura 9.1 também mostra efeitos não locais, nomeadamente a restrição de habilitações. O link direcionado (em vermelho) de $G_1$ para $G_{22}$ indica que $G_1$ ativa $G_{22}$ o que significa

NON LOCAL EFFECTS

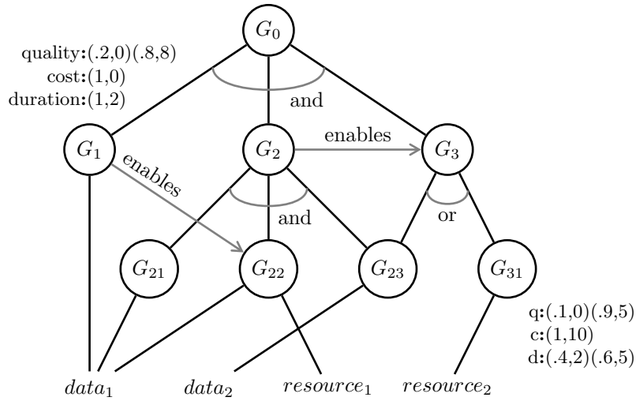



**Figure 9.1:** An example of a simple TÆMS structure.

| Function | Description |
|---|---|
| $q_{min}$ | minimum quality of all subtasks |
| $q_{max}$ | maximum quality of all subtasks |
| $q_{sum}$ | aggregate quality of all subtasks |
| $q_{last}$ | quality of most recently completed subtask |
| $q_{sum_all}$ | as with $q_{sum}$ but all subtasks must be completed |
| $q_{seq_min}$ | as with $q_{min}$ but all subtasks must be completed in order |
| $q_{seq_max}$ | as with $q_{max}$ but all subtasks must be completed in order |

**Figure 9.2:** Some of the quality accumulation functions available for a TÆMS structure.

that $G_1$ must be achieved before we can start to work on $G_{22}$ if we want $G_{22}$ to be achieved with quality.

The TÆMS formal model calls for each leaf goal or task to be described by three parameters: quality, cost, and duration. Figure 9.1 shows examples of these parameter values for only two nodes ($G_1$ and $G_{31}$) due to space constraints. In practice all leaf nodes should have similar values. These parameters represent the quality we expect to get when the goal is achieved (that is, how well it will get done), the cost the system will incur in achieving that goal, and the duration of time required to achieve the goal (or perform the task). The values are probabilistic. For example, $G_{31}$ will with .1 probability be achieved with a quality of 0 (that is, it will fail to get achieved) while with a probability of .9 it will be achieved with quality of 5. $G_{31}$ also has a fixed cost of 10 and it will take either 2 time units to finish, with probability .4, or 5 units to finish, with probability .6.

Figure 9.1 shows two ways of aggregating the quality of children nodes: the and and or functions. These are boolean functions and require us to establish a quality threshold below which we say that a goal has not been achieve. TÆMS usually assumes that a quality of 0 means that the task was not achieved and anything higher means that the task was achieved. For example, $G_0$ will only be achieved if (because of the *and*) $G_1$ is achieved, which only happens with probability .8 each time an agent tries.

QUALITY ACCUMULATION FUNCTIONS

TÆMS allows us to use many different functions besides simply *and* and *or*. Table 9.2 shows some of the **quality accumulation functions** (qafs) that are defined in TÆMS. Of course, the more different qafs we use the harder it will be, in general, to find a good schedule for the agents. Scheduling algorithms must always limit themselves to a subset of qafs in order to achieve realistic running times.



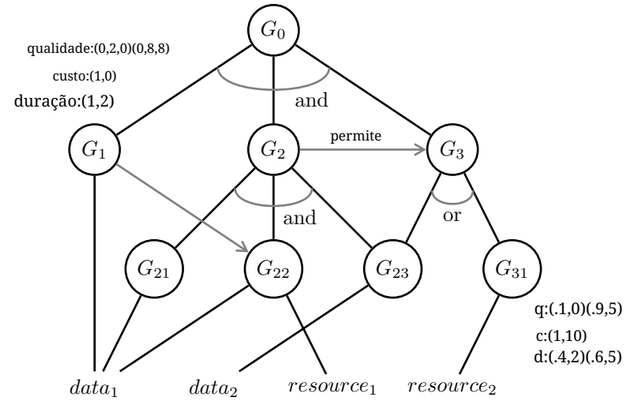



Figura 9.1: Um exemplo de estrutura TÆMS simples.

| Função | Descrição |
|---|---|
| $q_{min}$ | qualidade mínima de todas as subtarefas |
| $q_{max}$ | qualidade máxima de todas as subtarefas |
| $q_{sum}$ | qualidade agregada de todas as subtarefas |
| $q_{last}$ | qualidade da subtarefa concluída mais recentemente |
| $q_{sum_all}$ | como acontece com $q_{sum}$ mas todas as subtarefas devem ser concluídas |
| $q_{seq_min}$ | como acontece com $q_{min}$, mas todas as subtarefas devem ser concluídas em ordem |
| $q_{seq_max}$ | como acontece com $q_{max}$, mas todas as subtarefas devem ser concluídas em ordem |

Figura 9.2: Algumas das funções de acumulação de qualidade disponíveis para uma estrutura TÆMS.

isso $G_1$ deve ser alcançado antes de podermos começar a trabalhar $G_{22}$ se quisermos $G_{22}$ ser alcançado com qualidade.

O modelo formal TÆMS exige que cada meta ou tarefa folha seja descrita por três parâmetros: qualidade, custo e duração. A Figura 9.1 mostra exemplos desses valores de parâmetros para apenas dois nós ($G_1$ e $G_{31}$) devido a restrições de espaço. Na prática, todos os nós folhas devem ter valores semelhantes. Esses parâmetros representam a qualidade que esperamos obter quando a meta for alcançada (isto é, quão bem ela será realizada), o custo que o sistema incorrerá para atingir essa meta e a duração do tempo necessário para atingir a meta (ou executar a tarefa). Os valores são probabilísticos. Por exemplo, $G_{31}$ com probabilidade 0,1 será alcançado com qualidade 0 (ou seja, não será alcançado), enquanto com probabilidade 0,9 será alcançado com qualidade 5. $G_{31}$ também tem um custo fixo de 10 e levará 2 unidades de tempo para terminar, com probabilidade 0,4, ou 5 unidades para terminar, com probabilidade 0,6.

A Figura 9.1 mostra duas maneiras de agregar a qualidade dos nós filhos: as f unções e e ou. Estas são funções booleanas e exigem que estabeleçamos um limite de qualidade abaixo do qual dizemos que um objetivo não foi alcançado. TÆMS geralmente assume que uma qualidade 0 significa que a tarefa não foi alcançada e qualquer valor superior significa que a tarefa foi alcançada. Por exemplo, $G_0$ só será alcançado se (por causa de e) $G_1$ for alcançado, o que só acontece com probabilidade 0,8 cada vez que um agente tenta.

QUALITY ACCUMULATION FUNCTIONS

TÆMS nos permite usar muitas funções diferentes além de simplesmente and e or . A Tabela 9.2 mostra algumas das funções de acumulação de qualidade (qafs) que são definido em TÆMS. É claro que quanto mais qafs diferentes usarmos, mais difícil será, em geral, encontrar um bom cronograma para os agentes. Os algoritmos de escalonamento devem sempre limitar-se a um subconjunto de qafs para atingir tempos de execução realistas.

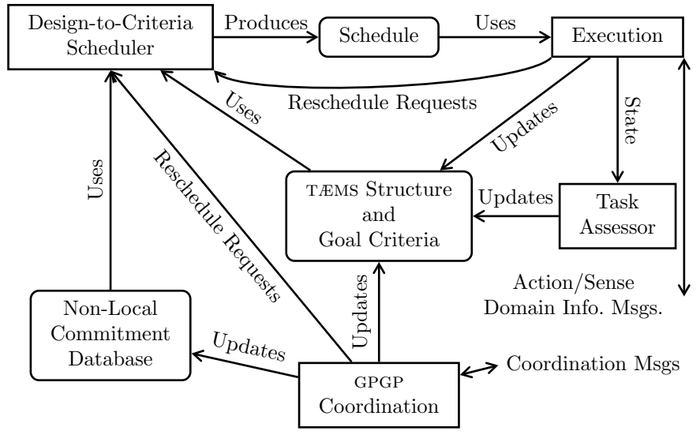



**Figure 9.3:** GPGP agent architecture. Components are in squares and data structures are in rounded squares.

## 9.2 GPGP

Generalized Partial Global Planning (GPGP) is the complete framework for multiagent coordination which includes, as one of its parts the TÆMS data structure. Over the years there have been many GPGP agent implementations each one slightly different from the others but all sharing some basic features. They have been applied to a wide variety of applications, from airplane repair scheduling, to supply chain management, to distributed sensor networks. The TÆMS data structure and the GPGP approach to coordination via hierarchical task decomposition are among the most successful and influential ideas in multiagent systems.

### 9.2.1 Agent Architecture

Figure 9.3 shows a typical agent architecture using GPGP. The red squares are software components. These typically operate in parallel within the agent. The black rounded squares are some of the most important data structures used by a GPGP agent.

The TÆMS data structure lies at the heart of a GPGP agent. This TÆMS structure is also annotated with the goal criteria. That is, it specifies which goals the agent should try to achieve. Remember that the basic TÆMS structure only lists all possible goals, it does not say which agent should try to achieve which goal. The TÆMS structure used by an agent does contain information that tells the agent who is trying to achieve which task. The agent gets this information from the GPGP coordination module which we will explain later.

The design to criteria scheduler is a component (program) which takes as input a TÆMS data structure, along with some commitments the agent has made, and generates a schedule. We will discuss the commitments in more detail later but, basically, the agent has the ability to make commitments to other agents that say that it will achieve a certain goal by a certain time. Once an agent has made a commitment the design to criteria scheduler does everything possible to ensure that the schedule it produces satisfies that commitment. The schedules produced by the scheduler are simply lists of tasks or goals the agent must execute and the times at which the agent will execute them.

The execution module handles plan execution as well as monitoring. That is, it uses the schedule generated by the scheduler and instructs the agent to perform the next task according to the schedule. It then uses the agent's sensors to make sure that the task was accomplished. In the case of a failure, or if it notices that the next task/goal is impossible to achieve in the current world state, it will ask the design to criteria scheduler to generate a new schedule. The execution module also sends



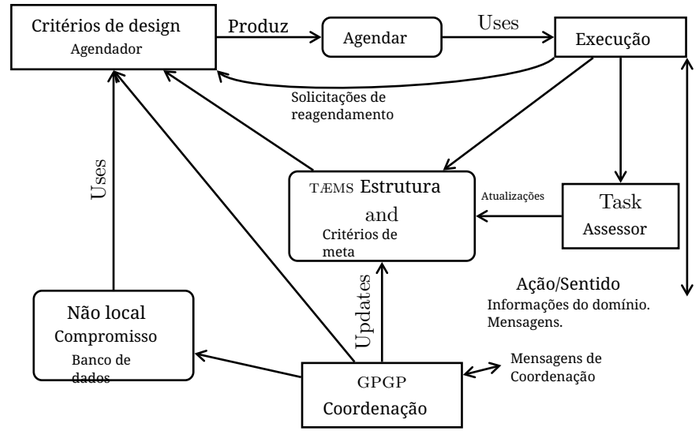



Figura 9.3: GPGP arquitetura de agentes. Os componentes estão em quadrados e as estruturas de dados estão em quadrados arredondados.

## 9.2GPGP

O Planejamento Global Parcial Generalizado (GPGP) é o framework completo para coordenação multiagente que inclui, como uma de suas partes, a estrutura de dados TÆMS . Ao longo dos anos, houve muitas implementações de agentes GPGP, cada uma ligeiramente diferente das outras, mas todas compartilhando alguns recursos básicos. Eles têm sido aplicados a uma ampla variedade de aplicações, desde a programação de reparos de aviões até o gerenciamento da cadeia de suprimentos e redes de sensores distribuídas. A estrutura de dados TÆMS e a abordagem GPGP para coordenação por meio da decomposição hierárquica de tarefas estão entre as ideias mais bem-sucedidas e influentes em sistemas multiagentes.

### 9.2.1 Arquitetura do Agente

A Figura 9.3 mostra uma arquitetura típica de agente usando GPGP. Os quadrados vermelhos são componentes de software. Eles normalmente operam em paralelo dentro do agente. Os quadrados arredondados pretos são algumas das estruturas de dados mais importantes usadas por umGPGP agente.

A estrutura de dados TÆMS está no centro de um agente GPGP . Essa TÆMS estrutura também é anotada com os critérios de meta. Ou seja, especifica quais objetivos o agente deve tentar alcançar. Lembre-se de que a estrutura básica TÆMSlista apenas todos os objetivos possíveis, não diz qual agente deve tentar atingir qual objetivo. A estrutura TÆMS usada por um agente contém informações que informam ao agente quem está tentando realizar qual tarefa. O agente obtém essas informações do módulo de coordenação GPGP que explicaremos mais tarde.

O agendador de design para critérios é um componente (programa) que recebe como entrada uma estrutura de dados TÆMS , junto com alguns compromissos que o agente assumiu, e gera um cronograma. Discutiremos os compromissos com mais detalhes posteriormente, mas, basicamente, o agente tem a capacidade de assumir compromissos co m outros agentes que dizem que atingirá um determinado objetivo em um determinado momento. Depois que um agente assume um compromisso, o escalonador de projeto de acordo com os critérios faz todo o possível para garantir que o cronograma que ele produz satisfaça esse compromisso. Os cronogramas produzidos pelo escalonador são simplesmente listas de tarefas ou objetivos que o agente deve executar e os horários em que o agente irá executá-las.

O módulo de execução cuida da execução do plano e também do monitoramento. Ou s eja, utiliza o agendamento gerado pelo agendador e instrui o agente a executar a próxima tarefa de acordo com o agendamento. Em seguida, usa os sensores do agente para garantir que a tarefa foi concluída. No caso de uma falha, ou se perceber que a próxima tarefa/objetivo é impossível de ser alcançada no estado mundial atual, ele solicitará ao planejador de critérios de design que gere um novo cronograma. O módulo de execução também envia

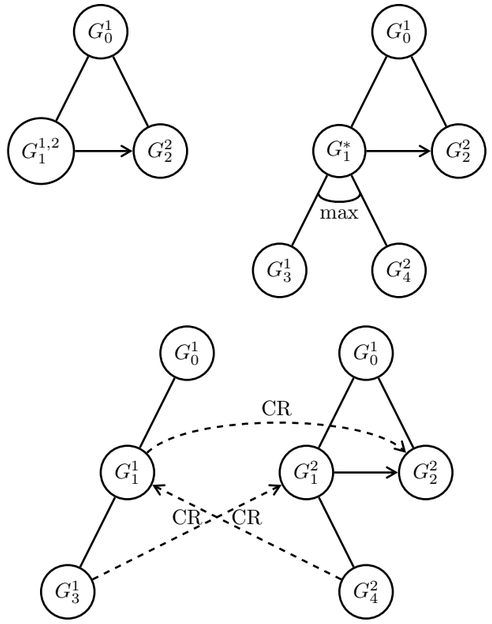



**Figure 9.4:** Example of how GPGP forms coordination relationships. Superscripts on the goals indicate the number of the agents that can achieve that goal. The original TÆMS structure, on the top left, is expanded, top right, and two graphs generated from it and given to agents 1 and 2, bottom. The coordination relationships are then added.

all the information it has about the world and the schedule's execution state to the task assessor module. The task assessor module has domain-specific knowledge and can determine if there are changes that need to be made to the TÆMS structure. For example, it might determine that one of the subgoals is no longer needed because of some specific change in the world. The execution module can also, in a more generalized manner, make changes to the TÆMS structure. These changes are all incorporated into the current TÆMS structure.

### 9.2.2 Coordination

In parallel with all these modules, the GPGP coordination module is always in communication with other agents and tries to keep the agent's commitments and TÆMS structure updated. The GPGP coordination module implements (mostly) domain independent coordination techniques and is the module that tells the agent which of the goals and tasks in its TÆMS structure it should achieve.

Figure 9.4 gives a pictorial view of the general steps the GPGP module follows in order to achieve coordination among agents. The coordination module starts with the TÆMS structure shown on the left. The structure is then modified so that every goal that can be achieved by more than one agent is replaced by a new dummy goal which has one child for every agent that could perform the goal. Each one of the children is a new goal that can only be performed by one child. The children are joined by a max qaf. In Figure 9.4 we see show $G_1^{1,2}$, which could be achieved by either 1 or 2 (as denoted by the superscript) is replaced by a new $G_1$ with two children $G_3^1$ and $G_4^2$.

The resulting graph is then divided among the agents such that each agent has the subgraph with all the goals that the agent can achieve plus all the ancestors of these goals (all the way to the root). This division can be seen in the last two graphs of Figure 9.4.

COORDINATION RELATIONSHIPS

The GPGP coordination module identifies **coordination relationships** (CR). These come from two different situations: when a non-local effect in the original graph now starts in one graph and ends in another, or when a non-local effect



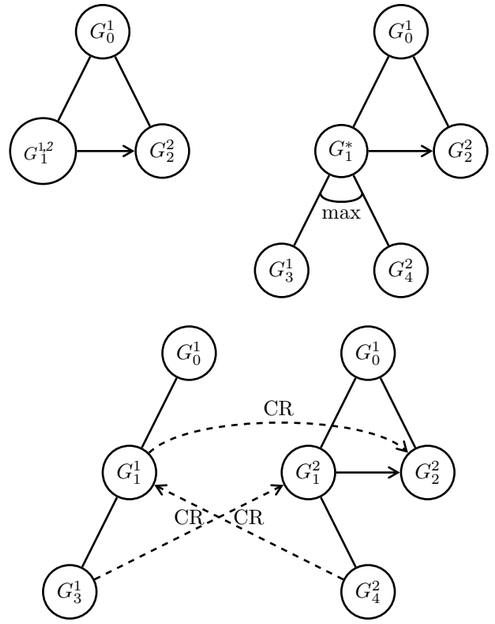



Figura 9.4: Exemplo de como GPGPforma relações de coordenação. Os sobrescritos nas metas indicam o número de agentes que podem atingir essa meta. A estrutura TÆMS original, no canto superior esquerdo, é expandida, no canto superior direito, e dois gráficos gerados a partir dela e fornecidos aos agentes 1 e 2, na parte inferior. As relações de coordenação são então adicionadas.

todas as informações que possui sobre o mundo e o estado de execução do cronograma para o módulo avaliador de tarefas. O módulo avaliador de tarefas tem conhecimento específico do domínio e pode determinar se há alterações que precisam ser feitas na TÆMS estrutura. Por exemplo, pode determinar que um dos subobjetivos não é mais necessário devido a alguma mudança específica no mundo. O módulo de execução também pode, de forma mais generalizada, fazer alterações na estrutura TÆMS . Todas essas mudanças estão incorporadas à estrutura TÆMS atual.

### 9.2.2 Coordenação

Paralelamente a todos esses módulos, o módulo de coordenação GPGP está sempre em comunicação com outros agentes e tenta manter os compromissos e a estrutura TÆMS do agente atualizados. O módulo de coordenação GPGP implementa (principalmente) técnicas de coordenação independentes de domínio e é o módulo que informa ao agente quais objetivos e tarefas em sua estrutura TÆMS ele deve alcançar.

A Figura 9.4 fornece uma visão pictórica das etapas gerais que o módulo GPGP segue para obter coordenação entre os agentes. O módulo de coordenação começa com a estrutura TÆMS mostrada à esquerda. A estrutura é então modificada para que cada objetivo que possa ser alcançado por mais de um agente seja substituído por um novo objetivo fictício que tenha um filho para cada agente que possa realizar o objetivo. Cada uma das crianças é uma nova meta que só pode ser realizada por uma criança. As crianças são acompanhadas por um max qaf. Na Figura 9.4 vemos mostrar $G^1$,2 1 , que poderia ser alcançado por 1 ou 2 (conforme indicado pelo sobrescrito) é substituído por um novo $G_1$ com dois filhos $G_3^1$ e $G_4^2$.

O gráfico resultante é dividido entre os agentes de forma que cada agente tenha o subgrafo com todos os objetivos que o agente pode atingir mais todos os ancestrais desses objetivos (até a raiz). Essa divisão pode ser observada nos dois últimos gráficos da Figura 9.4.

COORDINATION RELATIONSHIPS

O módulo de coordenação GPGP identifica relações de coordenação (CR). Estes vêm de duas situações diferentes: quando um efeito não local no gráfico original agora começa em um gráfico e termina em outro, ou quando um efeito não local

or a subtask relationship has one end in one subgraph but the other end in both subgraphs.

Notice that these CRs denote situations where one of the agents needs to either wait for the other to finish or to determine whether the other one has failed. The GPGP module uses some mechanism to remove the uncertainty that these CRs cause. Specifically, it makes the agent commit to do those actions that are at the initiating end of the CR to the other agent. For example, in Figure 9.4 the GPGP module would make agent 1 send to agent 2 the messages: "Commit ($\mathbf{Do}(G_1)$)" and "Commit ($\mathbf{Do}(G_3)$)".

### 9.2.3 Design-to-Criteria Scheduler

The design-to-criteria scheduler is a complex piece of software which uses search along with a bunch of heuristics to find a good schedule given the current TÆMS structure and the set of commitments. The scheduler only tries to schedule the first few tasks since it knows that it is likely that the problem will change and it will need to re-schedule. The scheduler also sometimes needs to create temporary commitments and generate schedules using those commitments for the GPGP coordination module. That is, when the GPGP coordination module wants to know how a certain commitment might affect the agent's other commitments it will ask the design-to-criteria scheduler to create a new schedule using those commitments.

The job of the scheduler is very complicated. It must decide how to satisfy goals in the TÆMS() structure and which ones are likely to generate the maximum utility. As such, the scheduler is a large piece of code which is generally considered a black box by GPGP users. They just feed it the TÆMS structure and out comes a schedule. The scheduler solves this problem by using a number of search heuristics and other specialized techniques.

### 9.2.4 GPGP/tæms Summary

The GPGP/TÆMS framework utilizes some key concepts. It views the problem of coordination as distributed optimization, in fact, it reduces complex problems to something similar to the distributed constraint optimization problem of Chapter 2. It provides a family of coordination mechanism for situation-specific control. It uses TÆMS to provide proven domain-independent representation of agent tasks, with support for multiple goals of varying worth and different deadline. It also uses quantitative coordination relationships among tasks.

The general approach of GPGP has been widely successful and has proven itself in many domains. The TÆMS structure has been adopted not just as a programming tools but also as a design tool to be used to better understand the problem. However, one problem with GPGP has been that it has required modifications for each one of its applications. As such, GPGP is best thought of as a general approach to designing multiagent systems rather than as a specific tool/architecture.



ou um relacionamento de subtarefa tem uma extremidade em um subgráfico, mas a outra extremidade em ambos os subgráficos.

Observe que esses CRs denotam situações em que um dos agentes precisa esperar que o outro termine ou determinar se o outro falhou. OGPGP módulo usa algum mecanismo para remover a incerteza que esses CRs causam. Especificamente, faz com que o agente se comprometa a realizar as ações que estão no final inicial do CR para o outro agente. Por exemplo, na Figura 9.4 o módulo GPGP faria com que o agente 1 enviasse ao agente 2 as mensagens: "Commit (Do($G_1$))" e "Commit (Do($G_3$))".

#### 9.2.3 Agendador de Design para Critérios

O agendador de design de acordo com critérios é um software complexo que usa pesquisa junto com um monte de heurísticas para encontrar um bom cronograma, dada a estrutura TÆMSatual e o conjunto de compromissos. O agendador tenta agendar apenas as primeiras tarefas, pois sabe que é provável que o problema mude e será necessário reprogramar. Às vezes, o agendador também precisa criar compromissos temporários e gerar cronogramas usando esses compromissos para o módulo de coordenação GPGP . Ou seja, quando o módulo de coordenação GPGP quiser saber como um determinado compromisso pode afetar os outros compromissos do agente, ele solicitará ao escalonador design-to-criteria que crie um novo cronograma usando esses compromissos.

O trabalho do agendador é muito complicado. Ele deve decidir como satisfazer os objetivos na estrutura TÆMS() e quais deles provavelmente gerarão a utilidade máxima. Como tal, o agendador é um grande pedaço de código que geralmente é considerado uma caixa preta pelos usuários GPGP . Eles apenas alimentam a TÆMS estrutura e surge um cronograma. O escalonador resolve esse problema usando uma série de heurísticas de busca e outras técnicas especializadas.

#### 9.2.4 Resumo GPGP/Tæms

A estrutura GPGP/TÆMS utiliza alguns conceitos-chave. Ele vê o problema de coordenação como otimização distribuída; na verdade, reduz problemas complexos a algo semelhante ao problema de otimização de restrição distribuída do Capítulo 2. Ele fornece uma família de mecanismos de coordenação para controle específico de situação. Ele usa TÆMS para fornecer representação comprovada independente de domínio das tarefas do agente, com suporte para vários objetivos também de valor variável e prazos diferentes. Também utiliza relações de coordenação quantitativa entre tarefas.

A abordagem geral de GPGP tem sido amplamente bem-sucedida e comprovada em muitos domínios. A estrutura TÆMS foi adotada não apenas como ferramenta de programação, mas também como ferramenta de design a ser utilizada para melhor compreensão do problema. No entanto, um problema com GPGP é que ele exigiu modificações para cada um de seus aplicativos. Como tal, GPGP é melhor pensado como uma abordagem geral para projetar sistemas multiagentes, em vez de uma f erramenta/arquitetura específica.

# Chapter 10

# Nature-Inspired Approaches

(Abelson et al., 2000), (Nagpal, 2002)

## 10.1 Ants and Termites

(Kube and Bonabeau, 2000), (Bonabeau et al., 1999), (Parunak et al., 2002), (Sauter et al., 2002), (Parunak, 1997)

## 10.2 Immune System

(Kephart and White, 1991), (Kephart, 1994)

## 10.3 Physics

(Cheng et al., 2005)



# Capítulo 10

# Abordagens inspiradas na natureza

(Abelson et al., 2000), (Nagpal, 2002)

## 10.1 Formigas e Cupins

(Kube e Bonabeau, 2000), (Bonabeau et al., 1999), (Parunak et al., 2002), (Sauter et al., 2002), (Parunak, 1997)

## 10.2 Sistema Imunológico

(Kephart e White, 1991), (Kephart, 1994)

## 10.3 Física

(Cheng et al., 2005)

## Bibliography


Abelson, H., Allen, D., Coore, D., Hanson, C., Homsy, G., Knight, T. F., Nagpal, R., Rauch, E., Sussman, G. J., and Weiss, R. (2000). Amorphous computing. *Communications of the ACM*, 43(5):74–82.

Aknine, S., Pinson, S., and Shakun, M. F. (2004). An extended multi-agent negotiation protocol. *Autonomous Agents and Multi-Agent Systems*, 8(1):5–45. doi:10.1023/B:AGNT.0000009409.19387.f8.

Andersson, M. R. and Sandholm, T. (1998). Contract types for satisficing task allocation: Ii experimental results. In *AAAI Spring Symposium: Satisficing Models*.

Arrow, K. J. (1951). *Social choice and individual values*. Yale University Press.

Axelrod, R. M. (1984). *The Evolution of Cooperation*. Basic Books.

Bessière, C., Maestre, A., Brito, I., and Meseguer, P. (2005). Asynchronous backtracking without adding links: a new member in the ABT family. *Artificial Intelligence*, 161:7–24. doi:10.1016/j.artint.2004.10.002.

Binmore, K. and Vulkan, N. (1999). Applying game theory to automated negotiation. *Netnomics*, 1(1):1–9. doi:10.1023/A:1011489402739.

Bomze, I. M. (1986). Noncooperative two-person games in biology: A classification. *International Journal of Game Theory*, 15(1):31–57. doi:10.1007/BF01769275.

Bonabeau, E., Dorigo, M., and Theraulaz, G. (1999). *Swarm Intelligence: From Natural to Artificial Systems*. Oxford.

Brooks, C. H. and Durfee, E. H. (2003). Congregation formation in multiagent systems. *Journal of Autonomous Agents and Multi-agent Systems*, 7(1–2):145–170.

Camerer, C. F., Lowenstein, G., and Rabin, M., editors (2003). *Advances in Behavioral Economics*. Princeton.

Cheng, J., Cheng, W., and Nagpal, R. (2005). Robust and self-repairing formation control for swarms of mobile agents. In *Proceedings of the Twentieth National Conference on Artificial Intelligence*, pages 59–64, Menlo Park, California. AAAI Press.

Clarke, E. H. (1971). Multipart pricing of public goods. *Public Choice*, 11(1). doi:10.1007/BF01726210.

Conen, W. and Sandholm, T. (2001a). Minimal preference elicitation in combinatorial auctions. In *Proceedings of the International Joint Conference of Artificial Intelligence Workshop on Economic Agents, Models, and Mechanisms*.

Conen, W. and Sandholm, T. (2001b). Preference elicitation in combinatorial auctions. In *Proceedings of the Third ACM conference on Electronic Commerce*, pages 256–259, New York, NY, USA. ACM Press. doi:10.1145/501158.501191.




## Bibliografia


Abelson, H., Allen, D., Coore, D., Hanson, C., Homsy, G., Knight, TF, Nagpal, R., Rauch, E., Sussman, GJ, e Weiss, R. (2000). Computação amorfa. Comunicações da ACM, 43(5):74–82. Aknine, S., Pinson, S. e Shakun, MF (2004). Um protocolo estendido de negociação multiagente. Agentes Autônomos e Sistemas Multiagentes, 8(1):5–45. doi:10.1023/B:AGNT.0000009409.19387.f8. Andersson, MR e Sandholm, T. (1998). Tipos de contratos para atribuição satisfatória de tarefas: Ii resultados experimentais. No Simpósio AAAI Spring: Modelos Satisfazendo. Arrow, KJ (1951). Escolha social e valores individuais. Imprensa da Universidade de Yale. Axelrod, RM (1984). A evolução da cooperação. Livros Básicos. Bessi`ere, C., Maestre, A., Brito, I., e Meseguer, P. (2005). Retrocesso assíncrono sem adição de links: um novo membro na família ABT. Inteligência Artificial, 161:7–24. doi:10.1016/j.artint.2004.10.002. Binmore, K. e Vulkan, N. (1999). Aplicando a teoria dos jogos à negociação automatizada. Netnomia, 1(1):1–9. doi:10.1023/A:1011489402739. Bomze, IM (1986). Jogos não cooperativos para duas pessoas em biologia: uma classificação. Jornal Internacional de Teoria dos Jogos, 15(1):31–57. doi:10.1007/ BF01769275. Bonabeau, E., Dorigo, M. e Theraulaz, G. (1999). Inteligência de enxame: dos sistemas naturais aos artificiais. Oxford. Brooks, CH e Durfee, EH (2003). Formação de congregações em sistemas multiagentes. Journal of Autonomous Agents and Multi-agent Systems, 7(1–2):145–170. Camerer, CF, Lo wenstein, G., e Rabin, M., editores (2003). Avanços na Economia Comportamental. Princeton. Cheng, J., Cheng, W. e Nagpal, R. (2005). Controle de formação robusto e autorreparável para enxames de agentes móveis. Em Proceedings of the Twentieth National Conference on Artificial Intelligence, páginas 59–64, Menlo Park, Califórnia. Imprensa AAAI. Clarke, EH (1971). Preços multiparte de bens públicos. Escolha Pública, 11(1). doi:10.1007/BF01726210. Conen, W. e Sandholm, T. (2001a). Elicitação de preferência mínima em leilões combinatórios. Nos Anais da Conferência Conjunta Internacional de Workshop de Inteligência Artificial sobre Agentes Econômicos, Modelos e Mecanismos. Conen, W. e Sandholm, T. (2001b). Elicitação de preferências e m leilões combinatórios. Em Anais da Terceira Conferência ACM sobre Comércio Eletrônico, páginas 256–259, Nova York, NY, EUA. Imprensa ACM. doi:10.1145/501158. 501191.

Conen, W. and Sandholm, T. (2002). Partial-revelation VCG mechanism for combinatorial auctions. In *Proceedings of the National Conference on Artificial Intelligence*, pages 367–372.

Conitzer, V. and Sandholm, T. (2003). AWESOME: A general multiagent learning algorithm that converges in self-play and learns a best response against stationary opponents. In *Proceedings of the Twentieth International Conference on Machine Learning*.

Conitzer, V. and Sandholm, T. (2006). AWESOME: A general multiagent learning algorithm that converges in self-play and learns a best response against stationary opponents. *Machine Learning*. doi:10.1007/s10994-006-0143-1.

Cramton, P., Shoham, Y., and Steinberg, R., editors (2006). *Combinatorial Auctions*. MIT Press.

Dang, V. D. and Jennings, N. (2004). Generating coalition structures with finite bound from the optimal guarantees. In *Proceedings of the Third International Joint Conference on Autonomous Agents and MultiAgent Systems*, pages 564–571. ACM.

Dash, R., Ramchurn, S., and Jennings, N. (2004). Trust-based mechanism design. In *Proceedings of the Third International Joint Conference on Autonomous Agents and MultiAgent Systems*, pages 748–755. ACM.

Dash, R. K., Jennings, N. R., and Parkes, D. C. (2003). Computational mechanism design: A call to arms. *IEEE Intelligent Systems*, 18(6):40–47. doi:10.1041/X6040s-2003.

Davin, J. and Modi, P. J. (2005). Impact of problem centralization in distributed constraint optimization algorithms. In (Dignum et al., 2005), pages 1057–1066.

Davis, R. and Smith, R. G. (1983). Negotiation as a metaphor for distributed problem solving. *Artificial Intelligence*, 20:63–109.

Delgado, J. (2002). Emergence of social conventions in complex networks. *Artificial Intelligence*, 141:171–185. doi:10.1016/S0004-3702(02)00262-X.

Dignum, F., Dignum, V., Koenig, S., Kraus, S., Pechoucek, M., Singh, M., Steiner, D., Thompson, S., and Wooldridge, M., editors (2005). *Proceedings of the Fourth International Joint Conference on Autonoumous Agents and Multi Agent Systems*, New York, NY. ACM Press.

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271.

Durfee, E. H. (1999). Practically coordinating. *AI Magazine*, 20(1):99–116.

Durfee, E. H. and Yokoo, M., editors (2007). *Proceedings of the Sixth International Joint Conference on Autonomous Agents and Multiagent Systems*.

Endriss, U., Maudet, N., Sadri, F., and Toni, F. (2006). Negotiating socially optimal allocations of resources. *Journal of Artificial Intelligence Research*, 25:315–348.

Fagin, R., Halpern, J. Y., Moses, Y., and Vardi, M. Y. (1995). *Reasoning About Knowledge*. The MIT Press, Cambridge, MA.

Faratin, P., Sierra, C., and Jennings, N. R. (1998). Negotiation decision functions for autonomous agents. *Robotics and Autonomous Systems*, 24(3–4):159–182. doi:10.1016/S0921-8890(98)00029-3.



Conen, W. e Sandholm, T. (2002). Mecanismo VCG de revelação parcial para leilões combinatórios. Em Anais da Conferência Nacional sobre Inteligência Artificial, páginas 367–372. Conitzer, V. e Sandholm, T. (2003). INCRÍVEL: Um algoritmo geral de aprendizado multiagente que converge em autojogo e aprende a melhor resposta c ontra oponentes estacionários. Nos Anais da Vigésima Conferência Internacional sobre Aprendizado de Máquina. Conitzer, V. e Sandholm, T. (2006). INCRÍVEL: Um algoritmo geral de aprendizado multiagente que converge em autojogo e aprende a melhor resposta contra oponentes estacionários. Aprendizado de máquina. doi:10.1007/s10994-006-0143-1. Cramton, P., Shoham, Y. e Steinberg, R., editores (2006). Leilões Combinatórios. Imprensa do MIT. Dang, VD e Jennings, N. (2004). Geração de estruturas de coalizão com limites finitos a partir das garantias ótimas. Em Anais da Terceira Conferência Conjunta Internacional sobre Agentes Autônomos e Sistemas MultiAgentes, páginas 564–571. ACM. Dash, R., Ramchurn, S. e Jennings, N. (2004). Projeto de mecanismo baseado em confiança. Nos Anais da Terceira Conferência Conjunta Internacional sobre Agentes Autônomos e Sistemas MultiAgentes, páginas 748–755. ACM. Dash, RK, Jennings, NR e Parkes, DC (2003). Projeto de mecanismo computacional: um chamado às armas. Sistemas Inteligentes IEEE, 18(6):40–47. doi:10.1041/ X6040s-2003. Davin, J. e Modi, PJ (2005). Impacto da centralização de problemas em algoritmos de otimização de restrições distribuídas. Em (Dignum et al., 2005), páginas 1057–1066. Davis, R. e Smith, RG (1983). A negociação como metáfora para a resolução distribuída de problemas. Inteligência Artificial, 20:63–109. Delgado, J. (2002). Emergência de convenções sociais em redes complexas. Inteligência Artificial, 141:171–185. doi:10.1016/S0004-3702(02)00262-X. Dignum, F., Dignum, V., Koenig, S., Kraus, S., Pechoucek, M., Singh, M., Steiner, D., Thompson, S., e Wooldridge, M., editores (2005). Anais da Quarta Conferência Conjunta Internacional sobre Agentes Autônomos e Sistemas Multiagentes, Nova York, NY. Imprensa ACM. Dijkstra, EW (1959). Uma nota sobre dois problemas relacionados com gráficos. Numerische Mathematik, 1:269–271. Durfee, EH (1999). Praticamente coordenando. Revista AI, 20(1):99–116. Durfee, EH e Yokoo, M., editores (2007). Anais da Sexta Conferência Conjunta Internacional sobre Agentes Autônomos e Sistemas Multiagentes. Endriss, U., Maudet, N., Sadri, F. e Toni, F. (2006). Negociar alocações de recursos socialmente ideais. Jornal de Pesquisa em Inteligência Artificial, 25:315–348. Fagin, R., Halpern, JY, Moses, Y. e Vardi, MY (1995). Raciocínio sobre o conhecimento. The MIT Press, Cambridge, MA. Faratin, P., Sierra, C. e Jennings, NR (1998). Funções de decisão de negociação para agentes autônomos. Robótica e Sistemas Autônomos, 24(3–4):159–182. doi:10.1016/S0921-8890(98)00029-3.

Fatima, S. S., Wooldridge, M., and Jennings, N. (2004). Optimal negotiation of multiple issues in incomplete information settings. In *Proceedings of the Third International Joint Conference on Autonomous Agents and MultiAgent Systems*, pages 1080–1089. ACM.

Feigenbaum, J., Papadimitriou, C., Sami, R., and Shenker, S. (2005). A BGP-based mechanism for lowest-cost routing. *Distributed Computing*. `doi:10.1007/s00446-005-0122-y`.

Feigenbaum, J., Papadimitriou, C. H., and Shenker, S. (2001). Sharing the cost of multicast transmissions. *Journal of Computer and System Sciences*, 63(1):21–41. `doi:10.1006/jcss.2001.1754`.

Fudenberg, D. and Kreps, D. (1990). Lectures on learning and equilibrium in strategic-form games. Technical report, CORE Lecture Series.

Fudenberg, D. and Levine, D. K. (1998). *The Theory of Learning in Games*. MIT Press.

Fujishima, Y., Leyton-Brown, K., and Shoham, Y. (1999). Taming the computational complexity of combinatorial auctions: Optimal and approximate approaches. In *Proceedings of the Sixteenth International Joint Conference on Artificial Intelligence*, pages 548–553. Morgan Kaufmann Publishers Inc.

Gmytrasiewicz, P. J. and Durfee, E. H. (1995). A rigorous, operational formalization of recursive modeling. In *Proceedings of the First International Conference on Multi-Agent Systems*, pages 125–132. AAAI/MIT press.

Gmytrasiewicz, P. J. and Durfee, E. H. (2001). Rational communication in multi-agent systems. *Autonomous Agents and Multi-Agent Systems Journal*, 4(3):233–272.

Goradia, H. J. and Vidal, J. M. (2007). An equal excess negotiation algorithm for coalition formation. In *Proceedings of the Autonomous Agents and Multi-Agent Systems Conference*.

Groves, T. (1973). Incentives in teams. *Econometrica*, 41(4):617–631.

Harford, T. (2005). *The Undercover Economist: Exposing Why the Rich Are Rich, the Poor Are Poor–and Why You Can Never Buy a Decent Used Car!* Oxford University Press.

Harsanyi, J. C. (1965). Approaches to the bargaining problem before and after the theory of games: A critical discussion of zeuthen's, hicks', and nash's theories. *Econometrica*, 24(2):144–157.

Hirayama, K. and Yokoo, M. (2005). The distributed breakout algorithms. *Artificial Intelligence*, 161(1–2):89–115. `doi:10.1016/j.artint.2004.08.004`.

Hu, J. and Wellman, M. P. (2003). Nash q-learning for general-sum stochastic games. *Journal of Machine Learning Research*, 4:1039–1069.

Hudson, B. and Sandholm, T. (2004). Effectiveness of query types and policies for preference elicitation in combinatorial auctions. In (Jennings et al., 2004).

Jennings, N. R., Faratin, P., Lomuscio, A. R., Parsons, S., Wooldridge, M., and Sierra, C. (2001). Automated negotiation: Prospects methods and challenges. *Group Decision and Negotiation*, 10(2):199–215.

Jennings, N. R., Sierra, C., Sonenberg, L., and Tambe, M., editors (2004). *Proceedings of the Third International Joint Conference on Autonoumous Agents and Multi Agent Systems*, New York, NY. ACM Press.



Fátima, SS, Wooldridge, M. e Jennings, N. (2004). Negociação ideal de múltiplos problemas em configurações de informações incompletas. Em Proceedings of the Third International Joint Conference on Autonomous Agents and MultiAgent Systems, páginas 1080–1089. ACM. Feigenbaum, J., Papadimitriou, C., Sami, R. e Shenker, S. (2005). Um mecanismo baseado em BGP para roteamento de menor custo. Computação Distribuída. faça:10. 1007/s00446-005-0122-y. Feigenbaum, J., Papadimitriou, CH e Shenker, S. (2001). Compartilhando o custo das transmissões multicast. Jornal de Ciências da Computação e Sistemas, 63(1):21– 41. doi:10.1006/jcss.2001.1754. Fudenberg, D. e Kreps, D. (1990). Aulas sobre aprendizagem e equilíbrio em jogos de forma estratégica. Relatório técnico, série de palestras CORE. Fudenberg, D. e Levine, DK (1998). A Teoria da Aprendizagem em Jogos. Imprensa do MIT. Fujishima, Y., Leyton-Brown, K. e Shoham, Y. (1999). Domando a complexidade computacional dos leilões combinatórios: abordagens ótimas e aproximadas. Nos Anais da Décima Sexta Conferência Conjunta Internacional sobre Inteligência Artificial, páginas 548–553. Morgan Kaufmann Publishers Inc. Gmytrasiewicz, PJ e Durfee, EH (1995). Uma formalização rigorosa e operacional de modelagem recursiva. Em Anais da Primeira Conferência Internacional sobre Sistemas Multiagentes, páginas 125–132. Imprensa AAAI/MIT. Gmytrasiewicz, PJ e Durfee, EH (2001). Comunicação racional em sistemas multiagentes. Jornal de Agentes Autônomos e Sistemas Multiagentes, 4(3):233–272. Goradia, HJ e Vidal, JM (2007). Um algoritmo de negociação de excesso igual para formação de coalizões. Nos Anais da Conferência de Agentes Autônomos e Sistemas Multiagentes. Groves, T. (1973). Incentivos nas equipes. Econometrica, 41(4):617–631. Harford, T. (2005). O economista disfarçado: expondo por que os ricos são ricos, os pobres são pobres - e por que você nunca poderá comprar um carro usado decente! Imprensa da Universidade de Oxford. Harsanyi, JC (1965). Abordagens ao problema da negociação antes e depois da teoria dos jogos: uma discussão crítica das teorias de Zeuthen, Hicks e Nash. Econometrica, 24(2):144–157. Hirayama, K. e Yokoo, M. (2005). Os algoritmos de breakout distribuídos. Inteligência Artificial, 161(1–2):89–115. doi:10.1016/j.artint.2004.08.004. Hu, J. e Wellman, MP (2003). Nash q-learning para jogos estocásticos de soma geral. Jornal de Pesquisa de Aprendizado de Máquina, 4:1039–1069. Hudson, B. e Sandholm, T. (2004). Eficácia de tipos de consulta e políticas para elicitação de preferências em leilões combinatórios. Em (Jennings et al., 2004). Jennings, NR, Faratin, P., Lomuscio, AR, Parsons, S., Wooldridge, M. e Sierra, C. (2001). Negociação automatizada: métodos e desafios de perspectivas. Decisão e Negociação do Grupo, 10(2):199–215. Jennings, NR, Sierra, C., Sonenberg, L. e Tambe, M., editores (2004). Anais da Terceira Conferência Conjunta Internacional sobre Agentes Autônomos e Sistemas Multiagentes, Nova York, NY. Imprensa ACM.

Jiang, H. and Vidal, J. M. (2005). Reducing redundant messages in the asynchronous backtracking algorithm. In *Proceedings of the Sixth International Workshop on Distributed Constraint Reasoning*.

Jiang, H. and Vidal, J. M. (2006). The message management asynchronous backtracking algorithm. *Journal of Experimental and Theoretical Artificial Intelligence*. To appear.

Kalai, E. and Smorodinsky, M. (1975). Other solutions to nash's bargaining problem. *Econometrica*, 43:513–518.

Kelly, F. and Stenberg, R. (2000). A combinatorial auction with multiple winners for universal service. *Management Science*, 46(4):586–596.

Kephart, J. O. (1994). A biologically inspired immune system for computers. In Brooks, R. A. and Maes, P., editors, *Proceedings of the Fourth International Workshop on Synthesis and Simulatoin of Living Systems*, pages 130–139. MIT Press, Cambridge, MA.

Kephart, J. O. and White, S. R. (1991). Directed-graph epidemiological models of computer viruses. In *Proceedings of the IEEE Computer Society Symposium on Research in Security and Privacy*, pages 343–359. doi:10.1109/RISP.1991.130801.

Klein, M., Faratin, P., Sayama, H., and Bar-Yam, Y. (2003). Negotiating complex contracts. *Group Decision and Negotiation*, 12:111–125. doi:10.1023/A:1023068821218.

Kraus, S. (2001). *Strategic Negotiation in Multiagent Environments*. MIT Press.

Kube, C. R. and Bonabeau, E. (2000). Cooperative transport by ants and robots. *Robotics and Autonomous Systems*, pages 85–101.

Lesser, V., Decker, K., Wagner, T., Carver, N., Garvey, A., Horling, B., Neiman, D., Podorozhny, R., Prasad, M. N., Raja, A., Vincent, R., Xuan, P., and Zhang, X. Q. (2004). Evolution of the GPGP/TAEMS domain-independent coordination framework. *Autonomous Agents and Multi-Agent Sytems,*, 9:87–143. doi:10.1023/B:AGNT.0000019690.28073.04.

Leyton-Brown, K., Pearson, M., and Shoham, Y. (2000). Towards a universal test suite for combinatorial auction algorithms. In *Proceedings of the 2nd ACM conference on Electronic commerce*, pages 66–76. ACM Press. http://cats.stanford.edu, doi:10.1145/352871.352879.

Littman, M. L. (2001). Friend-or-foe q-learning in general-sum games. In *Proceedings of the Eighteenth International Conference on Machine Learning*, pages 322–328. Morgan Kaufmann.

MacKie-Mason, J. K. and Varian, H. R. (1994). Generalized vickrey auctions. Technical report, University of Michigan.

Mailler, R. T. and Lesser, V. (2004a). Solving distributed constraint optimization problems using cooperative mediation. In (Jennings et al., 2004), pages 438–445.

Mailler, R. T. and Lesser, V. (2004b). Using cooperative mediation to solve distributed constraint satisfaction problems. In (Jennings et al., 2004), pages 446–453.

Mas-Colell, A., Whinston, M. D., and Green, J. R. (1995). *Microeconomic Theory*. Oxford.



Jiang, H. e Vidal, JM (2005). Reduzindo mensagens redundantes no algoritmo de retrocesso assíncrono. Nos Anais do Sexto Workshop Internacional sobre Raciocínio de Restrições Distribuídas. Jiang, H. e Vidal, JM (2006). O algoritmo de backtracking assíncrono de gerenciamento de mensagens. Journal of Experimental and Theoretical Artificial Intelligence. Para aparecer. Kalai, E. e Smorodinsky, M. (1975). Outras soluções para o problema de negociação de Nash. Econometrica, 43:513–518. Kelly, F. e Stenberg, R. (2000). Um leilão combinatório com múltiplos vencedores para o serviço universal. Ciência de Gestão, 46(4):586–596. Kephart, JO (1994). Um sistema imunológico de inspiração biológica para computadores. Em Brooks, R. A. e Maes, P., editores, Proceedings of the Fourth International Workshop on Synthesis and Simulatoin of Living Systems, páginas 130–139. MIT Press, Cambridge, MA. Kephart, JO e White, SR (1991). Modelos epidemiológicos de gráficos direcionados de vírus de computador. Em Anais do Simpósio da IEEE Computer Society sobre Pesquisa em Segurança e Privacidade, páginas 343–359. doi:10.1109/RISP.1991. 130801. Klein, M., Faratin, P., Sayama, H. e Bar-Yam, Y. (2003). Negociação de contratos complexos. Decisão e Negociação do Grupo, 12:111–125. doi:10.1023/A: 1023068821218. Kraus, S. (2001). Negociação Estratégica em Ambientes Multiagentes. Imprensa do MIT. Kube, CR e Bonabeau, E. (2000). Transporte cooperativo por formigas e robôs. Robótica e Sistemas Autônomos, páginas 85–101. Lesser, V., Decker, K., Wagner, T., Carver, N., Garvey, A., Horling, B., Neiman, D., Podorozhny, R., Prasad, MN, Raja, A., Vincent, R., Xuan, P., e Zhang, XQ (2004). Evolução da estrutura de coordenação independente de domínio GPGP/TAEMS. Agentes Autônomos e Sistemas Multiagentes, 9:87–143. doi:10.1023/B:AGNT.0000019690.28073.04. Leyton-Brown, K., Pearson, M. e Shoham, Y. (2000). Rumo a um conjunto de testes universal para algoritmos combinatórios de leil ão. Nos Anais da 2ª Conferência ACM sobre Comércio Eletrônico, páginas 66–76. Imprensa ACM. http://gatos. stanford.edu, doi:10.1145/352871.352879. Littman, ML (2001). Q-learning amigo ou inimigo em jogos de soma geral. Em Anais da Décima Oitava Conferência Internacional sobre Aprendizado de Máquina, páginas 322–328. Morgan Kaufmann. MacKie-Mason, JK e Varian, HR (1994). Leilões generalizados de Vicrey. Relatório técnico, Universidade de Michigan. Mailler, RT e Lesser, V. (2004a). Resolvendo problemas de otimização de restrições distribuídas usando mediação cooperativa. Em (Jennings et al., 2004), páginas 438–445. Mailler, RT e Lesser, V. (2004b). Usando mediação cooperativa para resolver problemas de satisfação de restrições distribuídas. Em (Jennings et al., 2004), páginas 446–453. Mas-Colell, A., Whinston, MD e Green, JR (1995). Teoria Microeconômica. Oxford.

McKelvey, R. D., McLennan, A. M., and Turocy, T. L. (2006). Gambit: Software tools for game theory. Technical report, Version 0.2006.01.20.

Mendoza, B. and Vidal, J. M. (2007). Bidding algorithms for a distributed combinatorial auction. In *Proceedings of the Autonomous Agents and Multi-Agent Systems Conference*.

Minton, S., Johnston, M. D., Philips, A. B., and Laird, P. (1992). Minimizing conflicts: a heuristic repair method for constraint satisfaction and scheduling problems. *Artificial Intelligence*, 58:161–205. doi:10.1016/0004-3702(92)90007-K.

Mitchell, T. M. (1997). *Machine Learning*. McGraw Hill.

Modi, P. J., Shen, W.-M., Tambe, M., and Yokoo, M. (2005). Adopt: Asynchronous distributed constraint optimization with quality guarantees. *Artificial Intelligence*, 16(1–2):149–180. doi:10.1016/j.artint.2004.09.003.

Monderer, D. and Tennenholtz, M. (1999). Distributed games: From mechanisms to protocols. In *Proceedings of the Sixteenth National Conference on Artificial Intelligence*, pages 32–37. AAAI Press. Menlo Park, Calif.

Muthoo, A. (1999). *Bargaining Theory with Applications*. Cambridge University Press.

Nagpal, R. (2002). Programmable self-assembly using biologically-inspired multiagent control. In *Proceedings of the 1st International Joint Conference on Autonomous Agents and Multi-Agent Systems*.

Nash, J. F. (1950). The bargaining problem. *Econometrica*, 18:155–162.

Neumann, J. V. (1928). Zur theorie der gesellschaftsspiele. *Mathematische Annalen*, 100(1):295–320. doi:10.1007/BF01448847.

Neumann, J. V. and Morgenstern, O. (1944). *Theory of Games and Economic Behavior*. Princeton University Press.

nevar, C. C., McGinnis, J., Modgil, S., Rahwan, I., Reed, C., Simari, G., South, M., Vreeswijk, G., and Willmott, S. (2006). Towards and argument interchange format. *The Knowledge Engineering Review*, 21(4):293–316. doi:10.1017/S0269888906001044.

Nguyen, T. D. and Jennings, N. (2004). Coordinating multiple concurrent negotiations. In *Proceedings of the Third International Joint Conference on Autonomous Agents and MultiAgent Systems*, pages 1064–1071. ACM.

Nisan, N. (2000). Bidding and allocation in combinatorial auctions. In *Proceedings of the ACM Conference on Electronic Commerce*, pages 1–12.

Nisan, N. and Ronen, A. (2001). Algorithmic mechanism design. *Games and Economic Behavior*, 35:166–196. doi:10.1006/game.1999.0790.

Nudelman, E., Wortman, J., Leyton-Brown, K., and Shoham, Y. (2004). Run the GAMUT: A comprehensive approach to evaluating game-theoretic algorithms. In (Jennings et al., 2004).

Osborne, M. J. and Rubinstein, A. (1990). *Bargaining and Markets*. Academic Press.

Osborne, M. J. and Rubinstein, A. (1999). *A Course in Game Theory*. MIT Press.

Parkes, D. C. and Shneidman, J. (2004). Distributed implementations of vickrey-clarke-groves auctions. In *Proceedings of the Third International Joint Conference on Autonomous Agents and MultiAgent Systems*, pages 261–268. ACM.



McKelvey, RD, McLennan, AM e Turocy, TL (2006). Gambit: Ferramentas de software para teoria dos jogos. Relatório técnico, versão 0.2006.01.20. Mendoza, B. e Vidal, JM (2007). Algoritmos de licitação para um leilão combinatório distribuído. Nos Anais da Conferência de Agentes Autônomos e Sistemas Multiagentes. Minton, S., Johnston, MD, Philips, AB e Laird, P. (1992). Minimizando conflitos: um método heurístico de reparo para satisfação de restrições e problemas de escalonamento. Inteligência Artificial, 58:161–205. doi:10.1016/0004-3702(92) 90007-K. Mitchell, TM (1997). Aprendizado de máquina. McGraw Hill. Modi, PJ, Shen, W.-M., Tambe, M. e Yokoo, M. (2005). Adotar: Otimização de restrições distribuídas assíncronas com garantias de qualidade. Inteligência Artificial, 16(1–2):149–180. doi:10.1016/j.artint.2004.09.003. Monderer, D. e Tennenholtz, M. (1999). Jogos distribuídos: Dos mecanismos aos protocolos. Nos Anais da Décima Sexta Conferência Nacional sobre Inteligência Artificial, páginas 32–37. Imprensa AAAI. Menlo Park, Califórnia. Muthoo, A. (1999). Teoria da Negociação com Aplicações. Imprensa da Universidade de Cambridge. Nagpal, R. (2002). Automontagem programável usando controle multiagente de inspiração biológica. Nos Anais da 1ª Conferência Conjunta Internacional sobre Agentes Autônomos e Sistemas Multiagentes. Nash, JF (1950). O problema da negociação. Econometrica, 18:155–162. Neumann, JV (1928). Zur theorie der gesellschaftsspiele. Mathematische Annalen, 100(1):295–320. doi:10.1007/BF01448847. Neumann, JV e Morgenstern, O. (1944). Teoria dos Jogos e Comportamento Econômico. Imprensa da Universidade de Princeton. nevar, CC, McGinnis, J., Modgil, S., Rahwan, I., Reed, C., Simari, G., South, M., Vreeswijk, G., e Willmott, S. (2006). Formato de intercâmbio de argumentos. A Revisão de Engenharia do Conhecimento, 21(4):293–316. doi:10.1017/ S0269888906001044. Nguyen, TD e Jennings, N. (2004). Coordenar múltiplas negociações simultâneas. Em Anais da Terceira Conferência Conjunta Internacional sobre Agentes Autônomos e Sistemas MultiAgentes, páginas 1064–1071. ACM. Nissan, N. (2000). Licitação e alocação em leil ões combinatórios. Nos Anais da Conferência ACM sobre Comércio Eletrônico, páginas 1–12. Nisan, N. e Ronen, A. (2001). Projeto de mecanismo algorítmico. Games and Economic Behavior, 35:166–196. doi:10.1006/game.1999.0790. Nudelman, E., Wortman, J., Leyton-Brown, K. e Shoham, Y. (2004). Execute o GAMUT: uma abordagem abrangente para avaliar algoritmos de teoria de jogos. Em (Jennings et al., 2004). Osborne, MJ e Rubinstein, A. (1990). Negociação e Mercados. Imprensa Acadêmica. Osborne, MJ e Rubinstein, A. (1999). Um curso de teoria dos jogos. Imprensa do MIT. Parkes, DC e Shneidman, J. (2004). Implementações distribuídas de leilões Vickrey-Clarke-groves. E m Anais da Terceira Conferência Internacional Conjunta sobre Agentes Autônomos e Sistemas MultiAgentes, páginas 261–268. ACM.

Parunak, H. V. D. (1997). “go to the ant”: Engineering principles from natural agent systems. *Annals of Operation Research*, 75:69–101.

Parunak, H. V. D., Brueckner, S., and Sauter, J. (2002). Synthetic pheromone mechanisms for coordination of unmanned vehicles. In *Proceedings of the First Intenational Joint Conference on Autonomous Agents and Multiagent Systems*, pages 448–450, Bologna, Italy. ACM Press, New York, NY.

Rahwan, I., Ramchurn, S. D., Jennings, N. R., McBurney, P., Parsons, S., and Sonenberg, L. (2004). Argumentation-based negotiation. *The Knowledge Engineering Review*, 18(4):343–375. doi:10.1017/S0269888904000098.

Resnick, M. (1994). *Turtles, Termites, and Traffic Jams: Explorations in Massively Parallel Microworlds*. MIT Press.

Rosenschein, J. S. and Zlotkin, G. (1994). *Rules of Encounter*. The MIT Press, Cambridge, MA.

Rothkopf, M. H., Pekec, A., and Harstad, R. M. (1998). Computationally manageable combinational auctions. *Management Science*, 44(8):1131–1147.

Rubinstein, A. (1982). Perfect equilibrium in a bargaining model. *Econometrica*, 50(1):97–110.

Russell, S. and Norvig, P. (2003). *Artificial Intelligence: A Modern Approach*. Prentice Hall, second edition.

Sandholm, T. (1997). Necessary and sufficient contract types for optimal task allocation. In *Proceedings of the Fourteenth International Joint Conference on Artificial Intelligence*.

Sandholm, T. (1998). Contract types for satisficing task allocation: I theoretical results. In *AAAI Spring Symposium: Satisficing Models*.

Sandholm, T. (1999). An algorithm for winner determination in combinatorial auctions. In *Proceedings of the Sixteenth International Joint Conference on Artificial Intelligence*, pages 542–547.

Sandholm, T. (2002). An algorithm for winner determination in combinatorial auctions. *Artificial Intelligence*, 135(1-2):1–54. doi:10.1016/S0004-3702(01)00159-X.

Sandholm, T., Larson, K., Anderson, M., Shehory, O., and Tohmé, F. (1999). Coalition structure generation with worst case guarantees. *Artificial Intelligence*, 111(1-2):209–238. doi:10.1016/S0004-3702(99)00036-3.

Sandholm, T. and Lesser, V. (2002). Leveled-commitment contracting: A backtracking instrument for multiagent systems. *AI Magazine*, 23(3):89–100.

Sandholm, T., Suri, S., Gilpin, A., and Levine, D. (2005). CABOB: a fast optimal algorithm for winner determination in combinatorial auctions. *Management Science*, 51(3):374–391.

Sandholm, T. and Zhou, Y. (2002). Surplus equivalence of leveled commitment contracts. *Artificial Intelligence*, 142(2):239–264. doi:10.1016/S0004-3702(02)00275-8.

Sarne, D. and Arponen, T. (2007). Sequential decision making in parallel two-sided economic search. In (Durfee and Yokoo, 2007).

Sauter, J. A., Matthews, R., Parunak, H. V. D., and Brueckner, S. (2002). Evolving adaptive pheromone path planning mechanisms. In *Proceedings of the First Intenational Joint Conference on Autonomous Agents and Multiagent Systems*, pages 434–440, Bologna, Italy. ACM Press, New York, NY.



Parunak, HVD (1997). “vá até a formiga”: princípios de engenharia a partir de sistemas de agentes naturais. Anais de Pesquisa Operacional, 75:69–101. Parunak, HVD, Brueckner, S. e Sauter, J. (2002). Mecanismos de feromônios sintéticos para c oordenação de veículos não tripulados. Em Anais da Primeira Conferência Conjunta Internacional sobre Agentes Autônomos e Sistemas Multiagentes, páginas 448–450, Bolonha, Itália. ACM Press, Nova York, NY. Rahwan, I., Ramchurn, SD, Jennings, NR, McBurney, P., Parsons, S. e Sonenberg, L. (2004). Negociação baseada em argu mentação. The Knowledge Engineering Review, 18(4):343–375. doi:10.1017/S0269888904000098. Resnick, M. (1994). Tartarugas, cupins e engarrafamentos: explorações em micromundos massivamente paralelos. Imprensa do MIT. Rosenschein, JS e Zlotkin, G. (1994). Regras do Encontro. The MIT Press, Cambridge, MA. Rothkopf, MH, Pekec, A. e Harstad, RM (1998). Leilões combinacionais gerenciáveis computacionalmente. Ciência de Gestão, 44(8):1131–1147. Rubinstein, A. (1982). Equilíbrio perfeito em um modelo de barganha. Econometrica, 50(1):97–110. Russell, S. e Norvig, P. (2003). Inteligência Artificial: Uma Abordagem Moderna. Prentice Hall, segunda edição. Sandholm, T. (1997). Tipos de contratos necessários e suficientes para uma alocação ideal de tarefas. Nos Anais da Décima Quarta Conferência Conjunta Internacional sobre Inteligência Artificial. Sandholm, T. (1998). Tipos de contratos para atribuição satisfatória de tarefas: resultados teóricos. No Simpósio AAAI Spring: Modelos Satisfazendo. Sandholm, T. (1999). Um algoritmo para determinação de vencedores em leilões combinatórios. Em Anais da Décima Sexta Conferência Conjunta Internacional sobre Inteligência Artificial, páginas 542–547. Sandholm, T. (2002). Um algoritmo para determinação de vencedores em leilões combinatórios. Inteligência Artificial, 135(1-2):1–54. doi:10.1016/S0004-3702(01) 00159-X. Sandholm, T., Larson, K., Anderson, M., Shehory, O. e Tohm´e, F. (1999). Geração de estrutura de coalizão com garantias de pior caso. Inteligência Artificial, 111(1-2):209–238. doi:10.1016/S0004-3702(99)00036-3. Sandholm, T. e Lesser, V. (2002). Contratação de compromisso nivelado: um instrumento de retrocesso para sistemas multiagentes. Revista AI, 23(3):89–100. Sandholm, T., Suri, S., Gilpin, A. e Levine, D. (2005). CABOB: u m algoritmo rápido ótimo para determinação de vencedores em leilões combinatórios. Ciência de Gestão, 51(3):374–391. Sandholm, T. e Zhou, Y. (2002). Equivalência e xcedente de contratos de compromisso nivelado. Inteligência Artificial, 142(2):239–264. doi:10.1016/S0004-3702(02) 00275-8. Sarne, D. e Arponen, T. (2007). Tomada de decisão sequencial em busca econômica bilateral paralela. Em (Durfee e Yokoo, 2007). Sauter, JA, Matthews, R., Parunak, HVD e Brueckner, S. (2002). Evolução de mecanismos adaptativos de planejamento de caminhos de feromônios. Em Anais da Primeira Conferência Conjunta Internacional sobre Agentes Autônomos e Sistemas Multiagentes, páginas 434–440, Bolonha, Itália. ACM Press, Nova York, NY.

Sen, S. (2002). Believing others: Pros and cons. *Artificial Intelligence*, 142(2):179–203. doi:10.1016/S0004-3702(02)00289-8.

Shehory, O. and Kraus, S. (1998). Methods for task allocation via agent coalition formation. *Artificial Intelligence*, 101(1-2):165–200.

Shoham, Y. and Tennenholtz, M. (1997). On the emergence of social conventions: modeling, analysis, and simulations. *Artificial Intelligence*, 94:139–166.

Smith, R. G. (1981). The contract net protocol: High-level communication and control in a distributed problem solver. *IEEE Transactions on Computers*, C-29(12):1104–1113.

Squyres, S. (2005). *Roving Mars: Spirit, Opportunity, and the Exploration of the Red Planet*. Hyperion.

Stone, P. (2000). *Layered Learning in Multiagent Systems*. MIT Press.

Stone, P. and Weiss, G., editors (2006). *Proceedings of the Fifth International Joint Conference on Autonoumous Agents and Multi Agent Systems*, New York, NY. ACM Press.

Surowiecki, J. (2005). *The Wisdom of Crowds*. Anchor.

Sutton, R. S. and Barto, A. G. (1998). *Reinforcement Learning: An Introduction*. MIT Press.

Taylor, P. and Jonker, L. (1978). Evolutionary stable strategies and game dynamics. *Mathematical Biosciences*, 16:76–83.

Tumer, K. and Wolpert, D., editors (2004). *Collectives and the Design of Complex Systems*. SPringer.

Vidal, J. M. and Durfee, E. H. (1998a). Learning nested models in an information economy. *Journal of Experimental and Theoretical Artificial Intelligence*, 10(3):291–308.

Vidal, J. M. and Durfee, E. H. (1998b). The moving target function problem in multi-agent learning. In *Proceedings of the Third International Conference on Multi-Agent Systems*, pages 317–324. AAAI/MIT press.

Vidal, J. M. and Durfee, E. H. (2003). Predicting the expected behavior of agents that learn about agents: the CLRI framework. *Autonomous Agents and Multi-Agent Systems*, 6(1):77–107. doi:10.1023/A:1021765422660.

Waltz, D. (1975). Understanding line drawings of scenes with shadows. In Winston, P., editor, *The Psychology of Computer Vision*, pages 19–91. McGraw-Hill.

Watkins, C. J. C. H. and Dayan, P. (1992). Q-learning. *Machine Learning*, 8(3-4):279–292. doi:10.1023/A:1022676722315.

Weibull, J. W. (1997). *Evolutionary Game Theory*. The MIT Press.

Wellman, M. P. (1996). Market-oriented programming: Some early lessons. In Clearwater, S., editor, *Market-Based Control: A Paradigm for Distributed Resource Allocation*. World Scientific.

Wilensky, U. (1999). NetLogo: Center for connected learning and computer-based modeling, Northwestern University. Evanston, IL. http://ccl.northwestern.edu/netlogo/.

Willer, D., editor (1999). *Network Exchange Theory*. Praeger Publishers, Westport CT.



Sen, S. (2002). Acreditar nos outros: prós e contras. Inteligência Artificial, 142(2):179–203. doi:10.1016/S0004-3702(02)00289-8. Shehory, O. e Kraus, S. (1998). Métodos para atribuição de tarefas através da formação de coligações de agentes. Inteligência Artificial, 101(1-2):165–200. Shoham, Y. e Tennenholtz, M. (1997). Sobre a emergência das convenções sociais: modelagem, análise e simulações. Inteligência Artificial, 94:139–166. Smith, RG (1981). O protocolo de rede de contrato: comunicação e controle de alto nível em um solucionador de problemas distribuído. Transações IEEE em Computadores, C-29(12):1104–1113. Squyres, S. (2005). Roving Mars: Espírito, Oportunidade e a Exploração do Planeta Vermelho. Hipérion. Pedra, P. (2000). Aprendizagem em camadas em sistemas multiagentes. Imprensa do MIT. Stone, P. e Weiss, G., editores (2006). Anais da Quinta Conferência Conjunta Internacional sobre Agentes Autônomos e Sistemas Multiagentes, Nova York, NY. Imprensa ACM. Surowiecki, J. (2005). A Sabedoria das Multidões. Âncora. Sutton, RS e Barto, AG (1998). Aprendizagem por Reforço: Uma Introdução. Imprensa do MIT. Taylor, P. e Jonker, L. (1978). Estratégias evolutivas estáveis e dinâmica de jogo. Biociências Matemáticas, 16:76–83. Tumer, K. e Wolpert, D., editores (2004). Coletivos e o Projeto de Sistemas Complexos. Primavera. Vidal, JM e Durfee, EH (1998a). Aprendendo modelos aninhados em uma economia da informação. Jornal de Inteligência Artificial Experimental e Teórica, 10(3):291–308. Vidal, JM e Durfee, EH (1998b). O problema da função alvo móvel na aprendizagem multiagente. Em Anais da Terceira Conferência Internacional sobre Sistemas Multiagentes, páginas 317–324. Imprensa AAAI/MIT. Vidal, JM e Durfee, EH (2003). Prever o comportamento esperado de agentes que aprendem sobre agentes: a estrutura CLRI. Agentes Autônomos e Sistemas Multiagentes, 6(1):77–107. doi:10.1023/A:1021765422660. Valsa, D. (1975). Compreender desenhos de cenas com sombras. Em Winston, P., editor, The Psychology of Computer Vision, páginas 19–91. McGraw-Hill. Watkins, CJCH e Dayan, P. (1992). Q-aprendizagem. Aprendizado de Máquina, 8(3-4):279–292. doi:10.1023/A:1022676722315. Weibull, JW (1997). Teoria Evolucionária dos Jogos. A imprensa do MIT. Wellman, MP (1996). Programação orientada para o mercado: algumas lições iniciais. In Clearwater, S., editor, Market-Based Control: A Paradigm for Distributed Resource Allocation. Científico Mundial. Wilensky, U. (1999). NetLogo: Centro de aprendizagem conectada e modelagem baseada em computador, Northwestern University. Evanston, Illinois. http://ccl. noroeste.edu/netlogo/. Willer, D., editor (1999). Teoria da troca de rede. Editores Praeger, Westport CT.

Wolpert, D. and Tumer, K. (2001). Optimal payoff functions for members of collectives. *Advances in Complex Systems*, 4(2–3):265–279.

Wolpert, D., Wheeler, K., and Tumer, K. (1999). General principles of learning-based multi-agent systems. In *Proceedings of the Third International Conference on Automomous Agents*, pages 77–83, Seattle, WA.

Wolpert, D. H. and Macready, W. G. (1995). No free lunch theorems for search. Technical report, Santa Fe Institute.

Wolpert, D. H. and Tumer, K. (1999). An introduction to collective intelligence. Technical report, NASA. NASA-ARC-IC-99-63.

Wurman, P. R., Wellman, M. P., and Walsh, W. E. (2002). Specifying rules for electronic auctions. *AI Magazine*, 23(3):15–23.

Yokoo, M., Conitzer, V., Sandholm, T., Ohta, N., and Iwasaki, A. (2005). Coalition games in open anonymous environments. In *Proceedings of the Twentieth National Conference on Artificial Intelligence*, pages 509–514, Menlog Park, California. AAAI Press.

Yokoo, M., Durfee, E. H., Ishida, T., and Kuwabara, K. (1998). The distributed constraint satisfaction problem: Formalization and algorithms. *IEEE Transactions on Knowledge and Data Engineering*, 10(5):673–685.

Yokoo, M. and Hirayama, K. (1996). Distributed breakout algorithm for solving distributed constraint satisfaction problems. In *Proceedings of the Second International Conference on Multiagent Systems*, pages 401–408.

Yokoo, M. and Hirayama, K. (2000). Algorithms for distributed constraint satisfaction: A review. *Autonomous Agents and Multi-Agent Systems*, 3(2):185–207. doi:10.1023/A:1010078712316.

Zhang, X., Lesser, V., and Abdallah, S. (2005a). Efficient management of multi-linked negotiation based on a formalized model. *Autonomous Agents and Multi-Agent Systems*, 10(2). doi:10.1007/s10458-004-6978-6.

Zhang, X., Lesser, V., and Podorozhny, R. (2005b). Multi-dimensional, multistep negotiation for task allocation in a cooperative system. *Autonomous Agents and Multi-Agent Systems*, 10(1):5–40.



Wolpert, D. e Tumer, K. (2001). Funções de recompensa ótimas para membros de coletivos. Avanços em Sistemas Complexos, 4(2–3):265–279. Wolpert, D., Wheeler, K. e Tumer, K. (1999). Princípios gerais de sistemas multiagentes baseados em aprendizagem. Em Anais da Terceira Conferência Internacional sobre Agentes Autônomos, páginas 77–83, Seattle, WA. Wolpert, DH e Macready, WG (1995). Não há teoremas de almoço grátis para pesquisa. Relatório técnico, Instituto Santa Fé. Wolpert, DH e Tumer, K. (1999). Uma introdução à inteligência coletiva. Relatório técnico, NASA. NASA-ARC-IC-99-63. Wurman, PR, Wellman, MP e Walsh, WE (2002). Especificação de regras para leilões eletrônicos. Revista AI, 23(3):15–23. Yokoo, M., Conitzer, V., Sandholm, T., Ohta, N. e Iwasaki, A. (2005). Jogos de coalizão em ambientes anônimos abertos. Em Anais da Vigésima Conferência Nacional sobre Inteligência Artificial, páginas 509–514, Menlog Park, Califórnia. Imprensa AAAI. Yokoo, M., Durfee, EH, Ishida, T. e Kuwabara, K. (1998). O problema de satisfação de restrições distribuídas: Formalização e algoritmos. Transações IEEE sobre Conhecimento e Engenharia de Dados, 10(5):673–685. Yokoo, M. e Hirayama, K. (1996). Algoritmo de breakout distribuído para resolver problemas de satisfação de restrições distribuídas. Em Anais da Segunda Conferência Internacional sobre Sistemas Multiagentes, páginas 401–408. Yokoo, M. e Hirayama, K. (2000). Algoritmos para satisfação de restrições distribuídas: Uma revisão. Agentes Autônomos e Sistemas Multiagentes, 3(2):185–207. doi:10.1023/A:1010078712316. Zhang, X., Lesser, V. e Abdallah, S. (2005a). Gestão eficiente da negociação multiligada baseada num modelo formalizado. Agentes Autônomos e Sistemas Multiagentes, 10(2). doi:10.1007/s10458-004-6978-6. Zhang, X., Lesser, V. e Podorozhny, R. (2005b). Negociação multidimensional e em várias etapas para alocação de tarefas em um sistema cooperativo. Agentes Autônomos e Sistemas Multiagentes, 10(1):5–40.

# Index


NashQ-learning, 69
friend-or-foe, 70
TÆMS, 137
0-level, 73
1-level, 73
1-sided proposal, 97
2-level, 73

ABT, 25
additive cost function, 90
additive valuation, 116
ADEPT, 87
Adopt, 32
agenda, 95
algorithmic mechanism design, 131
annealing, 95
APO, 37
appeals, 97
argument-based protocols, 97
aristocrat utility, 75
artificial intelligence planning, 15
asynchronous backtracking, 25
Asynchronous Weak-Commitment, 27
atomic bids, 115
auctions, 103
AWESOME, 66
axiomatic, 78

backtracking, 24
bargaining problem, 77
battle of the sexes, 43
belief state, 15
Bellman equation, 13
Bellman update, 13
bi-connected, 133
bidder collusion, 106
Bidtree, 111
Borda count, 122
Border Gateway Protocol, 132
branch and bound, 32
branch on bids, 110
branch on items, 109
Byzantine generals problem, 40

CABOB, 111
CASS, 112
CATS, 113
characteristic form, 49
characteristic function, 49
CLRI model, 71
coalition, 49
coalition formation, 57
coalition structure, 50
COIN, 74
combinatorial auction, 107
common knowledge, 39
common value, 103
concurrent negotiations, 98
constraint satisfaction problem, 19
contract net protocol, 89
cooperative games, 49
coordination problem, 43
coordination relationships, 140
core, 51
correlated value, 103
counter-proposal, 97
critique, 97
CSP, 19

DCSP, 20
de-commit, 92
deal, 77
decision version, 108
deductive, 9
depth first search, 20
descriptive approach, 99
desirable voting protocol, 123
deterministic, 12
direct revelation mechanism, 127
discount factor, 13
discounted rewards, 13
distributed algorithmic mechanism design, 131
distributed breakout, 29
distributed constraint optimization, 31
distributed constraint satisfaction, 20
divide and conquer, 16
dominant, 41
dominant strategy equilibrium, 126
double auction, 105
Dutch, 104
dynamic programming, 13

efficient, 41
efficient best first, 118
egalitarian social welfare, 79
egalitarian solution, 79
English auction, 104




**Índice**


NashQ-learning, 69
friend-or-foe, 70TÆMS, 137 nível 0, 73 nível 1, 73 proposta unilateral, 97 nível 2, 73

ABT, 25 função de custo aditiva, 90 avaliação aditiva, 116 ADEPT, 87 Adotar, 32 agenda, 95 design de mecanismo algorítmico, 131 recozimento, 95 APO, 37 apelos, 97 protocolos baseados em argumentos, 97 utilidade aristocrática, 75 planejamento de inteligência artificial, 15 retrocesso assíncrono, 25 compromisso fraco assíncrono, 27 lances atômicos, 115 leilões, 103 INCRÍVEIS, 66 axiomáticos, 78

retrocesso, 24 problemas de barganha, 77 batalha dos sexos, 43 estado de crença, 15 equação de Bellman, 13 atualização de Bellman, 13 bi-conectados, 133 conluio de licitantes, 106 Bidtree, 111 contagem de Borda, 122 Border Gateway Protocol, 132 ramificação e limite, 32 ramificação em licitações, 110 ramificação em itens, 109 problema de generais bizantinos, 40

CABOB, 111 CASS, 112 CATS, 113 forma característica, 49 função característica, 49

Modelo CLRI, 71 coalizões, 49 formações de coalizões, 57 estruturas de coalizões, 50 COIN, 74 leilões combinatórios, 107 conhecimentos comuns, 39 valores comuns, 103 negociações simultâneas, 98 problemas de satisfação de restrições, 19 protocolos de rede de contratos, 89 jogos cooperativos, 49 problemas de coordenação, 43 relacionamentos de coordenação, 140 núcleos, 51 valores correlacionados, 103 contrapropostas, 97 crítica, 97CSP, 19

DCSP, 20 descomprometimento, 92 acordo, 77 versão de decisão, 108 dedutivo, 9 pesquisa em profundidade, 20 abordagem descritiva, 99 protocolo de votação desejável, 123 determinístico, 12 mecanismo de revelação direta, 127 fator de desconto, 13 recompensas com desconto, 13 design de mecanismo algorítmico distribuído, 131 ruptura distribuída, 29 otimização de restrição distribuída, 31 restrição distribuída satisfação, 20 dividir e conquistar, 16 dominante, 41 dominante estratégia equilíbrio, 126 leilão duplo, 105 holandês, 104 programação dinâmica, 13

eficiente, 41
eficiente melhor primeiro, 118 bem-estar social igualitário, 79 solução igualitária, 79 leilão inglês, 104

envy-free, 114
equal excess, 54
equi-resistance point, 100
evolutionary game theory, 63
evolutionary stable strategy, 65
excess, 53
expected utility, 11
exploration rate, 68
extended form, 45

factored, 75
feasible, 50
fictitious play, 61
filtering algorithm, 21
first-price open-cry ascending, 104
First-price sealed-bid, 104
folk theorem, 45
free disposal, 116

Gambit, 47
game of chicken, 43
game theory, 39
Groves-Clarke mechanism, 129

hierarchical learning, 16
hill climbing, 90
hyper-resolution rule, 23

implements, 126
implements social choice function $f(\cdot)$ in dominant strategies, 127
individual hill-climbing, 115
induction, 60
induction bias, 60
inductive, 9
inefficient allocation, 106
iterated dominance, 41
iterated equi-resistance algorithm, 100
iterated prisoner's dilemma, 44

justify, 97

$k$-consistency, 22
Kalai-Smorodinsky solution, 81
kernel ABT, 27

learning rate, 68
leveled commitment contracts, 93
linear programming, 109
lying auctioneer, 106

machine learning, 59
marginal utility, 10
Markov decision process, 12
maximum expected utility, 12
maxmin strategy, 40
mechanism, 125
mechanism design, 125
mediator, 95
minimax theorem, 41
mixed strategy, 40
monetary payments, 89
monotonic concession protocol, 83
moving target function, 60
Multi-DB$^{++}$, 31
multi-dimensional deal, 94

n-level, 73
n-queens problem, 20
Nash bargaining solution, 80
Nash equilibrium, 41
Nash equilibrium point, 69
negotiation network, 98
network exchange theory, 99
no free lunch theorem, 60
nogood, 23
non local effects, 137
non-cooperative games, 39
non-deterministic, 12
normal form, 39
nucleolus, 53, 54

observation model, 15
OCSM contracts, 91
one-step negotiation, 86
opacity, 75
open-cry descending price, 104
optAPO, 37
optimal policy, 12
OR bids, 115
OR$^*$ bids, 116
outcome, 49, 125

pairwise, 122
PAR algorithm, 117
paretian, 125
Pareto frontier, 118
pareto frontier, 78
Pareto optimal, 41
partially observable Markov decision process, 15
PAUSE, 113
pausebid, 114
payoff matrix, 39
persuade, 97
plurality vote, 121
policy, 12
postman problem, 88
preference elicitation, 116
preference ordering, 10
Prisoner's Dilemma, 42
private value, 103
pure strategy, 40

Q-learning, 68
quality accumulation functions, 138
quasi-local minimum, 29



livre de inveja, 114 excesso igual, 54 ponto de equi-resistência, 100 teoria dos jogos evolutivos, 63 estratégia evolutiva estável, 65 excesso, 53 utilidade esperada, 11 taxa de exploração, 68 forma estendida, 45

fatorado, 75 viáveis, 50 jogos fictícios, 61 algoritmo de filtragem, 21 primeiro preço aberto, 104 la nce selado de primeiro preço, 104 teorema popular, 45 descarte gratuito, 116

Gambito, 47 jogo de frango, 43 teoria dos jogos, 39 mecanismo Groves-Clarke, 129

aprendizagem hierárquica, 16 subidas, 90 regra de hiper-resolução, 23

implementa, 126 implementa função de escolha social $f(\cdot)$ em estratégias dominantes, 127 escalada individual, 115 indução, 60 viés de indução, 60 indutivo, 9 alocação ineficiente, 106 dominância iterada, 41 algoritmo de equi-resistência iterado, 100 dilema do prisioneiro it erado, 44

justificar, 97

$k$-consistência, solução 22 Kalai-Smorodinsky, kernel 81 ABT,27

taxa de aprendizagem, 68 contratos de compromisso nivelados, 93 programação linear, 109 leiloeiro mentiroso, 106

aprendizado de máquina, 59 utilidade marginal, 10 processo de decisão de Markov, 12 utilidade máxima esperada, 12 maxminstrategy, 40 mecanismo, 125 design de mecanismo, 125 mediador, 95

teorema minimax, 41 estratégia mista, 40 pagamentos monetários, 89 protocolo de concessão monotônica, 83 função de alvo móvel, 60 Multi-DB$^{++}$, 31 acordo multidimensional, 94

nível n, problema de 73 n-rainhas, 20 solução de negociação de Nash, 80 equilíbrio de Nash, 41 ponto de equilíbrio de Nash, 69 rede de negociação, 98 teoria de troca de rede, 99 teorema sem almoço grátis, 60 nada de bom, 23 efeitos não locais, 137 jogos não cooperativos, 39 não determinísticos, 12 forma normal, 39 nucléolo, 53, 54

modelo de observação, 15 contratos OCSM, 91 negociação em uma etapa, 86 opacidade, 75 preço descendente de grito aberto, 104 optAPO, 37 política ideal, 12OR licitações, 115OR$^*$ licitações, 116 resultado, 49, 125

pairwise, 122 algoritmo PAR, 117 paretian, 125 fronteira de Pareto, 118 fronteira de pareto, 78 Pareto ótimo, 41 processo de decisão de Markov parcialmente observável, 15 PAUSE, 113 pausebid, 114 matriz de payoff, 39 persuadir, 97 voto de pluralidade, 121 política, 12 problema do carteiro, 88 elicitação de preferência, 116 preferência ordenação, 10 Dilema do Prisioneiro, 42 valor privado, 103 estratégia pura, 40

Q-learning, 68 funções de acumulação de qualidade, 138 mínimo quase local, 29

rational, 40
reciprocity, 48
Reflectional symmetry, 122
reinforcement learning, 68
repeated games, 61
replicator dynamics, 63
reservation price, 104
resistance, 99
revelation principle, 127
revenue equivalence, 130
rewards, 97
Rotational symmetry, 122
Rubinstein's alternating offers, 81
runoff, 122

second-price sealed-bid, 104
self-enforce collusion, 106
selfish, 10
Shapley value, 52
simplex plot, 66
social choice function, 125
social law, 47
social welfare, 41
Stirling, 56
Stirling number of the second kind, 108
stochastically, 62
strategic form, 39
strategy, 40
strategy-proof, 127
subgame perfect equilibrium, 46
super-additive, 50
symmetry, 122

tactics, 87
task allocation problem, 88
testing set, 59
threats, 97
tit-for-tat, 45
training set, 59
transferable utility, 49
transition function, 11
type, 125

utilitarian, 55
utilitarian solution, 79
utility function, 9

valuation, 103
value function, 49
value iteration, 13
value of information, 11
Vickrey, 104
Vickrey-Clarke-Groves, 130
VSA, 114

winner determination, 108
winner's curse, 104
wonderful life, 75

Xor bids, 116

zero-sum, 40
Zeuthen strategy, 84



racional, 40 reciprocidade, 48
simetria reflexiva, 122
aprendizagem por reforço, 68 j
ogos repetidos, 61 dinâmica de
replicador, 63 preço de reserva,
104 resistência, 99 princípio de
revelação, 127 equivalência de
receita, 130 recompensas, 97
simetria rotacional, 122 ofertas
alternadas de Rubinstein, 81
escoamento, 122

lance selado de segundo preço, 104
conluio auto-imposto, 106 egoísta, 10
valor de Shapley, 52 enredo simplex, 66
função de escolha social, 125 direito
social, 47 bem-estar social, 41 Stirling, 56
número de Stirling de segundo tipo, 108
estocasticamente, 62 forma estratégica,
39 estratégia, 40 prova de estratégia, 127
equilíbrio perfeito de subjogo, 46 super
aditivo, 50 simetria, 122

táticas, 87 problemas de
alocação de tarefas, 88
conjuntos de testes, 59
ameaças, 97 olho por olho,
45 conjuntos de
treinamento, 59 utilidades
transferíveis, 49 funções de
transição, 11 tipos, 125

utilitário, 55 solução
utilitária, 79 função de
utilidade, 9

avaliação, 103 função de
valor, 49 iteração de valor,
13 valor de informação, 11
Vickrey, 104
Vickrey-Clarke-Groves, 130
VSA, 114

determinação do vencedor,
108 maldição do vencedor,
104 vida maravilhosa, 75

Xor lances, 116

soma zero, 40
estratégia Zeuthen, 84